

LINGUAGEM ASSEMBLY

INTRODUÇÃO AO PADRÃO INTEL 8086

```
=====
[LINE]      LOC: MACHINE CODE      SOURCE
=====
[  1]      :                      ORG 100H
[  2]      :
[  3]      0100: B8 0A 00          MOV AX, 000AH
[  4]      0103: BB 0B 00          MOV BX, 000BH
[  5]      0106: 50                PUSH AX
[  6]      0107: 53                PUSH BX
[  7]      0108: E8 01 14          CALL PROCEDIMENTO
[  8]      010B: 40                INC AX
[  9]      010C: 43                INC BX
[ 10]      010D: E8 01 14          CALL PROCEDIMENTO
[ 11]      0110: 5B                POP BX
[ 12]      0111: 58                POP AX
[ 13]      0112: CD 20            INT 20H
[ 14]      :
[ 15]      0114: B8 01 00          PROCEDIMENTO: MOV AX, 0001H
[ 16]      0117: BB 02 00          MOV BX, 0002H
[ 17]      011A: 40                INC AX
[ 18]      011B: 43                INC BX
[ 19]      011C: C3                RET
=====
```



José Augusto N. G. Manzano



ORCID: 0000-0001-9248-7765

Linguagem Assembly

INTRODUÇÃO AO PADRÃO INTEL 8086

São Paulo
2021 - Propes Vivens



© Copyright 2021 by José Augusto N. G. Manzano / Propes Vivens.

Todos os direitos reservados. Proibida a reprodução total ou parcial, por qualquer meio ou processo, especialmente por sistemas gráficos, microfilmicos, fotográficos, reprográficos, fonográficos, videográficos, internet, e-books. Vedada a memorização e/ou recuperação total ou parcial em qualquer sistema de processamento de dados e a inclusão de qualquer parte da obra em qualquer programa jus cibernético existentes ou que a venham existir. Essas proibições aplicam-se também às características gráficas da obra e à sua editoração, exceto pelo exporto no próximo parágrafo. A violação dos direitos autorais é punível como crime (art. 184 e parágrafos, do Código Penal, conforme Lei nº 10.695, de 07.01.2003) com pena de reclusão, de dois a quatro anos, e multa, conjuntamente com busca e apreensão e indenizações diversas (artigos 102 e 103 parágrafo único, 104, 105, 106 e 107 itens 1, 2 e 3 da Lei nº 9.610, de 19.06.1998, Lei dos Direitos Autorais).

Esta obra é distribuída gratuitamente em formato digital (somente em PDF) apenas e tão somente no sítio do autor (www.manzano.pro.br) e para o que o deseja de forma impressa junto as plataformas **Clube de Autores** e **Agbook**. Nenhum outro local da Internet ou fora dela está autorizado a distribuir este material. Não é permitido este compartilhamento em outros locais ou por qualquer meio exceto o exposto neste parágrafo. Os infratores estão sujeitos a processo judicial.

O Autor acredita que as informações aqui apresentadas estão corretas e podem ser utilizadas para qualquer fim legal. Entretanto, não existe qualquer garantia, explícita ou implícita, de que o uso de tais procedimentos conduzirá sempre ao resultado desejado. O autor e a editora não poderão ser responsabilizados civilmente ou criminalmente. Os nomes de sítios e empresas, mencionados, foram utilizados apenas como ilustração, não havendo nenhum vínculo com a obra, não garantindo a sua existência nem divulgação a posteriori.

Conteúdo de acordo com Acordo Ortográfico da Língua Portuguesa, desde 1 de janeiro de 2009.

Dados Internacionais de Catalogação na Publicação (CIP)
(Câmara Brasileira do Livro, SP, Brasil)

Manzano, José Augusto N. G.
Linguagem Assembly [livro eletrônico] : introdução
ao padrão Intel 8086 / José Augusto N. G. Manzano. --
São Paulo : Propes Vivens, 2021.

PDF

Bibliografia.

ISBN 978-65-00-24498-4

1. Intel 8086 (Microprocessador) 2. Linguagens de
montagem (Computadores) 3. Microcomputadores IBM
I. Título.

21-70472

CDD-005.136

Índices para catálogo sistemático:

1. Assembly : Linguagem de programação :
Computadores : Processamento de dados 005.136

Cibele Maria Dias - Bibliotecária - CRB-8/9427

Produção e Editoração:
Imagens:

José Augusto Navarro Garcia Manzano
canva.com e freepng.es (ibm-pc)
<https://www.freepng.es/png-evckf6/>

Edição:

Propes Vivens

FERRAMENTAS UTILIZADAS

Produto: **EMU8086 - MICROPROCESSOR EMULATOR [Versão 4.08r11]**

Sítio: <http://www.emu8086.com> (descontinuado)

Alternativa: <https://github.com/AhmadNaserTurnkeySolutions/emu8086>

Custo: Private Licence	4,95 USD
------------------------	----------

1 Year School Licence	52,00 USD
-----------------------	-----------

Permanent Size Licence	295,00 USD
------------------------	------------

Produto: **ENHANCED DEBUG [Versão 1.32b]**

Sítio: <https://sites.google.com/site/pcdosretro/enhdebug>

Produto: **VDOS [Versão 2021.05.01]**

Sítio: <https://vdos.info/>

REQUISITOS DE HARDWARE E SOFTWARE

Requisitos Comuns

- ◆ 1 MB de memória RAM;
- ◆ 10 MB de espaço em disco para instalação do compilador;
- ◆ Sistema Operacional Windows XP, Vista, 7, 8, 8.1 ou 10;
- ◆ Monitor com 1024 x 768 pontos ou superior para melhor visualização;
- ◆ Mouse ou outro periférico de apontamento;
- ◆ Modem e acesso à Internet;
- ◆ Programa emu8086 adquirido com custo financeiro;
- ◆ Programa DOS Debug versão 1.25.

OBS.: No Microsoft Windows 7 de 32 bits após atualizações poderão ocorrer problemas na execução do programa emu8086. Neste caso é aconselhável trabalhar com o programa em modo XP Mode. Use o modo XP Mode para rodar os programas debug do próprio XP ou o emu8086 quando estiver em uso a versão de 64 bits do sistema operacional Microsoft Windows 7 ou 10. Outra dica é executar todos os programas como administrador para minimizar eventuais problemas.

Foram feitos testes de execução do programa emu8086 no Microsoft Windows 8.1 e 10 de 32 e 64 bits e aparentemente não ocorreram problemas. No entanto na versão 64 bits o programa Microsoft Debug não é encontrado e não poderá ser executado. No entanto, há uma alternativa para executar do programa no sistema operacional Windows 8, 8.1 e 10 de 64 bits por meio do programa de virtualização vDos retratado nesta obra.

Contato com o autor: *certificado@mail.com*

Colocar em assunto o título *Linguagem Assenbly*.

Somente serão respondidos e-mails com dúvidas relacionadas, estritamente, ao que é apresentado neste trabalho. Qualquer outro tipo de dúvida ou questão fora deste contexto, não será atendida.

RESTRIÇÃO OPERACIONAL

Devido a estrutura operacional de uso dos microprocessadores Intel 8086/8088 e do foco deste trabalho devem ser consideradas as seguintes restrições operacionais:

- ◆ Os programas apresentados nesta obra são desenvolvidos numa esfera simplista, apresentam apenas os recursos em média discutidos nos capítulos que são focados;
- ◆ Não são tratadas as entradas de valores numéricos com ponto flutuante, apenas entradas de dados baseada em valores numéricos inteiros e dados alfabéticos;
- ◆ O objetivo deste trabalho é ser uma introdução didática, o mais simples possível, apenas e tão somente para leitores que não conhecem absolutamente nada em programação Assembly;
- ◆ Não são tratados nenhum aspecto considerado intermediário ou avançado no nível de desenvolvimento dos programas ou do conteúdo da obra;
- ◆ Não é apresentado nenhum programa contextualizando o uso da linguagem Assembly para componentes de automação ou de uso comercial.

AGRADECIMENTOS

À minha esposa Sandra e à minha filha Audrey, motivos de constante inspiração ao meu trabalho. Pela paciência nos momentos de ausência que tenho, quando estou absorto em escrever, dedicando-me ao ensino;

A Peter Norton, bandeirante desbravador, que sempre direcionou seu trabalho literário ao conhecimento mais refinado no uso dos microcomputadores padrão IBM-PC e principalmente ao processo de entendimento do funcionamento dos microprocessadores (micro controladores) Intel, como o fez com John Socha em sua obra Peter Norton's Assembly Language Book for IBM-PC.

A Alexander Popov que, com seu brilhantismo profissional, iniciou o desenvolvimento da ferramenta de estudo da linguagem de programação Assembly emu8086, e também a Tomasz Grystar, desenvolvedor do programa Flat Assembler que tem sua contribuição registrada neste trabalho, por tornarem o estudo da programação em baixo nível muito agradável e de fácil compreensão.

Ao amigo Stephen P. Morse, engenheiro que desenvolveu o microprocessador 8086/8088 e quando o fez, não imaginou que sua criação seria utilizada para compor um dos microcomputadores mais utilizados no mundo. Obrigado amigo pelo auxílio que ofereceu a mim quando interagimos sobre seu trabalho.

Há ainda uma pessoa que merece grande atenção, você, leitor, que me prestigia adquirindo esta e outras obras de minha autoria, sempre de forma legal, na livraria de sua confiança. Devido à sua postura tenho incentivo para continuar escrevendo, mesmo num País em que está enraizada a cultura da pirataria, das famigeradas apostilas e da cópia ilegal do trabalho alheio. Muito obrigado por sua confiança e respeito.

A todos os alunos que passaram e passam por minhas mãos, que acreditaram e acreditam na minha pessoa e seguiram e seguem as orientações passadas; por me incentivarem continuamente quando me questionam sobre temas que ainda não conheço, por me levarem a um patamar maior por exigirem assim que eu pesquise mais.

Vida longa e próspera.

SUMÁRIO

Parte I - Noções preliminares

Capítulo 1 - Introdução

1.1 - Assembly ou Assembler	21
1.2 - Por que aprender Assembly?	22
1.3 - Restrições deste trabalho	23
1.4 - Por que usar o padrão 8086 e não outro mais recente?	25
1.5 - Arquitetura: Princípios básicos	25
1.6 - Código ASCII	27

Capítulo 2 - Conceitos fundamentais

2.1 - Sistema computacional	31
2.1.1 - Unidade central de processamento	32
2.1.2 - Unidade de memória	32
2.1.3 - Unidades de entrada e de saída	32
2.2 - Organização de dados	33
2.2.1 - Nibble	33
2.2.2 - Byte	34
2.2.3 - Word	34
2.2.4 - Double word	35
2.2.5 - Quad word	35
2.2.6 - Unidades de medidas computacionais	35
2.2.7 - Sistemas de numeração e potências computacionais	36
2.3 - Arquitetura 8086	40
2.3.1 - Registradores gerais	44
2.3.2 - Registradores de segmento	45
2.3.3 - Registradores de deslocamento	46
2.3.4 - Registradores de estado (status flags)	47
2.4 - Interrupções	48
2.5 - Segmentos e deslocamentos	49
2.6 - Endereçamento de memória	50
2.7 - A linguagem Assembly 8086	52
2.8 - Os modos 32 e 64 bits	53

Parte II - Programação com Enhanced DEBUG

Capítulo 3 - Ferramentas de desenvolvimento

3.1 - Como começou, como está, como fica	57
3.2 - Os Programas de depuração DEBUG	58
3.3 - Programa emulador assembler emu8086	59
3.4 - Preparação do ambiente de trabalho	59

Capítulo 4 - Cálculos matemáticos básicos

4.1 - O Programa Enhanced DEBUG	61
4.2 - Aritmética em modo hexadecimal.....	64
4.3 - Representação de valores negativos	65
4.4 - Cálculos em códigos de máquina.....	67
4.4.1 - Adição de valores hexadecimais.....	68
4.4.2 - Subtração de valores hexadecimais	72
4.4.3 - Multiplicação de valores hexadecimais.....	74
4.4.4 - Divisão de valores hexadecimais	75

Capítulo 5 - Apresentação de dados

5.1 - Apresentação de um caractere.....	79
5.2 - Movimentação de dados	83
5.3 - Apresentar sequência de caracteres	87
5.4 - Apresentar valores binários	89
5.5 - Registradores de estado	95
5.6 - Instruções de salto condicional.....	99
5.7 - Execução básica de desvios	101
5.8 - Apresentar valor hexadecimal	105

Capítulo 6 - Entrada de dados

6.1 - Entrada de dados por teclado	111
6.2 - Utilização de procedimento	117
6.3 - Utilização da pilha	119
6.4 - Consistência da entrada de dados	125

Parte III - Programação com emu8086

Capítulo 7 - Operações essenciais

7.1 - Simulador e assembler 8086	131
7.2 - Programa "Alo Mundo!"	132
7.3 - De volta ao programa Enhanced DEBUG	142
7.4 - Depuração com uso da pilha.....	144
7.5 - Instruções da linguagem de montagem 8086.....	152
7.6 - Uso do modo ajuda	159

Capítulo 8 - Programação básica

8.1 - Manipulação de registradores e dados.....	161
8.1.1 - Endereçamento imediato.....	161
8.1.2 - Endereçamento por registrador.....	161
8.1.3 - Endereçamento por deslocamento (offset)	162
8.1.4 - Outras formas de deslocamento	163
8.2 - Tipos de dados em Assembly	163

8.3 - Cálculos matemáticos intermediários.....	168
8.3.1 - Adição	168
8.3.2 - Subtração	177
8.3.3 - Divisão.....	180
8.3.4 - Multiplicação.....	187
8.4 - Procedimento próximo x procedimento distante	196

Capítulo 9 - Saltos, decisões e repetições

9.1 - Salto incondicional	199
9.2 - Salto condicional	201
9.3 - Desvios condicionais.....	205
9.3.1 - Desvio condicional simples.....	205
9.3.2 - Desvio condicional composto	207
9.4 - Operações lógicas.....	209
9.5 - Repetições	217
9.6 - Utilização de macros.....	225

Capítulo 10 - Detalhes complementares

10.1 - Mais sobre segmentos e deslocamentos	229
10.2 - Programas executáveis.....	237
10.3 - Bibliotecas em Assembly	248
10.4 - Apresentação de valores negativos	254
10.5 - Biblioteca de funções externas	258
10.6 - Manipulação de cadeias.....	263

Capítulo 11 - Recursos específicos

11.1 - Funcionalidades da biblioteca emu8086.inc.....	269
11.1.1 - Funções de macro	269
11.1.2 - Funções de procedimento	272
11.2 - Endereçamento e acesso à memória.....	275
11.2.1 - Acesso direto	276
11.2.2 - Acesso indireto com registradores	280
11.2.3 - Acesso indexado.....	285
11.2.4 - Acesso de base indexada.....	288
11.2.5 - Acesso de base indexada mais deslocamento	289
11.3 - Prefixos de anulação.....	291
11.4 - Operações com matrizes	291
11.5 - Programas exemplo	298

Capítulo 12 - Como em alto nível

12.1 - Tomada de decisão.....	301
12.1.1 - Decisão simples	302
12.1.2 - Decisão composta	305
12.1.3 - Decisões com duas condições.....	307
12.1.4 - Decisões sequências.....	313
12.1.5 - Decisão seletiva	316
12.1.6 - Decisão encadeada	319

12.2 - Laços de repetição	322
12.2.1 - Laço condicional pré-teste.....	322
12.2.2 - Laço condicional pós-teste	325
12.2.3 - Laço incondicional	328
12.3 - Demonstrações Assembly.....	330
12.4 - Linguagem assembly versus código de máquina.....	338

Parte IV - Apêndice

Apêndice A - Referência operacional	341
Apêndice B - Enhanced DEBUG	363
Referência Bibliográfica.....	383

PREFÁCIO

O objetivo central deste trabalho é ser um veículo de apresentação dos princípios preliminares e básicos que norteiam o início do aprendizado da linguagem de programação Assembly 8086/8088 de microcomputadores IBM-PC em baixo nível. Não há pretensão de explorar o tema com profundidade. Este livro é direcionado tão somente a leitores iniciantes. É uma introdução a atividade de programação em baixo nível. Se o leitor possui noções de programação em linguagem Assembly ou programação em baixo nível com *opcodes*, este material não é para você, pois não lhe acrescentará novos conhecimentos.

A escolha do modelo Assembly 8086/8088 para este estudo e das funções de programação básica de um microprocessador padrão Intel ou AMD se justifica por dois motivos: primeiro pelo fato de ser a mais usada e difundida para o ensino e aprendizado de programação de computadores em baixo nível em várias escolas espalhadas pelo mundo; segundo pelo fato das ferramentas *debug* e *emu8086* fazerem uso do padrão Intel 8086/8088.

Nos vários cursos de linguagem de programação Assembly nas mais variadas instituições de ensino superior espalhadas pelo mundo, normalmente escolhe-se o modelo 8086/8088 de microprocessador para o início do estudo, pois a família de computadores IBM-PC se desenvolveu e tornou-se popular sobre esta arquitetura. Não adianta tentar apresentar os últimos recursos do último modelo de um microprocessador Intel ou AMD para um iniciante que não conhece as bases mínimas desse ambiente de desenvolvimento.

O livro está dividido em quatro partes lógicas, nas quais se procurou agrupar o tema em áreas de concentração operacional (o que não é algo simples em se tratando de programação em baixo nível, pois os temas facilmente se misturam), no sentido de formarem as partes mínimas do todo, para o desenvolvimento do entendimento mental voltado à programação de computadores padrão IBM-PC em baixo nível.

A parte I apresenta as noções preliminares do estudo. São estudados a diferença entre assembly e assembler; o contexto do sistema computacional, a organização básica dos dados, os registradores, as interrupções, os segmentos, os deslocamentos e endereçamento de memória. Essa parte dá a noção básica relacionada à arquitetura interna do modelo de microprocessador 8086/8088.

A parte II destaca o uso do programa DEBUGX. Aqui são apresentados conceitos de aritmética em modo hexadecimal, representação de valores negativos, cálculos com códigos de máquina, movimentação de dados, apresentação de caracteres, valores binários e hexadecimais, utilização básica dos registradores de estado, apresentação inicial de salto condicional, entrada de dados por teclado, utilização de procedimentos, de pilha e o controle da consistência da entrada de dados. É pertinente salientar que boa parte dos programas exemplo utilizados foram baseados ou são adaptações dos programas encontrados na obra dos autores NORTON & SOCHA de 1987, que melhor descreveram as bases iniciais para o aprendizado da programação em baixo nível. Em alguns trechos dos capítulos desta parte são usados elementos colorizados para permitir melhor visualização dos elementos técnicos abordados.

A parte III trata das informações de aperfeiçoamento em relação ao estudo da parte anterior. É feita uma descrição dos recursos básicos da ferramenta emu8086, e são apresentadas informações teóricas de manipulação de registradores e dados (uma abordagem mais prática encontrada no final da obra). Outros pontos apresentados são a definição de tipos de dados e de cálculos matemáticos mais elaborados, o uso de saltos com decisões e laços de repetição, o uso de instruções lógicas, utilização de macros em complementação ao procedimento, apresentação mais detalhada de segmentos e deslocamentos, criação de biblioteca externa, apresentação de valores negativos, utilização da biblioteca de recursos que acompanha a ferramenta emu8086, abordagem de endereçamento e acesso à memória.

A parte IV apresenta anexo com informações complementares.

Por se tratar de um trabalho de apresentação dos recursos básicos da linguagem de programação Assembly 8086/8088, não é feito um estudo aprofundado das instruções do microprocessador Intel 8086/8088. Apresenta-se em torno de 60% do conjunto de instruções, o que já dá muito material para um estudo bem demorado.

Como sempre, a todos um grande abraço e um bom aprendizado. Que este trabalho seja de grande proveito e o investimento financeiro, ora nele feito, seja válido e proveitoso no final do seu estudo desta obra.

Augusto Manzano

"O aprendizado é um tesouro que segue seu dono por todos os lugares." - Provérbio chinês

SOBRE O AUTOR

José Augusto Navarro Garcia Manzano é professor com mestrado possuindo formação em Análise e Desenvolvimento de Sistemas, Ciências Econômicas e Licenciatura em Matemática atuando na área de Tecnologia da Informação, Computação e Informática (desenvolvimento de software, ensino e treinamento) desde 1986. Participou do desenvolvimento de aplicações computacionais para áreas de telecomunicações e comércio. Na carreira docente iniciou sua atividade docente em cursos livres, trabalhando posteriormente em empresas de treinamento e atuando desde então nos ensinos técnico e superior.

Atualmente é professor com dedicação exclusiva no IFSP (Instituto Federal de Educação, Ciência e Tecnologia de São Paulo, antiga Escola Técnica Federal). Em sua carreira desenvolveu competências e habilidades para ministrar componentes curriculares de Lógica de Programação (Algoritmos), Estrutura de Dados, Microinformática, Informática, Linguagens de Programação Estruturada, Linguagens de Programação Orientada a Objetos, Engenharia de Software, Sistemas de Informação, Engenharia da Informação, Arquitetura de Computadores e Tecnologias Web. Possui conhecimento de uso de diversas linguagens de programação imperativas, descritivas e de marcação, tais como: BASIC (clássico e estruturado), COBOL, COMAL, Logo, Scratch, Assembly, Pascal, FORTRAN (77 e 90), C, C++, D, Java, F#, Modula-2, C#, Lua, HTML, XHTML, CSS, Javascript, VBA, Ada, Rust, Python, LISP, Haskell, Standard ML, OCaml, Miranda, Hope, Groovy, Julia e Elixir. Possui mais de uma centena de obras publicadas, além de diversos artigos no Brasil e exterior.

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69
0D82:0109 64	DB	64
0D82:010A 61	DB	61
0D82:010B 64	DB	64
0D82:010C 65	DB	65
0D82:010D 206573	AND	[DI+73],AH
0D82:0110 7065	JO	0177
0D82:0112 63	DB	63
0D82:0113 69	DB	69
0D82:0114 66	DB	66
0D82:0115 69	DB	69
0D82:0116 63	DB	63
0D82:0117 61	DB	61

Parte

I

Noções preliminares

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 1

INTRODUÇÃO

O objetivo deste capítulo é apresentar conceitos introdutórios e básicos para a utilização da linguagem de programação para computadores Assembly baseada nos microprocessadores Intel e AMD usados nos computadores pertencentes a linha IBM-PC baseados na família x86, mais precisamente o modelo 8086. São apresentadas as diferenças entre os termos *assembly* e *assembler*, o que são mnemônicos e o que motiva o aprendizado da programação com a linguagem *assembly*.

1.1 Histórico da linguagem

É oportuno esclarecer um ponto de grande confusão por parte de pessoas leigas no assunto. Não é possível programar computadores com linguagem **Assembler**. Programa-se computadores quando em baixo nível ou com linguagem de máquina ou com linguagem **Assembly** (lê-se *assembli*).

O termo *assembly* significa *montagem*, ou seja, *linguagem de montagem* (*assembly language*), utilizada para programar um computador próximo ao nível operacional do seu microprocessador, denominado baixo nível, sendo necessário montar o programa na memória do computador para que o programa seja então executado. *Assembly* não é uma linguagem de máquina (como muitos afirmam), mas é a linguagem de programação que está mais próxima disso. Um programa escrito em *Assembly* é montado para ser executado dentro do microprocessador.

A linguagem de máquina é uma ferramenta utilizada por um microprocessador para controlar as funções internas de um computador digital por meio de comandos chamados *opcodes*. Segundo (VISCONTI, 1991, p. 13), o microprocessador utilizado em um computador "é um circuito que possui a capacidade de executar diversos tipos de funções distintas". A linguagem de máquina (em seu sentido mais puro) só aceita e manipula informações numéricas expressas em notação de códigos binários (os quais matematicamente representam os estados de tensão alta "1" ou tensão baixa "0" para os circuitos eletrônicos de um computador conectados a energia elétrica) na forma de grupos numéricos como: *nibble*, *byte*, *word* entre outros agrupamento que serão adiante comentados. Por questões de facilidade operacional, os agrupamentos de números binários usados em microprocessadores são representados no formato hexadecimal.

A programação de computadores digitais em linguagem de máquina foi muito utilizada, principalmente durante a década de 1940 (século XX), após o surgimento do primeiro computador eletrônico, o ENIAC (*Electronic Numerical Integrator and Computer*) de 1939 a 1946, Figura 1.1. Entre o final da década de 1940 e início da década de 1950, foi desenvolvida a linguagem de programação *Assembly* com o objetivo de facilitar o trabalho de codificação de um programa de computador. Nessa ocasião os códigos numéricos (binários ou hexadecimais) da linguagem de máquina foram substituídos por um código alfabético que era muito mais fácil de ser utilizado, pois montar um programa em linguagem de máquina era uma atividade muito dispendiosa (como poderá ser constatado nos próximos capítulos). Um programa que "rodasse" em meia hora para conseguir um tipo de processamento poderia levar para ser escrito, ou seja, montado talvez um, dois ou mais dias de trabalho. No final do capítulo 11 é dada uma ideia deste tipo de trabalho com um simples programa para apresenta a mensagem "alô mundo". A primeira linguagem de programação de alto nível surgiu em 1954 com o lançamento da linguagem FORTRAN (*FORmula TRANslator*).

A linguagem para programação de computadores *Assembly* possui uma estrutura sintática particular. Ela é formada por um conjunto de instruções que, por meio de códigos mais legíveis, representam as instruções do código de máquina.

As instruções da linguagem *Assembly* são conhecidas pelo nome de **mnemônicos** (lê-se menemônicos). É muito mais fácil olhar para um *mnemônico* e lembrar o que ele faz do que seu equivalente em código binário ou hexadecimal, legível apenas pelo microprocessador (CPU¹) do computador.

A linguagem de programação de computadores *Assembly* tem ainda uma grande vantagem sobre a linguagem de máquina, que é o fato de requer menos atenção a detalhes que são exigidos para programar em linguagem de máquina. Desta forma, é uma linguagem de fácil alteração se comparada com a linguagem de máquina.

O programa utilizado para compilar um programa escrito em linguagem de montagem (*assembly*), tornando-o executável em um computador chamasse *assembler*. Um programa **Assembler** é basicamente o ambiente de programação. É a ferramenta responsável por traduzir o programa-fonte (escrito em linguagem *Assembly*) para o programa objeto (programa em código de máquina) a ser interpretado por um processador. O programa *Assembler* é uma ferramenta que tem características semelhantes em alguns aspectos aos compiladores para uma determinada linguagem de alto nível.

O erro em confundir *Assembler* (programa montador - compilador) com *Assembly* (linguagem de montagem) é muito comum. Por exemplo, é comum dizer que se programa nas linguagens Delphi, Turbo Pascal, Turbo C, Visual Basic, Quick Basic, Visual C etc. Observe a Tabela 1.1 com os nomes das principais linguagens de programação de computadores e seus respectivos ambientes de programação.

Tabela 1.1 - Ambientes de Programação versus Linguagens de Programação

Linguagem	Ambiente de Programação
Assembly	MASM (Macro Assembler - Microsoft)
	MASM (Macro Assembler - IBM)
	TASM (Turbo Assembler - Borland)
	emu8086
C	Visual C / Visual Studio (Microsoft)
	Turbo C (Borland)
C++	GPP (GNU Project)
	Borland C++ / C++ Builder (Borland)
	Visual C++ (Microsoft)
Object Pascal	Delphi (Borland)
	Turbo Pascal (Borland)
	Kylix (Borland)
	Free Pascal Compiler (Software Livre)

Confundir nomes de compiladores com nomes de linguagens de programação de computadores é inadmissível para um profissional que diz pertencer à área de Tecnologia da Informação, mais precisamente para aquele que se diz um desenvolvedor de *software*, o que infelizmente é muito comum no Brasil. Jamais confunda o nome do ambiente de programação (seja em DOS, Windows, Linux) com a linguagem de programação que se utiliza.

1.2 - Por que aprender Assembly?

A aprendizagem de uso da linguagem de programação *Assembly* por parte de estudantes foi sempre cercada de grande misticismo em relação a ser considerada uma linguagem de difícil aprendizagem. Não se trata de ser uma linguagem de difícil aprendizagem, mas uma linguagem de aprendizagem complexa uma vez que diversos detalhes operacionais no uso de certo microprocessador precisa ser considerado para a execução de diversas operações que são consideradas simples e de fácil assimilação em linguagens de alto nível.

Existe certo preconceito na utilização da linguagem *Assembly* por parte de algumas pessoas. Muitos afirmam que programar nessa linguagem é coisa para "louco". Felizmente estão errados, pois quem programa nessa linguagem não é

¹ Central Processing Unit - Unidade Central de Processamento.

louco. É um profissional que conhece e sabe usar melhor os requisitos mais íntimos de um microprocessador, e, por conseguinte, sabe controlar melhor as funções de um computador digital.

Grosso modo pode-se comparar programar em linguagem *Assembly* com o fato de dirigir um automóvel com câmbio mecânico. Programar em uma linguagem de alto nível é como dirigir um automóvel que tem câmbio automático. Para quem gosta de dirigir, a verdadeira emoção está em controlar um automóvel com sistema mecânico de câmbio, pois o automático tira a sensação de controlar verdadeiramente a máquina, mas não tira sua dirigibilidade. Assim sendo, a primeira motivação para aprender a linguagem *Assembly* é ter vontade de controlar melhor as funções internas de um microprocessador. Desta forma, é possível desenvolver programas mais eficientes.

Muitos afirmam que a linguagem *Assembly* não é mais usada, que aprendê-la é perda de tempo. Como diria o menino-prodígio "Santa ignorância, Batman!". *Assembly* é a única linguagem que dá a um programador a capacidade de controlar totalmente as funções internas de um computador. Tanto que é comum muitas linguagens de programação de alto nível possuírem algum tipo de interação com rotinas de programas escritos em linguagem *Assembly*.

A linguagem de programação *Assembly* é muito utilizada na criação e desenvolvimento de rotinas escritas nas formas de DLLs, *drivers* (para controle de periféricos), programas embutidos para computadores de bordo, entre outras aplicações que podem ser encontradas no mercado. *Assembly* é uma linguagem de programação usada para se "conversar" diretamente com o *hardware* de um computador. Na década de 1980, século XX, um programa aplicativo de planilha eletrônica muito conhecido denominado Lotus 1-2-3 fez muito sucesso. Esse programa foi totalmente escrito em *Assembly*.

Não pense você que os melhores jogos do mercado são apenas escritos em linguagem de alto nível. Sempre existe em algum canto do programa algum trecho escrito em *Assembly*. Conhecer a linguagem *Assembly* é conhecer a verdadeira liberdade de programar um computador.

Os programas escritos em linguagem *Assembly* são rápidos e de pequeno tamanho (após a compilação) se comparados com códigos similares escritos em linguagem de alto nível. Em contrapartida, seu código-fonte é sempre maior do que o escrito em uma linguagem de alto nível. Por esta razão muitos acham que programar nessa linguagem é mais difícil, quando na verdade é apenas um pouco mais trabalhoso.

Norton & Socha (1988) afirmam que "em relação a todas as outras linguagens de programação, a linguagem *Assembly* é o denominador comum entre elas". Acrescentam ainda que aprender a linguagem *Assembly* é entender melhor o funcionamento de um microprocessador dentro de um computador, e torna-se possível entender melhor muitos dos elementos encontrados em outras linguagens de alto nível.

Como desvantagem no uso da linguagem de programação *Assembly* pode-se apontar o fato desta linguagem estar vinculada ao microprocessador em uso. A linguagem *Assembly* usada num microprocessador Intel ou AMD (para computadores da linha IBM-PC) é diferente da usada em um microprocessador da Motorola, por exemplo, o que obrigaria um programa a ser reescrito de um microprocessador Intel para um Motorola. Outra desvantagem é o fato da linguagem *Assembly* exigir do programador bom nível de conhecimento da arquitetura do computador em uso, principalmente na parte que tange ao uso do processador a ser programado e sua relação com os periféricos conectados.

Há ainda a questão do tamanho dos códigos de programa, que tendem a ser grandes, pois são necessárias muitas instruções para a realização de tarefas muitas vezes pequenas. Mas isto ocorre devido ao fato de se estar programando um computador praticamente *byte a byte* em sua memória e não por meio de instruções em alto nível. No entanto, a estrutura de escrita de um programa *Assembly* é muito mais simples que a estrutura utilizada nas linguagens de alto nível.

1.3 - Restrições deste trabalho

O objetivo desta obra é fazer um trabalho de introdução e apresentação dos fundamentos iniciais e básicos da linguagem de programação *Assembly 8086/8088*. Não se trata de um trabalho de grande profundidade técnica. Para tanto, está sendo levado em consideração o uso de um microcomputador da família IBM-PC dotado de um microprocessador Intel ou AMD que operem o modo 8086/8088.

O principal fator que levou à escolha da linguagem *Assembly* em modo 8086/8088 é o fato de ser o primeiro padrão dessa linguagem ensinado em vários cursos nas escolas do Brasil e no restante do mundo até os dias atuais para microcomputadores da família IBM-PC.

Outro fator considerado é a disponibilidade dos programas **DEBUG** da Microsoft (encontrado em todas as versões dos sistemas operacionais MS-DOS e nas versões 32 *bits* do sistema operacional Microsoft Windows XP, Vista, 7, 8, 8.1 e 10. Infelizmente o modo 64 *bits* dos sistemas operacionais Microsoft Windows não possui o programa **DEBUG**), **DOS Debug** (desenvolvido para o sistema operacional FreeDOS) e da ferramenta de emulação assembler **emu8086** (que pode ser executada em todas as versões do sistema operacional Microsoft Windows, tanto em modo 32 *bits* como em modo 64 *bits*). Essas ferramentas operam com base na estrutura de registradores de 16 *bits* por estarem voltadas ao padrão 8086/8088, exceto o programa **DOS Debug** que possibilita trabalhar com registradores de 16 e 32 *bits*.

Em relação aos programas de depuração (**DEBUG** e **DOS Debug**) será dada atenção ao programa **DOS Debug** por ser uma ferramenta distribuída de forma livre, não se tendo nenhum problema de direito autoral com seu uso, além deste ser distribuído com seu código fonte que poderá ser usado como fonte de estudo após a apresentação desta obra.

O programa **emu8086** que também será usado é um ambiente de programação e simulação do modo de operação interna de um microprocessador (micro controlador) padrão 8086/8088. A vantagem dessa ferramenta é ser executada em modo gráfico, além de apresentar todos os detalhes da operação de um microprocessador de um modo mais concreto, pode ser executado no Microsoft Windows XP, 7, 8, 8.1 e 10 de 32 ou de 64 *bits*. No entanto, há de se considerar que o programa **emu8086** é um *software* pago, devendo-se adquirir sua licença para uso além do tempo de experiência. A Figura 1.2 exibe algumas telas do programa **emu8086**.

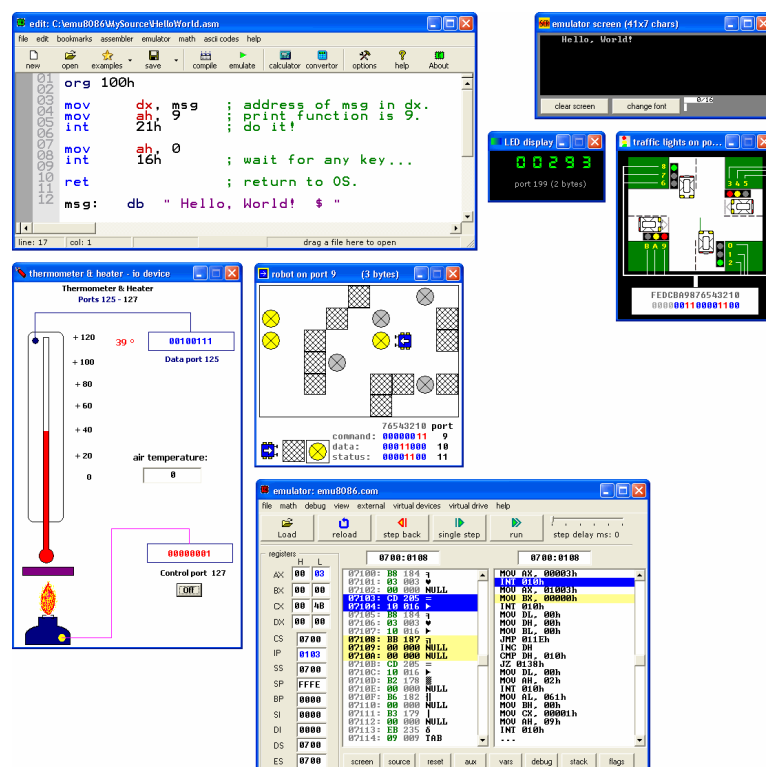


Figura 1.2 - Telas do Programa emu8086.

Como ambiente inicial de aprendizagem será utilizada a ferramenta **DOS Debug** e posteriormente será utilizado o programa **emu8086**.

Pelo fato da programação de computadores em linguagem *Assembly* para processadores de 32 ou 64 *bits* ser um pouco mais complexa que os objetivos desta obra este modo de operação não será aqui abordado. No entanto, a visão dada em modo 16 *bits* permitirá fácil entendimento de outros modos de uso.

Para estudar o conteúdo aqui apresentado, é necessário que o leitor possua alguns pré-requisitos básicos:

1. Sólidos conhecimentos das técnicas de programação (programação sequencial, utilização de decisões, laços de repetição, estruturas de dados homogêneas e heterogêneas, sub-rotinas e passagens de parâmetros) com o uso

de algoritmos computacionais², e preferencialmente tenha conhecimentos básicos de estruturas de dados³, principalmente no que tange às técnicas de controle de listas (pilhas e filas), além de conhecimentos sobre o tema organização de computadores.

2. Apesar de não ser obrigatório, é interessante possuir conhecimentos de uso e aplicação de alguma linguagem de programação de alto nível, preferencialmente Pascal, C, ou C++⁴.
3. Grande predisposição pessoal para aprender a utilizar a linguagem *Assembly*, pois talvez seja necessário estudar um determinado capítulo mais de uma vez e recorrer a materiais adicionais para suprir alguma deficiência que possa existir neste livro por ser o seu conteúdo básico e introdutório.
4. Conhecimento da manipulação de bases numéricas (binário, decimal, hexadecimal, octal). Conhecer e dominar as etapas do processo de cálculo nas várias bases numéricas⁵.

Cabe lembrar que os pré-requisitos de conhecimento mínimos necessários para este estudo não serão tratados no livro por estarem fora do seu escopo básico.

1.4 - Por que usar o padrão 8086 e não outro mais recente?

Muitos podem se perguntar por qual motivo esta obra, contemporânea, faz uso do padrão 8086 de 1978 ao invés de fazer uso, por exemplo, de um modelo mais recente como os modelos i7 da Intel ou os modelos FX-Seies, Sempron, Athlon ou Opteron da AMD? Não seria melhor usar o padrão 64 *bits* ao invés de usar o padrão 16 *bits*?

A resposta a essa questão, pode parecer, muito simplista aos olhos da maioria, mas de fato não o é. O que na prática pedagógica de sala de aula adiantaria pensar em usar os modelos mais modernos de processadores de 64 *bits* ou até processadores de maior capacidade que venham a ser lançados com um conjunto de instruções muito extenso? Seria isso adequado ao educando? A resposta mais simples a tais questões seria que uma coisa não tem a ver com a outra, o fato de se fazer uso de um modelo mais sofisticado de processador não garante efetivamente boa aprendizagem por parte do educando. Seria apenas uma atitude em vão para ficar “bonito na fita”.

A ideia deste trabalho é passar as bases essenciais de entendimento do uso da linguagem *Assembly* operada a partir do padrão Intel/AMD. Se o educando entender adequadamente o funcionamento do padrão 8086 estará apto a entender o funcionamento de outros padrões mais sofisticados. É o mesmo raciocínio de se aprender a dirigir, nenhuma autoescola brasileira possui para aprendizagem carros muito sofisticados como, por exemplo, Ferrari ou Lamborghini; possuem modelos simples, na sua maioria com motorização 1.0. E nem por isso o motorista deixa de aprender a dirigir e poderá futuramente dirigir carros de alto desempenho.

Assim sendo, considere o conteúdo deste livro como a aprendizagem da linguagem *Assembly* em modelo 1.0 e posteriormente poderá ampliar seu foco de aprendizagem em um modelo mais sofisticado. Lembre-se de que ninguém atinge o último degrau de uma longa escada sem passar pelo primeiro degrau.

1.5 - Arquitetura: Princípios básicos

Antes de iniciar o estudo e aplicação da linguagem *Assembly* é pertinente abordar algumas questões relacionadas a arquitetura dos computadores padrão IBM-PC.

² Sugere-se a leitura da obra "Algoritmos - Lógica para o Desenvolvimento de Programação de Computadores", dos autores José Augusto N. G. Manzano e Jayr Figueiredo, Editora Érica.

³ É aconselhável consultar a obra "Estrutura de Dados Fundamentais", do autor Sílvio do Lago Pereira, Editora Érica.

⁴ Seria conveniente, neste caso, consultar um dos livros: "Estudo Dirigido de Linguagem C", do autor José Augusto N. G. Manzano; "Free Pascal - Programação de Computadores - Guia Básico de Orientação e Desenvolvimento para Programação em Linux, MS-Windows e MS-DOS", dos autores José Augusto N. G. Manzano e Wilson Y. Yamatumi; "Programação de Computadores com C++ ANSI (ISO/IEC 14882:2003) - Guia Prático de Orientação e Desenvolvimento", do autor José Augusto N. G. Manzano, Editora Érica.

⁵ Para aprofundamento de alguns dos detalhes aqui apresentados, o leitor pode consultar as obras: "Estudo Dirigido de Informática Básica", publicada pela Editora Érica, dos autores André Luiz N. G. Manzano e Maria Izabel N. G. Manzano, e "Introdução à Informática", publicada pela Editora Pearson Education, do autor Peter Norton.

O termo computador advém da palavra *compute* que no idioma Latim refere-se a calcular. Um computador é um equipamento eletrônico que efetua o processamento de dados. O processamento realizado por um computador pode ser feito nas esferas matemática e lógica. Como equipamento, esta máquina é composta por uma série de circuitos interligados que permitem a realização de diversas tarefas controladas por programas.

Os computadores usados no dia-a-dia da humanidade obedecem a uma estrutura computacional (arquitetura) chamada Eckert-Mauchly, proposta por John von Neumann e criada por John Presper Eckert e John William Mauchly. A arquitetura proposta por von Neumann propõe um equipamento computacional a partir de quatro seções operacionais, sendo:

- ◆ Unidade Lógica e Aritmética (Arithmetic Logic Unit – ULA) – responsável por executar as operações matemáticas e lógicas;
- ◆ Unidade de Controle – responsável pela execução de instruções junto ao processador, que é por sua natureza o “cérebro” de um computador por transformar dados em;
- ◆ Memória Principal – dispositivo responsável em armazenar dados e códigos de programas;
- ◆ Dispositivos de Entrada e Saída – responsáveis pelas operações de comunicação efetuadas em um ser humano e um computador. Estes dispositivos são conhecidos como periféricos.

Todas estas partes encontram-se interligadas por meio de vias de acesso denominadas barramentos (*bus*).

Todas as ações computacionais como já informado, são executadas por meio de programas, que são por sua natureza, um conjunto de instruções organizadas de forma lógica com o objetivo de realizar certa ação e operação por meio de instruções. Uma instrução de programa é uma ordem enviada a um computador por meio da definição de um comando e um dado. No contexto da programação em baixo nível uma instrução é definida a partir de um código de operação (*opcode*) e de um ou mais operandos que são elementos que representam dados e a armazenagem de endereços de acesso à memória. O *opcode* representa a instrução a ser executada e o operando representa o dado a ser tratado. O *opcode* é representado por um código binário que identifica uma instrução e determina o tipo de ação a ser realizada pelo processador. Uma instrução *Assembly* pode conter, além do *opcode* opcionalmente um ou mais campos de *operandos*. Uma instrução somente com *opcode* caracteriza-se por ser uma instrução que não faz acesso à memória. Assim sendo uma instrução pode ser definida a partir do layout:

OPCODE	OPERANDO 1	OPERANDO 2	...	OPERANDO N
--------	------------	------------	-----	------------

As instruções *Assembly* podem ser usadas para:

- ◆ Efetuar ações matemáticas e lógicas;
- ◆ Efetuar movimentação de dados no processador;
- ◆ Efetuar operações de entrada e saída de dados/informações;
- ◆ Efetuar o controle de desvios condicionais e a execução de laços.

A execução de instruções em um processador é efetuada pelos registradores, que são dispositivos que efetuam o armazenamento de dados e de informações utilizadas para a execução de instruções de um programa. Existem dois tipos de registradores:

- ◆ Registradores gerais;
- ◆ Registradores de propósito especial (segmento, deslocamento e estado).

Os registradores de propósito geral são usados no armazenamento de dados e sua movimentação entre a memória e o processador. Já os registradores de propósito especial são usados na execução de instruções, normalmente, gerenciados pela unidade de controle.

A execução de instruções de um programa *Assembly* pelo processador segue as seguintes etapas:

- ◆ Busca da instrução na memória do computador;
- ◆ Decodificação da instrução pelo processador;
- ◆ Busca do operando na memória do computador;
- ◆ Execução da instrução pelo processador;
- ◆ Escrita do resultado, normalmente, na memória do computador.

As etapas na execução das instruções pelo processador são internamente independentes. Enquanto certa instrução está na segunda etapa, a unidade de controle coloca a próxima instrução na primeira etapa. Esse tipo de operação é

conhecida como *pipeline*. As instruções de um programa são executadas pelo *pipeline* de forma simultânea, desde que, cada instrução seja processada em uma etapa diferente da outra.

Para que um programa seja executado é importante que este tenha acesso ao endereçamento de memória do computador. O endereço de memória determina o local onde o operando se encontra, sendo possível fazer uso de cinco tipos de endereçamento de memória, sendo:

- ◆ Endereçamento imediato (modo imediato);
- ◆ Endereçamento direto (modo direto);
- ◆ Endereçamento indireto (modo indireto);
- ◆ Endereçamento por registrador;
- ◆ Endereçamento indexado (modo indexado).

O endereçamento imediato é usado quando o valor de um operando é representado pelo próprio dado. Neste caso, sendo este um valor constante. Neste endereçamento o dado não precisa ser procurado na memória, o que faz com que sua execução seja rápida. No entanto, o tamanho do dado fica limitado ao tamanho do campo do operando.

O endereçamento direto é usado quando o valor do operando é o endereço de memória onde certo dado se encontra armazenado. Este modo de endereçamento é usado quando da necessidade de operar com valores que se alteram ao longo da execução do programa. Esta técnica de ação é mais lenta que o endereçamento imediato e mais rápida que o endereçamento indireto. Este tipo de endereçamento limita-se ao tamanho do campo do operando que é reduzido, não sendo possível endereçar toda a memória.

O endereçamento indireto é usado quando o valor do operando é uma referência a uma posição de memória que contenha o endereço do próprio operando. O endereço apontado é um ponteiro vinculado a certo dado na memória principal. Não há para este tipo de endereçamento limitação de células endereçáveis de memória, o endereço da memória principal não fica limitado ao tamanho do operando, como ocorre no endereçamento direto. O endereçamento indireto é mais lento que o modo direto. Este realiza mais acessos a memória: busca de endereço (ponteiro) e busca de dado.

O endereçamento por registrador é usado quando há a necessidade do valor do operando fazer referência a um dado (endereçamento por registrador direto) ou referencia a um endereço (endereçamento por registrador indireto) quando se realiza o apontamento para um local de memória que contenha um dado. O acesso por registradores ocorre de forma mais rápida do que o acesso realizado a memória e as outras maneiras de endereçamento existentes. O endereçamento por registrador não é adequado para a transferência de dados entre memória e processador, possui como limitação a quantidade de registradores gerais disponíveis que no caso da arquitetura Intel são quatro identificados como **A**, **B**, **C** e **D**.

Por fim, não menos importante o endereçamento indexado é usado a partir da soma do campo operando com o valor que esteja armazenado em um registrador, sendo este chamado de índice. Este tipo de endereçamento é adequado para a manipulação de estruturas de dados como matrizes de uma dimensão na forma de vetores e de mais dimensões na forma de tabelas.

Note que foi muito mencionado a palavra “memória”. No contexto computacional a memória de um computador é um dispositivo que permite estabelecer o armazenamento de dados e programas de forma temporária (memória principal) ou “permanente” (memória secundária). O termo “permanente” entre aspas se refere deste tipo de armazenamento ser controlado pelo usuário.

A memória principal, conhecida como memória de acesso randômico (Random Access Memory – RAM) tem por característica ser volátil e manter dados e programas enquanto os componentes internos do computador se encontram energizados.

A memória secundária caracteriza-se por ser usada para o armazenamento permanente de dados e programas na forma de mídias externas (discos rígidos, discos ópticos, pen-drives, entre outras possibilidades).

1.6 Código ASCII

O código ASCII (*American Standard Code for Information Interchange*) foi desenvolvido a partir de 1963 e concluído em 1968, com a participação de várias companhias de comunicação norte-americanas, com o objetivo de substituir o até então utilizado código de Baudot.

O código de Baudot usava apenas 5 bits, o que possibilitava obter apenas 32 combinações diferentes de caracteres (números e letras maiúsculas), muito utilizado entre teleimpressores.

Este apêndice apresenta a tabela de códigos ASCII (lê-se *asquí* e não *asqui* 2) com a definição dos seus códigos numéricos em notação decimal, notação binária e notação hexadecimal.

O código ASCII estabelece a utilização de 128 símbolos (de 0 a 127) diferentes, utilizando efetivamente um conjunto de 7 bits dos 8 bits permitidos. O bit mais significativo permanece zerado. Do conjunto de 128 símbolos, 96 (de 32 a 127) são caracteres imprimíveis (números, letras minúsculas, letras maiúsculas e sinais de pontuação) e 32 (de 0 a 31) são caracteres não imprimíveis (retorno de carro, retrocesso, salto de linha, entre outros). Estes 128 caracteres são definidos como padrão para qualquer tipo de computador e seguem a estrutura da tabela seguinte.

Dec	Bin	Hex	Caractere
000	000000000	00	NUL
001	000000001	01	SOH
002	000000010	02	STX
003	000000011	03	ETX
004	000000100	04	EOT
005	000000101	05	ENQ
006	000000110	06	ACK
007	000000111	07	BEL
008	000001000	08	BS
009	000001001	09	HT
010	000001010	0A	LF
011	000001011	0B	VT
012	000001100	0C	FF
013	000001101	0D	CR
014	000001110	0E	SO
015	000001111	0F	SI
016	000100000	10	DEL
017	000100001	11	DC1
018	000100010	12	DC2
019	000100011	13	DC3
020	000100100	14	DC4
021	000100101	15	NAK
044	001011100	2C	,
045	001011101	2D	-
046	001011110	2E	.
047	001011111	2F	/
048	001100000	30	0
049	001100001	31	1
050	001100010	32	2
051	001100011	33	3
052	001100100	34	4
053	001100101	35	5
054	001100110	36	6
055	001100111	37	7

Dec	Bin	Hex	Caractere
022	000101110	16	SYN
023	000101111	17	ETB
024	000110000	18	CAN
025	000110001	19	EM
026	000110010	1A	SUB
027	000110011	1B	ESC
028	000111000	1C	FS
029	000111001	1D	GS
030	000111010	1E	RS
031	000111011	1F	US
032	001000000	20	SPACE
033	001000001	21	!
034	001000010	22	"
035	001000011	23	#
036	001000100	24	\$
037	001000101	25	%
038	001000110	26	&
039	001000111	27	'
040	001010000	28	(
041	001010001	29	)
042	001010010	2A	*
043	001010011	2B	+
086	010101110	56	v
087	010101111	57	w
088	010110000	58	x
089	010110001	59	y
090	010110010	5A	z
091	010110011	5B	[
092	010110100	5C	\
093	010110101	5D	]
094	010110110	5E	^
095	010110111	5F	_
096	011000000	60	`
097	011000001	61	a

Dec	Bin	Hex	Caractere
056	00111000	38	8
057	00111001	39	9
058	00111010	3A	:
059	00111011	3B	;
060	00111100	3C	<
061	00111101	3D	=
062	00111110	3E	>
063	00111111	3F	?
064	01000000	40	@
065	01000001	41	A
066	01000010	42	B
067	01000011	43	C
068	01000100	44	D
069	01000101	45	E
070	01000110	46	F
071	01000111	47	G
072	01001000	48	H
073	01001001	49	I
074	01001010	4A	J
075	01001011	4B	K
076	01001100	4C	L
077	01001101	4D	M
078	01001110	4E	N
079	01001111	4F	O
080	01010000	50	P
081	01010001	51	Q
082	01010010	52	R
083	01010011	53	S
084	01010100	54	T
085	01010101	55	U

Dec	Bin	Hex	Caractere
098	01100010	62	b
099	01100011	63	c
100	01100100	64	d
101	01100101	65	e
102	01100110	66	f
103	01100111	67	g
104	01101000	68	h
105	01101001	69	i
106	01101010	6A	j
107	01101011	6B	k
108	01101100	6C	l
109	01101101	6D	m
110	01101110	6E	n
111	01101111	6F	o
112	01110000	70	p
113	01110001	71	q
114	01110010	72	r
115	01110011	73	s
116	01110100	74	t
117	01110101	75	u
118	01110110	76	v
119	01110111	77	w
120	01111000	78	x
121	01111001	79	y
122	01111010	7A	z
123	01111011	7B	{
124	01111100	7C	
125	01111101	7D	}
126	01111110	7E	~
127	01111111	7F	△

Os códigos ASCII existentes na faixa de 0 até 31 são de controle para envio de dados a um periférico de impressão, para controlar a comunicação entre dois computadores por rede telefônica ou definir algum tipo de comunicação entre o computador de um agente externo. Quando direcionados para a tela do monitor de vídeo, esses caracteres são substituídos por algum símbolo imprimível que não é nenhum dos caracteres imprimíveis. A título de curiosidade observe a tabela seguinte.

Dec	Bin	Hex	Controle	Imprimível	Teclas	Descrição
000	00000000	00	NUL		Ctrl + @	Caractere nulo
001	00000001	01	SOH	☺	Ctrl + A	Início de cabeçalho
002	00000010	02	STX	●	Ctrl + B	Início de texto
003	00000011	03	ETX	♥	Ctrl + C	Fim de texto
004	00000100	04	EOT	♦	Ctrl + D	Fim da transmissão
005	00000101	05	ENQ	♣	Ctrl + E	Caractere de consulta

Dec	Bin	Hex	Controle	Imprimível	Teclas	Descrição
006	00000110	06	ACK	♠	Ctrl + F	Confirmação
007	00000111	07	BEL	▪	Ctrl + G	Alarme ou chamada
008	00001000	08	BS	▣	Ctrl + H	Retrocesso
009	00001001	09	HT	○	Ctrl + I	Tabulação horizontal
010	00001010	0A	LF	▣	Ctrl + J	Avanço de linha
011	00001011	0B	VT	♂	Ctrl + K	Tabulação vertical
012	00001100	0C	FF	♀	Ctrl + L	Avanço de página
013	00001101	0D	CR	♪	Ctrl + M	Retorno de carro
014	00001110	0E	SO	♪	Ctrl + N	Shift desativado
015	00001111	0F	SI	☼	Ctrl + O	Shift ativado
016	00010000	10	DEL	▶	Ctrl + P	Caractere de supressão
017	00010001	11	DC1	◀	Ctrl + Q	Contr. de dispositivo
018	00010010	12	DC2	↑	Ctrl + R	Contr. de dispositivo
019	00010011	13	DC3	!!	Ctrl + S	Contr. de dispositivo
020	00010100	14	DC4	¶	Ctrl + T	Contr. de dispositivo
021	00010101	15	NAK	§	Ctrl + U	Confirmação negada
022	00010110	16	SYN	■	Ctrl + V	Sincronismo
023	00010111	17	ETB	↓	Ctrl + W	Fim de bloco de texto
024	00011000	18	CAN	↑	Ctrl + X	Cancelamento
025	00011001	19	EM	↓	Ctrl + Y	Fim de meio de dados
026	00011010	1A	SUB	→	Ctrl + Z	Substituição
027	00011011	1B	ESC	←	Ctrl + 9	Diferenciação
028	00011100	1C	FS	└	Ctrl + /	Separador de arquivo
029	00011101	1D	GS	↔	Ctrl + :	Separador de grupo
030	00011110	1E	RS	▲	Ctrl + ^	Separador de registro
031	00011111	1F	US	▼	Ctrl + _	Separador de unidade

Além dos 128 códigos padronizados, os computadores padrão IBM-PC possuem uma extensão a mais da tabela ASCII, totalizando 256 caracteres. No entanto, essa extensão não deve ser considerada parte da tabela ASCII, mas um complemento definido, que somente é válido em microcomputadores da linha IBM-PC operando com ambiente de software Microsoft. Os códigos estendidos situam-se na faixa de 128 até 255 e não serão apresentados.

```

0D82:0100 B402      MOV     AH,02
0D82:0102 B241      MOV     DL,41
0D82:0104 CD21      INT     21
0D82:0106 CD20      INT     20
0D82:0108 69        DB      69

```

Capítulo 2

CONCEITOS FUNDAMENTAIS

Este capítulo trata dos conceitos essenciais e preliminares da estrutura computacional, que fornecem a base para o estudo inicial da linguagem de programação Assembly para os microprocessadores da família x86, mais precisamente o 8086 ou seu irmão 8088. Assim sendo, destacam-se detalhes do sistema computacional (unidade central de processamento, unidade de memória e unidade de entrada e saída), a organização interna dos dados (nibble, byte, word, double word e quad word), o uso dos registradores (gerais, segmento, apontamento e estado), as interrupções (hardware, exceção e software), os segmentos, deslocamentos e endereçamento de memória.

2.1 Sistema computacional

Um sistema computacional é o conjunto de componentes que formam a configuração de um computador, incluindo seu sistema operacional e todos os periféricos conectados a esse computador. Dentre os vários componentes, os mais importantes são a unidade central de processamento, as unidades de memória (memória RAM, memória ROM e memória de massa ou secundária), as unidades de entrada e saída, a unidade aritmética, unidade de controle lógico, unidade de saída e registradores as quais são descritas a seguir. A Figura 2.1 mostra um modelo esquemático da organização interna de um computador em linha geral.

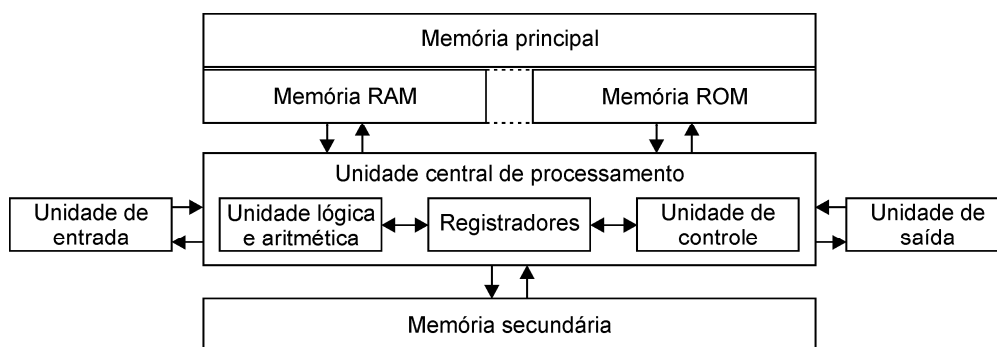


Figura 2.1 - Organização geral de um computador (TOKHEIN, 1985, p.2).

A memória principal se divide em memória RAM (*Random Access Memory* - memória de acesso randômico) e memória ROM (*Read Only Memory* - memória somente de leitura).

A memória RAM é o local onde os programas e dados são armazenados para a operação do sistema. Sendo volátil, ela não armazena os dados e programas para uso futuro. Quando o sistema é desligado, todo e qualquer dado ou programa armazenado nessa memória é perdido. Para sanar essa deficiência aparente, usa-se a memória secundária formada pelas unidades de armazenamento de dados conectadas ao computador, sendo a principal delas o HD (*Hard disk* - disco rígido) conectado dentro do gabinete.

Na memória ROM se encontra armazenado o programa básico de entrada e saída do sistema, e os dados armazenados nessa memória somente são lidos, não sendo nenhum dado escrito.

O sistema computacional de um microcomputador padrão IBM-PC (PC de *Personal Computer*) que faz uso da tecnologia Intel/AMD possui em seu microprocessador as várias unidades ligadas por um conjunto de componentes eletrônicos denominados *bus* (também conhecido como *barramento* ou *via*). Por meio do *bus* os sinais binários são transportados entre as unidades de barramento e de execução para a comunicação com a memória. Para que as operações sejam internamente executadas, existem o *bus de controle lógico*, o *bus de endereçamento de dados*, o *bus de operação externa* e o *bus de dados*. Mais detalhes sobre esta parte encontram-se no tópico sobre a arquitetura 8086, neste capítulo.

2.1.1 - Unidade central de processamento

A *unidade central de processamento*, também denominada CPU (*Central Processing Unit*), é erroneamente confundida com o gabinete do equipamento. O gabinete é apenas o compartimento (invólucro) que protege a placa mãe (*mainboard*) e seus componentes eletrônicos, entre eles o mais importante componente denominado CPU.

Dos componentes eletrônicos existentes em um microcomputador a CPU (microprocessador) é o mais importante, na qual se encontra a unidade de controle lógica e aritmética do computador, conhecida pelo nome ALU (*Aritmetic and Logic Unit*, ou ULA, unidade lógica e aritmética). Ela é responsável por executar as operações matemáticas e o controle lógico das ações que a máquina executa.

De forma mais ampla, a CPU tem a função de controlar a leitura e escrita de dados e informações nos registradores gerais, temporários, de endereçamentos e de estado (registradores são células de memória usadas para armazenamento temporário de dados, localizadas na memória RAM que se assemelham ao conceito de variáveis), além de controlar o tráfego de dados que passa pelo barramento de dados e executar as instruções de um programa em uso.

2.1.2 - Unidade de memória

É a parte responsável por armazenar dados para que eles possam ser controlados posteriormente. A unidade de memória opera em conjunto com a CPU, mas não faz parte direta da CPU. A unidade de memória se divide em duas partes, a saber:

- ♦ A memória principal conhecida pelo nome de memória RAM (Random Access Memory), ou seja, memória de acesso aleatório. Nessa memória a CPU executa as ações de controle, na qual se encontram os registradores. As informações armazenadas nessa memória são voláteis, ou seja, temporárias. Após o desligamento do computador os dados armazenados nos registradores são automaticamente perdidos. A memória principal é responsável por manter um programa em execução, o qual é controlado pela CPU. Pelo fato de essa memória ser volátil, torna-se necessária a utilização de memórias secundárias, quando se deseja manter uma informação ou dados gravados.
- ♦ A memória secundária também referenciada pelo termo memória auxiliar ou memória de massa. Nesse tipo de memória os dados de um programa permanecem armazenados por um determinado espaço de tempo. São memórias secundárias as fitas magnéticas, os discos magnéticos (disquetes e discos rígidos), os discos ópticos (CDs, CD-Rs, CD-RWs), cartões de memória, entre outros mecanismos que possam ser utilizados como elementos de armazenamento.

2.1.3 - Unidades de entrada e de saída

As unidades de entrada e de saída são os componentes responsáveis pela comunicação do mundo exterior com um computador e vice-versa. Efetivamente essa unidade faz uso do *bus* de operação externa.

Dos componentes para a efetivação de entrada, os mais conhecidos e utilizados são o teclado, o mouse, os scanners de leitura óptica (semelhantes aos usados nos caixas de supermercados). Com esse tipo de componente é possível obter dados do mundo externo para que sejam armazenados na memória principal e manipulados pela CPU, e posteriormente podem ser gravados em uma memória secundária.

Dos componentes para a efetivação de saída, os mais conhecidos são o vídeo e a impressora. Com esses componentes é possível enviar para o mundo externo os dados e informações existentes em uma memória secundária, ou mesmo fornecidos anteriormente por um componente de entrada.

2.2 - Organização de dados

Os valores binários utilizados internamente pelos vários circuitos de um computador digital para a formação dos caracteres alfanuméricos e também para a definição de suas operações internas são agrupados em conjuntos de *bits* (binary digit - dígito binário). *Bit* é a menor informação que pode ser manipulada em um computador.

O conjunto de *bits* mais conhecido é o *byte* (conjunto de oito *bits*, em português octeto), no entanto há outras formas de representar conjuntos numéricos de *bits* em um computador, que são menos discutidas e, por conseguinte, menos conhecidas, tais como *nibble*, *word*, *double word* e *quad word*. Esses conjuntos de agrupamento de *bits* são responsáveis pela forma de organização e representação interna dos dados na memória de um computador digital.

Partindo da premissa de que um computador digital manipula *bits* de dados em memória, e esses *bits* são representados apenas pelos valores binários 1 (um - circuito ligado) e 0 (zero - circuito desligado), torna-se óbvia a necessidade de combinar esses valores para que se consigam sinais diferentes.

Se um computador digital possuísse a capacidade de manipular apenas dois *bits*, ele conseguiria representar no máximo quatro valores diferentes, sendo eles: 00, 01, 11 e 10, ou seja, o valor 2 (quantidade de valores binários - 0 e 1) elevado a 2 (capacidade máxima de combinação entre os valores binários) num total de quatro combinações.

A partir desse detalhe técnico passou-se a considerar a necessidade de uma capacidade de manipulação de dados maior em um computador digital, e à medida que o tempo passa, os computadores digitais passam também a manipular quantidades maiores de dados. Esse cálculo (capacidade máxima de manipulação da quantidade de dados) é sempre feito com múltiplos de 2.

Considerando a quantidade máxima anterior de combinação entre os valores binários e multiplicando esse valor por 2, ter-se-á o valor 4, ou então, basta efetuar o cálculo $2^2 = 4$. A partir desse raciocínio fica fácil compor a capacidade das próximas combinações a serem manipuladas em um computador, que serão 8, 16, 32, 64, 128 e assim por diante.

2.2.1 - Nibble

É um conjunto numérico de quatro *bits* (quarteto), sendo a menor estrutura numérica manipulada internamente em um computador. A capacidade de armazenamento baseada em um *nibble* é obtida a partir do cálculo de 2^4 , o que possibilita representar até 16 valores diferentes (em hexadecimal de 0 a F, em decimal de 0 a 15 e em binário de 0000 a 1111). Um *nibble* é usado para facilitar a representação de valores binários em formato hexadecimal. Os valores em formato hexadecimal são representados pelos símbolos ("0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D", "E" e "F"), perfazendo um total de dezesseis dígitos diferentes.

A Tabela 2.2 apresenta os dezesseis valores que podem ser armazenados em um *nibble* de memória no formato decimal, hexadecimal e binário. Observe a coluna identificada como **Binário** com o agrupamento de valores que formam o conjunto de quatro *bits* (*nibble*).

Tabela 2.1 - Valores que podem ser armazenados em um nibble de memória

Decimal	Hexadecimal	Binário	Decimal	Hexadecimal	Binário
0	0	0000	8	8	1000
1	1	0001	9	9	1001
2	2	0010	10	A	1010
3	3	0011	11	B	1011
4	4	0100	12	C	1100
5	5	0101	13	D	1101
6	6	0110	14	E	1110
7	7	0111	15	F	1111

Perceba que na coluna **Binário** encontram-se as dezesseis combinações possíveis de serem conseguidas com quatro valores binários.

2.2.2 - Byte

É um conjunto de oito *bits* (octeto). Com esta estrutura é possível representar numericamente até 256 (que equivale a 2^8) valores diferentes que podem ser utilizados por um computador digital, principalmente nos equipamentos da família IBM-PC.

A estrutura interna de um único *byte* é numerada de 7 (sete) a 0 (zero) da esquerda para a direita, sendo os *bits* de posição 7, 6, 5 e 4 os mais significativos e os *bits* de posição 3, 2, 1 e 0 os menos significativos. Desta forma a estrutura interna de um *byte* é representada pelo formato indicado junto a Tabela 2.2.

Tabela 2.2 - Estrutura de um byte de memória

7	6	5	4	3	2	1	0
Bits mais significativos				Bits menos significativos			
Nibble 1 (mais significativo)				Nibble 0 (menos significativo)			

É pertinente salientar que o *bit* 7 (primeiro *bit* na posição esquerda, ou *bit* mais significativo) é reservado como *bit de sinal* usado na representação de valores positivos ou negativos. O valor de um *byte* é positivo quando o *bit* 7 é representado pelo valor binário 0 (zero) e será negativo caso este valor seja representado pelo valor binário 1 (um). Por exemplo, o valor binário **00001010** representa em decimal **10** positivo, enquanto que **10001010** representa em decimal **10** negativo. Assim sendo, observe na Tabela 2.3 o formato de um *byte* a partir da ótica de uso do *bit de sinal*.

Tabela 2.3 - Formato de um byte de memória a partir do bit de sinal

7	6	5	4	3	2	1	0
Sinal	Magnitude						

Observe que a indicação *Sinal* corresponde ao uso do *bit de sinal* que determina se o valor nas demais posições é positivo ou negativo. Os *bits* que representam a *Magnitude* são usados na representação do valor absoluto em si, independentemente deste valor ser positivo ou negativo. Um valor **+10** ou **-10** é **10** independentemente deste ser positivo ou negativo.

Perceba que um *byte* é formado por um conjunto de dois *nibbles*. Assim sendo, há um *nibble* (zero) menos significativo e outro *nibble* (um) mais significativo. O *nibble* mais significativo está representado pelos *bits* situados nas posições de 7 a 4 e o *nibble* menos significativo está representado pelos *bits* situados nas posições de 3 a 0.

2.2.3 - Word

Representa um conjunto de dezesseis *bits*. A estrutura de um tipo de dado *word* é numerada numa sequência de 15 (quinze) a 0 (zero) *bits*, sendo os *bits* 15, 14, 13, 12, 11, 10, 9 e 8 considerados os mais significativos e os *bits* 7, 6, 5, 4, 3, 2, 1 e 0 os menos significativos. Com a estrutura de dados *word* é possível representar numericamente até 65.536 (2^{16}) valores diferentes. O tipo de dado *word* possui seu formato como indicado na Tabela 2.4.

Tabela 2.4 - Estrutura de um word de memória

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Byte mais significativo								Byte menos significativo							
Nibble 3				Nibble 2				Nibble 1				Nibble 0			
Nibble mais sig.												Nibble menos sig.			

Um dado *word* é formado por dois *bytes*. Assim sendo, há um *byte* menos significativo e outro mais significativo. O *byte* mais significativo está representado pelos *bits* situados nas posições de 15 a 8 e o menos significativo está representado pelos *bits* situados nas posições de 7 a 0. O tipo de dado *word* pode ser dividido em quatro *nibbles*. Neste caso o *nibble* 3 é o mais significativo e o 0 é o menos significativo. Os *nibbles* 2 e 1 são simples, sem nenhuma classificação estrutural.

Como apresentado o tipo de dado *word* pode representar 65.536 valores diferentes entre 0 (zero) e 65.535. Também é possível representar valores numéricos entre -32.766 até 32.767. Normalmente se utiliza o tipo de dado *word* para representar valores inteiros e deslocamentos de segmento.

2.2.4 - Double word

É um conjunto de dois *words*, ou seja, um conjunto de 32 *bits*. A estrutura de um dado do tipo *double word* é numerada de 31 (trinta e um) a 0 (zero). Assim sendo, um *double word* é formado por dois *words*, ou quatro *bytes*, ou ainda oito *nibbles*.

Com o tipo *double word* é possível representar 4.294.967.296 valores numéricos diferentes (ou seja, 2 elevado a 32) entre 0 (zero) e 4.294.967.295. Também é possível representar com o dado do tipo *double word* valores numéricos entre -2.147.483.648 e 2.147.483.647. Normalmente se utiliza *double word* para representar valores inteiros de 32 *bits*, valores de ponto flutuante de 32 *bits* e endereços segmentados.

2.2.5 - Quad word

É um conjunto de dois *double words*, ou seja, um conjunto de 64 *bits*. A estrutura de um dado do tipo *quad word* é numerada de 63 (sessenta e três) a 0 (zero). Assim sendo, um *quad word* é formado por dois *double words*, ou quatro *words*, ou oito *bytes*, ou ainda dezesseis *nibbles*.

Com o tipo *quad word* é possível representar 18.446.744.073.709.551.616 valores numéricos diferentes (ou seja, 2 elevado a 64) entre 0 (zero) e 18.446.744.073.709.551.615. Pode-se também representar com o dado do tipo *double word* valores numéricos entre -9.223.372.036.854.775.808 e 9.223.372.036.854.775.807. O tipo de dado *quad word* é normalmente encontrado em computadores digitais dotados de microprocessadores de 64 *bits*. Por esta razão não será detalhado esse tipo de dado, pois foge do propósito do livro.

2.2.6 - Unidades de medidas computacionais

O volume de dados a ser processado numa memória principal ou armazenado numa memória secundária é medido em relação à quantidade de *bytes*, como já exposto. A Tabela 2.5 apresenta um resumo de algumas unidades de medida computacional utilizadas na área da computação.

Tabela 2.5 - Unidade de medida computacional

Unidade	Quantidade de caracteres
Bit (b)	Conjunto de dois <i>bits</i> que possibilita a ativação e a desativação de recursos e circuitos eletrônicos. Menor dado que pode ser operacionalizado dentro de um computador.
Nibble (N)	Conjunto de quatro <i>bits</i> que possibilita a definição do número mínimo de algarismos binários necessários para representar uma cifra decimal. Base para o sistema BDC (<i>Binary-Coded Decimal</i>).
Byte (B)	Conjunto de oito <i>bits</i> que possibilita a definição e o uso de duzentos a cinquenta e seis (2 ⁸) símbolos para representação de caracteres numéricos, alfabéticos, de pontuação e gráficos (opcional).
KByte (KB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 KByte (<i>kilobyte</i>) equivale a 1.024 caracteres, sendo obtido a partir de 2 ¹⁰ .
Mbyte (MB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Mbyte (<i>megabyte</i>) equivale a 1.048.576 caracteres (1.024 KBytes), sendo obtido a partir de 2 ²⁰ .
Gbyte (GB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Gbyte (<i>gigabyte</i>) equivale a 1.073.741.824 caracteres (1.024 Mbytes), sendo obtido a partir de 2 ³⁰ .
Tbyte (TB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Tbyte (<i>terabyte</i>) equivale a 1.099.511.627.776 caracteres (1.024 Gbytes), sendo obtido a partir de 2 ⁴⁰ .
Pbyte (PB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Pbyte (<i>petabyte</i>) equivale a 1.125.899.906.842.624 caracteres (1.024 Tbytes), sendo obtido a partir de 2 ⁵⁰ .

Unidade	Quantidade de caracteres
Ebyte (EB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Ebyte (<i>exabyte</i>) equivale a 1.152.921.504.606.846.976 caracteres (1.024 Pbytes), sendo obtido a partir de 2^{60} .
Zbyte (ZB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Zbyte (<i>zettabyte</i>) equivale a 1.180.591.620.717.411.303.424 caracteres (1.024 Ebytes), sendo obtido a partir de 2^{70} .
Ybyte (YB)	Definição da quantidade de caracteres a ser utilizada e armazenada em memórias: principal e secundária. 1 Ybyte (<i>yottabyte</i>) equivale a 1.208.925.819.614.629.174.706.176 caracteres (1.024 Zbytes), sendo obtido a partir de 2^{80} .

2.2.7 - Sistemas de numeração e potências computacionais

A seguir é apresentada uma tabela para facilitar a conversão de valores no sistema de numeração computacional básico. São mostrados valores grafados nos sistemas decimal, binário e hexadecimal, correspondentes às 256 combinações possíveis de serem obtidas a partir de um conjunto de oito *bits* (*byte*).

Para converter um valor de um sistema numérico de 8 *bits* no outro valor também de 8 *bits*, basta localizar o valor na base desejada e olhar para o seu equivalente na mesma linha. A conversão de valores acima de 8 *bits* pela tabela seguinte pode ser facilmente conseguida para valores binários e hexadecimais. Por exemplo, o valor binário de 16 *bits* **0101 1010 1100 0010** equivale em hexadecimal a **5AC2** e para fazer a conversão pela Tabela 2.6, divida os 16 *bits* em dois *bytes* e busque na tabela os valores correspondentes de cada *byte* e agrupe novamente os dois valores.

Tabela 2.6 - Tabela de conversão de bases

Decimal	Binário	Hexadecimal	Decimal	Binário	Hexadecimal
000	0000 0000	00	026	0001 1010	1A
001	0000 0001	01	027	0001 1011	1B
002	0000 0010	02	028	0001 1100	1C
003	0000 0011	03	029	0001 1101	1D
004	0000 0100	04	030	0001 1110	1E
005	0000 0101	05	031	0001 1111	1F
006	0000 0110	06	032	0010 0000	20
007	0000 0111	07	033	0010 0001	21
008	0000 1000	08	034	0010 0010	22
009	0000 1001	09	035	0010 0011	23
010	0000 1010	0A	036	0010 0100	24
011	0000 1011	0B	037	0010 0101	25
012	0000 1100	0C	038	0010 0110	26
013	0000 1101	0D	039	0010 0111	27
014	0000 1110	0E	040	0010 1000	28
015	0000 1111	0F	041	0010 1001	29
016	0001 0000	10	042	0010 1010	2A
017	0001 0001	11	043	0010 1011	2B
018	0001 0010	12	044	0010 1100	2C
019	0001 0011	13	045	0010 1101	2D

Decimal	Binário	Hexadecimal
020	0001 0100	14
021	0001 0101	15
022	0001 0110	16
023	0001 0111	17
024	0001 1000	18
025	0001 1001	19
052	0011 0100	34
053	0011 0101	35
054	0011 0110	36
055	0011 0111	37
056	0011 1000	38
057	0011 1001	39
058	0011 1010	3A
059	0011 1011	3B
060	0011 1100	3C
061	0011 1101	3D
062	0011 1110	3E
063	0011 1111	3F
064	0100 0000	40
065	0100 0001	41
066	0100 0010	42
067	0100 0011	43
068	0100 0100	44
069	0100 0101	45
070	0100 0110	46
071	0100 0111	47
072	0100 1000	48
073	0100 1001	49
074	0100 1010	4A
075	0100 1011	4B
076	0100 1100	4C
077	0100 1101	4D
078	0100 1110	4E
079	0100 1111	4F
080	0101 0000	50
081	0101 0001	51
082	0101 0010	52

Decimal	Binário	Hexadecimal
046	0010 1110	2E
047	0010 1111	2F
048	0011 0000	30
049	0011 0001	31
050	0011 0010	32
051	0011 0011	33
105	0110 1001	69
106	0110 1010	6A
107	0110 1011	6B
108	0110 1100	6C
109	0110 1101	6D
110	0110 1110	6E
111	0110 1111	6F
112	0111 0000	70
113	0111 0001	71
114	0111 0010	72
115	0111 0011	73
116	0111 0100	74
117	0111 0101	75
118	0111 0110	76
119	0111 0111	77
120	0111 1000	78
121	0111 1001	79
122	0111 1010	7A
123	0111 1011	7B
124	0111 1100	7C
125	0111 1101	7D
126	0111 1110	7E
127	0111 1111	7F
128	1000 0000	80
129	1000 0001	81
130	1000 0010	82
131	1000 0011	83
132	1000 0100	84
133	1000 0101	85
134	1000 0110	86
135	1000 0111	87

Decimal	Binário	Hexadecimal
083	0101 0011	53
084	0101 0100	54
085	0101 0101	55
086	0101 0110	56
087	0101 0111	57
088	0101 1000	58
089	0101 1001	59
090	0101 1010	5A
091	0101 1011	5B
092	0101 1100	5C
093	0101 1101	5D
094	0101 1110	5E
095	0101 1111	5F
096	0110 0000	60
097	0110 0001	61
098	0110 0010	62
099	0110 0011	63
100	0110 0100	64
101	0110 0101	65
102	0110 0110	66
103	0110 0111	67
104	0110 1000	68
158	1001 1110	9E
159	1001 1111	9F
160	1010 0000	A0
161	1010 0001	A1
162	1010 0010	A2
163	1010 0011	A3
164	1010 0100	A4
165	1010 0101	A5
166	1010 0110	A6
167	1010 0111	A7
168	1010 1000	A8
169	1010 1001	A9
170	1010 1010	AA
171	1010 1011	AB
172	1010 1100	AC

Decimal	Binário	Hexadecimal
136	1000 1000	88
137	1000 1001	89
138	1000 1010	8A
139	1000 1011	8B
140	1000 1100	8C
141	1000 1101	8D
142	1000 1110	8E
143	1000 1111	8F
144	1001 0000	90
145	1001 0001	91
146	1001 0010	92
147	1001 0011	93
148	1001 0100	94
149	1001 0101	95
150	1001 0110	96
151	1001 0111	97
152	1001 1000	98
153	1001 1001	99
154	1001 1010	9A
155	1001 1011	9B
156	1001 1100	9C
157	1001 1101	9D
207	1100 1111	CF
208	1101 0000	D0
209	1101 0001	D1
210	1101 0010	D2
211	1101 0011	D3
212	1101 0100	D4
213	1101 0101	D5
214	1101 0110	D6
215	1101 0111	D7
216	1101 1000	D8
217	1101 1001	D9
218	1101 1010	DA
219	1101 1011	DB
220	1101 1100	DC
221	1101 1101	DD

Decimal	Binário	Hexadecimal	Decimal	Binário	Hexadecimal
173	1010 1101	AD	222	1101 1110	DE
174	1010 1110	AE	223	1101 1111	DF
175	1010 1111	AF	224	1110 0000	E0
176	1011 0000	B0	225	1110 0001	E1
177	1011 0001	B1	226	1110 0010	E2
178	1011 0010	B2	227	1110 0011	E3
179	1011 0011	B3	228	1110 0100	E4
180	1011 0100	B4	229	1110 0101	E5
181	1011 0101	B5	230	1110 0110	E6
182	1011 0110	B6	231	1110 0111	E7
183	1011 0111	B7	232	1110 1000	E8
184	1011 1000	B8	233	1110 1001	E9
185	1011 1001	B9	234	1110 1010	EA
186	1011 1010	BA	235	1110 1011	EB
187	1011 1011	BB	236	1110 1100	EC
188	1011 1100	BC	237	1110 1101	ED
189	1011 1101	BD	238	1110 1110	EE
190	1011 1110	BE	239	1110 1111	EF
191	1011 1111	BF	240	1111 0000	F0
192	1100 0000	C0	241	1111 0001	F1
193	1100 0001	C1	242	1111 0010	F2
194	1100 0010	C2	243	1111 0011	F3
195	1100 0011	C3	244	1111 0100	F4
196	1100 0100	C4	245	1111 0101	F5
197	1100 0101	C5	246	1111 0110	F6
198	1100 0110	C6	247	1111 0111	F7
199	1100 0111	C7	248	1111 1000	F8
200	1100 1000	C8	249	1111 1001	F9
201	1100 1001	C9	250	1111 1010	FA
202	1100 1010	CA	251	1111 1011	FB
203	1100 1011	CB	252	1111 1100	FC
204	1100 1100	CC	253	1111 1101	FD
205	1100 1101	CD	254	1111 1110	FE
206	1100 1110	CE	255	1111 1111	FF

A Tabela 2.7 apresenta os valores dos resultados das potências de dois de zero a dezesseis.

Tabela 2.7 - Tabela com os valores de potencia dois

Expoente	Valor	Expoente	Valor
0	1	17	131.072
1	2	18	262.144
2	4	19	524.288
3	8	20	1.048.576
4	16	21	2.097.152
5	32	22	4.194.304
6	64	23	8.388.608
7	128	24	16.777.216
8	256	25	33.554.432
9	512	26	67.108.864
10	1.024	27	134.217.728
11	2.048	28	268.435.456
12	4.096	29	536.870.912
13	8.192	30	1.073.741.824
14	16.384	31	2.147.483.648
15	32.768	32	4.294.967.296
16	65.536		

A Tabela 2.8 apresenta os valores dos resultados das potências de dezesseis de zero a oito.

Tabela 2.8 - Tabela com os valores de potencia dezesseis

Expoente	Valor	Expoente	Valor
0	1	5	1.048.576
1	16	6	16.777.216
2	256	7	268.435.456
3	4.096	8	4.294.967.296
4	65.536		

2.3 - Arquitetura 8086

O microprocessador 8086 foi apresentado pela empresa Intel no ano de 1978 (MORSE, 1987, p. 10), tendo sido desenvolvido pelo engenheiro e PhD Stephen P. Morse com a principal finalidade de ser usado em calculadoras desenvolvidas pela empresa Intel. Esse processador foi também utilizado para fazer o controle de automatização de semáforos de trânsito, como já ocorria com seus antecessores.

Ninguém imaginou, em 1978, que o microprocessador 8086 poderia vir a ser utilizado como “cérebro” de um microcomputador. Quando a IBM decidiu projetar e desenvolver seu microcomputador, optou por usar o microprocessador da Intel, mas percebeu que não seria possível, pois ele era demais avançado para sua máquina. Na época todos os periféricos fabricados operavam com barramento de dados (*bus*) de 8 *bits*. Assim sendo, o microprocessador 8086 não conseguia conversar com os periféricos existentes no mercado. O microprocessador 8086 operava internamente a 16 *bits* e se comunicava externamente a 16 *bits*. Era preciso que o microcomputador se comunicasse com os periféricos externos a 8 *bits*. Foi quando a Intel desenvolveu, em 1979, uma versão simplificada do 8086 chamado 8088. Essa versão simplificada operava internamente a 16 *bits*, comunicava-se externamente a 8 *bits*. Os microprocessadores 8086 e 8088 são idênticos no nível de processamento interno, tendo como diferença o barramento de dados para comunicação com periféricos externos.

A partir do lançamento do microprocessador 8088, foi possível à IBM lançar seu microcomputador IBM-PC (PC de *Personal Computer*), em 12 de agosto de 1981, e também o modelo IBM-XT (XT de *eXTended*), em 8 de março de 1983. O IBM-XT era idêntico ao IBM-PC, tendo como diferença principal a existência de uma unidade de disco rígido de 10 MB (*Megabyte*). Basicamente o microprocessador 8086 não chegou a ser utilizado em nenhum dos microcomputadores da IBM. O uso da tecnologia 8086, ou seja, um microprocessador de 16 *bits* com barramento 16 *bits* somente aconteceu quando a Intel redesenhou o microprocessador e o chamou de 80286, introduzindo-o no mercado em 1º de fevereiro de 1982. Isso possibilitou à IBM lançar, em agosto de 1984, o microcomputador IBM-AT (AT de *Advanced Technology*). Além do novo microprocessador, esse equipamento vinha com um disco rígido de 20 Mb.

A partir de então, a empresa Intel vem lançando microprocessadores pertencentes à família x86, destacando-se os já comentados, além dos microprocessadores 80186 (1982), 80188 (1982), 80286 (1982), 80386 (1985), 80486 (1990) e toda a série Pentium (a partir de 1993). No lançamento do 80286, os periféricos no mercado já se comunicam a 16 *bits* e atualmente a 32 *bits*.

Pelo fato de todos esses microprocessadores pertencerem à família x86, eles operam basicamente com o mesmo conjunto de instruções, e a cada lançamento novas instruções são inseridas, a menos nos casos de simplificação de operação de barramento de dados, sendo este um dos motivos em que se aprende a programação em linguagem de máquina a partir dos microprocessadores 8086.

O microprocessador 8086 e seu irmão 8088 são divididos em duas unidades, sendo BIU (*Bus Interface Unit* - unidade de interface de barramento de dados) e EU (*Execution Unit* - unidade de execução). Para entender melhor o que aconteceu no processo de simplificação do microprocessador 8086 para o microprocessador 8088, observe nas Figuras 2.3 e 2.4 a estrutura funcional simplificada da arquitetura desses microprocessadores.

Ao observar as Figuras 2.3 e 2.4, é possível notar as diferenças existentes nos diagramas entre os números de instruções de fila de 6, para o 8086, a 4, para o 8088, e o barramento de dados a partir do módulo somador de 16 para 8 *bits* no sentido de direcionar o fluxo de comunicação externa com os periféricos por meio do barramento de controle lógico.

A unidade BIU é responsável por efetivar todas as operações de transferências de endereços e de dados por meio do barramento de dados e dos registradores¹ de deslocamento **CS**, **DS**, **SS**, **ES** e **IP** para efetivar operações na memória, além de estabelecer comunicação com o barramento de controle lógico por meio dos barramentos de endereço e de dados no sentido de acessar o barramento externo e se comunicar com os periféricos agregados ao equipamento.

A unidade EU tem por base se comunicar por meio do barramento de dados da ALU (*Arithmetic Logic Unit* - unidade lógica e aritmética) com a unidade BIU e informar o local onde os dados ou instruções serão executados e em quais endereços. Essa unidade opera sobre os registradores gerais e registradores de estados (*flags*). Todas as operações dessa unidade são controladas pela ALU e pela EU Control System (*Execution Unit Control System* - unidade de controle de execução do sistema).

Enquanto a unidade EU executa alguma instrução que não exige o uso do barramento de dados da ALU, a unidade BIU apanha até seis *bytes* de instrução da próxima instrução e armazena-os no barramento de fila. Assim que a EU estiver pronta para executar a próxima instrução, ela coleta os *bytes* no barramento de fila. Esse ciclo de execução busca a próxima instrução enquanto outra instrução é executada pelo ciclo de controle denominado *pipelining* (canalização).

O funcionamento de um microprocessador 8086/8088 é baseado a partir do seguinte procedimento de trabalho: a unidade *BIU* coloca a instrução apontada no registrador **IP** somado ao registrador **CS** no barramento de fila para que esta instrução seja operacionalizada pela unidade EU. Em seguida o registrador **IP** é incrementado e passa a apontar o endereço de memória da próxima instrução, que após ser lida é executada. A unidade EU pega sempre a primeira instrução da fila e faz sua execução. Enquanto a unidade EU executa uma instrução a unidade BIU efetua nova busca de instrução e preenche a fila. Caso a instrução a ser executada pela unidade EU seja demorada, a unidade BIU fará o preenchimento de toda a fila.

As instruções da fila não são usadas quando estiver em execução instruções de acesso a memória ou estiver em uso instruções de desvios, que provocam o descarte e a sobreposição do conteúdo da fila.

¹ Registradores são células especiais de memória usadas para o armazenamento temporário de dados que tenham no máximo o tamanho de uma word.

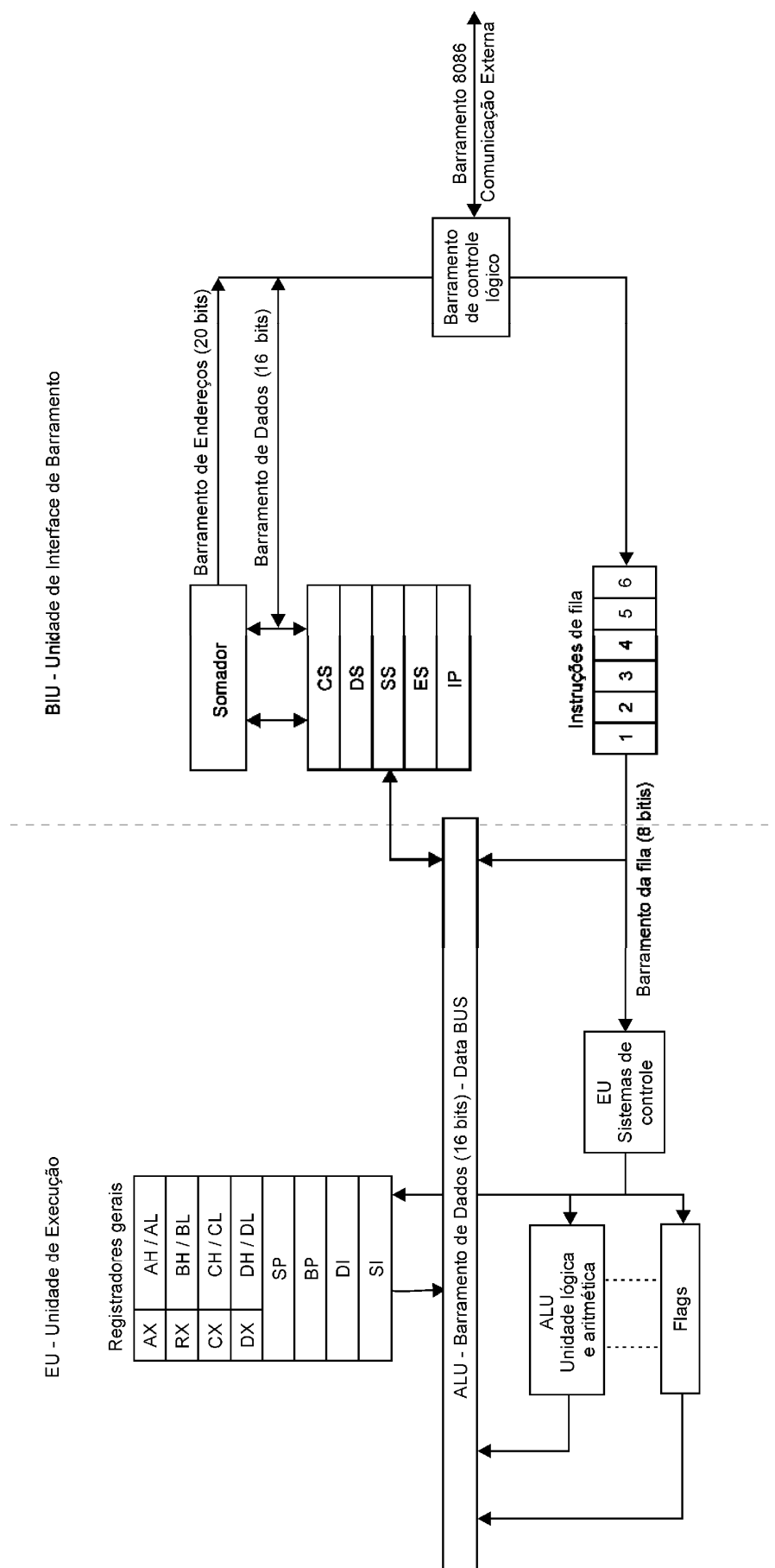


Figura 2.3 - Diagrama de bloco funcional do microprocessador 8086. Adaptado de INTEL, 1985, p. 1-4.

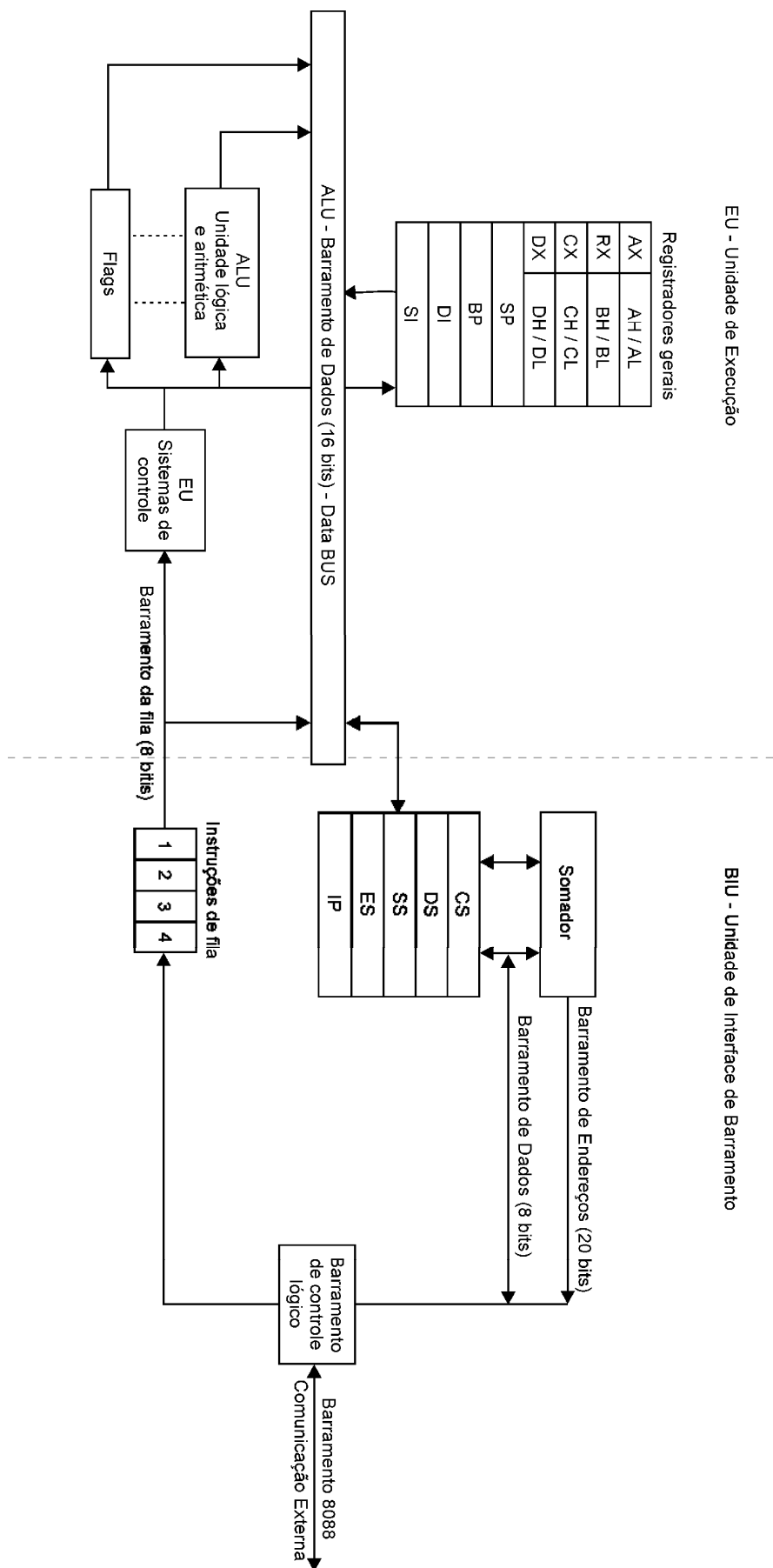


Figura 2.4 - Diagrama de bloco funcional do microprocessador 8088. Adaptado de INTEL, 1985, p. 1-5.

Como pode ser percebido, os microprocessadores 8086 e 8088 utilizam registradores para efetivar o armazenamento temporário de dados em memória, os quais serão manipulados por intermédio de um programa. Grosso modo, os registradores são semelhantes às variáveis encontradas nas linguagens de programação de alto nível para tratar a maior parte dos dados em memória.

O registrador está intimamente relacionado com a estrutura do microprocessador em uso. Para cada tipo de microprocessador existe uma forma peculiar de tratar este conceito. Será apresentada e considerada a estrutura de registradores para computadores digitais baseada na arquitetura do microprocessador 8086 da empresa Intel (podendo considerar também os microprocessadores fornecidos por outros fabricantes, como é o caso do AMD), conforme a Figura 2.6, a qual mostra de forma esquemática a estrutura interna da arquitetura de um microprocessador padrão 8086.

Perceba que a estrutura é formada por 14 registradores (**AX, BX, CX, DX, CS, DS, ES, SS, SI, DI, SP, BP, IP, F**), cada um com 16 *bits* (numerados de 0 a 15 da direita para a esquerda) divididos em quatro grupos funcionais denominados: registradores gerais (**AX, BX, CX, DX**), registradores de segmento (**CS, DS, ES, SS**), registradores de ponteiros (**SI, DI, SP, BP, IP**) e registradores de estado (**F**). A Figura 2.5 mostra um resumo dos registradores encontrados na arquitetura dos microprocessadores 8086/8088.

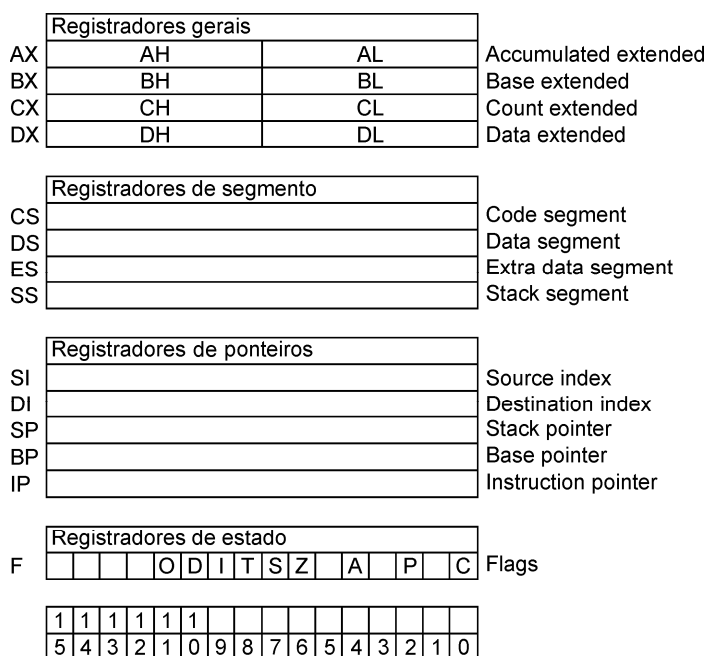


Figura 2.5 - Esquema da arquitetura interna do microprocessador 8086/8088.

Observação

Não se preocupe em entender plenamente tudo o que está exposto neste momento. Os necessários conceitos serão absorvidos durante a aplicação prática de exercícios de aprendizagem ao longo do livro.

2.3.1 - Registradores gerais

Os registradores gerais (*general registers*) **AX** (**A**ccumulator **eX**tended), **BX** (**B**ase **eX**tended), **CX** (**C**ounter **eX**tended) e **DX** (**D**ata **eX**tended) possuem 16 *bits* de dados, e cada um desses registradores pode ser dividido em duas partes, cada uma com 8 *bits*: parte mais significativa (do *bit* 8 até o *bit* 16), lado esquerdo e parte menos significativa (do *bit* 0 até o *bit* 7), lado direito. Desta forma, obtém-se oito registradores de 8 *bits* cada um, como indicado na Figura 2.6. Os registradores gerais fazem parte do grupo de dados (*data group*) e são utilizados com a finalidade de armazenar dados a serem processados.

Quando divididos, cada registrador geral é tratado com 8 *bits* mais significativos (alto) e 8 *bits* menos significativos (baixo). Desta forma, o registrador de 8 *bits* **AH** (**A**ccumulator **H**igh) e o registrador de 8 *bits* **AL** (**A**ccumulator **L**ow) são, respectivamente, divisões do registrador **AX** de 16 *bits*. Para os demais registradores gerais segue a mesma estrutura,

BH-BL, **CH-CL** e **DH-DL** no sentido de determinar registradores de 8 *bits* mais e menos significativos pertencentes, respectivamente, a cada um de seus registradores de 16 *bits* **BX**, **CX** e **DX**. A Figura 2.6 mostra como esta divisão deve ser considerada.

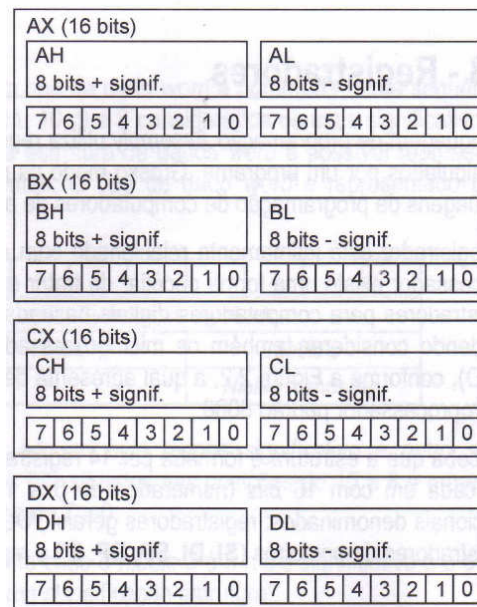


Figura 2.6 - Esquema da divisão interna dos registradores gerais.

O registrador geral **AX** (**AH-AL**) pode ser utilizado em operações aritméticas e lógicas, acessos de portas de entrada e saída, chamadas de interrupções, transferência de dados, entre outras possibilidades. A definição de seu valor pode ser feita sobre todo o registrador **AX** ou sobre os 4 *bits* à direita, ou seja, da sua parte menos significativa, sendo **AL**. Este é o principal registrador usado na manipulação de dados.

O registrador geral **BX** (**BH-BL**) pode ser utilizado como ponteiro para acessar a memória (apontador de endereço) no sentido de obter algum valor de retorno ou mesmo definir valores que serão usados para auxiliar operações aritméticas efetuadas com o registrador **AX**, além de servir de base para instruções que operem com vetores de dados.

O registrador geral **CX** (**CH-CL**) também é usado para receber alguns valores de interrupções, mas sua principal finalidade é servir como contador de laços de repetição e operações de deslocamento (*shift*) registradas normalmente em **CL**. Este registrador é usado em operações de ações repetitivas e iterativas.

O registrador geral **DX** (**DH-DL**) é usado em operações aritméticas (mais precisamente em operações de multiplicação para armazenamento da parte de um produto de 32 *bits* e também em operações de divisão para o armazenamento do resto da divisão), acessos de portas de entrada, acessos a portas de saída, acesso a portas lógicas e em chamadas de interrupções e apontamento de endereços de memória para deslocamentos.

2.3.2 - Registradores de segmento

Os registradores de segmento (*segment registers*) **CS** (**C**ode **S**egment - segmento de código), **DS** (**D**ata **S**egment - segmento de dados), **ES** (**E**xtra **S**egment - segmento extra) e **SS** (**S**tack **S**egment - segmento de pilha) têm 16 bits e são utilizados para acessar uma determinada área de memória, ou seja, para auxiliar o microprocessador a encontrar o caminho pela memória do computador. Eles não podem ser divididos em registradores de 8 bits.

O registrador de segmento **CS** aponta para uma área de memória que contém o segmento de código de programa que se encontra em execução. A mudança do valor existente nesse registrador de segmento pode resultar em travamento do computador. Quando esse segmento tem seu valor somado ao valor do segmento de deslocamento **IP** (comentado no próximo tópico), gera-se o endereço de 20 *bits* da instrução. O registrador **CS** contém as instruções que são executadas em um programa. Este registrador armazena o endereço inicial do segmento de código.

O registrador de segmento **DS** aponta para a área de memória que geralmente é utilizada para o armazenamento dos dados do programa em execução. A mudança do valor existente nesse registrador de segmento pode ocasionar a obtenção de dados incorretos. Quando utilizado em conjunto com o registrador de deslocamento **IP** (comentado no próximo tópico), permite definir o endereço efetivo de memória onde um dado será lido ou escrito por meio do controle de execução de instruções. Este registrador armazena o endereço inicial do segmento de dados.

O registrador de segmento **ES** é utilizado para determinar um segmento extra de endereço de dados (um novo segmento, ou seja, um segmento *far pointer*) distante da área em que se está operando. Normalmente necessário para acessar, por exemplo, a memória de vídeo. Esse registrador aponta para um segmento de dados do tipo *string* (texto).

Observação

Os registradores de segmento DS e ES são usados em conjunto com os registradores de deslocamento SI e DI, que serão comentados no tópico seguinte.

O registrador de segmento **SS** é utilizado para identificar a área de memória que será usada como **pilha** (*stack*). Desta forma, aponta para o segmento da pilha em uso no sentido de armazenar dados temporários para a execução de um determinado programa. Esse registrador de segmento pode algumas vezes conter o mesmo valor encontrado no registrador de segmento **DS**. A mudança do valor existente nesse registrador de segmento pode trazer resultados imprevisíveis, normalmente relacionados aos dados.

O segmento de pilha caracteriza-se por ser uma área a qual contém dados e endereços de retorno de procedimentos ou sub-rotinas. A pilha é implementada como uma estrutura de dados (**stack**). O registrador de segmento de pilha **SS** armazena o endereço inicial da pilha.

2.3.3 - Registradores de deslocamento

Os registradores de deslocamento (apontamento), também denominados registradores de índice, estão associados ao acesso de uma determinada posição de memória previamente conhecida, com a utilização dos *registradores de segmento*. Os registradores de deslocamento pertencem ao grupo de ponteiros e índices (*pointer and index group*).

Os registradores de ponteiros são cinco de 16 *bits*. Diferentemente dos registradores gerais, eles não podem ser divididos em registradores de 8 *bits*.

Dos cinco registradores de apontamento, quatro são manipuláveis, sendo **SI** (**Source Index** - índice de origem), **DI** (**Destination Index** - índice de destino), **SP** (**Stack Pointer** - ponteiro de pilha) e **BP** (**Base Pointer** - ponteiro de base). O registrador de apontamento (deslocamento) **IP** (**Instruction Pointer** - ponteiro de instrução), conhecido como *apontador da próxima instrução*, possui o valor de deslocamento (*offset*) do código da próxima instrução a ser executada.

O registrador de apontamento **IP** é de uso interno do microprocessador e tem por finalidade guardar o deslocamento (*offset*) da próxima instrução de um programa a ser executada, ou seja, esse registrador possui o valor da distância em *bytes* da próxima instrução a ser executada em relação ao início da instrução que se encontra em execução. O valor existente nesse registrador só pode ser lido. O registrador **IP** associado ao registrador **CS** fornece o endereço completo da instrução atual no segmento de código.

Os registradores de apontamentos **SI** (endereço fonte de operações com cadeias de caracteres) e **DI** (endereço destino de operações com cadeias de caracteres) são utilizados para manipular índices de uma tabela, sendo o registrador de apontamento **SI** usado para a leitura de dados do tipo *string* de uma tabela e o registrador de apontamento **DI** usado para a escrita de dados do tipo *string* em uma tabela. Em especial o registrador de apontamento **DI** também é utilizado na definição de endereçamento distante associado ao registrador de segmento **ES**.

Os registradores de apontamento **SP** (fornece o valor de deslocamento dentro da pilha do programa) e **BP** (usado na referência de parâmetro passado para uma sub-rotina) permitem o acesso à **pilha** de programa (memória para armazenamento de dados). A pilha possibilita armazenar dados em memória, sem utilizar registradores gerais. Uma pilha é um mecanismo que armazena dados de cima para baixo. Imagine uma pilha como uma matriz de uma dimensão. O registrador de apontamento **BP** armazena o endereço da base da pilha, enquanto o registrador de apontamento **SP** armazena o endereço do topo da pilha. O registrador **SP** associado ao registrador **SS** é usado na referência da posição atual dos dados ou endereço dentro da pilha do programa. Já o registrador **BP** combinado com o registrador **SS** fornece a

localização do parâmetro de uma sub-rotina, podendo ainda ser associado aos registradores **DI** e **SI** como registro de base para a definição de um endereçamento especial.

2.3.4 - Registradores de estado (status flags)

Os registradores de estado são considerados os mais importantes meios para a sinalização da efetivação de operações lógicas, aritméticas, manipulação de blocos e interrupções, pois indicam o estado de comportamento do microprocessador quando da execução de alguma instrução da linguagem de programação *Assembly*, pois são responsáveis pela definição das condições a serem utilizadas pelo microprocessador. Os registradores de estado são considerados registradores de controle. A Figura 2.7 apresenta o esquema do registrador de estado (*flag*) para o microprocessador 8086.

O registrador de estado **F** (*flags*) tem 16 *bits* que agrupam um conjunto de *flags* de um 1 *bit* (sendo um por *bit*), e cada *flag* define ou sinaliza um estado de comportamento particular do computador.

Se o valor de cada *bit* estiver sinalizado como 1, indica que o *flag* em questão está setado (acionado), caso esteja sinalizado com o valor 0, significa que o *flag* não está setado (desabilitado).

O *flag* está vinculado ao fato de uma ação ser ou não executada. Se uma determinada ação é executada, o *flag* (a bandeira) é levantado, indicando que a ação ocorreu (valor 1). Quando a bandeira está abaixada, é sinal de que a ação em questão não sofreu nenhum tipo de alteração (valor 0). Os *flags* do registrador de estado são independentes, mas por questões de conveniência são agrupados no mesmo conjunto de registrador de estado.

Registradores de estado (Flags)															
				O	D	I	T	S	Z		A		P		C
1	1	1	1	1	1										
5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0

C = Carry
P = Parity
A = Auxiliary carry
Z = Zero
S = Sign
T = Trap
I = Interrupt enable
D = Direction
O = Overflow

Figura 2.7 - Esquema do registrador de estado para o microprocessador 8086.

Os *flags* **CF**, **PF**, **AF**, **ZF** e **SF** são de estado; os *flags* **TF**, **IF** e **OF** são de sistema; o *flag* **DF** é de controle.

A Figura 2.7 apresenta o registrador de estado com o conjunto de nove *flags* associados a cada operação executada. Dos nove *flags*, seis são utilizados para indicar os resultados obtidos em operações aritméticas ou lógicas, são estes os registradores de estado (*status flags*) **CF** (*Carry Flag*), **PF** (*Parity Flag*), **AF** (*Auxiliary Flag*), **ZF** (*Zero Flag*), **SF** (*Sign Flag*) e **OF** (*Overflow Flag*). E os outros três *flags* são utilizados para as tarefas de controle (*control flags*): **TF** (*Trap Flag*), **IF** (*Interrupt Flag*) e **DF** (*Direction Flag*). Perceba que os *bits* 1, 3, 5, 12, 13, 14 e 15 do registrador estão indicados em branco (na figura). Internamente não existe como prever seus estados lógicos, se eles estarão com valores "0" ou "1", pois estão reservados propositalmente pela Intel.

O *flag* **CF** (*bit* 0) indica a sinalização da operação de *carry* (operação de vai um) para a próxima posição, após a efetivação de uma operação aritmética de adição ou operação de *carry* (operação de empresta 1) para a próxima posição, após a efetivação de uma operação de subtração que ultrapassar a capacidade de armazenamento daquele valor. Neste caso, será este sinalizado com o valor 1; se não ocorrer a movimentação de vai um, o valor sinalizado será 0. Por exemplo, ao se efetuar a soma dos valores 255 e 1, obter-se-á o valor 256. Considerando-se apenas a capacidade de armazenamento de um valor entre 0 e 255, este resultado estará fora da faixa, o que ocasiona a mudança do *flag* **CF** para 1.

O *flag* **PF** (*bit* 2) indica se a paridade do *byte* inferior do resultado é par ou ímpar após a execução de uma operação aritmética ou lógica resultar ou não um par de *bits* 1s (uns). Se o valor sinalizado for 1, será então considerada paridade par; se o valor sinalizado for 0, será considerada paridade ímpar.

O **flag AF** (*bit 4*) indica a sinalização de *carry* (operação de vai-um) do *bit 3* para o *bit 4* de uma adição ou com a sinalização de *carry* (operação de empresta-um) do *bit 4* para o *bit 3* de uma subtração com valores decimais, representados por sequências numéricas escritas em formato BCD (*Binary Coded Decimal*). Se o valor sinalizado for **1**, indica que ocorreu a operação de vai um; se o valor sinalizado for **0**, indica que ocorreu a operação de empresta um.

O **flag ZF** (*bit 6*) indica se o resultado após uma operação aritmética ou lógica. Em uma operação aritmética será igual a zero quando sinalizado pelo valor **1** ou diferente de zero quando sinalizado pelo valor **0**. Pode ser usada para a obtenção de resultado de comparação (ação lógica), sendo sinalizado pelo valor **1** quando os valores comparados forem iguais e **0** quando forem diferentes.

O **flag SF** (*bit 7*) mostra a obtenção de um resultado negativo ou positivo após uma operação aritmética de complemento 2, não ocorrendo estouro - *overflow*. Se o resultado for positivo, será sinalizado como valor **0**; caso seja negativo, será sinalizado com valor **1**.

O **flag TF** (*bit 8*) possibilita uma interrupção especial após executar uma única instrução, com a finalidade de acompanhar passo a passo a execução individual das instruções de um determinado programa em operações de *debug*. Quando sinalizado com valor **1**, indica a execução da ação passo a passo da própria instrução em foco e após essa ação ocorre uma interrupção e seu valor tornar-se-á **0**. Se o valor estiver em **0**, ocorre o contrário.

O **flag IF** (*bit 9*) habilita ou desabilita o reconhecimento à chamada de interrupções por meio da instrução **INT**. Se estiver uma ocorrência de interrupção habilitada, ele será sinalizado com o valor **1**; caso contrário, será sinalizado o valor **0**.

O **flag DF** (*bit 10*) aponta a direção das operações de manipulação de blocos da direita para a esquerda ou vice-versa em sequências de caracteres (*strings*). Quando estiver sinalizado com **0**, indica que os registradores de índice serão incrementados; caso esteja sinalizado com valor **1**, os registradores de índice são decrementados.

O **flag OF** (*bit 11*) indica a obtenção de um valor muito grande após uma operação aritmética ou lógica, estouro de capacidade quando um número positivo for muito grande ou um número negativo for muito pequeno para ser processado, ou seja, fora da faixa de valores permitida para a operação do microprocessador. Quando o estouro ocorrer, ele será sinalizado com valor **1**; permanecendo sinalizado o valor **0**, não ocorreu estouro na operação.

2.4 - Interrupções

A interrupção está associada à ação executada por um computador na realização de tarefas e a capacidade de interrompê-las a qualquer momento, para então realizar outras tarefas solicitadas e retornar àquilo que estava fazendo.

O processo de interrupção é de fundamental importância para o funcionamento adequado de um computador. É por meio das interrupções que se torna possível controlar vários dispositivos associados ao computador

O mecanismo de funcionamento ocorre pela comunicação entre o dispositivo externo (vídeo, teclado, mouse, unidades de disco, impressora, *scanner*, erros de execução de programa etc.) e o microprocessador.

Imagine o microprocessador trabalhando em uma determinada tarefa, quando é interrompido por um dispositivo externo que solicita autorização de ação. Nesse instante, o microprocessador para o que está fazendo e aceita a comunicação do dispositivo externo. Ao final da ação do dispositivo externo o microprocessador continua sua ação de trabalho a partir do ponto em que houve a interrupção. Esta característica torna os computadores máquinas eficientes e eficazes.

Graças à interrupção torna-se possível, por exemplo, utilizar um teclado para a entrada de dados quando da execução de um programa de cadastro. Para que uma interrupção seja acionada, usa-se a instrução **INT** seguida de um valor hexadecimal situado na faixa de valores entre **00h** e **FFh** para alterar o comportamento de execução do microcomputador, totalizando 256 interrupções possíveis. No entanto, nem toda interrupção pode ser usada, principalmente as interrupções de hardware.

No microprocessador 8086 os primeiros 1024 *bytes* da memória são reservados para a definição da tabela de interrupções (endereço físico de 0 a 1023). Essa tabela é formada por um conjunto de vetores que são indexados nos registradores **CS** e **IP**. Cada vetor de interrupção ocupa um total de 4 *bytes*, sendo 2 *bytes* usados no registrador **CS** e 2 *bytes* usados no registrador **IP**. A tabela de interrupções é conhecida como **ISR** (*Interrupt Service Routine*) e possui os endereços das funções de cada uma das interrupções existentes.

As interrupções que podem ser executadas em um computador padrão IBM-PC são divididas em três categorias, a saber:

- ◆ **Interrupção por hardware** - possibilita acionar periféricos externos, como unidades de disco, teclado, impressora, porta de comunicação etc. Esse tipo de interrupção conecta-se (comunica-se) ao microprocessador pelas linhas de comunicação de **IRQ** (*Interrupt ReQuest* - pedido de interrupção) que são controladas pelas rotinas de sistema operacional ou pelas rotinas da ROM-BIOS (BIOS é o nome do primeiro programa executado no computador após este ser acionado, significando *Basic Input Output System*, ou seja, sistema básico de entrada e saída, estando armazenado na memória ROM, daí chamá-lo de ROM-BIOS). Esse tipo de interrupção está na faixa hexadecimal de valores de **08h** a **0Fh**. É possível usar até oito dispositivos externos². As interrupções de hardware são gerenciadas pelo próprio computador. Assim sendo, não é possível modificar seu estado de comportamento.
- ◆ **Interrupção por exceção** - possibilita fazer o controle condicional interno do microprocessador, retornando algum tipo de erro. Um exemplo típico é a tentativa de um programa dividir um determinado valor por zero. Esse tipo de interrupção está na faixa de **00h** a **07h**.
- ◆ **Interrupção por software** - efetua o acionamento de determinadas interrupções desejadas por intermédio de um programa em execução, podendo interferir na ação de algum periférico conectado ao microcomputador.

Como exemplos de interrupções por hardware para o microprocessador 8086 pode-se indicar as seguintes: 08h (associada ao IRQ0 opera o *chip* temporizador do sistema), 09h (associada ao IRQ1 opera o teclado), 0Ah (associada ao IRQ2 é uma interrupção reservada), 0Bh (associada ao IRQ3 opera as portas de comunicação COM2), 0Ch (associada ao IRQ4 opera as portas de comunicação COM1), 0Dh (associada ao IRQ5 opera uma unidade de disco rígido), 0Eh (associada ao IRQ6 opera unidade de discos flexíveis) e 0Fh (associada ao IRQ7 opera a porta LPT1 para impressora do tipo paralela).

Como exemplos de interrupções por exceção para o microprocessador 8086 pode-se indicar as seguintes: 00h (falha de divisão quando o divisor é igual a zero), 01h (execução passo a passo num processo de *debug* quando *flag* TF é setado), 02h (interrupção reservada para a condição de operação para NMI - *Non-Maskable-Interrupt*), 03h (ocorre na definição da execução de um ponto de parada - *breakpoint* sobre a execução da instrução **INT 3**) e 04h (ocorre quando há estouro registrado no *flag* OF quando da execução da instrução **INT O**).

Como exemplos de interrupções de software para o microprocessador 8086 mais utilizadas pode-se indicar as interrupções de ROM-BIOS: 10h (escrita de caractere no monitor de vídeo), 13h (operações de leitura e escrita em disco), 14h (comunicação com porta serial), 16h (leitura de caractere no teclado), 17h (operações com impressora), 19h (ação de reset), 1Ah (retorna a hora do sistema), 20h (finaliza a operação do sistema), 21h (operações de leitura e escrita nos periféricos padrão conectados ao sistema) e 33h (utilização do mouse).

Durante a apresentação dos exemplos de programas dos próximos capítulos serão automaticamente utilizadas algumas das interrupções de software mais comuns das 256 interrupções existentes, pois são as aceitas para uso pelo programa **emu8086**.

2.5 - Segmentos e deslocamentos

A linguagem de programação *Assembly x86* utiliza as características operacionais do microprocessador 8086 (ou 8088) para controlar as funções de trabalho de um computador. Entre as funções operacionais de trabalho está o uso da memória.

A memória de um microcomputador da família IBM-PC se assemelha a uma tabela de dados, cujas linhas são os **segmentos** e as colunas são os **deslocamentos** (*offset*) com a capacidade de endereçar uma memória de até 1 MB, dividida em 16 blocos (numerados de 0 a F em hexadecimal) com 64 KBytes (65.536 caracteres) cada bloco. Assim sendo, cada bloco de memória é denominado **segmento** de memória com 64KB cada segmento.

Os 16 segmentos de memória são endereçados de **00000h** (**0000 0000h**) até **FFFFFh** (**000F FFFFh**), sendo **00000h** o valor do primeiro segmento e **FFFFFh** o valor do último segmento, que equivale a 1 MB de espaço em memória. Um endereço no formato **0xxxxh** faz referência ao segmento **0h**, assim como o endereço **1xxxxh** faz referência ao seg-

² Levando-se em consideração o uso de um microprocessador Intel 8086.

mento **1h** e assim por diante até o segmento **Fxxxh**. A indicação **xxxx** após o valor do segmento é o complemento da formação do endereço físico (que é formado ao todo por cinco dígitos) do segmento de memória em uso.

Cada segmento é dividido em deslocamentos que variam de **0yyyyh** até **Fyyyyh**, e cada posição de deslocamento tem um total de 16 *bits* (manipulados pelos registradores de segmento). O segmento de memória é de fato dividido em duas partes, sendo a primeira parte formada por um segmento de 16 *bits* e uma segunda parte formada por um deslocamento de 4 *bits*. Isto posto, ocorre o fato de um endereçamento de memória ocupar a quantidade de 20 *bits* na memória, ou seja, a posição de memória a ser especificada é definida pelo valor do segmento e pelo seu deslocamento em relação ao início do segmento.

O deslocamento para a representação do endereço de memória a ser usado com os programas é representado pela notação **xxxxh:yyyyh** (*segmento:deslocamento*), sendo **xxxxh** o endereço de segmento e **yyyyh** o endereço da posição de deslocamento desejado, e está situado na parte convencional da memória principal, ou seja, entre a posição de memória 0 Kb até 640 Kb.

Perceba que o uso de segmentos e deslocamento são uma forma de se fazer acesso as posições de memória. Para exemplificar o exposto da forma mais simples possível de um ponto de vista apenas didático, tome a Figura 2.8 como exemplo, a qual representa, em semelhança, como ocorre o armazenamento de dados. Para tanto, considere como dados armazenados os seguintes textos:

APRENDER ASSEMBLY REQUER ATENÇÃO E DEDICAÇÃO.

EDITORA ÉRICA LTDA.

QUEM DESCOBRIU O BRASIL FOI PEDRO ÁLVARES CABRAL.

D. PEDRO I PROCLAMOU A INDEPENDÊNCIA.

- ◆ Observe que o armazenamento dos dados na memória ocorre de forma contígua, um dado após o outro. Cada dado em particular ocupa uma posição de **deslocamento** (coluna) em relação a uma posição de **segmento** (linha). Assim sendo, tem-se como nomenclatura de posicionamento de dados na tabela a indicação **segmento:deslocamento**.
- ◆ Perceba que os dados indicados **EDITORA ÉRICA LTDA.** estão na memória a partir da posição de segmento **2h** e deslocamento **Eh**, ou seja, endereço **2h:Eh** e se estende até a posição de segmento **4h** e deslocamento **0h**, ou seja, endereço **4h:0h** (*segmento:deslocamento*). Efetivamente para saber o endereço de posicionamento de um dado na memória, basta multiplicar o valor do segmento desejado por **16d** ou **10h** e somar o valor do deslocamento.
- ◆ Tomando por base os dados **EDITORA ÉRICA LTDA.**, o endereço de inicialização dos dados é **2h * 10h + Eh**, sendo assim a posição **2Eh** em que a sequência de dados se inicia.
- ◆ Posteriormente no capítulo nove este assunto será tratado com maiores detalhes. Neste momento está sendo apresentado a título de ilustração e de formação da base conceitual para o entendimento da linguagem de programação *Assembly* apenas um mero exemplo.

Mais detalhes sobre segmentos, deslocamentos e outras informações são estendidas e apresentadas no capítulo 10.

2.6 - Endereçamento de memória

A memória de um computador padrão IBM-PC tem como função primária a tarefa de armazenar dados e executar programas. Para que isso ocorra, essa memória é formada por um grande conjunto de células de armazenamento, e cada célula é formada por um conjunto de 8 *bits*, podendo assumir um dos 256 significados existentes. Cada célula que compõe a memória tem um endereço absoluto de sua posição. Cada posição é considerada de modo individual com um valor numérico que representa seu endereço de posicionamento.

Segmento	Deslocamento															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	A	P	R	E	N	D	E	R		A	S	S	E	M	B	L
1	Y		R	E	Q	U	E	R		A	T	E	N	Ç	Ã	O
2		E		D	E	D	I	C	A	Ç	Ã	O	.		E	D
3	I	T	O	R	A		É	R	I	C	A		L	T	D	A
4	.		Q	U	E	M		D	E	S	C	O	B	R	I	U
5		O		B	R	A	S	I	L		F	O	I		P	E
6	D	R	O		A	L	V	A	R	E	S		C	A	B	R
7	A	L	.		D	.		P	E	D	R	O		I		P
8	R	O	C	L	A	M	O	U		A		I	N	D	E	P
9	E	N	D	Ê	N	C	I	A	.							
A																
B																
C																
D																
E																
F																

Figura 2.8 - Exemplo simplificado da forma de composição da memória.

Esta obra se baseia no uso da estrutura de um microprocessador Intel 8086 ou 8088. O espaço máximo de memória para uso por programas e dados está na casa de 640 KBytes, sendo permitido o endereçamento máximo de 1 Mb. Do espaço total de 1 Mb de memória, 384 Kb são reservados para uso interno do sistema e não se pode usar esse espaço; sobram 640 KBytes que são divididos para guardar a tabela de interrupções que ocupa 1 Kb, os dados utilizados pelo programa BIOS que ocupa 1,5 Kb, sobrando um total de 637.5 Kb para serem usados pelo DOS (*Disk Operation System* - sistema operacional em disco) e também pelos outros programas que são utilizados, bem como o espaço para trabalhar com os dados desses programas.

O modelo de microprocessador 80286 conseguia endereçar no máximo 1 Gb de memória. A partir dos modelos de microprocessadores com 32 *bits* (acima do modelo 80386) já foi possível fazer uso máximo de 4 Gb de endereçamento de memória e para os microprocessadores de 64 *bits* é possível usar 128 Gb de endereçamento de memória. No entanto, em todos os modelos de microprocessadores padrão Intel existentes a estrutura de endereçamento de memória de 1 Mb do modelo 8086 ou 8088 é mantida por questões de compatibilidade e pelo fato de o mais moderno microprocessador da Intel ou da AMD ser originado do modelo 8086 e baseado nele, sendo este um bom começo para o estudo de programação da linguagem *Assembly*.

A Figura 2.9 representa graficamente a estrutura de endereçamento de memória de 1 MB típica usada pelos microprocessadores Intel 8086/8088 e também por seus sucessores.

A região de memória convencional vai de 0 KByte até 640 KBytes. Essa região é utilizada para armazenar a tabela de interrupções, os dados do programa BIOS e também pelo DOS, além de armazenar os dados de programas de uso geral, bem como os próprios programas. Assim sendo, os endereços de memória são:

- ◆ A partir da posição 0 até a posição 1.024 situa-se a faixa de memória reservada para o armazenamento da tabela de interrupções (pode ser controlado o acesso ao vídeo e teclado). Essa faixa equivale a 1 KByte de memória.
- ◆ A partir da posição 1.024 até a posição 1.536 situa-se a faixa de memória reservada para o armazenamento dos dados do programa BIOS. Essa faixa equivale a 0,5 KByte (512 bytes) de memória.
- ◆ A partir da posição 1.536 até a posição 655.360 situa-se a faixa de memória reservada para o armazenamento do sistema operacional (DOS) e também de qualquer programa a ser utilizado. Essa faixa equivale a 638,5 KBytes de memória livre.

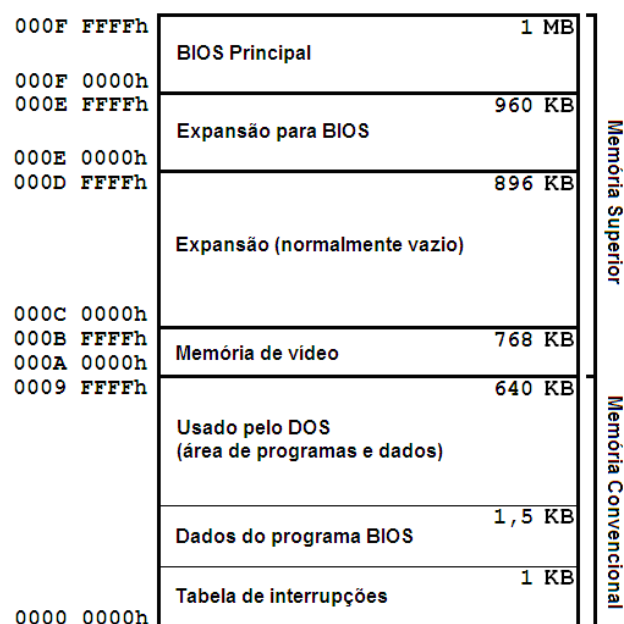


Figura 2.9 - Endereçamento de memória.

A área de memória livre para trabalho é menor que a marca de 640 KBytes, pois 1,5 KByte dos 640 KBytes de memória é reservado para a tabela de interrupções e para os dados do BIOS. Além da perda desse espaço, ocorre ainda a utilização de parte dele para arquivos de controle do sistema operacional como os arquivos IO.SYS, MSDOS.SYS e COMMAND.COM.

Pelo fato de os programas BIOS do sistema e do vídeo estarem gravados em memória ROM, é necessário ter suas informações temporárias armazenadas em uma área de memória RAM. Desta forma, é possível proceder com algum tipo de alteração nessas informações.

Acima da marca dos 640 KBytes encontra-se a área de memória de vídeo que ocupa um total de 128 Kb da parte de memória superior. Essa área é utilizada exclusivamente pelas placas de vídeo para que os programas executados possam "enxergar" a capacidade de memória que a placa de vídeo conectada no computador possui.

Entre 768 Kb e 896 Kb encontra-se a área de expansão com um total de 128 Kb. Normalmente essa parte da memória encontra-se vazia e quando utilizada, é por meio de programas de gerenciamento de memória no sentido de deixar mais espaço para a memória convencional.

No espaço de 64 Kb compreendido na faixa de 896 Kb até 960 kb encontra-se a área de expansão para o BIOS que complementa o espaço da área de dados do programa BIOS na memória convencional.

Por último, os 64 Kb acima de 960 Kb são usados pelo sistema para armazenar o programa BIOS propriamente dito. O programa BIOS encontra-se gravado na memória ROM.

A parte de memória convencional é usada para a execução do DOS e dos programas, gerenciamento dos dados utilizados pelos programas executados, armazenamento dos dados da BIOS e da tabela de interrupções. Já a área de memória superior é utilizada pelo próprio computador e não é diretamente acessível.

2.7 - A linguagem Assembly 8086

A denominação da linguagem de programação de computadores *Assembly* para computadores IBM-PC é feita normalmente com algumas siglas de identificação, tais como *ASM86*, *x86*, *ASM8086*, entre outras. Esta denominação refere-se ao uso da linguagem de programação de computadores *Assembly* para o microprocessador Intel ou equivalentes de mercado, como os microprocessadores fornecidos pela empresa AMD.

Os demais lançamentos de microprocessadores da família Intel, tais como 80186, 80286, 80386, 80486 e também a série Pentium, possuem basicamente o mesmo conjunto de instruções existentes no padrão de microprocessador 8086

e 8088, além de cada um possuir em sua evolução novas instruções e recursos. Os recursos desses outros microprocessadores não serão abordados nesta obra. Este trabalho limita-se somente às instruções básicas e características da linguagem *Assembly* para o microprocessador 8086/8088. A linguagem de programação *Assembly 8086* (ou *ASM86*, *x86*) possui 124 comandos (mnemônicos) diferentes destinadas ao controle do microprocessador. Os demais processadores da família Intel possuem a cada série um número maior de novas instruções. Para quem está começando a estudar a linguagem *Assembly*, é aconselhável primeiro conhecer basicamente as instruções para o microprocessador modelo 8086 ou modelo 8088. O *Assembly* do modelo 8086 é o mesmo do modelo 8088, tanto que se faz normalmente referência aos modelos com a nomenclatura 8086/8088, doravante utilizada nesta obra.

Os comandos da linguagem *Assembly* estão divididas em seis grupos funcionais, a saber:

- ◆ **Transferência de dados** - instruções destinadas à movimentação de dados. Os dados podem ser movimentados entre registradores, entre registradores e posições de memória e entre registradores e unidades de entrada e de saída.
- ◆ **Aritméticas** - instruções destinadas aos cálculos matemáticos básicos, como adição, subtração, multiplicação e divisão.
- ◆ **Manipulação de bits** - instruções que fazem o deslocamento de *bits* em um registrador ou posição de memória. As funções de operações lógicas de conjunção, disjunção e negação são incluídas nesse grupo.
- ◆ **Manipulação de strings** - elas fazem o controle (comparação, análise e movimentação) de grupos de sequências de caracteres.
- ◆ **Controle de programa** - instruções que controlam a execução do código de programa. O controle pode ser uma execução sequencial, com laços (*loopings*) e com sub-rotinas ou subprogramas. As instruções podem manipular as interrupções de um programa em execução. A ação de interromper um programa durante sua execução pode ocorrer por vários motivos, como, por exemplo, aceitar uma entrada de dados via teclado. Para que o dado possa ser digitado, é necessário fazer uma interrupção no programa que após aceitar o dado continua sua execução.
- ◆ **Controle do microprocessador** - instruções que possibilitam o acesso dos registradores de controle do microprocessador, com o objetivo de mudar seu estado de comportamento.

A linguagem *Assembly* como qualquer outra linguagem de programação manipula na memória dados. Os tipos de dados reconhecidos pela linguagem *Assembly 8086* são: constantes, variáveis e rótulos. Segundo Weber (2004, p. 240-241) os tipos de dados da linguagem *Assembly* operam da seguinte forma:

- ◆ **Constantes** – refere-se a identificação de um rótulo que será associado a determinado valor numérico sem o uso de algum atributo;
- ◆ **Variáveis** – refere-se a objetos manipuláveis usados como operandos para as instruções de manipulação de dados;
- ◆ **Rótulos** – refere-se a definição de nomes usados na elaboração de sub-rotinas e definição de pontos de salto a serem executados por instruções de saltos condicionais e incondicionais.

Uma variável ou um rótulo em *Assembly* podem ser associados a um segmento de memória, a um deslocamento dentro de certo segmento da memória, a definição de um tipo de dado (*byte*, *word*, entre outros) associado a uma variável ou a definição de acesso *near* ou *far* quando se usa definição de sub-rotinas.

2.8 - Os modos 32 e 64 bits

Apesar de não ser o foco central deste trabalho a linguagem *Assembly* para processadores que operam em 32 ou 64 *bits*, é oportuno comentar rapidamente as diferenças encontradas nesses microprocessadores no que tange à sua estrutura interna de registradores em relação ao modelo de 16 *bits*. No modo de operação 32 *bits*, os registradores utilizados em memória possuem o dobro da capacidade dos registradores de 16 *bits*, consequentemente os registradores de 64 *bits* possuem como capacidade o quádruplo dessa potência.

Na estrutura 32 *bits*, a maioria dos registradores é acrescida do identificador de prefixo **E** (de *extended*) para sua representação, destacando-se **EAX** [**AX** (**AH-AL**)], **EBX** [**BX** (**BH-BL**)], **ECX** [**CX** (**CH-CL**)], **EDX** [**DX** (**DH-DL**)], **ESI** [**SI**], **EDI** [**DI**], **EBP** [**BP**], **ESP** [**SP**], **EIP** [**IP**] e registradores de estado **EF** [**F**]. Os registradores de segmento **CS**, **DS**, **ES** e **SS** permanecem com a estrutura de 16 *bits*, acrescentando-se a eles **FS** e **GS** como registradores de segmentos extras. Nos registradores de estado **EF** de 32 *bits*, mantêm-se os mesmos registradores existentes no modo de 16 *bits*, mas são acrescentados os registradores de estado **IOPL** (décimo segundo *bit*), **NT** (décimo terceiro *bit*), **R** (décimo sexto *bit*), **VM**

(décimo sétimo *bit*), **AC** (décimo oitavo *bit*), **VI** (décimo nono *bit*), **VIP** (vigésimo *bit*) e **ID** (vigésimo primeiro *bit*). Os *bits* 1, 3, 5, 14, 15 e de 22 até 31 não são usados.

Na estrutura de 64 *bits*, a maioria dos registradores é acrescida do identificador de prefixo **R** (de *rex*) para sua representação, destacando-se os registradores **RAX** {**EAX** [**AX** (**AH-AL**)]} como **R0**, **RBX** {**EBX** [**BX** (**BH-BL**)]} como **R1**, **RCX** {**ECX** [**CX** (**CH-CL**)]} como **R2**, **RDX** {**EDX** [**DX** (**DH-DL**)]} como **R3**, **RBP** {**EBP** [**BP**]} como **R4**, **RSI** {**ESI** [**SI**]} como **R5**, **RDI** {**EDI** [**DI**]} como **R6**, **RSP** {**ESP** [**SP**]} como **R7**, além dos registradores reservados **R8**, **R9**, **R10**, **R11**, **R12**, **R13**, **R14** e **R15**, o registrador de ponteiro **RIP** {**EIP** [**IP**]} e registradores de estado **RF** {**EF** [**F**]}. Os registradores de segmento **CS**, **DS**, **ES**, **SS**, **FS** e **GS** permanecem com a estrutura de 16 *bits*. Nos registradores de estado **RF** de 64 *bits*, mantêm-se os mesmos registradores encontrados no modo de 32 *bits*, acrescentando-se a sequência dos *bits* de 32 até 63 que estão reservados.

A diferença encontrada nos processadores de 16, 32 e 64 *bits* reside na quantidade de dados e também de instruções que o processador pode executar por vez. Um processador de 16 *bits* consegue manipular um valor numérico inteiro de até 65.536 (2^{16}) numa única operação. Caso necessite trabalhar com um valor maior, será preciso alocar mais uma porção de 16 *bits* da memória. Um processador de 32 *bits* consegue de uma única vez manipular um valor inteiro de até 4.294.967.296 (2^{32}). Em relação a um processador de 64 *bits*, esse valor.

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69
0D82:0109 64	DB	64
0D82:010A 61	DB	61
0D82:010B 64	DB	64
0D82:010C 65	DB	65
0D82:010D 206573	AND	[DI+73],AH
0D82:0110 7065	JO	0177
0D82:0112 63	DB	63
0D82:0113 69	DB	69
0D82:0114 66	DB	66
0D82:0115 69	DB	69
0D82:0116 63	DB	63
0D82:0117 61	DB	61

Parte



II

Programação com Enhanced DEBUG

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 3

FERRAMENTAS DE DESENVOLVIMENTO

*Este capítulo descreve a obtenção, instalação e configuração do ambiente de trabalho para a produção de programas escritos em linguagem Assembly. São apresentadas as ferramentas operacionais **DEBUGs** e **emu8086** que serão usadas ao longo do estudo desta obra.*

3.1 - Como começou, como está, como fica

Este trabalho surgiu com o objetivo de ser um material introdutório para a apresentação inicial da linguagem de programação *Assembly* voltada aos microprocessadores padrão 8086/8088 da empresa Intel e demais microprocessadores compatíveis com são os da empresa AMD. O modelo 8086/8088 do microprocessador Intel foi usado no desenvolvimento do microcomputador pessoal da empresa IBM lançado no ano de 1981 e conhecido como IBM-PC.

Em 1980, o programador Tim Paterson trabalhando para a empresa **SCP** (*Seattle Computer Products*) desenvolveu o sistema operacional **QDOS** (*Quick and Dirty Operating System*) de 16 *bits*, posteriormente chamado de **86-DOS** para uma placa de circuitos integrados, chamada **8086 S-100**, baseada no microprocessador 8086 da empresa Intel, a qual o tinha projetado um ano antes.

Na fase de desenvolvimento do sistema operacional QDOS Tim Paterson escreveu um pequeno programa de depuração que veio a facilitar o desenvolvimento de rotinas internas desse sistema e de um dos chips de **ROM** (*Read Only Memory*) usado na placa 8086 S-100. Um tempo depois Tim Paterson adaptou o código de seu programa depurador para ser executado dentro de seu sistema operacional com extensão **.COM** chamando-o de **DEBUG** (lê-se *debâgui*), adicionando a este programa a capacidade de desmontar o código de máquina do microprocessador 8086.

No decorrer dessas ações a empresa Microsoft comprou os direitos do sistema operacional 86-DOS e o adaptou para uso na linha de computadores pessoais da família IBM. Nesse período a Microsoft tinha como seu principal cliente a empresa *IBM* que estava desenvolvendo secretamente o primeiro microcomputador de 16 *bits* para concorrer com os microcomputadores de 8 *bits* que estavam sendo comercializados no mercado.

A Microsoft contratou temporariamente o programador Tim Paterson como principal autor de “seu” primeiro produto: o sistema operacional **IBM-DOS 1.00**. Assim que o “novo” sistema operacional foi concluído Tim Paterson incluiu o utilitário **DEBUG.COM** neste “novo” sistema operacional.

Posteriormente a empresa Microsoft lança o sistema operacional **MS-DOS** para os computadores clone *IBM-PC*, mantendo dentro de seu conjunto de utilitários o programa **DEBUG**, existente até os dias de hoje em todas as versões de 32 *bits* do sistema operacional Windows.

O utilitário **DEBUG** passou ao longo dos anos de sua existência por diversas mudanças e correções de erros. Nas mãos de um bom programador é uma ferramenta muito eficaz para operar arquivos e escrever pequenos programas em baixo nível. No entanto, com o surgimento do sistema operacional Windows acabou por perder um pouco de seu *glamour*. O programa **DEBUG** basicamente deixou de ser apreciado e utilizado quando seus comandos de I/O se tornaram menos confiáveis, seja devido a um ‘bug’ ou ao próprio sistema operacional Windows. O fato é que os comandos de I/O no Windows não são confiáveis para acesso direto a discos rígidos via **DEBUG**.

Embora tenha sido criado no início da era do microprocessador de 16 *bits* as versões mais recentes do utilitário *DEBUG* são muito úteis para técnicos de PCs no que tange ao acesso direto a determinadas posições de memória. O *DEBUG* nos dias de hoje pode ser útil para fins educacionais como será demonstrado neste trabalho no sentido de auxiliar o entendimento de alguns princípios internos da arquitetura do microprocessador 8086 e da apresentação de códigos em linguagem de máquina e linguagem *Assembly*.

Um dos maiores problemas do programa *DEBUG* desde seu lançamento no MS-DOS é a disponibilidade de uma documentação efetiva para seu uso. Isso se explica pelo fato de ter sido um programa criado para uso particular de Tim Paterson na elaboração do sistema operacional QDOS. Não era intenção inicial de que tal ferramenta fizesse parte de um sistema operacional comercial como ocorreu com o MS-DOS. A ferramenta *DEBUG* é um programa *nerd* voltado para *nerds* e *hackers* e sua documentação passa a não necessária a este público. No entanto, isso não justifica plenamente sua ausência e neste sentido um material mais acentuado a respeito da ferramenta *DEBUG* pode ser consultado no sítio <http://debug.manzano.pro.br/>.

Além do programa *DEBUG*, Tim Paterson escreveu um programa assembler (compilador de linguagem de montagem) chamado **ASM** que foi usado para escrever o sistema operacional 86-DOS. Este programa foi base para que a Microsoft tivesse para seu sistema operacional MS-DOS um compilador de linguagem de montagem chamado **MASM** (Microsoft ASseMbler) que foi evoluindo ao longo dos anos, desde o lançamento de sua primeira versão em 1981, sendo disponibilizado atualmente em conjunto com o ambiente de programa *Visual Studio*.

Em paralelo a existência da ferramenta *MASM* outras ferramentas de compilação para linguagem de montagem surgiram em sua época, destacando-se o **TASM** (Turbo ASseMbler) de 1989 da empresa Borland (não mais distribuída), **FASM** (Flat ASseMbler) de 2000 escrito por Tomasz Grysztar e **NASM** (Netwide ASseMbler) de 2016 escrito por Simon Tatham e Julian Hall.

Os programas *MASM*, *TASM*, *FASM* e *NASM* são ferramentas para a produção de software, principalmente, comerciais podendo ser usadas pela área de educação. No entanto, aprendizagem de programação de baixo nível para novatos pode ser considerada difícil e ferramentas com toque profissional podem ser intimidadoras para alguns. Assim sendo, para uso didático há como alternativa o programa **emu8086** para o sistema operacional Windows que pode ser executado no sistema operacional Linux via programa **Wine**.

3.2 - Os Programas de depuração DEBUG

A partir do desenvolvimento do utilitário *DEBUG.COM* escrito para o sistema operacional QDOS/86-DOS e sua aquisição pela Microsoft para a IBM ocorreu a transição do sistema operacional QDOS/86-DOS para o IBM-DOS e posteriormente para o MS-DOS. Desta forma, passam a existir duas versões simultâneas do utilitário *DEBUG.COM* que foram amplamente usadas até o surgimento do sistema operacional Windows que em modo 32 *bits* possui internamente uma versão do utilitário *DEBUG* de 16 *bits* que pode ser normalmente executado em uma janela de comando do sistema.

No entanto, não é possível executar nas edições de 64 *bits* do sistema operacional Windows programas escritos para o padrão de 16 *bits* estando o programa *DEBUG* nesta condição, a menos que se utilize alguma versão clone do utilitário *DEBUG* (evitando-se a prática de pirataria ao fazer uso de uma versão própria da Microsoft) executado dentro de algum ambiente de virtualização para o sistema operacional MS-DOS, como é o caso do emulador **vDos** (<https://vdos.info/>).

Alternativamente ao sistema operacional MS-DOS de 16 *bits* da Microsoft há um sistema operacional de 32 *bits* chamado **FreeDOS** que trás entre seus utilitários um clone do programa *DEBUG* chamado *DOS Debug* que pode ser executado em sistemas operacionais Windows de 64 bits por meio do ambiente de emulação vDos. O programa **DOS Debug** foi escrito originalmente pelo Professor Paul Vojta (lê-se *pôu vóida*) e mantido posteriormente pelo programador Andreas Grech (<https://www.ibiblio.org/pub/micro/pc-stuff/freedos/files/repositories/1.2/base/debug.zip>) para o sistema operacional *FreeDOS* até a versão 1.25, tendo esta versão melhorias em relação a versão original do utilitário *DEBUG* da Microsoft. A partir da versão 1.26 o utilitário do Professor Paul Vojta vem sendo mantido pelo programador de pseudônimo VCB (<https://sites.google.com/site/pcdosretro/enhdebug>) com o nome **Enhanced DEBUG** (*DEBUG* Aprimorado). O pacote *Enhanced DEBUG* inclui o utilitário *DEBUG.COM* que é um substituto para o *DEBUG* original da Microsoft e o utilitário **DEBUGX.COM**, que depura programas em modo **DPMI** (*DOS Protected Mode Interface*), possuindo recursos, além dos encontrados no programas *DEBUG* (original) e mesmo do programa *DOS Debug*.

Além dos programas *DOS Debug/Enhanced DEBUG* há alternativamente um programa de depuração chamado **GRDB** (Get Real Debugger) desenvolvido pela **LADSoft** (http://ladsoft.tripod.com/grdb_debugger.html) compatível com as ferramentas *DEBUG* comentadas que pode também ser utilizado dentro de ambientes de emulação.

Para o estudo desta obra será utilizado o pacote *Enhanced DEBUG* versão 1.32b a partir do utilitário *DEBUGX.COM* que será executado dentro do programa de emulação *vDos*.

3.3 - Programa emulador assembler emu8086

O programa **emu8086** foi lançado em 1997 sendo escrito pelos programadores Barry Allyn e Yuri Margolin. Este programa possui uma estrutura operacional e visual muito didática por permitir a visualização simultânea de diversos detalhes operacionais da execução de instruções de baixo nível de um microprocessador 8086.

O *emu8086* é um programa emulador do microprocessador 8086 possuindo um modo montador (assembler) integrado, além de conter um conjunto de tutoriais e outros documentos voltados a iniciantes. O emulador executa programas como se fosse um microprocessador real permitindo visualizar a execução desses programas em modo passo a passo, mostrando as ocorrências geradas em todos os registradores, memória, pilha e variáveis, os quais podem ser acompanhados e editados com um duplo clique. As instruções de um programa podem ser executadas no *emu8086* passo a passo tanto para frente como para trás (LVSEVEN, 2018).

O objetivo do programa *emu8086* é ser um instrumento que facilite a aprendizagem da linguagem *Assembly 8086* com alto grau de facilidade de uso não encontrado nos programas montadores. O pacote de software inclui vários dispositivos virtuais externos, como: robô, motor de passo, display de LED e interseção de semáforos. Além de emular um microprocessador 8086 esta ferramenta executa dentro de sua instância (máquina virtual) de operação programas escritos em modo 16 *bits* mesmo em um sistema operacional Windows de 64 *bits*. Além de emular didaticamente um ambiente de 16 *bits* para o microprocessador 8086, o programa *emu8086* pode ser usado como um programa montador por ter integrado a ele o programa *FASM*.

É propício comentar que apesar do programa *emu8086* ser uma excelente ferramenta de estudo ela possui um pequeno problema de distribuição. Algumas vezes fica seu sítio oficial fora do ar o que prejudica a compra da licença para seu uso uma vez que o *emu8086* não é uma ferramenta *freeware*. Quanto a obtenção do programa *emu8086* este pode ser conseguido a partir de diversos serviços de *downloads* existente na *Internet* e estará seu uso restrito ao tempo de teste que é oferecido para um usuário não registrado. Por exemplo, durante o segundo semestre de 2018 o sítio do programa <http://emu8086.com/> esteve fora do ar, sendo esta a segunda vez desta ocorrência desde 1997. Além do sítio oficial, na documentação da versão do programa usada para a elaboração deste trabalho encontra-se como opção de contato o e-mail air.plane@dr.com.

3.4 - Preparação do ambiente de trabalho

Para o estudo deste livro é necessário o uso de três programas: **emu8086**, **vDos** e **Enhanced DEBUG**. O programa *vDos* é necessário para a execução do programa *Enhanced DEBUG* junto ao sistema operacional Windows de 64 *bits*.

Para obter o programa *vDos* acesse o sítio <https://vdos.info/> e selecione em seu menu a opção **Download** e na página apresentada acesse o **link Installation program 2018.05.01** em **Download vDos: Installation program 2018.05.01 (1.4 MB) | Source files or older versions** para que o programa de instalação **vDosSetup.exe** seja copiado para seu computador. A partir da cópia do programa **vDosSetup.exe** selecione-o com um duplo clique do ponteiro do *mouse* e proceda a instalação junto a pasta **vDos** no disco **C:** de seu computador.

Após copiar e instalar o programa *vDos* é necessário obter e instalar o programa de depuração *Enhanced DEBUG*. Para tanto, acesse o sítio <https://sites.google.com/site/pcdosretro/enhdebug>, e selecione no final da página o **link Download DEBUG 1.32b** que copiará o arquivo **DEBUGX.ZIP**. A partir da cópia do arquivo **DEBUGX.ZIP** descompacte-o e copie a pasta gerada para a raiz do disco **C:**.

A partir da instalação dos programas *vDos* e *Enhanced DEBUG* é necessário fazer a configuração para uso do programa *Enhanced DEBUG* dentro do ambiente *vDos*. Para tanto, utilizando-se do programa **Explorer** do Windows abra na pasta *vDos* do disco **C:** o arquivo **autoexec.txt** e acrescente no final do arquivo a linha de instrução **use e: c:\debugx**.

A partir da instalação e configuração dos programas *vDos/Enhanced DEBUG* é necessário obter e instalar o programa **emu8086**. Assim sendo, acesse o sítio <http://www.emu8086.com/> (ou outro serviço de sua confiança que possa fornecer-lo) e selecione o link **emu8086v408r10.zip** (17 de novembro de 2015 quando a versão **4.08r10** estava disponível) e baixe para sua pasta de trabalho, descompacte o arquivo e execute o programa **setup** e siga as orientações apresentadas acionando os botões **Next** e **Finish** quando apresentados.

Observação

Caso seja apresentada mensagem de erro com arquivo **dll** vá ao sítio <http://support.microsoft.com/gp/vbruntime>, selecione a partir da seção **Visual Basic 6.0** selecione a opção **FILE: VBRUN60.EXE Installs Visual Basic Runtime Files**. Na página apresentada selecione o link **Baixe o pacote VBRun60.exe agora** e baixe-o para sua pasta de trabalho. Após obtenção do arquivo **vbrun60.exe** proceda sua instalação.

Concluído o processo de instalação, o programa pode ser acionado por intermédio do ícone na área de *desktop* do Windows **emu8086**. Assim sendo, dê um duplo clique com o ponteiro do *mouse* sobre o ícone **emu8086**, selecione o botão **New** na caixa de diálogo **welcome**. Na caixa de diálogo **choose code template** mantenha a seleção da opção **COM template** e acione o botão **OK**.

Após os passos anteriores será apresentada a tela do ambiente de trabalho do programa **emu8086**. Inicialmente, como primeira execução será definida a configuração do tipo de caractere em que os programas serão apresentados dentro do ambiente do **emu8086**. Acione na barra de ferramentas do programa o botão **options** (nono botão) e aparece a caixa de diálogo **options** para ter acesso a caixa de diálogo **options**.

Acione o botão **set font** da área **fonts and colors** e defina para a opção **control** como **source code editor** a fonte de trabalho **Courier New** no formato **Negrito**, deixe como tamanho o valor **12** e acione o botão **OK**. No quadro **background color** da área **fonts and colors** dê um clique e selecione no quadro de cores apresentados a opção para cor branca. Na área **fonts and colors** selecione para **control** a opção **memory list** e defina para **background color** a cor branca e em seguida para a opção **disassembler list** defina a mesma opção de **memory list**. No quadro **keywords** junto à opção **item** selecione com o botão *drop-box* a opção **line numbers** e para a opção **background color** selecione a cor cinza claro e para a opção **foreground color** selecione a cor cinza escuro. Ao final acione o botão **close** para fechar a janela de caixa de diálogo **options**.

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 4

CÁLCULOS MATEMÁTICOS BÁSICOS

Antes mesmo de estudar a linguagem de programação Assembly 8086/8088 (mesmo em nível básico, como é a proposta desta obra), é importante ter uma noção do funcionamento de um computador controlado pelo seu código nativo, ou seja, em linguagem de máquina, pois esse conhecimento dá uma ideia do quão mais fácil é usar a linguagem de programação Assembly. Este capítulo dá noção das operações computacionais de máquina (cálculos matemáticos, baseados nas quatro operações aritméticas) realizadas com o programa **Enhanced DEBUG**. Ensina a executar algumas instruções internas do programa, além de utilizar a aritmética com valores hexadecimais, negativos e a operação de cálculos matemáticos simples (adição, subtração, multiplicação e divisão) com códigos de máquina.

4.1 - O Programa Enhanced DEBUG

Os sistemas operacionais MS-DOS de 16 *bits* e Microsoft Windows em todas as suas versões de 32 *bits* até a versão 10 que necessita da instalação do programa **NTVDM**, possuem no diretório básico de operação o programa **DEBUG**. No sistema operacional MS-DOS o programa fica instalado no diretório C:\DOS, e no Microsoft Windows o programa fica instalado na pasta C:\WINDOWS\COMMAND ou na pasta C:\WINDOWS\SYSTEM32. O programa **DEBUG** (foi originalmente escrito por Tim Paterson em 1980) não sendo encontrado nas edições de 64 *bits* do sistema operacional da Microsoft Windows. Neste caso, pode-se fazer uso de uma versão alternativa do programa desenvolvida pelo professor Paul Vojta para o projeto **FreeDOS**, denominado **Enhanced DEBUG** que opera apenas sob 32 *bits*.

O programa **DEBUG** (versão original) pode ser executado no sistema operacional Microsoft Windows 7 de 64 *bits*, por meio do recurso de virtualização **XP Mode**, desde que este recurso esteja instalado. No caso dos sistemas operacionais Microsoft Windows Vista, 7, 8, 8.1 e 10 de 64 *bits* uma solução é o uso do programa **Enhanced DEBUG** executado preferencialmente com auxílio do emulador de sistema operacional MS-DOS denominado **vDos**.

O nome *debug* está associado à retirada de defeitos e erros existentes em um programa. O termo *bug* (bãgui), que em inglês significa inseto, foi associado a falhas computacionais quando ocorreu uma parada inesperada do computador ENIAC por ser encontrada uma mariposa presa em um dos circuitos eletrônicos (MANZANO & MANZANO, 1998). Há autores que afirmam ter sido uma parada no computador MARK I (NORTON & SOCHA, 1988). Independentemente da versão histórica, o fato é sempre associado ao evento de ter sido encontrado um inseto. Assim sendo, *debug* significa algo como dedetizar, ou seja, retirar insetos, remover problemas.

O programa **Enhanced DEBUG** é uma ferramenta básica que permite estabelecer a manipulação de dados e de registradores de memória em linguagem de máquina, dando suporte à ação de comandos da linguagem de programação *Assembly* para o padrão de microprocessadores 8086/8088 (IBM, 1995). É uma ferramenta simples, mas poderosa, que possui a capacidade de criar pequenos programas em linguagem de máquina. Sua capacidade de trabalho está limitada à criação de programas com extensão **.COM**. Por esta razão, esses programas podem ter no máximo 64 Kb de memória e devem sempre ser iniciados no endereço de deslocamento de memória **0100h**.

É oportuno esclarecer onde se encontra contextualmente as ações de trabalho das linguagens de programação relacionadas a códigos de máquina, ao código *Assembly* e as linguagens de alto nível. A Figura 4.1, adaptada de Yadav (2008, p. 25) apresenta o esquema de distribuição de acesso entre os *hardwares* e as linguagens de programação de máquina, *assembly* e alto nível.

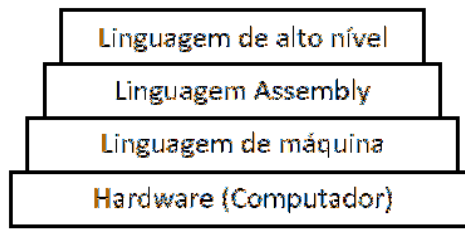


Figura 4.1 - Esquema de distribuição de acesso entre o hardware e as linguagens de programação.

Para iniciar a execução do programa **Enhanced DEBUG** selecione na área de trabalho do sistema operacional Microsoft Windows o ícone do programa **vDos** e na linha de comando apresentada informe os comandos:

```
C:\>E:      <Enter>
E:\>DEBUGX  <Enter>
```

A indicação **<Enter>** sugere que a tecla **<Enter>** seja acionada a cada comando fornecido, isto é, não se deve escrever após a indicação de cada comando o indicativo **<Enter>**. Assim que o programa é carregado, ele apresenta seu *prompt* de trabalho com um pequeno traço (um hífen) à esquerda da tela. Observe na Figura 4.2 a tela após a chamada do programa **Enhanced DEBUG**.

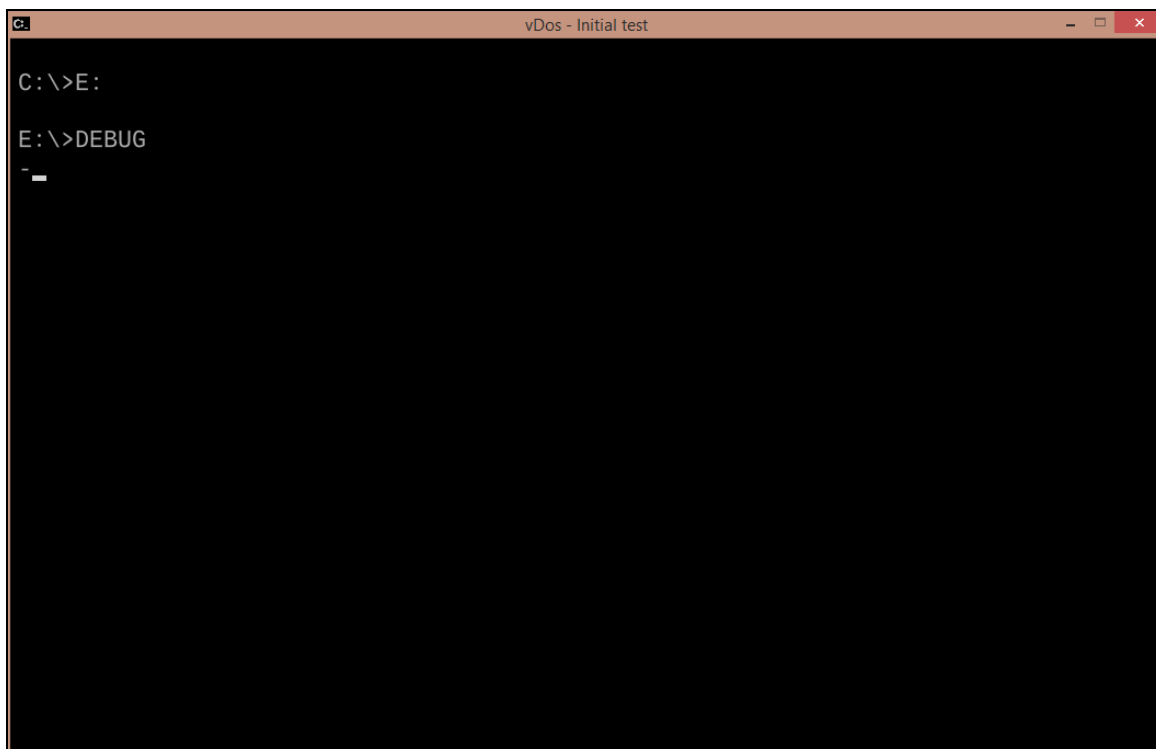


Figura 4.2 - Aparência da tela com o programa DEBUG em operação (Microsoft).

Para sair do programa **Enhanced DEBUG**, basta informar no seu *prompt* de trabalho o código **Q** (*quit*).

É pertinente indicar que todos os comandos internos de referência ao programa **Enhanced DEBUG** podem ser escritos em caracteres maiúsculos ou minúsculos. No entanto, neste capítulo as referências aos comandos são apresentadas sempre em caracteres maiúsculos.

O programa **Enhanced DEBUG** tem uma série de comandos que facilitam as operações em memória. Para ter uma ideia dos comandos disponíveis, acione no *prompt* do programa o comando **?** (*help*), o qual apresenta a lista completa de comandos, conforme exhibe as Figuras 4.3 e 4.4. No decorrer do livro, alguns desses comandos internos serão utilizados em várias oportunidades.

```

C:\ vDos - Initial test
assemble      A [address]
compare       C range address
dump          D[B|W|D] [range]
dump interrupt DI interrupt [count]
dump LDT      DL selector [count]
dump MCB chain DM
dump ext memory DX [physical_address]
enter         E address [list]
fill          F range list
go            G [=address] [breakpoints]
hex add/sub   H value1 value2
input         I[W|D] port
load file     L [address]
load sectors  L address drive sector count
move          M range address
set CPU mode  M [x|N|T] (x=0..6, N=no FPU, T=386 with 287)
name          N [[drive:][path]filename [arglist]]
output        O[W|D] port value
proceed       P [=address] [count]
proceed return PR
quit          Q
register       R [register [value]]
MMX register  RM
FPU register  RN[R]
[more]

```

Figura 4.3 - Lista de comandos do programa Enhanced DEBUG – Parte 1.

```

C:\ vDos - Initial test
hex add/sub   H value1 value2
input         I[W|D] port
load file     L [address]
load sectors  L address drive sector count
move          M range address
set CPU mode  M [x|N|T] (x=0..6, N=no FPU, T=386 with 287)
name          N [[drive:][path]filename [arglist]]
output        O[W|D] port value
proceed       P [=address] [count]
proceed return PR
quit          Q
register       R [register [value]]
MMX register  RM
FPU register  RN[R]
toggle 386 regs RX
search        S range list
trace         T [=address] [count]
trace mode    TM [0|1]
unassemble    U [range]
view flip     V
write file     W [address]
write sectors  W address drive sector count

prompts: '-' = real/v86-mode; '#' = protected-mode
-

```

Figura 4.4 - Lista de comandos do programa Enhanced DEBUG – Parte 2.

É pertinente salientar que não serão vistos todos os comandos do utilitário **Enhanced DEBUG**, apenas os comandos que são encontrados igualmente em todos os programas *DEBUG*. Maiores detalhes sobre o programa *DEBUG* podem ser obtidos junto ao sítio www.debug.manzano.pro.

4.2 - Aritmética em modo hexadecimal

O recurso de cálculo numérico oferecido no programa **Enhanced DEBUG** permite obter os resultados de adição e subtração de dois valores hexadecimais (não é possível trabalhar com números decimais) fornecidos no *prompt* do programa.

Para adição e subtração de valores hexadecimais é necessário utilizar o comando **H** (*hex*) e informar sempre dois valores hexadecimais. Escreva no *prompt* do programa o comando **H** seguido dos valores hexadecimais **0005** e **0001** (que possuem o mesmo significado dos seus equivalentes em decimal) e acione a tecla **<Enter>**. Veja a indicação:

```
H 0005 0001 <Enter>
```

Após executar a operação anterior, o programa apresenta como resultado a informação:

```
0006 0004
```

O valor hexadecimal **0005** e o valor hexadecimal **0001** resultaram na apresentação dos valores **0006** e **0004**, sendo **0006** o valor da adição de **0005** com **0001**, e **0004** o valor da subtração entre **0005** e **0001**. Observe que o programa **DEBUG** faz os dois cálculos aritméticos simultaneamente e fornece as duas respostas.

No entanto, um valor hexadecimal é diferente de um valor decimal. Por exemplo, execute o seguinte comando de cálculo aritmético para calcular os resultados de adição e subtração entre os valores hexadecimais **0009** e **0001**.

```
H 0009 0001 <Enter>
```

Após a execução da ação anterior, o programa apresenta como resultado a informação:

```
000A 0008
```

O valor **000A** (hexadecimal) corresponde ao valor **10** (decimal). Apesar de ser uma operação de fácil execução, é necessário ter muito cuidado com a leitura do valor numérico, uma vez que não está sendo utilizada base decimal de representação numérica.

Outro cuidado a ser tomado é em relação à utilização de valores numéricos maiores que a capacidade de cálculo do programa **Enhanced DEBUG**. Você deve ter notado que a resposta dada à solicitação de um determinado cálculo aritmético (adição e subtração) é sempre apresentada em quatro posições (dígitos). Números hexadecimais com quatro posições usam a estrutura de dados *word* para serem armazenados e processados. Números com dígitos acima dessa capacidade resultam em um estouro de cálculo, e isso é considerado um erro de operação.

Se estiver em uso o programa **DEBUG** (original) este tem a limitação de trabalhar com valores até o tamanho de um *word*, compatíveis com microprocessador de 16 bits. Por exemplo, se houver a tentativa de realizar o cálculo aritmético entre os valores hexadecimais **B000** (45.056 em decimal) e **A000** (40.960 em decimal), ocorre um erro de estouro:

```
H B000 A000 <Enter>
```

Após a ação anterior o programa apresenta como resultado a informação:

```
5000 1000
```

O valor hexadecimal **5000**, resultado da adição dos valores **B000** e **A000** no programa **DEBUG** original que deveria ser apresentado como sendo o valor hexadecimal **15000** (86.016 em decimal) como ocorre no uso do programa **Enhanced DEBUG** ao ser apresentado o valor **00015000**. Já o valor hexadecimal **1000** (4.096 decimal) está sendo apresentado de forma correta tanto no **DEBUG** original como no **Enhanced DEBUG**. O programa **DEBUG** original mostra apenas os quatro valores à direita do resultado da adição. Essa restrição do programa limita o seu uso em algumas operações. Já o programa **Enhanced DEBUG** apresenta o resultado de forma completa pelo fato de operar com microprocessadores de 32 *bits* e não 16 *bits* como ocorre com o programa **DEBUG** original.

Em particular o programa **Enhanced DEBUG** possui a capacidade de efetuar suas operações também em modo 32 *bits*. Para fazer uso deste recurso basta executar no *prompt* do programa o comando **RX** que apresentará a mensagem **386 regs on**. A partir deste instante o modo 32 *bits* de operação do programa torna-se ativo. Para voltar o programa **Enhanced DEBUG** ao modo de operação 16 *bits* basta executar novamente a instrução **RX**. A Figura 4.5 mostra a ativação do modo 32 e 16 *bits* e o uso do comando **R** que apresenta o estado dos registradores manipulado pelo programa.

```

C:\vDos - Initial test
trace mode      TM [0|1]
unassemble     U [range]
view flip      V
write file      W [address]
write sectors   W address drive sector count

prompts: '-' = real/v86-mode; '#' = protected-mode
-R
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
-RX
386 regs on
-R
EAX=00000000 EBX=00000000 ECX=00000000 EDX=00000000 ESP=0000FFFE EBP=00000000
ESI=00000000 EDI=00000000 EIP=00000100 EFL=00007202 NV UP EI PL NZ NA PO NC
DS=07E9 ES=07E9 SS=07E9 CS=07E9 FS=0000 GS=0000
07E9:0100 C3          RET
-RX
386 regs off
-R
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
-

```

Figura 4.5 - Modelos de de uso de registradores em 16 ou 32 bits.

Os exemplos de operação apresentados neste trabalho consideram apenas o uso de registradores em modo 16 *bits* atendendo o uso do microprocessador 8086.

4.3 - Representação de valores negativos

Como diz Hyde (2003) os valores numéricos representados em um computador digital não podem ser infinitos, como ocorre no universo matemático, pelo simples fato de que a quantidade de *bits* disponível para representá-los em um computador é restrita (8, 16, 32, 64, 128 ou qualquer outra quantidade que nunca será muito grande, ou seja, sempre haverá um limite máximo). Com um número fixo de *bits*, o valor máximo também é fixo. Por exemplo, com 8 *bits* (1 *byte*) pode-se obter no máximo 256 combinações de valores diferentes. Se houver a necessidade de expressar números negativos (o que é comum e necessário), será preciso dividir as 256 possibilidades existentes em dois conjuntos numéricos.

Assim sendo, ter-se-á metade de um conjunto para representar os valores positivos e a outra metade para representar os valores negativos. Essa atitude diminui o valor máximo de combinações, porém aumenta o valor mínimo de combinações. Se a divisão for benfeita, será possível obter valores na faixa numérica de -128 até 127. O mesmo raciocínio pode ser aplicado para valores numéricos de 16 *bits*, 32 *bits* etc.

Para exemplificar uma operação aritmética que represente a obtenção de valores negativos, considere o cálculo entre os valores hexadecimais **0005** e **0006** (nessa ordem). Informe na linha de *prompt* do programa:

H 0005 0006 <Enter>

Após a ação anterior o programa apresenta como resultado a informação:

000B FFFF

Certamente o leitor estava esperando a apresentação do valor hexadecimal de adição **000B** (o que ocorreu) e o valor hexadecimal de subtração **-0001** (que está sendo representado pelo valor hexadecimal **FFFF**). No contexto exposto o valor hexadecimal **FFFF** está representando realmente o valor decimal **-1**. Assim sendo, a operação está correta. Tanto que, se for feito o cálculo da soma do valor hexadecimal **0005** com o valor hexadecimal **FFFF**, a resposta será os valores **0004** e **0006**:

H 0005 FFFF <Enter>

O programa apresenta como resultado a informação:

00010004 0006

O valor hexadecimal **0005** com o valor hexadecimal **FFFF** (equivalente a **-1** em decimal) resultam no valor de adição hexadecimal **00010004** (valor decimal **5**, mais o valor decimal **-1**) e no valor de subtração hexadecimal **0006** (valor decimal **5**, menos valor decimal **-1**). Se estiver em uso o programa **DEBUG** (original) o resultado apresentado será diferente como **0004 0006**. O valor hexadecimal **FFFF**, dependendo do contexto em que é aplicado, pode ser interpretado como o valor decimal **65.535**. O valor **65.535** é o maior valor positivo que pode ser representado em um *word*.

A consideração em relação ao valor hexadecimal **FFFF** ser interpretado como o valor decimal **-1** ou como o valor decimal **65.535** depende de levar em conta se o valor hexadecimal **FFFF** está ou não sinalizado. Se o valor hexadecimal **FFFF** for positivo (equivalente ao valor decimal **65.535**), será considerado um valor não sinalizado. Agora, se o valor hexadecimal **FFFF** for negativo (equivalente ao valor decimal **-1**), será considerado um valor sinalizado, ou seja, um valor que possui o sinal de subtração à sua frente para determinar sua condição. Desta forma, todos os valores numéricos que gerarem um estouro e estiverem situados na faixa numérica hexadecimal de **8000** até **FFFF** poderão ser representados como valores sinalizados (valores negativos) ou como valores positivos (valores não sinalizados).

A informação que determina se um valor hexadecimal é negativo toma por base o estouro do valor quando o resultado de uma operação aritmética é apresentado (ou seja, um valor com cinco posições numéricas), desde que o estouro esteja situado na faixa numérica hexadecimal de **8000** até **FFFF**; caso contrário, ter-se-á um erro como o exemplificado no final do tópico anterior.

Por exemplo, a subtração do valor hexadecimal **0005** do hexadecimal **FFFF** (considerado **-1**) resulta na resposta **0004**. No entanto, internamente ocorre, como já mencionado, a soma dos valores e não a subtração, ou seja, o valor hexadecimal **0005** é somado ao valor hexadecimal **FFFF**, que resulta em um valor hexadecimal equivalente a **00010004**.

No caso do programa **DEBUG** (original) ocorre a desconsideração dos quatro valores à esquerda, ou seja, despreza-se o valor **0001** mantendo-se apenas as quatro posições à direita do valor **0004**. O valor **1** é, na verdade, a indicação de um estouro (ocorrência de *overflow*) da capacidade máxima de armazenamento dos valores numéricos num dado *word*, ficando armazenado num registrador de *flag* (que será apresentado um pouco mais adiante).

É conveniente levar em consideração o fato de que os computadores digitais somente processam informações em formato binário. O programa **Enhanced DEBUG** (e mesmo o programa **DEBUG**) aceita a entrada de valor numérico em formato hexadecimal e o apresenta em formato hexadecimal, mas o converte internamente em seu respectivo valor binário. A real determinação de um valor ser expresso em formato positivo ou negativo ocorre na esfera binária do processamento da máquina, e não na esfera hexadecimal.

Para entender melhor a representação de valores negativos, considere como exemplos os valores hexadecimais e binários exibidos na Tabela 4.1, a qual mostra os limites mínimos e máximos dos valores positivos e negativos que podem ser representados em um *word*.

Note que os valores positivos estão situados na faixa de **0000h** (valor **0** em decimal) até **7FFFh** (valor **32.767** em decimal) e os valores negativos estão situados na faixa de **8000h** (valor **-32.768** em decimal) até **FFFFh** (valor **-1** decimal).

Observe na tabela a indicação da segunda coluna com a apresentação dos valores binários equivalentes aos valores hexadecimais indicados na primeira coluna.

Tabela 4.1 - Faixa de Valores Positivos e Negativos

Faixa de Valores Positivos e Negativos	
Valor Hexadecimal	Valor Binário
0000 (positivo)	0000 0000 0000 0000
7FFF (positivo)	0111 1111 1111 1111
8000 (negativo)	1000 0000 0000 0000
FFFF (negativo)	1111 1111 1111 1111

Os valores binários negativos possuem o seu décimo sexto *bit* (primeiro valor à esquerda) mais significativo iniciado com o valor binário **1** (consideram-se então valores com sinalização), enquanto os valores binários positivos possuem o décimo sexto *bit* iniciado com o valor binário **0** (consideram-se valores sem sinalização). Por meio do décimo sexto *bit* (considerando-se o uso de 16 bits) o processador reconhece o uso ou não de um número negativo.

Um *word* pode representar valores positivos na faixa numérica decimal de **0** até **65.535**, e pode também representar valores numéricos positivos ou negativos na faixa decimal de **-32.768** até **32.767**. A forma de representação numérica depende do tipo de instrução em uso no processador para que o décimo sexto *bit* seja ou não considerado.

A título de ilustração, apresenta-se a seguir a regra de complementação por dois que estabelece o critério para determinar se um valor é ou não positivo. Considere o fato de converter o valor positivo decimal **10** no valor negativo decimal **-10**. Na esfera decimal, basta multiplicar o valor positivo por **-1**, e pronto. Mas na esfera binária, o processo é diferente.

O valor decimal **10** equivale ao valor hexadecimal **000A** que corresponde ao valor binário **1010**. Para efetuar a conversão do valor decimal **10** no respectivo valor binário negativo, é necessário:

- ◆ Converter o valor decimal **10** (valor hexadecimal **A**) em sua forma binária, ou seja, **1010**.
- ◆ Acrescentar à esquerda do valor binário valores **0** para formar um *word* completo, ou seja, no caso do valor **1010**, é necessário acrescentar zeros para formar o tamanho do *word* **0000 0000 0000 1010**.
- ◆ Converter o valor a ser definido em binário negativo com a inversão de todos os valores para seus opostos (a esse processo dá-se o nome de complementação – *complementação de base* ou *complemento de 1*), ou seja, o valor binário **0000 0000 0000 1010** deve ser escrito como **1111 1111 1111 0101**. Note que os valores **0** tornaram-se **1** e os valores **1** tornaram-se **0**.
- ◆ Somar o valor binário **1** ao valor binário **1111 1111 1111 0101** resulta no valor **1111 1111 1111 0110** que será considerado negativo decimal **-10** ou seu equivalente em hexadecimal **FFF6**. O valor binário **1111 1111 1111 0101** é considerado negativo, pois seu bit mais significativo (décimo sexto bit) está sinalizado com valor **1**. Se o valor binário **1111 1111 1111 0101** não for sinalizado (isso ocorre se estiver em uso mais de 16 bits e nesta condição possuir no novo tamanho o bit mais significativo igual a zero) este valor será considerado positivo equivalente a **65525**.

De acordo com a mesma estrutura de raciocínio veja, por exemplo, como fica o valor negativo decimal **-10** apresentado em formato positivo. Acompanhe os passos:

- ◆ A partir do valor negativo binário **1111 1111 1111 0110** faça a inversão dos valores pela técnica de complementação para obter o valor binário **0000 0000 0000 1001**.
- ◆ Some o valor binário **1** ao valor binário **0000 0000 0000 1001** e será obtido o valor **0000 0000 0000 1010**, que será considerado o valor positivo decimal **10** ou seu equivalente em hexadecimal **000A**.

A regra de complementação por dois é realizada em duas etapas (daí a denominação complemento por dois ou complementação por dois). Na primeira etapa inverte-se o valor binário e na segunda soma-se o valor **1** ao número invertido.

4.4 - Cálculos em códigos de máquina

No tópico anterior foram apresentadas instruções para fazer cálculos simples com dois valores numéricos hexadecimais utilizando o comando **H** dos programas **Enhanced DEBUG**. Neste tópico as operações matemáticas serão realizadas

com o armazenamento de valores hexadecimais (códigos escritos em linguagem de máquina) nos registradores gerais **AX** e **BX**, e em linguagem de máquina e não em linguagem *Assembly*, pelo menos por ora.

A ideia geral do que são registradores foi explanada no capítulo anterior. No entanto, é bom lembrar que um registrador tem comportamento semelhante a uma variável em uma linguagem de alto nível.

Para visualizar a estrutura de registradores do processador do computador em uso, é necessário informar no *prompt* dos programas **Enhanced DEBUG** o comando **R** (*register*):

R <Enter>

Ao acionar o comando **R**, aparece uma listagem como a indicada a seguir:

```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
```

Em seu computador a primeira linha é idêntica à primeira linha da imagem anterior, mas é bem provável que na segunda e terceira linhas sejam apresentados dados com valores diferentes. A diferença encontrada nos valores se refere à quantidade de memória do computador em uso.

Considere apenas neste momento as quatro informações da primeira linha (lado esquerdo) que fazem menção ao uso dos registradores gerais **AX**, **BX**, **CX** e **DX**, os quais estão definidos com valor hexadecimal **0000**. Os demais itens serão vistos mais adiante.

Todos os registradores apresentados têm um valor numérico hexadecimal de quatro posições (dígitos), os quais representam o armazenamento de um *word*, cada um com o tamanho de 16 *bits*.

Os registradores gerais podem armazenar valores numéricos inteiros sem sinalização de 0 até 65.535 (em decimal) ou valores numéricos inteiros com sinalização de -32.768 até 32.768.

4.4.1 - Adição de valores hexadecimais

O comando **R**, além de apresentar informações sobre o estado dos registradores, possibilita fazer a atribuição de valores a um determinado registrador. Imagine que você queira armazenar o valor hexadecimal **000A** (equivalente ao valor **10** em decimal) no registrador geral **AX**. Neste caso, informe na linha de *prompt* do programa a instrução:

R AX 000A <Enter>

Em seguida execute o comando:

R <Enter>

Observe o resultado apresentado na tela de acordo com o trecho sinalizado em negrito a seguir:

```
AX=000A BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
```

É possível realizar a entrada de qualquer valor hexadecimal em qualquer um dos registradores existentes pelo comando **R**. Sendo assim, informe para o registrador geral **BX** o valor hexadecimal **0001** (equivalente ao valor **1** em decimal), como demonstrado a seguir:

R BX 0001 <Enter>

Em seguida execute o comando:

R <Enter>

Observe o resultado apresentado na tela de acordo com o trecho sinalizado em negrito a seguir:

```
AX=000A BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3 RET
```

A partir do momento em que há dois valores armazenados nos registradores gerais **AX** e **BX**, torna-se possível fazer os cálculos matemáticos de adição, subtração, multiplicação e divisão.

Dentre as quatro operações será demonstrado primeiro como somar os valores armazenados nos registradores. Para que essa ação seja efetivada, é necessário colocar na memória do computador dois códigos numéricos (código de máquina) que correspondem à operação de adição.

Observação

O programa **Enhanced DEBUG** tem uma característica que o programa **DEBUG** não tem. Ele permite a entrada direta de um valor em um registrador, usando no *prompt* do programa uma entrada como **R BX 0001** para então acionar a tecla <Enter>. No caso do programa **DEBUG** usa-se primeiro o comando **R BX** e após acionar a tecla <Enter> informa-se o valor **0001** e aciona-se <Enter> novamente.

O problema agora é saber em que lugar da memória serão armazenados os dois códigos (código numérico em linguagem de máquina da operação matemática desejada) que permitirão fazer o cálculo. Por questões técnicas do próprio computador, a memória é segmentada em lotes de 64 KB, denominados *segmentos de dados* (este assunto será tratado mais adiante).

Numa programação em grande escala é necessário saber em que posição de qual segmento de memória uma determinada operação será definida. Pelo fato de estar sendo utilizado o programa **Enhanced DEBUG**, é preciso apenas informar em qual posição (deslocamento) de um determinado segmento (escolhido pelos programas **Enhanced DEBUG**) as instruções de operação de cálculo devem ser definidas.

Fica estabelecido, pela própria engenharia operacional do programa **Enhanced DEBUG**, o uso para a entrada dos códigos em linguagem de máquina do endereço de memória a partir de **0100** (valor em hexadecimal que representa o *byte* de deslocamento a partir do endereço inicial do segmento de memória em uso, neste caso **07E9**, podendo muitas vezes ser definido pelo programa outro segmento de memória). Como é necessário informar dois códigos de máquina, o primeiro será guardado no endereço hexadecimal **0100** e o segundo no endereço hexadecimal **0101**.

Para a entrada de valores em um determinado endereço de memória, deve-se utilizar o comando **E** (*enter*). Neste caso, informe na linha de *prompt* do programa a instrução:

E 0100 <Enter>

O programa apresenta algo semelhante as linhas seguintes:

```
07E9:0100 C3.
```

O valor **07E9** (que em seu computador pode vir a ser outro) apresentado antes do valor de deslocamento **0100** mostra o segmento de memória eleito pelos programas **Enhanced DEBUG**, o qual será usado para a ação aritmética ser implementada. Observe que, ao lado direito do endereço, aparece um valor antes de um ponto. Esse valor pode também ser diferente em seu computador. Ao lado do símbolo de ponto informe o código hexadecimal:

```
03 <Enter>
<Enter>
```

Em seguida, na linha de *prompt* do programa, informe a instrução:

E 0101 <Enter>

O programa apresenta então algo semelhante a:

07E9:0101 00.

Ao lado do símbolo de ponto, informe o código hexadecimal:

C3 <Enter>
<Enter>

Os valores em hexadecimal **03** e **C3** representam os códigos em linguagem de máquina responsáveis pela operação de adição dos valores armazenados nos registradores gerais **AX** e **BX**, mais precisamente da adição do valor do registrador geral **BX** com o valor definido no registrador geral **AX**. Esses códigos em hexadecimal são os *opcodes* (códigos operacionais) que permitem o controle operacional de um microprocessador em linguagem de máquina. Em linguagem de programação *Assembly* o código de operação em linguagem de máquina (*opcode*) **03C3** faz a soma (**ADD**) entre os conteúdos dos registradores **AX** e **BX**.

Em seguida execute o comando **R** para que seja apresentado o estado atual dos registradores:

R <Enter>

Observe atentamente os trechos em negrito como é mostrado em seguida:

```
AX=000A BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 03C3          ADD     AX,BX
```

Atente para os pontos marcados em negrito. Os registradores gerais **AX** e **BX** apresentam, respectivamente, os valores hexadecimais **000A** e **0001**. Observe a mudança ocorrida na terceira linha, a qual mostra o segmento em uso, no caso **07E9**, e o endereço de deslocamento **0100** (**07E9:0100**) definido com o par de códigos de máquina **03C3** (que foram informados separadamente), os quais representam a operação de adição identificada pela instrução **ADD** (*addition*) que será executada sobre os registradores gerais **AX** e **BX**. Note que a operação de um computador em linguagem de máquina via processador ocorre com instruções armazenadas em forma de números, neste caso, hexadecimais.

Em seguida é necessário informar ao processador onde encontrar na memória os códigos de ação da operação de adição. Todo o trabalho anterior foi apenas para entrar os valores e determinar a operação que será ainda realizada. Para a ação desejada o processador precisa saber o segmento e o deslocamento que devem ser utilizados, ou seja, onde está a instrução de processamento. Essas duas informações foram armazenadas nos registradores de apontamentos **CS** (*code segment* - registrador de segmento) e **IP** (*instruction pointer* - registrador de deslocamento), como pode ser notado nos trechos em negrito. Note também a terceira linha com as posições de segmento **07E9** (que pode ser diferente em seu computador) e de deslocamento **0100** marcadas em negrito.

Observação

Caso o registrador de apontamento **CS** e o registrador de deslocamento **IP** não apontem para os endereços definidos, respectivamente, para o segmento e para o deslocamento, eles necessitam ser informados manualmente por intermédio do comando **R**, de forma semelhante à entrada de valores nos registradores gerais **AX** e **BX**.

Assim que todas as verificações são feitas, basta pedir para o programa **Enhanced DEBUG** executar com o comando **T** (*trace*). Observe que o programa tem apenas uma instrução (**ADD AX,BX**):

T <Enter>

Após executar o comando **T** será efetuado o cálculo da adição e apresentado o estado dos registradores, semelhante à forma seguinte:

```
AX=000B BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 0000          ADD     [BX+SI],AL          DS:0001=20
```

Após a execução do comando **T**, observe os pontos marcados em negrito. O valor da adição foi armazenado no registrador geral **AX** (como sendo o valor hexadecimal **000B**, equivalente ao valor **11** em decimal), ou seja, ocorreu uma operação aritmética semelhante a **AX ← AX + BX**.

O valor do deslocamento (*offset*) do registrador de apontamento **IP** está indicando para o endereço de deslocamento **0102**, porque o comando **T** executa as instruções do programa passo a passo, a partir do primeiro até o último deslocamento sem nenhuma instrução. Isso pode ser percebido pela indicação da terceira linha.

Por exemplo, aponte o registrador de deslocamento **IP** para o endereço de deslocamento **0100** novamente. Execute no *prompt* do programa a instrução:

```
R IP 0100 <Enter>
```

Em seguida acione o comando **R** e observe o valor existente no registrador **IP**, como é indicado a seguir em negrito. Note também a alteração da informação da terceira linha.

```
AX=000B BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 03C3          ADD     AX,BX
```

Execute o comando **T** novamente:

```
T <Enter>
```

O valor hexadecimal **0001** do registrador geral **BX** é somado ao valor hexadecimal **000B** do registrador geral **AX**, colocando no registrador geral **AX** o valor hexadecimal **000C** (equivalente ao valor **12** em decimal), como é indicado a seguir:

```
AX=000C BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 0000          ADD     [BX+SI],AL          DS:0001=20
```

A ocorrência de deslocamento registrada no registrador de deslocamento **IP** indica novamente o posicionamento em **0102**. Se forem repetidas as ações descritas, ter-se-á um acumulador de valores em memória.

Anteriormente foi usado o par de valores de *opcodes* **03** e **C3** para somar o valor do registrador geral **BX** com o valor do registrador geral **AX**. É possível também fazer a operação inversa, ou mesmo efetuar operações com a porção menos significativa dos registradores gerais **AX** e **BX**. A Tabela 4.2 apresenta as operações de adição com o *opcode* usual e também um *opcode* alternativo quando houver. O *opcode* alternativo é uma segunda opção para executar uma determinada ação computacional.

Teste os *opcodes* apresentados na tabela para os valores atuais dos registradores gerais **AX** e **BX**, considerando inclusive as partes menos significativas dos respectivos registradores gerais mencionados, como foram os valores apresentados (valores de um *byte*). Caso queira entrar um valor que ocupe um *word*, faça-o utilizando todas as posições numéricas. Por exemplo, experimente os valores **1111** e **2222** para os Registradores gerais **AX** e **BX**.

Tabela 4.2 - Opcode usual e alternativo para adição

Operação Opcode		Adição	
Usual	Alternativo	Registradores Gerais	Operação
02 C3	00 D8	AL, BL	$AL \leftarrow AL + BL$
03 C3	01 D8	AX, BX	$AX \leftarrow AX + BX$
02 D8	-	BL, AL	$BL \leftarrow BL + AL$
03 D8	-	BX, AX	$BX \leftarrow BX + AX$

A instrução **ADD** quando executada altera o estado dos registradores de estados (*flags*) **AF**, **CF**, **OF**, **PF**, **SF** e **ZF**, possuindo como possibilidade de trabalho as operações sobre:

- ♦ registrador, registrador (para este tipo de ação usa 2 *bytes* de memória);
- ♦ memória, registrador (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ♦ registrador, memória (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ♦ registrador, constante (para este tipo de ação usa de 3 a 4 *bytes* de memória);
- ♦ memória, constante (para este tipo de ação usa de 3 a 6 *bytes* de memória).

Encerre a execução do programa **Enhanced DEBUG**.

4.4.2 - Subtração de valores hexadecimais

Carregue para a memória o programa **Enhanced DEBUG** e para a subtração de valores considere o fato de fornecer para os registradores gerais **AX** e **BX**, respectivamente, os valores hexadecimais **000A** e **0002**. Informe na linha de *prompt* do programa a instrução:

```
R AX 000A <Enter>
R BX 0002 <Enter>
R IP 0100 <Enter>
```

Ao lado do símbolo de dois-pontos informe o valor hexadecimal **000A** e acione a tecla **<Enter>**. Acione o comando **R** e observe o valor existente no registrador geral **AX**, como é indicado a seguir em negrito.

Acione o comando **R** e observe os valores existentes nos registradores gerais **AX** e **BX**, como é indicado a seguir em negrito.

```
AX=000A BX=0002 CX=0000 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PE NC
07E9:0100 C3 RET
```

O processo de subtração é conseguido com o par de códigos *opcodes* (valores hexadecimais) **2B** e **C3**, os quais devem ser armazenados, respectivamente, nos endereços de deslocamento **0100** e **0101**. Para efetuar a entrada de valores de endereço, utilize novamente o comando **E**:

```
E 0100 2B C3 <Enter>
```

Observe que os códigos hexadecimais **2B** e **C3** são responsáveis pela operação de subtração dos valores armazenados nos registradores gerais **AX** e **BX**. Veja a seguir o detalhamento completo da operação anterior.

Na sequência acione o comando **R** para que seja apresentado o estado atual dos registradores, como é mostrado em seguida:

```

AX=000A BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PE NC
07E9:0100 2BC3 SUB AX,BX

```

As informações dos registradores **AX**, **BX** e **IP** já são conhecidas. Atente especialmente para a terceira linha, na qual se encontra a instrução **SUB** (*subtract*), indicando que será executada a subtração dos valores armazenados nos registradores gerais **AX** e **BX** (**SUB AX,BX**), ou seja, ocorrerá a operação aritmética semelhante a $AX \leftarrow AX - BX$. Na sequência execute o comando **T** e observe o cálculo da subtração, como indicado a seguir:

```

AX=0008 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 0000 ADD [BX+SI],AL DS:0002=FF

```

Observe que o *flag* **PE** é mostrado como **PO** a partir da ação de uma subtração. Mais adiante maiores detalhes serão dados em relação aos registradores de estado (*flags*).

Seria conveniente agora fazer um cálculo de subtração para obter um resultado negativo. Entre para o registrador geral **BX** o valor hexadecimal **000A** e ajuste o endereço do registrador de deslocamento **IP** para **0100** com as instruções:

```

R BX 000A <Enter>
R IP 0100 <Enter>

```

Em seguida execute o comando **T** e será efetuada a subtração com resultado negativo. Observe atentamente o resultado **FFFE** apresentado no registrador **BX** e a mudança dos registradores de *flags* **PL** (anteriormente sinalizado como **NG**), **NA** (anteriormente sinalizado como **AC**) e **CY** (anteriormente sinalizado como **NC**) marcados em negrito e indicados a seguir:

```

AX=FFFE BX=000A CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI NG NZ AC PO CY
07E9:0102 0000 ADD [BX+SI],AL DS:000A=39

```

As alterações sobre os registradores de *flag* **PL** para **NG**, **NA** para **AC** e **NC** para **CY** ocorreram porque a subtração calculada gerou um resultado negativo. Neste momento, não se preocupe com esses detalhes.

A apresentação do valor hexadecimal **FFFE** no registrador geral **BX** equivale ao valor decimal **-2**. Lembre-se do estouro que ocorre para a representação de valores negativos.

Anteriormente foi usado o par de valores de *opcodes* **2B C3** para subtrair o valor do registrador geral **BX** do valor do registrador geral **AX**. É possível também realizar a operação inversa, ou mesmo operações com a porção menos significativa dos registradores gerais **AX** e **BX**. A Tabela 4.3 mostra as operações de subtração possíveis.

Tabela 4.3 - Opcode usual e alternativo para subtração

Operação Opcode		Subtração	
Usual	Alternativo	Registradores Gerais	Operação
2A C3	28 D8	AL, BL	$AL \leftarrow AL - BL$
2B C3	29 D8	AX, BX	$AX \leftarrow AX - BX$
2A D8	-	BL, AL	$BL \leftarrow BL - AL$
2B D8	-	BX, AX	$BX \leftarrow BX - AX$

Teste os *opcodes* apresentados na tabela para os valores atuais dos registradores gerais **AX** e **BX**, considerando inclusive as partes menos significativas dos respectivos registradores gerais mencionados, como foram os valores apresentados (valores de um *byte*).

A instrução **SUB** quando executada altera o estado dos registradores de estados (*flags*) **AF**, **CF**, **OF**, **PF**, **SF** e **ZF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, registrador (para este tipo de ação usa 2 bytes de memória);
- ◆ memória, registrador (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, memória (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, constante (para este tipo de ação usa de 3 a 4 bytes de memória);
- ◆ memória, constante (para este tipo de ação usa de 3 a 6 bytes de memória).

Encerre a execução do programa **Enhanced DEBUG**.

4.4.3 - Multiplicação de valores hexadecimais

Carregue para a memória o programa **Enhanced DEBUG** e para a multiplicação de valores considere o fato de fornecer para os registradores gerais **AX** e **BX**, respectivamente, os valores hexadecimais **0005** e **0003**. Informe na linha de *prompt* do programa as instruções:

```
R AX 0005 <Enter>
R BX 0003 <Enter>
R IP 0100 <Enter>
```

Acione o comando **R** e observe os valores existentes nos registradores gerais **AX** e **BX** e no registrador **IP** como é indicado a seguir em negrito.

```
AX=0005 BX=0003 CX=0000 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3 RET
```

O processo de multiplicação é conseguido com o par de códigos *opcodes* (valores hexadecimais) **F7** e **E3**, os quais devem ser armazenados, respectivamente, nos endereços de deslocamento **0100** e **0101**. Assim sendo, execute a instrução:

```
E 0100 F7 E3 <Enter>
```

Execute o comando **R** para que seja apresentado o estado atual dos registradores, como é mostrado em seguida:

```
AX=0005 BX=0003 CX=0000 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 F7E3 MUL BX
```

As informações dos registradores **AX**, **BX** e **IP** já são conhecidas. Atente especialmente para a terceira linha, na qual se encontra a definição da instrução **MUL** (*multiply*) apontando para o registrador geral **BX** (**MUL BX**).

Execute o comando **T** para que o resultado da multiplicação seja armazenado no registrador geral **AX** (neste caso será armazenado o valor hexadecimal **000F**) e para que as informações dos registradores sejam apresentadas.

```
AX=000F BX=0003 CX=0000 DX=0000 SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 0000 ADD [BX+SI],AL DS:0003=9F
```

No exemplo anterior foi realizada a multiplicação de dois valores numéricos hexadecimais muito pequenos. Mas imagine se a multiplicação ocorresse entre os valores bem maiores. Neste caso, deve-se considerar que a multiplicação de dois valores numéricos de 16 *bits* (*word*) pode resultar num valor numérico de resposta na casa de 32 *bits* (*double word*).

Por exemplo, informe para o registrador geral **AX** o valor hexadecimal **7D3C** (equivalente ao valor **32.060** em decimal) e para o registrador geral **BX** o valor hexadecimal **0100** (equivalente ao valor **256** em decimal). Depois execute o comando **R IP** e forneça como endereço de deslocamento inicial o valor hexadecimal **0100** de acordo com as instruções:


```
R AX 7D3C <Enter>
R BX 0100 <Enter>
R IP 0100 <Enter>
```

Na sequência acione o comando **R** para que seja apresentado o estado atual dos registradores, depois acione o comando **T** para que a multiplicação seja processada. Veja a seguir a sequência de resultados:

```
AX=7D3C BX=0100 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 F7E3          MUL     BX

AX=3C00 BX=0100 CX=0000 DX=007D SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 OV UP EI NG NZ AC PO CY
07E9:0102 0000          ADD     [BX+SI],AL          DS:0100=F7
```

Observe também a mudança ocorrida nos registradores de *flags* **OV** (anteriormente sinalizado como NV) e **CY** (anteriormente sinalizado como NC). Mais adiante serão dados mais detalhes sobre estas ocorrências.

O resultado da multiplicação é muito grande para ser armazenado apenas no registrador geral **AX**. A leitura do valor calculado deve ser feita na sequência de registradores gerais **DX** e **AX**, ou seja, **DX:AX**. O registrador geral **DX** (que armazena os 16 *bits* mais significativos - mais altos - do resultado de uma multiplicação) possui o valor hexadecimal **007D** e o registrador geral **AX** (que armazena os 16 *bits* menos significativos - mais baixos - do resultado de uma multiplicação) possui o valor hexadecimal **3C00**, o que resulta um valor hexadecimal de 32 *bits* representado como **007D3C00** (equivalente ao valor **8.207.360** decimal), ou seja, ocorre a operação aritmética semelhante a **DX:AX ← AX * BX**.

Anteriormente foi usado o par de valores de *opcodes* **F7 E3** para multiplicar o valor do registrador geral **BX** pelo valor do registrador geral **AX**. A Tabela 4.4 apresentada as operações de multiplicação possíveis.

Tabela 4.4 - Opcode usual para subtração

Operação Opcode	Multiplicação	
	Registradores Gerais	Operação
F6 E3	BL	AX ← AL * BL
F7 E3	BX	DX:AX ← AX * BX

Teste os *opcodes* apresentados na tabela para os valores atuais dos registradores gerais **AX** e **BX**, considerando inclusive as partes menos significativas dos respectivos registradores gerais mencionados, como foram os valores apresentados (valores de um *byte*). Caso queira entrar um valor que ocupe um *byte*, faça-o utilizando as posições numéricas da direita. Por exemplo, experimente multiplicar os valores **007F** e **0002** para os registradores gerais **AX** e **BX** e forneça o par de *opcodes* **F6 E3**, que resulta o valor **00FE** no registrador geral **AX**.

A instrução **MUL** quando executada altera o estado dos registradores de estados (*flags*) **CF** e **OF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória (para este tipo de ação usa de 2 a 4 *bytes* de memória).

Encerre a execução do programa **Enhanced DEBUG**.

4.4.4 - Divisão de valores hexadecimais

Carregue para a memória o programa **Enhanced DEBUG** e para a divisão de dois valores considere o fato de fornecer para os registradores gerais **AX** e **BX**, respectivamente, os valores hexadecimais **0009** e **0002**. Assim sendo, informe as instruções:

```
R AX 0009 <Enter>
R BX 0002 <Enter>
R IP 0100 <Enter>
```

Acione o comando **R** e observe os valores existentes nos registradores gerais **AX** e **BX**, como é indicado a seguir em negrito.

```
AX=0009 BX=0002 CX=0000 DX=007D SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI NG NZ NA PO NC
07E9:0100 C3 RET
```

O processo de divisão é conseguido com o par de códigos em linguagem de máquina (valores hexadecimais) **F7** e **F3**, os quais devem ser armazenados, respectivamente, nos endereços de deslocamento **0100** e **0101**. Desta forma, execute a instrução:

E **0100 F7 F3** <Enter>

Acione o comando **R** para que seja apresentado o estado atual dos registradores, como indicado em seguida:

```
AX=0009 BX=0002 CX=0000 DX=007D SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI NG NZ NA PO NC
07E9:0100 F7F3 DIV BX
```

As informações dos registradores **AX**, **BX** e **IP** já são conhecidas. Atente especialmente para a terceira linha, na qual se encontra a definição da instrução **DIV** (*divide*) apontando para o registrador geral **BX** (**DIV BX**).

Execute o comando **T** para que o resultado da divisão seja armazenado nos registradores gerais **AX** e **DX**, como segue:

```
AX=0004 BX=0002 CX=0000 DX=0001 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI NG NZ NA PO NC
07E9:0102 0000 ADD [BX+SI],AL DS:0002=FF
```

O resultado da divisão (quociente) é armazenado no registrador geral **AX**, enquanto o resultado do resto é armazenado no registrador geral **DX**. Assim como a multiplicação, a operação de divisão utiliza o par de registradores gerais **AX** e **DX**.

Anteriormente foi usado o par de valores de *opcodes* **F7 F3** para efetuar a multiplicação do valor do registrador geral **BX** pelo valor do registrador geral **AX**. A Tabela 4.5 apresentada as operações de divisão possíveis.

Teste os *opcodes* apresentados na tabela para os valores atuais dos registradores gerais **AX** e **BX**, considerando inclusive as partes menos significativas dos respectivos registros gerais mencionados, como foram os valores apresentados (valores de um byte). Caso queira entrar um valor que ocupe um byte, faça-o utilizando as posições numéricas da direita. Por exemplo, experimente multiplicar os valores hexadecimais **0009** e **0002** para os registradores gerais **AX** e **BX** e forneça o par de *opcodes* **F6 F3**, que resulta o valor hexadecimal 0104 no registrador geral **AX**, em que o valor **04** (lado menos significativo) é o quociente da operação e o valor **01** (lado mais significativo) é o resto da divisão.

Tabela 4.5 - Opcode usual para divisão

Operação Opcode	Divisão	
Usual	Registradores Gerais	Operação
F6 F3	BL	$AX \leftarrow AL / BL$ AL = quociente AH = resto
F7 F3	BX	$DX:AX \leftarrow AX / BX$ AX = quociente DX = resto

A instrução **DIV** quando executada não altera o estado dos registradores de estados (*flags*), possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória (para este tipo de ação usa de 2 a 4 *bytes* de memória).

Encerre a execução do programa **Enhanced DEBUG**.

Anotações

0D82:0100 B402	MOV	AH, 02
0D82:0102 B241	MOV	DL, 41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 5

APRESENTAÇÃO DE DADOS

*Este capítulo aborda as instruções para a exibição de dados no monitor de vídeo. Ele ensina a apresentar um único caractere, movimentação de dados, sequência de caracteres (apresentação de strings), valores binários, registradores de estado e aplicação simples de saltos condicionais. Novas instruções de uso das ferramentas **Enhanced DEBUG** e novos códigos de máquina para controlar a apresentação de dados no monitor de vídeo do computador em uso também são assuntos estudados no capítulo.*

5.1 - Apresentação de um caractere

Para apresentar apenas um caractere na tela do monitor de vídeo, é necessário utilizar um recurso chamado *interrupções*. Até este momento o uso das ferramentas **Enhanced DEBUG**, bem como a manipulação de dados numéricos, ocorreu somente em memória. Assim sendo, carregue para a memória o programa **Enhanced DEBUG**.

A interrupção responsável por controlar a apresentação de um caractere no console padrão de saída (que pode ser a tela do monitor de vídeo) é definida com o código hexadecimal **21** (essa interrupção permite também controlar a entrada de dados, como será visto no próximo capítulo). É importante ressaltar que a comunicação do microprocessador de um computador digital que é realizada com os seus periféricos ocorre por meio das interrupções e a interrupção **21** é usada para controlar o acesso aos periféricos de entrada e saída de dados.

A apresentação de um caractere é realizada com o uso dos registradores gerais **AX** e **DX**, em que **AX** armazena o código de controle para a exibição de apenas um caractere e **DX** armazena o código do caractere a ser apresentado. O registrador geral **DX** normalmente não deve ser usado para o armazenamento de valores para entrada ou saída de dados (como está aqui sendo exemplificado), pois sua finalidade é apontar para um endereço de memória que possui algum dado a ser operacionalizado pelo programa. Mas no presente momento, isto pode ser realizado por ser a apresentação de apenas um caractere. Para apresentar um *string* o procedimento de trabalho deve ser um pouco diferente como será ainda constatado.

Com o programa **Enhanced DEBUG** carregado informe ao registrador geral **AX** o valor hexadecimal **0200** (AH = 02 e AL = 00) que tem por finalidade permitir a escrita de um caractere no periférico padrão de saída; caso contrário, não é possível apresentar o caractere na tela do monitor de vídeo. Neste caso, informe na linha de *prompt* do programa a instrução:

R AX 0200 <Enter>

Na sequência informe para o registrador geral **DX** o valor hexadecimal **0040** (valor do caractere @ na tabela ASCII)¹, como demonstrado a seguir:

R DX 0040 <Enter>

¹ Para visualizar a tabela ASCII com os 128 códigos padrão definidos em decimal, binário e hexadecimal, consulte o apêndice A.

Acione o comando **R** e observe os valores existentes nos registradores gerais **AX** e **DX**, como é indicado a seguir em **negrito**.

```
AX=0200 BX=0000 CX=0000 DX=0040 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3 RET
```

É pertinente salientar que o valor **0200** do registrador geral **AX** será usado para indicar ao sistema operacional que deve ser apresentado um caractere no monitor de vídeo. O registrador de 16 *bits* **AX** é formado pelos registradores **AH** (*bits* mais significativos) e **AL** (*bits* menos significativos), de 8 *bits* cada um. Neste caso o valor **0200** possui a parte do valor hexadecimal **02** alocada no registrador **AH** e a parte **00** alocada no registrador **AL**. Se fosse colocado outro valor no registrador **AH** diferente de **02**, ele informaria ao sistema operacional para executar uma ação diferente de apresentar algo na tela. O registrador mais significativo **AH** caracteriza-se por ser utilizado para informar o código de controle padrão que efetua a saída de um caractere no monitor de vídeo. Neste caso, o valor hexadecimal **02** que é o código de função para uso da interrupção do monitor de vídeo.

Outro detalhe a ser observado com relação ao valor **0040** do registrador geral **DX** é que ele também possui sua parte mais significativa (registrador **DH**) e menos significativa (registrador **DL**), e o código hexadecimal **40** está sendo alocado na parte menos significativa do registrador geral **DX**.

A seguir é necessário informar nos endereços de deslocamento **0100** e **0101** o par de valores hexadecimais (*opcodes*) **CD** (que representa o comando de interrupção) e **21** (que representa a interrupção a ser acionada) que executam o código em linguagem de máquina que ativa o serviço de acionamento do monitor de vídeo para que um determinado caractere seja apresentado através do código de serviço **0200** armazenado no registrador geral **AX**. Informe o comando:

E **0100 CD 21** <Enter>

Na sequência verifique se o registrador de deslocamento **IP** está apontando para o endereço de deslocamento **0100**. Caso não esteja (o que é difícil, pois o programa **Enhanced DEBUG** inicia sua execução nesse endereço), execute a instrução **R IP 0100**.

Depois acione o comando **R** para que seja apresentado o estado atual dos registradores, como é mostrado em seguida. Atente para os pontos em **negrito** e verifique se esses valores estão presentes na tela do programa **Enhanced DEBUG**:

```
AX=0200 BX=0000 CX=0000 DX=0040 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 CD21 INT 21
```

Observe atentamente as informações dos registradores **AX**, **DX** e **IP** e a terceira linha, na qual se encontra a definição da instrução **INT** (*interrupt*) indicando o valor hexadecimal **21** (**INT 21**). À esquerda veja a definição do par de códigos de máquina (*opcode*) **CD21**.

A instrução **INT** quando executada altera o estado dos registradores de estados (*flags*) **TF** e **IF**, possuindo como possibilidade de trabalho a definição direta de um valor (constante) e para este tipo de ação consome-se 1 byte de memória.

O *opcode* **CD21** (**INT 21**) é uma instrução de interrupção do sistema operacional responsável por efetuar acesso aos periféricos de entrada e saída conectados ao computador. Neste caso particular será realizada uma apresentação de dado no monitor de vídeo devido ao valor **0200** armazenado no registrador **AX**, ou seja, o valor **02h** que está no **AH**. O valor **02h** caracteriza-se por representar a sub função responsável por indicar para a interrupção **21h** a ação de apresentação de um caractere no periférico de saída.

Será solicitada a execução dos dados em memória, mas desta vez não será usado o comando **T**, mas o comando **G** (*go*), porque o comando **T** executa linha a linha, e isso gera uma execução inconveniente, pois ao ser executada a interrupção (**INT 21**), ela desvia para uma sub-rotina interna, a qual dará muito trabalho para ser executada passo a passo. A interrupção acessada pelo código **INT 21**, busca no registrador menos significativo **DL** a informação a ser

apresentada no monitor de vídeo, que está sendo controlado pelo código **02h** armazenado no registrador mais significativo **AH**.

O comando **G** age de forma direta, necessitando apenas que seja informado até que ponto de um deslocamento de um segmento ele deve ir, ou seja, é necessário informar o endereço de parada para a execução. Neste caso (considerando que o registrador de deslocamento **IP** está no endereço de deslocamento **0100**) será solicitado que o comando **G** execute os endereços de deslocamento **0100**, **0101** e pare no endereço de deslocamento **0102**. Informe na linha de *prompt* do programa o comando:

G 0102 <Enter>

O programa apresenta o caractere **@** (note o ponto marcado em negrito) e indica a listagem de estado dos registradores, conforme a seguir:

```
@AX=0240 BX=0000 CX=0000 DX=0040 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 F726FC26          MUL      WORD PTR [26FC]          DS:26FC=0000
```

A primeira linha tem a indicação do caractere **@**, e os valores existentes no registrador geral **AX** e no registrador de deslocamento **IP**. O registrador geral **AX** assumiu no seu registrador geral menos significativo (**AL**) o código **40** correspondente ao caractere **@**.

A instrução **INT 21** (*opcode* **CD21**) executa a chamada de uma sub-rotina do sistema operacional que acessa o serviço de entrada ou de saída. Para a execução de sua ação é necessário passar e receber parâmetros por meio de registradores (WEBER, 2004, p. 247). No caso apresentado o registrador **AH** foi usado com o código **02** (código que informa ao sistema operacional que será usado o monitor de vídeo para a apresentação de um caractere) para indicar a função que deverá ser executada pela instrução **INT 21** e o registrador **AL** foi usado para receber a resposta da ação, ou seja, o valor **40** referente ao caractere a ser apresentado. Veja a seguir alguns exemplos de ações normalmente executadas com a instrução **INT 21**:

- ◆ Registrador **AH** com valor **01h** espera que um caractere seja informado via teclado, mostra o caractere na tela e retorna seu código ASCII no registrador **AL**;
- ◆ Registrador **AH** com valor **08h** espera que um caractere seja informado via teclado, não mostra o caractere na tela e retorna seu código ASCII no registrador **AL**;
- ◆ Registrador **AH** com valor **09h** mostra a cadeia de caracteres (finalizada com o caractere **\$** - valor **24h**) que se encontra apontada pelo par de registradores **DS:DX**.

Todos os programas em código de máquina desenvolvidos até aqui utilizaram apenas uma linha de código para suas ações. No entanto, um programa costuma ter bem mais do que isso.

Mesmo um programa de apresentação de um único caractere acaba precisando de mais de um código, pois é sempre bom após a execução de um programa solicitar o retorno do controle ao sistema operacional, ou seja, solicitar o encerramento do programa. Neste caso, deve-se utilizar a interrupção **20**, e é claro que ela deve ser colocada no final do código de programa.

Lembre-se de que o programa de apresentação do caractere **@** está utilizando os endereços de deslocamento **0100** e **0101** para a definição da interrupção **21**. A interrupção **20** deve ser colocada nos endereços de segmentos **0102** e **0103** com os códigos de máquina (*opcode*) **CD** e **20**. Informe o comando:

E 0102 CD 20 <Enter>

Na sequência verifique se o registrador de deslocamento **IP** está apontando para o endereço de deslocamento **0102**. Caso não esteja, execute o comando **R IP** e forneça como endereço de apontamento inicial o valor hexadecimal **0102**. Depois acione o comando **R** para que seja apresentado o estado atual dos registradores, como é mostrado em seguida:

```
AX=0240 BX=0000 CX=0000 DX=0040 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 CD20          INT      20
```

A terceira linha traz a definição do rótulo **INT** indicando o valor hexadecimal **20** (**INT 20**) e também à esquerda a definição do par de códigos de máquina **CD20**.

Para executar o programa que agora tem duas instruções, ajuste o registrador de deslocamento **IP** para o deslocamento **0100** e execute o comando **G** sem a indicação do endereço de término, pois como foi definida a interrupção **20**, ela informa o término da execução do programa, retornando o controle de operação do computador para o sistema operacional sem necessidade de indicar em que segmento deve a execução parar.

```
R IP 0100 <Enter>
G          <Enter>
```

```
@
Program terminated (0000)
```

Assim que o comando **G** é acionado, o programa apresenta o caractere **@**, em seguida sinaliza o término da sua ação com a mensagem "**Program terminated (0000)**". É apresentada a sequência de execução dos comandos e do programa a partir do ajuste do registrador de deslocamento **IP**.

Para ver uma listagem do código de programa criado, que agora possui duas instruções, será utilizado o comando **U** (*unassemble*) que faz a listagem do programa de um ponto inicial até um ponto definido como intervalo de deslocamentos. Informe no *prompt* do programa o comando:

```
U 0100 0102 <Enter>
```

O programa apresenta algo semelhante a:

```
07E9:0100 CD21          INT      21
07E9:0102 CD20          INT      20
```

A listagem anterior apresenta apenas os códigos de instrução situados nos deslocamentos **0100** e **0102**. Os deslocamentos **0101** e **0103** foram usados apenas para auxiliar a entrada dos pares de códigos de máquina no programa que controlam respectivamente a apresentação do caractere e o encerramento do programa. O primeiro bloco de números separados por dois pontos mostram a posição de memória onde uma instrução encontra-se definida, o segundo bloco apresenta o código operacional (*opcode*) e o terceiro bloco apresenta a instrução escrita em linguagem *Assembly*. Note que a instrução **INT 21** está na posição de memória **0100** e a instrução **INT 20** está situada duas posições a frente, ou seja, na posição **0102**. Isto mostra que a instrução **INT 20** está ocupando dois *bytes* de memória e certamente uma próxima instrução após a interrupção **INT 20** seria então posicionada na posição de memória **0104**. A distância de ocupação de memória usada por uma linha de instrução pode ser de um a seis *bytes*, dependendo da instrução em uso. O apêndice B apresenta este tipo de informação.

O uso de código de máquina pelo programa **Enhanced DEBUG** pode ser simplificado com o fornecimento direto de códigos de programas na linguagem *Assembly* para o modo de montagem (*assembler*).

Para testar essa nova possibilidade, saia dos programas **Enhanced DEBUG** com o comando **Q** e retorne ao programa imediatamente. Essa ação fará com que todos os registradores utilizados sejam zerados.

Para o próximo teste forneça para o registrador geral **AX** e para o registrador geral **DX**, respectivamente, os valores hexadecimais **0200** e **0046** (valor do caractere **F** na tabela ASCII).

```
R AX 0200 <Enter>
R DX 0046 <Enter>
```

Acione o comando **R** e observe os valores existentes nos registradores gerais **AX** e **DX**, como é indicado a seguir em negrito.


```
AX=0200 BX=0000 CX=0000 DX=0046 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3 RET
```

A partir do ponto em que os valores de controle do programa estão armazenados nos registradores, é necessário informar as interrupções **21** e **20**, as quais serão feitas com códigos em *Assembly* e não em código de máquina, por meio de *opcodes*. Acione o comando **A** (*assemble*) fornecendo como ponto inicial de armazenamento de código o deslocamento **0100**, como indicado a seguir:

A 0100 <Enter>

Automaticamente será apresentada a linha (segmento:deslocamento) em que o código deve ser definido. É sempre bom lembrar que o endereço de segmento é um valor escolhido pelos programas **Enhanced DEBUG**. Nesta etapa forneça as seguintes linhas de código:

```
07E9:0100 INT 21 <Enter>
07E9:0102 INT 20 <Enter>
07E9:0104 <Enter>
```

Ao informar a primeira linha de código e acionar a tecla <Enter>, é apresentado automaticamente o segundo endereço de deslocamento. Nesse ponto, ao entrar a segunda linha de código e acionar <Enter>, passa-se para o próximo segmento, que se nada for entrado e for acionada a tecla <Enter>, o modo de trabalho do comando **A** é encerrado. A sequência de comandos apresenta um exemplo visual da ocorrência aqui descrita:

Observação

A entrada de instruções no programa **Enhanced DEBUG** pode ser feita tanto com letras minúsculas como maiúsculas. O programa sempre converte os caracteres no seu formato maiúsculo quando da apresentação das instruções listadas.

Para fazer um teste de execução do programa, utilize o comando **G** sem indicar nenhum endereço de deslocamento inicial. Veja o resultado que será apresentado:

```
F
Program terminated (0000)
```

O uso de comandos na forma mnemônica é bem mais fácil de que comandos em código de máquina. A partir deste ponto os próximos programas serão criados com esse recurso.

5.2 - Movimentação de dados

Os programas feitos até aqui utilizam as informações armazenadas nos registradores de forma manual. No entanto, é possível controlar essas ações por meio de um programa de forma mais automática.

Para conseguir esse intento, é necessário utilizar uma instrução *Assembly* denominada **MOV** (*move*)², que faz a movimentação de dados (*byte* ou *word*) nos registradores e entre os registradores. No entanto, é necessário levar em consideração que não é possível movimentar dados de uma posição da memória para outra posição de forma direta; para este tipo de ação, é sempre necessário fazer uso de um registrador. O dado posicionado em um local de origem deve ser posicionado primeiramente em um registrador, para depois, ser movimentado do registrador para um local de destino.

² Lembre-se de que já foram transmitidas noções sobre as instruções *ADD* (addition), *SUB* (subtract), *MUL* (multiply), *DIV* (divide) e *INT* (interrupt).

A instrução **MOV** quando executada não altera o estado dos registradores de estados (*flags*), possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, registrador (para este tipo de ação usa 2 bytes de memória);
- ◆ memória, registrador (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, memória (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, constante (para este tipo de ação usa de 2 a 3 bytes de memória);
- ◆ memória, constante (para este tipo de ação usa de 3 a 6 bytes de memória);
- ◆ segmentoregistro, registrador (para este tipo de ação usa 2 bytes de memória);
- ◆ segmentoregistro, memória (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ memória, segmentoregistro (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, segmentoregistro (para este tipo de ação usa 2 bytes de memória).

Para assimilar um pouco de intimidade com essa nova instrução, serão executadas algumas movimentações de valores entre os registradores. Para começar, informe para o registrador geral **AX** o valor **1122** e para o registrador **DX** o valor **AABB**.

```
R AX 1122 <Enter>
R DX AABB <Enter>
```

Verifique com o comando **R** se os valores estão como os seguintes:

```
AX=1122 BX=0000 CX=0000 DX=AABB SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 CD21 INT 21
```

Como exemplo será realizada a movimentação da parte menos significativa do registrador **DX** para a parte mais significativa do operador **AX**. Na linha de *prompt* dos programas **Enhanced DEBUG** execute as instruções:

```
A 0100 <Enter>
```

Informe na linha a instrução **MOV AH,DL** e acione a tecla **<Enter>** por duas vezes.

Observe na sequência a aparência que deve ter a tela do programa após as execuções anteriores:

```
07E9:0100 MOV AH,DL <Enter>
07E9:0102 <Enter>
```

Caso queira poderá fazer uso do par de *opcodes* **8A E2** para os endereços **0100** e **0101** informados com o uso do comando **E** a partir da instrução **E 0100 8A E2**. Execute em seguida o comando **T** para visualizar o estado atual dos registradores durante a execução do programa, como indicado a seguir:

```
AX=BB22 BX=0000 CX=0000 DX=AABB SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 CD20 INT 20
```

O valor hexadecimal **BB** armazenado na parte menos significativa do registrador **DL** pertencente ao registrador geral **DX** foi copiado (foi atribuído ou implicado) para o registrador mais significativo do registrador geral **AX**.

Na sequência, atualize o valor do registrador de deslocamento **IP** para **0100** e execute o comando **A 0100** como indicado a seguir e informando as ações grafadas em negrito seguintes:

```
R IP 0100 <Enter>
A 0100    <Enter>
```

```
07E9:0100 MOV AX,DX <Enter>
07E9:0102 <Enter>
```

Caso queira poderá fazer uso do par de *opcodes* **8B C2** para os endereços **0100** e **0101** informados com o uso do comando **E**. Execute o comando **T** para visualizar o estado atual dos registradores durante a execução do programa, como indicado a seguir:

```
AX=AABB BX=0000 CX=0000 DX=AABB SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 CD20                INT     20
```

O valor hexadecimal **AABB** armazenado no registrador geral **DX** foi copiado por inteiro para o registrador geral **AX**.

Na execução anterior foi copiado entre os registradores gerais **AX** e **DX** o conteúdo armazenado na estrutura *word*. No entanto, no exemplo anterior ao último foi copiado entre os registradores **AH** e **DL** o conteúdo armazenado em um *byte*.

É importante estar atento à instrução **MOV** que opera sempre entre pares da mesma estrutura. Não é possível, por exemplo, executar uma linha de código como **MOV AX,DL**.

Após essa noção básica execute o comando **A 0100** e informe as linhas de código indicadas em negrito a seguir:

```
07E9:0100 MOV AH,02 <Enter>
07E9:0102 MOV DL,41 <Enter>
07E9:0104 INT 21    <Enter>
07E9:0106 INT 20    <Enter>
07E9:0108 <Enter>
```

O código anterior (**MOV AH,02**) movimenta o valor hexadecimal **02** (serviço de apresentação em vídeo realizado pelo sistema operacional) para o registrador **AH** (parte mais significativa do registrador geral **AX**) e movimenta também (**MOV DL,41**) o valor hexadecimal **41** (código que representa o caractere maiúsculo **A**) para o registrador **DL** (registrador menos significativo do registrador geral **DX**). Esse tipo de ação não é possível realizar manualmente quando utilizar o comando **R** para a definição de valores nos registradores, que somente aceita valores no tamanho de um *word*. As linhas **INT 21** e **INT 20** são usadas para acionar, respectivamente, as interrupções **21** (comunicação com o monitor de vídeo) e **20** (sinalização de encerramento de programa pelo sistema operacional).

Caso queira a partir do uso do comando **E** poderá fazer a entrada do programa com uso dos conjuntos de *opcodes* **B4 02** para os endereços **0100** e **0101**, **B2 41** para os endereços **0102** e **0103**, **CD 21** para os endereços **0104** e **0105** e por fim **CD 20** para os endereços **0106** e **0107**.

Com o comando **R IP** posicione o endereço de deslocamento em **0100** e execute o comando **G** para visualizar a apresentação do caractere **A** na tela, como é mostrado a seguir:

```
A
Program terminated (0000)
```

O último programa composto por um conjunto de quatro linhas de código já pode ser gravado. No entanto, a gravação de um programa deve seguir alguns passos básicos de ação:

1. É necessário saber o número de *bytes* que o programa ocupará em disco. Pegue o endereço de deslocamento da última instrução do programa e da primeira instrução (que deve sempre ser iniciada por **0100**) e calcule a diferença entre esses valores. No programa anterior a primeira linha após a última instrução é a de valor de deslocamento **0108**. Neste caso, é fácil saber o tamanho, porém em programas maiores sugere-se usar o comando **H** para obter o valor correto. Neste caso poder-se-ia usar o comando como **H 0108 0100** que resultaria nos valores **0208** e **0008**, sendo a segunda resposta o valor da diferença.
2. Com o valor do número de *bytes* em mãos é necessário informá-lo aos registradores gerais **BX** e **CX**. O registrador geral **BX** somente será usado no caso de o valor do tamanho em *bytes* não caber no registrador geral **CX**.

Com o comando **R CX** informe o valor **0008**. Este valor determinará para o registrado **CX** a quantidade de bytes que deverá ser contada para a execução da gravação.

3. Após executar os procedimentos anteriores, use o comando **N** (*name*) para atribuir um nome ao programa. Para esse teste forneça o nome **MOSTRA.COM**. Os programas gravados com a ferramenta **Enhanced DEBUG** devem ter a extensão de identificação **.COM**. Utilize a linha de código **N MOSTRA.COM**. um detalhe importante nesta ação o nome do arquivo deve respeitar o padrão **8.3** (8 caracteres máximo para o nome e 3 caracteres máximo para o nome da extensão).
4. Em seguida é necessário gravar o programa com o comando **W** (*write*). Basta apenas acionar **W** e aparece a mensagem de que o arquivo foi gravado. Essa mensagem informa também o número de *bytes* usados na gravação.

Agora que o programa foi criado, saia do programa **Enhanced DEBUG**. No *prompt* do sistema operacional execute o programa **MOSTRA.COM** e observe a apresentação da letra **A**.

Volte ao programa **Enhanced DEBUG** e execute novamente o comando **N MOSTRA.COM** para associá-lo à memória. Depois execute o comando **L** (*load*) para que o programa associado seja carregado na memória.

Execute o comando **U** (*unassemble*) para que o código *decompilado* seja mostrado com a instrução **U 0100**, a qual mostrará uma listagem semelhante à:

0727:0100	B402	MOV	AH,02
0727:0102	B241	MOV	DL,41
0727:0104	CD21	INT	21
0727:0106	CD20	INT	20
0727:0108	0000	ADD	[BX+SI],AL
0727:010A	0000	ADD	[BX+SI],AL
0727:010C	0000	ADD	[BX+SI],AL
0727:010E	0000	ADD	[BX+SI],AL
0727:0110	0000	ADD	[BX+SI],AL
0727:0112	0000	ADD	[BX+SI],AL
0727:0114	0000	ADD	[BX+SI],AL
0727:0116	0000	ADD	[BX+SI],AL
0727:0118	0000	ADD	[BX+SI],AL
0727:011A	0000	ADD	[BX+SI],AL
0727:011C	0000	ADD	[BX+SI],AL
0727:011E	0000	ADD	[BX+SI],AL

Leve em consideração apenas as quatro primeiras linhas, as quais estão grafadas em negrito e correspondem ao programa montado anteriormente. As demais linhas apresentam dados que estão dispostos na memória do computador, mas não interferem no programa criado, os quais poderão ser diferentes em seu computador. Depois saia dos programas **Enhanced DEBUG**.

Na listagem apresentada são indicadas algumas informações importantes, tais como a posição da instrução no endereço de memória (por exemplo, do endereço **0727:0100** até **0727:0106**, onde cada uma das posições de memória é formada pelo par **SEGMENTO:DESLOCAMENTO**), a instrução em código de máquina representada pelo valor hexadecimal (como, por exemplo, **B402** existente na primeira linha de código) ao lado do endereço de memória é o código definido em *Assembly* em modo *opcode* (**B402** corresponde ao uso da instrução **MOV AH,02**, encontrada na primeira linha de código).

No pequeno programa apresentado nota-se que no endereço de memória **0727** a um pulo nos valores associados ao **DESLOCAMENTO** de dois em dois (**0100**, **0102**, **0104** e **0106**). Esse salto é calculado automaticamente pelo microprocessador e se refere a quantidade de *bytes* que cada instrução ocupa na memória quando usada. Por exemplo, a instrução **MOV AH, 02** para fazer a movimentação do valor **02** (constante) para o registrador **AH** ocupa 2 *bytes* de memória. Toda vez que se faz uma operação do tipo **MOV registrador, constante** usa-se de 2 a 3 *bytes* de memória.

Encerre a execução do programa **Enhanced DEBUG**.

5.3 - Apresentar sequência de caracteres

O desafio agora é escrever uma linha de texto na tela do monitor de vídeo. É necessário saber que se deseja escrever e codificar cada caractere do *string* em seu equivalente hexadecimal. Imagine a necessidade de apresentar a mensagem “Alo mundo!” (sem nenhuma acentuação, se existir).

Para efetuar a conversão da mensagem “Alo mundo!”, é necessário estar com a tabela ASCII em mãos para saber quais são os valores numéricos hexadecimais de cada caractere da frase a ser escrita. No caso da mensagem em uso os valores hexadecimais são os apresentados na Tabela 5.1.

Tabela 5.1 - Valores ASCII da mensagem “Alo mundo!”

A	l	o		M	u	n	d	o	!
41	6C	6F	20	6D	75	6E	64	6F	21

É preciso colocar os códigos do *string* na memória. Carregue um dos programas **Enhanced DEBUG**, execute o comando **E** e informe o endereço de deslocamento a partir de **0200**.

Informe cada *byte* da tabela anterior, separando-os por espaços em branco. Ao final acrescente o código hexadecimal **24** que representa o símbolo \$, o qual define para os programas **Enhanced DEBUG** o fim do *string*. A seguir são apresentados os dados após a entrada dos valores hexadecimais da mensagem “Alo mundo!”.

E 0200 41 6C 6F 20 6D 75 6E 64 6F 21 24

É necessário informar para o registrador geral **DX** o início do endereço de deslocamento em que se encontra a sequência de caracteres. Neste caso, o endereço é **0200**. É necessário também definir o valor hexadecimal **09** (função para apresentação de uma sequência de caracteres no monitor de vídeo) no registrador mais significativo **AH**, para que um *string* seja impresso. Execute o comando **A 0100**.

```
07E9:0100 MOV AH,09    <Enter>
07E9:0102 MOV DX,0200  <Enter>
07E9:0105 INT 21      <Enter>
07E9:0107 INT 20      <Enter>
07E9:0109             <Enter>
```

No código anterior não está sendo utilizado o código **02** para o registrador mais significativo **AH**, mas sim o código **09** quando do uso da instrução **MOV AH,09** (caso prefira poderá usar o *opcode* **B4 09**). O código **02** aciona o recurso de impressão de apenas um caractere, enquanto o código **09** aciona o serviço de apresentação de uma sequência de caracteres.

A instrução **MOV DX,0200** (*opcode* **BA 00 02**) para ser usada ocupa três posições de memória, pois armazena no registrador **DX** o endereço de memória onde se encontra o *string* a ser apresentado. O registrador geral **DX** é normalmente usado para apontar o endereço onde certo dado se encontra na memória.

Execute o comando **G** para que a mensagem seja apresentada, como é indicado a seguir:

```
Alo mundo!
Program terminated (0000)
```

O comando **U** permite visualizar a sequência de instruções de um programa. No entanto, é possível também visualizar as informações referentes ao armazenamento dos códigos hexadecimais dos caracteres da mensagem. Utilize o comando **D** (*dump*) com o endereço do deslocamento desejado. Neste caso, acione o comando:

D 0200 <Enter>

Observe os detalhes apresentados na sequência:

07E9:0200	41 6C 6F 20 6D 75 6E 64-6F 21 24	00 00 00 00 00 00	Alo mundo!\$.
07E9:0210	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	
07E9:0220	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	
07E9:0230	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	
07E9:0240	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	
07E9:0250	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	
07E9:0260	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	
07E9:0270	00 00 00 00 00 00 00 00-00	00 00 00 00 00 00	

Observe o trecho em negrito. Os demais dados são restos de conteúdo da memória e certamente serão apresentados de forma diferente no computador do leitor. Perceba a definição do caractere \$ indicando o fim da mensagem. A informação apresentada é composta das informações de segmento e de deslocamento (segmento:deslocamento) e de 16 bytes hexadecimais com os códigos referentes aos caracteres que também são apresentados no extremo direito da relação.

Na parte extrema à direita da relação há também alguns pontos (.). Isso ocorre quando o caractere é um ponto ou outro caractere especial, ou seja, o comando D apresenta apenas 96 códigos de caracteres dos 256 códigos encontrados na tabela ASCII. Os outros 160 códigos são apresentados como pontos.

Agora o programa de escrita de mensagem deve ser gravado. Este programa ocupa um espaço de memória bem maior que o programa anterior, pois a sequência de caracteres está armazenada a partir do endereço de deslocamento **0200**. Para verificar essa informação, execute o comando:

U 0200 <Enter>

Observe os detalhes apresentados na listagem a seguir:

07E9:0200	41	INC	CX
07E9:0201	6C	INSB	
07E9:0202	6F	OUTSW	
07E9:0203	206D75	AND	[DI+75],CH
07E9:0206	6E	OUTSB	
07E9:0207	646F	FS:OUTSW	
07E9:0209	2124	AND	[SI],SP
07E9:020B	0000	ADD	[BX+SI],AL
07E9:020D	0000	ADD	[BX+SI],AL
07E9:0211	0000	ADD	[BX+SI],AL
07E9:0213	0000	ADD	[BX+SI],AL
07E9:0215	0000	ADD	[BX+SI],AL
07E9:0217	0000	ADD	[BX+SI],AL
07E9:0219	0000	ADD	[BX+SI],AL
07E9:021B	0000	ADD	[BX+SI],AL
07E9:021D	0000	ADD	[BX+SI],AL
07E9:021F	0000	ADD	[BX+SI],AL

O trecho marcado em negrito corresponde exatamente à sequência de caracteres definida para a escrita do *string* entre os endereços **0200** e **0209**. Considere a primeira posição de memória após a indicação do endereço **0209**, ou seja, considere o valor do endereço **020B**.

A sequência de caracteres que forma o *string* "Alô mundo!" é visível na definição dos valores de códigos de máquina existentes após a indicação dos endereços de memória, como mostra o trecho em negrito de código a seguir.

07E9:0200	41	INC	CX
07E9:0201	6C	INSB	
07E9:0202	6F	OUTSW	
07E9:0203	206D75	AND	[DI+75],CH
07E9:0206	6E	OUTSB	
07E9:0207	646F	FS:OUTSW	
07E9:0209	2124	AND	[SI],SP

Não se preocupe com os mnêmicos apresentados do lado direito dos valores hexadecimais que forma o texto do *string*. Em seguida calcule a diferença entre os valores final e inicial para determinar o tamanho do programa em memória. Note que o programa começa no endereço de memória **0100** e faz uso da sequência de caracteres definida a partir do endereço **0200**, a qual se estende até o endereço **020B**. Desta forma, basta fazer o cálculo com a instrução:

H 020B 0100 <Enter>

Os resultados apresentados são **030B** para a soma e **010B** para a diferença. Considerando o valor de diferença **010B** coloque-o no registrador **CX** com a instrução:

R CX 010B <Enter>

Assim sendo, execute o comando **R** e atente para os pontos marcados em negrito como mostrado a seguir:

```
AX=0000 BX=0000 CX=010B DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 B409          MOV     AH,09
```

Execute o comando **N MENSA.COM** e depois o comando **W**. Saia de um dos programas **Enhanced DEBUG** e execute o programa no *prompt* do sistema operacional para que a mensagem seja apresentada.

Há outra forma um pouco mais simples de definir sequências de caracteres na memória. Para tanto, execute a instrução:

E 0200 "Olá, leitor" 24 <Enter>

```
A 0100                                <Enter>
07E9:0100 MOV AH,09                    <Enter>
07E9:0102 MOV DX,0200                  <Enter>
07E9:0105 INT 21                        <Enter>
07E9:0107 INT 20                        <Enter>
07E9:0109                                <Enter>
```

Observe que sequência de caracteres está definida entre aspas. É importante manter ao final o código hexadecimal **24** para a definição do caractere de controle \$. Com o comando **G** execute o programa e observe a apresentação da mensagem **Olá, leitor**.

5.4 - Apresentar valores binários

A seguir serão criados alguns programas que apresentam valores numéricos binários. Para este contexto são indicados novos comandos de operação.

Mas antes de apresentar os valores numéricos, é necessário considerar outros detalhes relacionados ao estouro da capacidade de cálculo. Por exemplo, carregue novamente um dos programas **Enhanced DEBUG** e para o registrador geral **AX** forneça o valor hexadecimal **FFFF**, depois forneça para o registrador geral **BX** o valor hexadecimal **0001** a partir das instruções:

```
R AX FFFF <Enter>
R BX 0001 <Enter>
```

Execute o comando **R** para ver a listagem da posição de memória e observe se a primeira linha está semelhante à indicação em negrito a seguir:

```
AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
```


Na segunda linha da encontra-se um conjunto de oito pares de registradores de estado (*flags*), como indicado em negrito:

```
AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3 RET
```

Direcione a sua atenção para o último par de valores que deve estar sinalizado com as letras **NC** - *No Carry* (dependendo da ocorrência em uso, esse par de valores poderá estar identificado com as letras **CY** - *Carry Yes*). Os valores **NC** e **CY** são as respostas geradas para o estado de operação do registrador **CF** (*Carry Flag*). O registrador **CF** será sinalizado com o valor **CY** quando determinado cálculo gerar um estouro da capacidade máxima de representação numérica na memória.

Os registradores gerais **AX** e **BX** possuem, respectivamente, os valores hexadecimais **FFFF** e **0001**. Assim, execute a instrução:

```
E 0100 01 D8 <Enter>
```

Os valores **01** e **D8** são os *opcodes* utilizados para realizar a adição do valor do registrador geral **BX** sobre o valor do registrador geral **AX**. Com a certeza de que o registrador de deslocamento **IP** aponta para o endereço **0100**, execute o comando **T** e observe o que ocorreu.

```
AX=0000 BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL ZR AC PE CY
07E9:0102 F726FC26 MUL WORD PTR [26FC] DS:26FC=0000
```

Os valores apresentados nos registradores gerais **AX** e **BX** são, respectivamente, **0000** e **0001**. O último valor do registrador de estado que era **NC** agora é **CY**. Se prestar atenção, verá que outros três registradores de estado também foram afetados (**NZ** para **ZR**, **NA** para **AC** e **PO** para **PE**). Mas no momento, interessa apenas fazer um rápido estudo do último valor do registrador de estado, pois com ele será realizada a apresentação de valores binários.

O que ocorreu? Já é sabido que os registradores gerais operam com valores numéricos de 16 *bits*, e quando um valor ultrapassa esta capacidade, ocorre automaticamente um estouro. Foi exatamente isso que ocorreu ao tentar somar o valor hexadecimal **0001** com o valor hexadecimal **FFFF** (que é o maior possível).

Ocorreu um estouro no valor do registrador **AX** que deveria ser **10000** e apresenta apenas o valor **0000**, ou seja, são considerados apenas os quatro dígitos da direita. O valor 1 (primeiro dígito à esquerda "desprezado") gerou um estouro da capacidade numérica de armazenamento. Ao ocorrer o estouro, o décimo quinto *bit* (está em uso um valor hexadecimal de 16 *bits*. Se fosse um valor de 8 *bits*, seria então considerado o sétimo *bit*), o registrador de *Carry*, é automaticamente "setado" em 1, indicando a ocorrência do estouro, pois o registrador de estado *Carry* indica o conceito de vai-um quando ocorre um estouro, e neste caso esse registrador passa do estado **NC** (sem estouro, quando está com valor binário zero) para o estado **CY** (com estouro, passando a possuir o binário um).

Se neste exato momento for executada a soma do valor hexadecimal **0001** no registrador geral **AX**, o registrador de estado *Carry* volta ao seu estado original sinalizado como **NC**. Posicione o registrador **IP** no endereço **0100** e execute o comando **T**, que apresenta a indicação do registrador de estado *Carry*, como exibido em negrito:

```
AX=0001 BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 F726FC26 MUL WORD PTR [26FC] DS:26FC=0000
```

A primeira forma de apresentação numérica será de valores binários. Neste caso, serão utilizados apenas os recursos do registrador de estado *Carry* para a finalidade de representar um valor numérico na forma binária. Considere a apresentação do valor hexadecimal **00AA** (valor 170 em decimal) na forma binária **10101010**. Forneça para o registrador geral **AX** o valor hexadecimal **0000**, para o registrador **BX** o valor hexadecimal **00AA** e para o registrador **IP** o valor hexadecimal **0100**. Acione o comando **R** e veja o resultado conforme a seguir:


```

AX=0000 BX=00AA CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 01D8          ADD     AX,BX

```

Para conseguir o intento de escrever um número em formato binário, é necessário utilizar o comando **RCL** (*Rotate through Carry Left*), que tem por finalidade mover para a esquerda o ponteiro de *bit* dentro do registrador de estado *Carry Flag* (**CF**), percorrendo todos os *bits* existentes, a partir da primeira posição (posição zero) até a última (que pode ser a posição sete para a varredura de um *byte* ou a posição dezesseis para a varredura de um *word*). Entre com as instruções:

```

A 0100          <Enter>
07E9:0100 RCL BL, 1 <Enter>
07E9:0102          <Enter>

```

A instrução **RCL BL, 1** tem por finalidade somar o valor 1 à parte baixa do registrador geral **BX**.

O comando **RCL** quando executada altera o estado dos registradores de estados (*flags*) **CF** e **OF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, 1 (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, 1 (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ registrador, CL (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, CL (para este tipo de ação usa de 2 a 4 *bytes* de memória).

Observação

A instrução **RCL** também pode operar a movimentação do registrador de estado *Carry Flag* para dados do tipo *word*.

Isso fará com que o valor hexadecimal **00AA** (em modo binário **10101010**) seja somado com o valor 1. Toda vez que ocorrer internamente um estouro da soma do valor 1 do registrador menos significativo **BL** com o valor armazenado no registrador geral **BX**, este sinalizará o registrador de estado *Carry Flag* com o valor **CY** (1) ou **NC** (0).

A primeira ação ocorre no *bit* zero (primeiro *bit* da direita para a esquerda). Quando for solicitada uma segunda ação do comando **RCL**, ele passa a operação para o segundo *bit*. É necessário repetir a ação por oito vezes (*byte*) para conseguir obter os oito valores binários relacionados ao valor hexadecimal **00AA**. Tendo uma ideia do processo de execução, acompanhe cada passo indicado a seguir:

1. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 1 numa folha de papel, pois o registrador *Carry Flag* está indicando **CY**.
2. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 0 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **NC**.
3. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 1 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **CY**.
4. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 0 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **NC**.
5. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 1 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **CY**.
6. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 0 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **NC**.
7. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 1 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **CY**.
8. Atualize o registrador **IP** com o valor de endereço **0100**, execute o comando **T** e anote o valor 0 numa folha de papel ao lado do valor anteriormente obtido, pois o registrador de estado *Carry Flag* está indicando **NC**.

Observe na Tabela 4.2 o resumo da anotação dos valores de *Carry* durante os oito passos com o comando **T**. Ao executar as ações indicadas, confira cada etapa do processo com a Tabela 5.2.

Tabela 5.2 - Valores de *carry* após uso do comando **T**

Valor Inicial do Registrador BX: 00AA (ou apenas AAh)					
Ação	IP	Execução	Carry Flag	Anotação	(BX) BL
1	R IP 0100	T	CY	1	0054
2	R IP 0100	T	NC	0	00A9
3	R IP 0100	T	CY	1	0052
4	R IP 0100	T	NC	0	00A5
5	R IP 0100	T	CY	1	004A
6	R IP 0100	T	NC	0	0095
7	R IP 0100	T	CY	1	002A
8	R IP 0100	T	NC	0	0055

O valor do registrador geral **BL** está sendo indicado na tabela anterior para facilitar o acompanhamento de cada passo. Em nenhum momento o valor existente nesse registrador será usado. Será usada apenas a informação **CY** e **NC** do registrador de estado *Carry Flag*.

Se por curiosidade forem executados mais uma vez os comandos descritos na tabela anterior, obter-se-á o valor **00AA** no registrador geral **BX**, provando que o comando **RCL** efetuou a varredura *bit a bit* sempre no sentido esquerdo.

Até o presente momento foram informadas as instruções para movimentação dos *bits* pela instrução **RCL**, sendo apenas uma parte do processo de apresentação de um valor hexadecimal na forma binária.

É necessário ainda fazer com que o registrador de estado *Carry Flag* quando "setado", ou seja, sinalizado (**CY**), forneça para o programa condição de escrever o valor **1** (um) e quando não estiver "setado" (**NC**), forneça condição de escrever **0** (zero). Além disso, é necessário desenvolver um laço de repetição para automatizar o processo de tradução de valor hexadecimal em valor binário.

O laço de repetição será construído com o comando **LOOP** que determina o número de vezes a ser executado o trecho de código subordinado ao laço. Essa instrução utiliza o registrador geral **CX** para armazenar o valor do contador, ocupa 2 *bytes* de memória e não afeta os registradores de estado. No exemplo apresentado anteriormente o programa deve executar o trecho de conversão definido no laço oito vezes.

Primeiramente saia do programa **Enhanced DEBUG** em uso e faça o seu carregamento para a memória novamente a fim de inicializa-lo de forma zerada. Acrescente ao registrador geral **BX** o valor hexadecimal **00AA**, depois execute o comando **A 0100** e informe as linhas seguintes:

```
07E9:0100 MOV  CX,0008 <Enter>
07E9:0103 RCL  BL,1    <Enter>
07E9:0105 LOOP 0103   <Enter>
07E9:0107 INT  20     <Enter>
07E9:0109                <Enter>
```

A primeira linha **07E9:0100** faz a colocação do valor hexadecimal **0008** no registrador geral **CX**. A segunda linha **07E9:0103** movimenta um *bit* à esquerda no registrador de estado *Carry Flag*. A terceira linha **07E9:0105** executa o laço desviando a execução do programa para a linha de deslocamento **0103**. Esse processo será executado oito vezes. A cada execução do comando **LOOP** (que será internamente considerado como comando **LOOPW** para o programa **Enhanced DEBUG**) será realizado um decréscimo de **0001** até que o registrador geral **CX** chegue ao valor **0000**, quando o fluxo de execução do programa será desviado para a quarta linha do programa (**07E9:0107**).

Para realizar um teste de execução, utilize o comando **T** que possibilita a execução de um programa passo a passo até que seja apresentada a mensagem de término do programa. A seguir é apresentada a sequência completa de execução dessa etapa. Observe detalhadamente cada trecho marcado em negrito na sequência indicada a seguir.

```

-T
AX=0000 BX=00AA CX=0008 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 NV UP EI PL NZ NA PO NC
07E9:0103 D0D3          RCL    BL,1

-T
AX=0000 BX=0054 CX=0008 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO CY
07E9:0105 E2FC          LOOPW  0103

-T
AX=0000 BX=0054 CX=0007 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO CY
07E9:0103 D0D3          RCL    BL,1

-T
AX=0000 BX=00A9 CX=0007 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO NC
07E9:0105 E2FC          LOOPW  0103

-T
AX=0000 BX=00A9 CX=0006 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO NC
07E9:0103 D0D3          RCL    BL,1

-T
AX=0000 BX=0052 CX=0006 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO CY
07E9:0105 E2FC          LOOPW  0103

-T
AX=0000 BX=0052 CX=0005 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO CY
07E9:0103 D0D3          RCL    BL,1

-T
AX=0000 BX=00A5 CX=0005 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO NC
07E9:0105 E2FC          LOOPW  0103

-T
AX=0000 BX=00A5 CX=0004 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO NC
07E9:0103 D0D3          RCL    BL,1

-T
AX=0000 BX=004A CX=0004 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO CY
07E9:0105 E2FC          LOOPW  0103

-T
AX=0000 BX=004A CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO CY
07E9:0103 D0D3          RCL    BL,1

-T
AX=0000 BX=0095 CX=0003 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO NC
07E9:0105 E2FC          LOOPW  0103

-T
AX=0000 BX=0095 CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO NC
07E9:0103 D0D3          RCL    BL,1

```

```
-T
AX=0000 BX=002A CX=0002 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 OV UP EI PL NZ NA PO CY
07E9:0105 E2FC          LOOPW  0103
```

```
-T
AX=0000 BX=002A CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0103 OV UP EI PL NZ NA PO CY
07E9:0103 D0D3          RCL     BL,1
```

```
-T
AX=0000 BX=0055 CX=0001 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 NV UP EI PL NZ NA PO NC
07E9:0105 E2FC          LOOPW  0103
```

```
-T
AX=0000 BX=0055 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0107 NV UP EI PL NZ NA PO NC
07E9:0107 CD20          INT     20
```

-T

Program terminated (0000)

Observe pela sequência anterior que o registrador geral **CX** é decrementado de **0001** em **0001** até chegar a **0000**, em que a execução passo a passo deve ser interrompida.

Falta pouco para conseguir a apresentação do valor no formato binário. A próxima etapa é interpretar no registrador de estado *Carry Flag* cada **CY** como **1** e cada **NC** como **0**. Será utilizado o comando **ADC** (*Add Carry Flag*). O comando **ADC** tem como diferença do comando **ADD** a capacidade de, além de adicionar dois valores, efetuar a adição do *bit* do estado do registrador de estado *Carry Flag*.

Para conseguir o efeito de apresentação de valor binário (que será formado pelos caracteres **0** e **1**, os quais possuem, respectivamente, os valores ASCII **30** (indica o caractere **0**) e **31** (indica caractere **1**) em formato hexadecimal), será adicionado o valor atual do registrador de estado *Carry Flag* com o valor hexadecimal **30**. Neste caso, se o *bit* do *Carry Flag* for **NC** (valor 0), será apresentado o caractere de código hexadecimal **30**; caso o *bit* do *Carry Flag* venha a ser **CY** (valor 1), será apresentado o caractere de código hexadecimal **31**.

Assim sendo, defina para o registrador **BX** o valor **00AA** e para o registrador **IP** o valor **0001**, depois insira o código a seguir a partir do deslocamento **0100**. Faça-o com o comando **A 0100**.

```
07E9:0100 MOV  AH,02    <Enter>
07E9:0102 MOV  CX,0008  <Enter>
07E9:0105 MOV  DL,00    <Enter>
07E9:0107 RCL  BL,1     <Enter>
07E9:0109 ADC  DL,30    <Enter>
07E9:010C INT  21      <Enter>
07E9:010E LOOP 0105    <Enter>
07E9:0110 INT  20      <Enter>
07E9:0112          <Enter>
```

A primeira linha (**07E9:0100**) do código anterior define o valor hexadecimal **02** para o registrador geral **DL**. Lembre-se de que esse valor informa ao sistema operacional que é para imprimir um único caractere armazenado no registrador **DL** (registrador geral menos significativo do registrador geral **DX**).

A segunda linha (**07E9:0102**) define o valor total do contador de laço, representado pelo registrador geral **CX**. É importante lembrar que o registrador geral **CX** é exclusivo para a definição de contadores, por esta razão a instrução **LOOP** o utiliza. Outro ponto a ser observado é que a instrução **LOOP** efetua sempre um laço decrescente, diminuindo o valor do registrador **CX**.

A terceira linha (**07E9:0105**) movimenta o valor **00** para o registrador geral menos significativo **DL**. Isso faz com que o registrador seja limpo (zerado) toda vez que for executada essa linha de código. Esta atitude retira o último valor utilizado. Lembre-se de que os registradores possuem o efeito de ir acumulando a carga de valores que são a eles imputados.

A quarta linha (**07E9:0107**) utiliza a instrução **RCL** que movimenta para a esquerda um *bit* durante todo o *byte* utilizado do registrador geral menos significativo **BL** para representar o valor **00AA** (o valor interno realmente considerado é **AA**; o **00** à esquerda não possui nenhum valor).

A quinta linha (**07E9:0109**) utiliza a instrução **ADC** que efetua a adição do valor hexadecimal **30** (caractere **0** - zero) ao valor existente no registrador menos significativo **DL**, que pode ser **1** ou **0**, dependendo do valor do registrador de estado *Carry Flag*. Toda vez que a instrução **ADC** efetua uma adição, ela também muda o valor do registrador de estado *Carry Flag* de **NC** (não sinalizado) para **CY** (sinalizado) e vice-versa. Desta forma, é possível simular a geração e a apresentação de um valor binário convertido a partir de um valor hexadecimal definido.

A sexta linha (**07E9:010C**) efetua a apresentação na tela do monitor de vídeo do valor existente no registrador geral menos significativo **DL** pela instrução **INT 21**.

A sétima linha (**07E9:010E**) efetua o decremento em **1** do valor existente no registrador geral **CX** e transfere a execução do fluxo de programa para a linha de código identificada pelo valor **0105**. Tudo isso é controlado pela instrução **LOOPW**.

A oitava (**07E9:0110**) e última linha de código do programa efetua a devolução do controle operacional do programa ao sistema operacional por meio da instrução **INT 20**.

Observe a seguir um paralelo entre o código escrito em *opcodes* e linguagem *Assembly* equivalentes do programa anterior que pode ser obtido a partir da instrução **U 0100 0110**.

07E9:0100	B4 02	MOV	AH,02
07E9:0102	B9 08 00	MOV	CX,0008
07E9:0105	B2 00	MOV	DL,00
07E9:0107	D0 D3	RCL	BL,1
07E9:0109	80 D2 30	ADC	DL,30
07E9:010C	CD 21	INT	21
07E9:010E	E2 F5	LOOPW	0105
07E9:0110	CD 20	INT	20

Execute o programa com o comando **G** para visualizar na forma binária o valor hexadecimal **10101010** definido para o registrador geral **BX**.

5.5 - Registradores de estado

No tópico anterior foi bastante utilizado o registrador de estado **CF** (*Carry Flag*), que manifesta sua ação quando uma operação de cálculo matemático é efetuada. No entanto, o conjunto de registradores de estado possui outros registradores que se alteram quando uma operação de cálculo é realizada, e por esta razão merecem alguma atenção. Por exemplo, quando se utiliza os comandos **ADD** e **SUB** os registradores **AF**, **CF**, **OF**, **PF**, **SF** e **ZF** são alterados; ao se usar os comandos **MUL CF** e **OF**; o uso do comando **DIV** não altera nenhum registrador de *flag*.

O modelo 8086/8088 possui nove registradores de estado, sendo este valor de 32 em modelos mais recentes de microprocessadores da Intel e AMD. Os registradores de estado são elementos que sinalizam um estado de ação para que o microprocessador “saiba” como efetuar certa tarefa.

Entre o conjunto disponível estão os registradores de estado que serão apresentados neste tópico: registrador de estado *Zero Flag* (**ZF**), que pode estar sinalizado com os valores **ZR** - *zero* - ou com **NZ** - *not zero*; registrador de estado *Sign Flag* (**SF**), que pode estar sinalizado com os valores **NG** - *negative* - ou com **PL** - *plus* - positivo, registrador de estado *Overflow Flag* (**OF**), que pode estar sinalizado com os valores **OV** - *overflow* ou com **NV** - *not overflow*, o registrador de estado **PF** (*Parity Flag*) que indica se a paridade após uma ação é ímpar com valor **PO** - *parity odd* ou par com valor **PE** - *parity even* e o registrador de estado **AF** (*Auxiliary Carry Flag*) que opera com os valores **NA** - *not auxiliary carry* e **AC** - *auxiliary carry*.

Os registradores de estado *Direction Flag* (**DF**) que opera com os valores **DN** - *direction down* e **UP** - *direction up* é usado nas ações de contagem com os registradores *Source Index* (**SI**) e *Destination Index* (**DI**) quando da operação e tratamento de seqüências de caracteres, *Interrupt Flag* (**IF**) que opera com os valores **EI** - *enabled interrupt* e **DI** - *disabled interrupt* quando ações de interrupção são habilitadas e desabilitadas e *Trap Flag* (**TF**) que não tem seus valores de estado visíveis no programa **Enhanced DEBUG** e efetua uma interrupção na execução do programa, permitindo ao comando **T** por exemplo, executar um programa passo-a-passo e ter acesso a ação de cada uma das instruções separadamente. Os registradores **DF**, **IF** e **TF** não serão demonstrados além do que aqui está sendo exposto por não ser isto tão necessário.

Para realizar testes com os registradores de estado *Zero Flag*, *Sign Flag* e *Overflow Flag*, sugere-se sair do programa **Enhanced DEBUG** em uso e fazer novo carregamento do programa em memória, para que todos os registradores sejam iniciados com seus valores padrão, como é indicado em seguida após a execução do comando **R**:

```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
```

Observe atentamente o lado direito da segunda linha. Em especial, note os registradores sinalizados com os valores **NV** (primeiro na lista), **PL** (quarto na lista) e **NZ** (quinto na lista) que estão sinalizados em negrito.

```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
```

Para um primeiro teste será verificada uma operação de subtração de valores. Forneça o valor **0005**, respectivamente, para os registradores gerais **AX** e **BX**. Assim sendo execute as instruções:

```
R AX 0005 <Enter>
R BX 0005 <Enter>
```

Observe a seguir como devem estar os dados em memória após executar o comando **R**.

```
AX=0005 BX=0005 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3          RET
```

Para realizar a subtração, execute a instrução **E 0100 29 D8** e acione a tecla **<Enter>**. Lembre-se de que este é o código de máquina para realização de uma subtração.

Observação

Verifique sempre se o registrador **IP** está com o valor de deslocamento **0100**. Caso não esteja, lembre-se de executar o comando **R IP** e fornecer o valor correto.

Na seqüência acione o comando **R** para que seja apresentado o estado atual dos registradores. Confirme se o registrador **IP** aponta para o endereço **0100** como é mostrado em seguida:

```
AX=0005 BX=0005 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 29D8      SUB     AX,BX
```

Execute o comando **T** para que o resultado da subtração seja armazenado no registrador geral **AX** (neste caso será armazenado o valor hexadecimal **0000**) e a informação do registrador de estado *Zero Flag* indicado como **NZ** seja apresentado sinalizado com o identificador **ZR**, indicando que a operação executada zerou o valor do registrador geral **AX**. Além do registrador de estado *Zero Flag* o registrador de estado *Parity Flag* também é alterado de **PO** para **PE**.


```

AX=0000 BX=0005 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL ZR NA PE NC
07E9:0102 F726FC26          MUL      WORD PTR [26FC]          DS:26FC=0000

```

O registrador de estado *Overflow Flag* é sinalizado todas as vezes que ocorre um estouro da capacidade numérica a ser armazenada na estrutura de um *word*. Para testar essa ocorrência, entre para o registrador geral **AX** o valor hexadecimal **6000** e para o registrador geral **BX** o valor hexadecimal **7000**. Ajuste o registrador de deslocamento **IP** para o endereço de deslocamento **0100** e entre para a adição o código em linguagem de máquina **01** e **D8**, a partir do endereço **0100** por meio do comando **E**. Assim sendo, execute as instruções:

```

R AX 6000    <Enter>
R BX 7000    <Enter>
R IP 0100    <Enter>
E 0100 01 D8 <Enter>

```

Observe se os dados de seu computador estão semelhantes aos pontos marcados em negrito após execução do comando **R**:

```

AX=6000 BX=7000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL ZR NA PE NC
07E9:0100 01D8          ADD      AX,BX

```

Execute o comando **T** e observe o valor armazenado no registrador **AX** após a operação (valor hexadecimal negativo **D000**).

```

AX=D000 BX=7000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 OV UP EI NG NZ NA PE NC
07E9:0102 F726FC26          MUL      WORD PTR [26FC]          DS:26FC=0000

```

Observe também a indicação dos registradores de estado *Overflow Flag* e *Sign Flag*, os quais mostram, respectivamente, os valores **OV** (indicando que ocorreu um estouro da capacidade de cálculo, ou seja, ocorreu um erro) e **NG** (indicando que o valor apresentado é considerado negativo).

O registrador de estado *Sign Flag* também pode manifestar-se quando ocorrer a subtração de um valor menor em relação a um valor maior. Por exemplo, estabeleça para o registrador **AX** o valor hexadecimal **0000** e para o registrador geral **BX** o valor hexadecimal **0001**. Ajuste o registrador de deslocamento **IP** para o endereço de deslocamento **0100** e entre para a subtração o código em linguagem de máquina **29** e **D8**, a partir do endereço **0100** por meio do comando **E** a partir das instruções:

```

R AX 0000    <Enter>
R BX 0001    <Enter>
R IP 0100    <Enter>
E 0100 29 D8 <Enter>

```

Observe se os dados de seu computador estão semelhantes aos pontos marcados em negrito após execução do comando **R**:

```

AX=0000 BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 OV UP EI NG NZ NA PE NC
07E9:0100 29D8          SUB      AX,BX

```

Execute o comando **T** e observe o valor armazenado no registrador geral **AX** após a operação (valor hexadecimal negativo **FFFF**).

AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI NG NZ AC PE CY
07E9:0102 F726FC26 MUL WORD PTR [26FC] DS:26FC=0000

O registrador de estado *Overflow* com o valor **NV**, anteriormente marcado como **OV** sinaliza a ocorrência de um estouro na capacidade numérica do cálculo efetivado.

Outros dois registradores de estado que tiveram seus valores alterados são: *Auxiliary Carry Flag* (indicado que ocorreu na operação o conceito pega-um) que teve seu valor de **NA** alterado para **AC** e *Parity Flag* (indicando se a quantidade de *bits* manipulado na memória é par ou ímpar) que neste caso mantém o valor **PE** ao indicar que o resultado **FFFF** é um valor com quantidade de *bits* par.

É comum utilizar o registrador de estado (*flags*) em operações de programação, principalmente naquelas que envolvem saltos condicionais. Cada *flag* que compõe o conjunto do registrador de estado possui um valor particular, como mostra a Tabela 5.3.

Tabela 5.3 - Valores dos registradores de *flag*

Bit	Flag	Nome	Finalidade	Ativado	Desativado
11	OF	Overflow	Sinaliza uma operação aritmética com estouro da capacidade de cálculo.	OV	NV (*)
10	DF	Direction	Determina a direção de contagem de SI/DI para cima ou para baixo em sequências de caracteres.	DN	UP (*)
9	IF	Interrupt	Identifica se alguma interrupção de hardware está ou não ativa quando do uso do comando INT .	EI (*)	DI
8	TF	Trap	Permite que um programa seja executado linha a linha durante a fase de depuração (comando T).	Não é visível para o DEBUG	
7	SF	Sign	Indica a obtenção de um valor positivo ou negativo em uma operação aritmética.	NG	PL (*)
6	ZF	Zero	Indica se o resultado após uma operação aritmética é zero, ou se o resultado de comparação após uma ação lógica é igual.	ZR	NZ (*)
4	AF	Auxiliary Carry	Indica a sinalização de <i>carry</i> quando da adição e subtração de valores binários baseados no formato BCD.	AC	NA (*)
2	PF	Parity	Indica se a paridade de um valor é par ou ímpar após uma operação lógica ou aritmética.	PE	PO (*)
0	CF	Carry	Indica a sinalização de <i>carry</i> (vai um) para a próxima posição, após a efetivação de uma operação aritmética.	CY	NC (*)

Na Tabela 4.3 as colunas **Ativado** e **Desativado** indicam os valores que cada um dos registradores de estado pode assumir. Internamente os valores de ativação serão **1** e os valores de desativação serão **0**. As células da Tabela 4.3 com a indicação dos *status* assinalados com asterisco mostram os valores dos registradores definidos automaticamente por padrão pelo sistema.

Assim como é possível fazer a definição e alteração de valores para os registradores gerais **AX**, **BX**, **CX** e **DX** e de apontamento como **IP** os registradores de estado podem ter seus valores manualmente alterados a partir do uso do comando **R F** (*Register Flag*), que pode ser usado como **RF**.

Quando o comando **RF** é usado ocorre a apresentação de um *prompt* com os valores dos registradores de estado que estão ativos. Neste instante basta informar o valor referente ao registrador que se deseja alterar para que a mudança ocorra. Por exemplo, para mudar o valor de **CY** para **NC** do registrador de estado **CF** basta informar no *prompt* apresentado pelo comando **RF** o valor **NC**.

5.6 - Instruções de salto condicional

Anteriormente foi apresentado o comando **LOOPW** que efetua a ação de laço incondicional tendo seu controle definido junto ao registrador geral **CX**. No entanto, existem outros comandos de salto controlados por ações condicionadas aos registradores de estado, sendo esses comandos reesponsáveis por ações de saltos condicionais.

Os comandos usados para a realização de saltos condicionais são recursos que possibilitam executar ações de desvio em um programa, a partir da verificação de condições e valores dos registradores de estado (*flags*). Os comandos de saltos condicionais ocupam de 2 a 4 *bytes* de memória, sendo:

- ♦ **JZ** (*jump on zero*) - salta para um determinado endereço de deslocamento caso o registrador de estado *Zero Flag* esteja sinalizado como zero (**ZR**);
- ♦ **JNZ** (*jump on not zero*) - salta para um determinado endereço de deslocamento caso o registrador de estado *Zero Flag* esteja sinalizado como não zero (**NZ**).

Para demonstrar o salto condicional com o comando **JNZ**, considere apresentar na tela de vídeo a letra **A** enquanto o registrador geral menos significativo **BL** for maior que zero. Quando o registrador geral **BL** estiver com zero, o programa apresenta a letra **B**, e encerra a sua execução. Informe o código seguinte a partir do endereço de deslocamento **0100** com o comando **A**:

```
07E9:0100 MOV BL,03 <Enter>
07E9:0102 MOV AH,02 <Enter>
07E9:0104 MOV DL,41 <Enter>
07E9:0106 INT 21 <Enter>
07E9:0108 SUB BL,01 <Enter>
07E9:010B JNZ 0106 <Enter>
07E9:010D MOV DL,42 <Enter>
07E9:010F INT 21 <Enter>
07E9:0111 INT 20 <Enter>
07E9:0113 <Enter>
```

A primeira linha do programa (**07E9:0100**) determina o valor hexadecimal **03** ao registrador geral menos significativo **BL**. Esse registrador será utilizado para armazenar o valor de contagem do laço de repetição até que ele seja zero, ou seja, até que o registrador de deslocamento *Zero Flag* seja sinalizado com **NZ**.

A segunda linha do programa (**07E9:0102**) determina o código hexadecimal **02** de controle da apresentação de caracteres (apenas um caractere) ao registrador geral mais significativo **AH**.

A terceira linha do programa (**07E9:0104**) determina o armazenamento do código hexadecimal **41** correspondente ao valor da letra **A**. O código **41** é armazenado no registrador geral menos significativo **DL**.

A quarta linha do programa (**07E9:0106**), por meio da instrução **INT 21**, apresenta o caractere armazenado no registrador **DL**.

A quinta linha do programa (**07E9:0108**) efetua com a instrução **SUB** (*subtract*) a subtração de **01** (**SUB BL,01**) no valor armazenado no registrador **BL**, fazendo com que o valor passe a ser **02**, depois **01** e por último **00**.

A sexta linha do programa (**07E9:010B**) salta o programa voltando a execução para o endereço de deslocamento **0106** (quarta linha, que efetua uma nova apresentação do caractere armazenado no registrador **DL**). Esse ciclo se repete até o momento em que o registrador menos significativo **BL** se torna **00** e gera o sinal **ZR** para o registrador de estado *Zero*.

A sétima linha do programa (**07E9:010D**) determina o armazenamento do código hexadecimal **42** correspondente ao valor da letra **B** no registrador geral menos significativo **DL**. Essa ação somente ocorre quando a sexta linha deixar de operar, passando o fluxo de execução do programa para a próxima linha.

A oitava linha do programa (**07E9:010F**), por meio da instrução **INT 21**, apresenta o caractere armazenado no registrador **DL**, que nesse momento é **B**.

A nona e última linha do programa (**07E9:0111**) finaliza o programa e retorna o controle de execução para o sistema operacional.

Observe a seguir um paralelo entre o código escrito em *opcodes* e linguagem *Assembly* equivalentes do programa anterior que pode ser obtido a partir da instrução **U 0100 0111**.

07E9:0100	B3 03	MOV BL,03
07E9:0102	B4 02	MOV AH,02
07E9:0104	B2 41	MOV DL,41
07E9:0106	CD 21	INT 21
07E9:0108	80 EB 01	SUB BL,01
07E9:010B	75 F9	JNZ 0106
07E9:010D	B2 42	MOV DL,42
07E9:010F	CD 21	INT 21
07E9:0111	CD 20	INT 20

Em seguida execute o programa com o comando **G** para visualizar na tela do monitor de vídeo a sequência de caracteres **AAAB**.

Além dos comandos de salto **JZ** e **JNZ**, há ainda o comando **CMP** (*compare*), a qual não altera o valor inicial de um determinado registrador geral. Sua finalidade é apenas comparar os valores. Se eles forem iguais, esse comando altera a sinalização do registrador de estado *Zero Flag* para **ZR**.

O comando **CMP** não efetua a comparação de locais de memória ou dois valores informados na mesma instrução. Os conteúdos (operandos) informados para o comando **CMP** devem ser do mesmo tamanho, podendo ser valores do tipo *byte* ou *word*.

O comando **CMP** quando executada altera os valores dos registradores de estados (*flags*) **AF**, **CF**, **OF**, **PF**, **SF** e **ZF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, registrador (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, registrador (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ registrador, memória (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ registrador, constante (para este tipo de ação usa de 3 a 4 *bytes* de memória);
- ◆ memória, constante (para este tipo de ação usa de 3 a 6 *bytes* de memória).

Por exemplo, entre o valor hexadecimal **AAAA** para os registradores gerais **AX** e **BX** e ajuste o registrador **IP** para **0100**, como é mostrado a seguir:

```
R AX AAAA <Enter>
R BX AAAA <Enter>
R IP 0100 <Enter>
```

Observe se os dados de seu computador estão semelhantes aos pontos marcados em negrito após execução do comando **R**:

```
AX=AAAA BX=AAAA CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 B303          MOV     BL,03
```

Com o comando **A 0100** entre a linha de código **SUB AX, BX**, verifique se o registrador **IP** aponta para o endereço de deslocamento **0100** e execute o comando **T**. Note o valor alterado do registrador geral **AX** e, em especial, do registrador de estado *Zero Flag*:

```
AX=0000 BX=AAAA CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL ZR NA PE NC
07E9:0102 B402          MOV     AH,02
```

Saia do programa **Enhanced DEBUG** e retorne ao programa novamente. Defina o valor hexadecimal **AAAA** nos registradores **AX** e **BX**, acione o comando **A 0100** entre a linha de código **CMP AX, BX**, verifique se o registrador **IP** aponta para o endereço de deslocamento **0100** e execute o comando **R**.

```
AX=AAAA BX=AAAA CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0101 NV UP EI PL NZ NA PO NC
07E9:0100 39D8          CMP      AX,BX
```

Na sequência execute o comando **T** e observe atentamente todos os pontos indicados a seguir em negrito:

```
AX=AAAA BX=AAAA CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL ZR NA PE NC
07E9:0102 7261          JB      0165
```

Os valores dos registradores **AX** e **BX** estão inalterados, mas a sinalização do registrador de estado *Zero Flag* indica **ZR** quando anteriormente indicava **NZ** e o registrador *Parity Flag* indica **PE** quando anteriormente indicava **PO**. Mais adiante esta característica de comportamento do comando **CMP** será utilizada.

5.7 - Execução básica de desvios

A partir do conjunto de detalhes anteriormente apresentado torna-se possível trabalhar em algo um pouco mais sofisticado, como, por exemplo, escrever na tela do monitor de vídeo um valor de um dígito expresso em notação hexadecimal.

O programa deve apresentar um valor hexadecimal entre **0** (zero) e **F** (quinze em decimal). É preciso levar em consideração o código ASCII em hexadecimal de cada caractere que compõe a formação de um único valor hexadecimal. Considere o trecho apresentado na Tabela 5.4.

Tabela 5.4 - Trecho com caracteres e seus valores hexadecimais

Caractere	Código ASCII (Hexadecimal)	Caractere	Código ASCII (Hexadecimal)
0	30	<	3C
1	31	=	3D
2	32	>	3E
3	33	?	3F
4	34	@	40
5	35	A	41
6	36	B	42
7	37	C	43
8	38	D	44
9	39	E	45
:	3A	F	46
;	3B		

A parte numérica de um valor hexadecimal representado na Tabela 4.4 vai do código ASCII **30** até o código ASCII **39**, enquanto a parte alfabética de um valor hexadecimal vai do código ASCII **41** até o código ASCII **46**. Para formar a apresentação de um valor hexadecimal, é necessário usar dois grupos de caracteres com sequências diferentes da tabela ASCII com uma disparidade de sete posições. O programa em questão deve considerar este fato.

Para conseguir a apresentação na tela do monitor de vídeo de um valor hexadecimal, é necessário utilizar o comando de comparação **CMP** (*compare*) em conjunto com os comandos de salto condicional. O comando **CMP** efetua a subtração dos valores existentes entre os registradores gerais **AX** e **BX** sem que o resultado da operação seja armazenado na memória e sem efetuar a alteração do valor do registrador **AX**. O resultado da operação altera apenas os valores dos ponteiros dos registradores de estado, permitindo que se utilize na sequência algum dos comandos de saltos condicionais.

Antes de se preocupar com a rotina de apresentação de um único valor hexadecimal, é pertinente testar um pouco o funcionamento do comando **CMP** em conjunto com um dos vários comandos de saltos condicionais existentes. Neste

caso, será utilizado o comando **JG** (*jump on greater*) que não afeta nenhum dos registradores de estado e ocupa de 2 a 4 bytes de memória.

Para realizar o teste condicional do comando **JG**, defina, respectivamente, para os registradores gerais **AX**, **BX** e **IP** os valores hexadecimais **0003**, **0002** e **0100** com as instruções:

```
R AX 0003 <Enter>
R BX 0002 <Enter>
R IP 0100 <Enter>
```

Em seguida, execute a instrução **A 0100** e entre a sequência de código seguinte:

```
07E9:0100 CMP AX,BX <Enter>
07E9:0102 JG 0108 <Enter>
07E9:0104 MOV DL,BB <Enter>
07E9:0106 JMP 010A <Enter>
07E9:0108 MOV DH,AA <Enter>
07E9:010A SUB DX,DX <Enter>
07E9:010C <Enter>
```

O programa executa na primeira linha (**07E9:0100**) a comparação dos valores dos registradores gerais **AX** e **BX** com o comando **CMP**, o qual realiza uma subtração entre os valores dos registradores, afetando apenas valores dos registradores de estado: **AF** (*Auxiliar Flag*), **CF** (*Carry Flag*), **OF** (*Overflow Flag*), **PF** (*Parity Flag*), **SF** (*Signal Flag*) e **ZF** (*Zero Flag*).

Na segunda linha (**07E9:0102**) o programa efetua um desvio condicional, caso o valor do registrador geral **AX** seja maior que o valor do registrador geral **BX**, por meio da instrução condicional **JG**. O fluxo do programa passa para a linha de código **0108**, caso a condição seja verdadeira. Para uma instrução condicional funcionar, ela necessita do uso prévio da instrução **CMP**.

A terceira linha (**07E9:0104**) carrega o valor hexadecimal **BB** para dentro da parte menos significativa (**DL**) do registrador geral **DX**. Essa linha somente será executada caso a linha **0102** não faça o desvio programado.

A quarta linha (**07E9:0106**), por meio da instrução **JMP**, realiza um salto incondicional para a linha de código **010A**. Essa linha somente será executada caso a linha **0102** não efetue o desvio programado. A instrução **JMP** não afeta nenhum registrador de estado, ocupando de 2 até 4 bytes de memória. Esta instrução será apresentada em mais detalhes no capítulo 8.

A quinta linha (**07E9:0104**) carrega o valor hexadecimal **AA** para dentro da parte mais significativa (**DH**) do registrador geral **DX**. Essa linha somente será executada caso a condição da linha **0102** efetue o desvio programado.

A sexta e última linha (**07E9:010A**) faz a subtração de todo o conteúdo do registrador geral **DX** pelo próprio conteúdo do registrador geral **DX**, ou seja, desta forma o valor do registrador é zerado.

Observe a seguir um paralelo entre o código escrito em *opcodes* e linguagem *Assembly* equivalentes do programa anterior que pode ser obtido a partir da instrução **U 0100 010A**.

07E9:0100	39 D8	CMP AX,BX
07E9:0102	7F 04	JG 0108
07E9:0104	B2 BB	MOV DL,BB
07E9:0106	EB 02	JMP 010A
07E9:0108	B6 AA	MOV DH,AA
07E9:010A	29 D2	SUB DX,DX

Ao executar o programa, utilize o comando **T** para fazê-lo passo a passo, mas antes disso verifique se o registrador de deslocamento **IP** aponta para o endereço de deslocamento **0100**. Caso não esteja, execute o comando **R IP** e forneça o valor **0100**.

Na sequência execute o comando **R** e observe a apresentação das informações pertinentes ao estado atual dos valores em memória marcados em negrito.

```

AX=0003 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL ZR NA PE NC
07E9:0100 39D8 CMP AX,BX

```

Em seguida execute o programa com o comando T. A seguir veja o resultado das operações solicitadas:

```

AX=0003 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 7F04 JG 0108

```

Nesta etapa pode-se perceber que, ao ser executado o comando T, o registrador de deslocamento **IP** encontra--se na posição **0102**, na qual se encontra a instrução **JG 0108**.

Pelo fato de o valor do registrador geral **AX** ser maior que o valor do registrador geral **BX**, ele ocasiona um resultado da operação de subtração positivo, o que caracteriza que o primeiro valor é maior que o segundo valor e desvia o fluxo de execução do programa para a linha **0108**, como pode ser constatado na próxima execução do comando T.

```

AX=0003 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0108 NV UP EI PL NZ NA PO NC
07E9:0108 B6AA MOV DH,AA

```

Ocorreu o desvio para a linha **0108** na qual se encontra a instrução **MOV DH,AA** que carrega o registrador geral **DX** com o valor **AA00**, como pode ser constatado na próxima execução do comando T.

```

AX=0003 BX=0002 CX=0000 DX=AA00 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010A NV UP EI PL NZ NA PO NC
07E9:010A 29D2 SUB DX,DX

```

Observe os pontos marcados em negrito. O registrador de deslocamento **IP** está apontando para o endereço de deslocamento **010A** e neste momento mostra o registrador geral **DX** com o valor **AA00**. Dê sequência à execução do programa com o comando T.

```

AX=0003 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010C NV UP EI PL ZR NA PE NC
07E9:010C 26283F SUB ES:[BX],BH ES:0002=FF

```

Após a última execução do comando T o registrador geral **DX**, devido à execução das linhas **SUB DX,DX**, passa a ter o valor **0000**. Perceba também a posição de deslocamento em que se encontra o registrador de deslocamento **IP**.

Para fazer outro teste, altere o valor do registrador geral **AX** para **0001** e do registrador de deslocamento **IP** para **0100** com as instruções:

```

R AX 0001 <Enter>
R IP 0100 <Enter>

```

Em seguida acione uma vez o comando R e mais quatro vezes o comando T, prestando muita atenção no que ocorre. A seguir é apresentada a sequência de ocorrência da ação solicitada:

```

-R
AX=0001 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL ZR NA PE NC
07E9:0100 39D8 CMP AX,BX

```

```

-T
AX=0001 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI NG NZ AC PE CY
07E9:0102 7F04 JG 0108

```

```
- T
AX=0001 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0104 NV UP EI NG NZ AC PE CY
07E9:0104 B2BB          MOV     DL,BB
```

```
- T
AX=0001 BX=0002 CX=0000 DX=00BB SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0106 NV UP EI NG NZ AC PE CY
07E9:0106 EB02          JMP     010A
```

```
- T
AX=0001 BX=0002 CX=0000 DX=00BB SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010A NV UP EI NG NZ AC PE CY
07E9:010A 29D2          SUB     DX,DX
```

```
- T
AX=0001 BX=0002 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010C NV UP EI PL ZR NA PE NC
07E9:010C 26283F        SUB     ES:[BX],BH          ES:0002=FF
```

Nesta sequência ocorreu algo de diferente na execução do programa. Pelo fato de o valor do registrador geral **AX** ser menor que o valor do registrador geral **BX** (observe as indicações do registrador de estado **CF** com a indicação **NC** e **CY**), ele gerou um resultado negativo. A instrução da linha **0102** não é considerada, passando o controle do fluxo de execução do programa para a linha seguinte (linha **0104** que atribui o valor **00BB** ao registrador geral **DX**). Depois o programa encontra a linha **0106** que por meio da instrução **JMP 010A** desvia a execução do programa para a linha **010A**, que limpa o valor existente no registrador geral **DX**.

A partir da explicação anteriormente exposta, já é possível escrever um trecho de programa que apresente na tela o valor de um número hexadecimal de uma única posição. Para tanto, ajuste o valor do registrador de deslocamento **IP** para **0100**; zere os valores dos registradores gerais **AX** e **BX** com as instruções:

```
R AX 0000 <Enter>
R BX 0000 <Enter>
R IP 0100 <Enter>
```

A partir da execução da instrução **A 0100** entre as instruções do seguinte programa:

```
07E9:0100 MOV AH,02 <Enter>
07E9:0102 MOV DL,BL <Enter>
07E9:0104 ADD DL,30 <Enter>
07E9:0107 CMP DL,39 <Enter>
07E9:010A JLE 010F <Enter>
07E9:010C ADD DL,07 <Enter>
07E9:010F INT 21 <Enter>
07E9:0111 INT 20 <Enter>
07E9:0113 <Enter>
```

O programa anterior possui alguns detalhes já familiares. A primeira linha (**07E9:0100**) estabelece para a parte mais significativa (**AH**) do registrador geral **AX** o valor hexadecimal **02**, responsável por apresentar um único caractere na tela do monitor de vídeo.

A segunda linha (**07E9:0102**) movimenta para a parte menos significativa do registrador geral **DX** o conteúdo armazenado na parte menos significativa do registrador geral **BX**.

A terceira linha (**07E9:0104**) adiciona o valor hexadecimal **30** ao valor atual armazenado no registrador geral **BX**. O valor hexadecimal **30** equivale ao código ASCII do caractere **0** (zero). Os valores **30**, **31**, **32**, **33**, **34**, **35**, **36**, **37**, **38** e **39** correspondem ao código ASCII equivalente para a apresentação dos caracteres **0**, **1**, **2**, **3**, **4**, **5**, **6**, **7**, **8** e **9**.

A quarta linha (**07E9:0107**) faz a comparação (subtração) do valor armazenado na parte menos significativa do registrador geral **DX** com o valor **39**, que é o código ASCII do número **9** (nove).

A quinta linha (**07E9:010A**) utiliza a instrução condicional **JLE** para verificar se o resultado da comparação (linha de código **0107**) do registrador geral **DX** é menor ou igual ao valor **39**. Se a condição for verdadeira, o fluxo de execução do programa desvia a execução para a linha **010F** que apresenta o valor equivalente encontrado no registrador geral **BX**. Caso a condição seja falsa, automaticamente o programa executa a linha de código **010C** (sexta linha). A instrução **JLE** não afeta nenhum registrador de estado, ocupando de 2 até 4 *bytes* de memória.

A sexta linha (**07E9:010C**) será executada quando a quinta linha for falsa. Neste caso o valor armazenado no registrador **BX** corresponde às sete letras que compõem a representação numérica de um valor em formato hexadecimal. A letra **A** possui código ASCII hexadecimal **41**. Ela está distante do valor nove posições. Por esta razão a sexta linha faz a adição do valor hexadecimal **07** com o valor existente na parte menos significativa do registrador geral **DX**.

A sétima (**07E9:010F**) e a oitava (**07E9:0111**) linhas do programa executam, respectivamente, a ativação da tela do monitor de vídeo para apresentação dos caracteres e também devolvem o controle de execução do programa para o sistema operacional.

Observe a seguir um paralelo entre o código escrito em *opcodes* e linguagem *Assembly* equivalentes do programa anterior que pode ser obtido a partir da instrução **U 0100 0111**.

07E9:0100	B4 02	MOV AH,02
07E9:0102	88 DA	MOV DL,BL
07E9:0104	80 C2 30	ADD DL,30
07E9:0107	80 FA 39	CMP DL,39
07E9:010A	7E 03	JLE 010F
07E9:010C	80 C2 07	ADD DL,07
07E9:010F	CD 21	INT 21
07E9:0111	CD 20	INT 20

Estabeleça para o registrador geral **BX** um valor hexadecimal na faixa de **0000** até **000F**. Para um teste execute o comando **R BX** e informe o valor **0007**, mantenha **IP** em **0100**. Depois execute os comandos **R** e **G** e observe os dados apresentados em tela, como indicado a seguir:

```
-R
AX=0000 BX=0007 CX=0000 DX=0000 SP=FFE8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL ZR NA PE NC
07E9:0100 B402          MOV     AH,02
```

```
-G
7
Program terminated (0000)
```

Experimente alterar os valores hexadecimais da parte menos significativa do registrador geral **BX**. Lembre-se de limitar as alterações entre os valores de **00h** até **0Fh**.

5.8 - Apresentar valor hexadecimal

Anteriormente foi conseguido o efeito de apresentar em tela um valor hexadecimal de um único dígito. A questão agora é apresentar um número hexadecimal com mais de um dígito, ou seja, com dois dígitos.

Para apresentar um número hexadecimal com dois dígitos, é necessário utilizar o comando **SHR** (*Shift Logical Right*) para fazer a rotação de *bits* no sentido da direita. Lembre-se de que anteriormente esse trabalho foi feito pelo comando **RCL**, a qual fazia a rotação no sentido da esquerda. É pertinente, nesta etapa, sair do programa **Enhanced DEBUG** em uso e retornar a ele em seguida para que todos os registradores sejam inicializados com seus valores padrão.

O comando **SHR** possibilita a rotação de *bits* no sentido da direita com uma pequena diferença em relação ao comando **RCL**, uma vez que é possível efetuar a movimentação de mais de um *bit* na mesma instrução. É necessário apenas fornecer para o comando **SHR** o número de vezes que a rotação deve ser realizada (considerando o fato de ser um tipo *byte* ou um tipo *word*). O valor da rotação fica definido na parte menos significativa (**CL**) do registrador geral **CX**, estando limitado ao valor máximo de dezesseis (**10** em hexadecimal) que é o tamanho de um tipo *word*.

O comando **SHR** quando executada altera os valores dos registradores de estados (*flags*) **CF**, **OF**, **PF**, **SF** e **ZF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, 1 (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, 1 (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ registrador, CL (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, CL (para este tipo de ação usa de 2 a 4 *bytes* de memória).

Para executar a apresentação em tela de valor numérico hexadecimal de dois dígitos, é necessário fazer a rotação de 4 *bits* no sentido da direita, para que o valor hexadecimal armazenado na parte menos significativa (**DL**) do registrador geral **DX** seja manipulado adequadamente. Para tanto, execute as instruções:

```
R DX 006B <Enter>
R CX 0004 <Enter>
E 0100 D2 EA <Enter>
```

O *opcode* **D2 EA** é código de operação para a instrução **SHR DL,CL** e execute o comando **R** que apresenta valores semelhantes à relação seguinte:

```
AX=0000 BX=0000 CX=0004 DX=006B SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 D2EA SHR DL,CL
```

Antes de executar qualquer ação, preste bem atenção nos pontos marcados em negrito, principalmente nos valores armazenados nos registradores gerais **CX** (contador de deslocamento de *bits* que será de quatro em quatro, ou seja, de *nibble* em *nibble*) e **DX** (valor que será rotacionado à direita). Observe atentamente o registrador de estado **CF** que do valor **NC** passará para o valor **CY**.

Em seguida execute o comando **T**.

```
AX=0000 BX=0000 CX=0004 DX=0006 SP=FFFF BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ AC PE CY
07E9:0102 F726FC26 MUL WORD PTR [26FC] DS:26FC=0000
```

O valor do registrador geral **DX** passou a ser **0006**, ou seja, ele foi movimentado quatro *bits* (um *nibble*) para a direita. Além disso, o registrador de estado **CF** passou do valor **NC** para o valor **CY**, indicando a movimentação de "*carry*", neste caso para a direita.

Para entender melhor essa ação execute a instrução **A 0100** e entre o código:

```
07E9:0100 MOV AH,02 <Enter>
07E9:0102 MOV DL,BL <Enter>
07E9:0104 MOV CL,04 <Enter>
07E9:0106 SHR DL,CL <Enter>
07E9:0108 ADD DL,30 <Enter>
07E9:010B CMP DL,39 <Enter>
07E9:010E JLE 0113 <Enter>
07E9:0110 ADD DL,07 <Enter>
07E9:0113 INT 21 <Enter>
07E9:0115 INT 20 <Enter>
07E9:0117 <Enter>
```

Em seguida execute as instruções:

```
R BX 0012 <Enter>
R IP 0100 <Enter>
```

Na sequência execute o comando **R**.


```
AX=0000 BX=0012 CX=0004 DX=0006 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PE CY
07E9:0100 B402          MOV     AH,02
```

Para fazer um teste passo a passo da execução do programa, utilize o comando **T** e não o comando **G**. Na primeira execução do comando **T** o valor hexadecimal **02** é armazenado no segmento mais significativo do registrador geral **AX** e o registrador de deslocamento **IP** aponta para o endereço **0102**, conforme a seguir:

```
AX=0200 BX=0012 CX=0004 DX=0006 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PE CY
07E9:0102 88DA          MOV     DL,BL
```

Execute novamente o comando **T** e observe a movimentação do valor existente no registrador menos significativo **BL** para o registrador menos significativo **DL**. Observe também o valor do registrador de deslocamento **IP** apontando para o endereço **0104**.

```
AX=0200 BX=0012 CX=0000 DX=0012 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0104 NV UP EI PL NZ NA PE CY
07E9:0104 B104          MOV     CL,04
```

Na sequência execute o comando **T** novamente e observe a movimentação do valor **04** para dentro do registrador menos significativo **CL**. Mais uma vez observe o valor do registrador de deslocamento **IP** apontando para o endereço **0106**.

```
AX=0200 BX=0012 CX=0004 DX=0012 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0106 NV UP EI PL NZ NA PE CY
07E9:0106 D2EA          SHR     DL,CL
```

Novamente acione o comando **T** e observe que no registrador geral **DL**, ocorre um deslocamento à direita de quatro *bits*. Note a alteração do valor do registrador de deslocamento **IP** apontando para o endereço **0108**.

```
AX=0200 BX=0012 CX=0004 DX=0001 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0108 NV UP EI PL NZ NA PE CY
07E9:0108 80C230        ADD     DL,30
```

A partir da linha **0108** o programa é idêntico ao modelo do tópico anterior. Neste caso acione desse ponto o comando **G** para a conclusão e término adequado do programa, pois o valor hexadecimal **0001** do registrador geral **DX** será apresentado como 1:

```
1
Program terminated (0000)
```

Observe a seguir um paralelo entre o código escrito em *opcodes* e linguagem *Assembly* equivalentes do programa anterior obtido a partir da instrução **U 0100 0115**.

```

07E9:0100    B4 02    MOV AH,02
07E9:0102    88 DA    MOV DL,BL
07E9:0104    B1 04    MOV CL,04
07E9:0106    D2 EA    SHR DL,CL
07E9:0108    80 C2 30  ADD DL,30
07E9:010B    80 FA 39  CMP DL,39
07E9:010E    7E 03    JLE 0113
07E9:0110    80 C2 07  ADD DL,07
07E9:0113    CD 21    INT 21
07E9:0115    CD 20    INT 20

```

Até este ponto foi conseguido o mesmo efeito do tópico anterior, mas não é este o objetivo. É preciso ainda o suporte de mais uma instrução para conseguir a impressão do segundo dígito do valor. Entrará em uso o comando de ação lógica **AND** (*and logical*) que faz a conjunção de dois valores e os considera com resultado verdadeiro se todos os valores forem também verdadeiros.

O comando **AND** quando executado altera o estado dos registradores de estados (*flags*) **CF**, **OF**, **PF**, **SF** e **ZF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, registrador (para este tipo de ação usa 2 bytes de memória);
- ◆ memória, registrador (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, memória (para este tipo de ação usa de 2 a 4 bytes de memória);
- ◆ registrador, constante (para este tipo de ação usa de 3 a 4 bytes de memória);
- ◆ memória, constante (para este tipo de ação usa de 3 a 6 bytes de memória).

Observação

O comando **T** (*trace*) utilizado até aqui foi executado sempre passo a passo. No entanto, é possível definir para ele o número de passos que deseja que ele execute. Por exemplo, experimente no *prompt* do programa **Enhanced DEBUG** a seguinte linha de comando **T = 0100 4** e observe que será apresentado o resultado de quatro passagens do comando. O comando **T** pode ocasionar execuções desconfortáveis quando direciona sua ação para dentro de uma rotina de interrupção (como é o caso do comando **INT 21**, que é na verdade uma sub-rotina adjacente ao código em uso). No entanto, o procedimento pode ser executado com o comando **P** (*proceed*) que não entra nas rotinas adjacentes, executando-as como se fossem comandos comuns. O comando **P** aceita os mesmos parâmetros que o comando **T**. Após o uso dos comandos **T** e **P** é sempre importante retornar o valor do registrador de deslocamento **IP** para **0100**.

Para o modo de programação em baixo nível, um valor é considerado verdadeiro quando está "setado" em **1** (um) e um valor é considerado falso quando está "setado" em **0** (zero). Isso é semelhante ao utilizado pelos registradores de estado. Considere a Tabela 5.5 para dois valores quaisquer avaliados em um estado de conjunção:

Tabela 5.5 - Trecho para ação de conjunção

Valor 1	Valor 2	Resultado
1	1	1
1	0	0
0	1	0
0	0	0

Na tabela o resultado será considerado verdadeiro (valor 1) quando todos os valores (no caso valores 1 e 2) forem também verdadeiros, ou seja, definidos com o valor 1 (um).

O comando **AND** faz a comparação em um determinado valor *bit a bit* (modo binário) em dados do tipo *byte* ou *word*, tendo seu uso destinado a três situações típicas (DANDAMUDI, 2000, p. 301):

1. Apoiar as composições de expressões lógicas;
2. Limpar um ou mais *bits* de um *byte* ou *word*;
3. Isolar um ou mais *bits* de um *byte* ou *word*.

Para os objetivos propostos o comando **AND** será usada para fazer a limpeza (zerar) dos *bits* do *byte* em uso que não forem de interesse. Por exemplo, imagine que se possua o valor hexadecimal **12**, o qual equivale em binário ao valor **0001 0010**, e que se deseja pegar apenas a parte da direita do valor, ou seja, o número **2** desprezando-se assim o valor **1**. Assim sendo, basta comparar o valor binário **0001 0010** com o valor binário **0000 1111** (que equivale ao valor hexadecimal **0F**), como mostra a Tabela 5.6.

Tabela 5.6 - Ação de limpeza de *bits*

<i>Byte</i>	<i>Nibble</i> mais alto				<i>Nibble</i> mais baixo			
<i>Bits</i>	7	6	5	4	3	2	1	0
Valores comparados com instrução lógica AND								
Valor original	0	0	0	1	0	0	1	0
Código para limpeza	0	0	0	0	1	1	1	1
Resultado	0	0	0	0	0	0	1	0

Observe na tabela a indicação dos valores hexadecimais **12** (equivalente a **0001 0010** em binário) e **0F** (equivalente a **0000 1111** binário). Ao ser realizada uma comparação *bit a bit* com o comando **AND** de cada parte de ambos os valores e respeitando a regra definida na tabela-verdade anterior, fica evidente que no *nibble* mais alto sua posição torna-se zerada e no *nibble* mais baixo o valor fica mantido.

A partir deste fato é possível fazer a outra parte do trabalho. Já existe o conhecimento para apresentar o primeiro valor de um número hexadecimal de um *byte*. No entanto, será demonstrado um programa que apresenta apenas o segundo valor de um número hexadecimal para depois ambas as técnicas serem trabalhadas em conjunto.

Observação

Uma forma rápida de fazer verificações condicionais com valores dos tipos binários e hexadecimais é utilizar a calculadora existente no ambiente Windows. Neste caso, selecione o modo de operação científica, o modo de operação Bin (binário) ou Hex (hexadecimal). Depois entre o primeiro valor e acione o botão [And], entre o segundo valor e acione o botão [=]. Experimente fazer o teste com o valor hexadecimal 12 no sentido de obter a parte menos significativa de seu valor, ou seja, o valor 2. Execute na calculadora 12 [And] F [=] e observe a apresentação do valor 2.

Em seguida execute as instruções:

```
R BX 0012 <Enter>
R IP 0100 <Enter>
```

Na sequência entre o código a seguir a partir da execução da instrução **A 0100**.

```
07E9:0100 MOV AH,02 <Enter>
07E9:0102 MOV DL,BL <Enter>
07E9:0104 AND DL,0F <Enter>
07E9:0107 ADD DL,30 <Enter>
07E9:010A CMP DL,39 <Enter>
07E9:010D JLE 0112 <Enter>
07E9:010F ADD DL,07 <Enter>
07E9:0112 INT 21 <Enter>
07E9:0114 INT 20 <Enter>
07E9:0116 <Enter>
```

Após entrar o código anterior, execute o comando **G** e observe a apresentação do valor hexadecimal **2**. Se desejar fazer uma execução passo a passo, utilize o comando **T**, tomando o cuidado de parar a ação do comando na linha **0112**, para não cair na rotina de programa que realiza o comando **INT 21**.

Observe a seguir um paralelo entre o código escrito em *opcodes* e linguagem *Assembly* equivalentes do programa anterior

```

07E9:0100  B4 02      MOV AH,02
07E9:0102  88 DA      MOV DL,BL
07E9:0104  80 E2 0F    AND DL,0F
07E9:0107  80 C2 30    ADD DL,30
07E9:010A  80 FA 39    CMP DL,39
07E9:010D  7E 03      JLE 0112
07E9:010F  80 C2 07    ADD DL,07
07E9:0112  CD 21      INT 21
07E9:0114  CD 20      INT 20

```

Para apresentar dois valores numéricos basicamente basta unir os dois trechos de programa conhecidos e apresentados anteriormente em um código só. O trecho de programa que utiliza o comando **SHR** deve vir à frente do trecho de programa que usa o comando **AND**.

Trechos anteriores	Código completo
<pre> 07E9:0100 MOV AH,02 07E9:0102 MOV DL,BL 07E9:0104 MOV CL,04 07E9:0106 SHR DL,CL 07E9:0108 ADD DL,30 07E9:010B CMP DL,39 07E9:010E JLE 0113 07E9:0110 ADD DL,07 07E9:0113 INT 21 07E9:0115 INT 20 </pre>	<pre> 07E9:0100 MOV AH,02 07E9:0102 MOV DL,BL 07E9:0104 MOV CL,04 07E9:0106 SHR DL,CL 07E9:0108 ADD DL,30 07E9:010B CMP DL,39 07E9:010E JLE 0113 07E9:0110 ADD DL,07 07E9:0113 INT 21 </pre>
<pre> 07E9:0100 MOV AH,02 07E9:0102 MOV DL,BL 07E9:0104 AND DL,0F 07E9:0107 ADD DL,30 07E9:010A CMP DL,39 07E9:010D JLE 0112 07E9:010F ADD DL,07 07E9:0112 INT 21 07E9:0114 INT 20 </pre>	<pre> 07E9:0115 MOV DL,BL 07E9:0117 AND DL,0F 07E9:011A ADD DL,30 07E9:011D CMP DL,39 07E9:0120 JLE 0125 07E9:0122 ADD DL,07 07E9:0125 INT 21 07E9:0127 INT 20 </pre>

Informe o código de programa indicado na coluna da esquerda sem pular linhas em branco a partir do comando **A 0100**: Observe a seguir as instruções a serem definidas:

```

07E9:0100 MOV AH,02 <Enter>
07E9:0102 MOV DL,BL <Enter>
07E9:0104 MOV CL,04 <Enter>
07E9:0106 SHR DL,CL <Enter>
07E9:0108 ADD DL,30 <Enter>
07E9:010B CMP DL,39 <Enter>
07E9:010E JLE 0113 <Enter>
07E9:0110 ADD DL,07 <Enter>
07E9:0113 INT 21 <Enter>
07E9:0115 MOV DL,BL <Enter>
07E9:0117 AND DL,0F <Enter>
07E9:011A ADD DL,30 <Enter>
07E9:011D CMP DL,39 <Enter>
07E9:0120 JLE 0125 <Enter>
07E9:0122 ADD DL,07 <Enter>
07E9:0125 INT 21 <Enter>
07E9:0127 INT 20 <Enter>
07E9:0129 <Enter>

```

Ao rodar o programa com o comando **G**, você obterá na tela o valor definido no registrador menos significativo (**BL**) do registrador geral **BX**. Altere alguns valores para a parte menos significativa do registrador geral **BX**. Se você desejar, execute o programa passo a passo com o comando **P** até a apresentação da mensagem: **Program terminated**.

```

0D82:0100 B402      MOV     AH,02
0D82:0102 B241      MOV     DL,41
0D82:0104 CD21      INT     21
0D82:0106 CD20      INT     20
0D82:0108 69        DB      69

```

Capítulo 6

ENTRADA DE DADOS

*Este capítulo mostra como capturar dados fornecidos via teclado, utilizar rotina do tipo procedimento, usar pilha, a fazer consistência de entrada de dados, além de ler valores numéricos hexadecimais de um e dois dígitos por meio do programa **Enhanced DEBUG**.*

6.1 - Entrada de dados por teclado

Anteriormente os programas desenvolvidos efetuaram operações de processamento e apresentação de sequências de caracteres (mensagem) e dígitos (valores numéricos em hexadecimais) por intermédio do uso da interrupção **21** que na ocasião utilizou as funções hexadecimais de controle de saída: **02** para a saída de um caractere e **09** para a saída de mais de um caractere, ou seja, saída de um *string*. A entrada de dados também utiliza a interrupção **21**, mas com os códigos de função de controle de entrada **01** para a parte alta do registrador geral **AX**. O programa que fará a leitura de um caractere (que deve ser um dígito hexadecimal válido) é semelhante ao programa de apresentação de dados desenvolvido no capítulo anterior.

Para a realização da entrada de dados numéricos ou alfanuméricos, bem como para a saída desses dados o computador por meio de seu microprocessador faz uso da tabela ASCII (ver apêndice A) para efetivar esta ação. Assim sendo, quando se efetua a entrada de certa sequência de dados por meio do teclado o dado inserido é decomposto nos caracteres que o formam e são então convertidos internamente para seus códigos ASCII em hexadecimal. Por exemplo, a entrada da sequência de caracteres "Brasil3000" será decomposta nos caracteres "B", "r", "a", "s", "l", "3", "0", "0" e "0", ou seja, será decomposta em seus códigos hexadecimais ASCII "42h", "72h", "61h", "73h", "69h", "6Ch", "33h", "30h", "30h" e "30h". Após a inserção de cada um dos caracteres via teclado estes são transferidos para a memória e são nela armazenados no formato hexadecimal. Em seguida os dados são lidos da memória em seu formato hexadecimal e são transferidos para o monitor de vídeo, onde são apresentados visualmente para o usuário convertidos no formato dos caracteres efetivamente fornecidos junto ao teclado. A Figura 5.1 demonstra o esquema de ação de entrada de dados.

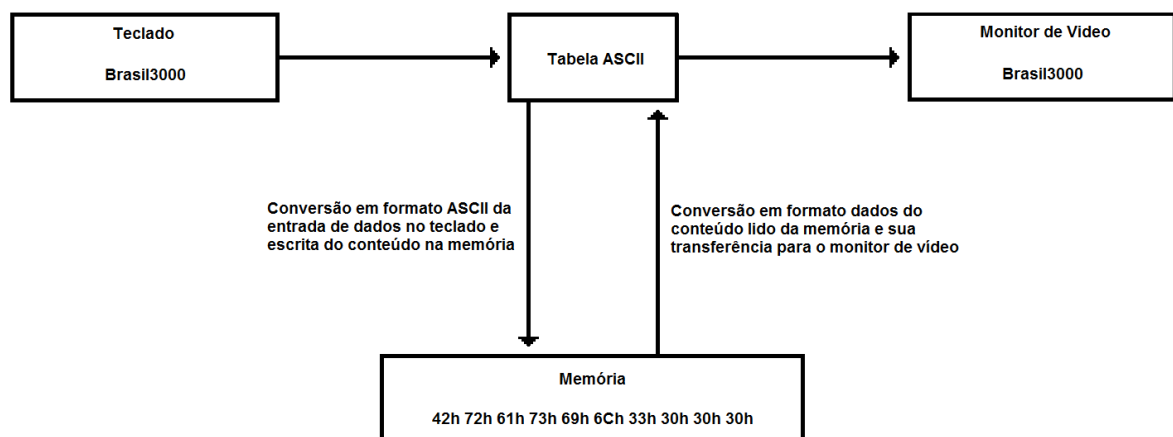


Figura 5.1 - Esquema de ação da entrada de dados.

Antes de iniciar qualquer novo teste apresentado neste capítulo sugere-se sair do programa **Enhanced DEBUG** e inicia-lo novamente para que todos os registradores sejam devidamente inicializados em seus valores padrão.

Em seguida execute a instrução **A 0100** e informe o código seguinte de programa. Atente para o código expresso em *Assembly* no lado esquerdo. O código expresso no lado direito está sendo indicado em *opcodes*:

07E9:0100 MOV AH,01	B4 01
07E9:0102 INT 21	CD 21
07E9:0104 SUB AL,30	2C 30
07E9:0106 CMP AL,09	3C 09
07E9:0108 JLE 010C	7E 02
07E9:010A SUB AL,07	2C 07
07E9:010C INT 20	CD 20

A primeira linha do programa (**07E9:0100**) carrega para o registrador mais significativo (**AH**) do registrador geral **AX** o valor hexadecimal **01** que é o código da função de entrada de dados de um teclado.

A segunda linha (**07E9:0102**) executa a interrupção **21**. Neste momento o programa permite que um valor hexadecimal seja informado e automaticamente armazenado na parte menos significativa (**AL**) do registrador geral **AX**. Ao fazer a entrada de um valor hexadecimal entre **0** e **F**, ele será capturado de forma direta, ou seja, não é necessário acionar a tecla **<Enter>**.

A terceira linha (**07E9:0104**) faz a subtração do valor hexadecimal **30**. Por exemplo, ao entrar o valor hexadecimal **05** pelo teclado, ele é armazenado no registrador **AL** com o valor hexadecimal **35** que é o código hexadecimal ASCII referente ao caractere **"5"**, ou seja, ao acionar a tecla **<5>** no teclado, o seu código ASCII é que, na verdade, é armazenado. É preciso tirar o valor hexadecimal **30** para obter o valor correto da tecla acionada.

A quarta linha (**07E9:0106**), por meio da instrução **CMP**, faz a subtração do valor hexadecimal **09** em relação ao conteúdo existente no registrador **AL** e atualiza os *flags* de controle, fornecendo o parâmetro base para a execução da quinta linha de código. O valor **09** no registrador **AL** refere-se ao limite máximo dos dígitos hexadecimais aceitos, que vão de **0** a **9**. Os demais valores são representados pelas letras de **A** até **F**.

A quinta linha (**07E9:0108**), por meio da instrução **JLE**, desvia o código de execução do programa para a linha **010C** caso o valor de **CMP** (quarta linha) seja menor ou igual a **09**. Valores hexadecimais de **0** até **9** serão desviados para a linha de código **010C**. Os valores de **A** até **F** não serão desviados por essa linha, transferindo a execução do programa para a próxima linha (**010A**).

A sexta linha (**07E9:010A**) subtrai do registrador **AL** o valor hexadecimal **07**, além do valor **30** já subtraído pela linha de código **0104**. Isso é necessário, uma vez que na tabela ASCII os valores numéricos de **0** até **9** estão a uma distância de sete posições em relação aos caracteres de **A** até **F**, como pode ser constatado no trecho indicado junto à Tabela 6.1.

Tabela 6.1 - Valores numéricos versus valores caracteres

Caractere	Código ASCII (Hexadecimal)	Caractere	Código ASCII (Hexadecimal)
0	30	8	38
1	31	9	39
2	32	A	41
3	33	B	42
4	34	C	43
5	35	D	44
6	36	E	45
7	37	F	46

Por último a sétima linha (**07E9:010C**) executa a interrupção **20** a qual devolve o controle ao sistema operacional.

Verifique se o ponteiro de deslocamento **IP** está apontando para o endereço de deslocamento **0100**. Antes de iniciar a execução do programa, acione o comando **R** e observe se a informação de tela é semelhante à indicada a seguir:

```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 B401          MOV     AH,01
```

Para executar passo a passo pode-se Fazer uso do comando **T**, mas agora será utilizado outro comando para a mesma ação. Neste caso, o comando **P** (*Proceed*). Assim sendo execute a instrução:

P <Enter>

Será apresentada a seguinte informação, como mostra os trechos em negrito:

```
AX=0100 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 CD21          INT     21
```

O registrador **AX** passa a ter o valor **0100**, sendo o valor **01** o código de função de entrada de dados. Na sequência acione novamente o comando **P** com a instrução:

P <Enter>

Observe que aparentemente nada acontece. Neste momento forneça um valor hexadecimal válido. Para um teste forneça o valor **5**, com a instrução:

5 <Enter>

Observe na sequência a apresentação dos valores nos registradores:

```
AX=0135 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0104 NV UP EI PL NZ NA PO NC
07E9:0104 2C30          SUB     AL,30
```

O registrador **AX** está com o valor **0135**, sendo o valor **01** o código da função de entrada de dados e **35** o valor ASCII referente ao caractere **5**. Está sendo indicada a execução da próxima instrução (**SUB AL,30**) que tirará do registrador **AL** que é **35** o valor **30**. Na sequência execute a instrução:

P <Enter>

Observe os pontos em negrito:

```
AX=0105 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0106 NV UP EI PL NZ NA PE NC
07E9:0106 3C09          CMP     AL,09
```

O registrador geral **AX** está atualizado e a próxima instrução efetua a comparação (subtração) do valor **09** com o valor armazenado no registrador **AL**. Acione mais uma vez o comando **P**:

P <Enter>

Observe os pontos em negrito:

```
AX=0105 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0108 NV UP EI NG NZ AC PE CY
07E9:0108 7E02          JLE     010C
```


Uma forma de saber a ação do comando **CMP** é observar as informações da sequência de *flags* do registrador de estado. Ao fazer a subtração (**CMP AL,09**), o valor **09** é subtraído do valor **05** que gera um resultado no registrador de estado de um valor negativo, e é esse valor que será usado como parâmetro de desvio para o próximo comando, como indica a próxima linha de código: **07E9:0108 7E02 JLE 010C**. Na sequência acione o comando **P** novamente:

P <Enter>

Observe em negrito a ocorrência do desvio para a linha **010C** saltando a execução da linha **010A**:

```
AX=0105 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010C NV UP EI NG NZ AC PE CY
07E9:010C CD20                INT      20
```

A partir deste ponto o programa já está quase encerrado. Ao acionar mais uma vez o comando **P**, ele apresenta a mensagem **"Program terminated (0000)"** por ter atingido a linha de código **INT 20**.

Procure fazer alguns testes com o comando **P** para alguns valores hexadecimais de um dígito apenas. Lembre-se de ir ajustando o endereço de deslocamento **0100** para o registrador de deslocamento **IP** com a instrução **R IP 0100**, antes de iniciar um novo teste.

A partir deste ponto já é possível, com certo grau de facilidade, desenvolver uma rotina de programa que leia um valor hexadecimal de dois dígitos. Essa rotina é similar à rotina de saída do capítulo anterior, mas exige um controle operacional um pouco maior.

Para auxiliar essa nova tarefa, será utilizado o comando **SHL** (*Shift Logical Left*), que possui sua ação contrária ao modo como o comando **SHR** opera. O comando **SHL** efetua a rotação de *bits* no sentido da esquerda, operando em estruturas de dados do tipo *byte* e *word* e quando executada altera o estado dos registradores de estados (*flags*) **CF**, **OF**, **PF**, **SF** e **ZF**, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador, 1 (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, 1 (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ registrador, CL (para este tipo de ação usa 2 *bytes* de memória);
- ◆ memória, CL (para este tipo de ação usa de 2 a 4 *bytes* de memória).

Basicamente se fará com a rotina de programa de entrada de dois dígitos o sentido inverso realizado pela rotina de saída de dois dígitos. A partir do endereço de deslocamento **0100** com o comando **A** informe o código seguinte definido do lado esquerdo, pois ao lado direito estão indicados os *opcodes* dos comandos apresentados do lado esquerdo:

07E9:0100 MOV AH,01	B4 01
07E9:0102 INT 21	CD 21
07E9:0104 MOV DL,AL	8A D0
07E9:0106 SUB DL,30	80 EA 30
07E9:0109 CMP DL,09	80 FA 09
07E9:010C JLE 0111	7E 03
07E9:010E SUB DL,07	80 EA 07
07E9:0111 MOV CL,04	B1 04
07E9:0113 SHL DL,CL	D2 E2
07E9:0115 INT 21	CD 21
07E9:0117 SUB AL,30	2C 30
07E9:0119 CMP AL,09	3C 09
07E9:011B JLE 011F	7E 02
07E9:011D SUB AL,07	2C 07
07E9:011F ADD DL,AL	02 D0
07E9:0121 INT 20	CD 20

Antes de escrever a rotina de programa anterior, informe o valor **0000** para o registrador **AX** (**R AX 0000**). Estabeleça o endereço de deslocamento **0100** para o registrador de deslocamento **IP**, em seguida execute o comando **R** (**R IP 0100**) e verifique se as posições de memória em uso são equivalentes aos dados seguintes a partir da execução do comando **R**:


```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 B401          MOV     AH,01
```

Na sequência acione o comando **P** para que a primeira linha do programa (**07E9:0100**) seja executada, como é apresentado em seguida. Note também o apontamento para a execução da segunda linha de código do programa **07E9:0102**:

```
AX=0100 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 CD21          INT     21
```

O código de função de entrada de dados **01** é estabelecido para a parte mais significativa (**AH**) do registrador geral **AX**. Acione novamente o comando **P** e entre o primeiro dígito de um valor de duas posições.

Para este teste será fornecido o valor hexadecimal **C9**. Entre o caractere **C** (em formato maiúsculo) e automaticamente ocorre um salto da execução do programa da linha **0102** para a linha **0104**, mostrando a seguinte posição de memória:

```
CAX=0143 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0104 NV UP EI PL NZ NA PO NC
07E9:0104 88C2          MOV     DL,AL
```

O valor **43** (valor hexadecimal correspondente à letra **C** na tabela ASCII) é armazenado na parte menos significativa (**AL**) do registrador geral **AX**. Acione o comando **P**.

```
AX=0143 BX=0000 CX=0000 DX=0043 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0106 NV UP EI PL NZ NA PO NC
07E9:0106 80EA30        SUB     DL,30
```

O valor **43** foi copiado (**MOV DL,AL**) para dentro da parte menos significativa (**DL**) do registrador geral **DX**. Em seguida acione novamente o comando **P** e observe a alteração do valor dentro do registrador geral **DX**.

```
AX=0143 BX=0000 CX=0000 DX=0013 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0109 NV UP EI PL NZ NA PO NC
07E9:0109 80FA09        CMP     DL,09
```

Agora que a parte menos significativa do registrador geral **DX** está com o valor **13**, ele será comparado (**CMP DL,09**), que resultará na subtração do valor **09** do valor **13** do registrador geral **DX**, o qual será avaliado pelo desvio condicional da linha de código **07E9:0109**. Acione o comando **P** e observe a alteração das informações nos registradores de estado:

```
AX=0143 BX=0000 CX=0000 DX=0013 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010C NV UP EI PL NZ AC PE NC
07E9:010C 7E03          JLE     0111
```

A linha de código **07E9:010C** em que se encontra a instrução **JLE 0111** desvia o programa para a linha de código **07E9:0111** caso o resultado da instrução **CMP DL,09** executada na linha de código **07E9:0109** seja menor ou igual a zero. Para verificar essa ação, utilize o comando **P** e observe os detalhes marcados em negrito a seguir:

```
AX=0143 BX=0000 CX=0000 DX=0013 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010E NV UP EI PL NZ AC PE NC
07E9:010E 80EA07        SUB     DL,07
```

A condição de execução da linha de programa **07E9:010C** é considerada falsa. A execução do programa para a linha de código **07E9:010E** fará a subtração do valor **07** do valor existente na parte menos significativa do registrador geral **DX**. Para observar esta ação, acione o comando **P**.

```
AX=0143 BX=0000 CX=0000 DX=000C SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0111 NV UP EI PL NZ AC PE NC
07E9:0111 B104          MOV     CL,04
```

Observe o armazenamento do valor hexadecimal **C** no registrador geral **DL**, sendo este a representação do valor hexadecimal do caractere **C** informado na ação da linha de programa **07E9:0102**.

A partir deste ponto o programa faz o tratamento da entrada do segundo dígito do valor hexadecimal **C9**. Acione o comando **P** e observe o armazenamento do valor **04** na parte menos significativa (**CL**) do registrador geral **CX**.

```
AX=0143 BX=0000 CX=0004 DX=000C SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0113 NV UP EI PL NZ AC PE NC
07E9:0113 D2E2          SHL     DL,CL
```

Lembre-se de que se utiliza o registrador geral **CX** normalmente quando há necessidade de fazer a contagem de algum passo de ação. Neste caso objetiva-se mover em quatro *bits* (uma posição - um *nibble*) o conteúdo do registrador **DX**, ou seja, mover o valor **C** da parte menos significativa uma posição para a esquerda com o comando **SHL** que se encontra na linha de código **07E9:0113**. Acione o comando **P**:

```
AX=0143 BX=0000 CX=0004 DX=00C0 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0115 NV UP EI NG NZ NA PE NC
07E9:0115 CD21          INT     21
```

Observe a posição em que se encontra o valor **C** no registrador geral **DX**. A primeira posição mais à direita do registrador ficou disponível para o armazenamento do próximo valor, que será entrado pela instrução **INT 21** da linha de código **07E9:0115**. Para visualizar essa ação, acione o comando **P** e entre o valor **9** e observe as partes sinalizadas em negrito que se sucedem a ação:

```
9AX=0139 BX=0000 CX=0004 DX=00C0 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0117 NV UP EI NG NZ NA PE NC
07E9:0115 2C30          SUB     AL,30
```

Observe que a parte menos significativa do registrador geral **AX** (que antes da ação possuía o valor **0143** e passa a ter o valor **0139**, mas o que interessa é o valor da parte menos significativa, ou seja, o valor hexadecimal **39**). O valor hexadecimal **39** armazenado na parte menos significativa do registrador geral **AX** corresponde ao código ASCII para a representação do valor numérico **9**.

Em seguida acione o comando **P** para visualizar a ação do comando **SUB AL,30** que fará a subtração do valor hexadecimal **30** do valor atual do registrador **AL**. Observe o ponto marcado em negrito:

```
AX=0109 BX=0000 CX=0004 DX=00C0 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0119 NV UP EI PL NZ NA PE NC
07E9:0119 3C09          CMP     AL,09
```

O registrador geral **AX** tem na sua parte menos significativa o valor hexadecimal **09**. Em seguida, a linha de código **07E9:0119** calcula a subtração interna para definição da comparação, que resulta em um valor **00** e faz com que a instrução da linha de código **07E9:011B** execute o desvio condicional indicado. Acione o comando **P** e observe atentamente os valores do registrador de estado:

```
AX=0109 BX=0000 CX=0004 DX=00C0 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=011B NV UP EI PL ZR NA PE NC
07E9:011B 7E02          JLE     011F
```

Nessa etapa (linha de código **07E9:011B**) o comando **JLE 011F** desvia o programa para a linha de código **07E9:011F**, a qual executa a instrução **ADD DL,AL**. Acione o comando **P** e observe atentamente este detalhe:

```
AX=0109 BX=0000 CX=0004 DX=00C0 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=011F NV UP EI PL ZR NA PE NC
07E9:011F 00C2          ADD     DL,AL
```

Ao ser executado o comando **P** nessa etapa, ocorre a movimentação do valor **09** existente no registrador **AL** para dentro do registrador **DL**. Perceba que após essa ação o registrador geral **DX** ficará com o valor **00C9** referente à entrada **C9**:

```
AX=0109 BX=0000 CX=0004 DX=00C9 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0121 NV UP EI NG NZ NA PE NC
07E9:0121 CD20          INT     20
```

O próximo acionamento do comando **P** finaliza a execução do programa em uso.

6.2 - Utilização de procedimento

O *procedimento* utilizado em linguagem de programação de computadores *Assembly* tem um significado similar às sub-rotinas existentes em linguagens de programação de computadores de alto nível, ou seja, um procedimento é um trecho de código de programa definido em uma determinada área de memória que pode ser chamado e utilizado várias vezes.

Para utilizar o procedimento em linguagem de programação *Assembly*, é necessário trabalhar com dois comandos adicionais: **CALL** (*Call procedure*) que ocupa de 2 até 5 bytes de memória e **RET** (*Return*) que ocupa de 1 a 3 bytes de memória. Os comandos **CALL** e **RET** não afetam nenhum registrador de estado.

Imagine um programa que apresente em vídeo a sequência de números **0123456789**. Saia do programa **Enhanced DEBUG** e faça um novo carregamento em memória, depois informe as linhas de código a seguir posicionadas do lado esquerdo, lembrando que ao lado direito estão os códigos em *opcodes*:

E 0200 "0123456789" 24 A 0100 07E9:0100 MOV AH,09 07E9:0102 MOV DX,0200 07E9:0105 INT 21 07E9:0107 INT 20	30 31 32 33 34 35 36 37 38 39 24 B4 09 BA 00 02 CD 21 CD 20
--	---

Ao final peça a execução do comando **G** para comprovar o resultado da saída pretendida. No entanto, o objetivo é apresentar caractere por caractere até concluir a apresentação dos dados **0123456789**.

Primeiramente será desenvolvido um programa que não faz uso da técnica de procedimento, em seguida será apresentado uma versão do mesmo programa com uso da técnica de procedimento.

A partir da instrução **A 0100** informe o código seguinte definido do lado esquerdo, lembrando que do lado direito estão indicados os códigos *opcodes* dos comandos apresentados do lado esquerdo:

07E9:0100 MOV AH,02 07E9:0102 MOV DL,30 07E9:0104 INT 21 07E9:0106 MOV DL,31 07E9:0108 INT 21 07E9:010A MOV DL,32 07E9:010C INT 21 07E9:010E MOV DL,33 07E9:0110 INT 21	B4 02 B2 30 CD 21 B2 31 CD 21 B2 32 CD 21 B2 33 CD 21
--	---

07E9:0112	MOV DL,34	B2 34
07E9:0114	INT 21	CD 21
07E9:0116	MOV DL,35	B2 35
07E9:0118	INT 21	CD 21
07E9:011A	MOV DL,36	B2 36
07E9:011C	INT 21	CD 21
07E9:011E	MOV DL,37	B2 37
07E9:0120	INT 21	CD 21
07E9:0122	MOV DL,38	B2 38
07E9:0124	INT 21	CD 21
07E9:0126	MOV DL,39	B2 39
07E9:0128	INT 21	CD 21
07E9:012A	INT 20	CD 20

Para cada valor apresentado é feita uma chamada à interrupção **21h (INT 21)** para que o caractere correspondente ao código hexadecimal seja apresentado na tela do monitor de vídeo. Ao executar o programa com o comando **G**, observe a saída da informação **0123456789**.

Imagine então apresentar os 256 caracteres da tabela ASCII desta forma. Para facilitar esse tipo de trabalho é que entra em ação o *procedimento*. A partir do endereço de deslocamento **0100** será fornecido o código do programa principal. Utilizando o comando **A**, informe o código a seguir indicado do lado esquerdo:

07E9:0100	MOV AH,02	B4 02
07E9:0102	MOV CL,0A	B1 0A
07E9:0104	MOV DL,30	B2 30
07E9:0106	CALL 0200	E8 F7 00
07E9:0109	LOOP 0106	E2 FB
07E9:010B	INT 20	CD 20

A partir do endereço de deslocamento **0200** é fornecido o código de programa correspondente ao trecho da rotina de procedimento. Utilizando o comando **A**, informe o código a seguir indicado do lado esquerdo:

07E9:0200	INT 21	CD 21
07E9:0202	INC DL	FE C2
07E9:0204	RET	C3

Na sequência execute o comando **G** e será apresentada a mensagem:

```
0123456789
Program terminated (0000)
```

A rotina principal (**0100** até **010B**) do programa define na primeira linha de código (**07E9:0100**) a apresentação de um único caractere. Depois na segunda linha (**07E9:0102**) é definido o valor **0A** (valor decimal **10**) que será usado para contar o laço executado pela linha de código (**07E9:0109**). Toda vez que o comando **LOOP** é executada, ocorre um decremento de **1** no valor armazenado no registrador **CL**.

A terceira linha do programa principal (**07E9:0104**) estabelece o valor do primeiro caractere a ser apresentado (no caso o caractere **0** – zero representado pelo código ASCII do caractere zero como **30h**). A quarta linha de código do programa principal (**07E9:0106**), por meio do comando **CALL**, faz a chamada do trecho secundário do programa, ou seja, de uma sub-rotina.

Antes da execução do comando **CALL** ocorre a atualização do registrador **IP** com o endereço no qual se encontra a primeira instrução após a chamada do comando **CALL** que representa o valor de endereço de retorno da sub-rotina a ser utilizado pelo comando **RET**. Dependendo do tipo de procedimento em uso ter-se-á a atualização do registrador **CS** com o endereço do segmento de memória.

A chamada de uma sub-rotina pode ser feita de forma *direta* ou *indireta*, tanto na modalidade *intra-segmento* (**NEAR**) como na modalidade *inter-segmento* (**FAR**). Uma chamada intra-segmento ocorre no mesmo segmento de memória e uma chamada intra-segmento ocorre em segmentos diferentes de memória.

O uso de procedimentos faz com que o recurso de pilha da memória seja usado, onde *pilha* é uma região da memória que efetua o armazenamento temporário de dados.

Na chamada **direta intra-segmento** o registrador **SP** é subtraído de **02h**, é colocado o valor do registrador **IP** na pilha e efetua-se a soma do deslocamento entre o destino e a instrução seguinte a definição do comando **CALL** ao registrador **IP**.

Na chamada **direta inter-segmento** o registrador **SP** é subtraído de **02h**, é colocado o valor do registrador **CS** na pilha, o registrador **IP** é subtraído de **02h**, é colocado o valor do registrador **IP** na pilha e carrega-se os registradores **CS** e **IP** com o endereço de destino.

Na chamada **indireta intra-segmento** o registrador **SP** é subtraído de **02h**, é colocado o valor do registrador **IP** na pilha e carrega-se o registrador **IP** com o conteúdo do operando.

Na chamada **indireta inter-segmento** o registrador **SP** é subtraído de **02h**, é colocado o valor do registrador **CS** na pilha, o registrador **IP** é subtraído de **02h**, coloca-se o valor do registrador **CS** na pilha e carrega-se os registradores **CS** e **IP** com o conteúdo dos operados.

Os tipos de chamadas de um procedimento em *Assembly* dependem do tipo de operando em uso, que podem ser:

- ◆ rótulo **NEAR** usa ação **direta intra-segmento**;
- ◆ rótulo **FAR** usa ação **direta inter-segmento**;
- ◆ **registrador** usa ação **indireta intra-segmento**;
- ◆ variável **word** usa ação **indireta intra-segmento**;
- ◆ variável **dword** usa ação **indireta inter-segmento**.

Na primeira linha do trecho secundário do programa (**07E9:0200**) é feita a interrupção **21** que apresenta o caractere correspondente ao código ASCII armazenado no registrador geral **DX**. Na segunda linha (**07E9:0202**) do programa secundário encontra-se a instrução **INC** (*increment*), que faz o incremento de **1** ao valor atual armazenado no registrador geral **DX**.

A título de ilustração a instrução **INC** faz o incremento de mais um, a operação inversa pode ser realizada com o uso da instrução **DEC**.

Observe a terceira linha (**07E9:0204**) do programa secundário que executa a instrução **RET**, a qual retorna a execução do programa para a próxima linha do programa principal, após a instrução **CALL**.

O comando **RET** carrega o registrador **IP** com o valor (conteúdo) que estiver no topo da pilha, incrementa com **02h** o registrador **SP**. Se o retorno vier de uma chamada do tipo inter-segmento carrega o registrador **CS** com o valor do topo da pilha e adiciona **02h** ao registrador **SP**.

É muito importante se tomar o cuidado nas operações com procedimentos de não se alterar o valor do topo da pilha, o qual representa o valor do endereço de retorno para o programa principal. Se este valor for alterado o programa perderá a referência de retorno.

6.3 - Utilização da pilha

Quando os comandos **CALL** e **RET** chamam, respectivamente, um procedimento e seu posterior retorno, elas utilizam como parâmetro de controle operacional as informações armazenadas nos registradores de segmento **CS** (*Code Segment*), **DS** (*Data Segment*), **ES** (*Extra Data Segment*) e **SS** (*Stack Segment*), em conjunto com as informações do registrador de apontamentos **SP** (*Stack Pointer*). A pilha é usada para manter o endereço de retorno de uma instrução quando realizada a chamada de um procedimento ou para manter armazenado qualquer dado manipulado pelos comandos **PUSH** e **POP**, que serão vistos mais adiante.

As principais ações efetuadas pela pilha são: passar parâmetros para sub-rotinas como procedimentos e funções; efetuar a chamada de sub-rotinas e retornar as ações dessas sub-rotinas; preservar os valores dos registradores quando utilizados por sub-rotinas; preservar dados na memória; transferir dados sem que sejam utilizados os registradores; alterar a ordem de dados armazenados.

Para visualizar o controle da operação de manipulação da pilha, o programa atual é executado com auxílio dos comandos **T** e **P**. Antes de iniciar a execução do programa, acione o comando **R** e observe os pontos marcados em negrito, os quais indicam os registradores que são usados para as ações de controle da pilha:

```
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 B402          MOV     AH,02
```

Os registradores de segmento **DS**, **ES**, **SS** e **CS** apontam para o segmento de memória **07E9** e o registrador de apontamento **SP** tem o valor hexadecimal **FFFE** (ou outro endereço qualquer que representa o topo da pilha dos registradores de segmento, sendo este valor apresentado pelo próprio programa **Enhanced DEBUG**).

Os valores dos registradores de segmento **DS**, **ES**, **SS** e **CS** não serão alterados por estar sendo executada a chamada de procedimento no modo intra-segmento. Caso a execução da chamada de um procedimento ocorra fora do segmento apontado por **DS**, **ES**, **SS** e **CS** este será em modo inter-segmento.

Nessa etapa inicial acione o comando **P** por três vezes e observe as informações dos registradores de segmento e de apontamento marcadas em negrito mantendo-se inalteradas:

```
- P
AX=0200 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0100 B10A          MOV     CL,0A
```

```
- P
AX=0200 BX=0000 CX=000A DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0104 NV UP EI PL NZ NA PO NC
07E9:0104 B230          MOV     DL,30
```

```
- P
AX=0200 BX=0000 CX=000A DX=0030 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0106 NV UP EI PL NZ NA PO NC
07E9:0106 E8F700       CALL     0200
```

Após a terceira etapa de execução do comando **P** a linha de código **07E9:0106** aponta para a chamada do procedimento por meio do comando **CALL 0200**. Neste momento execute o comando **T**.

```
AX=0200 BX=0000 CX=000A DX=0030 SP=FFFC BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0200 NV UP EI PL NZ NA PE NC
07E9:0200 CD21          INT      21
```

O registrador de apontamento **IP** mostra o valor **0200** que é o valor inicial da rotina do procedimento. Veja também a informação do registrador de apontamento **SP** que anteriormente mostrava o valor **FFFE** e agora apresenta o valor **FFFC** (lembrando que esses valores poderão ser diferentes no computador em uso). Acione o comando **P** mais três vezes e observe os pontos em negrito:

```
- P
0AX=0230 BX=0000 CX=000A DX=0030 SP=FFFC BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0202 NV UP EI PL NZ NA PE NC
07E9:0202 FEC2          INC      DL
```

```
- P
AX=0230 BX=0000 CX=000A DX=0031 SP=FFFC BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0204 NV UP EI PL NZ NA PO NC
07E9:0204 C3           RET
```

-P

```
AX=0230 BX=0000 CX=000A DX=0031 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0109 NV UP EI PL NZ NA PO NC
07E9:0109 E2FB                LOOPW    0106
```

Na última execução do comando **P** o valor do registrador de apontamento **SP** volta a ser **FFFE** e o valor do registrador de apontamento **IP** passa de **0204** para **0109**.

Repita a ação descrita mais algumas vezes observando vagarosamente os detalhes indicados. Os comandos **CALL** e **RET** operam de acordo com as informações dos registradores de segmento e apontamento. **CALL** faz a chamada de um procedimento e armazena o endereço desta chamada na pilha, a instrução **RET** recebe o valor de endereço da pilha e retorna o fluxo de ação do programa de acordo com o deslocamento (*offset*). O mesmo efeito ocorre com o uso do comando **INT** que faz a chamada de sub-rotinas internas para efetuar seu controle operacional.

O uso da pilha é um processo que facilita muitas operações de manipulação de valores (*byte* ou *word*), pois é possível armazenar valores na pilha ou retirá-los via programação pelas instruções específicas para essa finalidade, tais como **PUSH** (envia um dado para a pilha) e **POP** (retira um dado da pilha). Os comandos **PUSH** e **POP** são outra forma de efetuar movimentação de dados na memória.

O comando **PUSH** quando executado não altera nenhum registrador de estado, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador (para este tipo de ação usa 1 *byte* de memória);
- ◆ memória (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ segmento_registro, memória (para este tipo de ação usa 1 *byte* de memória);
- ◆ constante (para este tipo de ação usa de 2 a 3 *bytes* de memória).

O comando **POP** quando executado não altera nenhum registrador de estado, possuindo como possibilidade de trabalho as operações sobre:

- ◆ registrador (para este tipo de ação usa 1 *byte* de memória);
- ◆ memória (para este tipo de ação usa de 2 a 4 *bytes* de memória);
- ◆ segmento_registro, memória (para este tipo de ação usa 1 *byte* de memória);

Manipular uma pilha de valores é útil em situações nas quais é preciso guardar um valor de um determinado registrador antes de um procedimento ser executado e recuperar o valor antes da finalização do procedimento.

Para demonstrar essa ação, saia do programa **Enhanced DEBUG** e carregue-o em seguida novamente na memória. Depois entre o valor hexadecimal **000A** para o registrador geral **AX** e o valor **000B** para o registrador geral **BX**, de forma semelhante à seguinte informação:

```
R AX 000A <Enter>
R BX 000B <Enter>
R IP 0100 <Enter>
```

Após execute o comando **R**:

```
AX=000A BX=000B CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL NZ NA PO NC
07E9:0100 C3                RET
```

A partir do endereço de deslocamento **0100** entre com o comando **A** o código do programa principal indicado a seguir e posicionado do lado esquerdo:

07E9:0100	PUSH AX	50
07E9:0101	PUSH BX	53
07E9:0102	CALL 0200	E8 FB 00
07E9:0105	INC AX	40
07E9:0106	INC BX	43
07E9:0107	CALL 0200	E8 F6 00
07E9:010A	POP BX	5B
07E9:010B	POP AX	58
07E9:010C	INT 20	CD 20

Na sequência, a partir do endereço de deslocamento **0200** com o comando **A** entre o código da rotina de procedimento indicado a seguir e posicionado do lado esquerdo:

07E9:0200	MOV AX,0001	B8 01 00
07E9:0203	MOV BX,0002	BB 02 00
07E9:0206	INC AX	40
07E9:0207	INC BX	43
07E9:0208	RET	C3

No código do programa principal a primeira (**07E9:0100**) e a segunda (**07E9:0101**) linhas guardam na pilha os valores atuais dos registradores gerais **AX** e **BX**.

Na terceira linha (**07E9:0102**) do programa principal é feita a chamada pela instrução **CALL** da rotina de procedimento definida no endereço de deslocamento **0200**.

Dentro do trecho da rotina de procedimento o programa move os valores **0001** e **0002**, respectivamente, para os registradores gerais (primeira linha **07E9:0200**) **AX** e (segunda linha **07E9:0203**) **BX**. Nessa etapa de execução os registradores gerais **AX** e **BX** perdem os valores originais e assumem os novos valores devido ao uso da instrução **MOV**.

Depois na terceira (**07E9:0206**) e quarta (**07E9:0207**) linhas da rotina de procedimento o programa incrementa em **1** os valores existentes nos registradores gerais **AX** e **BX**. A quinta linha (**07E9:0208**) da rotina de procedimento executa a instrução **RET** que devolve o fluxo de execução do programa para a primeira linha do programa principal após a execução do comando **CALL**. Após a primeira chamada do procedimento o retorno ocorre na quarta linha do programa principal (**07E9:0105**).

A quarta (**07E9:0105**) e quinta (**07E9:0106**) linhas do programa principal incrementam o valor **1** aos valores atuais dos registradores gerais **AX** e **BX**. Depois na sexta linha (**07E9:0107**) do programa principal faz nova chamada à rotina de procedimento, que repete sua ação incrementando o valor **1** ao valor atual dos registradores gerais **AX** e **BX** e executando o comando **RET**. Após a segunda chamada da rotina de procedimento o retorno ocorre na sétima linha (**07E9:010A**) do programa principal.

De volta ao programa principal o comando **POP** encontrada na sétima (**07E9:010A**) e oitava (**07E9:010B**) linhas do programa recupera da pilha os valores iniciais dos registradores gerais **AX** e **BX**, ou seja, os registradores gerais **AX** e **BX** voltam, respectivamente, a ter os valores originais **000A** e **000B**. O uso do comando **POP** sempre ocorre no sentido inverso ao uso do comando **PUSH**. Por exemplo, se executada as instruções **PUSH AX**, **PUSH BX**, **PUSH CX** para guardar na pilha os valores dos registradores **AX**, **BX** e **CX** e desejando-se retornar os valores da pilha para seus respectivos registradores deverá ser usada a sequência de instruções **POP CX**, **POP BX** e **POP AX**.

A seguir, a partir do registrador de deslocamento **IP** posicionado no endereço de deslocamento **0100**, execute o comando **R** uma vez para verificar se os registradores gerais **AX** e **BX** estão com os valores **000A** e **000B**, atentando para o valor **FFFE** do registrador **SP**.

```
AX=000A BX=000B CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0100 NV UP EI PL ZR NA PO NC
07E9:0100 50          PUSH    AX
```

Observe cuidadosamente cada informação apresentada, principalmente nos pontos marcados em negrito. Na sequência execute o comando **T** para que a primeira linha do programa principal seja executada.


```
AX=000A BX=000B CX=0000 DX=0000 SP=FFFC BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0101 NV UP EI PL ZR NA PO NC
07E9:0101 53          PUSH    BX
```

O valor do registrador de apontamento **SP** mudou de **FFFE** para **FFFC**. A mudança ocorreu pelo fato de o valor **000A** do registrador geral **AX** ter sido armazenado no topo da pilha. Houve uma atualização do valor para determinar o novo topo da pilha. Na sequência execute o comando **T** para que a segunda linha do programa principal seja executada.

```
AX=000A BX=000B CX=0000 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0102 NV UP EI PL NZ NA PO NC
07E9:0102 E8FB00      CALL    0200
```

O valor do registrador de apontamento **SP** mudou de **FFFC** para **FFFA**. A mudança ocorreu pelo fato de o valor **000B** do registrador geral **BX** ter sido armazenado no topo da pilha. Houve uma atualização do valor para determinar o novo topo da pilha. Toda vez que for usado o comando **PUSH** o registrador de apontamento **SP** será atualizado para determinar o próximo topo da pilha.

A próxima instrução a ser executada pelo programa principal é a chamada da execução da rotina de procedimento, que se encontra no endereço de deslocamento **0200**. Na sequência execute o comando **T** para que a terceira linha do programa principal seja executada e ocorra o desvio do fluxo de execução do programa para dentro do procedimento.

```
AX=000A BX=000B CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0200 NV UP EI PL NZ NA PO NC
07E9:0200 B80100      MOV     AX,0001
```

Ao ser executada a rotina de procedimento, o programa atualiza o valor do registrador de apontamento **SP** de **FFFA** para **FFF8**, pois o comando **CALL** também utiliza a informação da pilha para sua execução. O valor **FFFA** será restaurado quando da execução do comando **RET**.

Na posição atual do registrador de apontamento **IP** que marca o endereço **0200** encontra-se a primeira instrução da rotina de procedimento. Execute o comando **T** por duas vezes para que a primeira e segunda linhas da rotina de procedimento sejam executadas.

```
- T
AX=0001 BX=000B CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0203 NV UP EI PL NZ NA PO NC
07E9:0203 BB0200      MOV     BX,0002
```

```
- T
AX=0001 BX=0002 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0206 NV UP EI PL NZ NA PO NC
07E9:0206 40          INC     AX
```

Os registradores gerais **AX** e **BX** perdem o valor anterior e assumem um "novo" valor devido ao uso do comando **MOV**. Para preservar os valores originais dos registradores **AX** e **BX** foi utilizada a instrução **PUSH**. Perceba que o valor do registrador **SP** se manteve constante nas duas ações anteriores.

Na sequência execute o comando **T** por mais duas vezes e observe a atualização dos valores dos registradores gerais **AX** e **BX** e o comportamento do registrador de apontamento **SP**.

```
- T
AX=0002 BX=0002 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0207 NV UP EI PL NZ NA PO NC
07E9:0207 43          INC     BX
```

```
- T
AX=0002 BX=0003 CX=0000 DX=0000 SP=FFE8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0208 NV UP EI PL NZ NA PE NC
07E9:0208 C3          RET
```

Nesse ponto, ao ser solicitada a execução da última linha de código do programa de procedimento com o comando **T**, ocorre o retorno à quarta linha do programa principal.

```
AX=0002 BX=0003 CX=0000 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0105 NV UP EI PL NZ NA PE NC
07E9:0105 40          INC     AX
```

Observe nesse ponto a alteração do valor do registrador **SP** que volta a marcar o topo da pilha como sendo **FFFA**, valor este da última posição da pilha antes da chamada da rotina de procedimento. Execute em seguida o comando **T** duas vezes.

```
- T
AX=0003 BX=0003 CX=0000 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0106 NV UP EI PL NZ NA PE NC
07E9:0106 43          INC     BX
```

```
- T
AX=0003 BX=0004 CX=0000 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0107 NV UP EI PL NZ NA PO NC
07E9:0107 E8F600     CALL    0200
```

O endereço do registrador de apontamento **SP** se manteve constante durante os passos anteriores executados no programa principal. Nesse último momento o programa principal aponta para a chamada da rotina de procedimento no endereço de deslocamento **0200**. Nessa etapa acione o comando **T** uma vez.

```
AX=0003 BX=0004 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0200 NV UP EI PL NZ NA PO NC
07E9:0200 B80100     MOV     AX,0001
```

Ao ser executado o acesso à rotina de procedimento, o ponteiro de apontamento **SP** volta a ficar com o valor **FFF8**. Na sequência execute o comando **T** quatro vezes.

```
- T
AX=0001 BX=0004 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0203 NV UP EI PL NZ NA PO NC
07E9:0203 BB0200     MOV     BX,0002
```

```
- T
AX=0001 BX=0002 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0206 NV UP EI PL NZ NA PO NC
07E9:0206 40          INC     AX
```

```
- T
AX=0002 BX=0002 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0207 NV UP EI PL NZ NA PO NC
07E9:0207 43          INC     BX
```

```
- T
AX=0002 BX=0003 CX=0000 DX=0000 SP=FFF8 BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=0208 NV UP EI PL NZ NA PE NC
07E9:0208 C3          RET
```

Perceba a mudança dos valores dos registradores gerais **AX** e **BX**. Em relação à anterior os valores mudaram novamente devido ao uso dos comandos **MOV** e **INC**. Nessa etapa, acione o comando **T** e observe o local de retorno do programa, principalmente na informação do registrador **SP** e na indicação da próxima linha de execução do programa.

```
AX=0002 BX=0003 CX=0000 DX=0000 SP=FFFA BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010A NV UP EI PL NZ NA PE NC
07E9:010A 5B POP BX
```

Acione o comando **T** uma vez. Perceba o retorno do valor **000B** para o registrador geral **BX**. Este foi o último valor inserido na pilha, ou seja, o último valor empilhado é o primeiro a ser desempilhado, pois de outra forma, a operação não pode ser realizada.

```
AX=0002 BX=000B CX=0000 DX=0000 SP=FFFC BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010B NV UP EI PL NZ NA PE NC
07E9:010B 58 POP AX
```

Ao ser retirado um valor da pilha, o registrador de apontamento **SP** é atualizado para **FFFC**. Execute mais uma vez o comando **T**.

```
AX=000A BX=000B CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=07E9 ES=07E9 SS=07E9 CS=07E9 IP=010C NV UP EI PL NZ NA PE NC
07E9:010C CD20 INT 20
```

Note o retorno do valor **000A** para o registrador geral **AX** e o ajuste do valor do registrador de apontamento **SP** para **FFFE**, valor inicial antes da execução da primeira linha de código do programa principal. Depois, para finalizar o programa, acione apenas uma vez o comando **P**.

A pilha permite manipular a preservação de valores, dando a possibilidade de trabalhar em baixo nível do mesmo modo que em linguagens de programação de alto nível, quando se utilizam passagens de parâmetro por valor ou referência em sub-rotinas.

6.4 - Consistência da entrada de dados

A explanação do tópico anterior foi necessária, uma vez que não se deve confiar na ação de um usuário de programa. Devido a esta questão, o desenvolvedor de um programa precisa se preocupar com alguns detalhes, como o que aconteceria na entrada de valores hexadecimais se ele entrasse um caractere inválido, diferente da sequência **0123456789ABCDEF**.

O objetivo do próximo programa é apenas aceitar a entrada de caracteres válidos, ou seja, valores numéricos de 0 até 9 e caracteres alfabéticos de A até F em maiúsculos. Qualquer outro caractere não será aceito pelo programa.

O programa seguinte é formado por três partes de código, sendo uma parte o programa principal e as outras duas partes as rotinas de procedimento (uma secundária e outra terciária).

A rotina principal do programa é responsável por todo o controle aceitando até dois dígitos válidos. Para a definição dessa rotina utilize o comando **A** e a partir do endereço de deslocamento **0100**, informe o código a seguir indicado do lado esquerdo:

07E9:0100	SUB DX,DX	29 D2
07E9:0102	CALL 0200	E8 FB 00
07E9:0105	MOV DL,AL	88 C2
07E9:0107	MOV CL,04	B1 04
07E9:0109	SHL DL,CL	D2 E2
07E9:010B	CALL 0200	E8 F2 00
07E9:010E	INT 20	CD 20

A primeira instrução do programa efetua a limpeza do registrador geral **DX** com a instrução **SUB DX,DX** eliminando eventualmente qualquer valor que exista na posição preparando o registrador para as ações do programa.

A segunda instrução **CALL 0200** efetua a chamada do procedimento que se encontra no endereço de deslocamento **0200** (procedimento secundário) responsável pela leitura do primeiro caractere advindo do teclado. O valor numérico aceito precisa ser reposicionado na memória para que a segunda entrada ocorra. Esta ação é realizada pelas próximas três instruções.

A terceira instrução **MOV DL,AL**, após a entrada do caractere numérico validado, pega o conteúdo informado e armazenado no registrador **AL** e o transfere para o registrador **DL**.

A quarta instrução **MOV CL,04** coloca no registrador **CX** o valor **04** para rotacionar os bits do valor numérico informado.

A quinta instrução **MOV CL,DL** faz o rotacionamento para à esquerda dos caracteres junto ao registrador **DL** deixando-o pronto para ser apresentado em conjunto com o segundo caractere lido pela sexta instrução.

A sexta instrução **CALL 0200**, assim como a segunda instrução faz a chamada do procedimento que efetua a leitura do caractere.

A sétima instrução **INT 20** faz o encerramento do programa devolvendo o controle de operação ao sistema operacional.

A rotina de procedimento secundária (será chamada pelo programa principal), responsável pela recepção de apenas um caractere hexadecimal válido por vez. Esta rotina de código deve ser codificada a partir do endereço de deslocamento **0200**, por meio do comando **A**, como apresentado em seguida:

07E9:0200	PUSH DX	52
07E9:0201	MOV AH,08	B4 08
07E9:0203	INT 21	CD 21
07E9:0205	CMP AL,30	3C 30
07E9:0207	JL 0203	7C FA
07E9:0209	CMP AL,46	3C 46
07E9:020B	JG 0203	7F F6
07E9:020D	CMP AL,39	3C 39
07E9:020F	JG 0219	7F 08
07E9:0211	CALL 0300	E8 EC 00
07E9:0214	SUB AL,30	2C 30
07E9:0216	POP DX	5A
07E9:0217	JMP 0223	EB 0A
07E9:0219	CMP AL,41	3C 41
07E9:021B	JL 0203	7C E6
07E9:021D	CALL 0300	E8 E0 00
07E9:0220	SUB AL,37	2C 37
07E9:0222	POP DX	5A
07E9:0223	RET	C3

A primeira instrução **PUSH DX** pega o valor existente em **DX** e o coloca na pilha, deixando este registrador livre para outro uso no programa. Isto é importante para salvaguardar o valor existente em **DL**. É pertinente perceber que o registrador **DL** também é usado pelo trecho de código do programa no sentido de seu conteúdo ser apresentado quando da execução da instrução **CALL 0300**.

A segunda instrução **MOV AH,08** habilita no registrador **AH** o valor de função **08** responsável por permitir que o teclado opere sem que apresente o caractere lido. Esta função faz o efeito de teclado sem eco. Este recurso está sendo implementado para que não seja apresentado no monitor de vídeo nenhum caractere que seja inválido. O programa somente apresenta caracteres de 0 até 9 e de A até F.

A terceira instrução **INT 21** dá acesso ao uso do teclado configurado para efetuar a leitura de caracteres sem que os mesmos sejam apresentados no monitor de vídeo. A apresentação dos valores hexadecimais ocorrerá somente pelo controle do procedimento no endereço de deslocamento **0300** para os caracteres validados.

A quarta instrução **CMP AL,30** faz a subtração do valor **30** do valor do caractere numérico lido e armazenado no registrador **AL** no sentido de verificar se o caractere ASCII informado é válido. Desta forma, é possível ter o valor que será tratado condicionalmente por uma instrução de desvio condicional, neste caso **JL** (*Jump on Less*) definido junto a quinta instrução.

A quinta instrução **JL 0203** pega o valor avaliado pela instrução **CMP AL,30** e verifica se o valor numérico do caractere informado é um valor abaixo de **0** (zero). Se a condição for verdadeira o fluxo do programa será desviado para a linha **0203** (terceira linha) para que a entrada continue. Se o caractere numérico informado for válido o programa acata automaticamente o valor validado.

A sexta instrução, assim como a quarta instrução **CMP AL,46** subtrai o valor **46** do caractere informado para saber se o caractere tem valor acima do código ASCII da letra **F**. esta instrução gera um valor lógico que será tratado pela instrução de desvio condicional **JG** (*Jump on Greater*).

A sétima instrução **JG 0203** desvia o fluxo do programa para o endereço de deslocamento **0203** caso o caractere informado seja maior que a letra **F**. Se o caractere numérico informado for válido o programa acata automaticamente o valor validado.

A oitava instrução **CMP AL,39** efetua a subtração para ter o valor que permita verificar caracteres acima do valor **9**. Se o valor estiver acima de **9** este precisará ser verificado se é ou não um caractere alfabético.

A nona instrução **JG 0219** desvia o fluxo do programa para o endereço de deslocamento **0219** caso o caractere informado seja maior que o número **9**. Não sendo maior que **9** o primeiro caractere já válido pode na sequência ser apresentado no monitor de vídeo por meio da chamada **CALL 0300**.

A décima instrução **CALL 0300** efetua a apresentação do primeiro caractere validado.

A décima primeira instrução **SUB AL,30** ajusta o registrador **AL** subtraindo o valor **30** do valor primeiro valor informado na entrada.

A décima segunda instrução **POP DX** recupera o primeiro valor colocado anteriormente na pilha.

A décima terceira instrução **JMP 0223** efetua um salto incondicional para o endereço de deslocamento **0223** onde é realizado o encerramento do programa. O comando **JMP** (*Jump*) é usado quando se deseja desviar o fluxo de um programa para certo ponto do código sem o uso de avaliação condicional.

A décima quarta instrução **CMP AL,41** efetua a subtração em relação ao segundo valor hexadecimal para ter o valor que permita verificar caracteres abaixo da letra **A**. Se o valor estiver abaixo da letra **A** este precisará ser verificado se é ou não um caractere alfabético.

A décima quinta instrução **JL 0203** desvia para a reentrada de um caractere que tenha sido informado de forma inválida.

A décima sexta instrução **CALL 0300** efetua a apresentação do segundo caractere validado.

A décima sétima instrução **SUB AL,37** ajusta o registrador **AL** subtraindo o valor **37** do valor segundo valor informado na entrada.

A décima oitava instrução **POP DX** recupera o segundo valor colocado anteriormente na pilha.

A décima nona instrução **RET** faz o retorno da chamada do procedimento ao programa principal.

A rotina de procedimento terciário (chamada pelo procedimento secundário) é responsável pela apresentação do caractere válido, deve ser codificada a partir do endereço de deslocamento **0300**, pelo comando **A**, informe o código a seguir indicado do lado esquerdo:

07E9:0300	MOV AH,02	B4 02
07E9:0302	MOV DL,AL	88 C2
07E9:0304	INT 21	CD 21
07E9:0306	RET	C3

A primeira instrução **MOV AH,02** é usada para armazenar o código de função **02** responsável por apresentar o caractere armazenado no registrador **DL**.

A segunda instrução **MOV DL,AL** transporta para **DL** o conteúdo do registrador **AL** que será apresentado.

A terceira instrução **INT 21** efetua a apresentação do caractere em **DL** a partir do código de função **02**.

A quarta instrução **RET** faz o retorno da chamada do procedimento ao procedimento secundário.

Em seguida execute o comando **G**. O programa espera que dois caracteres válidos sejam fornecidos. Se o caractere for inválido o programa não faz sua recepção, aceitando apenas caracteres para a entrada de valores numéricos hexadecimais.

Anotações

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69
0D82:0109 64	DB	64
0D82:010A 61	DB	61
0D82:010B 64	DB	64
0D82:010C 65	DB	65
0D82:010D 206573	AND	[DI+73],AH
0D82:0110 7065	JO	0177
0D82:0112 63	DB	63
0D82:0113 69	DB	69
0D82:0114 66	DB	66
0D82:0115 69	DB	69
0D82:0116 63	DB	63
0D82:0117 61	DB	61

Parte



Programação com emu8086

0D82:0100 B402	MOV	AH, 02
0D82:0102 B241	MOV	DL, 41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 7

OPERAÇÕES ESSENCIAIS

*Este capítulo mostra recursos e restrições da ferramenta de montagem e simulação Assembly **emu8086** (versão 4.08r). Traz informações sobre o processo de compilação de programas, execução (depuração) passo a passo do código de programa e o conjunto de instruções que formam a linguagem de programação de computadores Assembly 8086/8088..*

7.1 Simulador e assembler 8086

O programa **Enhanced DEBUG** anteriormente demonstrado permite a construção de programas tanto em linguagem de máquina, por meio de *opcode* (códigos de operação) como em linguagem *Assembly*. No entanto, é uma ferramenta que possui algumas restrições operacionais, sendo uma delas exigir que se saiba em que parte da memória está se construindo os programas.

É fato que existem outras ferramentas que facilitam o trabalho de codificação e compilação dos programas, tais como: **TASM** (*Turbo Assembler da Borland*) e **MASM** (*Macro Assembler da Microsoft*) que são ferramentas comerciais, além de ferramentas baseadas em código livre como é o caso do programa **FASM** (*Flat Assembler* - <http://flatassembler.net>) e **NASM** (*The Netwide Assembler* - <http://www.nasm.us/>).

Pelo fato deste livro ser focado a leitores iniciantes cabe apresentar uma ferramenta com apelo mais didático e que facilite o entendimento deste tipo de programação, chamado **emu8086**. O programa **emu8086** é um simulador gráfico do processador 8086/8088 padrão *shareware* que também compila programas escritos em linguagem *Assembly*. Além de sua praticidade, possui boa documentação e pode ser adquirido a um baixo custo considerando o potencial que a ferramenta oferece e sua execução nos sistemas operacionais Microsoft Windows de 32 e 64 *bits*.

Efetue o carregamento para a memória do computador do programa **emu8086** como orientado no capítulo 3. Selecione na caixa de diálogo **welcome** o botão **New**. Na caixa de diálogo **choose code template** mantenha a seleção da opção **COM template** e acione o botão **OK**. A Figura 7.1 apresenta a tela inicial do programa.

Observe atentamente detalhes que já são conhecidos e estão presentes no trecho de código apresentado:

- ◆ Na linha **05** está a definição da diretiva¹ **org 100h** que estabelece o critério de criação de um programa simples com extensão **.COM**. A diretiva está grafada em tom azul-marinho e o valor associado em tom preto. A diretiva **org** é usada para marcar a posição inicial de carga do código objeto (WEBER, 2004, p. 239). O valor **100** é colocado no registrador de segmento **CS** quando o programa for executado. Anteriormente o acesso a esse endereço foi realizado por meio da instrução **A 0100** no programa **Enhanced DEBUG**.
- ◆ A linha **09** indica o uso do comando **RET** (ou **ret**) que tem por finalidade retornar o controle de execução do programa ao sistema operacional após seu encerramento.

Observação

A instrução **org 100h** pode ser substituída sem nenhum problema pela instrução **org 0x100**. O indicativo **0x** possui o mesmo efeito do indicativo **h**. A forma **org 0x100** será explorada neste texto.

¹ Uma diretiva é um comando usado para controlar a montagem de um programa objeto (WEBER, 2004, p. 239). É uma maneira de se passar ao programa montador opções que o programador deseja executar na ação de montagem de seu programa.

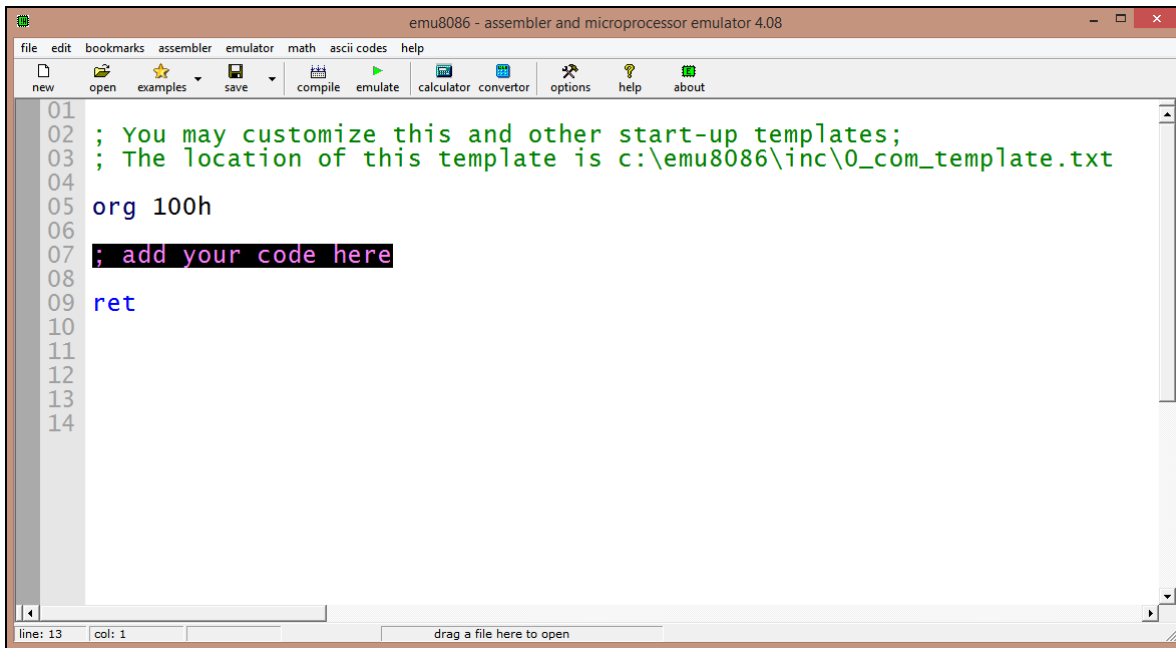


Figura 7.1 - Site da ferramenta emu8086.

O programa **emu8086** permite de maneira simples, a visualização gráfica de toda a estrutura da arquitetura de um microprocessador padrão Intel 8086. É possível visualizar diversos elementos de composição de um microprocessador como: *stack*, *alu*, *memória*, *variáveis*, *flags* entre outras possibilidades. Por ser esta ferramenta um simulador há algumas poucas instruções *assembly* 8086 que não são executadas no ambiente, mas nada que comprometa o entendimento e aprendizagem.

7.2 - Programa “Alo Mundo”

Em seguida será desenvolvido um pequeno programa que apresenta a mensagem “**Alo mundo**”, na tela do monitor de vídeo como o programa já apresentado no capítulo 5. Observe o código a seguir similar ao código que foi utilizado anteriormente no programa **Enhanced DEBUG**:

```
0D0B:0100 MOV AH,09
0D0B:0102 MOV DX,0200
0D0B:0105 INT 21
0D0B:0107 INT 20
```

```
0D0B:0200 41 6C 6F 20 6D 75 6E 64 6F 21 24
```

Observe em seguida o mesmo programa codificado no estilo utilizado pela ferramenta **emu8086**. Note a existência de algumas pequenas diferenças entre os dois códigos. Assim sendo, a partir da linha **04** escreva o programa seguinte:

```
MOV AH, 09h
LEA DX, mensagem
INT 21h
INT 20h
```

```
mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 24h
```

As instruções da linguagem de programação de computadores *Assembly* podem ser escritas tanto com letras maiúsculas como minúsculas. Assim sendo, torna-se necessário definir um padrão de trabalho. Nesta obra as instruções sempre são escritas com caracteres maiúsculos.

A Figura 7.2 apresenta o código do programa na tela do editor do programa **emu8086**. Note a similaridade existente entre as duas formas de codificação. No primeiro programa a sequência de códigos hexadecimais da mensagem está

definida no endereço de memória **0200h**. Já no segundo programa a sequência de códigos hexadecimais para a escrita da mensagem está definida na variável **mensagem** (que será criada em algum endereço de memória desconhecido) por meio da diretiva **DB** (que será explanada no próximo capítulo). Outro detalhe do segundo programa é o uso da instrução **LEA DX, mensagem** no lugar da instrução **MOV DX,0200**. A instrução **MOV DX,0200** funciona no programa **Enhanced DEBUG**, mas não pode ser usada da mesma forma no programa **emu8086**, no qual essa operação é realizada pelo comando **LEA**.

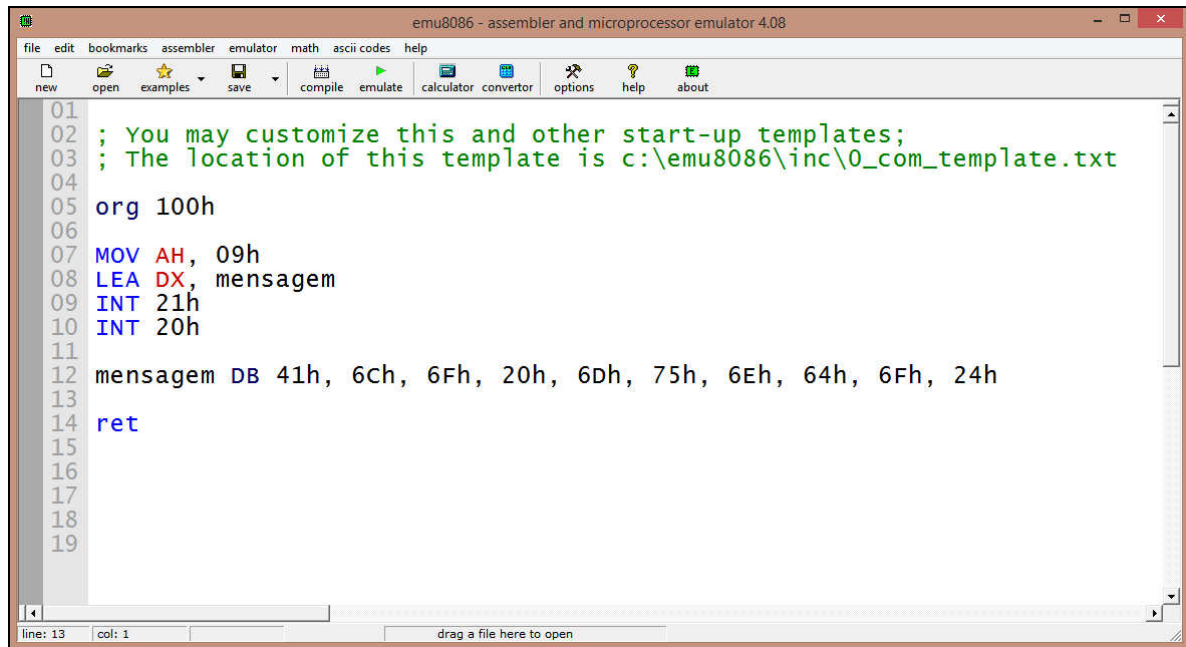


Figura 7.2 - Programa Alô mundo.

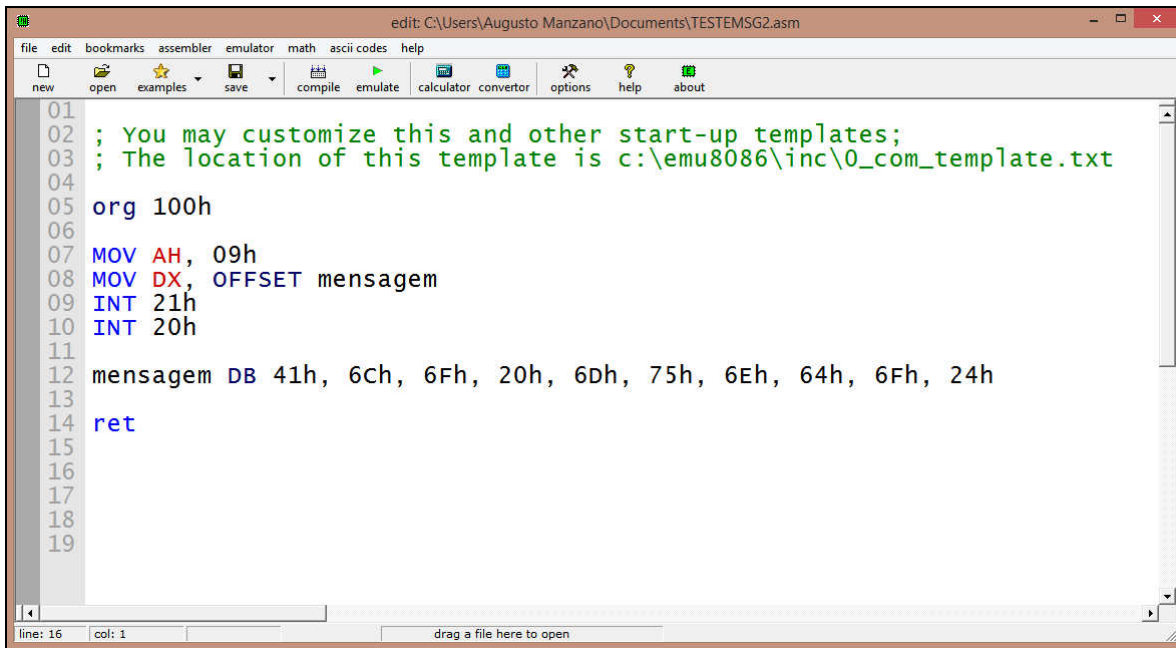
O comando **LEA** funciona com dois parâmetros: um **registrador** (representado por **DX**) e o **endereço** de memória (representado pela variável **mensagem**). Esta instrução não afeta nenhum registrador de estado e ocupa de 2 a 4 bytes de memória. No caso apresentado, o parâmetro *registrador* está definido pelo uso do registrador geral **DX** de 16 bits, e só é possível usar o comando **LEA** com registradores gerais que sejam de 16 bits; o parâmetro *endereço* representado pelo rótulo de identificação da variável **mensagem** indica o local da memória onde se encontra armazenada os caracteres da mensagem a ser apresentada.

O comando **LEA** obtém de forma automática o valor do deslocamento de endereço de memória a partir do local efetivo apontado, ou seja, o comando **LEA** calcula o valor efetivo do deslocamento de endereço a ser utilizado, ou seja, carrega em um registrador do tipo *word* (neste caso **DX**) o deslocamento de um operando (neste caso **mensagem**). Assim, o comando **LEA** carrega no **registrador** o endereço onde se encontra definido o conteúdo apontado em **endereço**. Apesar dessa facilidade, essa instrução é mais lenta em sua execução que a execução do comando **MOV**. Em seguida, grave o programa na pasta **Documentos** usando o comando de menu **file/save** com o nome **TESTMSG1**.

Para fazer uso do comando **MOV** no lugar do comando **LEA** definido na linha **05** deve-se em seu lugar usar a instrução **MOV DX, OFFSET mensagem**, em que o registrador geral **DX** é carregado com o valor do deslocamento (instrução **OFFSET**) de endereço da posição de memória onde se encontra a variável **mensagem**. É como pedir para o comando **MOV** ir até o local da memória onde se encontra a variável **mensagem** por meio do comando **OFFSET**.

O comando **OFFSET** é uma diretiva que obtém o endereço relativo, ou seja, o deslocamento da variável indicada no segmento. O comando **MOV DX, OFFSET mensagem** esta armazenando no registrador **DX** o *offset* do endereço onde a variável **mensagem** se encontra.

Observe a seguir o código do programa com essa alteração e também sua forma de apresentação na tela do programa **emu8086**, Figura 7.3. Grave o programa na pasta **Documentos** com o nome **TESTMSG2** usando para tanto o comando de menu **file/save as...**



```
edit: C:\Users\Augusto Manzano\Documents\TESTMSG2.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
01
02 ; You may customize this and other start-up templates;
03 ; The location of this template is c:\emu8086\inc\0_com_template.txt
04
05 org 100h
06
07 MOV AH, 09h
08 MOV DX, OFFSET mensagem
09 INT 21h
10 INT 20h
11
12 mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 24h
13
14 ret
15
16
17
18
19
line: 16 col: 1 drag a file here to open
```

Figura 7.3 - Programa Alô mundo com MOV DX, OFFSET mensagem.

Você sabia que o comando **MOV**:

- ◆ não afeta nenhum dos registradores de estado (*flags*);
- ◆ que as movimentações entre a origem e o destino devem ser do mesmo tipo de dado;
- ◆ que o registrador de apontamento **IP** não pode ser usado;
- ◆ que o registrador de segmento **CS** não pode ser usado como destino, mas pode ser definido como origem;
- ◆ que nenhum operador de segmento pode receber um dado de forma imediata;
- ◆ que as definições de origem e destino não podem ser ambos indicados como memória.

O comando **MOV** pode ser utilizado para efetuar movimentação:

- ◆ de dado imediato para a memória;
- ◆ de dado imediato para registrador;
- ◆ de registrador para registrador;
- ◆ de memória/registrador para registrador/memória;
- ◆ de memória ou registrador para registrador de segmento;
- ◆ de registrador de segmento para memória ou registrador;
- ◆ de acumulador/memória para memória/acumulador.

Assim sendo, observe alguns exemplos de utilização do comando **MOV**.

- ◆ De dado imediato para memória:

MOV byte ptr [0800h], ABh - movimenta o word **ABh** para o deslocamento de memória **0800h**. A indicação **byte ptr** refere-se ao uso de um byte na memória;

MOV word ptr [AX+ABCDh], CS - movimenta o word de **CS** para o deslocamento de memória **[AX+ABCDh]**. A indicação **word ptr** refere-se ao uso de um word na memória.

- ◆ De dado imediato para registrador:

MOV AX, 1357h - movimenta o valor hexadecimal **1357** para o registrador geral **AX**;

MOV CX, 10101010b - movimenta o valor binário **10101010** para o registrador geral **CX**.

- ◆ De registrador para registrador:
 - MOV AX, BX** - movimenta o valor do registrador **BX** para o registrador **AX**;
 - MOV SI, AX** - movimenta o valor do registrador **AX** para o registrador **SI**.
- ◆ De memória/registrador para registrador/memória:
 - MOV AL, [BX]** - movimenta o conteúdo armazenado no deslocamento **[BX]** para o registrador **AL**;
 - MOV AL, [BX + SI]** - movimenta o conteúdo armazenado no deslocamento **[BX + SI]** para o registrador **AL**.
- ◆ De memória ou registrador para registrador de segmento:
 - MOV ES, AX** - movimenta o valor do registrador **AX** para o registrador **ES**;
 - MOV DS, [SI]** - movimenta o conteúdo do deslocamento **[SI]** para o registrador **DS**.
- ◆ De registrador de segmento para memória ou registrador:
 - MOV AX, CS** - movimenta o valor do registrador **CS** para o registrador **AX**;
 - MOV [0700h], DS** - movimenta o conteúdo do registrador **DS** para o deslocamento de memória **0700h**.
- ◆ De acumulador/memória para memória/acumulador:
 - MOV [SI], AH** - movimenta o byte armazenado no registrador **AH** para o deslocamento **[SI]**;
 - MOV AH, [BX+04h]** - movimenta o conteúdo do deslocamento **[BX+04h]** para o registrador **AH**;
 - MOV DESLOC, AL** - movimenta o valor do registrador **AL** para o deslocamento identificado pelo rótulo **DESLOC**;
 - MOV AL, DESLOC** - movimenta o conteúdo do deslocamento identificado pelo rótulo **DESLOC** para o registrador **AL**.

Outro detalhe a ser observado é que no programa **emu8086** (como também é comum em outros programas montadores) não é necessário preocupar-se com o local onde a sequência de caracteres deve ser armazenada na memória. Esse trabalho fica a cargo da ferramenta de montagem (do programa *Assembler*).

A partir deste momento o programa pode ser compilado dentro da ferramenta **emu8086** ou apenas executado passo a passo. Se for compilado, será criado, neste caso, o arquivo de programa com a extensão **.COM**. Se for executado passo a passo, é possível acompanhar a execução de cada linha e também o processo de execução das instruções.

Abra o programa **TESTMSG1** com o comando de menu **file/open**. Essa ação fecha o programa aberto, no caso o programa **TESTMSG2** e coloca em uso o programa carregado. Atente para a instrução **MOV AH, 09h** definida na linha **04**, como foi indicado na Figura 6.6. Essa instrução está movimentando o valor **09h** para o registrador **AH** (parte mais alta do registrador geral **AX**). O código **09h** é usado quando se deseja apresentar um *string* armazenado no registrador geral **DX** (como na instrução da linha **05** que movimenta para o registrador geral **DX** o endereço de memória no qual se encontra o conteúdo da variável **mensagem** por meio da instrução **LEA DX, mensagem**). A partir do uso da interrupção **21h**, comandada pela linha **06** por meio da instrução **INT 21h**, ocorre a apresentação da mensagem no monitor de vídeo.

A interrupção **21h** é um recurso que estabelece o controle do serviço de entrada e saída operacionalizado pelo MS-DOS (entenda-se sistema operacional). A definição da ação de uma entrada ou da ação de uma saída dessa interrupção depende do valor armazenado sempre no registrador **AH**. Alguns códigos válidos para operacionalizar a interrupção **21h** são:

- ◆ **AH = 01h** - efetua a leitura com eco de um caractere a partir do periférico padrão para a entrada de dados conectado ao computador, representado pelo teclado. O caractere informado é armazenado no registrador **AL**. Se não houver nenhum caractere armazenado no *buffer* do teclado, a função espera até que alguma tecla seja acionada.
- ◆ **AH = 02h** - faz a escrita de um caractere a partir do periférico padrão para a saída de dados conectado ao computador, representado pelo monitor de vídeo. O caractere a ser escrito é armazenado no registrador **DL** após a execução de **AL = DL**.

- ◆ **AH = 05h** - efetua a escrita de um caractere a partir do periférico padrão para a saída de dados conectado ao computador, representado pela impressora. O caractere a ser escrito é armazenado no registrador **DL** após a execução de **AL = DL**.
- ◆ **AH = 06h** - efetua a entrada ou a saída a partir do teclado ou monitor de vídeo conforme o caso. Para efetivar uma saída, o registrador **DL** deve possuir um valor entre **00h..FEh** (em decimal 0..254) que representa o código ASCII do caractere a ser utilizado em **AL = DL** quando do retorno desta ação. Para efetivar a entrada, o registrador **DL** deve possuir o valor **FFh** (em decimal 255) e neste caso a entrada retorna o registrador **ZF** setado em 1 caso não se tenha nenhum caractere disponível no *buffer* do teclado e o registrador **AL** esteja com valor **00h**. O registrador **ZF** fica com seu valor em 0 quando existir algum caractere armazenado no *buffer* do teclado. Neste caso o registrador **AL** conterá a entrada e o *buffer* de teclado será limpo.
- ◆ **AH = 07h** - realiza a leitura sem eco de um caractere a partir do periférico padrão para a entrada de dados conectado ao computador, representado pelo teclado. O caractere informado é armazenado no registrador **AL**. Se não houver nenhum caractere armazenado no *buffer* do teclado, a função espera até que alguma tecla seja acionada.
- ◆ **AH = 09h** - efetua a escrita de uma sequência de caracteres a partir do periférico padrão para a saída de dados conectado ao computador, representado pela impressora. O caractere a ser escrito é armazenado no registrador **DS:DX**.
- ◆ **AH = 0Ah** - faz a leitura de uma sequência de caracteres (*string*) armazenando no registrador **DS:DX**. O primeiro *byte* representa o tamanho do *buffer* e o segundo *byte* representa o número de caracteres efetivamente lidos. Essa função não coloca ao final do *string* o caractere \$, necessário para a identificação do final de uma sequência de caracteres. Para fazer a impressão dessa entrada com **INT 21** e **AH = 09h**, é necessário acrescentar ao final da sequência de caracteres o caractere \$ e iniciar a apresentação a partir do endereço de memória **DS:DX+2**.
- ◆ **AH = 2Ah** - obtém a data do sistema, tendo como retorno o ano de 1980 até 2099 no registrador **CX**, o mês no registrador **DH**, o dia no registrador **DL**, o dia da semana no registrador **AL**, sendo o valor **00h** o domingo, o valor **01h** a segunda-feira e assim por diante.
- ◆ **AH = 2Ch** - obtém a hora do sistema, tendo como retorno a hora no registrador **CH**, o minuto no registrador **CL**, o segundo no registrador **DH** e o centésimo de minuto no registrador **DL:DX**.

Além dos valores apresentados, há também outras funções usadas com a interrupção **INT 21**.

A interrupção **INT 20** finaliza o programa, retornando o controle ao sistema operacional.

Com o objetivo de demonstrar o real uso da ferramenta **emu8086**, ou seja, executar um programa de baixo nível em modo passo a passo, acione o comando de menu **assembler/compile and load in emulator**, ou utilize o botão **emulate** da barra de ferramentas do programa, ou ainda utilize a tecla de função **<F5>**. A Figura 7.4 mostra um exemplo da tela de depuração do programa **original source code** com a janela **emulator: TESTMSG1.asm_** sobre a janela de edição do programa.

Nesta etapa do processo utilize a tecla de função **<F8>** para ver a execução de cada linha do programa. A primeira linha do programa indicada é a de código **MOV AH, 09h**. Neste momento, acione a tecla **<F8>** e observe que a linha de código **LEA DX, mensagem**, Figura 7.5, da janela **original source code** é indicada como **MOV DX, 00109h** em operação na janela **emulator: TESTMSG1.com_**, Figura 7.6.

Note que na janela **emulator: TESTMSG1.com_**, Figura 7.6, a execução do comando **LEA DX, mensagem** é indicada como sendo **MOV DX, 00109h** (semelhante ao que ocorreu no programa **Enhanced DEBUG**). Essa janela apresenta três áreas de trabalho importantes:

- ◆ A parte à esquerda assinalada como **registers** (abaixo do botão **Load**) mostra o estado dos registradores e como estes estão se comportando à medida que é possível avançar o programa passo a passo por meio da tecla de função **<F8>**.
- ◆ A parte central exibe a execução do programa em linguagem de máquina por meio da indicação do endereço real de memória, que são os valores numéricos hexadecimais com cinco dígitos definidos na primeira coluna (na Figura 7.5 esse recurso está assinalado nas linhas **07100** e **07101**), os valores em hexadecimal na segunda coluna referente ao código de operação (*Opcode*) em execução (indicados na Figura 7.5 por **B4** e **09**, em que o valor hexadecimal **09** será movimentado para o registrador **AH**, sendo o valor hexadecimal **B4** o *Opcode* referente à ação da instrução **MOV AH**).

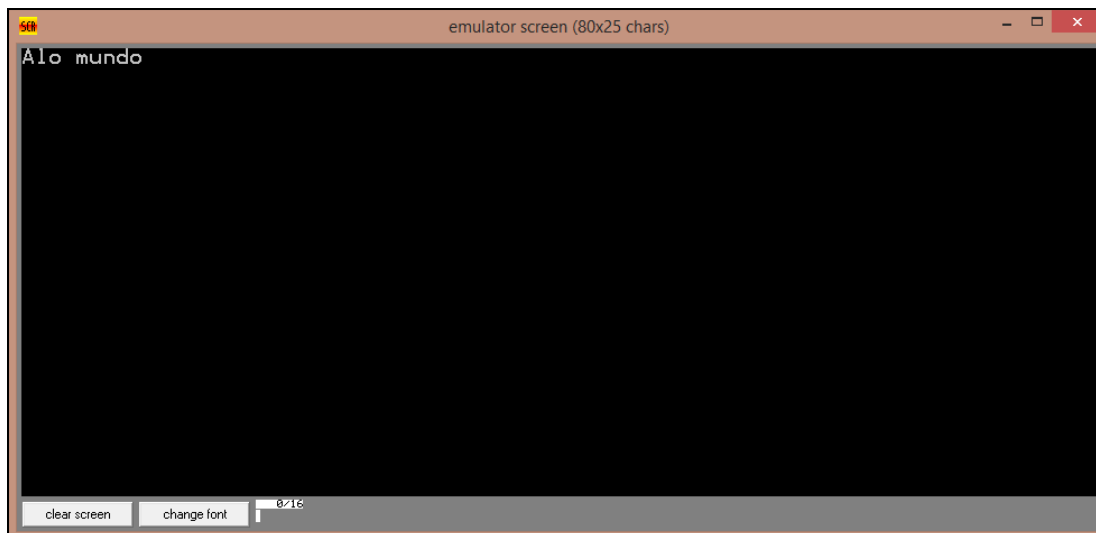


Figura 7.7 - Apresentação da saída do programa.

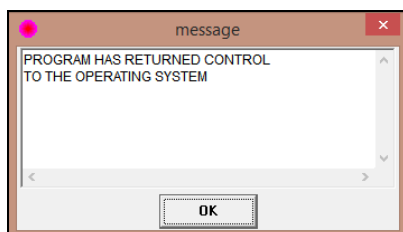


Figura 7.8 - Término da execução do programa.

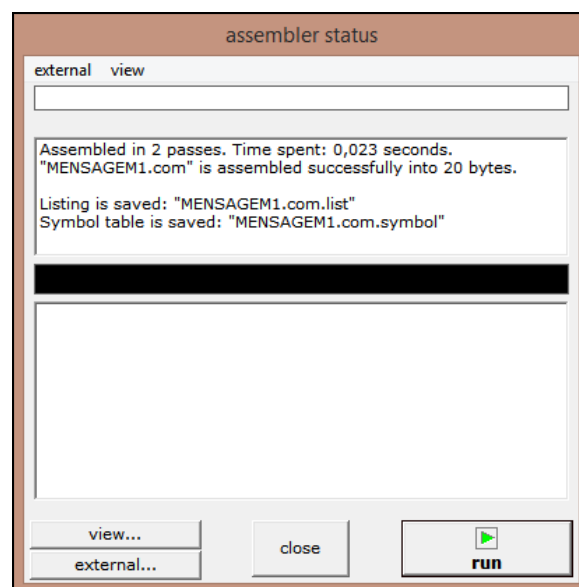


Figura 7.9 - Caixa de diálogo Assembler status. Compiler Status.

A caixa de diálogo traz algumas informações a respeito do processo de compilação do programa. Em seguida, acione o botão **close** para fechar a caixa de diálogo **compiler status** e retornar ao editor.

O programa anterior desenvolvido na ferramenta **emu8086** fez uso de um pequeno conjunto de instruções similares as mesmas instruções usadas no programa **Enhanced DEBUG**. Neste conjunto de instruções foi utilizada certa estrutura sintática na escrita das instruções em código de linguagem *Assembly* para microprocessadores Intel (ou AMD) destinados a computadores pertencentes a família IBM-PC. Cabe neste ponto da apresentação da linguagem *Assembly 8086/8088* descrever a estrutura de escrita de suas instruções que obedecem a seguinte forma:

```
[rótulo:] mnemônico [argumento1][, argumento2][, argumento3] [;comentário]
```

onde

- ♦ **rótulo** é um identificador opcional seguido de dois pontos usado para indicar principalmente nas ferramentas de *assembler* pontos de retorno quando do uso de instruções de desvios.
- ♦ **mnemônico** é o uso da instrução *assembly* para a definição de alguma ação pretendida.

- ◆ **argumento1**, **argumento2** ou **argumento3** são operandos opcionais usados de zero até três argumentos dependendo do código operacional (*opcode*) em uso para a definição de valores ou literais.
- ◆ **Comentário** é a definição de uma linha explicativa opcional da instrução sendo implementada.

Quando são usados dois operandos em uma instrução aritmética ou lógica, o operando posicionado a esquerda representa o destino e o operando da direita representa a origem. Por exemplo:

`carrega_registrador: MOV AX, 50h ; move o valor hexa 50 no registrador AX`

A linha de instrução identificada com o rótulo **carrega_registrador**: efetua a movimentação com o mnemônico **MOV** do valor **50h** para o registrador **AX**. O comentário que faz uma explicação sobre a ação da instrução está definida após o uso do símbolo de ponto-e-vírgula.

O programa **emu8086** possui um simulador do programa **DEBUG** original da Microsoft que pode ser utilizado quando um programa está em execução passo a passo. Assim sendo, carregue para a memória o programa **TESTMSG1.asm** gravado na pastas **Documentos** a partir do uso do comando de menu **file/open...**. Acione no menu principal do programa o comando **assembler/compile and load in emulator**.

Considerando apenas o uso da janela **emulator: TESTMSG1.com**, apresentada selecione na parte inferior o botão **debug** para ser apresentada a janela **debug log - debug.exe emulation** como indicado na Figura 7.10.

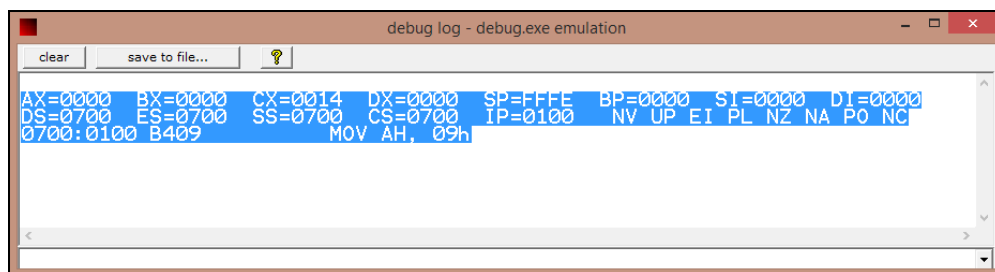


Figura 7.10 - Janela debug log – debug.exe emulation.

A janela **debug log - debug.exe emulation** é formada por três áreas operacionais, sendo: uma pequena barra de ação com os botões: **clear** (limpa o conteúdo da área de display apresentado no simulador de *debug*), **save to file...** (salva o conteúdo da área de display do emulador em um arquivo texto com o mesmo nome do programa em operação) e **inter-rogação** (mostra o modo de ajuda com os comandos aceitos pelo simulador de *debug*); uma área de display contendo a apresentação do estado dos registradores internos de um microprocessador padrão 8086; a última área é definida pelo campo de comando existente abaixo da área de display que permite a entrada de alguns comandos similares aos existentes nas várias versões dos programas **DEBUG**.

Os comandos aceitos pelo simulador **DEBUG** na terceira área de acesso são similares aos comandos dos programas **DEBUG** estando disponível um pequeno conjunto deles, destacando-se apenas os comandos: **C** (*compare*), **I** (*input*), **O** (*output*), **D** (*dump*), **T** (*trace*), **P** (*proceed*) e **R** (*register*). Desse pequeno conjunto os comandos conhecidos anteriormente são **D**, **T**, **P** e **R**.

Para um pequeno teste acione o botão **clear** na área de ação da janela para limpar a área de display e entre na área do campo de comando o código de ação **R** para ver os registradores de um microprocessador padrão 8086. As figuras 7.11 e 7.12 mostram respectivamente o uso do comando **R** e do resultado apresentado.

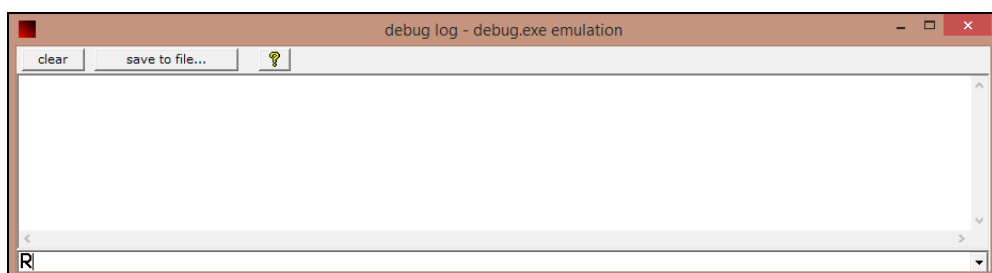


Figura 7.11 - Janela debug log com definição de uso do comando "R".

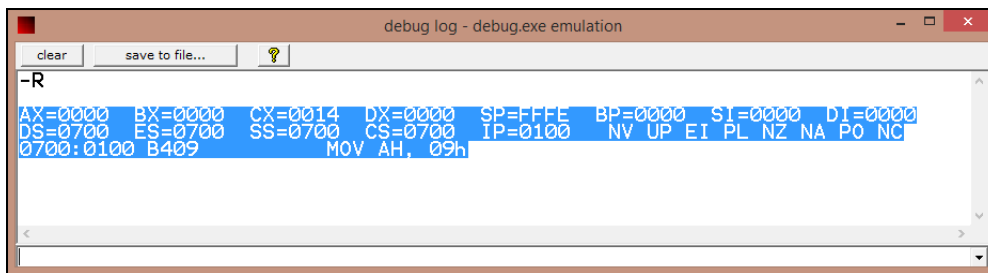


Figura 7.12 - Janela debug log com resultado apresentado após comando "R".

Na janela **debug log - debug.exe emulation** é possível executar passo a passo o programa da memória de duas formas: uma utilizando-se o botão **single run** existente na janela **emulador: TESTEMSG1.com_**; outra se utilizando o código de ação **P** definido no campo de comando apresentado na janela **debug log - debug.exe emulation**.

Neste momento, acione uma vez o botão **single run** da janela **emulador: TESTEMSG1.com_** e note a execução da linha de instrução **MOV AH, 09h** para a linha de instrução **MOV DX, 00109h** como mostra a Figura 7.13.

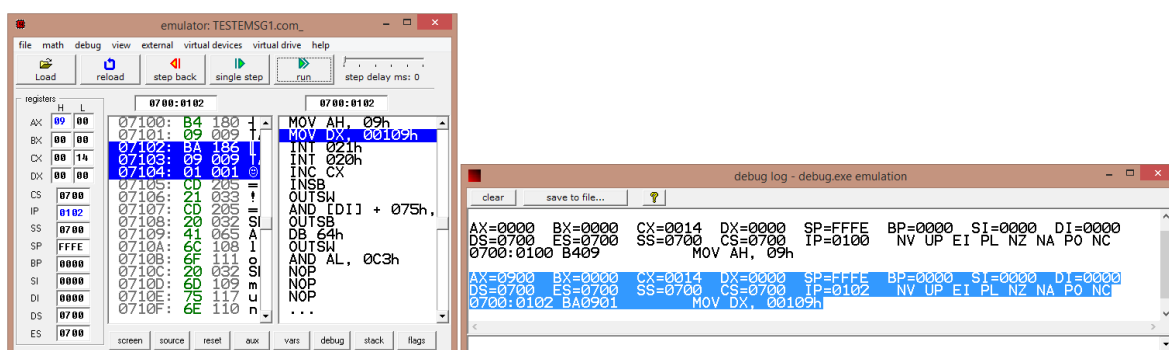


Figura 7.13 - Ação passo a passo após instrução MOV AH, 09h.

A próxima ação de execução para a instrução **MOV DX, 00109h** será efetuada com o uso do código de ação **P** definido no campo de comando da janela **debug log - debug.exe emulation**. Assim sendo, entre o comando **P** e acione a tecla **<Enter>**. Ao fazer esta ação o efeito ocorre tanto na janela **debug log - debug.exe emulation** como na janela **emulador: TESTEMSG1.com_** como pode ser constatado na figura 7.14.

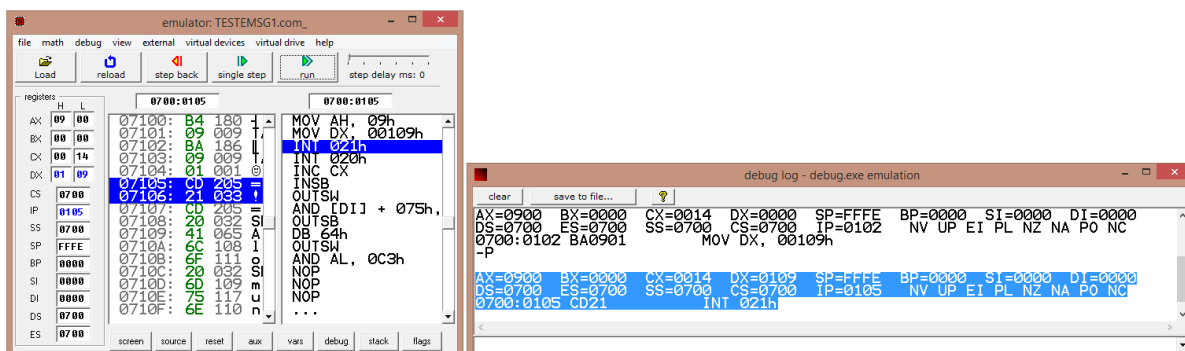


Figura 7.14 - Ação passo a passo após instrução MOV DX, 00109h.

A partir das duas ações anteriormente executadas percebe-se que a execução passo a passo de um programa pode ser realizada tanto pelo uso do botão **single run** como com o uso do comando **P**. A única diferença percebida é que quando a ação é efetivada pelo botão **single run** da janela **emulador: TESTEMSG1.com_** a apresentação do resultado na janela **debug log - debug.exe emulation** ocorre sequencialmente sem a indicação do uso, por exemplo, do comando **P**, como pode ser percebido junto a Figura 7.14 ao se indicar na janela **debug log - debug.exe emulation** o indicativo **-P**.

Como continuidade da ação de execução do programa efetue mais duas vezes o uso do código de ação **P** no campo de comando da janela **debug log - debug.exe emulation** para concluir a ação do programa. Atente para a apresentação

da tela de saída do programa com a resposta da operação na janela **emulator screen (80x25chars)** e a indicação de encerramento do programa com a janela **message** de forma similar as imagens das Figuras 7.7 e 7.8. A Figura 7.15 mostra a janela **debug log - debug.exe emulation** com todas as ações executadas.

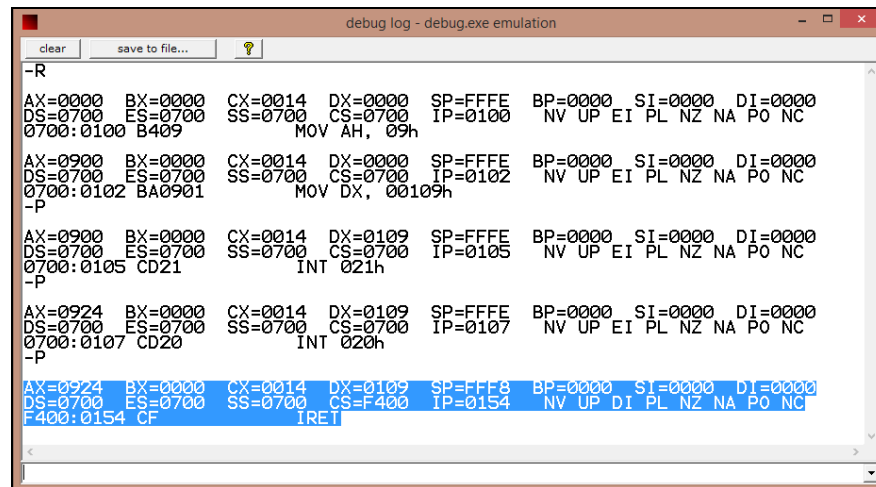


Figura 7.15 - Janela debug log - debug.exe emulation com todas as ações executadas.

Após a apresentação da caixa de diálogo **message** acione o botão **OK** e junto a janela **emulator: TESTMSG1.com_** selecione o comando **close the emulator** do menu **file** para voltar a tela de edição do programa **emu8086**.

Além dos recursos básicos já apresentados, o programa **emu8086** tem uma série de outros como um conversor de bases numéricas, que pode ser acionado pelo comando de menu **math/base converter** ou pelo botão **converter** da barra de ferramentas, Figura 7.16, e uma calculadora de expressões aritméticas que pode ser acionada pelo comando de menu **math/multi base calculator** ou pelo botão **Calculator** da barra de ferramentas, Figura 7.17.

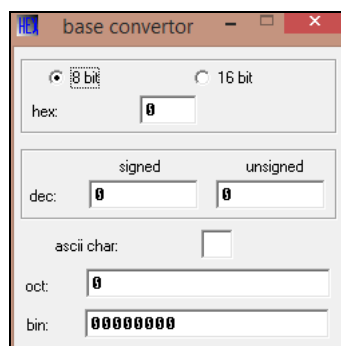


Figura 7.16 - Conversor numérico.

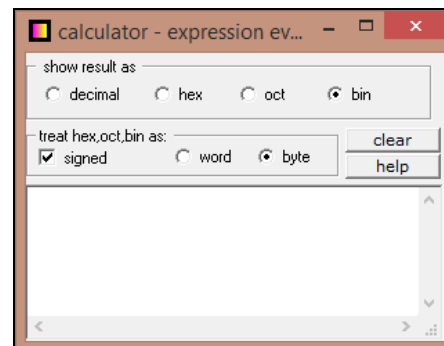


Figura 7.17 - Calculadora de expressões aritméticas.

A maior parte dos recursos do programa **umu8086** está disponível quando se executa um programa com a tecla de função **<F5>**, quando é apresentada a tela do programa **emulator**. Entre os vários recursos da janela **emulator** estão disponíveis no menu **view** boa parcela deles, tais como **extended value viewer** (Figura 7.18), **stack** (Figura 7.19), **variables** (Figura 7.20), **memory** (Figura 7.21), **arithmetic & logical unit/ALU** (Figura 7.22), **flags**, Figura 7.23, e **lexical flag analyser**, Figura 7.24. Os recursos **actual source** e **user screen** são exibidos automaticamente quando se usa o modo de execução do programa.

A Figura 7.19 mostra a tela **extended value viewer** com os valores atuais armazenados e existentes em um determinado registrador. Neste sentido, é possível visualizar o valor nas formas hexadecimal, binária, octal, decimal de 8 e 16 *bits* com ou sem sinal.

A Figura 7.20 apresenta a tela **stack** com os valores existentes na pilha. Observe o caractere **<** indicando o topo da pilha.

A Figura 7.21 exhibe a tela **variables** com a lista de variáveis em uso no programa, bem como os valores iniciais encontrados nas variáveis relacionadas.

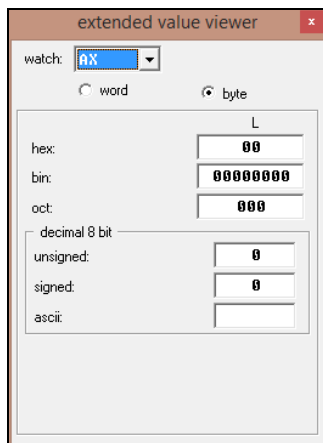


Figura 7.18 - Extended value viewer.

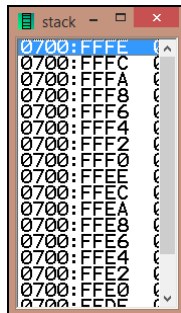


Figura 7.19 - Stack.

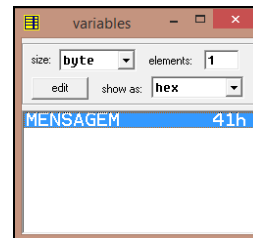


Figura 7.20 - Variables.

A Figura 7.21 mostra a tela **Memory** com a lista de valores externos armazenados nos endereços de memória a partir das coordenadas de segmento e deslocamento definidas na memória RAM (*Random Access Memory*).

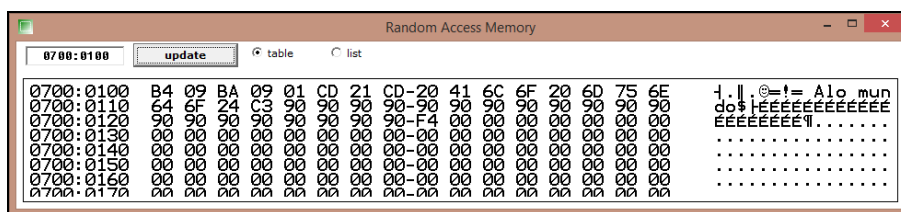


Figura 7.21 - Memory.

Na Figura 7.22 a tela **ALU - arithmetic & logical unit** mostra os dados existentes nas unidades lógica e aritmética.

Na Figura 7.23 a tela **flags** indica os valores do registrador de estado. Com o acionamento do botão **analyse** é apresentada a tela **lexical flag analyser**, Figura 7.24.

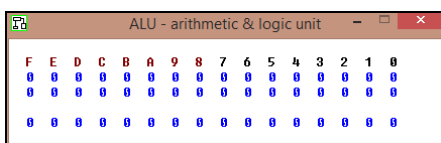


Figura 7.22 - ALU - arithmetic & logical unit.



Figura 7.23 - Flags.

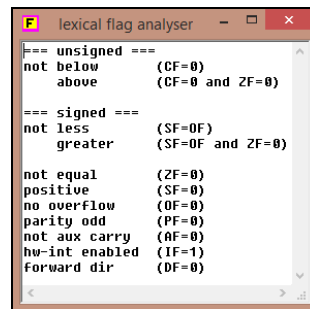


Figura 7.24 - Lexical flag analyser.

Na Figura 7.24 a tela **lexical flag analyser** indica na forma escrita os valores e o conteúdo de cada *flag* do registrador de estado.

7.3 - De volta ao programa Enhanced DEBUG

A partir da criação do programa **MENSAGEM1.com** é interessante visualizar seu código *Assembly* escrito no programa **emu8086** dentro do programa **Enhanced DEBUG**. Assim sendo, copie o arquivo **MENSAGEM1.com** da pasta **Documentos** para a pasta **DEBUGX** e efetue a chamada do ambiente **vDos**, mantendo o programa **emu8086** carregado e execute os seguintes comandos:

```
C:\>E: <Enter>
E:\>DEBUG MENSAG~1.COM <Enter>
```

A indicação **MENSAG~1.COM** refere-se ao programa **MENSAGEM1.com**. A forma de referência **MENSAG~1** ocorre pelo fato do programa **vDos** reconhecer apenas e até os oito primeiros caracteres do nome de um arquivo em modo MS-DOS, como o nome do programa **MENSAGEM1** possui mais de oito caracteres este é renomeado internamente e automaticamente pelo programa **vDos** como sendo **MENSAG~1**.

Assim que o programa **vDos** é carregado na memória juntamente do programa **MENSAG~1.COM** execute o comando a seguir que tem por finalidade decompilar o programa e apresentar seu código originalmente escrito ou um código perto da forma que o código original foi escrito:

```
U 0100 0112 <Enter>
```

Em seguida é apresentado o código do programa definido em memória, que pode ser visualizado a partir de duas partes.

A primeira parte representa o trecho do código em negrito do programa em si definido entre os endereços **0100** e **0107**:

```
07EF:0100 B409          MOV     AH,09
07EF:0102 BA0901       MOV     DX,0109
07EF:0105 CD21         INT      21
07EF:0107 CD20         INT      20
```

A segunda parte representa o trecho do código em negrito que representa a mensagem a ser escrita definida entre os endereços **0109** e **0112** (efetivamente no endereço **0111**):

```
07EF:0109 41           INC      CX
07EF:010A 6C           INSB
07EF:010B 6F           OUTSW
07EF:010C 206D75       AND      [DI+75],CH
07EF:010F 6E           OUTSB
07EF:0110 646F       FS:OUTSW
07EF:0112 24C3       AND      AL,C3
```

Atente para o trecho grafado em negrito (coluna de *opcodes*), o qual indica os códigos ASCII em valores hexadecimais dos caracteres que compõem a mensagem a ser apresentada "**Alo mundo!**" definidas entre os endereços **0109** e **0111** que está anexo abaixo da última linha do trecho do código de apresentação da mensagem, identificado entre os endereços **0100** e **0107**.

Quando se utiliza uma ferramenta de montagem, um "*assemblador*", não é necessário se preocupar com o ajuste de endereço de deslocamento inicial, pois os programas montadores fazem esse controle automaticamente como é o caso de uso da variável **mensagem** no programa escrito no programa **emu8086** acessado pela instrução **LEA DX, mensagem**. No entanto, programas como o **Enhanced DEBUG** necessitam que este trabalho seja efetuado manualmente, o que acaba exigindo certo grau de atenção e cuidado como mostra o uso do endereço **0109** acessado pela instrução **MOV DX, 0109**.

Na sequência encerre a execução dos programas **Enhanced DEBUG** e **vDos** e de volta ao programa **emu8086** acione a tecla de função <F5>. Na janela **emulator: TESTMSG1.com** selecione no menu **view** a opção **listing** e será apresentada um relatório detalhado do código do programa grafado em *opcodes* e *assembly* como mostra a Figura 7.25.

O histórico apresentado mostra detalhes do código do programa em uso. Neste relatório há a indicação do número de linha usada no programa na coluna **[LINE]**, o local de memória onde o código do programa é montado identificado pela coluna **LOC:**, o código de máquina na forma de *opcode* indicado na coluna **MACHINE CODE** e o código escrito em *Assembly* identificada pela coluna **SOURCE**.

Além das informações apresentadas no relatório de um programa verificado no programa **emu8086** o relatório indica a data e hora em que o relatório foi processado.

```

TESTMSG1.com_list - Bloco de notas
Arquivo  Editar  Formatar  Exibir  Ajuda
EMU8086 GENERATED LISTING. MACHINE CODE <- SOURCE.

TESTMSG1.com_ -- emu8086 assembler version: 4.08

[ 20/10/2018 -- 17:48:32 ]

=====
[LINE]      LOC: MACHINE CODE      SOURCE
=====
[ 1]      :
[ 2]      :
[ 3]      :      ; You may customize this and other start-up templates;
[ 4]      :      ; The location of this template is c:\emu8086\inc\0_com_template.txt
[ 5]      :
[ 6]      :      org 100h
[ 7]      :
[ 7]      0100: B4 09      MOV AH, 09h
[ 8]      0102: BA 09 01      LEA DX, mensagem
[ 9]      0105: CD 21      INT 21h
[10]      0107: CD 20      INT 20h
[11]      :
[12]      0109: 41 6C 6F 20 6D 75 6E 64 6F 24      mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 24h
[13]      :
[14]      0113: C3      ret

```

Figura 7.25 - Listagem de código de um programa em opcode e assembly.

O relatório apresentado com o comando **view/listing** da janela **emulator** é uma maneira de se ter em mãos um paralelo entre as formas de escrita de um programa em baixo nível codificado em linguagem de máquina e linguagem de montagem.

Um detalhe a ser observado junto ao relatório apresentado é a indicação da instrução da linha **8** onde o código de máquina **BA 09 01** equivalente a instrução em linguagem de montagem **LEA DX, mensagem** mostra que a variável **mensagem** se encontra definida no endereço de memória **0901**. No entanto, é pertinente salientar que a variável **mensagem** definida na linha **12** como **mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 24h** é indicada no trecho descrito em linguagem de máquina como **0109: 41 6C 6F 20 6D 75 6E 64 6F 24**. Isto posto, mostra que o endereço de memória **0901** indicado na linha **8** é identificado na listagem do programa na linha **12** como endereço **0109**.

Note que os valores são a mesma informação, mas mostrados de forma invertida **0901 x 0109**. Isto decorre da forma como o endereçamento de dados na memória é gerenciado pelo microprocessador. Observe que o endereço de memória **0109** onde se encontra a mensagem a ser apresentada requer para ser armazenado dois *bytes* de memória, sendo no *byte* mais significativo do registrador **DX** (**DH**) armazenado o valor **09** e no *byte* menos significativo do registrador **DX** (**DL**) armazenado o valor **01**.

O microprocessador necessita “conversar” com a memória para nela fazer o armazenamento de dados. Esta ação ocorre de forma reversa, isto é, um *byte* de baixa ordem é armazenado em certo endereço de memória e o outro *byte* de alta ordem é armazenado no próximo endereço de memória. Desta forma, para que o microprocessador possa operar o valor **0109** do registrador **DX** na memória os faz transferindo primeiro o valor **09** para certo endereço de memória e em seguida transfere o valor **01** para o próximo endereço de memória, daí vem o significado da indicação dos valores **0901** da linha **8** estar definido na linha **12** como o valor **0109**. Assim que o microprocessador obtém os dados do endereço de memória a ser utilizado no registrador os inverte novamente.

7.4 - Depuração com uso da pilha

Para um teste com o uso da pilha, considere um programa com uso de chamadas de procedimento semelhante a um exemplo utilizado anteriormente quando do uso do programa **Enhanced DEBUG**.

No programa **emu8086** execute o comando de menu **file/new/com template**. A partir da linha **07** informe a sequência de código seguinte de forma que fique semelhante à Figura 7.26, mantenha a instrução **RET** do código seguinte e retire a instrução **ret** apresentada automaticamente no programa:

```

MOV AX, 000Ah
MOV BX, 000Bh
PUSH AX
PUSH BX
CALL procedimento
INC AX

```

```

INC BX
CALL procedimento
POP BX
POP AX
INT 20h

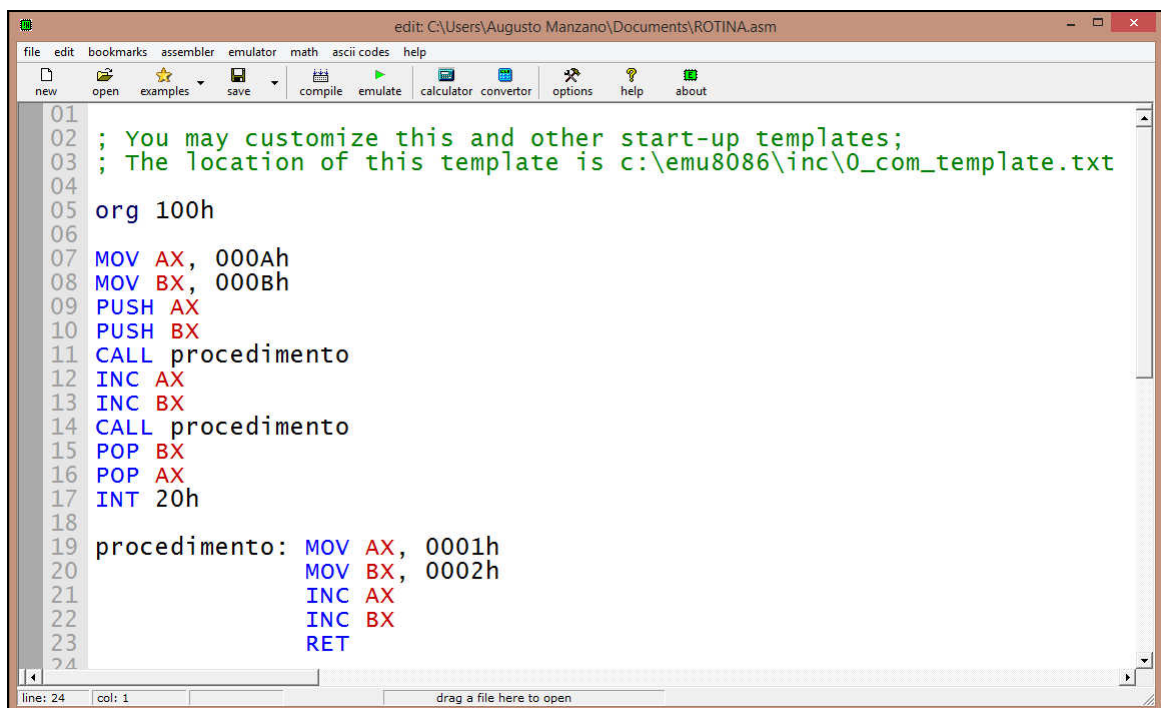
```

```

procedimento: MOV AX, 0001h
               MOV BX, 0002h
               INC AX
               INC BX
               RET

```

Em seguida grave o programa com o nome **ROTINA** e acione a tecla de função <F5> e na medida em que a janela **original source code** vá sendo apresentada feche-a sucessivamente. Por meio da janela **emulator: ROTINA.com_**, execute os comandos de menu **view/log and debug.exe emulation** e **view/stack** que respectivamente apresentarão as janelas **debug log - debug.com emulator** como mostra a Figura 7.27 e **stack** como mostra a Figura 7.28 (que mostra esta janela ajustada com seu tamanho para a direita e para baixo).



The screenshot shows the emu8086 editor window with the file 'ROTINA.asm' open. The code is as follows:

```

01
02 ; You may customize this and other start-up templates;
03 ; The location of this template is c:\emu8086\inc\0_com_template.txt
04
05 org 100h
06
07 MOV AX, 000Ah
08 MOV BX, 000Bh
09 PUSH AX
10 PUSH BX
11 CALL procedimento
12 INC AX
13 INC BX
14 CALL procedimento
15 POP BX
16 POP AX
17 INT 20h
18
19 procedimento: MOV AX, 0001h
20               MOV BX, 0002h
21               INC AX
22               INC BX
23               RET
24

```

Figura 7.26 - Programa escrito no ambiente de programação emu8086.

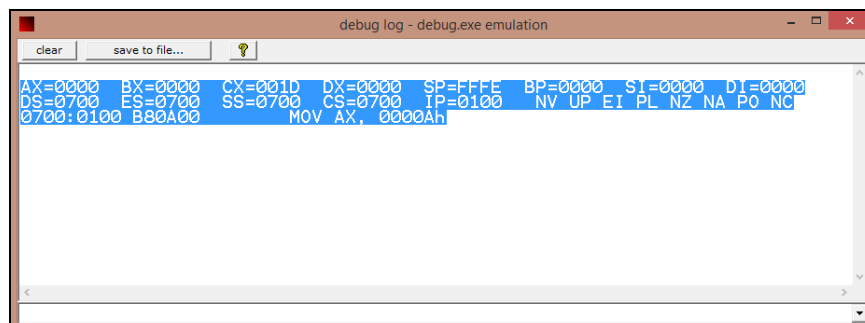


Figura 7.27 - Visualização do modo log and debug.exe emulation.

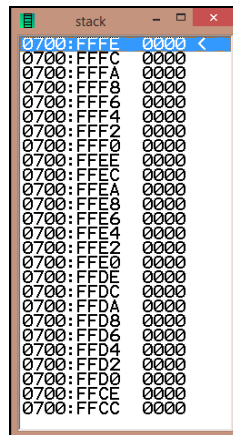


Figura 7.28 - Visualização do modo stack.

Na sequência vá acionando a tecla <F8> e observe atentamente as informações sendo apresentadas nas janelas apresentadas. Inicialmente a Figura 7.29 mostra as janelas **emulador: ROTINA.com**, **debug log - debug.com emulador** e **stack** em conjunto antes da execução de qualquer ação.

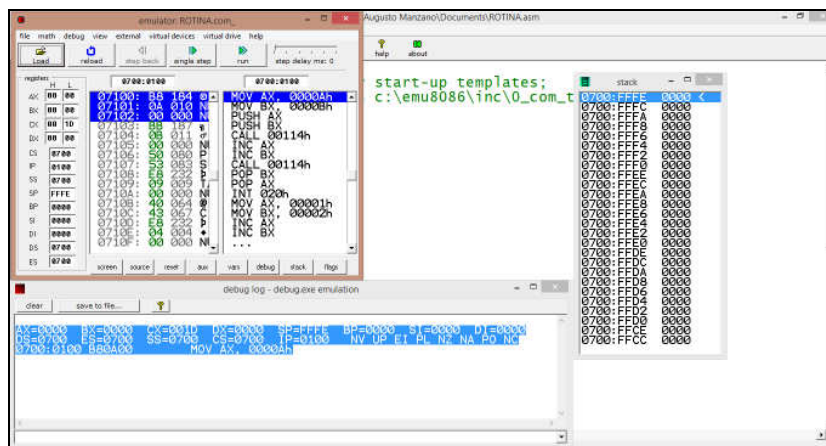


Figura 7.29 - Janelas de ação passo a passo do programa ROTINA.

A primeira ação a ser executada é a movimentação do valor **000A** para o registrador **AX**. Assim sendo acione a tecla <F8> e observe o resultado ocorrido como mostra a Figura 7.30.

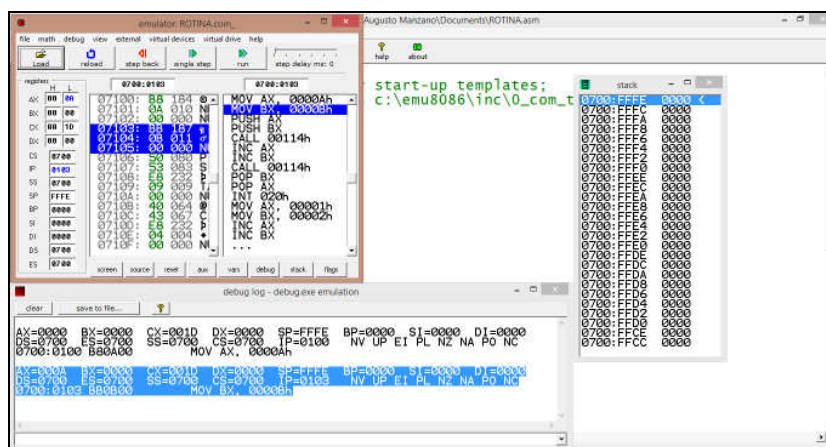


Figura 7.30 - Resultado da execução da instrução MOV AX, 0000Ah.

Observe a apresentação do valor **0A** no registrador **AL** e a mudança do valor do registrador **IP** de **0100** para **0103**. Na sequência acione a tecla de <F8> e observe o resultado ocorrido como mostra a Figura 7.31.

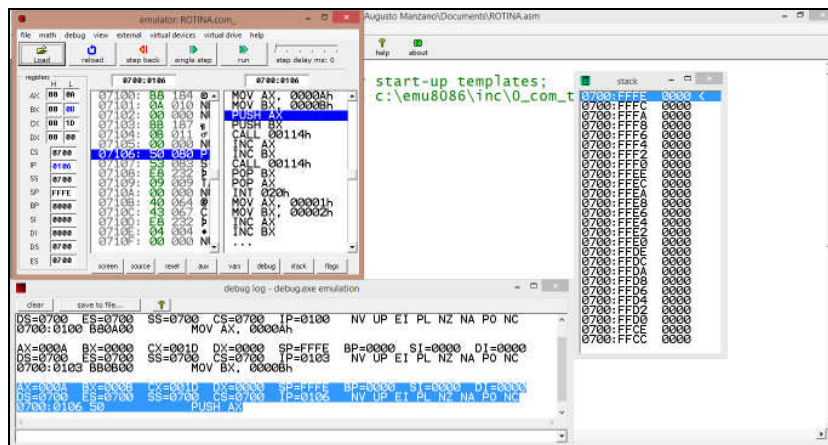


Figura 7.31 - Resultado da execução da instrução MOV AX, 0000Bh.

Observe a apresentação do valor **0B** no registrador **BL** e a mudança do valor do registrador **IP** de **0103** para **0106**. Na sequência acione a tecla de **<F8>** e note o que ocorre com o valor do registrador **SP** que marca o valor **FFFE** e passa a marcar o valor **FFFC** como indica a Figura 7.32.

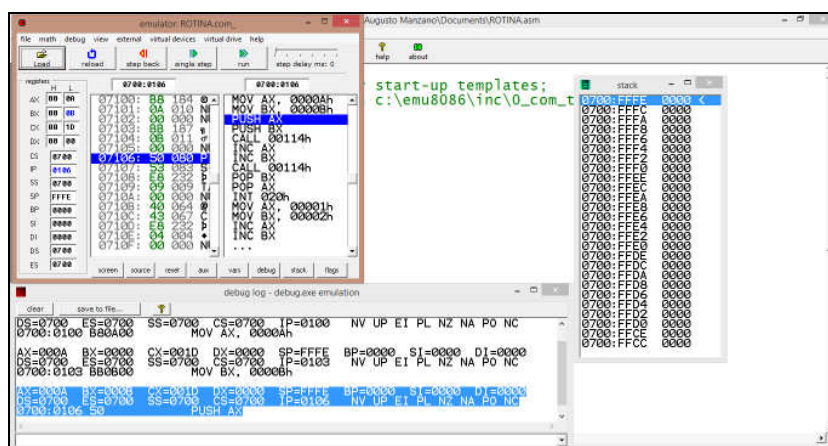


Figura 7.32 - Resultado da execução da instrução PUSH AX.

Observe a apresentação do valor **FFFC** no registrador **SP**, a mudança do valor do registrador **IP** de **0106** para **0107** e a mudança na pilha (janela **stack**) do valor **0000** para **000A** definido no endereço **0700:FFFC**. Na sequência acione a tecla de **<F8>** e note as mudanças comentadas a seguir como indica a Figura 7.33.

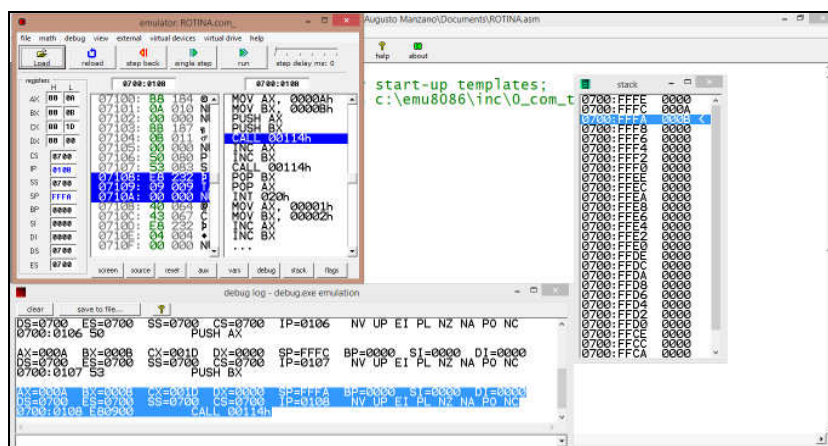


Figura 7.33 - Resultado da execução da instrução CALL 00114h.

A janela **stack** mostra os valores armazenados na pilha, sendo o valor **000A** no endereço de pilha **0700:FFFC** e o valor **000B** no endereço **0700:000A**. Já a janela **debug log - debug.exe emulation** mostra a mudança do valor do registrador **SP** de **FFFC** para **FFFA** e do registrador **IP** com o valor **0108**, além de indicar a execução da próxima instrução **CALL 00114h** equivalente a instrução **CALL procedimento** definido na linha de código **11**. O endereço **00114h** é o local onde se encontra o código do procedimento a ser executado.

O próximo passo a ser executado é a chamada do código do procedimento definido. Assim sendo, acione a tecla de **<F8>** e note as mudanças comentadas a seguir como indica a Figura 7.34.

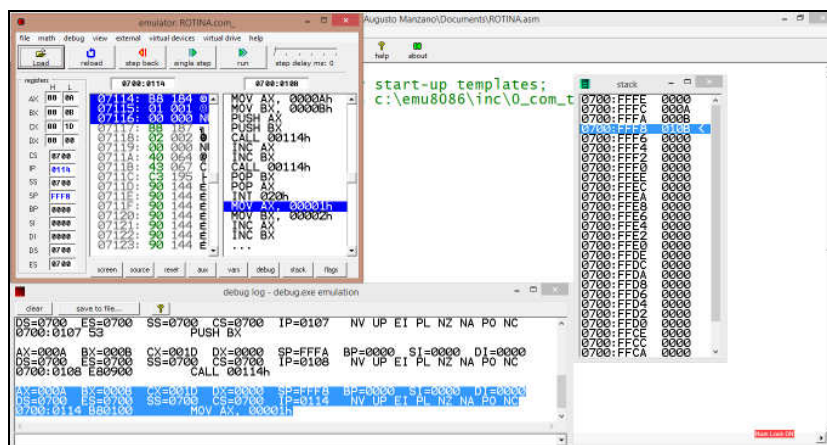


Figura 7.34 - Resultado da execução da instrução **MOV AX, 00001**.

Note as mudanças ocorridas nos registradores **SP** de **FFFA** para **FFF8** e **IP** de **0108** para **0114** e o armazenamento do valor **010B** na posição **0700:FFF8** da pilha que é o valor de retorno para uso do registrador **IP** após a execução do comando **RET** no retorno do procedimento executado. Acione a tecla de **<F8>** e note as mudanças comentadas a seguir como indica a Figura 7.35.

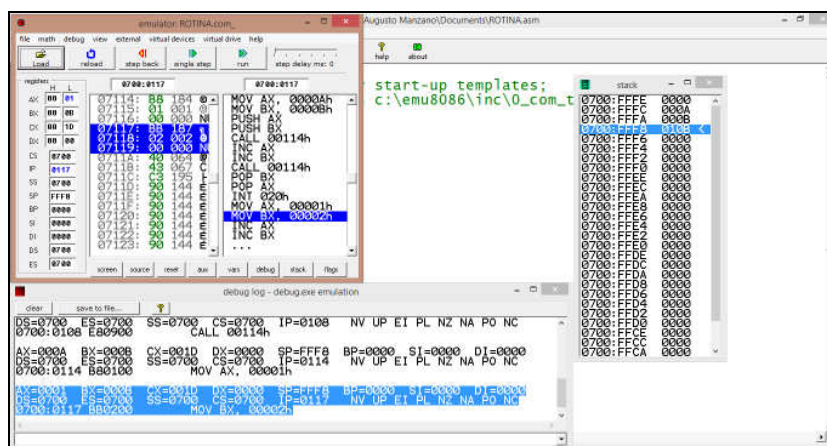


Figura 7.35 - Resultado da execução da instrução **MOV BX, 00002**.

Note que neste momento o registrador **SP** mantém foco sob o valor **FFF8** que não foi alterado, pois guarda o valor do endereço de retorno que será usado pelo comando **RET** para retornar o fluxo de execução do programa para a próxima linha após a chamada do procedimento. A alteração ocorre apenas no registrador **AX** que é alterado de **000A** para **0001** e no registrador **IP** de **0114** para **0117**. Acione em seguida a tecla de **<F8>** e note as mudanças comentadas a seguir como indica a Figura 7.36.

Observe a mudança do valor do registrador **BX** que passa a ser **0002** e do registrador **IP** de **0117** para **011A**. A Figura 7.36 apresenta a execução da próxima instrução **INC AX** que acrescentará o valor 1 ao valor existente no registrador **AX**, ou seja, passará a ser **0002**. Nesta etapa acione a tecla de função **<F8>** por duas vezes de forma que o registrador **AX** fique com o valor **0002** e o registrador **BX** fique com o valor **0003** como indica a Figura 7.37.

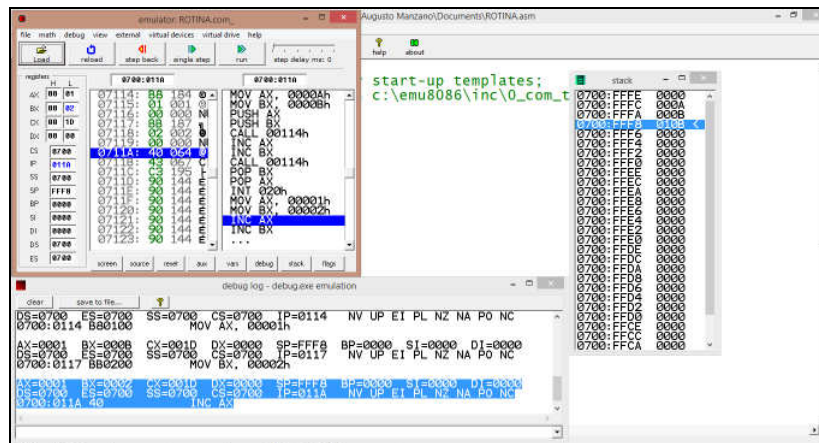


Figura 7.36 - Resultado da execução da instrução INC AX.

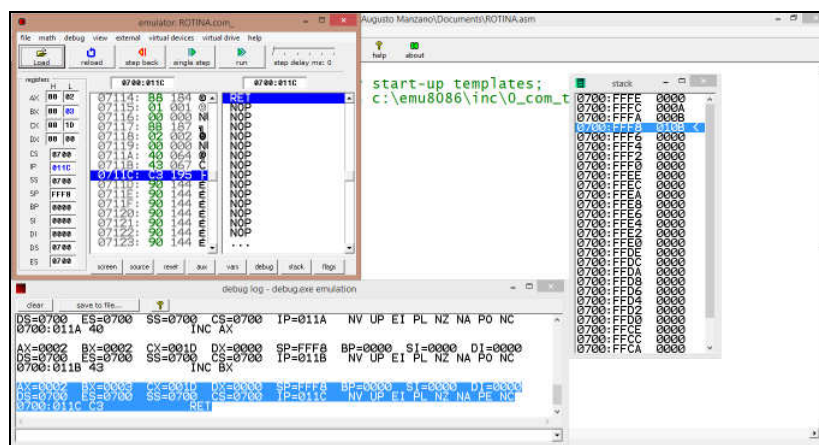


Figura 7.37 - Resultado da execução da instrução RET.

Após as últimas duas execuções os registradores **AX** e **BX** estão respectivamente os valores **0002** e **0003**. Na sequência acione o **<F8>** e observe a execução do comando **RET**. O programa retorna para o endereço **0700:010B**, sendo o valor **010B** obtido a partir da posição **0700:FFF8** da pilha, como demonstrado na Figura 7.38.

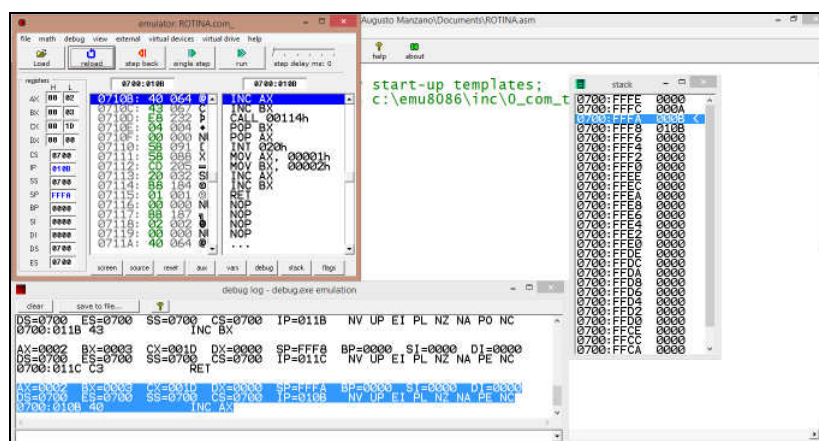


Figura 7.38 - Resultado da execução após uso da instrução RET.

Neste momento acione novamente por duas vezes a tecla de função **<F8>** e observe os registradores **AX** e **BX** assumirem respectivamente os valores **0003** e **0004** como mostra a Figura 7.39.

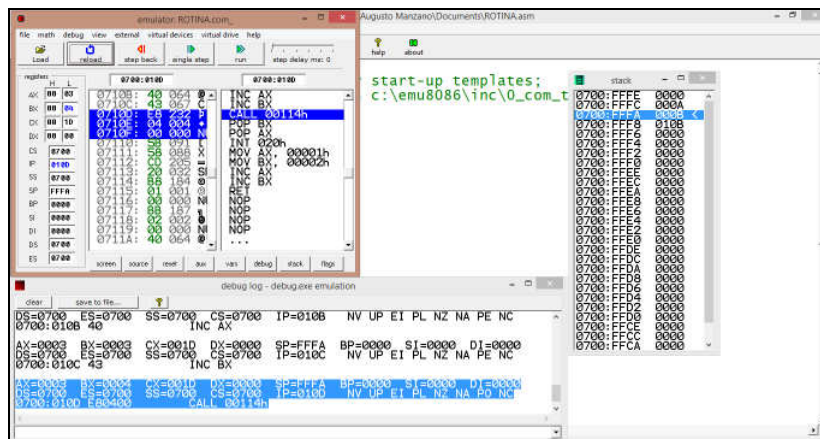


Figura 7.39 - Resultado da execução após os dois últimos incrementos.

A próxima etapa chama novamente o procedimento situado no endereço 00114h. Assim sendo, acione <F8> e observe que o endereço da pilha 0700:FFF8 armazena agora o valor 0110 (antes armazenava o valor 010B) que é o endereço de retorno a ser executado quando do uso do comando RET. A Figura 7.40 mostra essas última mudanças.

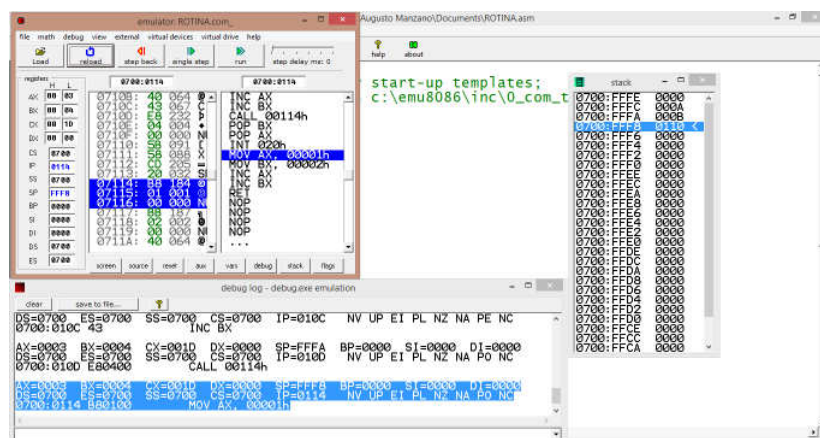


Figura 7.40 - Resultado da execução da segunda chamada do procedimento.

Os dois próximos passos substituem respectivamente os valores dos registradores AX e BX para 0001 e 0002. Assim sendo, acione <F8> mais duas vezes e observe o resultado indicado na Figura 7.41.

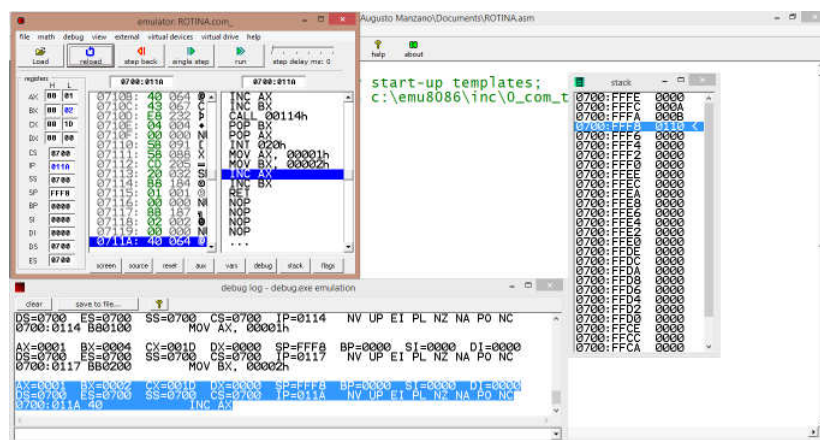


Figura 7.41 - Resultado da execução da definição de valores em AX e BX.

Os dois próximo passos são os incrementos do valor 1 nos valores existentes dos registradores AX e BX de forma que passem a possuir os valores 0002 e 0003. Assim sendo, acione <F8> duas vezes e observe a Figura 7.42.

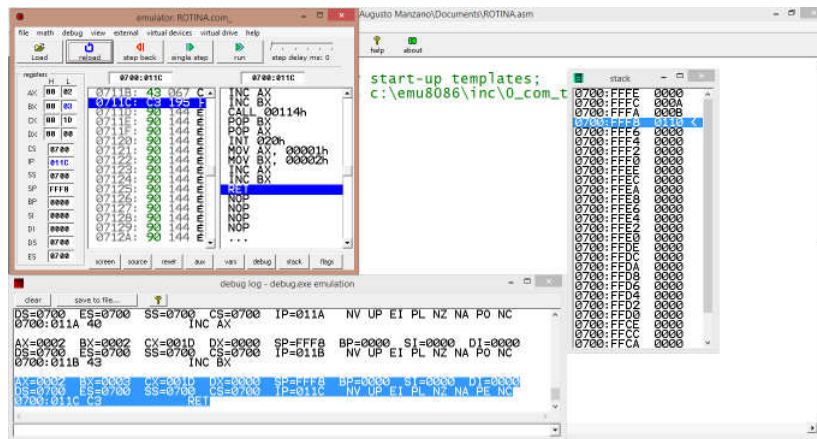


Figura 7.42 - Resultado da execução da definição de valores em AX e BX após incrementos.

O próximo passo é a execução do comando **RET** que retornará o fluxo de operação do programa para o endereço **0110** que é a primeira instrução após a segunda chamada de ação do procedimento. Assim sendo, acione <F8> e observe que a marcação de posição da janela **stack** aponta o endereço **0700:FFFF** que indica o acesso ao valor **000B** que será recuperado pela instrução **POP BX**. A Figura 7.43 mostra este momento.

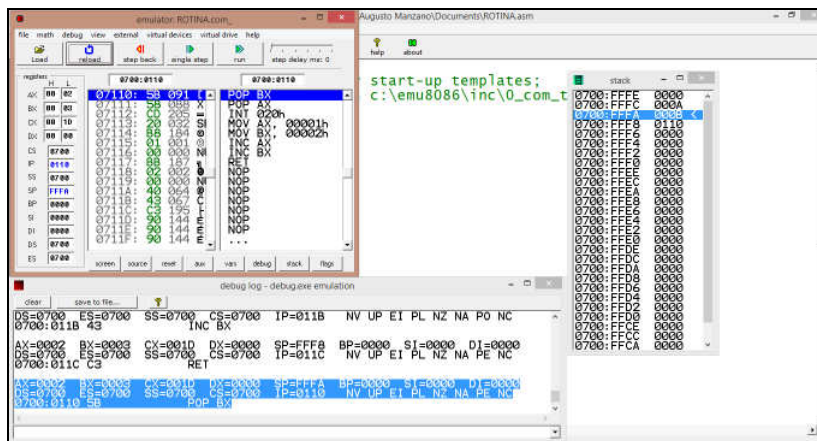


Figura 7.43 - Indicação do valor armazenado na pilha a ser recuperado.

Ao ser acionada a tecla de função <F8> ocorrerá a recuperação do valor **000B** para o registrador **BX**. Note esta ocorrência junto a Figura 7.44 e observe a mudança dos valores definidos no registrador **IP** e no registrador **SP** que mostra o endereço de recuperação dos valores armazenados na pilha.

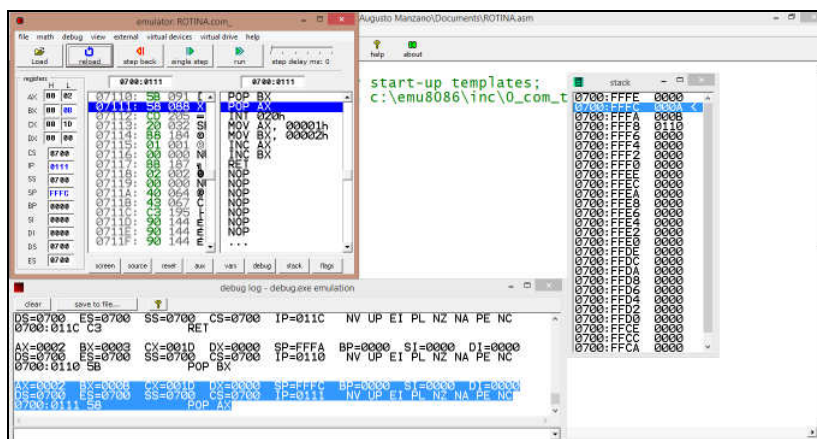


Figura 7.44 - Indicação da recuperação do valor 000B armazenado na pilha.

Na sequência acione <F8> para recuperar o valor **000A** para o registrador **AX** e em seguida acione mais duas vezes <F8> para encerrar a execução passo a passo do programa. Acione o botão **OK** e feche as janelas abertas.

7.5 - Instruções da linguagem de montagem 8086

O microprocessador 8086/8088 (da Intel e seus compatíveis, como os da AMD) tem um conjunto de instruções que possibilita o total controle de um computador. O conjunto de instruções do microprocessador 8086/8088 pode ser categorizado nos seguintes tipos, de acordo com instruções encontradas no programa **emu8086**:

- ◆ Transferência de dados para entrada e saída (IN, OUT);
- ◆ Transferência de dados gerais (MOV, POP, POPA, PUSH, PUSHA, XCHG, XLATB);
- ◆ Transferência de dados relacionada a endereços de segmento (LEA, LDS e LES);
- ◆ Transferência de dados para controle de registradores de estado (LAHF, SAHF, PUSHF e POPF);
- ◆ Aritmética para adição (AAA, ADD, ADC, DAA e INC);
- ◆ Aritmética para subtração (AAS, DAS, CMP, SBB, SUB, DEC e NEG);
- ◆ Aritmética para multiplicação (AAM, MUL e IMUL);
- ◆ Aritmética para divisão (AAD, CBW, CWD, DIV e IDIV);
- ◆ Manipulação de bit para operação lógica (AND, NOT, OR, TEST e XOR);
- ◆ Manipulação de bit para deslocamento (SHL/SAL, SHR e SAR);
- ◆ Manipulação de bit para rotação (RCL, RCR, ROL e ROR);
- ◆ Manipulação de sequências de caracteres (CMPSB, CMPSW, LODSB, LODSW, MOVSB, MOVSW, REP, REPE, REPNE, REPZ, REPNZ, SCASB, SCASW, STOSB e STOSW);
- ◆ Transferência de programa com instrução incondicional (CALL, JMP, RET e RETF);
- ◆ Transferência de programa com instrução condicional (JA, JAE, JB, JBE, JC, JE, JG, JGE, JL, JLE, JNA, JNAE, JNB, JNBE, JNC, JNG, JNGE, JNE, JNL, JNLE, JNO, JNP, JPO, JS, JZ, JNS, JNZ, JO, JP e JPE);
- ◆ Transferência de programa com instrução de controle iterativo (JCXZ, LOOP, LOOPE, LOPPNE, LOPPNZ e LOOPZ);
- ◆ Transferência de programa com instrução de interrupção (INT, INTO e IRET);
- ◆ Instruções de controle do processador em operações de estado (CLC, CLD, CLI, CMC, STC, STD e STI);
- ◆ Não classificada (NOP).

Toda linguagem de programação é composta por um conjunto de instruções, as quais, de forma geral, podem ser formadas por agrupamentos de comandos, diretivas, parâmetros e funções internas, que permitem a um programador "conversar" com um computador por meio do programa que desenvolve.

As instruções de uma linguagem formam o conjunto de *palavras reservadas* da linguagem. No caso da linguagem *Assembly*, o conjunto de palavras reservadas recebe o nome de *mnemônicos* ("menemônicos"). Uma *palavra reservada* não pode ser usada para qualquer tipo de operação que não seja aquela para a qual foi projetada.

A linguagem *Assembly* para a manipulação apenas do microprocessador padrão 8086/8088 de acordo com a documentação dos manuais da Intel possui um conjunto de 113 mnemônicos (instruções), chegando a um total de 124 instruções se considerar o uso dos microprocessadores 80186/80188.

O programa **emu8086** faz uso de 116 instruções, em que são omitidas as instruções BOUND, ESC, HLT, LOCK e WAIT da série 80186/10188, exceto a instrução PUSHA; faz-se a inserção da instrução RETF não existente nos processadores 8086/8088 (80186/80188), mas encontrada nos microprocessadores de 32 e 64 *bits*. Efetua o desmembramento das instruções encontradas nos processadores acima do 80286: CMPS (como CMPSB e MPSW), LODS (como LODSB e LODSW), MOVS (MOVSB e MOVSW), SCAS (SCASB e SCASW), STOS (STOSB e STOSW) e o uso da instrução XLAT como XLATB, como mostra a Tabela 7.1.

Tabela 7.1 - Comandos encontrados no programa emu8086

Mnemônico	Significado	Sintaxe
AAA	ASCII Adjust for Addition	AAA
AAD	ASCII Adjust for Division	AAD
AAM	ASCII Adjust for Multiply	AAM
AAS	ASCII Adjust for Subtraction	AAS
ADC	Add with Carry	ADC destino, origem
ADD	ADditiOn	ADD destino, origem
AND	AND logical	AND destino, origem
CALL	CALL procedure	CALL nome_procedimento
CBW	Convert Byte to Word	CBW
CLC	Clear Carry	CLC
CLD	Clear Direction flag	CLD
CLI	Clear Interrupt-enable flag	CLI
CMC	CoMplement Carry flag	CMC
CMP	CoMPare	CMP destino, origem
CMPSB	CoMPare String Byte	CMPSB texto_destino, texto_origem
CMPSW	CoMPare String Word	CMPSW texto_destino, texto_origem
CWD	Convert Word to Doubleword	CWD
DAA	Decimal Adjust for Addition	DAA
DAS	Decimal Adjust for Subtraction	DAS
DEC	DECrement	DEC
DIV	DIVide	DIV origem
HLT	HaLT	HLT
IDIV	Integer DIVide	IDIV origem
IMUL	Integer MULtiPLY	IMUL origem
IN	Input from port	IN acumulador, porta
INC	INCrement	INC destino
INT	INTerrupt	INT tipo_interrupção
INTO	INTerrupt on Overflow	INTO
IRET	Interrupt RETurn	IRET
JA	Jump on Above	JA
JAE	Jump on Above or Equal	JAE
JB	Jump on Below	JB
JBE	Jump on Below or Equal	JBE
JC	Jump on Carry	JC
JCXZ	Jump if CX register Zero	JCXZ rótulo
JE	Jump on Equal	JE
JG	Jump on Greater	JG
JGE	Jump on Greater or Equal	JGE
JL	Jump on Less	JL
JLE	Jump on Less or Equal	JLE
JMP	JuMP unconditionally	JMP rótulo
JNA	Jump on Not Above	JNA
JNAE	Jump on Not Above or Equal	JNAE

Mnemônico	Significado	Sintaxe
JNB	Jump on Not Below	JNB
JNBE	Jump on Not Below or Equal	JNBE
JNC	Jump on Not Carry	JNC
JNE	Jump on Not Equal	JNE
JNG	Jump on Not Greater	JNG
JNGE	Jump on Not Greater or Equal	JNGE
JNL	Jump on Not Less	JNL
JNLE	Jump on Not Less or Equal	JNLE
JNO	Jump on Not Overflow	JNO
JNP	Jump on Not Parity	JNP
JNS	Jump on Not Sign	JNS
JNZ	Jump on Not Zero	JNZ
JO	Jump on Overflow	JO
JP	Jump on Parity	JP
JPE	Jump on Parity Equal	JPE
JPO	Jump on Parity Odd	JPO
JS	Jump on Sign	JS
JZ	Jump on Zero	JZ
LAHF	Load register AH from Flags	LAHF
LDS	Load pointer using DS	LDS destino, origem
LEA	Load Effective Address	LEA destino, origem
LES	Load pointer using ES	LES destino, origem
LODSB	LOaD String Byte	LODSB texto_origem
LODSW	LOaD String Word	LODSW texto_origem
LOOP	LOOP	LOOP rótulo
LOOPE	LOOP while Equal	LOOPE rótulo
LOOPNE	LOOP while Not Equal	LOOPNE rótulo
LOOPNZ	LOOP while Not Zero	LOOPNZ rótulo
LOOPZ	LOOP while Zero	LOOPZ rótulo
MOV	MOVE (byte ou word)	MOV destino, origem
MOVSb	MOVE String Byte	MOVSb texto_destino, texto_origem
MOVSw	MOVE String Word	MOVSw texto_destino, texto_origem
MUL	MULTiply	MUL origem
NEG	NEGate	NEG destino
NOP	No Operation	NOP
NOT	logical NOT	NOT destino
OR	logical OR	OR destino, origem
OUT	OUTput	OR porta, acumulador
POP	POP	POP destino
POPA	POP All registers	POPA
POPF	POP Flags	POPF
PUSH	PUSH	PUSH origem
PUSHA	PUSH All registers	PUSHA
PUSHF	PUSh Flags	PUSHF

Mnemônico	Significado	Sintaxe
RCL	Rotate through Carry Left	RCL destino, contador
RCR	Rotate through Carry Right	RCR destino, contador
REP	REPeat	REP
REPE	REPeat while Equal	REPE
REPNE	REPeat while Not Equal	REPNE
REPNZ	REPeat while Not Zero	REPNZ
REPZ	REPeat while Zero	REPZ
RET	RETurn	RET valor_opcional_pop
RETF	RETurn from Far procedure	RETF
ROL	ROtate Left	ROL destino, contador
ROR	ROtate Right	ROR destino, contador
SAHF	Store register AH into Flags	SAHF
SAL	SHift Arithmetic Left	SAL destino, contador
SAR	SHift Arithmetic Right	SAR destino, contador
SBB	SuBtract with Berrow	SBB destino, origem
SCASB	SCAN String Byte	SCASB texto_destino
SCASW	SCAN String Word	SCASW texto_destino
SHL	SHifit logical Left	SHL destino, contador
SHR	SHift logical Right	SHR destino, origem
STC	SeT Carry	STC
STD	SeT Direction flag	STD
STI	SeT Interrupt enable flag	STI
STOSB	STOre String (byte ou word)	STOSB texto_destino
STOSW	STOre String (byte ou word)	STOSW texto_destino
SUB	SUBtract	SUB destino, origem
TEST	TEST	TEST destino, origem
XCHG	Exchange	XCHG destino, origem
XLATB	Translate table look-up	XLATB tabela
XOR	eXclusive OR	XOR destino, origem

A primeira coluna da tabela descreve o nome da instrução, a segunda coluna mostra sinalizando com caracteres maiúsculos o significado da instrução e a terceira coluna apresenta de forma resumida a sintaxe da instrução.

Note junto a terceira coluna algumas instruções que possuem em sua sintaxe a definição de elementos complementares, como *destino*, *origem*, *acumulador*, *porta*, *texto_destino*, *texto_origem*, *nome_procedimento*, *tipo_interrupção*, *rótulo*, *contador* e *tabela*, *valor_opcional_pop*, os quais definem o tipo de operação que pode ser efetuado com uma determinada instrução.

A Tabela 7.2 apresenta e descreve o significado de cada um dos elementos apontados no parágrafo anterior.

Tabela 7.2 - Elementos complementares ao uso dos comandos

Elemento	Usado em operações	Descrição
origem	de transferência de dados, aritméticas, de manipulação de bit	Um registro, um endereço de memória ou um valor definido que será usado na operação, mas que não alterará a ação da instrução.
destino	de transferência de dados, de manipulação de bit	Um registro ou endereço de memória que contém o dado que será operacionalizado pela instrução.
acumulador	com as instruções in e out	Transferência de dado do tipo Word para o registrador AX e de dado do tipo byte para o registrador AL.
porta	com as instruções in e out	Especificação de um número de 0 a 255 na definição da porta de entrada ou saída a ser utilizada. O valor fornecido ocorre de forma direta no registrador DX.
texto_destino	de manipulação de texto	Nome do <i>string</i> definido em memória por meio do endereçamento do registrador DI que será substituído pelo resultado da operação.
texto origem	de manipulação de texto	Nome do <i>string</i> definido em memória por meio do endereçamento do registrador SI que será usado na operação de substituição, mas não será alterado pela operação.
nome_procedimento	de sub-rotinas	Definição do nome de uma sub-rotina que será chamada dentro do programa.
tipo_interrupção	com a instrução int	Valor numérico entre 0 e 255 utilizado para definir a interrupção do sistema a ser utilizada.
rótulo	de condições, transferências, de controle iterativo	Definição de um nome de identificação que será usado como ponto de desvio para a ação pretendida.
contador	de deslocamento de rotações	Especificação de um valor numérico para operações de deslocamento e de rotação definidas com o registrador CS em valores entre 0 e 255.

Elemento	Usado em operações	Descrição
tabela	com a instrução xlatb	Nome do endereço de memória usado na tradução de uma tabela usada com o registrador BX.
valor_opcional_pop	com a instrução ret	Definição do número de <i>bytes</i> que será usado nas operações de pilha.

Do conjunto de instruções apresentadas, algumas delas, quando executadas, alteram os valores dos registradores de *flags*. Assim sendo, segue tabela com indicação apenas das instruções que afetam o conjunto de registradores de *flags*. As instruções que não operam ou não afetam seus valores foram omitidas. Cabe salientar que o registrador **TF** não é afetado.

Os registradores de *flags* podem assumir um de quatro comportamentos possíveis, sendo o valor binário **1** (um) ou **0** (zero) quando são assim explicitamente definidos por alguma instrução em execução (sinalizado na tabela como **UM** - um e **ZR** - zero); podem assumir um valor que depende da instrução em execução e neste caso seu valor binário pode ser **1** ou **0** (sinalizado na tabela como **DP** - depende); podem assumir um valor indefinido e neste caso talvez ser o valor binário **1** ou **0**, dependendo da instrução em execução (sinalizado na tabela como **ID** - independe) ou podem não assumir nenhum valor (sinalizado na tabela como **NV** - nenhum valor). Os registradores não afetados pela instrução em execução estão sinalizados com a identificação **NA** - não afetado. A Tabela 7.3 descreve os *flags* afetados com o uso das instruções.

Tabela 7.3 - Instruções e *flags* afetados

Instrução	Flags afetados							
	CF	DF	ZF	SF	OF	PF	AF	IF
AAA	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	IN	IN	IN	IN	DP	NA
AAD	CF	DF	ZF	SF	OF	PF	AF	IF
	IN	NA	DP	DP	IN	DP	IN	NA
AAM	CF	DF	ZF	SF	OF	PF	AF	IF
	IN	NA	DP	DP	IN	DP	IN	NA
AAS	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	IN	IN	IN	IN	DP	NA
ADC	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
ADD	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
AND	CF	DF	ZF	SF	OF	PF	AF	IF
	ZR	NA	DP	DP	ZR	DP	IN	NA
CLC	CF	DF	ZF	SF	OF	PF	AF	IF
	ZR	NA	NA	NA	NA	NA	NA	NA
CLD	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	ZR	NA	NA	NA	NA	NA	NA
CLI	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	NA	NA	NA	NA	NA	ZR
CMC	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	NA	NA	NA	NA

Instrução	Flags afetados							
CMP	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
CMPSB	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
CMPSW	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
DAA	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
DAS	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
DEC	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	DP	DP	DP	DP	DP	NA
DIV	CF	DF	ZF	SF	OF	PF	AF	IF
	IN	NA	IN	IN	IN	IN	IN	NA
IDIV	CF	DF	ZF	SF	OF	PF	AF	IF
	IN	NA	IN	IN	IN	IN	IN	NA
IMUL	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	IN	IN	DP	IN	IN	NA
INC	CF	DF	ZF	SF	OF	PF	AF	IF
	IN	NA	DP	DP	DP	DP	DP	NA
INT	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	NA	NA	NA	NA	NA	ZR
MUL	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	IN	IN	DP	IN	IN	NA
NEG	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
OR	CF	DF	ZF	SF	OF	PF	AF	IF
	ZR	NA	DP	DP	ZR	DP	IN	NA
RCL	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
RCR	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
REP	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	DP	NA	NA	NA	NA	NA
REPE	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	DP	NA	NA	NA	NA	NA
REPNE	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	DP	NA	NA	NA	NA	NA
REPNZ	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	DP	NA	NA	NA	NA	NA
REPZ	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	DP	NA	NA	NA	NA	NA

Instrução	Flags afetados							
ROL	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
ROR	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
SAHF	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
SAL	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
SAR	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
SBB	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
SCASB	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
SCASW	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
SHL	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
SHR	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	NA	NA	DP	NA	NA	NA
STC	CF	DF	ZF	SF	OF	PF	AF	IF
	UM	NA	NA	NA	NA	NA	NA	NA
STD	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	UM	NA	NA	NA	NA	NA	NA
STI	CF	DF	ZF	SF	OF	PF	AF	IF
	NA	NA	NA	NA	NA	NA	NA	UM
SUB	CF	DF	ZF	SF	OF	PF	AF	IF
	DP	NA	DP	DP	DP	DP	DP	NA
TEST	CF	DF	ZF	SF	OF	PF	AF	IF
	ZR	NA	DP	DP	ZR	DP	NA	NA
XOR	CF	DF	ZF	SF	OF	PF	AF	IF
	ZR	NA	DP	DP	ZR	DP	IN	NA

7.6 - Uso do modo ajuda

O estudo da linguagem *Assembly* para microcomputadores IBM-PC é sempre realizado a partir da edição 8086/8088 do microprocessador Intel. Uma vez que se tenha uma boa noção do funcionamento das instruções e da estrutura da arquitetura do microprocessador, fica mais fácil utilizar processadores de 32 e 64 *bits*.

O programa de simulação do microprocessador Intel 8086/8088 **emu8086** foi desenvolvido com objetivos didáticos. Ele possui uma série de recursos que o tornam uma ferramenta bastante prática para o uso escolar.

Este foi um dos principais fatores que incentivaram o uso dessa ferramenta para a elaboração desta obra, pois além da execução pausada das instruções *Assembly*, a ferramenta disponibiliza várias janelas que mostram informações sobre os registradores, *flags*, pilha, entre outros detalhes, como a existência de um *tutorial* muito bem elaborado. Esses detalhes vieram ao encontro dos objetivos deste trabalho.

Apesar do **emu8086** ser uma ferramenta prática e possuir diversos recursos, tem algumas pequenas limitações e restrições no uso de alguns recursos, como, por exemplo, não trabalhar com o código de função **08h** da interrupção **21h** para a entrada de dados sem eco de teclado, permitir apenas o uso das diretivas de definição de tipos de variáveis **DB** e **DW** (que ainda serão explanadas).

Para obter maiores detalhes a respeito de funcionalidades e restrições do uso da ferramenta, sugere-se uma leitura detalhada da documentação anexa à ferramenta. Execute no editor do programa (tela principal) o comando de menu **help/documentation and tutorial**, que apresenta o modo ajuda indicado na Figura 7.45.

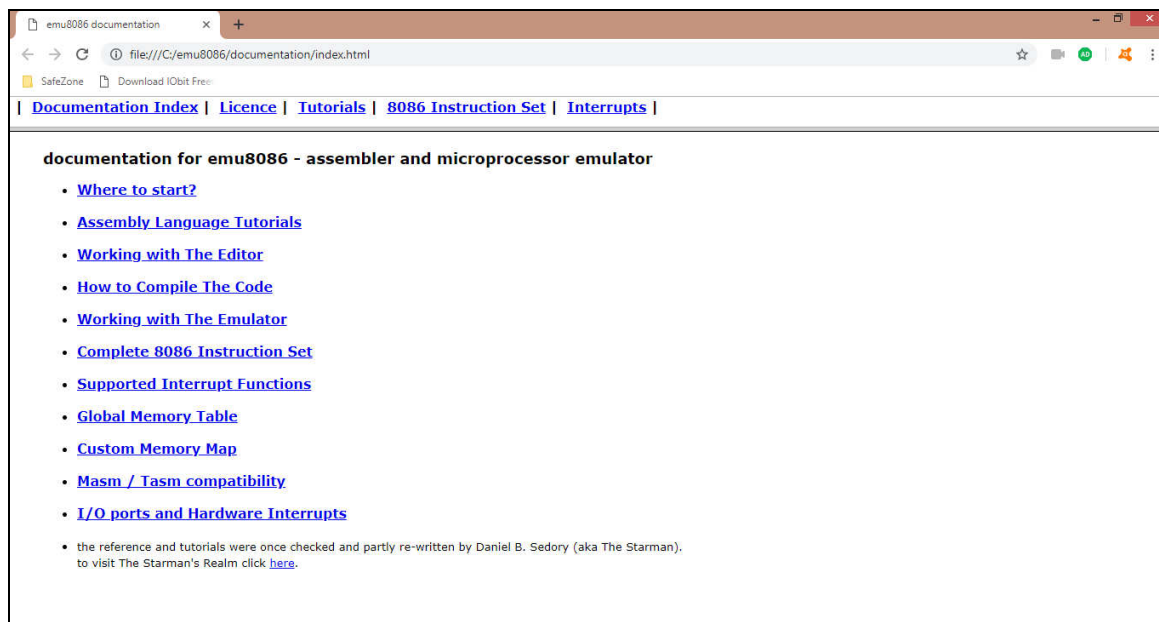


Figura 7.45 - Tela do modo ajuda.

0D82:0100 B402	MOV	AH, 02
0D82:0102 B241	MOV	DL, 41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 8

PROGRAMAÇÃO BÁSICA

Este capítulo traz alguns elementos básicos complementares utilizados na programação de baixo nível na linguagem de programação Assembly 8086/8088. São abordados manipulação de registradores, manipulação de dados, tipos de dados a serem associados a variáveis, processo de cálculos matemáticos, novas instruções matemáticas e valores hexadecimais com oito dígitos.

8.1 - Manipulação de registradores e dados

Anteriormente tanto no programa **Enhanced DEBUG** como no programa **emu8086** foi bastante usado o comando **MOV** para a movimentação de valores entre registradores. A característica básica do comando **MOV** é movimentar (carregar) um determinado dado de um endereço de memória fonte (registrador) para outro endereço de memória destino ou movimentar para um registrador certo valor de forma direta. Além das formas usadas a instrução **MOV** pode ser utilizada de outras formas para movimentar os dados com o acesso de memória (DANDAMUDI, 2000 e HYDE, 2003) baseado em acesso direto, indireto por registrador, indexado, acesso de base indexada e de base indexada mais deslocamento, os quais serão apresentados mais adiante com alguns novos detalhes.

8.1.1 - Endereçamento imediato

O endereçamento imediato está relacionado à possibilidade de carregar um registrador com um valor de 8 ou 16 *bits*. Imagine a necessidade de mover um valor hexadecimal de 8 *bits* para os 8 *bits* mais significativos do registrador geral **AX**. Neste caso o valor seria movido para o registrador mais significativo **AH** com a linha de código:

```
MOV AH, FFh
```

O valor **FFh** (equivalente a **1111 1111** em notação binária, pode ser entendido como valor decimal positivo **255** ou decimal negativo **-1**) está sendo movimentado de forma direta para o registrador **AH**, sendo esta uma das formas mais comuns desse tipo de operação. Lembre-se de que a consideração em relação ao fato de o valor ser positivo ou negativo depende da forma como este estará sinalizado. Para ser visto como negativo, os registradores **CF**, **SF** e **AF** devem estar setados com valor 1.

8.1.2 - Endereçamento por registrador

O endereçamento por registrador está relacionado com a possibilidade de carregar um determinado registrador com o valor existente em outro registrador, uma das formas utilizadas nos exemplos anteriores. Imagine a necessidade de mover um valor existente no registrador de 8 *bits* mais significativo do registrador geral **CX** para o de 8 *bits* menos significativo do registrador geral **BX**. Neste caso o valor seria movido do registrador **CH** para o **BL** com a linha de código:

```
MOV BL, CH
```

Após a transferência do valor, o registrador **CH** permanece inalterado, assim como também permanece inalterado o valor que porventura exista no registrador **BH**. Apenas o registrador **BH** foi alterado com o valor do registrador **CH**.

8.1.3 - Endereçamento por deslocamento (offset)

As duas formas comentadas anteriormente são os mecanismos mais rápidos e fáceis de usar, pois o endereçamento é passado de forma imediata, seja por intermédio de um dado (valor) informado a um registrador ou de transferência de valores entre registradores. Ocorre muitas vezes a necessidade de obter dados que estão armazenados na memória e não em registradores. Torna-se necessário utilizar o endereçamento por deslocamento (também conhecido como endereçamento por *offset*).

Para que seja possível obter o endereçamento em memória de um determinado dado, é necessário definir uma constante de 16 *bits* (a qual é denominada *deslocamento* ou *offset*) que somada ao conteúdo de um registrador (de 16 *bits*) fornece a real posição de memória em que se encontra aquele dado.

O endereço efetivo (deslocamento ou *offset*) pode ser obtido de diversos modos. Inicialmente basta conhecer de forma básica os endereçamentos indexado e de base indexada com deslocamento.

O endereçamento indexado, quando de sua indicação na própria instrução de movimentação, está relacionado a um valor numérico hexadecimal entre colchetes.

Imagine a necessidade de mover um valor de endereço de deslocamento de memória para o registrador **AX**, considerando que o valor em questão corresponde ao endereço de deslocamento **00100h**:

```
MOV AX, [00100h]
```

O endereço de memória **00100h** (valor hexadecimal) representa o valor de deslocamento (*offset*) do segmento em uso. O endereçamento indireto está relacionado, quando de sua indicação na própria instrução de movimentação, associado a uma posição de memória de um determinado registrador entre colchetes. Imagine a necessidade de mover para o registrador geral **AX** a posição de memória do conteúdo do registrador geral **BX**:

```
MOV AX, [BX]
```

É possível ainda determinar a posição de memória utilizando:

```
MOV BX, [00100h]  
MOV AX, [BX]
```

Neste caso, primeiramente se está carregando o registrador geral **BX** de forma direta com o endereço de memória **00100h**, em seguida, de forma indireta, o registrador geral **AX** com o endereço de memória armazenado no registrador geral **BX**.

O endereçamento de base indexada está relacionado, quando de sua indicação entre colchetes, ao endereço inicial de memória somado a ele um valor constante de deslocamento. Imagine a necessidade de mover para o registrador geral **AX** o conteúdo existente no registrador geral **BX** mais a posição do endereço de memória **00100h**:

```
MOV AX, [00100h+BX]
```

Esta indicação pode ainda ser escrita:

```
MOV AX, [BX+00100h]  
MOV AX, 00100h[BX]  
MOV AX, [BX]+00100h
```

Esse tipo de endereçamento permite o acesso a dados que estejam armazenados em lugares diferentes da memória. Esse tipo de indicação facilita o acesso de dados existentes em matrizes (tabelas), registros ou outras estruturas de dados que ainda serão apresentadas.

Além das formas indicadas, a passagem de endereço de deslocamento pode também ser definida como:

MOV DX, OFFSET var

Esse tipo de endereçamento permite definir o endereço de deslocamento utilizado pela variável (neste caso, variável **var**) para o registrador geral em uso (neste caso **DX**).

8.1.4 - Outras formas de deslocamento

Além das formas de endereçamentos de deslocamento apresentadas é possível realizar outras operações como: endereçamento com deslocamento direto; endereçamento com deslocamento indireto sobre registrador; endereçamento com deslocamento por registrador geral; endereçamento com deslocamento por registrador ponteiro e endereçamento de deslocamento sobre registrador geral com registrador de ponteiro.

A ação de endereçamento com deslocamento direto é definida a partir da instrução **MOV AX, var**, onde **var** é a definição do endereço de uma variável na memória.

O endereçamento com deslocamento indireto sobre registrador ocorre a partir da leitura de um registrador base **BX** ou **BP** ou mesmo de um registrador de ponteiro (índice) **SI** ou **DI** por meio de instrução similar a **MOV AX, [BX]**.

Para o endereçamento com deslocamento de registrador geral o endereço é lido de um registrador base **BX** ou **BP** adicionado um valor de deslocamento a partir da instrução **MOV AX, [BX+0010h]**.

Para o endereçamento com deslocamento por registrador ponteiro usa-se sintaxe de instrução idêntica ao uso de deslocamento de registrador geral. Neste caso, levando-se em conta o uso dos registradores de ponteiro **DI** ou **SI** com a instrução **MOV AX, [DI+0010h]**.

Os modos de endereçamento indexados são obtidos a partir da soma de um registrador base (**BX** ou **BP**) com um registrador de ponteiro (**SI** ou **DI**). O registrador **BP** é usado quando se necessita trabalhar com a pilha. A sintaxe deste tipo de ação pode ser estabelecida com a instrução **MOV AX, [BX][DI]** ou **MOV AX, tabela[BX][DI]**.

8.2 - Tipos de dados em Assembly

A linguagem de programação *Assembly 8086/8088*, por meio das diretivas **DB** (*Define Byte*), **DW** (*Define Word*), **DD** (*Define Doubleword*), **DQ** (*Define Quadword*) e **DT** (*Define Ten Bytes*), permite manipular alguns tipos dados básicos associados a valores e as suas variáveis. Analogamente a outras linguagens, é possível manipular valores do tipo inteiro, real e caractere (*string*). Os tipos de dados em *Assembly* podem tratar os seguintes dados:

- ◆ A diretiva **DB** pode ser utilizada para manipular dados do tipo *string* e também valores inteiros curtos (8 *bits*), com a capacidade de manipular valores de -128 até 255 (de -2^7 até $2^8 - 1$).
- ◆ A diretiva **DW** pode ser usada para manipular valores inteiros curtos e também valores reais curtos (16 *bits*, sendo 2 *bytes* consecutivos, 1 *word*), com a capacidade de manipular valores de -32.768 até 65.535 (de -2^{15} até $2^{16} - 1$).
- ◆ A diretiva **DD** pode ser usada para manipular valores inteiros longos e também valores reais curtos (32 *bits*, sendo 4 *bytes* consecutivos, 1 *doubleword*), com a capacidade de manipular valores de $-2.147.483.648$ até $4.294.967.295$ (de -2^{63} até $2^{64} - 1$).
- ◆ A diretiva **DQ** pode ser usada para manipular valores reais longos (64 *bits*, sendo 8 *bytes* consecutivos, 1 *quadword*), com a capacidade de manipular valores de $-9.223.372.036.854.775.808$ até $18.446.744.073.709.551.615$ (de -2^{63} até $2^{64} - 1$).
- ◆ A diretiva **DT** pode ser usada para manipular valores que ocupem até dez *bytes* consecutivos.

Das diretivas apresentadas o programa **emu8086** aceita apenas **DB** e **DW**, as quais são mais do que suficientes para os objetivos deste trabalho. Além disso, a ferramenta **emu8086** é um instrumento de apoio básico e didático ao aprendizado das noções preliminares da linguagem de programação de computadores *Assembly 8086/8088*.

A seguir são apresentados alguns exemplos das diretivas de definição de tipos de dados. Para o uso desse recurso deve-se observar a sintaxe seguinte:

variável diretiva valor1 [,<valor1>], ...

O parâmetro **variável** é um rótulo de identificação do nome da variável para utilização dos dados referenciados no programa, o parâmetro **diretiva** é a definição de um tipo de dado válido e o parâmetro **valor** é a definição do valor associado a uma variável.

Há a possibilidade de definir o parâmetro **valor** com o operador **?** (interrogação), que tem por finalidade reservar espaço na memória de acordo com o tipo de diretiva de definição em uso, sem efetuar a inicialização da variável com algum valor.

Após a definição do valor da expressão pode-se também reservar espaço na área de memória com a utilização da diretiva **DUP**. Assim sendo, poder-se-ia utilizar esse recurso de algumas maneiras.

```
caractere1 DB 'A'
caractere2 DB 41h
caractere3 DB 01000001b
```

A definição anterior estabelece para a variável com denominação **caractere** o espaço de um *byte* simples para o armazenamento do caractere **A**, ou dos valores em hexadecimal e binário para a representação do caractere **A**.

```
valor1 DW 26987d
```

A definição de um valor numérico positivo de 16 *bits* em notação decimal é internamente convertida de forma automática no seu valor equivalente em notação hexadecimal. O valor **26987d** é considerado internamente **696Bh**.

```
valor2 DW -25476d
```

A definição de um valor numérico negativo de 16 *bits* em notação decimal é internamente convertida de forma automática no seu valor equivalente em notação hexadecimal, considerando a complementação por 2. O valor **-25476d** é considerado internamente **9C7Ch**.

Na possibilidade de definir uma variável que não possua um valor inicial, ela pode ser definida como:

```
valor3 DB ?
valor2 DW ?
```

A definição de valores do tipo *string* para variáveis com a diretiva **DB** pode ser realizada no estilo de múltiplos dados. Considere a seguir a definição da variável **palavra** com o conteúdo **"Alo mundo!"**, a qual pode ser definida como:

```
palavra DB 'A', 'l', 'o', ' ', 'M', 'u', 'n', 'd', 'o', '!'
palavra DB 'A'
        DB 'l'
        DB 'o'
        DB ' '
        DB 'M'
        DB 'u'
        DB 'n'
        DB 'd'
        DB 'o'
        DB '!'
```

As duas formas anteriores equivalem à seguinte definição:

```
palavra DB 'Alo Mundo!'
```

O mesmo conceito também pode ser utilizado com as demais diretivas (considerando o fato de a ferramenta de compilação as aceitar). Considere uma matriz de dados denominada **vetor** com a capacidade de armazenar cinco valores numéricos.

```
vetor DW 0
      DW 0
      DW 0
      DW 0
      DW 0
```

Que também pode ser definida como:

```
vetor DW 0, 0, 0, 0, 0
```

No caso anterior poder-se-ia também reservar espaço de memória utilizando a diretiva **DUP**, como mostrado a seguir:

```
vetor DW 5 DUP (0)
```

A diretiva **DUP** é largamente utilizada quando há necessidade de definir uma variável que seja do tipo vetor (matriz de uma dimensão) ou tabela (matriz de duas ou mais dimensões). São formas válidas:

```
vetor1 DB 8 DUP (?)
vetor2 DW 4 DUP ('?')
vetor3 DB 2 DUP ('Alo ')
```

A primeira definição estabelece para a variável **vetor1** a reserva de 8 bytes de memória com valor de inicialização desconhecido. A segunda definição estabelece para a variável **vetor2** a reserva de 4 words com a definição do caractere ?. A terceira e última definição estabelece para a variável **vetor3** a definição de 2 bytes inicializados com Alo Alo.

Para os casos de definição de uma tabela, a diretiva **DUP** deve ser utilizada com a seguinte sintaxe:

```
matriz DW 5 DUP (3 DUP (0))
```

Neste caso está sendo definida uma variável **matriz**, a qual é inicializada com 15 words com valor zero. Essa forma seria algo similar a uma tabela de cinco linhas e três colunas.

A diretiva **DUP** pode ser usada de muitas formas para inicializar o valor de uma variável. Por exemplo, imagine que se deseja inicializar uma variável denominada **dados** com o seguinte conteúdo: **444433322espaço 444433322espaço 444433322espaço**. Neste caso, usar-se-ia a seguinte sintaxe:

```
dados DB 3 DUP (4 DUP ('4'), 3 DUP ('3'), 2 DUP ('2'), 1 DUP (' '))
```

A variável **dados** é uma matriz com três posições, e cada posição possui a inicialização de quatro caracteres 4; três caracteres 3; dois caracteres 2 e 1 caractere com um espaço em branco.

O uso de diretivas de definição para tipos de dados sugere o uso em conjunto das diretivas **DATA** e **CODE**, que permitem o estabelecimento de uma sequência de definições.

Por exemplo, considere o programa **MENSAGEM1** apresentado no capítulo anterior, conforme em seguida:

```
; You may customize this and other start-up templates;  
; The location of this template is c:\emu8086\inc\0_com_template.txt
```

```
org 100h
```

```
MOV AH, 09h  
LEA DX, mensagem  
INT 21h  
INT 20h
```

```
mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 24h
```

```
ret
```

No programa **MENSAGEM1** o código é definido a frente da definição da variável **mensagem**. Esta forma de definição permite que o programa montador crie na memória o código de apresentação da mensagem e coloque a mensagem após a última linha do código associando o endereço escolhido automaticamente para a variável **mensagem** junto a instrução **LEA DX, mensagem**. No entanto, a forma de escrita apresentada não é a forma mais comum de definição de um programa em *Assembly*, pois normalmente se define primeiro as variáveis do programa e depois seu código.

Como segundo exemplo, no programa **emu8086** execute o comando de menu **file/new/com template**, sobre a linha **07** escreva o código de programa a seguir e com o comando **file/save as...** grave o programa com o nome **MENSAGEM2**:

```
JMP inicio
```

```
mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 21h, 24h
```

```
inicio:  
    LEA DX, mensagem  
    MOV AH, 09h  
    INT 21h  
    INT 20h
```

A definição da variável **mensagem** está ocorrendo nessa versão antes do início das linhas de código de controle do programa. Por esta razão é necessário indicar no código do programa o uso da instrução **JMP** que salta para o rótulo indicado como **inicio**, onde estão as instruções do programa. Observe a Figura 8.1.

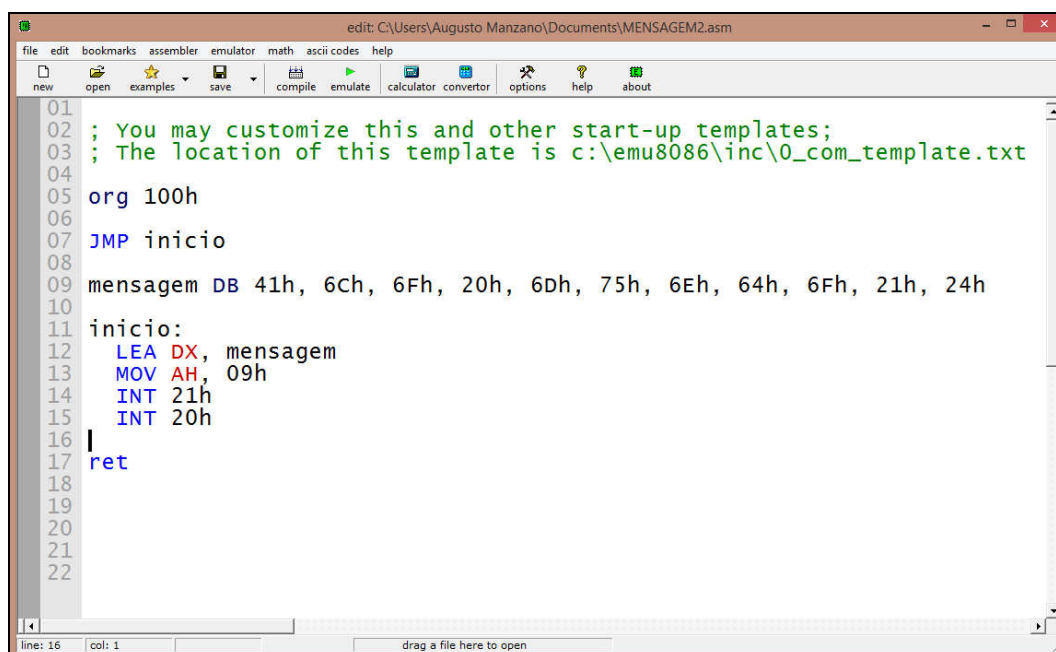


Figura 8.1 - Programa MENSAGEM2.

Se retirada a linha **JMP inicio** do código do programa, ocorrerá um erro na execução do programa. O erro ocorre quando há a tentativa de montar o programa na memória, pois o código do programa deve vir sempre à frente da definição dos dados a serem manipulados pelo programa. Por esta razão a segunda versão utiliza a instrução **JMP** que faz com que seja executado primeiramente o código do programa que buscará os dados na variável **mensagem**.

Uma forma de proceder com uma montagem de programa mais adequada é fazer uso das diretivas **.DATA** e **.CODE**. Desta forma é possível definir primeiramente os dados do programa antes da parte do código do programa.

Definir primeiramente os dados para depois definir o código é uma prática de programação considerada correta e elegante. Assim sendo, observe o trecho de código seguinte:

```
.DATA
mensagem DB 41h, 6Ch, 6Fh, 20h, 6Dh, 75h, 6Eh, 64h, 6Fh, 21h, 24h

.CODE
LEA DX, mensagem
MOV AH, 09h
INT 21h
INT 20h
```

Execute o comando de menu **file/new/com template**, informe o código anterior a partir da linha **04** e grave o programa com o comando **file/save** com o nome **MENSAGEM3**. Observe a Figura 8.2.

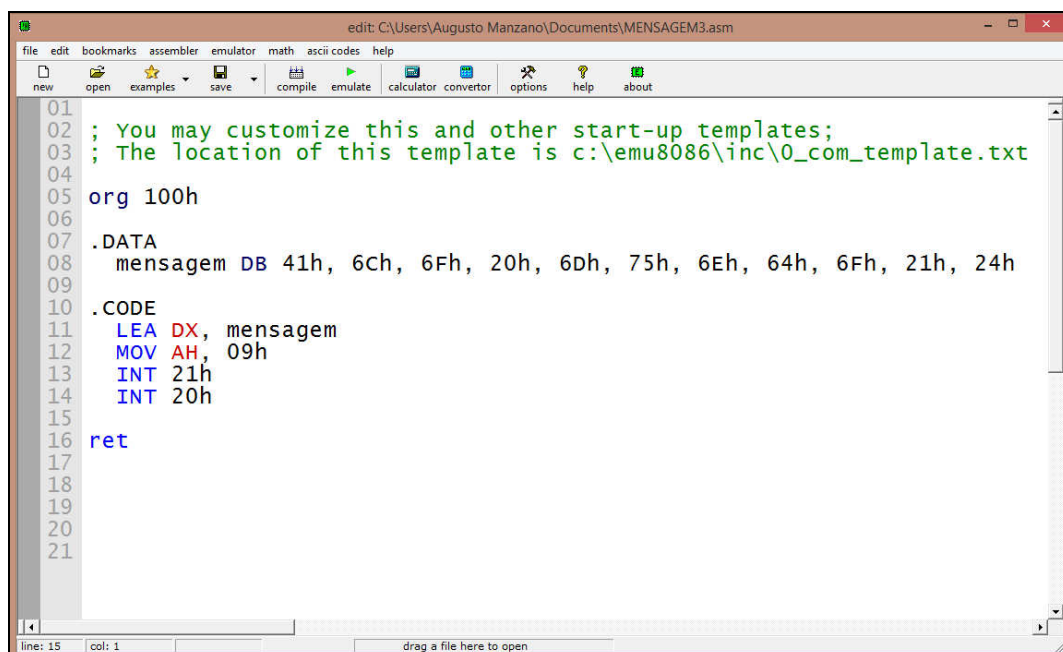


Figura 8.2 - Programa MENSAGEM3.

Veja o uso das diretivas **.DATA** e **.CODE** (atenção para os símbolos de ponto usados nas diretivas). Desta forma, definem-se duas áreas de programa: a área de dados (**.DATA**) e a área de código (**.CODE**). A partir desse instante esta será a abordagem usada para a codificação de todos os programas que, quando compilados, possuírem a extensão **.COM**.

É importante ressaltar que as ferramentas de montagem (os programas *assembler*) não fazem nenhuma questão quanto aos caracteres utilizados nos códigos serem maiúsculos ou minúsculos. É possível escrever um programa inteiro com todos os caracteres grafados no formato minúsculo ou no formato maiúsculo. No entanto, por questões de melhor visualização do código, nesta obra adotou-se o critério de grafar em caracteres maiúsculos as instruções e as diretivas. Rótulos de identificação de posição de código e de variáveis são escritos em formato minúsculo.

8.3 - Cálculos matemáticos intermediários

Anteriormente foram apresentadas algumas formas básicas para a realização de pequenos cálculos matemáticos com a ferramenta **Enhanced DEBUG**. Neste tópico este assunto volta à tona acrescido de alguns novos detalhes, como, por exemplo, levar em consideração no processo de cálculo o registrador de estado **CF** (*Carry Flag*).

A linguagem de programação de computadores *Assembly 8086/8088* tem alguns comandos reservados para cálculos aritméticos, tais como adição (**ADD** ou **ADC**), subtração (**SUB** ou **SBB**), multiplicação (**MUL** ou **IMUL**) e divisão (**DIV** ou **IDIV**). Lembre-se de que os comandos **ADD**, **SUB**, **MUL** e **DIV** já foram apresentados e serão revistos neste tópico (é sempre bom lembrar).

É conveniente considerar que os comandos de cálculos aritméticos alteram os registradores de estado (*flags*) **OF**, **SF**, **ZF**, **AF**, **PF** e **CF** existentes na memória. O registrador de estado afetado mais importante em operações aritméticas é **CF**, pois sinaliza na memória a ocorrência do efeito "vai um" quando do estouro da manipulação aritmética sobre a posição de um registrador geral.

8.3.1 - Adição

As operações de adição podem ocorrer com a utilização dos comandos **ADD** ou **ADC**. O comando **ADD** tem a mesma finalidade do operador aritmético "+" existente em outras linguagens de programação. Já o comando **ADC** possui um pequeno diferencial, pois além de executar a mesma operação do comando **ADD**, ele soma ao valor o valor do registrador de estado **CF** que pode ser **0** (zero) ou **1** (um).

O funcionamento lógico do comando **ADD** será **ADD DESTINO, ORIGEM** ($\text{DESTINO} \leftarrow \text{DESTINO} + \text{ORIGEM}$).

ADD AX, 5d

No exemplo apresentado o registrador geral **AX** está sendo adicionado com o valor decimal **5**. Neste contexto o valor **5** é a definição de um valor constante no registrador. Se no registrador geral **AX** existir algum valor anterior, o valor **5** será somado ao existente.

Observação

A definição de valores constantes pode ocorrer com o uso de sinalizadores da base numérica a que o número é referenciado. Por exemplo, o valor **5d** indica a definição de um valor do tipo *decimal*. Assim sendo pode-se usar como sinalização de base após um valor constante as letras: **d** (decimal), **h** (hexadecimal), **o** (octal) e **b** (binário).

O funcionamento lógico da instrução **ADC** será **ADC DESTINO, ORIGEM** ($\text{DESTINO} \leftarrow \text{DESTINO} + \text{ORIGEM} + \text{CF}$).

ADC AX, 5d

No exemplo apresentado o registrador geral **AX** está sendo adicionado com o valor decimal **5** mais o valor que estiver no registrador de estado **CF**, que pode ser **0** (zero) ou **1** (um).

Tome como base um programa que deva executar a soma de dois valores numéricos que ocupem no máximo 1 *word*. O primeiro valor deve estar associado a uma variável denominada **a**, o segundo valor a uma variável denominada **b** e definir uma terceira variável denominada **x** para armazenar o valor da soma.

Para a criação deste programa execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas **<Ctrl> + <a>** para selecionar o texto apresentado e em seguida acione ****. Em seguida escreva o código do programa seguinte:

```

;*****
;*      Programa: ADICA01.ASM      *
;*****

.MODEL small
.STACK 512d

.DATA
    a DW 6d
    b DW 2d
    x DW 0, '$'

.CODE
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, a
    ADD AX, b
    MOV x, AX
    ADD x, 30h
    MOV DX, OFFSET x

    MOV AH, 09h
    INT 21h

    MOV AH, 4Ch
    INT 21h

```

Grave o programa por meio dos comandos de menu **file/save** na pasta **Documentos** com o nome **ADICA01**. A Figura 8.3 apresenta o programa definido dentro do editor de textos do ambiente de programação **emu8086**.

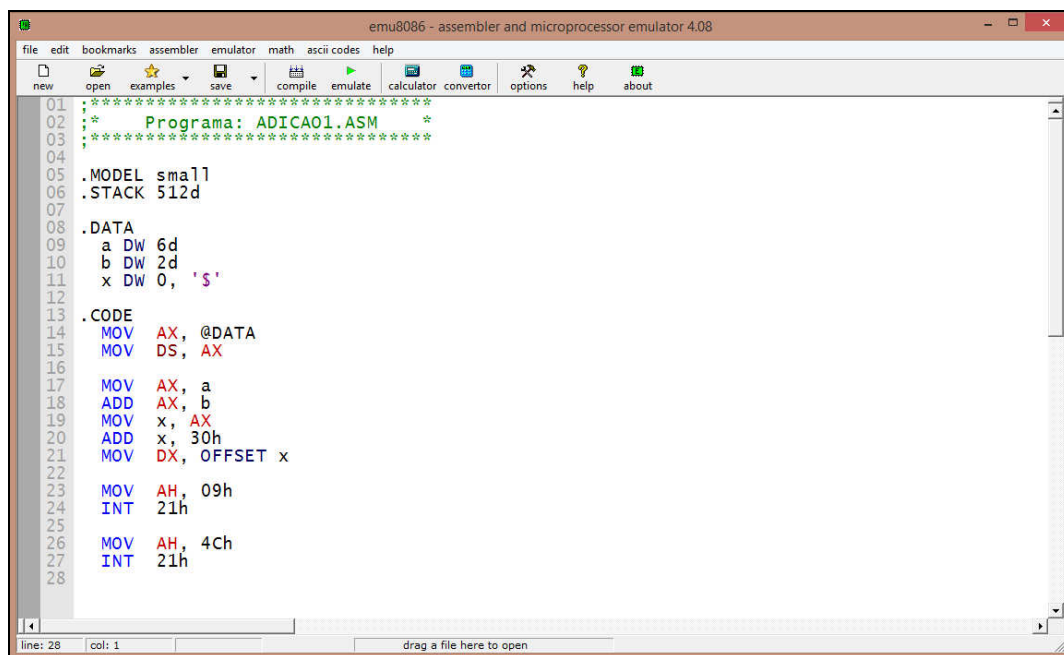


Figura 8.3 - Programa ADICA01 na ferramenta emu8086.

Nas linhas **05** e **06** estão sendo utilizadas duas novas diretivas, sendo **.MODEL** e **.STACK**.

A diretiva **.MODEL** indica o tipo de modelo de memória que deve ser usado pelo programa. Deve ser utilizada antes de qualquer definição de segmento de memória, ou seja, deve ser a primeira linha do programa.

O parâmetro utilizado após a diretiva **.MODEL** pode ser¹ **small**, **medium**, **compact**, **large** e **huge**, os quais possuem o seguinte significado:

O modelo **small** é o modo mais adequado para a maioria das aplicações, devido à sua rapidez de carga e facilidade de depuração. Esse modelo estabelece que o código do programa está em um segmento de memória e os dados estão em outro segmento, cada um ocupando menos de 64KB de memória.

- ◆ O modelo **medium** estabelece que o código do programa pode ultrapassar a marca de 64 KB e pode existir mais de um segmento de código, enquanto os dados estarão alocados em um único segmento com até 64 KB.
- ◆ O modelo **compact** estabelece que o código do programa não pode ser maior que 64 KB, porém permite que para os dados do programa possa ser utilizado mais de um segmento de dados, ou seja, mais de 64 KB.
- ◆ O modelo **large** estabelece que tanto o código do programa como os seus dados podem utilizar mais de 64 KB. No entanto arranjos de dados (*arrays*) não podem ultrapassar 64 KB.
- ◆ O modelo **huge** estabelece que código do programa, dados e *arrays* podem utilizar mais de 64 KB.

A diretiva **.STACK** tem por finalidade estabelecer a reserva de espaço na pilha do programa. O tamanho da pilha a ser definido depende de alguns fatores com relação à chamada de sub-rotinas, registradores salvos, interrupções e passagens de parâmetros. Normalmente, utiliza-se um tamanho em torno de 512 *bytes* (valor decimal). Desta forma, você pode usar o valor 512 como padrão.

Na linha **08** define-se o segmento de dados **.DATA** com a criação de três variáveis **a**, **b** e **x**, todas do tipo **DW** com seus respectivos valores decimais **6**, **2** e **0**. O caractere "\$" existente após a definição da variável **x** tem a mesma ação utilizada nos programas **MENSAGEM1**, **MENSAGEM2** e **MENSAGEM3**. A diretiva **.DATA** permite que seja definido o espaço do segmento de dados do programa.

A partir da linha **13** é definido o segmento **.CODE**, o qual possui a sequência de instruções do programa que soma os valores entre as variáveis **a** e **b**, faz a atribuição do valor à variável **x** e executa a apresentação do resultado na tela do monitor de vídeo. A diretiva **.CODE** permite que seja definido o espaço do segmento de código do programa.

Observação

Os programas apresentados neste capítulo têm estruturas simples e devem ser escritos como são exibidos. Não tente fazer nenhuma alteração, pois talvez não surtam os efeitos pretendidos. O programa **ADICAO1** está preparado apenas para tratar somas de unidades. Valores que utilizem dezenas, centenas ou milhares não serão apresentados. Portanto, não se preocupe ainda com esses detalhes.

As linhas **14** (**MOV AX, @DATA**) e **15** (**MOV DS, AX**) definem o acesso do segmento de código ao segmento de dados. Com a linha **17** (**MOV AX, a**) ocorre a movimentação do valor da variável **a** para o registrador **AX** e com a linha **18** (**ADD AX, b**) é efetuado o processamento da operação de adição do valor da variável **b** sobre o valor da variável **a** armazenado no registrador **AX**. A linha **19** (**MOV AX, x**) movimenta o valor somado do registrador **AX** para a variável **x**, efetuando a operação $x = a + b$.

Na linha **20** (**ADD x, 30h**) encontra-se a adição do valor **30h** à variável **x**. Lembre-se de que esse recurso de adicionar o valor **30h** a um resultado possibilita que esse resultado seja apresentado. A linha **21** (**MOV DX, OFFSET x**) efetua o cálculo do tamanho que a variável **x** ocupa na memória e armazena esta quantidade no registrador **DX** para que seja usada no momento da apresentação do resultado quando da execução das linhas **23** e **24**.

As linhas **23** (**MOV AH, 09h**) e **24** (**INT 21h**) fazem a apresentação do resultado da soma que está armazenado na variável **x** apontada no registrador **DX** e as linhas **26** (**AH, 4Ch**) e **27** (**INT 21h**) fazem o encerramento do programa. O valor **4Ch** armazenado no registrador **AX** na linha **27** estabelece que ocorrerá o encerramento do programa, fazendo-se o retorno do controle operacional para sistema operacional a partir da chamada da interrupção **21h**.

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <a>** e **** do editor de texto e escreva o programa anterior gravando-o na pasta **Documentos** com os comandos de menu **file/save** com o nome **ADICAO2**.

¹ Dependendo do programa montador e versão utilizada, pode existir outros modelos de memória como **tiny** que estabelece que tanto o código como os dados deverão estar no mesmo segmento de 64KB de memória.

Tome como base um programa que execute a equação $X \leftarrow A + B + CF$, em que a variável **a** possui o valor decimal **6**, a variável **b** o valor decimal **2** e **CF** é o valor existente no registrador de estado **CF**. Observe o código do programa a seguir:

```
;*****
;*   Programa: ADICA02.ASM   *
;*****

.MODEL small
.STACK 512d

.DATA
a DW 6d
b DW 2d
x DW 0, '$'

.CODE
MOV AX, @DATA
MOV DS, AX

STC
MOV AX, a
ADC AX, b
MOV x, AX

ADD x, 30h
MOV DX, OFFSET x

MOV AH, 09h
INT 21h

MOV AH, 4Ch
INT 21h
```

A Figura 8.4 mostra a disposição do código dentro do ambiente de desenvolvimento do programa **emu8086**.

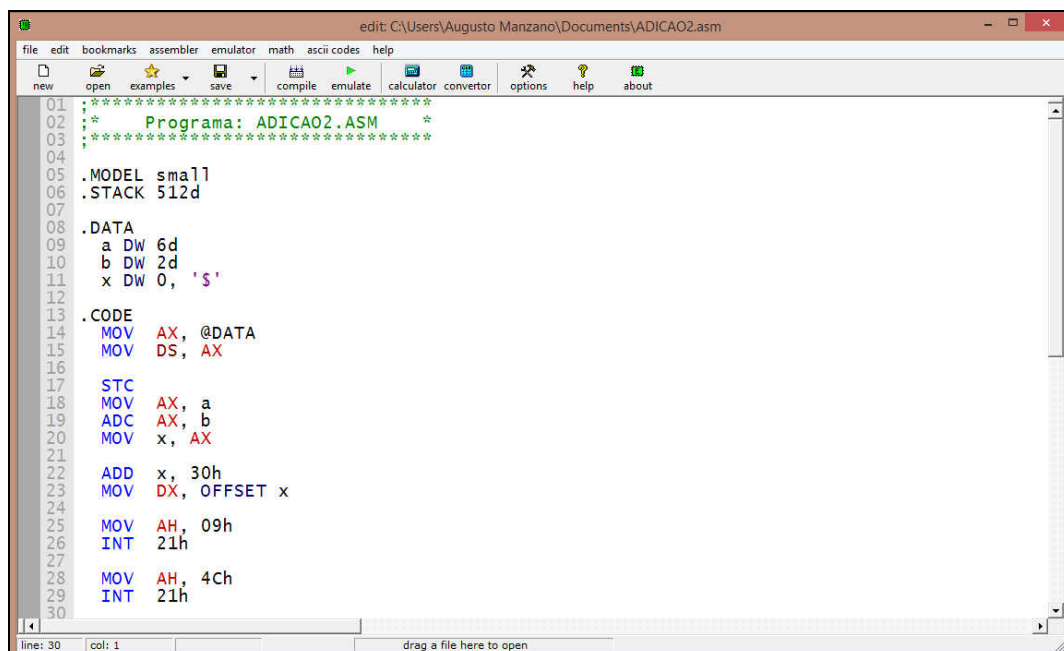


Figura 8.4 - Programa ADICA02 na ferramenta emu8086.

A título de ilustração e de maior intimidade com a operacionalização da ferramenta **emu8086**, serão descritos os passos de ação das instruções do programa. Execute o comando de menu **assembler/compile and load in emulator**, ou acione a tecla de função **<F5>**, ou o botão **emulate** da barra de ferramentas. Assim que a ação anterior de execução do programa for

solicitada, serão apresentadas as telas de demonstração do programa, como indica a imagem da Figura 8.5, que retrata o primeiro trecho de ação.

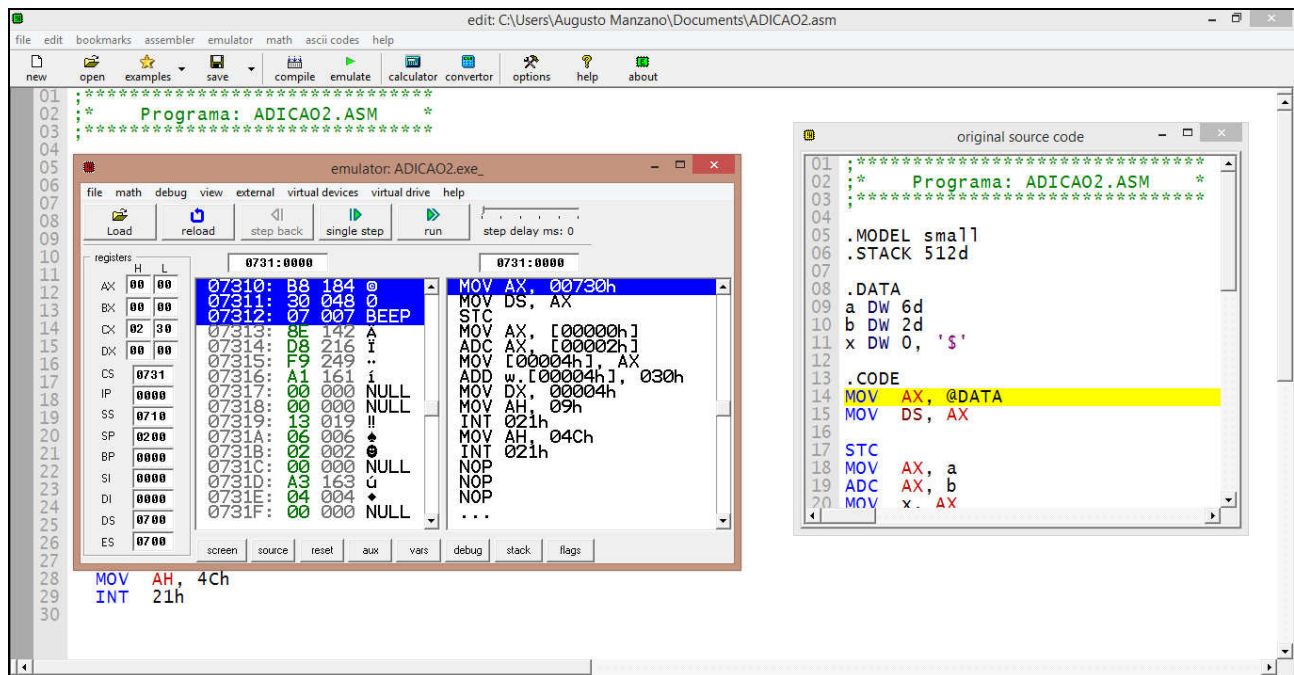


Figura 8.5 - Telas de ação com dados preliminares do programa.

A Figura 8.5 apresenta a barra de ação (barra amarela) da janela **original source code** posicionada após a diretiva **.CODE** da linha 14 do programa na linha **MOV AX, @DATA**. A janela **emulator: ADICAO2.exe** mostra os registradores de segmento **CS** posicionado no endereço **0731h** e **IP** posicionado no endereço **0000h** (**CS:IP – 0731:0000**). Mas lembre-se de que os valores de segmento e deslocamento apresentados podem variar conforme o computador ou o momento. Assim sendo, aqui são válidos apenas como referência. A estrutura **CS:IP** é usada para indicar o endereço de memória do código sendo executado.

Pressione a tecla de função **<F8>** (primeira vez) ou acione o botão **single step** da janela **emulator: ADICAO2.exe**. Observe que nessa primeira etapa o programa mostra a alteração do endereço dos registradores de deslocamento **IP** para o valor **0003h**, pois é o endereço de início da primeira instrução do programa (linha 15). Basta olhar para a barra de ação amarela da janela **original source code** sobre o comando **MOV DS, AX** que transfere para o registrador geral **DS** o conteúdo de **AX**, o valor do endereço de segmento em que as variáveis e seus valores estão definidos.

Acione a tecla de função **<F8>** pela segunda vez e observe os dados apresentados em tela. Note que o programa está posicionado sobre a linha de código **STC** (linha 17) que, ao ser executada, fará com que o registrador de estado **CF** passe do valor **0** (zero) para o valor **1** (um).

Na sequência execute o comando de menu **view/flags** na janela **emulator: ADICAO2.exe** e posicione a janela no lugar mais confortável da tela. Em seguida acione a tecla de função **<F8>** pela terceira vez e observe os dados apresentados em tela, como mostra a Figura 8.6 que indica o registrador de deslocamento **IP** com a indicação de seu valor como **0005h**, pois é o endereço de início da segunda instrução do programa.

A execução da linha de código **STC** (linha 17) define para o registrador **CF** o valor **1**, que será perceptível a partir da terceira execução da tecla de função **<F8>**. Observe junto a Figura 8.7 este efeito que mostra também o registrador **IP** com a definição do valor **0006h** e a barra amarela mostra a linha de código **MOV AX, a** (linha 18) que transfere o valor definido para a variável **a** para o registrador geral **AX**, que se encontra com o valor do endereço de deslocamento utilizado anteriormente para o registrador **DS**.

Nessa etapa na área no quadro direito da tela **emulator** encontra-se marcado o comando **MOV AX, [0000h]**, sendo o endereço de memória **0000h** o local onde está armazenado o valor hexadecimal **6**, como pode ser observado na Figura 8.7. Note o valor do registrador **CS** definido como **1**.

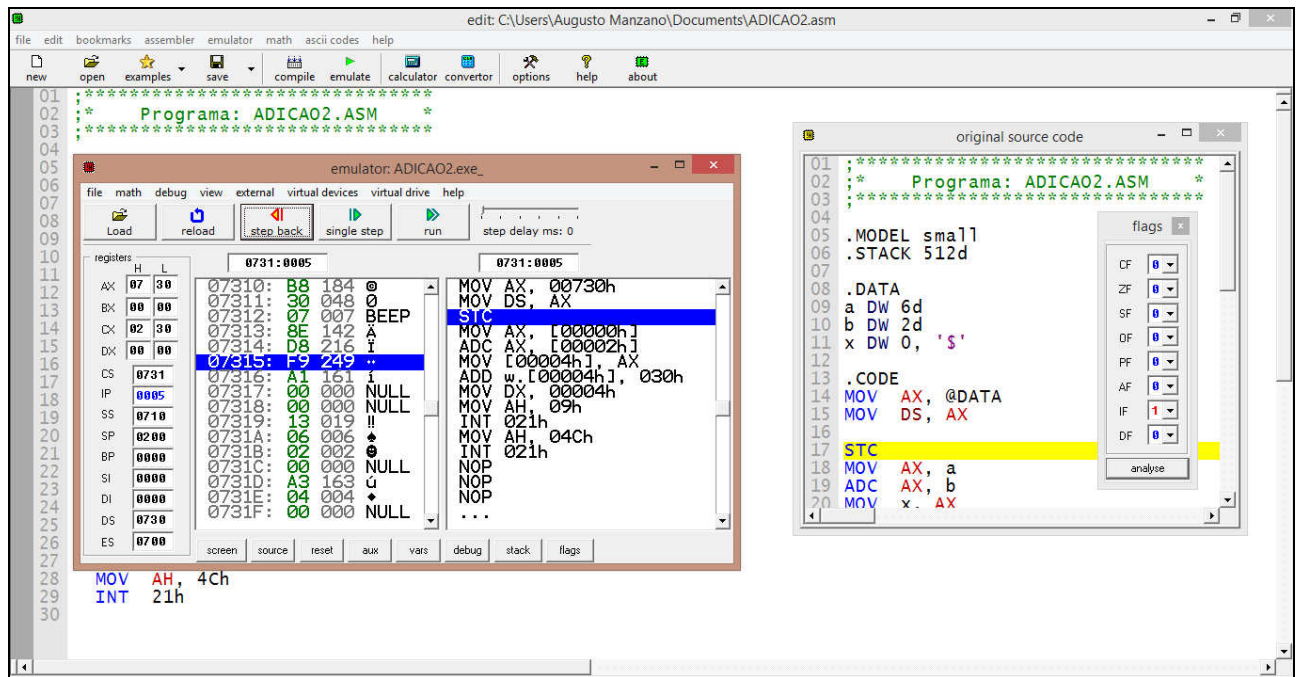


Figura 8.6 - Detalhes de ação do programa (segunda ação da tecla <F8>).

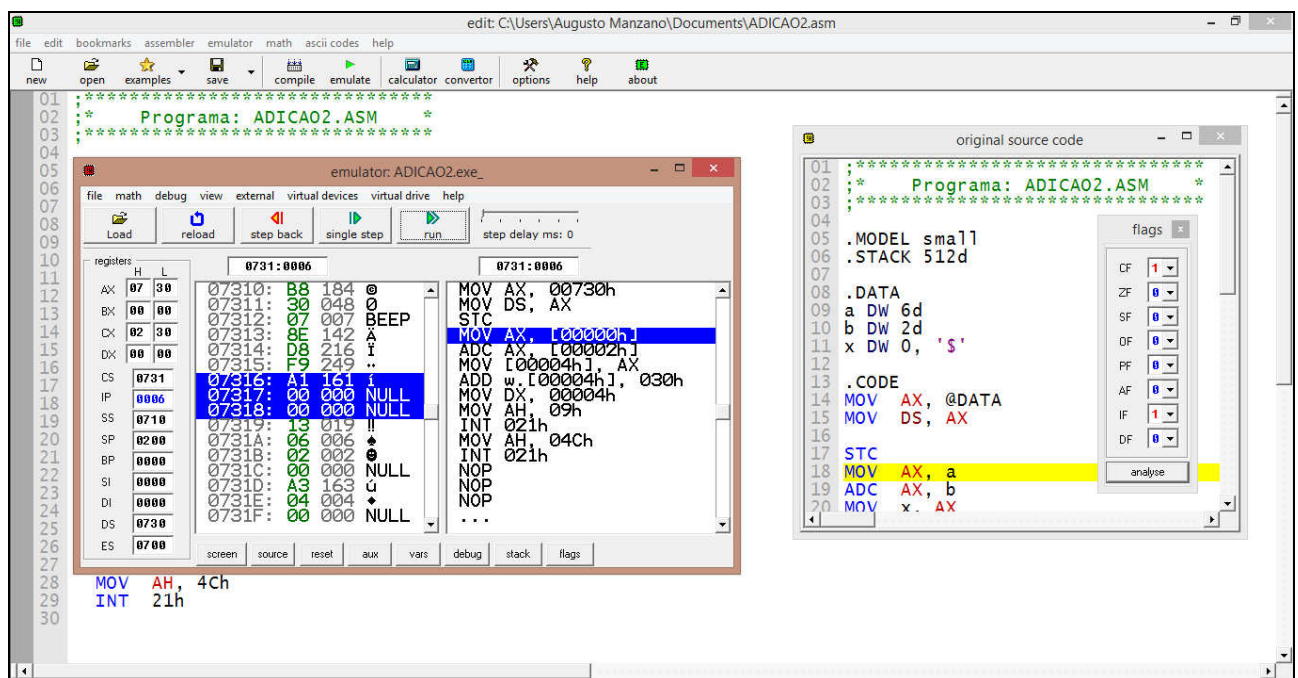


Figura 8.7 - Detalhes de ação do programa (terceira ação da tecla <F8>).

Na sequência de execução acione a tecla de função <F8> pela quarta vez e observe os dados apresentados em tela, como na Figura 8.8.

A instrução ADC AX, b (linha 19) transfere o valor definido para a variável **b** para o registrador geral AX. Na área no quadro direito da tela **emulador** encontra-se marcado o comando ADC AX, [0002h] e o endereço 0002h é o local em que se encontra o valor hexadecimal 2, como pode ser observado.

Acione a tecla de função <F8> pela quinta vez e observe os dados apresentados em tela, Figura 8.9. Nessa etapa o registrador geral AX já está com a soma 8 e o valor 1 do registrador de estado PF, como pode ser notado na janela ADICA02. O registrador CF volta a ter o valor 0 (zero).

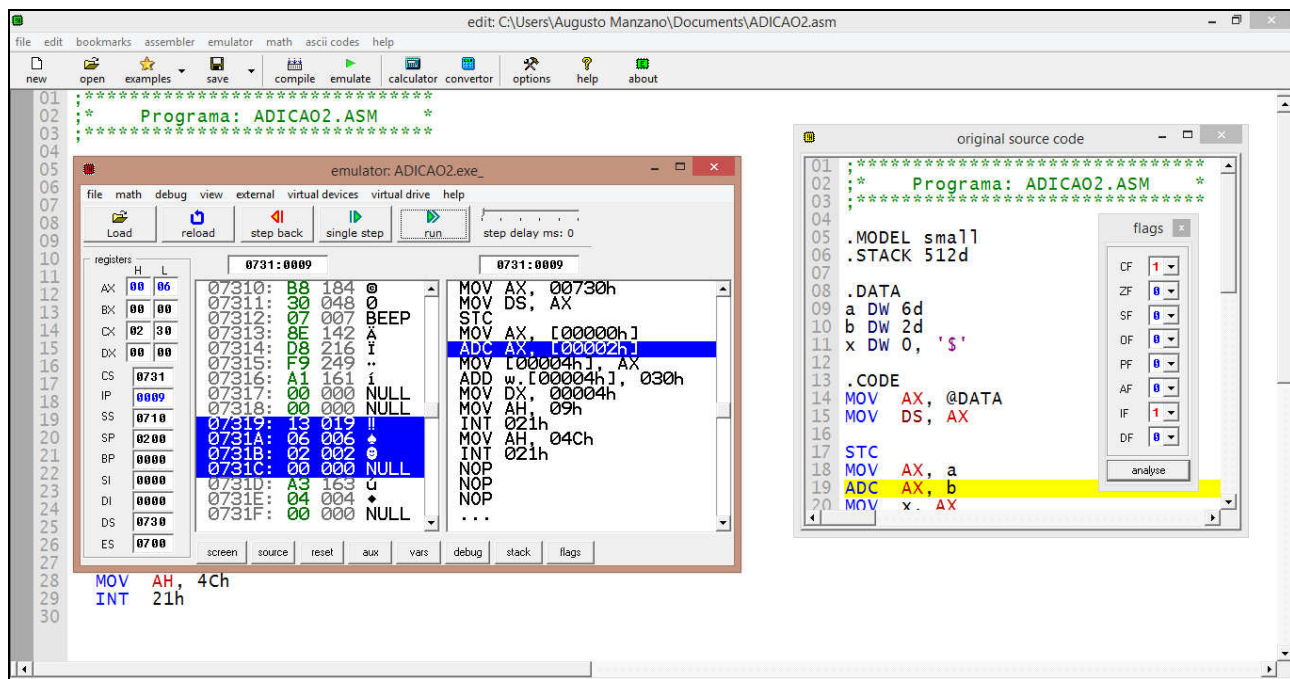


Figura 8.8 - Detalhes de ação do programa (quarta ação da tecla <F8>).

Pressione a tecla de função <F8> pela sexta vez e observe os dados apresentados em tela, como indica a Figura 8.10. A partir dessa etapa será adicionado o valor hexadecimal 030h ao valor existente no registrador geral AX, por meio da instrução `ADD x, 030h` (linha 22). Lembre-se de que essa estratégia é usada para obter o valor correspondente ao código ASCII do valor armazenado no registrador menos significativo do registrador geral AX.

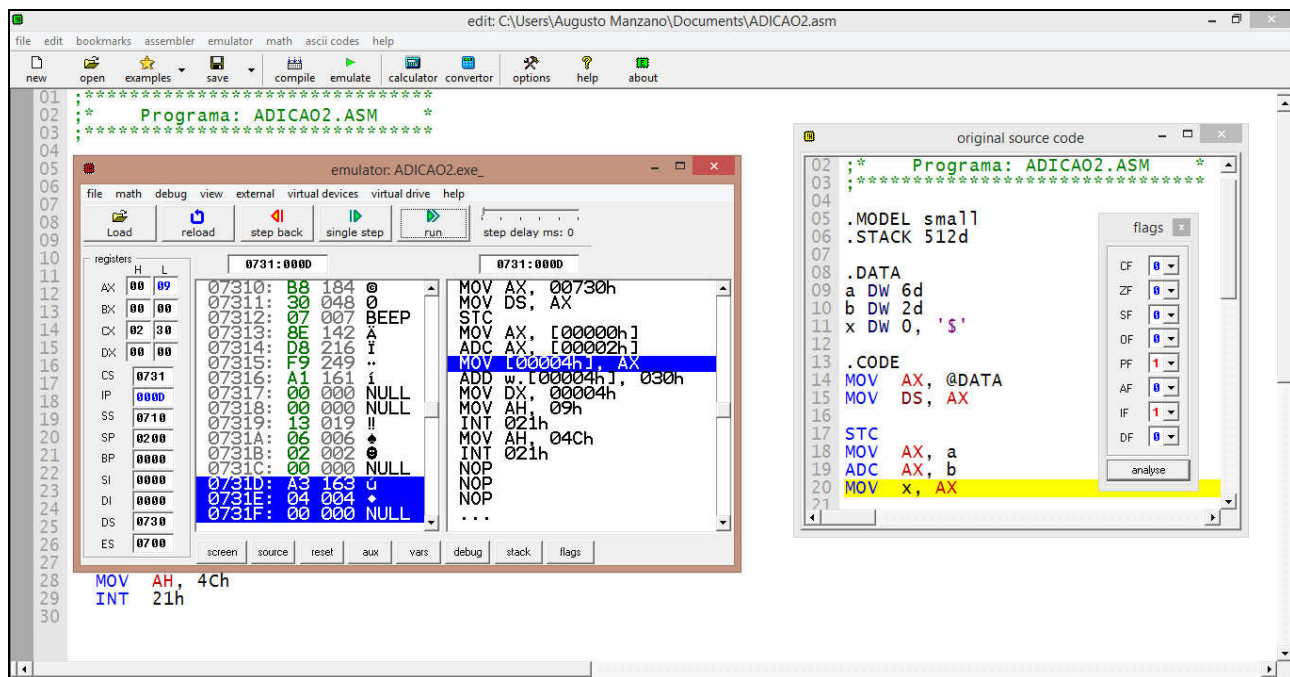


Figura 8.9 - Detalhes de ação do programa (quinta ação da tecla <F8>).

Observe também a identificação do quadro direito da tela **emulator** que apresenta marcada a linha de código `ADD w,[00004h], 030h`, demonstrando que o valor 030h será adicionado ao conteúdo existente no endereço de deslocamento 00004h.

Aperte a tecla de função <F8> pela sétima vez e observe os dados apresentados em tela, como na Figura 8.11.

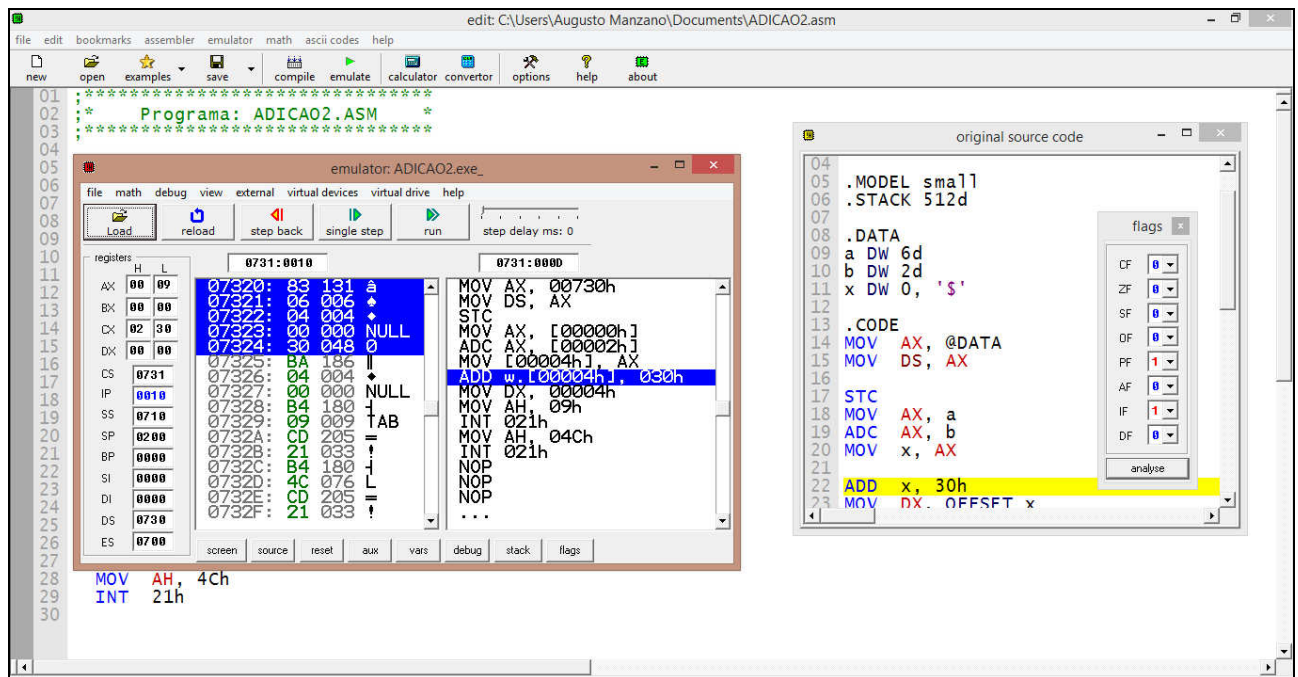


Figura 8.10 - Detalhes de ação do programa (sexta ação da tecla <F8>).

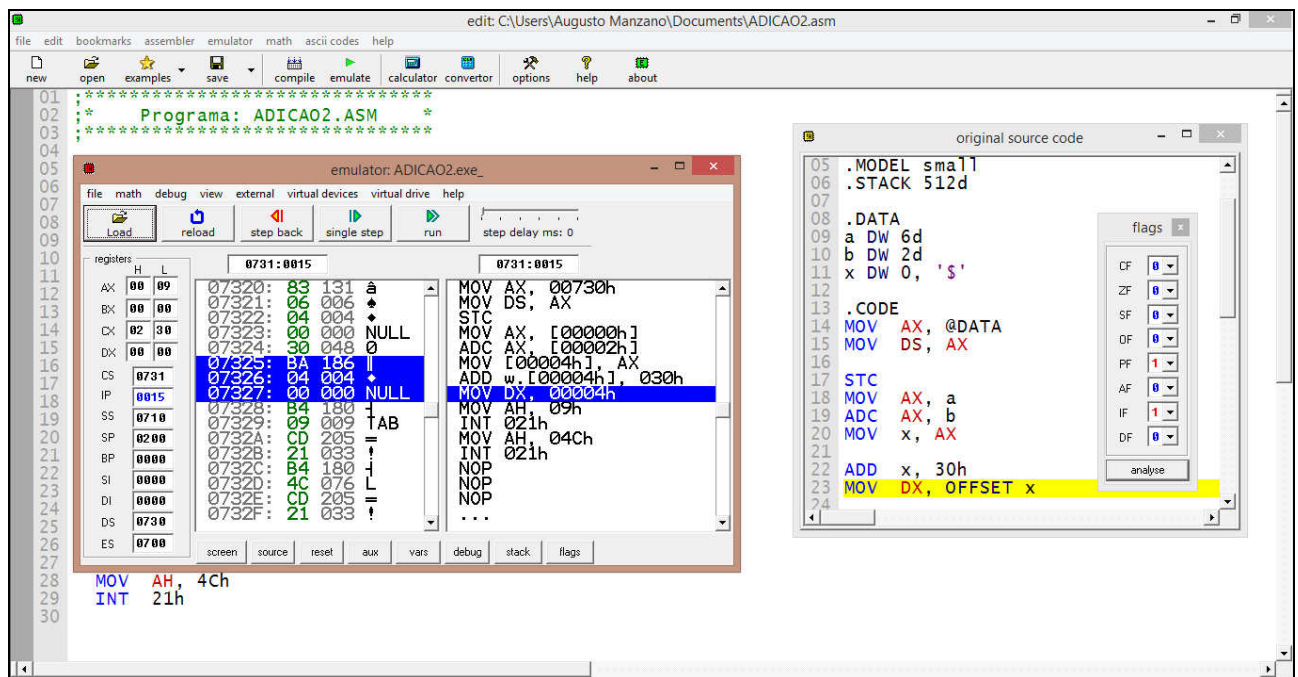


Figura 8.11 - Detalhes de ação do programa (sétima ação da tecla <F8>).

A linha de código **MOV DX, OFFSET x** (linha 23) mostra que o endereço de deslocamento em que se encontra definida a variável **x** (**00004h**) será movimentado para o registrador geral **DX**. Veja os detalhes da área do quadro direito da tela **emulador** na Figura 8.12.

Na sequência acione a tecla de função <F8> pela oitava vez e observe os dados apresentados em tela, conforme a Figura 8.13.

A partir desse ponto o programa apresenta o resultado da operação na tela do monitor de vídeo, ou seja, o conteúdo existente no endereço de deslocamento **0018h**.

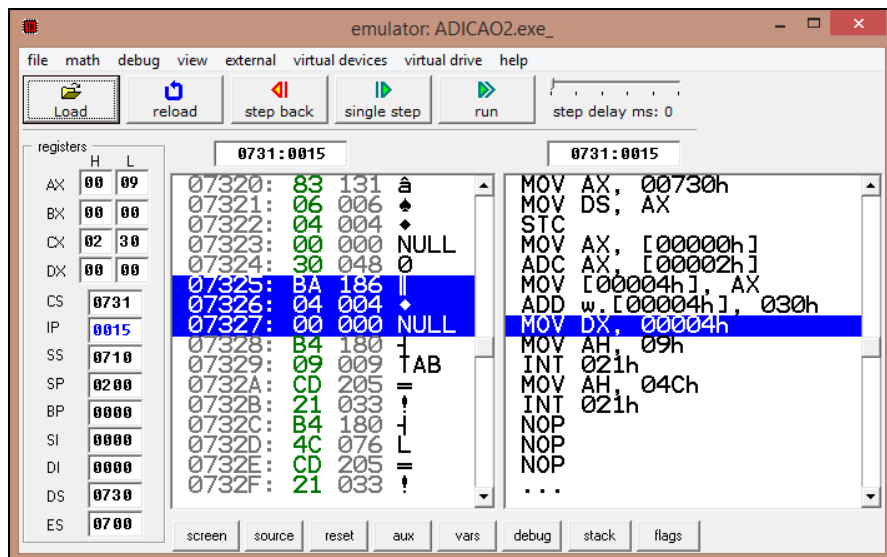


Figura 8.12 - Detalhes de ação do programa.

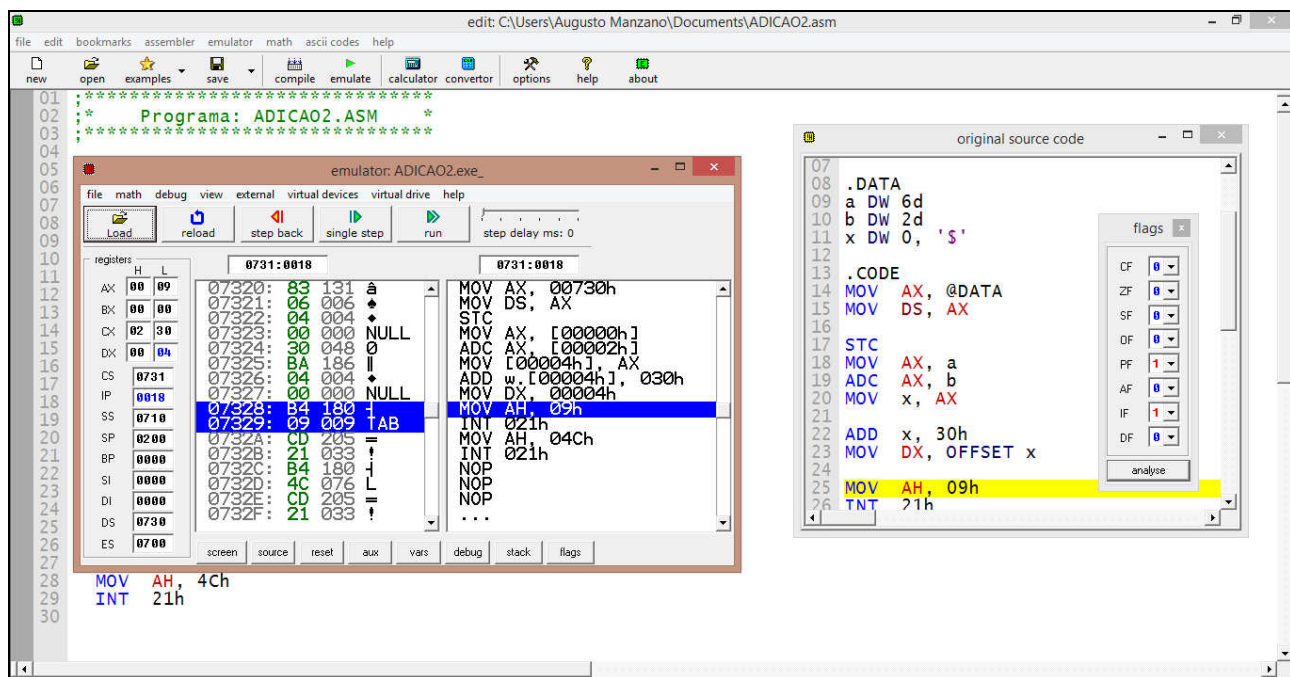


Figura 8.13 - Detalhes de ação do programa (oitava ação da tecla <F8>).

Em seguida, acione a tecla de função <F8> mais três vezes até que a caixa de diálogo com a mensagem de término seja apresentada, como indica a Figura 8.14. Para finalizar, pressione o botão **OK** da caixa de diálogo **message** e na janela **emulador: ADICAO2.exe_** acione o comando de menu **file/close the emulator**.

Em seguida feche todas as janelas do ambiente **emu8086**.

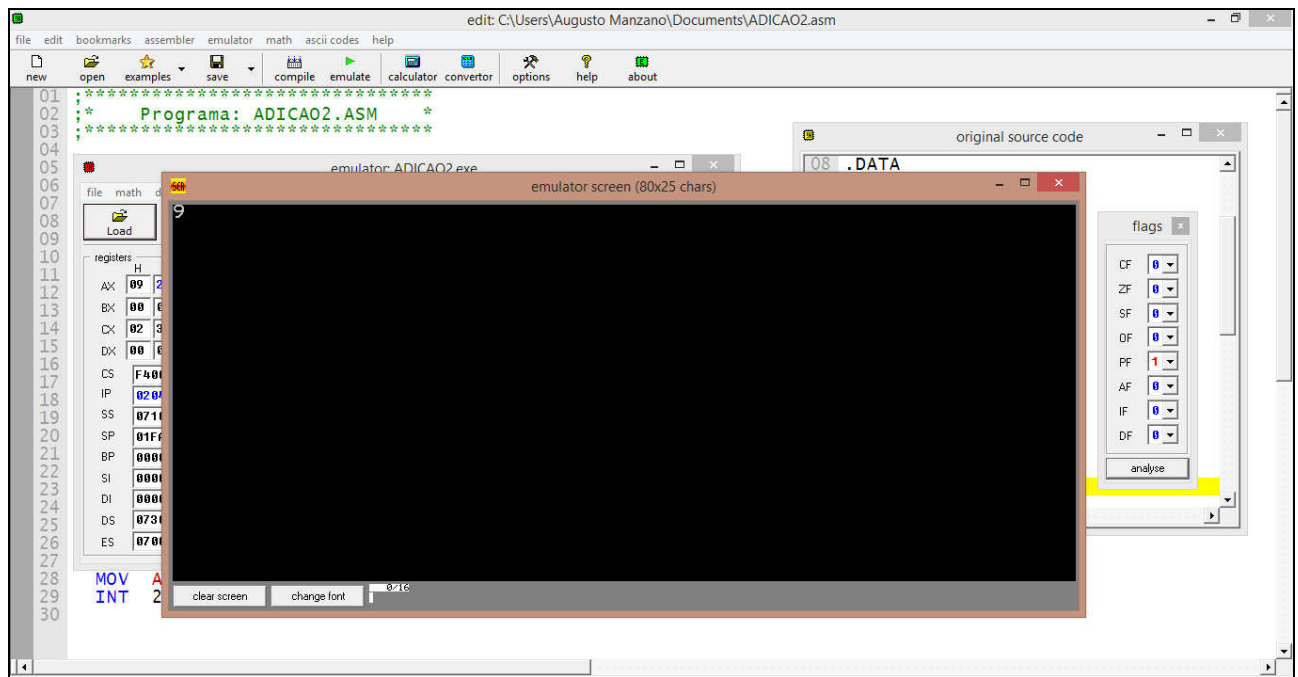


Figura 8.14 - Finalização da execução do programa.

8.3.2 - Subtração

As operações de subtração podem ocorrer com a utilização das instruções **SUB** ou **SBB**. A instrução **SUB** tem a mesma finalidade do operador aritmético "-" existente em outras linguagens de programação. Já a instrução **SBB** possui como diferencial a capacidade de subtrair, além dos valores existentes, o valor encontrado no registrador de estado **CF**.

O funcionamento lógico da instrução **SUB** será **SUB DESTINO, ORIGEM (DESTINO ← DESTINO – ORIGEM)**.

SUB AX, 5d

No exemplo apresentado, o registrador geral **AX** está sendo subtraído com o valor decimal **5**. Se no registrador geral **AX** existir algum valor anterior, o valor **5** será subtraído do valor existente.

O funcionamento lógico da instrução **SBB** será **SBB DESTINO, ORIGEM (DESTINO ← DESTINO – ORIGEM – CF)**.

SBB AX, 5d

No exemplo apresentado o registrador geral **AX** está sendo subtraído do valor decimal **5** mais o valor que estiver no registrador de estado **CF**, que pode ser **0** (zero) ou **1** (um).

Tome como base um programa que deva executar a equação $X \leftarrow A - B$, em que a variável **A** tem o valor decimal **6** e a variável **B**, o valor decimal **4**. Acompanhe o código do programa a seguir:

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa seguinte, gravando-o por meio dos comandos de menu **File/Save** com o nome **SUBTRAI1**, de forma que fique semelhante à Figura 8.15.

```
;*****
;*   Programa: SUBTRAI1.ASM   *
;*****

.MODEL small
.STACK 512d
```

```

.DATA
a DW 6d
b DW 4d
x DW 0, '$'

.CODE
MOV AX, @DATA
MOV DS, AX

MOV AX, a
SUB AX, b
MOV x, AX

ADD x, 30h
MOV DX, OFFSET x

MOV AH, 09h
INT 21h

MOV AH, 4Ch
INT 21h

```

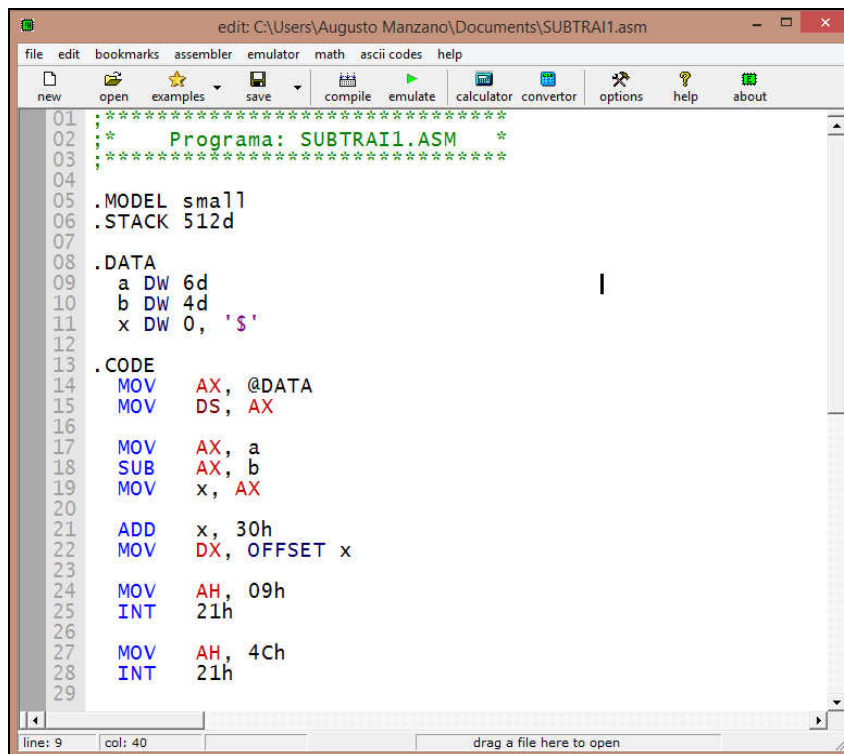


Figura 8.15 - Programa SUBTRAI1 na ferramenta emu8086.

A estrutura do programa de subtração é idêntica à do programa de adição. A execução ocorre de forma semelhante. Teste a execução passo a passo, observando os detalhes apresentados.

Em seguida, a título de ilustração de uso da instrução **SBB**, considere um programa que execute a equação $X \leftarrow A - B - CF$, em que a variável **A** tem o valor decimal 6, a variável **B** o valor decimal 4 e **CF** é o valor existente no registrador de estado **CF**. Acompanhe o código do programa a seguir:


```

;*****
;*      Programa: SUBTRAI2.ASM      *
;*****

.MODEL small
.STACK 512d

.DATA
    a DW 6d
    b DW 4d
    x DW 0, '$'

.CODE
    MOV     AX, @DATA
    MOV     DS, AX

    STC
    MOV     AX, a
    SBB     AX, b
    MOV     x, AX

    ADD     x, 30h
    MOV     DX, OFFSET x

    MOV     AH, 09h
    INT     21h

    MOV     AH, 4Ch
    INT     21h

```

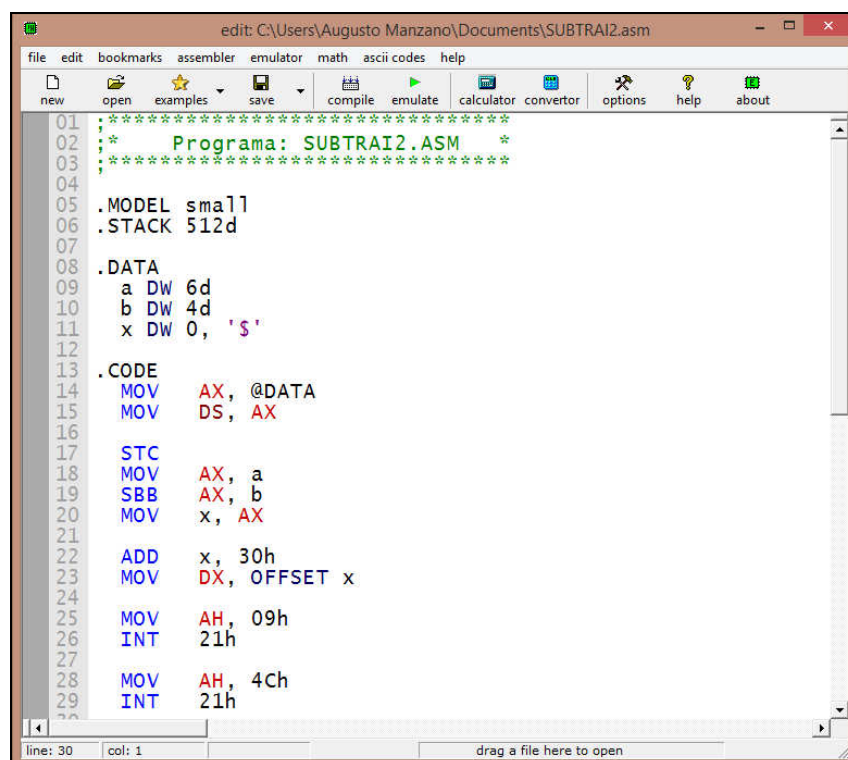


Figura 8.16 - Programa SUBTRA2 na ferramenta emu8086.

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save com** o nome **SUB-TRAI2.asm**, de forma que fique semelhante à Figura 8.16.

Observe o detalhe de uso da instrução **STC** na linha 17 do programa que apresenta como resultado de cálculo o valor 1.

8.3.3 - Divisão

As operações de divisão podem utilizar as instruções **DIV** (*divide*), a qual anteriormente já foi demonstrada, ou **IDIV** (*integer divide*). As instruções **DIV** e **IDIV** possuem a mesma finalidade do operador aritmético "/" existente em outras linguagens de programação.

As operações de divisão exigem que sejam tomados dois cuidados básicos:

- ◆ O primeiro cuidado é com relação à capacidade de armazenamento do número de *bits* em relação ao registrador em uso.
- ◆ O segundo é quanto ao uso do valor zero na indicação do divisor.

Observação

A operação de divisão pode ocorrer entre valores de 32 e 16 *bits* ou entre valores de 16 e 8 *bits*.

Caso um resultado obtido não caiba em um registrador ou o divisor seja zero, será gerado um erro de operação.

Após uma operação de divisão entre um dividendo de 16 *bits* (que deve estar armazenado no registrador geral **AX**) e um divisor de 8 *bits*, o valor do quociente é armazenado no registrador menos significativo **AL** do registrador geral **AX** e o resto da divisão é armazenado no registrador mais significativo **AH** do registrador geral **AX**. Assim sendo, a instrução:

DIV BL

Indica que em uma divisão de 16 *bits* por 8 *bits*, o registrador geral **AX** será armazenado com o valor de 16 *bits*, que será dividido pelo divisor armazenado no registrador menos significativo **BL** do registrador geral **BX**. O quociente estará armazenado no registrador menos significativo **AL** e o resto da divisão armazenado no registrador mais significativo **AH**. É como informar em uma linguagem de alto nível a linha de instrução **AL ← AX / BL**, em que **AH** é o resto de divisão.

Caso a divisão ocorra entre um dividendo de 32 *bits* (que deve estar armazenado no registrador geral **DX**) e um divisor de 16 *bits*, o valor do quociente será armazenado no registrador geral **AX** e o resto da divisão armazenado no registrador geral **DX**. Assim sendo, a instrução:

DIV BX

Indicaria que em uma divisão de 32 *bits* por 16 *bits*, o valor armazenado no par de registradores gerais **DX:AX** estaria com um valor de 32 *bits*, que será dividido pelo divisor armazenado no registrador geral **BX**. O quociente estaria no registrador geral **AX** e o resto da divisão armazenado no registrador geral **DX**. Seria como informar em uma linguagem de alto nível a linha de instrução **AX ← DX:AX / BX**, em que **DX** é o resto de divisão.

Observação

Quando são feitas operações matemáticas com operações de divisão, não se leva em conta o comportamento dos registradores de estado (*flags*).

Tome como base um programa que vai executar a equação $X \leftarrow A / B$, em que a variável **A** tem o valor decimal 9 de 16 *bits* (tipo **DW**) e a variável **B** o valor decimal 2 de 8 *bits* (tipo **DB**). Observe atentamente cada linha do código do programa a seguir:

```

;*****
;*      Programa: DIVIDE1.ASM      *
;*****
;

.MODEL small
.STACK 512d

.DATA
a DW 9d
b DB 2d
x DB 0, '$'

.CODE
MOV AX, @DATA
MOV DS, AX

MOV AX, a
MOV BL, b
DIV BL

MOV x, AL
ADD x, 30h
MOV DX, OFFSET x

MOV AH, 09h
INT 21h

MOV AH, 04Ch
INT 21h

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save** com o nome **DIVIDE1** de forma que fique semelhante à Figura 8.17.

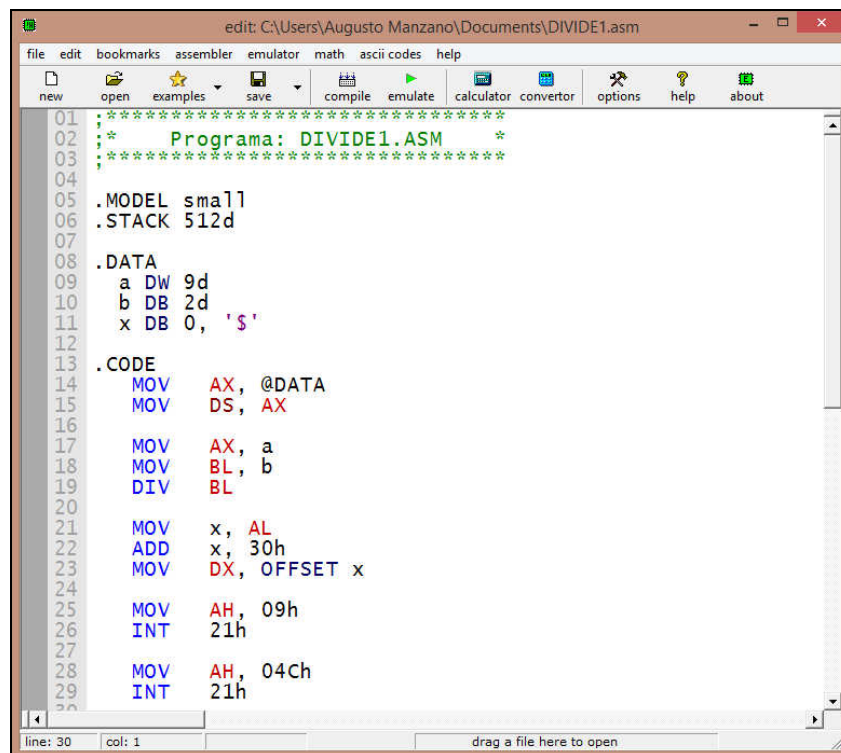


Figura 8.17 - Programa DIVIDE1 na ferramenta emu8086.

Conforme descrito anteriormente, considerando a divisão de valores de 16 *bits* (valor 9 definido para variável **a** do tipo **DW**) por um valor de 8 *bits* (valor 2 definido para a variável **b** do tipo **DB**), o valor do quociente estará no registrador menos significativo **AL** e o resto da divisão no registrador mais significativo **AH**.

Para verificar essa ocorrência, serão apresentados alguns detalhes da execução do programa passo a passo. Acione o comando de menu **file/compile and load in emulator**.

Acione a tecla de função **<F8>** por cinco vezes e observe na janela **emulate: DIVIDE1.com_** o resultado do quociente no registrador menos significativo **AL** e o resto da divisão no registrador mais significativo **AH**, como mostra a Figura 8.18.

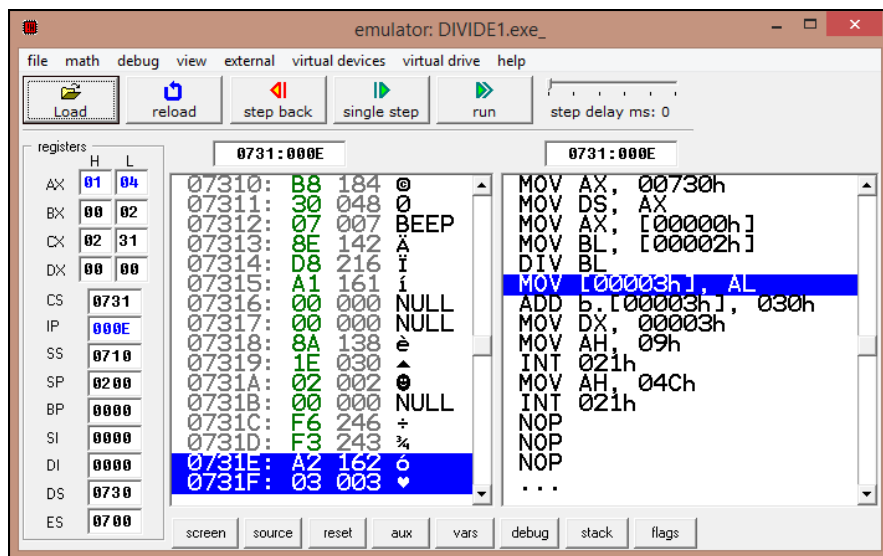


Figura 8.18 - Resultados do programa DIVIDE1.

Observe o valor **04h** do quociente no registrador menos significativo **AL**, resultado da divisão, e o valor **01h** do resto no registrador mais significativo **AH**. Em operações de divisão com valores hexadecimais, normalmente se leva em conta apenas o valor do quociente, desprezando o valor do resto. Haja vista se for executada a mesma operação no programa **Calculadora** do Windows.

Na sequência do programa acione a tecla de função **<F8>** algumas vezes ou acione o botão **run** até o programa apresentar o resultado da operação, quando deve ser acionado o botão **OK** da caixa de diálogo **message**.

Com o objetivo de demonstrar uma operação de divisão com valores do tipo *word*, considere um programa que deva executar a equação $X \leftarrow A / B$, em que a variável **A** possui o valor decimal 9 de 16 *bits* (tipo **DW**) e a variável **B** o valor decimal 2 de 16 *bits* (tipo **DW**). Acompanhe atentamente cada linha do código do programa a seguir:

```

;*****
;*      Programa: DIVIDE2.ASM      *
;*****

.MODEL small
.STACK 512d

.DATA
    a DW 9d
    b DW 2d
    x DB 0, '$'

.CODE
    MOV     AX, @DATA
    MOV     DS, AX

    MOV     AX, a
    MOV     BX, b

```

```

DIV    BX

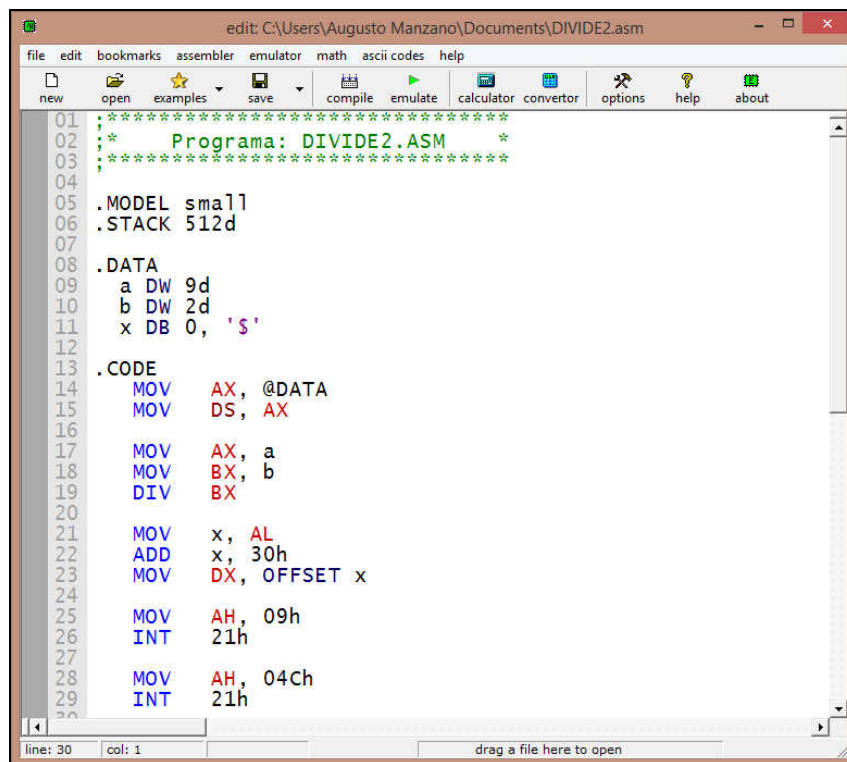
MOV    x, AL
ADD    x, 30h
MOV    DX, OFFSET x

MOV    AH, 09h
INT    21h

MOV    AH, 04Ch
INT    21h

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save** com o nome **DIVIDE2**, de forma que fique semelhante à Figura 8.19.



```

01  ;*****
02  ; Programa: DIVIDE2.ASM
03  ;*****
04
05  .MODEL small
06  .STACK 512d
07
08  .DATA
09  a DW 9d
10  b DW 2d
11  x DB 0, '$'
12
13  .CODE
14  MOV AX, @DATA
15  MOV DS, AX
16
17  MOV AX, a
18  MOV BX, b
19  DIV BX
20
21  MOV x, AL
22  ADD x, 30h
23  MOV DX, OFFSET x
24
25  MOV AH, 09h
26  INT 21h
27
28  MOV AH, 04Ch
29  INT 21h
30

```

Figura 8.19 - Programa DIVIDE2 na ferramenta emu8086.

Conforme descrito anteriormente, considerando a divisão de valores de 16 *bits* (valor 9 definido para variável **a** do tipo **DW**) por um valor de 16 *bits* (valor 2 definido para a variável **b** do tipo **DW**), o valor do quociente no registrador menos significativo **AX** e o resto da divisão estará no registrador mais significativo **DX**.

Para verificar essa ocorrência, serão apresentados alguns detalhes da execução do programa passo a passo. Acione o comando de menu **assembler/ compile and load in emulator**.

Acione a tecla de função **<F8>** por cinco vezes e observe na janela **emulator: DIVIDE2.com_** o resultado do quociente no registrador geral **AL (AX)** e o resto da divisão no registrador geral **DL (DX)**, como exibe a Figura 8.20.

Na sequência acione a tecla de função **<F8>** algumas vezes até o programa apresentar o resultado da operação, quando deve ser acionado o botão **OK** da caixa de diálogo **message**.

É importante salientar que a instrução **DIV** deve operar apenas com valores numéricos não sinalizados (valores positivos). Caso haja necessidade de efetuar operações de divisão com valores numéricos sinalizados (valores negativos), deve-se utilizar a instrução **IDIV**.

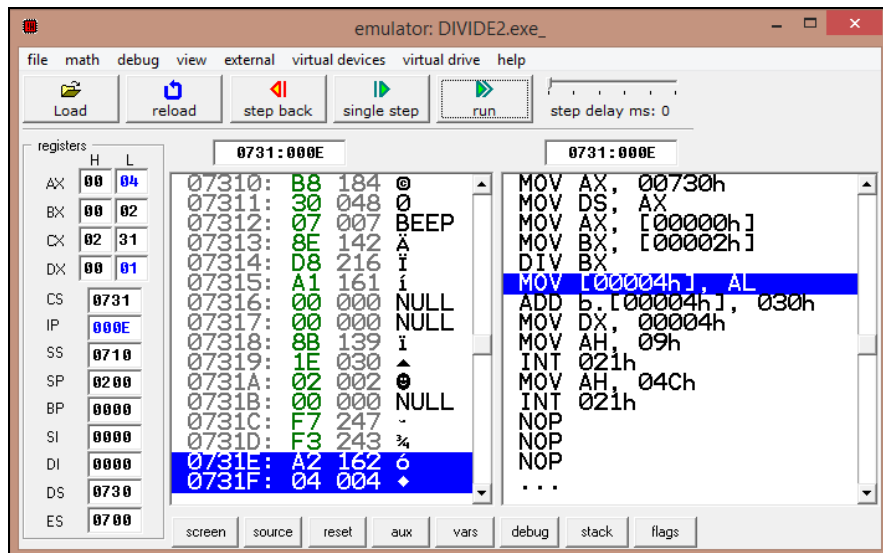


Figura 8.20 - Resultados do programa DIVIDE2.

Com o objetivo de demonstrar uma operação de divisão com a instrução **IDIV**, considere um programa que deva executar a equação $X \leftarrow A / B$, em que a variável **A** possui o valor decimal negativo **-9** de 16 *bits* (tipo **DW**) e a variável **B** o valor decimal **2** de 8 *bits* (tipo **DB**). Observe atentamente cada linha do código do programa em seguida:

```
;*****
;*      Programa: DIVIDE3.ASM      *
;*****
```

```
.MODEL small
.STACK 512d
```

```
.DATA
a DW -9d
b DB 2d
x DB 0, '$'
```

```
.CODE
MOV AX, @DATA
MOV DS, AX

MOV AX, a
MOV BL, b
IDIV BL

MOV x, AL
SUB BX, BX
MOV BL, x

MOV AH, 02h
MOV DL, BL
MOV CL, 04h
SHR DL, CL
ADD DL, 30h
CMP DL, 39h
JLE valor1
ADD DL, 07h
```

```

valor1:
    INT     21h

    MOV     DL, BL
    AND     DL, 0Fh
    ADD     DL, 30h
    CMP     DL, 39h
    JLE     valor2
    ADD     DL, 07h

valor2:
    INT     21h

    MOV     AH, 4Ch
    INT     21h

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save** com o nome **DIVIDE3**. A Figura 8.21 mostra um trecho de como o programa deve ficar.

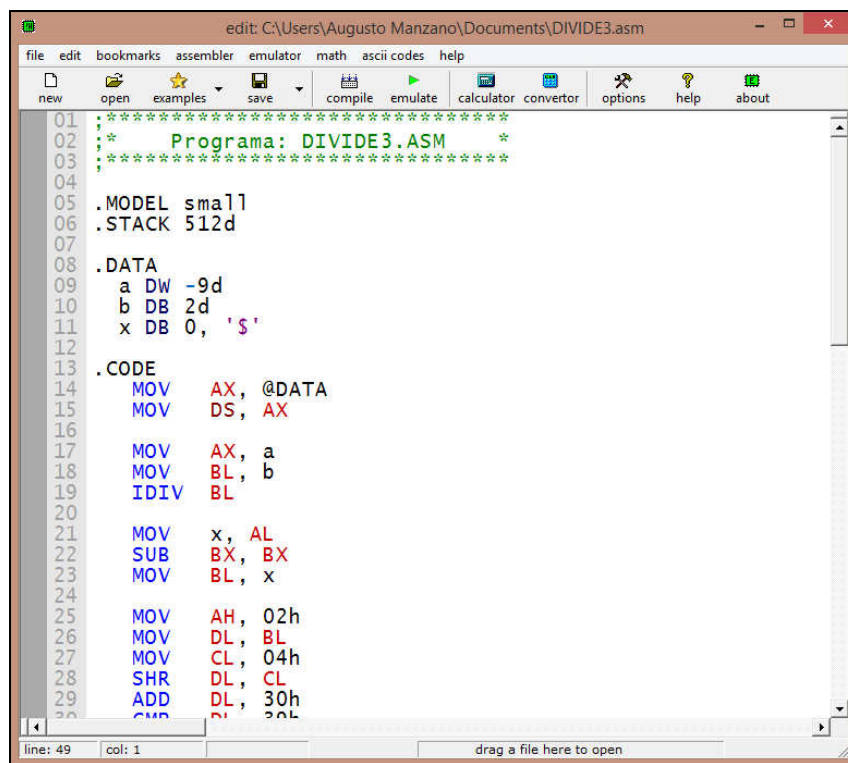


Figura 8.21 - Programa DIVIDE3 na ferramenta emu8086.

A operação de divisão de um dividendo negativo **-9** (que internamente será um valor hexadecimal) de 16 *bits* com um divisor **2** (também hexadecimal) de 8 *bits* resulta em um valor hexadecimal de duas posições, ou seja, o resultado da divisão dos valores hexadecimais **-9h / 2h** terá um quociente igual a **FCh** definido no registrador **AL (AX)**. Para esta comprovação acione a tecla de função **<F5>** uma vez e a tecla de função **<F8>** cinco vezes. A Figura 8.22 mostra o resultado na tela da etapa de simulação **emulator: DIVIDE3.exe**.

Na linha 17 é feita a transferência do valor armazenado na variável **a** para dentro do registrador geral **AX** por meio da instrução **MOV AX, a**. Observe que na linha 09 está sendo definido o tipo **DW** para armazenar o valor negativo (**-9d**). Ao ser processada essa linha, o registrador **AX** armazena o valor **FFF7h** (**-9** em decimal). Valores negativos para serem armazenados usam o tamanho *doubleword*.

Na linha 18 é feita a transferência do valor armazenado na variável **b** para dentro do registrador geral **BL** por meio da instrução **MOV BL, b**. Observe que na linha 10 está sendo definido o tipo **DB** para armazenar o valor **02h**. Ao ser processada essa linha, o registrador **BL** armazena então o valor **02h**, deixando o registrador **BX** com valor **0002h**.

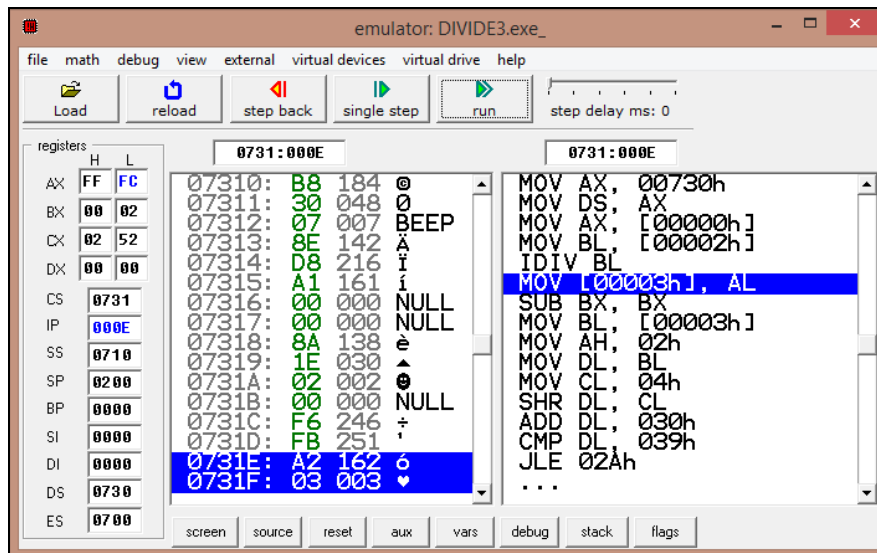


Figura 8.22 - Resultado de divisão no programa.

Na linha 19 é executada a instrução **IDIV BL**, a qual pega o valor do registrador **AX** e faz sua divisão com o valor do registrador **BL**, armazenando o quociente da operação no registrador **AL** e o resto da divisão no registrador **AH**. No caso da linha 19 o resultado obtido da divisão **FCh** é armazenado no registrador **AL**.

A instrução **IDIV** pode operar com valores do tipo *byte* ou *word*. Quando a operação de divisão envolver o uso do tipo *byte*, ocorre no registrador **AL** a atribuição da divisão do valor do registrador **AX** pelo valor do divisor que estará armazenado no registrador **BL** e se houver resto da divisão, esse valor será armazenado no registrador **AH**. Para a divisão que envolva o uso de valores do tipo *word*, ocorre no registrador **AX** a atribuição do valor da divisão que estará armazenado nos registradores **DX** e **AX** (**DX AX**) pelo valor do divisor que gera um quociente armazenado no registrador **AX** e se houver resto, será armazenado no registrador **DX**.

A partir da linha 21 são efetuadas as ações que possibilitam de certa forma apresentar o resultado na tela do monitor de vídeo.

O trecho da linha 21 até a linha 23 faz a transferência do valor do registrador menos significativo **AL** que contém o resultado da divisão efetuada na linha 19 para a variável **x** por meio de **MOV x, AL**. Depois por meio da linha **SUB BX, BX** faz a limpeza do registrador geral **BX**, ou seja, seu valor torna-se **00h** para que a sua parte menos significativa receba o valor armazenado na variável **x** (**MOV BL, x**). Neste caso o valor **FCh**, que é o resultado da operação de divisão, pois esse valor precisa ser apresentado na tela do monitor de vídeo, por esta razão o resultado está sendo preservado no registrador **BL** para seu tratamento no programa pelas linhas 26 e 37.

Da linha 25 até a linha 34 são efetuadas as ações responsáveis pela apresentação do caractere **F** (primeiro caractere do valor **FCh**) que ocorre na linha 34 quando da execução da instrução **INT 21h** identificada após o rótulo **valor1** definido na linha 33.

Da linha 36 até a linha 43 são efetuadas as ações responsáveis pela apresentação do caractere **C** (segundo caractere do valor **FCh**) que ocorre na linha 43 quando da execução da instrução **INT 21h** identificada após o rótulo **valor2** definido na linha 42.

A definição do valor **02h** na linha 25 para o registrador **AH** é o estabelecimento da saída de um caractere para a interrupção **21h** executada nas linhas 34 e 43 por meio da instrução **INT 21h**.

A linha 26 faz a transferência do valor **FCh** (resultado da divisão) do registrador **BL** para o registrador **DL** por meio da instrução **MOV DL, BL**, para que o primeiro caractere da sequência numérica seja preparado para apresentação.

As linhas 27 e 28 efetuam o tratamento para apresentação de valores hexadecimais. Na linha 27 está sendo fornecido o valor **04h** para o registrador **CL** por meio da instrução **MOV CL, 04h**, isso fará com que ocorra o deslocamento de 4 *bits* (1 *nibble*) para a direita quando a linha 28 for executada. Na linha 28 a instrução **SHR DL, CL** fará o deslocamento para a direita a quantidade de *bits* informada a partir do valor armazenado no registrador **CL**, ou seja, fará o deslocamento para a direita do *nibble* representado pelo caractere **F** do valor **FCh** que se encontra definido no registrador **DL**. Assim sendo, o registrador **DL** passa a possuir o valor **0Fh** (**F** é o conteúdo extraído do valor **FCh**).

A linha **29** prepara por meio da execução da instrução **ADD DL, 30h** o conteúdo do registrador **DL** para apresentação na tela do monitor de vídeo, semelhante ao recurso já explanado no capítulo quatro para a apresentação de valores binários.

A linha **30** estabelece a comparação do valor armazenado no registrador **DL** com o valor **39h**. Se for o valor da comparação verdadeiro, o programa será desviado pela linha de código **31** para a linha **33** de forma que ocorra a apresentação do caractere **F**. Se a condição não for verdadeira, será executada antes da apresentação do caractere **F** a instrução da linha **32**.

A linha **31** por meio da instrução **JLE valor1** fará o desvio do programa para a linha marcada com o rótulo **valor1**: (linha **33**) quando a condição assinalada pela linha **30** for verdadeira; caso contrário, a ação dessa instrução não é executada e o programa continua a partir da linha **32**.

A linha **32** efetua a soma de sete *bits* para compor a representação numérica de um valor em formato hexadecimal. Essa ação é executada quando o conteúdo a ser apresentado estiver distante do valor de nove posições, por isso essa linha faz a adição do valor **07h** ao valor armazenado no registrador **DL** por meio da instrução **ADD DL, 07h**.

Após a apresentação do caractere **F** que compõe o valor **FCh** o programa fará algo idêntico para apresentar o caractere **C**. Lembre-se de que a linguagem de programação *Assembly* efetua entradas e saídas sempre um caractere por vez.

A linha **36** faz a movimentação do valor **FCh** armazenado no registrador **BL** para o registrador **DL**, como ocorreu com a instrução da linha **26**.

A linha de código **37** para preparar a apresentação do segundo caractere do valor **FCh** usa a instrução **AND DL, 0Fh**. A instrução **AND** faz a comparação em um valor *bit a bit* (modo binário) em dados do tipo *byte* ou *word*. Neste sentido, para extrair o segundo caractere do valor **FCh** que é sua parte direita faz-se a comparação do valor armazenado no registrador **DL** com o valor **0Fh**. Lembre-se de que esse recurso foi apresentado no capítulo quatro. Após essa ação o registrador **DL** apresenta como valor armazenado o conteúdo **0Ch**, que é o segundo valor a ser apresentado.

As linhas de **38** a **43** efetuam ação idêntica à ação já descrita para as linhas de **29** a **34**. As linhas **45** e **46** são responsáveis pelo término da execução do programa, como já foi explicado.

8.3.4 - Multiplicação

As operações de multiplicação podem ser feitas com as instruções **MUL** (*multiply*) ou **IMUL** (*integer multiply*), as quais têm a mesma finalidade do operador aritmético "*" existente em outras linguagens de programação.

Para realizar operações de multiplicação, é necessário levar em conta o fato de trabalhar com registradores de 8 ou 16 *bits*. Elas são mais complexas que as de adição e subtração.

Se for feita a multiplicação entre dois registradores de 16 *bits*, obter-se-á um resultado de 32 *bits* (ou seja, um *double word*). Isso está acima da capacidade de trabalho do processador 8086/8088, e neste caso serão utilizados automaticamente os registradores gerais **DX** e **AX** para armazenar um valor de 32 *bits*. O registrador geral **DX** armazena o *word* mais significativo e o registrador geral **AX**, o *word* menos significativo do valor de 32 *bits* resultante após a multiplicação.

Imagine a necessidade de multiplicar o valor hexadecimal **AA1Fh** (43.551 decimal) pelo valor hexadecimal **FF2Ah** (65.322 decimal) que resultaria no valor **A990CA16h** (2.844.838.422 decimal). Neste caso a porção mais significativa (**A990h**) seria armazenada no registrador geral **DX** e a porção menos significativa (**CA16h**) no registrador geral **AX**.

É fundamental considerar também que as operações de multiplicação serão sempre imputadas sobre os registradores acumuladores **AL** ou **AX**, dependendo de os valores serem de 8 ou 16 *bits*. Desta forma a linha de instrução:

MUL BL

Indica que o registrador menos significativo **BL** está sendo multiplicado pelo valor existente no registrador menos significativo **AL** e armazenando o resultado obtido no registrador geral **AX**. Seria como informar em uma linguagem de alto nível a linha de instrução **AX ← AL * BL**, considerando que os valores trabalhados são de 8 *bits*.

Caso venha a utilizar valores de 16 *bits*, o resultado será armazenado no par de registradores gerais **DX:AX**. Assim sendo, a instrução:

MUL BX

indicaria que o registrador geral **BX** está sendo multiplicado pelo valor existente no registrador geral **AX** e armazenando o resultado obtido nos registradores **DX** e **AX**. Seria como informar em uma linguagem de alto nível a linha de instrução **DX:AX ← AX * CX**.

Observação

Quando se fazem operações de multiplicação, não se leva em conta o comportamento dos registradores de estado (*flags*).

Tome como base um programa que deva executar a equação $X \leftarrow A * B$, em que a variável **A** possui o valor decimal **5** de 16 *bits* (tipo **DW**) e a variável **B** o valor decimal **3** de 8 *bits* (tipo **DB**). Observe atentamente cada linha do código do programa a seguir:

```
*****
;*      Programa: MULTIP1.ASM      *
*****

.MODEL small
.STACK 512d

.DATA
a DW 5d
b DB 3d
x DB 0, '$'

.CODE
MOV AX, @DATA
MOV DS, AX

MOV AX, a
MOV BL, b
MUL BL

MOV x, AL
SUB BX, BX
MOV BL, x

MOV AH, 02h

MOV DL, BL
MOV CL, 04h
SHR DL, CL
ADD DL, 30h
CMP DL, 39h
JLE valor1
ADD DL, 07h

valor1:
INT 021h

MOV DL, BL
AND DL, 0Fh
ADD DL, 30h
CMP DL, 39h
JLE valor2
ADD DL, 07h

valor2:
INT 21h
```

```
MOV    AH, 04Ch
INT    21h
```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save** com o nome **MULTIP1**. A Figura 8.23 mostra um trecho de como o programa deve ficar.

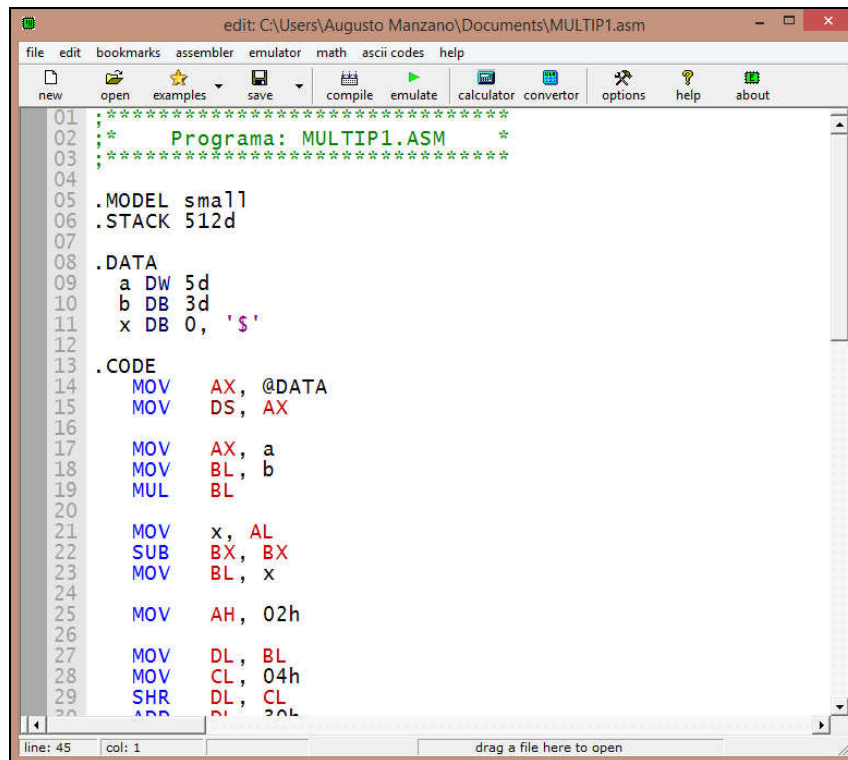


Figura 8.23 - Programa MULTIP1 na ferramenta emu8086..

O mecanismo de ação do programa **MULTIP1** é semelhante ao funcionamento do programa **DIVIDE3**, com exceção da linha de código **24** que apresenta o uso da instrução **MUL BL**.

A instrução **MUL** pode operar com valores do tipo *byte* ou *word*. Quando a operação de multiplicação envolver o uso do tipo *byte*, ocorre no registrador **AX** a atribuição da multiplicação do valor do registrador **AL** pelo valor do registrador **BL** como seu operando, como indicado nas linhas de código de **17** até **19**. Para multiplicação que envolva o uso de valores do tipo *word*, ocorre nos registradores **DX** e **AX** (**DX AX**) a atribuição do valor da multiplicação do valor armazenado no registrador **AX** pelo valor definido como operando.

A operação de multiplicação de um valor **5** (que internamente será um valor hexadecimal) de **16 bits** com um valor **3** (também hexadecimal) de **8 bits** resulta em um valor hexadecimal de duas posições, ou seja, o resultado do produto dos valores hexadecimais **5h * 3h** é **0Fh** (acione a tecla de função **<F5>** uma vez e a tecla de função **<F8>** cinco vezes para comprovar este resultado). A Figura 8.24 mostra o resultado obtido após a operação indicada.

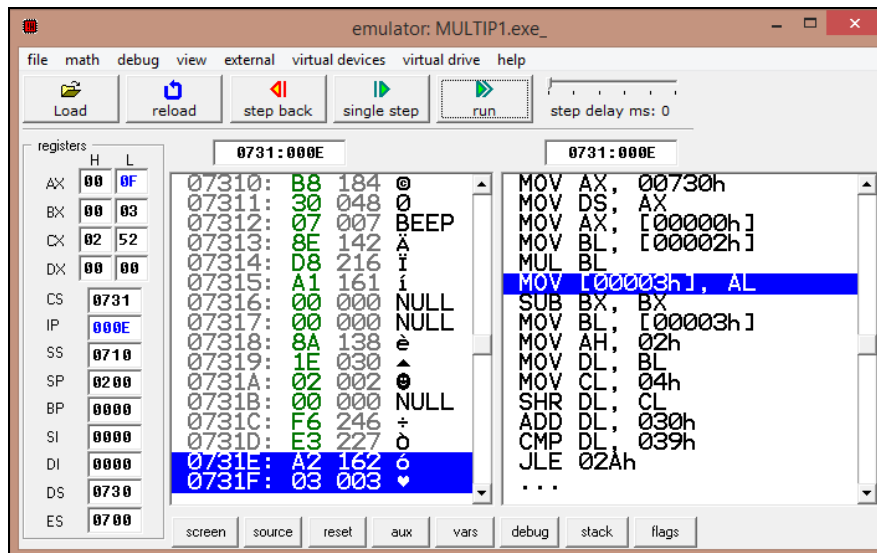


Figura 8.24 - Resultado de multiplicação no programa.

Considere como exemplo outro programa que deva apresentar o resultado da multiplicação de dois valores de 16 *bits*. Ele deve executar a equação $YX \leftarrow A * B$, em que a variável **A** possui o valor decimal **43551** (**AA1Fh**) de 16 *bits* (tipo **DW**) e a variável **B** o valor decimal **65322** (**FF2Ah**) de 16 *bits* (tipo **DW**). Observe atentamente cada linha do código do programa a seguir:

```
;*****
;*      Programa: MULTIP2.ASM      *
;*****
```

```
.MODEL small
.STACK 512d
```

```
.DATA
a DW 43551d
b DW 65322d
x DW 0
y DW 0, '$'
```

```
.CODE
MOV AX, @DATA
MOV DS, AX

MOV AX, a
MOV BX, b
MUL BX

MOV x, DX
MOV y, AX

SUB BX, BX
MOV BX, x
CALL valor1
CALL valor2

SUB BX, BX
MOV BX, y
CALL valor1
CALL valor2

MOV AH, 4Ch
INT 21h
```

```

saida PROC NEAR
    ADD    DL, 30h
    CMP    DL, 39h
    JLE    valor
    ADD    DL, 07h
    valor:
        INT    21h
    RET

```

```

saida ENDP

```

```

valor1 PROC NEAR
    MOV    AH, 02h
    MOV    DL, BH
    MOV    CL, 04h
    SHR    DL, CL
    CALL   saida
    MOV    DL, BH
    AND    DL, 0Fh
    CALL   saida
    RET

```

```

valor1 ENDP

```

```

valor2 PROC NEAR
    MOV    AH, 02h
    MOV    DL, BL
    MOV    CL, 04h
    SHR    DL, CL
    CALL   saida
    MOV    DL, BL
    AND    DL, 0Fh
    CALL   saida
    RET

```

```

valor2 ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho <Ctrl> + <A> do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save** com o nome **MULTIP2**. A Figura 8.25 mostra um trecho de como o programa deve ficar.

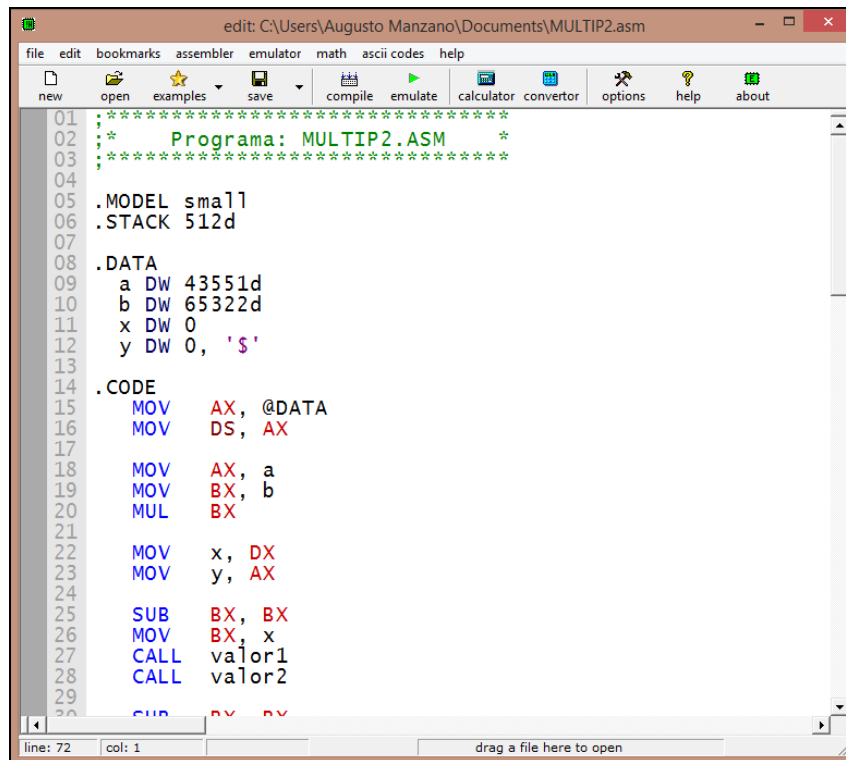
Esse programa utiliza muitos dos conceitos já abordados, entre eles a movimentação de *bits*, o uso de procedimentos e saltos condicionais. Note o trecho de código definido entre as linhas **39** e **46**, como apresentado em seguida:

```

saida PROC NEAR
    ADD    DL, 30h
    CMP    DL, 39h
    JLE    valor
    ADD    DL, 07h
    valor:
        INT    21h
    RET
saida ENDP

```

Para a definição desse procedimento (sub-rotina) estão sendo utilizadas as diretivas **PROC** e **ENDP**, que definem, respectivamente, o início (**PROC** - **PROC**edure) e o fim (**ENDP** - **END** Procedure) de um procedimento. A diretiva **ENDP** é utilizada por duas razões: a primeira razão por ela identificar o ponto de início (primeira instrução) do procedimento após a diretiva **PROC**; a segunda razão pelo fato de sinalizar ao programa *Assembler* o fim do trecho do procedimento em operação.



```

01;*****
02; Programa: MULTIP2.ASM
03;*****
04
05.MODEL small
06.STACK 512d
07
08.DATA
09 a DW 43551d
10 b DW 65322d
11 x DW 0
12 y DW 0, '$'
13
14.CODE
15 MOV AX, @DATA
16 MOV DS, AX
17
18 MOV AX, a
19 MOV BX, b
20 MUL BX
21
22 MOV x, DX
23 MOV y, AX
24
25 SUB BX, BX
26 MOV BX, x
27 CALL valor1
28 CALL valor2
29
30 SUB BX, BX

```

Figura 8.25 - Programa MULTIP2 na ferramenta emu8086.

O parâmetro opcional **NEAR** definido à frente da diretiva **PROC** estabelece para o programa *Assembler* (neste caso **emu8086**) informações sobre o uso e forma de acesso aos segmentos de memória pelo programa em execução. Dependendo da forma de acesso ao endereçamento de memória, pode também ser utilizado o parâmetro **FAR** após a diretiva **PROC**.

O programa *Assembler* utiliza a informação do parâmetro definido após a diretiva **PROC** para estabelecer a forma de acesso que ocorrerá na memória por meio de um procedimento. Esse acesso pode ocorrer no mesmo segmento de memória quando usada a diretiva **NEAR** que representa um segmento próximo, ou em segmentos diferentes do local em que o programa está definido por meio da diretiva **FAR** que representa um segmento distante. Mais adiante (próximo tópico) este tema será discutido com um pouco mais de detalhamento.

A rotina de programa tem por finalidade imprimir um caractere hexadecimal na tela do monitor de vídeo, levando em consideração a faixa numérica de **0** (zero hexadecimal) até **9** (nove hexadecimal) e a faixa de **A** (dez hexadecimal) até **F** (quinze hexadecimal) de acordo com o código da tabela ASCII.

Nas linhas **50** até **60** e nas linhas **62** até **72** encontram-se, respectivamente, os procedimentos **valor1** e **valor2** que são responsáveis pela obtenção do primeiro e do segundo caracteres hexadecimais que serão armazenados no registrador **DX**.

```

valor1 PROC NEAR
    MOV AH, 02h
    MOV DL, BH
    MOV CL, 04h
    SHR DL, CL
    CALL saida
    MOV DL, BH
    AND DL, 0Fh
    CALL saida
    RET
valor1 ENDP

```



```

valor2 PROC NEAR
    MOV     AH, 02h
    MOV     DL, BL
    MOV     CL, 04h
    SHR     DL, CL
    CALL    saida
    MOV     DL, BL
    AND     DL, 0Fh
    CALL    saida
    RET
valor2 ENDP

```

Com relação à parte do programa principal, ele amplifica a apresentação de valores hexadecimais de dois dígitos para oito dígitos. Observe o trecho de código seguinte:

```

.CODE
    MOV     AX, @DATA
    MOV     DS, AX

    MOV     AX, a
    MOV     BX, b
    MUL     BX

    MOV     x, DX
    MOV     y, AX

    SUB     BX, BX
    MOV     BX, x
    CALL    valor1
    CALL    valor2

    SUB     BX, BX
    MOV     BX, y
    CALL    valor1
    CALL    valor2

    MOV     AH, 4Ch
    INT     21h

```

Após a multiplicação (das linhas 19 até 21) são transferidos para as variáveis **x** e **y**, respectivamente, os valores do resultado da multiplicação armazenados nos registradores gerais **DX** e **AX**. Isso é necessário, pois os registradores devem estar disponíveis para a operação de processamento do programa.

Na sequência são definidos dois trechos de código, um existente entre as linhas 26 e 29 e outro no trecho das linhas 31 e 34. O trecho da linha 26 até a linha 29 manipula os valores da variável **x** (antigo conteúdo do registrador geral **DX**) e pelas chamadas de procedimento com a instrução **CALL** apresenta os primeiros quatro caracteres hexadecimais. Por meio do trecho de código da linha 31 até a linha 34 manipula os dados da variável **y** (antigo conteúdo do registrador geral **AX**).

A operação de multiplicação dos valores **43551d** e **65322d** fornece como resultado um valor decimal **2.844.838.422** (ou seu equivalente **A990CA16** em hexadecimal). Para ver esse resultado, acione a tecla de função <F5> e em seguida acione a tecla de função <F9> ou use o botão **Run**. A Figura 8.26 mostra o resultado obtido após a operação indicada.

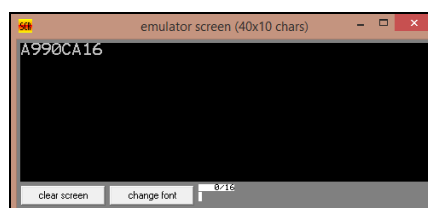


Figura 8.26 - Resultado de multiplicação no programa.

Em seguida considere como exemplo um programa que deva apresentar o resultado da multiplicação de dois valores de 16 *bits* utilizando a instrução **IMUL**. O programa deve executar a equação $YX \leftarrow A * B$, em que a variável **A** possui o valor decimal negativo **-32.700 (8044h)** de 16 *bits* (tipo **DW**) e a variável **B** o valor decimal **25 (19h)** de 16 *bits* (tipo **DW**), que resulta no valor decimal **-81.7500 (FFF386A4h)**. Observe atentamente cada linha do código do programa a seguir:

```

;*****
;*      Programa: MULTIP3.ASM      *
;*****

.MODEL small
.STACK 512d

.DATA
a DW -32700d
b DW 25d
x DW 0
y DW 0, '$'

.CODE
MOV     AX, @DATA
MOV     DS, AX

MOV     AX, a
MOV     BX, b
IMUL    BX
MOV     x, DX
MOV     y, AX

SUB     BX, BX
MOV     BX, x
CALL    valor1
CALL    valor2

SUB     BX, BX
MOV     BX, y
CALL    valor1
CALL    valor2

MOV     AH, 4Ch
INT     021h

saida PROC NEAR
ADD     DL, 30h
CMP     DL, 39h
JLE     valor
ADD     DL, 07h
valor:
INT     21h
RET
saida ENDP

valor1 PROC NEAR
MOV     AH, 02h
MOV     DL, BH
MOV     CL, 04h
SHR     DL, CL
CALL    saida
MOV     DL, BH
AND     DL, 0Fh
CALL    saida
RET

```

```

valor1 ENDP

valor2 PROC NEAR
    MOV     AH, 02h
    MOV     DL, BL
    MOV     CL, 04h
    SHR     DL, CL
    CALL    saida
    MOV     DL, BL
    AND     DL, 0Fh
    CALL    saida
    RET
valor2 ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o com os comandos de menu **file/save** com o nome **MULTIP3**. A Figura 8.27 mostra um trecho de como o programa deve ficar.

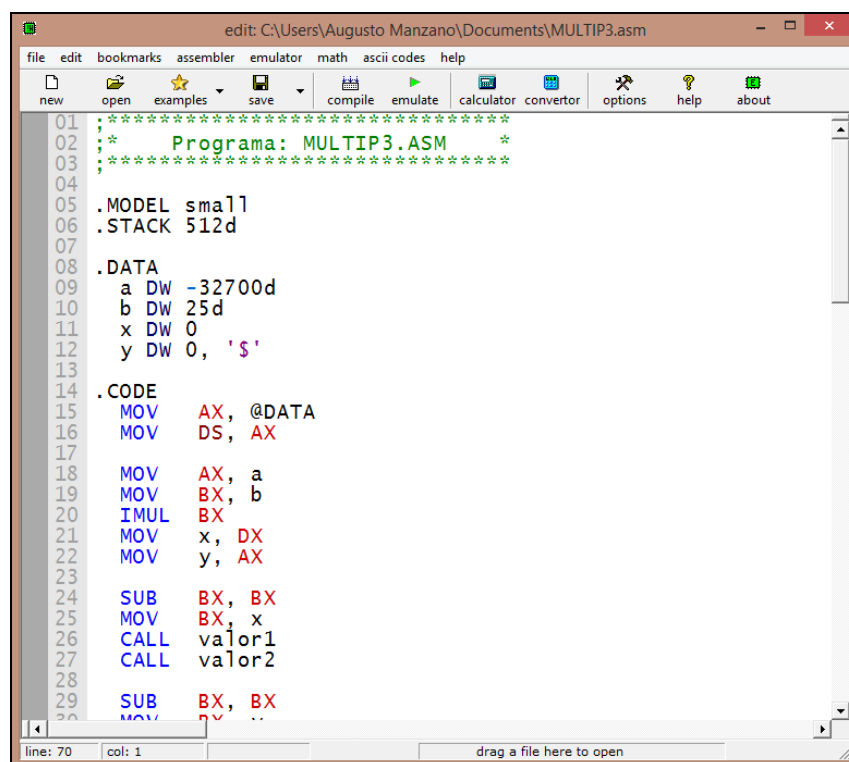
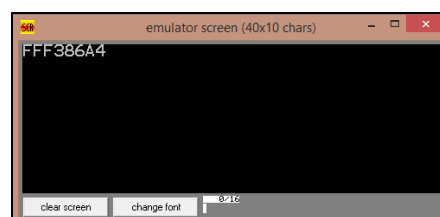


Figura 8.27 - Programa MULTIP3 na ferramenta Emu8086.

Os programas **MULTIP3** e **MULTIP2** são semelhantes. A diferença está na definição dos valores para as variáveis **a** e **b**, e também no uso do comando **IMUL** na linha **25** do programa.

A operação de multiplicação dos valores **-32700d** e **25d** fornece como resultado um valor decimal negativo **-817.500** (ou seu equivalente **FFF386A4** em hexadecimal). Para ver esse resultado, acione a tecla de função **<F5>** e em seguida acione a tecla de função **<F9>** ou use o botão **Run**. A Figura 8.28 mostra o resultado obtido após a operação indicada.



8.4 - Procedimento próximo x procedimento distante

O conceito de procedimento próximo ou distante está associado ao uso do tipo de parâmetro após a definição da diretiva **PROC** que pode ser **NEAR** (próximo) ou **FAR** (distante), dependendo de como se deseja acessar o endereçamento de memória.

O parâmetro **NEAR** (que pode ser omitido) é utilizado quando o acesso de um procedimento na memória ocorre no mesmo segmento onde se encontra o código de programa em execução (chamada direta intrassegmento), ou seja, o acesso ocorre apenas com a alteração do valor do registrador de ponteiro **IP** sem que ocorra a alteração do registrador de segmento **CS**. É importante levar em conta que cada segmento de memória está limitado a um tamanho de 64 *KBytes*, o que inviabiliza o desenvolvimento de grandes programas. Ao término de execução de um procedimento **NEAR** o retorno a parte chamadora da subrotina é efetuado com a instrução **RET**.

O parâmetro **FAR** (deve obrigatoriamente ser declarado) é utilizado quando o acesso a um procedimento na memória ocorre em um endereço de segmento diferente do endereço de segmento em que se encontra o código do programa em execução (chamada direta intersegmento), ou seja, ocorre a alteração do valor dos registradores **IP** e **CS**. Desta forma, torna-se possível ultrapassar a barreira do tamanho de 64 *KBytes* de cada segmento de memória. Ao término de execução de um procedimento **FAR** o retorno a parte chamadora da sub-rotina é efetuado com a instrução **RETF**.

A execução de um procedimento com parâmetro **NEAR** é mais simples que a execução de um procedimento com parâmetro **FAR**. Note que o parâmetro **NEAR** manipula apenas o registrador de ponteiro **IP**, enquanto o parâmetro **FAR** manipula os registradores **IP** e **CS**. Assim sendo, o parâmetro **NEAR** coloca na pilha um único *word* contendo o endereço do segmento **IP**, enquanto o parâmetro **FAR** coloca na pilha dois *words*, sendo um valor para o registrador de ponteiro **IP** e o outro valor para o registrador de segmento **CS**.

Os microprocessadores padrão 8086/8088 operam com base em uma arquitetura de memória segmentada, onde somente é possível acessar um segmento de memória de 64 KB por vez. É devido a esta característica que se tem a definição de procedimentos com parâmetro **NEAR** ou **FAR**. A Figura 8.29 exemplifica o uso conceitual de procedimentos **NEAR** e **FAR**.

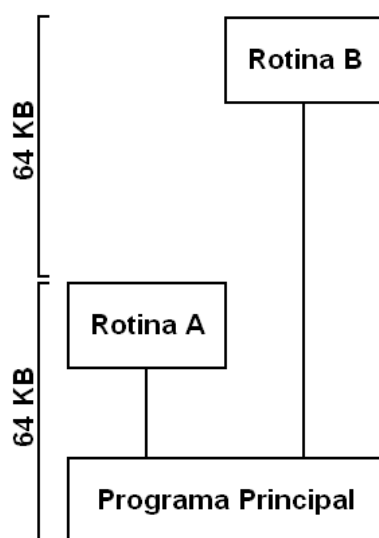


Figura 8.29 - Procedimentos NEAR e FAR.

O procedimento **Rotina A** está situado no mesmo segmento de memória em que se encontra definido o **Programa Principal**. Neste caso o procedimento **Rotina A** é um procedimento do tipo **NEAR** (próximo), pois para acessar o procedimento **Rotina A** não é necessário mudar o endereço do registrador de segmento **CS** uma vez que o registrador de segmento **CS** aponta para o mesmo segmento de memória onde está o **Programa Principal**, o que muda apenas é o endereço do registrador de ponteiro **IP**. Neste caso o valor do deslocamento muda, mas o valor do segmento permanece inalterado.

O procedimento **Rotina B** está situado em um segmento de memória diferente do segmento de memória onde se encontra o **Programa Principal**. Neste caso o procedimento **Rotina B** é um procedimento **FAR** (distante), pois para o **Programa Principal** acessar o procedimento **Rotina B** é necessário mudar o valor do endereço do registrador de segmento **CS** e o valor do registrador de ponteiro **IP**.

O parâmetro **NEAR** de procedimentos é utilizado em programas do tipo **.COM**, pois os programas desse tipo são sempre escritos no mesmo segmento de memória e o parâmetro **FAR** é utilizado em programas do tipo **.EXE**, como pode ser visto em um pequeno exemplo no capítulo nove desta obra, pois esses podem ser feitos em mais de um segmento.

Anotações

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 9

SALTOS, DECISÕES E REPETIÇÕES

Anteriormente foram apresentados superficialmente alguns desvios, laços e saltos em exemplos de programas aplicados. Este capítulo explica mais detalhadamente instrução de salto incondicional e condicional, tomada de decisões (simples e composta), instruções lógicas, laços de repetição e utilização de rotinas do tipo macro, além de apresentação de valores numéricos em notação decimal com o uso da pilha.

9.1 - Salto incondicional

Os saltos incondicionais estão associados ao desvio do fluxo de programa de um determinado ponto do código para outro ponto do mesmo código. Isso é conseguido com o comando de salto **JMP** (*jump*) utilizado e demonstrado anteriormente em várias ocasiões, similar ao comando **GOTO** encontrada em várias linguagens de programação de computadores de alto nível.

Esse tipo de salto deve ser usado com muita cautela, pois o uso excessivo pode tornar o código de programa confuso. Procure usá-lo o mínimo possível. Dê preferência à utilização de procedimentos. Assuma uma postura estruturada de programação. A título de ilustração considere o código de programa seguinte com o uso de saltos com o comando **JMP**:

```
;*****
;*   Programa: MENSAGEM4.ASM   *
;*****

org 100h

.DATA
mensagem1 DB 'Mensagem 1', 0Dh, 0Ah, 24h
mensagem2 DB 'Mensagem 2', 0Dh, 0Ah, 24h
mensagem3 DB 'Mensagem 3', 0Dh, 0Ah, 24h

.CODE
JMP  salto3

salto1:
    LEA  DX, mensagem1
    CALL escreva
    JMP  saida

salto2:
    LEA  DX, mensagem2
    CALL escreva
    JMP  salto1
```



```
salto3:
    LEA    DX, mensagem3
    CALL   escreva
    JMP    salto2
```

```
saida:
    INT    20h
```

```
escreva PROC NEAR
    MOV    AH, 09h
    INT     21h
    RET
escreva ENDP
```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **File/Save** com o nome **MENSAGEM4**, de forma que fique semelhante à imagem da Figura 9.1.

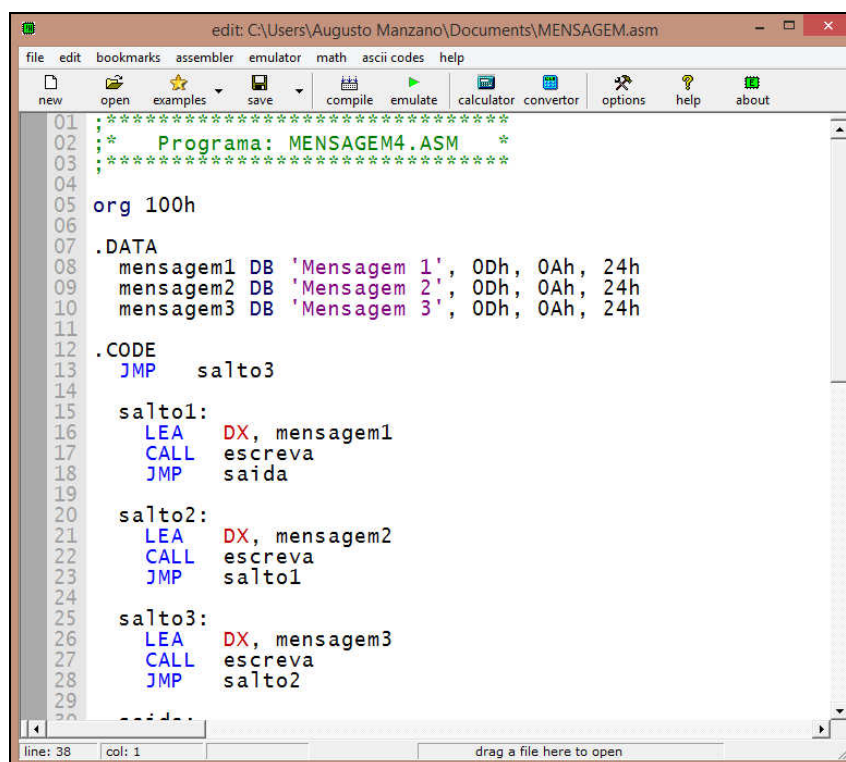


Figura 9.1 - Programa MENSAGEM4 na ferramenta emu8086.

Ao executar o programa, serão apresentadas as mensagens **Mensagem 3**, **Mensagem 2** e **Mensagem 1**, como mostra a Figura 9.2. Se forem omitidas as linhas com o comando **JMP**, a saída será **Mensagem 1**, **Mensagem2** e **Mensagem 3** (faça o teste).

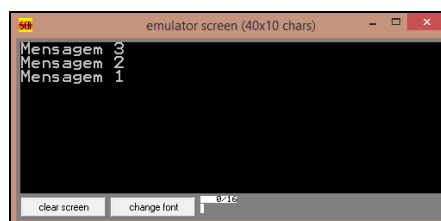


Figura 9.2 - Tela de saída do programa MENSAGEM4 na ferramenta emu8086.

O comando **JMP** desvia a execução do programa sem nenhum motivo aparente, apenas pela simples questão de ser esta a sua finalidade e de a instrução estar ali definida. Por esta razão o comando **JMP** é denominado *desvio incondici-*

onal. Essa instrução é executada após a atualização do registrador **IP** em relação ao acesso à instrução seguinte e pode ser utilizada de duas maneiras distintas.

Observe que na linha **05** do programa está sendo usada a diretiva de compilação **org 100h** (poderia também ter sido usado **org 0x100**) que define um arquivo de tamanho pequeno, de até 64 *KBytes*, similar aos arquivos de programas criados com o programa **Enhanced DEBUG**.

Nas linhas de **08** até **10** ao lado da definição das variáveis **mensagem1**, **mensagem2** e **mensagem3**, além do *string* a ser apresentado, são definidos após a vírgula de cada mensagem os valores **0Dh**, **0Ah** e **24h**, que correspondem à ação de mudança de linha do cursor após a escrita da mensagem.

O valor **0Dh** corresponde ao código de uso da tecla **<Enter>**, o valor **0Ah** corresponde ao código de mudança de linha e o código **24h** corresponde ao uso do caractere **\$**, necessário para identificar o final de uma sequência de caracteres. Note que quando se faz a entrada de um dado alfabético no teclado de um computador e aciona-se a tecla **<Enter>**, ocorre internamente a ação dos três códigos aqui apresentados.

Na linha **13** há o comando **JMP salto3** que envia o fluxo de execução do programa para a linha **25** (identificada com o rótulo **salto3**;) e a partir desse ponto executam-se o envio do conteúdo da **mensagem3** para o registrador **DX** na linha **26** e a linha **27** que faz a chamada do procedimento **escreva** por meio da instrução **CALL escreva** que se encontra definida a partir da linha **33**.

O procedimento **escreva** faz a definição do código de apresentação de caracteres **09h** para o registrador **AH** que será utilizado pela interrupção **21h**. Após a execução das linhas de código de **33** até **36**, ou seja, após a apresentação do conteúdo da variável **mensagem3**, a linha **36** por meio da instrução **RET** faz o retorno do procedimento para a primeira linha após sua chamada. Como a primeira chamada do procedimento ocorreu pela linha **27**, o retorno é feito na linha **28**.

A linha **28** (**JMP salto2**) faz um desvio para a linha **20** (identificada com o rótulo **salto2**;) e a partir dessa posição ocorre na linha **21** (**LEA DX, mensagem2**) o apontamento para o conteúdo da variável **mensagem2** para o registrador **DX**. Na linha **22** é feita nova chamada ao procedimento **escreva** que após sua execução retorna o fluxo de execução do programa para a linha **23**, que faz o desvio do programa para a linha **15**.

A partir da linha **15** ocorre na linha **16** o apontamento do conteúdo da variável **mensagem3** para o registrador **DX**. Em seguida na linha **17** é feita nova chamada do procedimento **escreva** que, após sua operacionalização, devolve o fluxo do programa à linha **18** que desvia o fluxo para a linha **30** em que o encerramento do programa é então processado.

9.2 - Salto condicional

O salto condicional é diferente de um salto incondicional representado pelo comando **JMP**, pois para ser executado, necessita de um comando condicional sobre a qual uma decisão é tomada. Normalmente uma condição é estabelecida com o uso de um dos comandos: **CMP** (*Compare*), **AND** (*Logical and*), **OR** (*Logical or*), **NOT** (*Logical not*) e **XOR** (*Exclusive or*), dos quais os comandos **CMP** e **AND** já foram apresentados.

Para auxiliar o processo de ação de desvios condicionais, são utilizadas os comandos de saltos **JA**, **JAE**, **JB**, **JBE**, **JC**, **JCXZ**, **JE**, **JG**, **JGE**, **JL**, **JLE**, **JNA**, **JNAE**, **JNB**, **JNBE**, **JNC**, **JNE**, **JNG**, **JNGE**, **JNL**, **JNS**, **JNO**, **JNP**, **JNLE**, **JNO**, **JNP**, **JNS**, **JNZ**, **JO**, **JP**, **JPE**, **JPO**, **JS**, **JZ**, dos quais os comandos **JG**, **JL**, **JLE**, **JNZ** já foram apresentadas e demonstradas. As instruções de desvios condicionais efetuam seus desvios numa escala entre -127 e 128 *bytes*.

A Tabela 9.1 exhibe o conjunto completo de comandos de saltos a serem efetivadas pela linguagem de programação *Assembly 8086/8088*, após a utilização de uma instrução condicional.

Tabela 9.1 - Instruções de salto condicional

Comando	Significado	Descrição
JA	jump on above	salte se acima de
JAE	jump on above or equal	salte se acima ou igual a
JB	jump on below	salte se abaixo de
JBE	jump on below or equal	salte se abaixo ou igual a
JC	jump on carry	salte se flag carry igual a 1

Comando	Significado	Descrição
JCXZ	jump if cx register zero	salte se CX registra zero
JE	jump on equal	salte se igual a
JG	jump on greater	salte se maior que
JGE	jump on greater or equal	salte se maior ou igual a
JL	jump on less	salte se menor que
JLE	jump on less or equal	salte se menor ou igual a
JNA	jump on not above	salte se não acima de
JNAE	jump on not above or equal	salte se não acima ou igual a
JNB	jump on not below	salte se não abaixo de
JNBE	jump on not below or equal	salte se não abaixo ou igual a
JNC	jump on not carry	salte se flag carry igual a 0
JNE	jump on not equal	salte se não igual a
JNG	jump on not greater	salte se não maior que
JNGE	jump on not greater or equal	salte se não maior ou igual a
JNL	jump on not less	salte se não menor que
JNLE	jump on not less or equal	salte se não menor ou igual a
JNO	jump or not overlay	salte se não ocorreu <i>overlay</i>
JNP	jump on not parity	salte se não for par
JNS	jump on not sign	salte se positivo
JNZ	jump on not zero	salte se não for zero
JO	jump on overflow	salte se ocorreu <i>overflow</i>
JP	jump on parity	salte se for par
JPE	jump on parity equal	salte se for par
JPO	jump on parity odd	salte se for ímpar
JS	jump on sign	salte se for negativo
JZ	jump on zero	salte se for zero

É importante considerar que alguns comandos de saltos condicionais possuem aparentemente certa ambiguidade operacional com outros comandos condicionais, pois executam a mesma ação lógica de processamento, mas são grafadas de forma diferente. São pares de ação as instruções da Tabela 9.2.

Tabela 9.2 - Instruções de salto opostas

Comando	Comando Oposto	Comando	Comando Oposto
JA	JNBE	JGE	JNL
JAЕ	JNB	JL	JNGE
JB	JNAE	JLE	JNG
JBE	JNA	JNP	JPO
JE	JZ	JP	JPE
JG	JNLE		

Esses comandos condicionais formam alguns pares de comandos. Um determinado comando pode ser utilizado perfeitamente no lugar do outro comando, dependendo da escolha ou necessidade do desenvolvedor ou do programa em desenvolvimento.

É importante considerar que para um comando de salto condicional operar ela necessita do auxílio e definição anteriores de um dos comandos de comparação **CMP**, **AND**, **OR**, **NOT** e **XOR**.

Para a realização de desvios condicionais baseados na estrutura relacional, os comandos de comparação devem ser usados com os comandos de saltos condicionais da Tabela 9.3.

Tabela 9.3 - Instruções de salto relacionais

Comando	Descrição	Equivalência
JE	salte se igual a	=
JG	salte se maior que	>
JGE	salte se maior ou igual a	>=
JL	salte se menor que	<
JLE	salte se menor ou igual a	<=
JNE	salte se não igual a (diferente de)	<>

Para a realização de desvios condicionais baseados na verificação dos registradores de estado, os comandos de comparação devem ser usados com os comandos de saltos condicionais indicados na Tabela 9.4.

Tabela 9.4 - Instruções de salto baseados em registradores

Comando	Descrição
JC	salta se registrador CF for igual a 1
JNC	salta se registrador CF for igual a 0
JNO	salta se registrador OF for igual a 0
JNS	salta se registrador SF for igual a 1
JNZ	salta se registrador ZF for igual a 0
JO	salta se registrador OF for igual a 1
JPE	salta se registrador PF for igual a 1 (paridade par)
JPO	salta se registrador PF for igual a 0 (paridade ímpar)
JS	salta se registrador SF for igual a 1
JZ	salta se registrador ZF for igual a 1

Todos os comandos de saltos condicionais são iniciados com o caractere "J" de *jump*. Qualquer um dos comandos de salto condicional deve estar associado a um rótulo de identificação para determinar o local de desvio no programa.

Os saltos condicionais, diferentemente dos saltos incondicionais, são feitos de forma relativa quanto à posição de sua definição. Não é possível realizar saltos condicionais muito longos. O tamanho máximo de salto está na faixa de -128 *bytes* a 127 *bytes* a partir da posição em que se encontra o comando de salto.

Além dos comandos condicionais, existem comandos de desvios condicionais de uso geral, indicados na Tabela 9.5.

Tabela 9.5 - Instruções de salto gerais

Comando	Descrição
JA	salte se acima de
JAЕ	salte se acima ou igual a
JB	salte se abaixo de
JBE	salte se abaixo ou igual a
JCXZ	salte se CX registra zero
JNA	salte se não acima de
JNAE	salte se não acima ou igual a
JNB	salte se não abaixo de
JNBE	salte se não abaixo ou igual a
JNG	salte se não maior que

Comando	Descrição
JNGE	salte se não maior ou igual a
JNL	salte se não menor que
JNLE	salte se não menor ou igual a
JNP	salte se não for par
JP	salte se for par

No universo de saltos, os comandos podem ser divididos em três grupos operacionais: o primeiro grupo são os que operam com apenas um registrador de estado, o segundo grupo são os que operam com valores sinalizados (valores negativos) e o terceiro são os que operam com valores não sinalizados (valores positivos), representadas na Tabela 9.6.

Tabela 9.6 - Instruções de salto separadas por grupos operacionais

Saltos com um Registrador de Estado		
Comando	Condição	Comando Oposto
JZ, JE	ZF = 1	JNZ, JNE
JC, JB, JNAE	CF = 1	JNC, JNB, JAE
JS	SF = 1	JNS
JO	OF = 1	JNO
JPE, JP	PF = 1	JPO
JNZ, JNE	ZF = 0	JZ, JE
JNC, JNB, JAE	CF = 0	JC, JB, JNAE
JNS	SF = 0	JS
JNO	OF = 0	JO
JPO, JNP	PF = 0	JPE, JP
Saltos com Valores Sinalizados (Negativos)		
Comando	Condição	Comando Oposto
JE, JZ	ZF = 1	JNE, JNZ
JNE, JNZ	ZF = 0	JE, JZ
JG, JNLE	ZF = 0 e SF = 0F	JNG, JLE
JL, JNGE	SF <> 0F	JNL, JGE
JGE, JNL	SF = 0F	JNGE, JL
JLE, JNG	ZF = 1 ou SF <> 0F	JNLE, JG
Saltos com Valores Não Sinalizados (Positivos)		
Comando	Condição	Comando Oposto
JE, JZ	ZF = 1	JNE, JNZ
JNE, JNZ	ZF = 0	JE, JZ
JA, JNBE	CF = 0 e ZF = 0	JNA, JBE
JB, JNAE, JC	CF = 1	JNB, JAE, JNC
JAE, JNB, JNC	CF = 0	JNAE, JB
JBE, JNA	CF = 1 ou ZF = 1	JNBE, JA

Os comandos de saltos JA, JAE, JBE, JE, JE, JG, JGE, JL, JLE, JNE, JNE e JB, bem como seus equivalentes, podem e devem ser utilizados após alguma ação aritmética ou lógica.

9.3 - Desvios condicionais

Com a finalidade de explorar o uso de alguns comandos de saltos condicionais, são exibidos em seguida exemplos simples de situações nas quais é necessário utilizar esses recursos (vale ressaltar que alguns recursos já haviam sido usados), como é o caso da tomada de decisão: simples, composta, sequencial, encadeada e por seleção.

É necessário primeiramente entender e conhecer o mecanismo de funcionamento das principais instruções de saltos condicionais, para depois melhorar a forma de estabelecer o processo entrada e saída de dados. Infelizmente por questão de espaço torna-se difícil exemplificar e utilizar todas as instruções de saltos condicionais. No entanto, muitas delas serão usadas de forma automática em outras situações a serem estabelecidas.

9.3.1 - Desvio condicional simples

A decisão simples ocorre quando uma ação necessita ser realizada caso uma determinada condição seja verdadeira. Se a condição for falsa, não deve ser executada a ação especificada como verdadeira, passando o controle operacional do programa para outro ponto, que normalmente também será executado após a ação considerada verdadeira.

Imagine a necessidade de verificar se o conteúdo do registrador **AL** e o conteúdo do registrador **BL** são de fato iguais. Se forem iguais, é necessário somar o valor **1** ao registrador geral **AL**. Após a execução da decisão sobre a condição ser verdadeira ou falsa, o conteúdo do registrador geral **BL** deve ser diminuído em **1**. O programa deve apresentar os valores dos registradores **AL** e **BL**.

Na linguagem *Assembly 8086/8088* de programação de computadores o programa deve ser codificado como:

```
;*****
;*      Programa: CONDIC1.ASM      *
;*****

.MODEL small
.STACK 512d

.DATA
a DB 6d
b DB 6d

.CODE
MOV AX, @DATA
MOV DS, AX

MOV AL, a
MOV BL, b

CMP AL, BL
JE  entao
JMP fimse

entao:
INC AL
CALL apoio
MOV DL, AL
CALL escreva
```

```

fimse:
    DEC    BL
    CALL  apoio
    MOV    DL, BL
    CALL  escreva

INT    20h

escreva PROC NEAR
    ADD    DL, 30h
    CMP    DL, 39h
    JLE    valor
    ADD    DL, 07h
valor:
    INT    21h
    RET
escreva ENDP

apoio PROC NEAR
    MOV    AH, 02h
    MOV    CL, 04h
    SHR    DL, CL
    RET
apoio ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e então escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **CONDIC1**, de forma que fique semelhante à imagem da Figura 9.3.

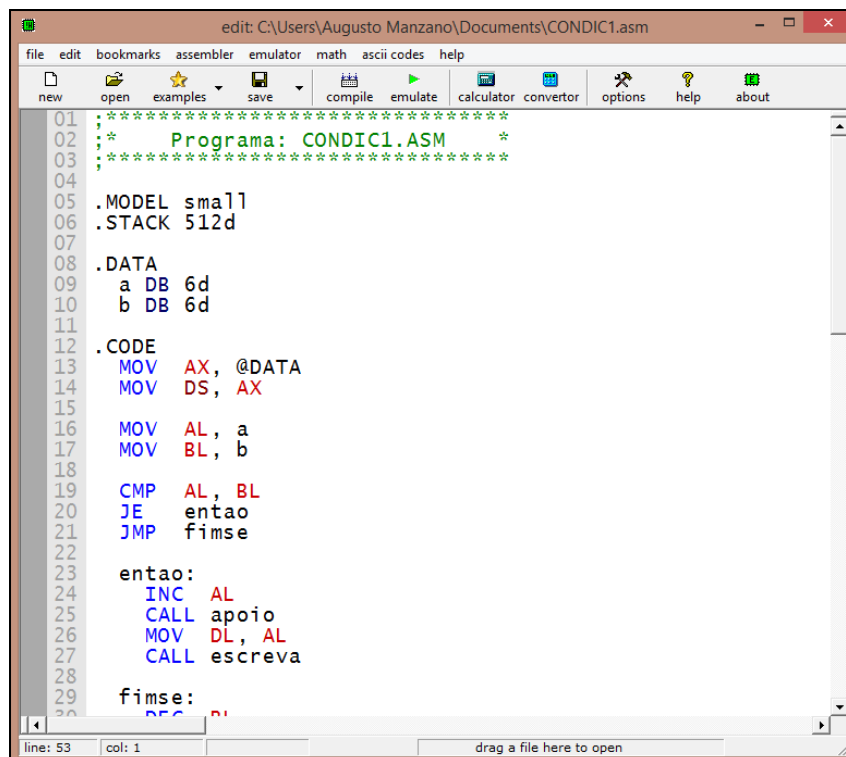


Figura 9.3 - Programa CONDIC1 na ferramenta emu8086.

Observe o trecho de código entre as linhas 19 e 21, o qual representa algo como **SE (AL = BL) ENTÃO**. No lugar da instrução **JE entao** (linha 20) pode ser utilizada sem problema a instrução **JZ entao**.

Os registradores **AL** e **BL**, respectivamente, possuem o valor **6** (decimal) das variáveis **a** e **b** (linhas **16** e **17**), e sendo a condição verdadeira, ela é desviada pela instrução **JE** para o ponto de execução **então** definido entre as linhas **23** e **27**.

No trecho de programa assinalado como **entao** ocorre o incremento de valor **01h** ao valor existente no registrador **AL** por meio da instrução **INC** (linha **24**). Na sequência é chamado o procedimento **apoio** (linha **25**) que prepara a memória para a apresentação do valor calculado do registrador **AL**. Em seguida é transferido o valor do registrador **AL** para o registrador **DL** (linha **26**) e na linha de código **27** é chamado o procedimento **escreva** que apresenta o valor armazenado em **AL**.

Após apresentar o valor do registrador **AL** (que estará com o valor **07h**), o programa executa o trecho identificado como **fimse** (linha **29**), no qual faz o decremento de **01h** sobre o registrador **BL** por meio da instrução **DEC** (linha **30**). Assim sendo, o registrador **BL** passa a possuir o valor **05h**.

Em seguida na linha **31** é efetuada a chamada ao procedimento **apoio** que após sua execução retorna seu processamento para a linha **32**. O conteúdo do registrador **BL** é movimentado para o registrador **DL**, depois na linha **33** ocorre a chamada ao procedimento **escreva**.

O procedimento denominado **apoio** definido entre as linhas **47** e **52** faz a preparação do ambiente para a apresentação de um caractere por vez do valor a ser mostrado na tela do monitor de vídeo. Já o procedimento **escreva** faz a apresentação de um caractere por vez.

Experimente trocar os valores das variáveis **a** e **b** e executar o programa para verificar a forma de funcionamento com valores diferentes.

9.3.2 - Desvio condicional composto

A decisão composta ocorre quando uma ação precisa ser realizada caso uma determinada condição seja verdadeira ou falsa. Se a condição **é** verdadeira, será executada uma determinada ação especificada **para esta finalidade**. Se a condição por falsa, será executada outra ação.

Imagine a necessidade de verificar se o conteúdo do registrador **AL** é maior que o conteúdo do registrador **BL**. Se for maior, é necessário somar o valor **1** ao registrador **AL** e apresentar seu valor; caso contrário, o conteúdo do registrador **BL** deve ser diminuído em **1** para então ser apresentado seu resultado.

Na linguagem *Assembly 8086/8088* de programação de computadores o programa deve ser codificado como:

```
;*****
;*      Programa: CONDIC2.ASM      *
;*****

.MODEL small
.STACK 512d

.DATA
    a DB 4d
    b DB 9d

.CODE
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, a
    MOV BL, b

    CMP AL, BL
    JG  entao
    JLE senao
```

```

entao:
    INC AL
    CALL apoio
    MOV DL, AL
    CALL escreva
    JMP fimse

senao:
    DEC BL
    CALL apoio
    MOV DL, BL
    CALL escreva
    JMP fimse

fimse:
    MOV AH, 4Ch
    INT 21h

escreva PROC NEAR
    ADD DL, 30h
    CMP DL, 39h
    JLE valor
    ADD DL, 07h
valor:
    INT 21h
    RET
escreva ENDP

apoio PROC NEAR
    MOV AH, 02h
    MOV CL, 04h
    SHR DL, CL
    RET
apoio ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **CONDIC2**, de forma que fique semelhante à imagem da Figura 9.4.

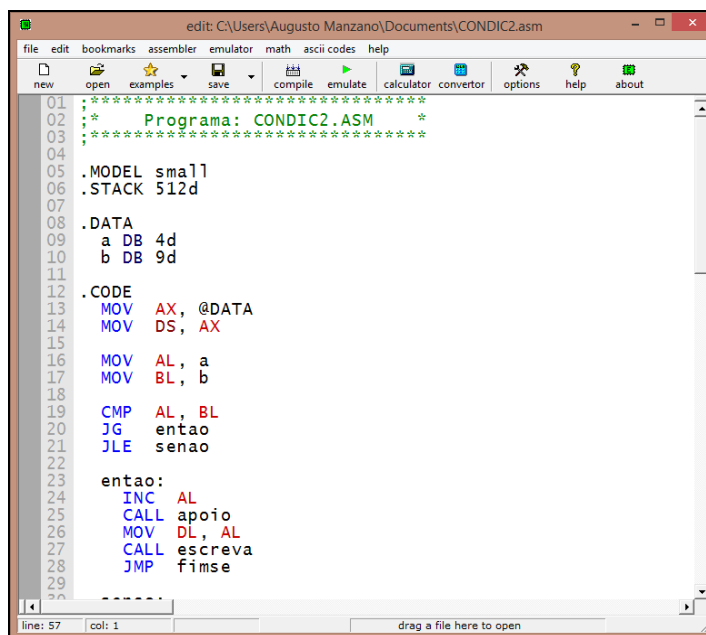


Figura 9.4 - Programa CONDIC2 na ferramenta emu8086.

Atente para os detalhes de código entre as linhas **19** e **21**. Note que estão sendo utilizadas após a instrução **CMP** duas instruções de desvio condicional. Uma é a instrução **JG** (condição maior que) na linha **20** e a outra **JLE** (condição menor que) na linha **21**.

Em linhas gerais o programa **CONDIC2** é muito semelhante ao programa **CONDIC1**, tendo como diferença principal as instruções defini-das entre as linhas **19** e **21**.

As duas instruções de desvio condicional permitem a representação da ação da tomada de decisões composta. O funcionamento da instrução **JG** e da instrução **JLE** é proporcionalmente inverso.

Experimente trocar os valores das variáveis **a** e **b** e executar o programa para verificar a forma de funcionamento com valores diferentes.

9.4 - Operações lógicas

O uso de operações lógicas em um programa são elementos importantes quando é necessário trabalhar com mais de uma condição (quando da utilização dos comandos operadores **AND**, **OR** e **XOR**) ou fazer a negação de uma condição (comando operador **NOT**) para a tomada de outra condição.

É importante ressaltar que as ações lógicas operam o conteúdo armazenado na memória no nível binário. Se estiver em análise um dado do tipo *byte* ou *word*, a instrução lógica em uso fará uma varredura do *bit* mais baixo para o *bit* mais alto, verificando o dado da direita para a esquerda. Se todos os *bits* corresponderem à condição estabelecida, ela será considerada *verdadeira*; caso contrário, será considerada falsa.

De forma operacional as instruções lógicas devem ser usadas seguindo esta sintaxe:

```
AND [destino], [origem]
OR  [destino], [origem]
XOR [destino], [origem]
NOT [destino]
```

Os parâmetros **destino** e **origem** são obrigatórios e o resultado lógico da avaliação da instrução lógica em uso será sempre implicado no parâmetro **destino**.

O comando **AND** estabelece a conjunção de dois valores em nível binário: **origem** e **destino**. O resultado de saída de **destino** será verdadeiro (valor **1**) se ambos os *bits* da posição em análise forem **1**; caso contrário, será considerado falso (valor **0**). Veja a tabela-verdade para o comando lógico **AND** indicado na Tabela 9.7.

Tabela 9.7 – Operador AND

Entrada (<i>bits</i>)		Saída (<i>bit</i>)
Origem	Destino	Destino
0	0	0
0	1	0
1	0	0
1	1	1

O comando **OR** estabelece a disjunção de dois valores em nível binário: **origem** e **destino**. O resultado de saída de **destino** será verdadeiro (valor **1**) se pelo menos um dos *bits* da posição em análise for **1**; caso contrário, será considerado falso (valor **0**). Veja a tabela-verdade para o comando lógico **OR** indicado na Tabela 9.8.

Tabela 9.8 – Operador OR

Entrada (<i>bits</i>)		Saída (<i>bit</i>)
Origem	Destino	Destino
0	0	0
0	1	1
1	0	1
1	1	1

O comando **XOR** estabelece a disjunção exclusiva de dois valores em nível binário: **origem** e **destino**. O resultado de saída de **destino** será verdadeiro (valor 1) quando os *bits* da posição em análise forem diferentes. Caso os valores binários sejam iguais, o resultado de **destino** será falso (valor 0). Veja ao lado a tabela-verdade para o comando lógico **XOR**. Veja a tabela-verdade para o comando lógico **XOR** indicado na Tabela 9.9.

Tabela 9.9 – Operador XOR

Entrada (<i>bits</i>)		Saída (<i>bit</i>)
Origem	Destino	Destino
1	0	0
0	1	1
1	0	1
1	1	0

O comando **NOT** estabelece a negação do valor em nível binário. O resultado de saída de **destino** será verdadeiro (valor 1) quando o valor de destino da entrada for falso (valor 0). Será considerado falso (valor 0) quando o valor de destino da entrada for verdadeiro (valor 1). Veja a tabela-verdade para o comando lógico **NOT** indicado na Tabela 9.10.

Tabela 9.10 – Operador NOT

Entrada (<i>bits</i>)	Saída (<i>bit</i>)
Destino	Destino
0	1
1	0

A título de ilustração geral considere a Tabela 9.11 com a avaliação lógica pelos comandos **AND**, **OR**, **XOR** e **NOT** de alguns valores binários.

Tabela 9.11 – Comparação entre operadores lógicos

AL	BL	AND AL, BL	OR AL, BL	XOR AL, BL	NOT AL
		AL	AL	AL	AL
1111 1111	1111 1111	1111 1111	1111 1111	0000 0000	0000 0000
1111 0000	0000 1111	0000 0000	1111 1111	1111 1111	0000 1111
1100 0011	1010 1010	0000 0010	1110 1011	0110 1001	0011 1100
0110 0011	0000 0011	0000 0011	0110 0011	0110 0000	1001 1100

Para fazer uma verificação rápida dos resultados lógicos obtidos em relação aos valores binários de 1 *byte*, representados pelos registradores **AL** e **BL** (tabela anterior), utilize a calculadora do Windows em modo de cálculo binário e use as operações **AND**, **OR**, **XOR** e **NOT**.

Um detalhe importante a ser considerado é que os comando lógicos **AND**, **OR**, **XOR** e **NOT** devem ser utilizados de forma semelhante ao comando **CMP**. Normalmente após a definição de um desses comandos há necessidade de utilizar um comando de desvio condicional.

Para exemplificar o uso do comando **AND**, considere um programa que faça a leitura de um valor numérico positivo entre 0 e 9 de apenas um dígito, e apresente uma mensagem informando se o número é par ou ímpar. Qualquer caractere que não seja um valor entre 0 e 9 deve ser recusado. Observe o código seguinte:

```
;*****
;*      Programa: CONDIC3.ASM      *
;*****
```

```
org 100h
```

```

.DATA
msg1 DB 'Entre um valor numerico positivo (de 0 ate 9): ', 24h
msg2 DB 0Dh, 0Ah, 'Valor impar', 24h
msg3 DB 0Dh, 0Ah, 'Valor par', 24h
msg4 DB 0Dh, 0Ah, 'Caractere invalido', 24h

.CODE
LEA    DX, msg1
CALL   escreva

MOV    AH, 01h
INT    21h

CMP    AL, 30h
JL     erro

CMP    AL, 39h
JG     erro

SUB    AL, 30h
AND    AL, 01h
JPE    par
JPO    impar

par:
LEA    DX, msg3
CALL   escreva
JMP    saida

impar:
LEA    DX, msg2
CALL   escreva
JMP    saida

erro:
LEA    DX, msg4
CALL   escreva
JMP    saida

saida:
INT    20h

escreva PROC NEAR
MOV    AH, 09h
INT    21h
RET
escreva ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho <Ctrl> + <A> do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **CONDIC3**, de forma que fique semelhante à imagem da Figura 9.5.

O programa possui após a definição da diretiva **.DATA** a definição das variáveis com as mensagens que serão apresentadas. Em especial, atente para as variáveis **msg2**, **msg3** e **msg4** (linhas 09 a 11) que, além de utilizarem o código **24h** (que representa o caractere de fim de *string* \$), utilizam o código **0Dh** (que executa o efeito de retorno de carro, tecla <Enter>) e o **0Ah** (que executa o efeito de mudança de linha, *line feed*).

Quando no programa for feita a chamada dos trechos em que se encontram as variáveis **msg2**, **msg3** e **msg4**, as suas mensagens serão sempre escritas após a apresentação da mensagem identificada na variável **msg1**.

```

01 ;*****
02 ; Programa: CONDIC3.ASM
03 ;*****
04
05 org 100h
06
07 .DATA
08 msg1 DB 'Entre um valor numerico positivo (de 0 ate 9):'
09 msg2 DB 0Dh, 0Ah, 'Valor impar', 24h
10 msg3 DB 0Dh, 0Ah, 'Valor par', 24h
11 msg4 DB 0Dh, 0Ah, 'Caractere invalido', 24h
12
13 .CODE
14 LEA DX, msg1
15 CALL escreva
16
17 MOV AH, 01h
18 INT 21h
19
20 CMP AL, 30h
21 JL erro
22
23 CMP AL, 39h
24 JG erro
25
26 SUB AL, 30h
27 AND AL, 01h
28 JPE par
29 JPO impar

```

Figura 9.5 - Programa CONDIC3 na ferramenta emu8086.

O programa executa algumas verificações após a entrada do valor pelo teclado. É definido na linha 17 o código 01h para a entrada de um caractere pelo teclado que será armazenado no registrador AH, sendo essa ação controlada pela instrução INT 21h da linha 18.

Entre as linhas 20 e 21 está definido o desvio de detecção de erro, caso o valor fornecido seja menor que o valor 30h (código ASCII para a definição do valor 0d). Note que o programa por meio da instrução JL erro efetua o desvio para a linha 41 que apresenta a mensagem armazenada na variável msg4.

Entre as linhas 23 e 24 está definido outro desvio de detecção de erro, caso o valor fornecido seja maior ou igual ao valor 39h (código ASCII para o valor 9h). Note que o programa por meio da instrução JG erro efetua o desvio para a linha 41 que apresenta a mensagem armazenada na variável msg4.

O trecho da linha 20 até a linha 24 verifica a validade da entrada. Qualquer caractere que resulte um valor abaixo de 0 ou acima de 9 será considerado inválido.

O trecho mais importante do programa concentra-se nas linhas 26 até 29, pois nesse ponto é realizada a verificação lógica com o comando AND. A linha 26 efetua apenas a subtração do valor 30h do código ASCII para deixar apenas no registrador menos significativo AL o valor numérico propriamente dito.

Na linha 27 está em uso a instrução AND AL, 01h. O valor 01h equivale ao valor 00000001b. Ao ser feita a comparação com o valor armazenado AL, o comando AND altera ou não o valor do registrador de estado PF. Neste caso, o registrador de estado PF estará com o valor 1 caso o comando AND seja verdadeiro. Se o comando AND estiver com valor falso, o registrador de estado PF estará com valor 0.

A Tabela 9.12 exemplifica o que ocorre internamente no programa quando utilizados os valores 7, 8, 2 e 5.

Tabela 9.12 – Demonstração do estado de execução do programa exemplo

AND		Resultado	
AL	01h	AL	PF
0000 0111	0000 0001	0000 0001	0
0000 1000	0000 0001	0000 0000	1
0000 0010	0000 0001	0000 0000	1
0000 0101	0000 0001	0000 0001	0

Observe também o uso dos comandos de desvio condicional **JPE** (linha 28) e **JPO** (linha 29), as quais desviam a execução do programa para o trecho pertinente à validade da condição que utilizam. O comando **JPE** é executado quando o comando **AND** resultar um valor lógico verdadeiro, enquanto que o comando **JPO** é executado quando o comando **AND** resultar um valor lógico falso.

Este exemplo merece um detalhamento quanto à sua execução passo a passo. Acione a tecla de função <F5>, em seguida a tecla de função <F8> e solicite a visualização da janela **flags** (acione o comando **view/flags** na janela **CONDIC3.com**). Ajuste a distribuição das janelas na tela de seu monitor da forma mais conveniente possível. A Figura 9.6 mostra esse momento de ação (primeira etapa da tecla de função <F8>) com as janelas solicitadas exibidas de forma distribuída manualmente. Acione a tecla <F8> mais onze vezes.

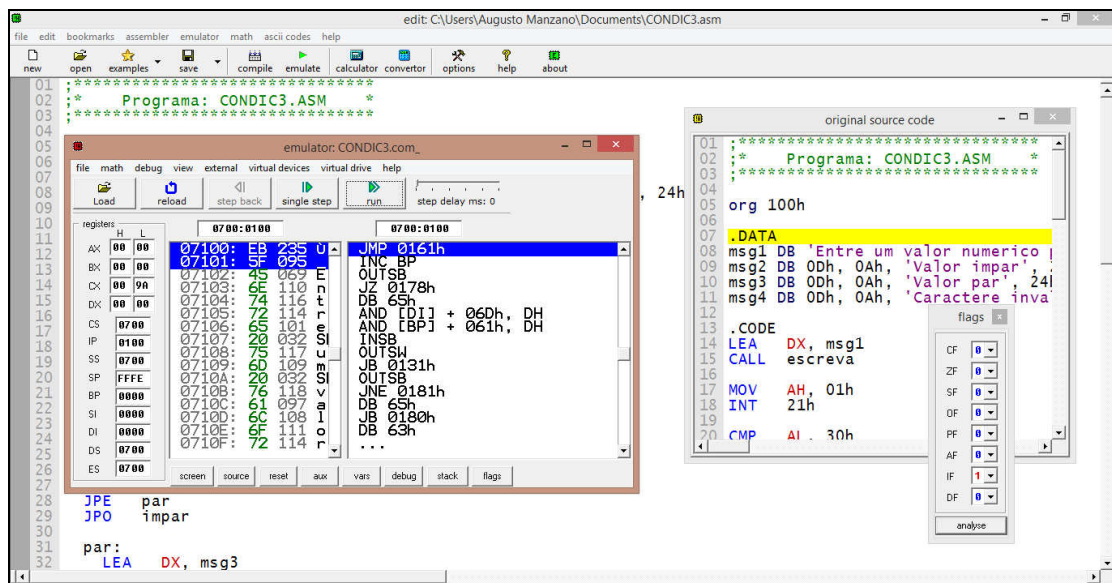


Figura 9.6 - Programa CONDIC3 em execução - início.

Para um teste de execução do programa informe o valor 8 quando a mensagem solicitando um valor numérico positivo for apresentada, acione em seguida a tecla <F8> mais seis vezes e observe a mudança da informação do registrador menos significativo **AL** com o valor **08**. Execute a partir da tela **emulator: CONDIC3.com_** o comando de menu **view/extended value viewer**. Será então mostrada a tela do recurso **extended value viewer**, como mostra a Figura 9.8.

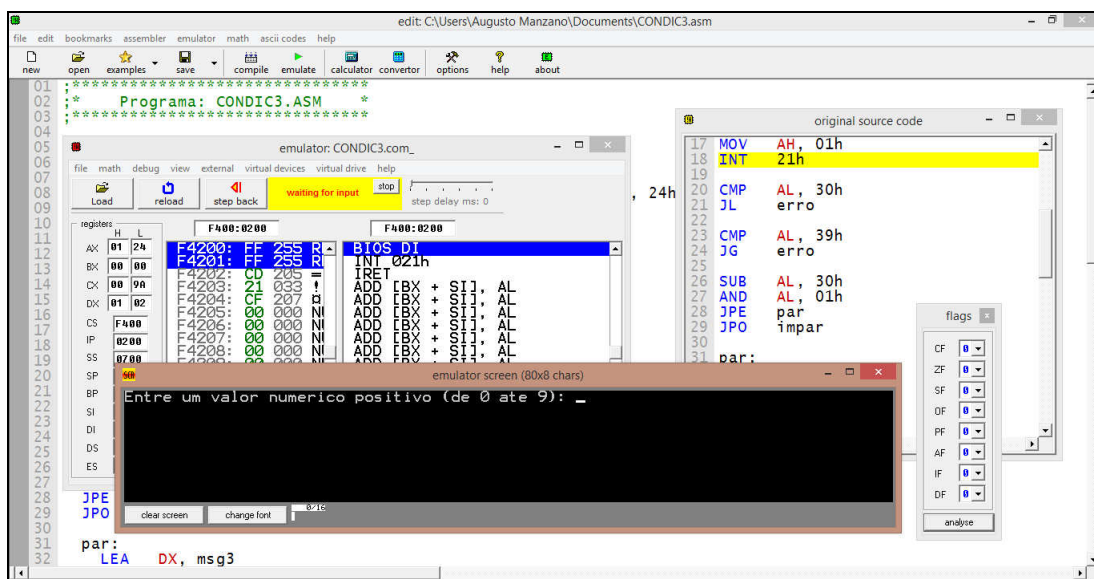


Figura 9.7 - Programa CONDIC3 em execução - entrada.

Observe as informações apresentadas na janela **extended value viewer**. O registrador menos significativo **AL** mostra o valor **08** (valor anteriormente informado). Na janela **original source code** a apresentação da barra de seleção amarela sobre a instrução **AND AL, 01h** mostra o comando que será executado no próximo passo. A Figura 9.9 apresenta a ocorrência.

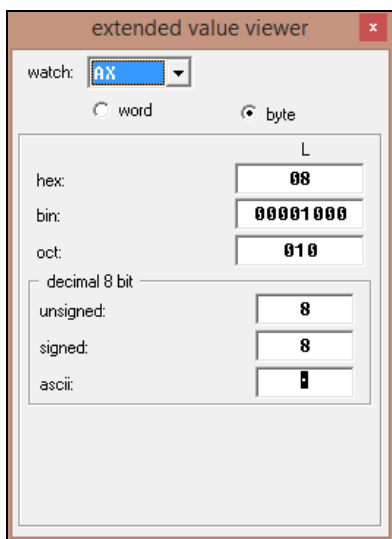


Figura 9.8 - Janela Extended value viewer.

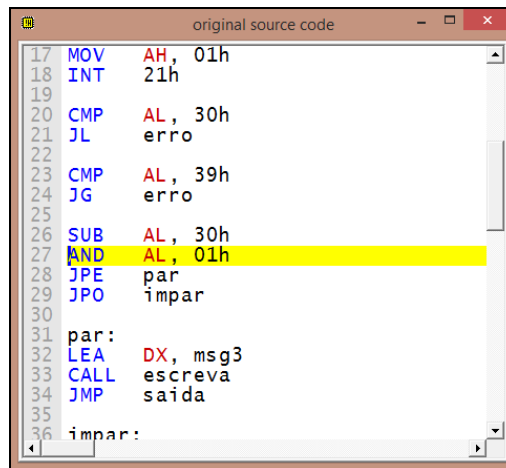


Figura 9.9 - Janela Original source code.

Acione a tecla de função **<F8>** e observe na janela **flags** a mudança do valor do registrador **PF** para **1**. Assim que a tecla de função **<F8>** é acionada ocorre a operação lógica do comando **AND** sobre o valor armazenado no registrador menos significativo **AL**, a qual altera o valor do registrador menos significativo **AL** para **0000000b**, como pode ser verificado na Figura 9.10.

Como o valor de **AL** é **0000000b**, a instrução **AND** resultou um valor verdadeiro, pois para ser verdadeiro o valor de **AL** deveria ser **0000001b**.

Na sequência do programa, acione a tecla de função **<F8>** uma vez e observe que a janela **Original source code** aponta para a instrução **JPE par**. Acione a tecla de função **<F8>** seis vezes e será apresentada a mensagem **Valor par**. Se houvesse ocorrido a entrada de um valor ímpar, a instrução **JPE par** seria saltada, e a barra de seleção amarela seria posicionada sobre a instrução **JPO ímpar** para que fosse então apresentada a mensagem **Valor ímpar**. Para concluir a execução do programa, acione a tecla **<F8>** mais cinco vezes. A Figura 9.11 apresenta a janela **emulator screen** com a mensagem de saída do programa.

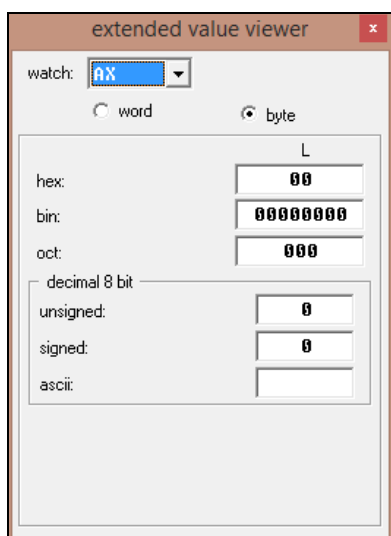


Figura 9.10 - Janela Extended value viewer com valor de AL alterado.

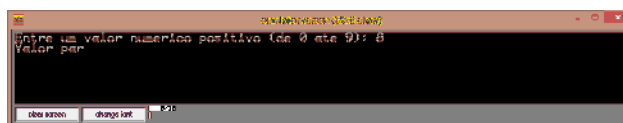


Figura 9.11 - Janela Emulator Screen.

Na caixa de diálogo **message** acione o botão **OK** e na janela **emulador: CONDIC3.com_** acione o comando de menu **file/close emulator** para retornar à tela do editor do programa.

Observação

Além da instrução **AND** utilizada anteriormente há a instrução **TEST** que pode ser usada em seu lugar. A diferença entre as duas está no fato de a instrução **TEST** não efetuar a alteração do valor no registrador de destino, mas mantém a alteração do registrador de estado **CF**.

Os comandos de operações lógicas **AND**, **OR**, **XOR** e **NOT** não devem ser confundidos com os operadores lógicos em linguagens de alto nível. Os comando lógicos no *Assembly* têm a capacidade de manipular os dados em nível de *bit*.

O programa seguinte tem a finalidade de solicitar a entrada de dois valores numéricos decimais positivos de um dígito, somar os valores e apresentar o resultado da operação como sendo um valor decimal. O programa exemplifica o uso do comando **OR**. Acompanhe o código seguinte:

```
;*****  
;*      Programa: CONDIC4.ASM      *  
;*****
```

org 100h

.DATA

```
msg1 DB 'Entre valor decimal 1 (de 0 ate 9): ', 024h  
msg2 DB 0Dh, 0Ah, 'Entre valor decimal 2 (de 0 ate 9): ', 24h  
  
msg3 DB 0Dh, 0Ah, 'Soma = ', 24h  
msg4 DB 0Dh, 0Ah, 'Caractere invalido', 24h
```

.CODE

```
LEA    DX, msg1  
CALL   escreva  
CALL   leia  
MOV     BH, AL  
LEA     DX, msg2  
CALL   escreva  
CALL   leia  
MOV     BL, AL  
  
LEA     DX, msg3  
CALL   escreva  
XCHG    AX, BX  
  
ADD     AL, AH  
SUB     AH, AH  
AAA  
MOV     DX, AX  
MOV     AH, 0Eh  
CMP     DH, 0h  
JE      nao_zero  
OR       DH, 30h  
MOV     AL, DH  
INT     10h  
nao_zero:  
OR       DL, 30h  
MOV     AL, DL  
INT     10h  
INT     20h
```

```

escreva PROC NEAR
    MOV     AH, 09h
    INT     21h
    RET
escreva ENDP

```

```

leia PROC NEAR
    MOV     AH, 01h
    INT     21h
    CMP     AL, 30h
    JL      erro
    CMP     AL, 3Ah
    JGE     erro
    SUB     AL, 30h
    RET
erro:
    LEA     DX, msg4
    CALL    escreva
    INT     20h
    RET
leia ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **CONDIC4**, de forma que fique semelhante à imagem da Figura 9.12.

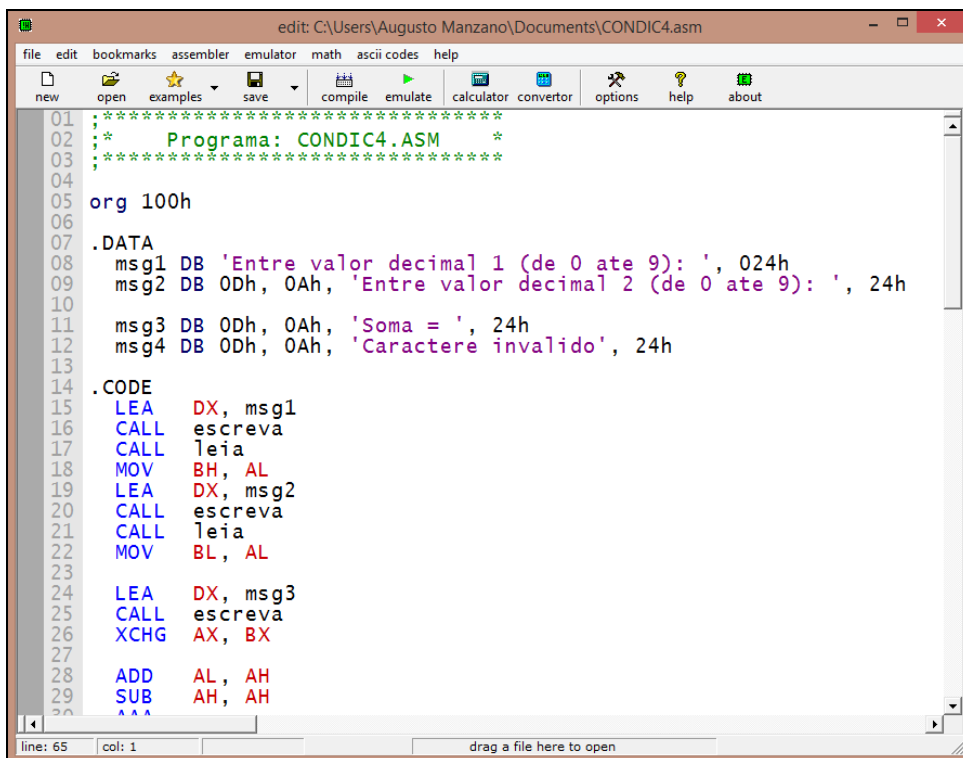


Figura 9.12 - Programa CONDIC4 na ferramenta emu8086.

Apesar do programa **CONDIC4** fazer uso de recursos já apresentados e conhecidos, acrescenta-se os comandos **XCHG**, **AAA** e a instrução **INT 10**.

Primeiramente atente para a definição dos procedimentos **escreva** (idêntico ao programa **CONDIC3**) existente entre as linhas **44** e **48** e o procedimento **leia** (com alterações em relação ao programa **CONDIC3**) existente entre as linhas **50** e **64**. O procedimento **leia** é responsável pela recepção dos valores fornecidos no teclado.

O trecho de programa entre as linhas **14** e **17** apresenta a primeira mensagem solicitando o primeiro valor (linhas **14** e **15**), faz a entrada do valor (linha **16**) e com a linha **17** faz a movimentação do conteúdo do registrador **AL** (primeiro valor informado) para o registrador **BH** no sentido de esse valor ficar temporariamente armazenado para o cálculo e liberar o registrador **AL** para a nova entrada.

O trecho de programa entre as linhas **19** e **22** apresenta a segunda mensagem solicitando o segundo valor (linha **21**) e com a linha **22** faz a movimentação do conteúdo do registrador **AL** (segundo valor informado), para o registrador **BL** no sentido de esse valor ficar temporariamente armazenado para o cálculo.

A partir desse ponto o registrador **BX** conterá armazenados os dois valores fornecidos. O primeiro valor está armazenado na parte mais significativa (**BH**) e o segundo valor armazenado na parte menos significativa (**BL**).

Em seguida o trecho de linhas entre **24** e **26** mostra a terceira mensagem para a apresentação do resultado do cálculo processado. Na linha **26** há a definição de uso do comando **XCHG** (*exchange*) que faz a troca dos conteúdos entre os registradores **AX** e **BX**. Essa troca é necessária uma vez que a operação de adição entre os valores será feita com as partes mais (**AH**) e menos (**AL**) significativas do registrador **AH**. Desta forma, os valores reservados no registrador **BX** são posicionados no registrador **AX**.

O comando **XCHG** não afeta nenhum dos registradores de estado (*flags*), exigindo que ambos os operandos sejam de mesmo tamanho e não estejam relacionados à memória. Essa instrução não opera com o registrador **IP**, bem como com os registradores de segmento.

Na sequência o trecho de programa da linha **28** até a linha **42** é o mais importante, pois é responsável pela apresentação do valor decimal da resposta da operação de adição dos valores fornecidos.

A linha **28** efetua a soma dos dois valores e armazena o resultado dessa operação na parte menos significativa do registrador **AX**, ou seja, no registrador **AL**. Depois com a linha **29** o programa limpa o registrador **AH**.

Na linha **30** encontra-se o comando **AAA** (*Ascii Adjust for Addition*) que tem por finalidade alterar o valor do registrador **AL**, tornando-o um valor decimal válido. O comando **AAA** trabalha com o ajuste de valor baseando-se no uso de valores em BCD.

A instrução **INT 10** tem por finalidade efetuar uma chamada a interrupção da BIOS que trata o modo de acesso ao monitor de vídeo. Este comando é usado após a definição de uma função de operação junto ao registrador **AL**. No programa a instrução **MOV AL, DH** faz com que o valor definido no registrador **DH** seja passado ao registrador **AL** no sentido de apresentar o resultado da soma quando este resultado for maior ou igual a 10 (limitado a 15).

A linha **31** faz a transferência do valor armazenado no registrador geral **AX** para o registrador geral **DX**. O registrador **AX** fica livre para ser usado nas operações de apresentação em tela dos caracteres armazenados na memória.

O programa na linha **32** usa o valor **0Eh** que é um código de função utilizado para a apresentação de dados na tela do monitor de vídeo por meio da interrupção **010h** (**INT 010h** existente nas linhas **37** e **41**) que é responsável pelo acesso aos recursos da placa de vídeo conectada no computador.

Esse recurso substitui o uso da interrupção **21h**. O comportamento da interrupção **10h** é na maioria das vezes melhor que o comportamento da interrupção **21h**, pois é mais rápido.

Nas linhas **33** e **34** existe uma comparação verificando se o valor do registrador **DL** é zero, e se for, o programa é desviado para a linha **38**. Esse recurso evita que numa operação de adição de unidades seja apresentada à frente da unidade o valor **0**.

As linhas **35** e **39** utilizam a instrução **OR** para subtrair, respectivamente, dos valores existentes nos registradores **DH** e **DL** o valor **30h**. Desta forma converte-se o valor existente no seu código ASCII correspondente e para fazer a apresentação dos caracteres que formam o valor decimal, são executadas as ações das linhas **36:37** e **40:41**.

9.5 - Repetições

Outra estrutura de programação muito utilizada e importante são os laços que proporcionam a repetição de certos trechos de um programa (também conhecidos como *ciclos*, *loopings* ou *malhas de repetição*) cuja característica operacional é a capacidade de executar um trecho de programa por determinado número de vezes. Normalmente um laço é estabelecido em *Assembly com os comandos* **LOOP**, **LOOPE**, **LOOPNE**, **LOOPNZ** e **LOOPZ**. O comando **LOOP** já fora usada anteriormente.

A Tabela 9.13 exibe as instruções de laços de repetição existentes na linguagem de programação *Assembly 8086/8088*.

Tabela 9.13 – Instruções para execução de laços

Instrução	Significado	Descrição
LOOP	loop	laço iterativo
LOOPE	loop while equal	enquanto laço igual a
LOOPNE	loop while not equal	enquanto laço não for igual a
LOOPNZ	loop while not zero	enquanto laço não for zero
LOOPZ	loop while zero	enquanto laço for zero

O laço baseado no comando **LOOP** é do tipo *iterativo*, ou seja, executa a ação do laço de repetição um determinado número de vezes, semelhante ao laço de repetição **FOR** encontrado em linguagens de alto nível.

Os laços **LOOPE**, **LOOPZ**, **LOOPNE** e **LOOPNZ** são do tipo *iterativo*, ou seja, são laços condicionais, pois para operarem dependem do valor sinalizado no registrador de estado **ZF**. Essa forma é semelhante ao laço de repetição **WHILE** encontrado em linguagens de alto nível.

Os comandos **LOOPE** e **LOOPZ** são executados enquanto o registrador de estado **ZF** for igual a 1 (um) e o registrador geral **CX** for diferente de 0 (zero). Já os comandos **LOOPNE** e **LOOPNZ** são executados enquanto o registrador de estado **ZF** for igual a 0 (zero) e o registrador geral **CX** for diferente de 0 (zero).

Os comandos **LOOPE** / **LOOPZ** e **LOOPNE** / **LOOPNZ** são escritos de forma diferente, mas possuem a mesma estrutura operacional. Não é necessário apresentar em separado um exemplo de cada comando de laço. Os próximos programas contemplam as instruções **LOOP**, **LOOPE** e **LOOPNE**.

Observação

É bom lembrar que as instruções de laço (sejam elas quais forem) usam o valor que estiver armazenado no registrador geral **CX** para a operação do contador de passos. O valor é sempre decrementado do registrador geral **CX**.

Para exemplificar operações com laços, considere um programa que apresenta cinco vezes na tela do monitor de vídeo a mensagem **Alô Mundo!**, conforme indicado a seguir:

```
;*****  
;*      Programa: LACO1.ASM      *  
;*****
```

```
org 100h
```

```
.DATA
```

```
msg DB 'Alô Mundo!', 13d, 12o, 24h
```

```
.CODE
```

```
LEA DX, msg
```

```
MOV CX, 5d
```

```
MOV AH, 09h
```

```
laco:
```

```
INT 21h
```

```
LOOP laco
```

```
INT 20h
```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO1**, de forma que fique semelhante à imagem da Figura 9.13.

```

edit: C:\Users\Augusto Manzano\Documents\LACO1.asm
file  edit  bookmarks  assembler  emulator  math  ascii codes  help
new  open  examples  save  compile  emulate  calculator  convertor  options  help  about

01  ;*****
02  ;*      Programa: LACO1.ASM      *
03  ;*****
04
05  org 100h
06
07  .DATA
08  msg DB 'Alo Mundo!', 13d, 12o, 24h
09
10  .CODE
11  LEA  DX, msg
12  MOV  CX, 5d
13  MOV  AH, 09h
14  laco:
15      INT 21h
16      LOOP laco
17  INT 20h
18  |

```

Figura 9.13 - Programa LACO1 na ferramenta emu8086

Observe na linha **08** o uso dos valores **13d** (equivalente a **0Dh**), **12o** (equivalente a **0Ah**) e **24h** após a definição da mensagem **Alô Mundo!**, que são responsáveis pela definição dos códigos de controle para apresentação de um *string*.

A definição na linha **12** da instrução **MOV CX, 5d** estabelece para o registrador geral **CX** o valor decimal **5** (valor que será usado para a efetivação da contagem dos laços de repetição), que será automaticamente decrementado em **1** toda vez que o comando **LOOP** (linha **16**) for executado.

As demais linhas dos programas possuem recursos já conhecidos e que foram explanados em exemplos anteriores.

Considere em seguida um segundo exemplo de programa que leia um valor numérico positivo de um dígito entre os valores **0** e **8** e apresente como resultado o valor da fatorial do valor fornecido. Valores iguais ou superiores a **9** não são aceitos por gerarem um resultado inteiro acima da capacidade numérica de trabalho dos registradores gerais (considerando o uso de um microprocessador padrão **16 bits**, sendo a solução para esta questão algo que foge do escopo deste trabalho). Observe detalhadamente o código a seguir:

```

;*****
;*      Programa: LACO2.ASM      *
;*****

org 100h

.DATA
msg1 DB 'Entre valor decimal positivo (de 0 ate 8): ', 24h
msg2 DB 0Dh, 0Ah, 'Fatorial de ', 24h
msg3 DB ' equivale a ', 24h
msg4 DB 0Dh, 0Ah, 'Valor invalido', 24h

.CODE
LEA  DX, msg1
CALL mensagem
CALL entrada
PUSH AX

```

```

    LEA    DX, msg2
    CALL   mensagem
    POP    AX
    MOV    DL, AL
    MOV    AH, 0Eh
    INT    10h
    SUB    AL, 30h
    MOV    CL, AL
    LEA    DX, msg3
    CALL   mensagem
    CALL   fatorial
    CALL   valor

fim:
    INT    20h

mensagem PROC NEAR
    MOV    AH, 09h
    INT    021h
    RET
mensagem ENDP

entrada PROC NEAR
    MOV    AH, 01h
    INT    021h
    CMP    AL, 030h
    JL     erro
    CMP    AL, 039h
    JGE    erro
    JMP     fim_validacao
erro:
    LEA    DX, msg4
    CALL   mensagem
    JMP     fim
fim_validacao:
    RET
entrada ENDP

fatorial PROC NEAR
    MOV    AX, 01h
    CMP    CX, 0h
    JE     fim_laco
repita1:
    MUL    CX
    LOOPNE repita1
fim_laco:
    RET
fatorial ENDP

valor PROC NEAR
    MOV    BX, 0Ah
    SUB    CX, CX
repita2:
    SUB    DX, DX
    DIV    BX
    PUSH   DX
    INC    CX
    CMP    AX, 0h
    JNZ    repita2
saida:
    POP    AX

```



```

ADD     AL, 30h
MOV     DL, AL
MOV     AH, 0Eh
INT     10h
DEC     CX
JNBE    saida
POP     DX
RET
valor ENDP

```

```

01  ;*****
02  ; Programa: LACO2.ASM
03  ;*****
04
05  org 100h
06
07  .DATA
08  msg1 DB 'Entre valor decimal positivo (de 0 ate 8): ', 24h
09  msg2 DB 0Dh, 0Ah, 'Fatorial de ', 24h
10  msg3 DB 'equivale a ', 24h
11  msg4 DB 0Dh, 0Ah, 'Valor invalido', 24h
12
13  .CODE
14  LEA   DX, msg1
15  CALL  mensagem
16  CALL  entrada
17  PUSH  AX
18
19  LEA   DX, msg2
20  CALL  mensagem
21  POP   AX
22  MOV   DL, AL
23  MOV   AH, 0Eh
24  INT   10h
25  SUB   AL, 30h
26  MOV   CL, AL
27  LEA   DX, msg3
28  CALL  mensagem
29  CALL  fatorial
30

```

Figura 9.14 - Programa LACO2 na ferramenta emu8086 - principal.

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO2**, de forma que fique semelhante às imagens das Figuras 9.14 (trecho do programa principal), 9.15 e 9.16 (trecho das sub-rotinas do programa).

No trecho de programa principal representado na Figura 9.14 encontram-se as definições da área de dados e da área de código, como de costume utilizadas até o momento.

O trecho de código de programa entre as linhas 14 e 17 apresenta a mensagem de entrada definida para a variável **msg1** pela chamada do procedimento **mensagem** (**CALL mensagem** na linha 15) e do procedimento **entrada** (**CALL entrada** na linha 16). A linha 17 exibe o armazenamento na pilha do valor atual do registrador geral **AX** que é o valor fornecido no teclado pela instrução **PUSH AX**.

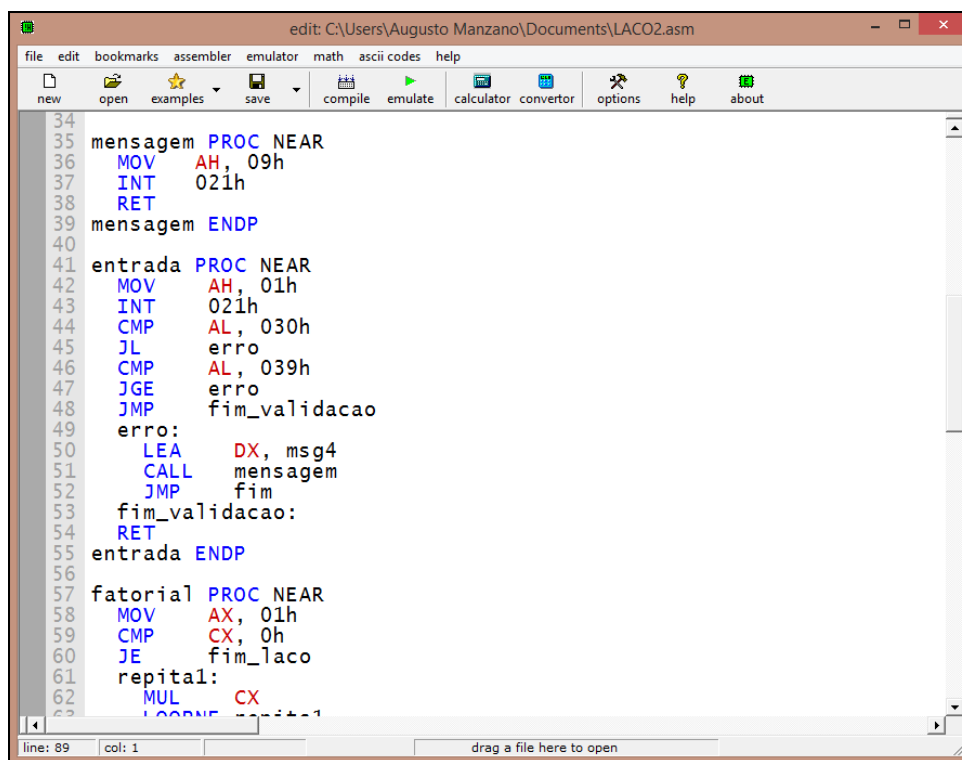
O armazenamento do valor do registrador **AX** na pilha é necessário porque as próximas operações realizadas afetam o valor desse registrador. É necessário preservar esse valor para posterior resgate pelo comando **POP AX** definida na linha 21. Na sequência são encontradas as linhas 19 e 27 que apresentam as mensagens definidas para as variáveis **msg2** e **msg3** e a apresentação na tela do valor informado para que em seguida ocorra o cálculo do resultado da fatorial e a apresentação do resultado calculado.

Em especial observe o código da linha 21 (instrução **POP AX**) que resgata da pilha o valor de entrada anteriormente preservado (quando da execução da linha 17). Em seguida (linha 22) é feita a movimentação do valor (trazido por **POP AX** na linha 21) do registrador **AL** para o registrador **DL** (nesse registrador está sendo colocado o valor que será apresentado). Depois o programa faz a apresentação do valor pelas linhas de programa 23 e 24. Na sequência a linha 25

faz a conversão do valor do código ASCII do registrador **AL** no seu valor numérico decimal correspondente e a linha **26** movimenta o valor agora decimal do registrador **AL** para o registrador **CL**.

O trecho de código situado entre as linhas **28** e **30** faz a apresentação do resultado da fatorial pelas chamadas dos procedimentos **fatorial** (**CALL fatorial** definido na linha **29**) e **valor** (**CALL valor** definido na linha **30**). Após a conclusão da operação o programa é finalizado com a instrução **INT 20h** da linha **33**.

No trecho de programa representado na Figura 9.15 encontram-se as definições dos procedimentos **mensagem** (que apresenta as mensagens definidas para as variáveis **msg1**, **msg2**, **msg3** e **msg4**), **entrada** (que captura no teclado o valor numérico de um dígito) e **fatorial** (que calcula e armazena na memória o valor da fatorial).



```
34 mensagem PROC NEAR
35 MOV AH, 09h
36 INT 021h
37 RET
38 mensagem ENDP
39
40
41 entrada PROC NEAR
42 MOV AH, 01h
43 INT 021h
44 CMP AL, 030h
45 JL erro
46 CMP AL, 039h
47 JGE erro
48 JMP fim_validacao
49 erro:
50 LEA DX, msg4
51 CALL mensagem
52 JMP fim
53 fim_validacao:
54 RET
55 entrada ENDP
56
57 fatorial PROC NEAR
58 MOV AX, 01h
59 CMP CX, 0h
60 JE fim_laco
61 repital:
62 MUL CX
63 LOOPNE repital
```

Figura 9.15 - Programa LACO2 na ferramenta emu8086 - procedimentos.

Os procedimentos **mensagem** e **entrada** de certa forma já são conhecidos e não requerem explicações. Atente para o procedimento **fatorial** situado entre as linhas **57** e **66**.

Na linha **58** é feita a movimentação do valor **01h** para o registrador geral **AX**, sendo este o menor resultado do cálculo de um fatorial válido para a entrada dos valores **0** (zero) e **1** (um). As instruções **CMP CX, 0h** (linha **59**) e **JE fim_laco** (linha **60**) verificam se o valor do registrador geral **CX** é zero, e se for, desvia para o trecho de programa identificado como **fim_laco** definido após a linha **64**. Esse trecho será executado quando o valor do registrador **CX** for zero; caso contrário, serão executadas as linhas de código **MUL CX** (linha **62**) e **LOOPNE repital** (linha **63**) que calculam o resultado da fatorial.

Imagine que o valor informado para a obtenção do cálculo da fatorial seja **5**. Nesse caso o procedimento (sub-rotina) **fatorial** realiza as seguintes operações:

- ◆ Lembre-se de que nesse momento, na execução do programa (linha **59**) o registrador **CL** está com o valor **0005h** recebido do registrador **AL** pela execução da linha de código **26**. Neste caso, a linha de código **58** faz a movimentação do valor **0001h** para o registrador geral **AX** e as linhas **59** e **60** verificam se o valor do registrador geral **CX** é zero. Se for, o procedimento é encerrado e se não for (que é o caso descrito), o programa será desviado para a linha **61**.
- ◆ A linha de código **62** multiplica o valor do registrador geral **CX** (valor **0005h**) pelo valor do registrador geral **AX** (valor **0001h**) e armazena o resultado da operação no par de registradores gerais **DX:AX**. Nesse momento o valor do registrador geral **AX** que é **0001h** é multiplicado pelo valor do registrador geral **CX** e passa a conter o valor **0005h**. O registrador geral **DX** possui neste momento o valor **0h**.

- ◆ Após o processamento da linha **62**, o programa, por meio da linha **63** (**LOOPNE repita1**), faz um retorno para a linha **61** enquanto o valor do registrador **CX** não for igual a zero.
- ◆ Ao retornar para a linha **62**, o registrador geral **CX** estará com o valor **0004h**, pois toda vez que uma instrução de laço é executada, o registrador geral **CX** é diminuído em **1h**.
- ◆ De volta à linha **62** ocorre nova multiplicação do valor atual do registrador geral **CX** (possui nesse momento **0004h**) pelo valor atual do registrador geral **AX** (possui nesse momento **0005h**). O registrador geral **AX** passa a ter o valor **0014h** (**20** em decimal).
- ◆ Na execução da linha **63** ocorre novo retorno para a linha **61** e na linha **62** por ser o valor do registrador geral **CX** diferente de **0000h**. O registrador geral **CX** estará com o valor **0003h**.
- ◆ De volta à linha **62** ocorre nova multiplicação do valor atual do registrador geral **CX** (possui nesse momento **03h**) pelo valor atual do registrador geral **AX** (possui nesse momento **0014h**). O registrador geral **AX** passa a ter valor **003Ch** (**60** em decimal).
- ◆ Na execução da linha **63** ocorre novo retorno para a linha **61** e na linha **62** por ser o valor do registrador geral **CX** diferente de **0000h**. O registrador geral **CX** estará com o valor **0002h**.
- ◆ De volta à linha **61** ocorre nova multiplicação na linha **61** do valor atual do registrador geral **CX** (possui nesse momento **0002h**) pelo valor atual do registrador geral **AX** (possui nesse momento **003Ch**). O registrador geral **AX** passa a ter o valor **0078h** (**120** em decimal).
- ◆ Na execução da linha **63** ocorre novo retorno para a linha **61** e na linha **62** por ser o valor do registrador geral **CX** diferente de **0000h**. O registrador geral **CX** estará com o valor **0001h**.
- ◆ De volta à linha **61** ocorre nova multiplicação do valor atual do registrador geral **CX** (possui nesse momento **0001h**) na linha **62** pelo valor atual do registrador geral **AX** (possui nesse momento **0078h**). O registrador geral **AX** passa a ter o valor **0078h** (**120** em decimal).
- ◆ Na execução da linha **63** ocorre novo retorno para a linha **61**. Neste momento o registrador geral **CX** estará com o valor **0000h**. O programa é transferido então para a linha **65** que encerra o procedimento **fatorial**.

No trecho de programa representado na Figura 9.16 encontra-se a definição do procedimento **valor** que apresenta o resultado da fatorial em notação decimal. Nessa etapa o trecho de programa situado na faixa da linha **72** até a linha **77** armazena o valor decimal na pilha (isso sempre ocorre de forma invertida) e o trecho das linhas **79** a **86** retira os valores da pilha, apresentando-os na forma decimal. Esse procedimento executa as seguintes ações:

- ◆ Na linha **69** é carregado o registrador geral **BX** com o valor **0Ah** (**10** em decimal). Esse registrador será usado para auxiliar o processo de conversão do valor em notação decimal.
- ◆ Na linha **70** o registrador geral **CX** é zerado e o mesmo ocorre com o registrador geral **DX** na linha **72**.
- ◆ Na linha **73** ocorre a divisão do valor atual do registrador geral **AX** (valor **0078h**) pelo valor atual do registrador geral **BX** (valor **0Ah**). O registrador geral **AX** passa a ter o valor do quociente da divisão **000Ch** e o registrador geral **DX** passa a possuir o valor do resto da divisão **0h**.
- ◆ Na linha **74** ocorre o armazenamento na pilha do valor armazenado no registrador geral **DX**.
- ◆ Na linha **75** ocorre um incremento de **0001h** no registrador geral **CX**. Este passa então a ter o valor **0001h**.
- ◆ Na linha **76** ocorre a comparação do valor atual do registrador **AX** (valor **000Ch**) com o valor **0h**. O valor do registrador geral **AX** foi diminuído, pois é o valor do quociente da divisão anterior.
- ◆ Na linha **77** ocorre o desvio para a linha de código **79** caso o valor do registrador geral **AX** não seja zero. Caso contrário o programa volta para a linha **71**.
- ◆ No retorno para a linha **71**, a linha **72** faz a limpeza do registrador geral **DX**. Na sequência (linha **73**) o programa efetua nova divisão do valor atual do registrador geral **AX** (**000Ch**) pelo valor do registrador geral **BX** (**0Ah**). Nesse momento o registrador geral **AX** passa a ter o valor do quociente da divisão **0001h** e o registrador geral **DX** passa a ter o valor do resto da divisão **0002h**.
- ◆ Quando se executa a linha **74**, o valor do registrador geral **DX** é armazenado na pilha sobre o seu valor anterior, ou seja, nesse momento a pilha possui os valores **0000h** e **0002h**.
- ◆ As linhas **75**, **76** e **77** efetuam, respectivamente, o ajuste do valor do registrador geral **CX** para **0002h** e transferem a execução do programa novamente para a linha **71**.

```

67 valor PROC NEAR
68 MOV BX, 0Ah
69 SUB CX, CX
71 repita2:
72 SUB DX, DX
73 DIV BX
74 PUSH DX
75 INC CX
76 CMP AX, 0h
77 JNZ repita2
78 saida:
79 POP AX
80 ADD AL, 30h
81 MOV DL, AL
82 MOV AH, 0Eh
83 INT 10h
84 DEC CX
85 JNBE saida
86 POP DX
87 RET
88 valor ENDP
89 |

```

Figura 9.16 - Programa LACO1 na ferramenta emu8086 - procedimentos.

- ◆ De volta à linha 71 o programa limpa o registrador geral DX. Na sequência (linha 72) o programa efetua nova divisão do valor atual do registrador geral AX (0001h) pelo valor do registrador geral BX (000Ah). Nesse momento o registrador geral AX passa a ter o valor do quociente da divisão 0000h e o registrador geral DX passa a possuir o valor do resto da divisão 0001h.
- ◆ Quando se executa a linha 74, o valor do registrador geral DX é armazenado na pilha sobre o seu valor anterior, ou seja, a pilha possui os valores 0000h, 0002h e 0001h. Nesse momento a pilha possui de forma invertida os valores que formarão a imagem decimal do valor 120d.
- ◆ As linhas 75, 76 e 77 ajustam, respectivamente, o valor do registrador geral CX para 0003h e transferem a execução do programa para a linha 79, uma vez que o registrador geral AX passou a ter o valor 0h.
- ◆ Na linha 79 ocorre a retirada da pilha do valor armazenado para o registrador geral AX. Depois na linha 80 ocorre a transformação do valor para o seu código ASCII equivalente.
- ◆ Na linha 81 ocorre a movimentação do valor do registrador menos significativo AL para o registrador menos significativo DL que armazena o código a ser apresentado na tela do monitor de vídeo. Em seguida com as linhas 82 e 83 ocorre a apresentação do primeiro valor puxado da pilha (neste caso o valor 1).
- ◆ Na linha 84 há a subtração (decremento) de 01h do registrador geral CX (que está com o valor 0003h e passa a possuir o valor 0002h). Na sequência a linha 85 verifica, por meio da instrução JNBE saida, se o registrador geral CX está com o seu valor abaixo ou igual a zero. Caso não esteja, retorna o fluxo de execução do programa para a linha 78 que envia o processamento para a linha 79.
- ◆ A partir desse retorno para a linha 79 o programa retira o valor 2 da pilha. Em seguida da linha 80 até a linha 83 é feita a apresentação do valor retirado da pilha. A linha 84 ajusta o valor do registrador geral CX de 0002h para 0001h e executa o novo retorno à linha 78 que envia o processamento para a linha 79.
- ◆ Nessa última etapa de execução da linha 79 o programa retira o valor 1 da pilha. Em seguida entre as linhas 80 e 83 é feita a apresentação do valor retirado da pilha. A linha 84 ajusta o valor do registrador geral CX de 0001h para 0000h. Isso fará com que o programa seja desviado para a linha de código 86 que remove da pilha o valor 0 (último valor a ser retirado, que foi o primeiro a ser inserido).

A partir daí o programa é encerrado e o valor do cálculo do resultado da fatorial é apresentado.

9.6 - Utilização de macros

Em vários exemplos de programas anteriores foi empregada a programação estruturada baseada em **procedimentos**.

Os **procedimentos** utilizados na linguagem de programação Assembly 8086/8088 são similares às sub-rotinas usadas em algumas linguagens de alto nível.

Cabe apresentar ainda uma segunda forma de trabalhar com a programação estruturada. Nesse caso utilizando **macros**, que têm certa similaridade com funções em algumas linguagens de alto nível. Uma rotina de **macro** pode ser definida com e sem parâmetros. Ela tem a seguinte estrutura de sintaxe:

```
nome MACRO [parâmetro1, parâmetro2, ...]
      corpo da macro
ENDM
```

Uma rotina de macro deve ter um nome definido antes da diretiva **MACRO** e entre as diretivas **MACRO** e **ENDM** deve ser colocado o código a ser executado. A definição de parâmetros é opcional.

Basicamente tudo que é definido com procedimento também pode ser definido com macro. A diferença está no fato de que um procedimento é chamado com o comando **CALL** e uma rotina de macro é chamada como se fosse um comando da própria linguagem. Outra diferença entre esses dois mecanismos é que uma rotina de macro utiliza mais memória. Por essa razão deve ser usada com parcimônia.

Com base no código do programa **LACO2** serão feitas algumas mudanças para transformar o procedimento **mensagem** na macro **msg**. Observe a seguir os pontos marcados em negrito no código do programa:

```
;*****
;*      Programa: LACO3.ASM      *
;*****

org 100h

.DATA
msg1 DB 'Entre valor decimal positivo (de 0 ate 8): ', 24h
msg2 DB 0Dh, 0Ah, 'Fatorial de ', 24h
msg3 DB ' equivale a ', 24h
msg4 DB 0Dh, 0Ah, 'Valor invalido', 24h

.CODE
LEA    DX, msg1
msg
CALL  entrada
PUSH  AX

LEA    DX, msg2
msg
POP    AX
MOV    DL, AL
MOV    AH, 0Eh
INT    10h
SUB    AL, 30h
MOV    CL, AL

LEA    DX, msg3
msg
CALL  fatorial
CALL  valor

fim:
INT    20h
```

```

msg MACRO
    MOV    AH, 09h
    INT    21h
ENDM

entrada PROC NEAR
    MOV    AH, 01h
    INT    21h
    CMP    AL, 30h
    JL     erro
    CMP    AL, 39h
    JGE    erro
    JMP     fim_validacao
erro:
    LEA     DX, msg4
    msg
    JMP     fim
fim_validacao:
    RET
entrada ENDP

fatorial PROC NEAR
    MOV    AX, 01h
    CMP    CX, 0h
    JE     fim_laco
repita1:
    MUL    CX
    LOOPNE repita1
fim_laco:
    RET
fatorial ENDP

valor PROC NEAR
    PUSH    AX
    MOV     BX, 0Ah
    SUB     CX, CX
repita2:
    SUB     DX, DX
    DIV     BX
    PUSH    DX
    INC     CX
    CMP     AX, 0h
    JNZ     repita2
saida:
    POP     AX
    ADD     AL, 30h
    MOV     DL, AL
    MOV     AH, 0Eh
    INT     10h
    DEC     CX
    JNBE    saida
    POP     DX
    RET
valor ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO3**.

Nos pontos em que existia a instrução **CALL mensagem** agora existe apenas a chamada da rotina de *macro msg*. Atente também para a mudança do código da rotina de procedimento **mensagem** para o código da rotina de macro **msg** exemplificado a seguir:

```
msg MACRO
    MOV     AH, 09h
    INT     21h
ENDM
```

A rotina de macro é de certa forma semelhante à de um procedimento. Houve a supressão do comando **RET**, pois macros não a utilizam porque fazem o retorno automaticamente.

O exemplo anterior apresentou a rotina de *macro* sem parâmetro. No entanto, o uso de parâmetro pode ser muito vantajoso. Tome por base o código seguinte, observando as linhas grafadas em negrito:

```
;*****
;*      Programa: LAC04.ASM      *
;*****

org 100h

.DATA
msg1 DB 'Entre valor decimal positivo (de 0 ate 8): ', 24h
msg2 DB 0Dh, 0Ah, 'Fatorial de ', 24h
msg3 DB ' equivale a ', 24h
msg4 DB 0Dh, 0Ah, 'Valor invalido', 24h

.CODE
msg    msg1
CALL  entrada
PUSH  AX

msg    msg2
POP   AX
MOV   DL, AL
MOV   AH, 0Eh
INT   010h
SUB   AL, 30h
MOV   CL, AL

msg    msg3
CALL  fatorial
CALL  valor

fim:
    INT     20h

msg MACRO mensagem
    LEA     DX, mensagem
    MOV     AH, 09h
    INT     21h
ENDM

entrada PROC NEAR
    MOV     AH, 01h
    INT     021h
    CMP     AL, 30h
    JL      erro
    CMP     AL, 39h
    JGE     erro
    JMP     fim_validacao
erro:
    msg     msg4
    JMP     fim
```

```

    fim_validacao:
    RET
entrada ENDP

fatorial PROC NEAR
    MOV     AX, 01h
    CMP     CX, 0h
    JE      fim_laco
    repita1:
        MUL     CX
        LOOPNE repita1
    fim_laco:
    RET
fatorial ENDP

valor PROC NEAR
    PUSH    AX
    MOV     BX, 0Ah
    SUB     CX, CX
    repita2:
        SUB     DX, DX
        DIV     BX
        PUSH    DX
        INC     CX
        CMP     AX, 0h
        JNZ     repita2
    saida:
        POP     AX
        ADD     AL, 30h
        MOV     DL, AL
        MOV     AH, 0Eh
        INT     010h
        DEC     CX
        JNBE    saida
        POP     DX
    RET
valor ENDP

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho <Ctrl> + <A> do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO4**.

O código da rotina de *macro msg* tem agora a definição de um parâmetro denominado **mensagem** que receberá o conteúdo da mensagem a ser apresentada.

0D82:0100 B402	MOV	AH, 02
0D82:0102 B241	MOV	DL, 41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 10

DETALHES COMPLEMENTARES

Este capítulo mostra alguns detalhes técnicos ainda não comentados, como, por exemplo, endereçamento de memória pela manipulação dos registradores de segmento e registradores de deslocamento, programas executáveis, bibliotecas externas, manipulações básicas (teclado e cursor) e valores negativos.

10.1 - Mais sobre segmentos e deslocamentos

No capítulo 2 o tema sobre segmentos e deslocamentos foram comentados superficialmente para situar o leitor nos parâmetros básicos do funcionamento da linguagem de programação de computadores *Assembly 8086/8088*. Nos demais capítulos o assunto não foi aprofundado. É propício, neste momento, relembrar alguns pontos apresentados e introduzir novos detalhes técnicos ainda desconhecidos:

- ◆ O microprocessador Intel 8086/8088¹ foi concebido com a capacidade de endereçar fisicamente no máximo uma memória de até 1 MB, ou seja, 2²⁰ (1.048.576), ou seja, de **00000h** até **FFFFFFh**.
- ◆ A memória de 1 MB é dividida em 16 blocos (segmentos de memória) de 64 KB endereçados da posição **0h** (primeiro segmento de memória) até a posição **Fh** (décimo sexto segmento de memória). Cada segmento de memória trabalha com até 64 KBytes (65.536) de dados, ou seja, cada segmento está dividido em deslocamentos (*offsets*) que variam de **0000h** até **FFFFh**. Logicamente a memória está organizada em quatro segmentos de trabalho, sendo: *code*, *data*, *stack* e *extra*.
- ◆ Cada deslocamento permite manipular um dado de até 16 *bits*, ou seja, um dado do tipo *word* (manipulado pelos registradores de segmento).
- ◆ Cada *word* (posição de memória) é representado por meio do endereço cartesiano **xxxxh:yyyyh** (segmento:deslocamento), em que **xxxxh** é o segmento e **yyyyh** é o deslocamento.

Para melhor entendimento da estrutura organizacional interna do microprocessador 8086/8088, observe todos os detalhes apresentados na Figura 10.1.

Para entender o motivo que levou a empresa Intel a adotar essa estrutura de endereçamento, é necessário observar alguns pontos históricos, apontados por Hyde (2003), Norton (1993) e também pela empresa Intel (2004):

- ◆ O microprocessador 8086/8088 foi lançado no ano de 1978, ocasião em que o custo de produção de memórias era alto para os padrões de mercado. Nesse período os microcomputadores em uso eram máquinas de 8 *bits* com capacidade de trabalhar com 48 KB de memória.

¹ A partir dessa limitação os demais microprocessadores da família Intel lançados a partir do modelo 8086/8088 possuem essa mesma característica, tendo como diferença a possibilidade de trabalhar com maior capacidade de memória, dependendo do modelo em uso.

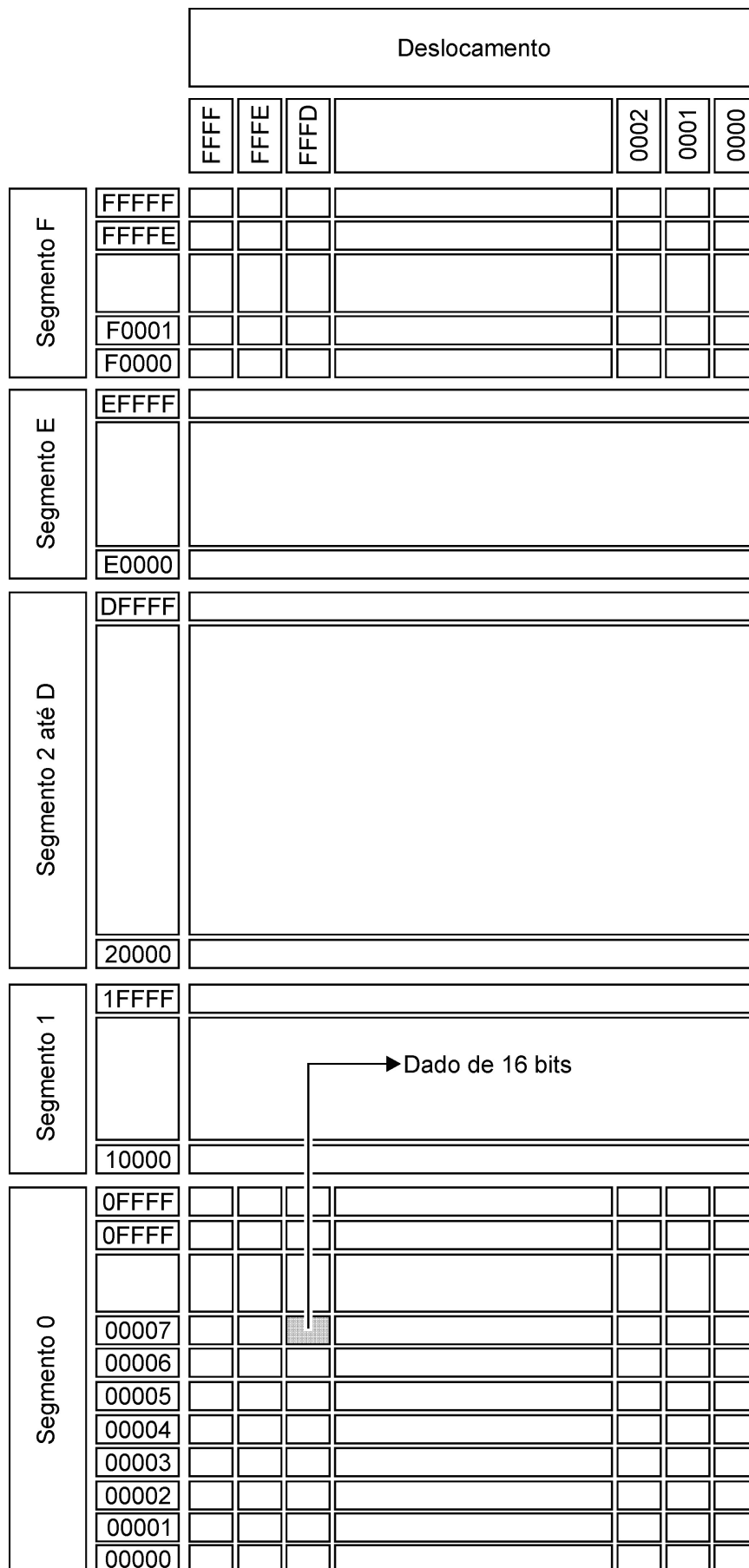


Figura 10.1 - Mapeamento de memória de 1 MB do processador 8086/8088.

- ◆ No ano de 1981, a empresa IBM lança seu microcomputador de uso pessoal (PC) trabalhando com 640 KB (dotado com o microprocessador Intel 80882, uma versão do processador 8086/8088 lançado em 1979) de memória com a capacidade de manipular dados de 16 bits. Nesse período os microcomputadores vendidos no mercado norte-americano possuíam a capacidade de memória de 64 KB, alguns chegavam a 128 KB, operando dados em processamento a 8 bits.
- ◆ Nessa ocasião poder usar uma memória de 1 MB era algo fantástico. A Intel não acreditava que o processador 8086/8088 duraria tanto, apesar de sua grande capacidade para a época, pois os modelos anteriores de microprocessadores tiveram um ciclo de vida em torno de cinco anos. Assim, não se preocupou em trabalhar em cima do processador 8086/8088, pois tinha planos para o desenvolvimento de outros processadores e a linha 8086/8088 não fazia parte desses planos.
- ◆ Devido ao sucesso estrondoso de vendas dos microcomputadores IBM-PC e da grande quantidade de softwares desenvolvidos, a Intel acabou sendo pega de surpresa e não pôde mais abandonar o projeto do processador 8086/8088, gerando uma família de processadores x86.
- ◆ Em 1982, esbarrou-se no limite de 1 MB do processador 8086/8088, foi quando a Intel lançou o processador 80286 e acabou ficando presa à estrutura interna do processador 8086/8088, pois todos os softwares escritos para os computadores da família IBM-PC estavam calcados na estrutura anterior utilizada no processador 8086/8088, ou seja, conseguiam chegar até 1 MB de memória e os 16 MB total de memória que o processador 80286 disponibilizava ficavam resumidos aos míseros 1 MB, com uma perda de 15 MB.
- ◆ O problema não era a memória endereçável, mas o processador 8086/8088 ser de 16 bits, com registradores de 16 bits e endereços de 16 bits, limitando a capacidade de endereçamento de memória em 64 KB.
- ◆ A forma que a Intel encontrou para solucionar o problema da capacidade de endereçamento da memória, criando a segmentação com 64 KB, foi muito inteligente. No entanto, a solução era viável para endereçar até 1 MB.
- ◆ O problema era como aumentar a capacidade de 64 KB para endereçar um volume maior de memória. Isso exigiria um esforço muito maior, o que foi de certa forma resolvido pelos engenheiros da Intel ao criarem a segmentação de blocos de memória.
- ◆ A segmentação não é ruim. O problema está na maneira como a implementação foi feita pela Intel em 1978, com o lançamento do processador 8086/8088 e que é de certa forma usada até hoje.
- ◆ Não se pode culpar a Intel por isso, pois ela não tinha grandes perspectivas para o processador 8086/8088 e se viu obrigada a manter a estrutura original mesmo em novo processador, como foi o caso do 80286. É importante considerar que a Intel resolveu o problema com o lançamento do processador 80386 em 1985.
- ◆ Devido ao grande volume de softwares desenvolvidos para os IBM-PCs durante os anos de 1980, incluindo-se o sistema operacional MS-DOS, a forma de segmentação adotada originariamente pela Intel não foi abandonada.
- ◆ Sistemas operacionais como LINUX e versões do Windows a partir da edição 95 não sofrem dos mesmos males que o MS-DOS, mas os processadores mais novos até os últimos lançamentos da linha Pentium, bem como do concorrente AMD, ainda por questões de compatibilidade utilizam segmentação.

A partir dos pontos apresentados fica mais fácil entender os motivos que levaram a empresa Intel a adotar a segmentação da memória e deslocamento de endereço, os quais acabaram sendo largamente utilizados na época.

O processador 8086/8088 utiliza quatro registradores de segmentos de memória: **CS** (*code segment*), **DS** (*data segment*), **SS** (*stack segment*) e **ES** (*extra segment*), desses quatro os mais utilizados são **CS**, **DS** e **SS**; cinco registradores de deslocamentos: **SI** (*source index*), **DI** (*destination index*), **SP** (*stack pointer*), **BP** (*base pointer*) e **IP** (*instruction pointer*). São sobre esses registradores que as operações de **segmento:deslocamento** são efetivadas. As operações de endereçamento de *segmento* são usadas nos registradores **CS**, **DS** e **SS** e as operações de endereçamento *deslocamento* são usadas nos registradores **SI**, **DI**, **SP**, **BP** ou **IP**, podendo-se também utilizar o registrador **BX** para a indicação de um endereço de deslocamento.

A Tabela 10.1 mostra as principais operações executadas nas áreas de segmentos de código, pilha, dados e extra e a Figura 10.2 apresenta esquema organização da memória de um microprocessador 8086/8088.

² Os microprocessadores 8086 e 8088 são idênticos, ou seja, ambos são processadores de 16 bits, exceto pela quantidade de dados que podiam receber e enviar de uma só vez quando da necessidade de comunicação com periféricos internos (unidade de disco) e externos (impressoras). O processador 8086 fazia essa comunicação a 16 bits, enquanto o processador 8088 fazia a 8 bits apesar de processar os dados internos a 16 bits. Isso tudo porque a maioria dos dispositivos e circuitos disponíveis na ocasião era de 8 bits (NORTON, 1993, p. 13).

Tabela 10.1 - Operações das áreas de segmento

Referência à memória	Identificador de Segmento	Identificador de Deslocamento
Segmento de código	CS	IP
Segmento de pilha	SS	SP, BP
Segmento de dados	DS	BX, SI, DI
Segmento extra	ES	DI

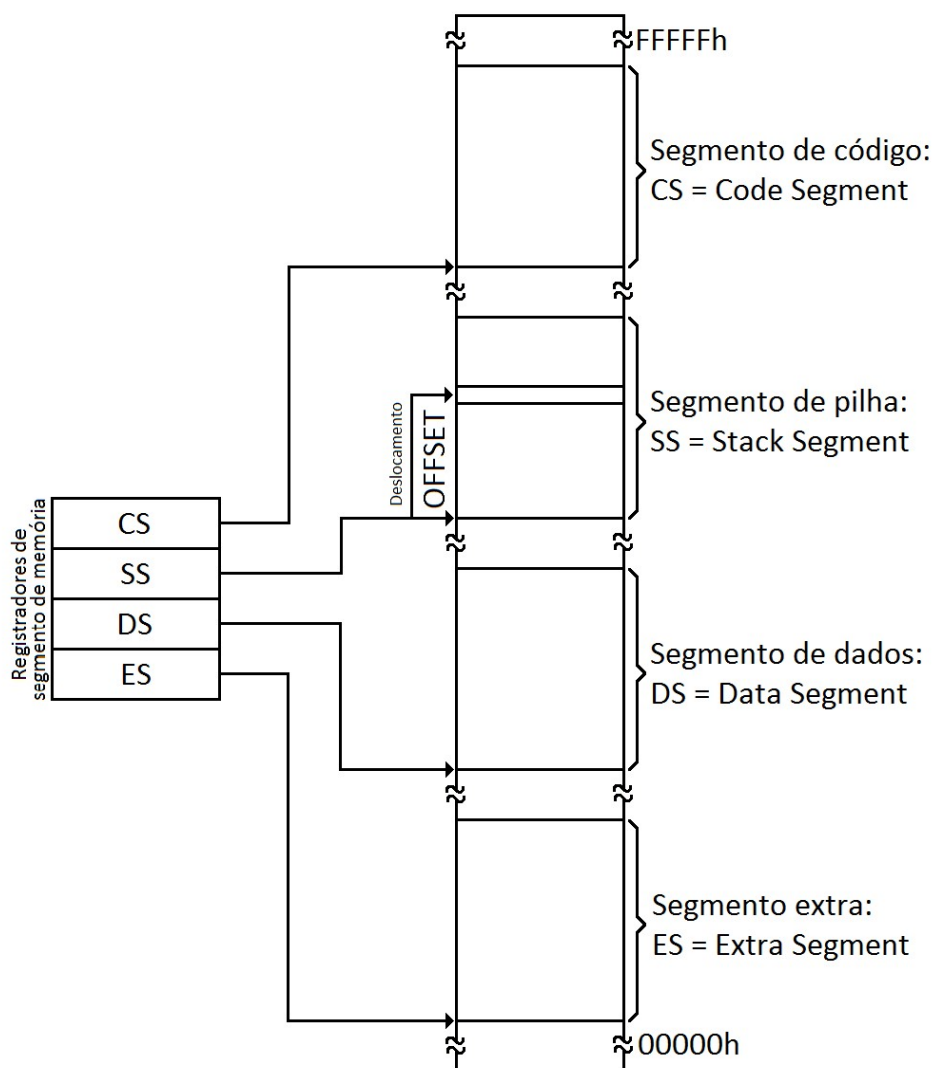


Figura 10.2 – Organização da memória de um microprocessador 8086/8088. Adaptado de YADAV, 2008.

Os registradores de segmento: **CS** tem por finalidade armazenar o endereço de início do código do programa a ser executado no segmento de memória, **DS** tem por finalidade armazenar o endereço onde se localizam os dados (variáveis) a serem usadas no programa, **SS** tem por finalidade armazenar os endereços da pilha e dos dados e **ES** tem por finalidade armazenar o endereço adicional de mais dados que venham a ser necessários ao programa se assim necessitar.

O microprocessador 8086/8088 opera com dois tipos de endereçamento de memória: *lógico* e *físico*. O endereço lógico é utilizado pelos programadores, enquanto o endereço físico é utilizado pelo *hardware* para acessar posições de memória. No entanto, é o próprio programador que direciona para o *hardware* o endereço físico que será usado a partir da informação do endereço lógico.

Os endereços físicos, para serem utilizados, ocupam um total de 20 *bits*. Internamente o microprocessador 8086/8088 opera com registradores de segmento com no máximo de 16 *bits*, o que gera a necessidade de utilizar mais 4 *bits* emprestados de um registrador de deslocamento.

O endereço do registrador de segmento é multiplicado por **10h** obtendo-se um endereço de 20 *bits*. A partir desta operação ocorre um deslocamento de 4 *bits* para a esquerda do valor e insere-se o valor obtido na parte direita do segmento, para então somar a ele o valor do endereço do registrador de deslocamento.

Os 4 *bits* excedentes que são a parte menos significativa do registro de 20 *bits* são sempre marcados com quatro *bits* setados a zero e os 16 *bits* mais significativos ficam definidos no par de registradores de base e de ponteiro, como ocorre, por exemplo, no caso do formato **CS:IP** que é uma das referências de endereço lógico mais usadas.

Vale relembrar que os registradores na sua forma geral possuem a capacidade de armazenar 16 *bits* de dados, ou seja, 2 *bytes*. Cada *byte* é subdividido em 2 *nibbles*, cada *nibble* é um conjunto de 4 *bits* e representa uma unidade de valor hexadecimal entre **0h** e **Fh**. Assim sendo, observe a Figura 10.3 que representa o cálculo do endereço físico **B2213h** a partir do segmento **A705h** e de deslocamento **B1C3h**, ou seja, **A705:B1C3**.

$$\begin{array}{r}
 \begin{array}{|c|c|c|c|} \hline \text{A} & 7 & 0 & 5 \\ \hline \end{array} \\
 \times \quad \begin{array}{|c|c|} \hline 1 & 0 \\ \hline \end{array} \\
 \hline
 = \begin{array}{|c|c|c|c|c|} \hline \text{A} & 7 & 0 & 5 & 0 \\ \hline \end{array} \\
 + \quad \begin{array}{|c|c|c|c|} \hline \text{B} & 1 & \text{C} & 3 \\ \hline \end{array} \\
 \hline
 = \begin{array}{|c|c|c|c|c|} \hline \text{B} & 2 & 2 & 1 & 3 \\ \hline \end{array}
 \end{array}$$

Figura 10.3 - Demonstração do cálculo do endereço físico de memória.

Um endereço de memória, segundo Hyde (2003), é aquele que "contém um componente *segmento* e outro componente *deslocamento*". Por esta razão é que normalmente se faz referência à posição lógica de memória utilizando a nomenclatura **segmento:deslocamento**, sendo esses dois valores constantes de 16 *bits*³.

A partir das coordenadas de *segmento* e *deslocamento* e de acordo com o exposto anteriormente, torna-se possível descobrir o endereço físico real a ser utilizado na memória. Isso é conseguido com a fórmula: **ENDEREÇO FÍSICO = SEGMENTO X 10h + DESLOCAMENTO**, em que, **SEGMENTO** é a informação de endereço existente no registrador de segmento **CS** (*code segment*) e **DESLOCAMENTO** é a informação do *offset* existente no registrador de deslocamento **IP** (*instruction pointer*). O valor **10h** (equivalente a 16 em notação decimal) é a capacidade em *bits* de armazenamento. O microprocessador utiliza as instruções de um programa com base no endereço de **segmento:deslocamento**, ou seja, com base nas informações dos registradores **CS:IP**.

Imagine obter o endereço físico (endereço real) da memória com base no endereço de segmento e deslocamento **2425:5121** (**segmento:deslocamento**). Basta substituir os valores na fórmula:

$$\text{ENDEREÇO FÍSICO} = 2425\text{h} \times 10\text{h} + 5121\text{h}$$

Após fazer o cálculo do valor **2425h** multiplicado pelo valor **10h** (valor 16 em decimal - 16 *bits*), obter-se-á o valor **24250h** que somado ao valor **5121h** resulta no endereço físico **29371h**. Apesar de esse trabalho ser realizado automaticamente pelo microprocessador, é interessante saber como funciona.

O programa **emu8086** pode ser usado para conferir o cálculo da geração de endereço físico de 20 *bits* a partir da definição de valores junto aos registradores **CS:IP**. Para proceder a este teste encerre o programa **emu8086** e carregue-o novamente. Em seguida execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** e sem escrever nenhum código acione o botão **emulate** ou execute a tecla de função **<F5>**. Ao ser apresentada a caixa de diálogo **8086 microprocessor emulate** indicada na Figura 10.4 entre os valores de segmento e deslocamento nos campos dos registradores **CS** e **IP** assinalados.

³ No processador 8086/8088 com deslocamentos de 16 bits, um segmento não pode ser maior do que 64 KB; pode ser menor (e a maioria é), mas nunca maior. Os processadores 80386 e posteriores permitem deslocamentos de 32 bits (o dobro do processadores 8086/8088) com segmentos de até 4 gigabytes (HYDE, 2003).

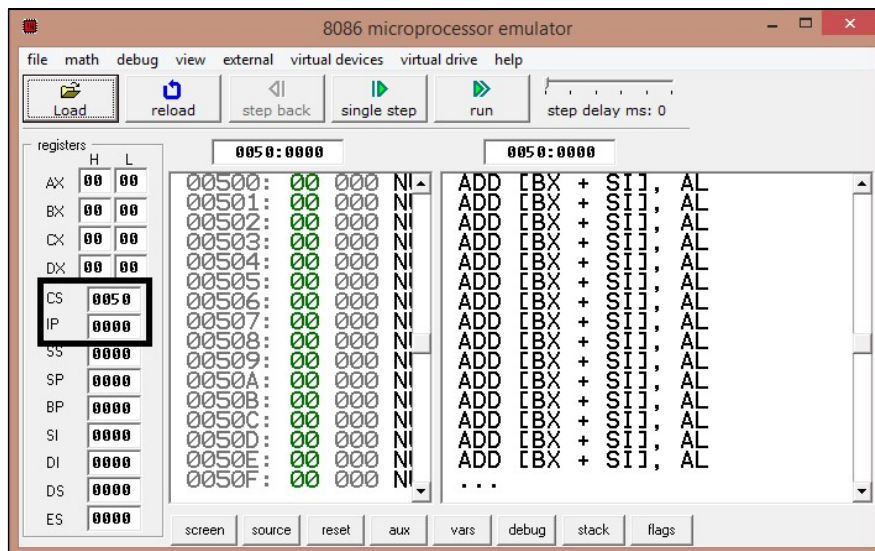


Figura 10.4 - Caixa de diálogo 8086 microprocessor emulate.

Junto aos campos dos registradores **CS** e **IP** informe respectivamente os valores **A705h** e **B1C3h** e observe a indicação automática do valor **B2213h** como mostrado junto a Figura 10.5 para a área que indica os valores do programa em *opcodes*.

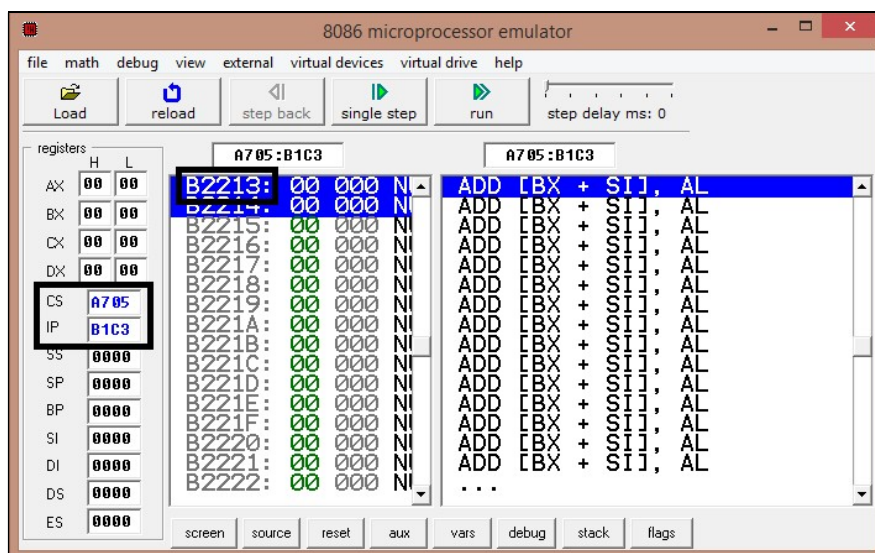


Figura 10.5 - Caixa de diálogo 8086 microprocessor emulate com cálculo de endereço físico.

Os valores dos registradores de segmento possuem inicialmente o mesmo valor quando o programa em desenvolvimento é montado como um arquivo do tipo **.COM**, normalmente definidos a partir do endereço de deslocamento **0100h** (**org 100h**). Para confirmar este fato, considere o seguinte programa:

```

;*****
;*   Programa: SEGM1.ASM   *
;*****

org 100h

.DATA
    mensagem DB 'Ola, Mundo$'

.CODE
    LEA DX, mensagem

```



```

MOV AH, 09h
INT 21h
INT 20h

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **SEGM1**, de forma que fique semelhante à imagem da Figura 10.6.

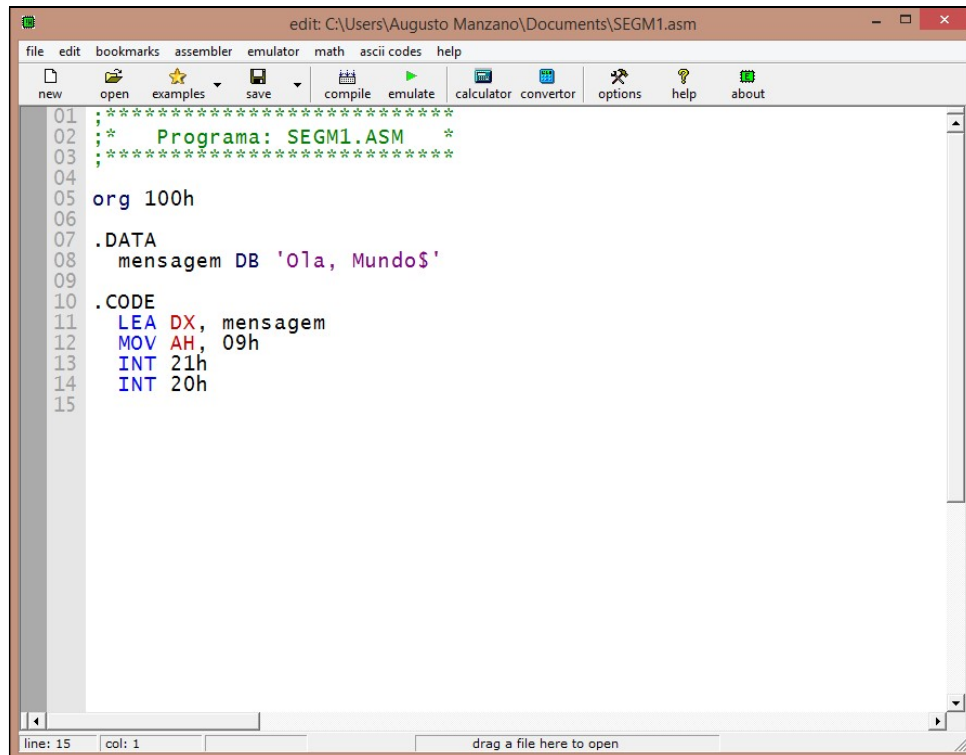


Figura 10.6 - Programa SEG1 na ferramenta emu8086.

Em seguida, execute o comando de menu **assembler/compile and load in the emulator** ou acione a tecla de função **<F5>**. Na janela **Emulator: SEG1.com_** são mostrados os valores dos registradores de segmento **CS, IP, DS, SS** e **ES**, como indica a Figura 10.7.

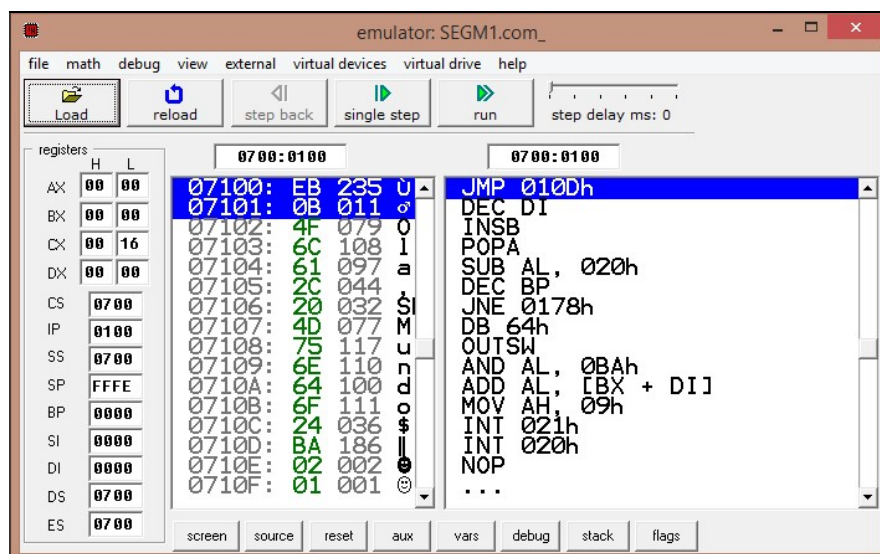


Figura 10.7 - Janela Emulator: SEG1.com_ indicando o endereço de segmento 0700:0100.

Note em especial os registradores **CS:IP** que indicam o uso do endereço lógico de memória **0700:0100**. Além do conjunto de registradores **CS:IP** mostrarem o valor de endereço lógico de memória correspondente ao endereço físico de memória **07100h** ($0700h \times 10h + 0100h$), os registradores **DS**, **SS** e **ES** também indicam o valor de endereço de segmento **0700**. Perceba que na área **Register** os registradores **CS** e **IP** mostram, respectivamente, os valores **0700h** e **0100h** que são o endereço lógico e na área **Memory** (quadro central do lado direito da área *register*) é indicado o valor **07100h** na primeira linha, informando assim o endereço físico na memória.

Ao ser executado o programa passo a passo, o valor do registrador **CS** permanece inalterado enquanto estiver no mesmo endereço de segmento de memória, enquanto o valor do registrador de deslocamento **IP** é alterado constantemente. Os demais valores dos registradores de segmento permanecem fixos.

Na Figura 10.7 note a indicação da marca de seleção sobre a instrução **JMP 010Dh**. Esta instrução faz um salto do fluxo de execução do programa para a linha de código **010Dh**. Observe o valor do registrador de segmento **IP** indicando **0100h**. Acione a tecla de função <F8> pela primeira vez para iniciar o processo de execução passo a passo do programa, como apresenta a Figura 10.8. Note o valor **010Dh** definido como endereço do registrador de segmento **IP**.

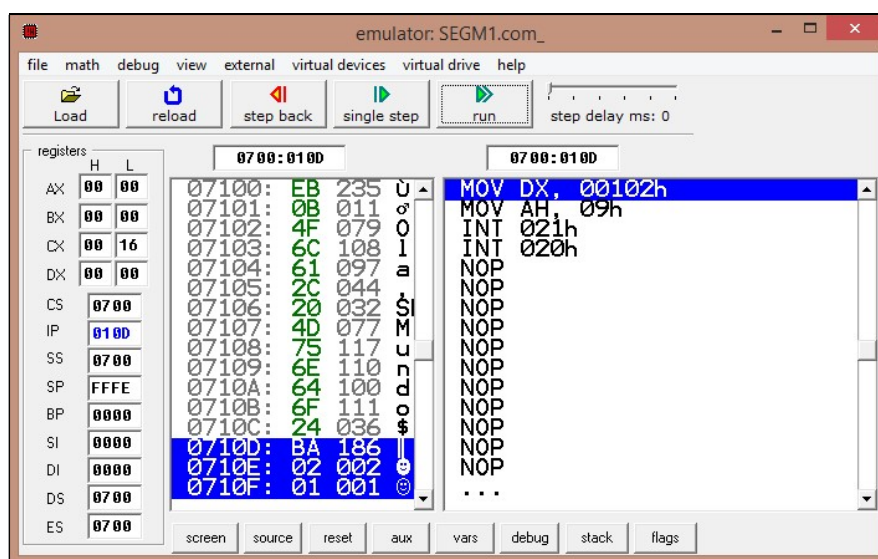


Figura 10.8 - Janela Emulador: SEGM1.com_ com ação da tecla <F8>.

Acione a tecla de função <F8> mais três vezes e aparece a indicação do valor do registrador de segmento **CS** apontando para a região de memória em que se encontra o código interno de execução da instrução **INT 21h**. Ao ser executada a linha de instrução **INT 21h**, o programa é desviado para uma sub-rotina interna que faz a interrupção. Ocorre a alteração do registrador de segmento **CS** e do registrador de deslocamento **IP** a fim de indicar para o programa em que posição da memória ele deve ir para executar a ação desejada.

A Figura 10.9 mostra essa ocorrência em que os registradores **CS** e **IP** possuem, respectivamente, os valores **F400h** e **0200h** (**CS:IP** como endereço de segmento **F400:0200**, ou seja, endereço físico **F4200h**). Note também que o registrador **SP** possui seu valor **FFFEh** alterado para o valor **FFF8h**.

Quando o registrador de deslocamento **SP** é alterado para **FFF8h** este indica que a pilha foi acionada. Esse efeito ocorreu devido à alteração do valor do registrador de segmento **CS**. Antes de o registrador de segmento **CS** ser alterado, o seu endereço **0700** foi armazenado na pilha, como pode ser conferido na Figura 10.10 (para ver execute o comando **view/stack** na janela **SEGM1.com**).

A Figura 10.10 mostra acima da barra de seleção atual (**0700:FFF8**) a linha definida com o endereço **0700:FFFA** em que se encontra o valor **0700**, endereço de memória onde se encontra o código do programa. Assim que a ação da instrução **INT 21h** for executada, o valor armazenado na pilha (que está na posição de memória **0700:FFF8**) retorna ao registrador de segmento **CS**.

Ao executar a tecla de função <F8> mais duas vezes, é possível ver a apresentação da mensagem e o retorno do valor **0700** para o registrador de segmento **CS**.

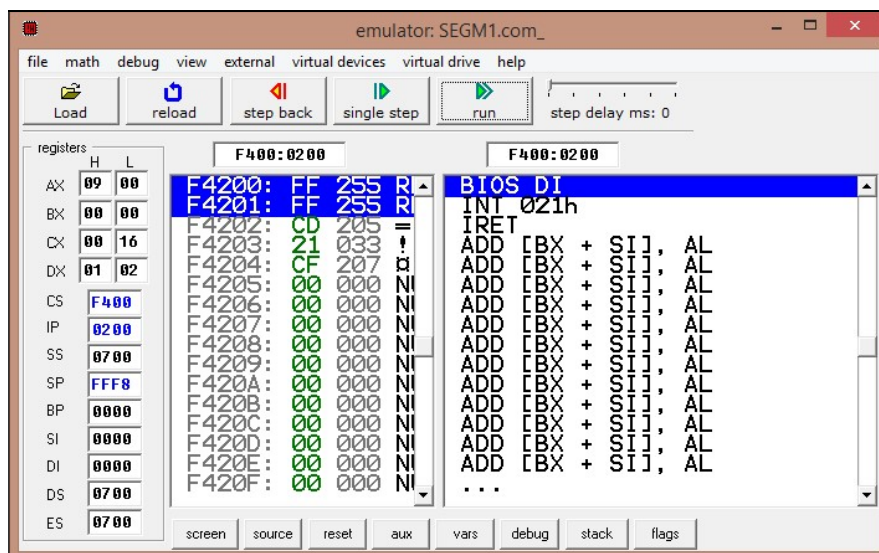


Figura 10.9 - Janela Emulador: SEGMENT1.com_ com alteração do registrador CS, IP e SP.

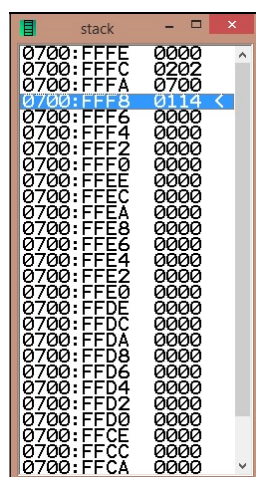


Figura 10.10 - Janela Stack com valor do registrador CS armazenado.

Na sequência execute a tecla de função <F8> mais duas vezes, acione o botão **OK** da caixa de diálogo **message** para encerrar o programa e saia do modo de execução do programa retornando à tela do editor de textos.

10.2 - Programas executáveis

Até esse momento os programas desenvolvidos em linguagem de programação *Assembly 8086/8088* utilizaram a diretiva **org 100h** ou as diretivas **.MODEL small** e **.STACK 512d**. Os programas que usam as diretivas **.MODEL small** e **.STACK 512d** realizaram operações com valores numéricos e os programas que usam a diretiva **org 100h**, além de usar valores numéricos, fizeram apresentação de sequências de caracteres (*string*).

Os programas que usam as diretivas **.MODEL small** e **.STACK 512d**, quando compilados, possuem a extensão **.EXE** e os programas que usam a diretiva **org 100h**, quando compilados, possuem a extensão **.COM**.

Os programas com extensão **.EXE** são definidos com algumas diferenças em relação aos programas com extensão **.COM**, pois os registradores de segmento **CS** e **SS** são iniciados com valores diferentes dos registradores **DS** e **ES**.

Os programas compilados com extensão **.COM**, como já explanado, são programas com estrutura simples que possuem seu código em linguagem de máquina muito pequeno, ocupando até 64 KBytes. Já os programas compilados com extensão **.EXE** são programas com estrutura avançada, pois é possível trabalhar com cabeçalhos, realocação de recursos, entre outras possibilidades.

Para trabalhar com arquivos de programas com extensão **.EXE**, a partir do programa **emu8086** é necessário acionar os comandos de menu **file/new/exe template**. Será apresentado, na sequência, o código de programa seguinte:

```
01 ; multi-segment executable file template..
02
03 data segment
04     ; add your data here!
05     pkey db "press any key...$"
06 ends
07
08 stack segment
09     dw 128 dup(0)
10 ends
11
12 code segment
13 start:
14 ; set segment registers:
15     mov ax, data
16     mov ds, ax
17     mov es, ax
18
19     ; add your code here
20
21     lea dx, pkey
22     mov ah, 9
23     int 21h          ; output string at ds:dx
24
25     ; wait for any key....
26     mov ah, 1
27     int 21h
28
29     mov ax, 4c00h ; exit to operating system.
30     int 21h
31 ends
32
33 end start ; set entry point and stop the assembler.
```

Se o programa for executado, neste momento. É apresentada a mensagem **"press any key..."**. Ao olhar para o modelo de programa indicado, percebe-se que este possui uma estrutura sintática um pouco mais complexa que as utilizadas nos exemplos anteriores deste livro. Num programa **.EXE**, é necessário se preocupar com a definição dos segmentos de dados (linhas de **03** até **06**), pilha (linhas de **08** até **10**) e código (linhas de **12** até **33**) de forma mais explícita que em um programa **.COM**. Não é necessário se preocupar com a posição de endereçamento da memória, a menos que estivesse sendo utilizada a ferramenta **DEBUG** para a geração do código.

As linhas iniciadas com o caractere ponto e vírgula se configuram por serem linhas de comentários, normalmente usadas para se estabelecer a identificação de ações do programa com a finalidade de orientar de forma mais clara o código do programa na documentação interna.

Nas linhas **03**, **08** e **12** são definidos os trechos de códigos identificados pelas diretivas **SEGMENT** após a identificação das áreas **data**, **stack** e **code** e também a diretiva **ENDS** para encerramento da diretiva **SEGMENT**.

Os segmentos com a utilização das diretivas **SEGMENT** e **ENDS** obedecem à seguinte estrutura sintática:

```
[nome]  SEGMENT
        [corpo do seguimento]
[nome]  ENDS
```

O rótulo *nome* é obrigatório para identificar o segmento em uso, podendo-se definir qualquer nome escolhido pelo programador.

O trecho para a área de dados, existente entre as linhas de código **03** e **06**, deve conter os dados que o programa irá manipular.

O trecho para a pilha, existente entre as linhas de código **08** e **10**, estabelece o valor de 128 *bytes* de espaço para a manipulação da pilha com dados do tipo **DW**. O operador **dup(0)** definido na linha **09** estabelece que o valor **0** (zero) será repetido um determinado número de vezes na memória. Na linha **09** encontra-se a instrução **dw 128 dup(0)**, que define que serão alocados na memória 128 *bytes* com valor zero. O operador **dup(?)** permite alocar um tamanho limite na memória sem a definição de valores.

O trecho para a definição da área de código, existente na linha **19**, deve ser utilizado para se inserir nessa posição o código de programa. Note que o trecho de linhas entre **12** e **33** possui a definição de parte do código de programa de forma automática (característica da ferramenta **emu8086**).

Para demonstrar o uso de programas **.EXE**, acompanhe as seguintes instruções de preenchimento do modelo de programa na ferramenta **emu8086** e grave o programa com o nome **SEGM2**:

- ◆ Vá até a linha **05** e sobre escreva a linha existente pela instrução **mensagem db "Ola, Mundo\$"** e ao final acione a tecla **<Enter>** e na linha **29** substitua a linha existente pela instrução **lead x, mensagem**. Remova as linhas de código **31**, **32** e **33**.
- ◆ A partir da linha **19** retire a linha de comentário e acrescente o código:

```
MOV    DX, OFFSET mensagem

MOV    AH, 09h
INT    21h

MOV    AH, 4Ch
INT    21h
```

O trecho de código com as instruções **MOV AH, 4Ch** e **INT 21h** procede com o encerramento do programa e a devolução do controle de execução do programa ao sistema operacional. Esta é uma forma alternativa ao modo **INT 20h**. As instruções **MOV AH, 4Ch** e **INT 21h** carregam o código de retorno para o registrador geral **AL**, sendo então considerado retorno normal quando o valor de **AL** for **0** (zero), sendo diferente de zero há algum erro no retorno do programa. O retorno do programa ao sistema operacional ocorre com a execução da interrupção com o código de função **4Ch** em **AH**.

Usar a forma **MOV AH, 4Ch** e **INT 21h** caracteriza a maneira adequada de retorno ao sistema operacional, mas não adequada para uso junto ao programa **Enhanced DEBUG** que faz uso da instrução **INT 20h**.

A seguir é apresentado o código completo do programa. Atente para as linhas grafadas em **negrito** contendo as inserções anteriormente solicitadas:

```
; multi-segment executable file template.
```

```
data segment
    ; add your data here!
    mensagem db "Ola, Mundo$"
ends
```

```
stack segment
    dw 128 dup(0)
ends
```

```
code segment
start:
; set segment registers:
    mov ax, data
    mov ds, ax
    mov es, ax

    MOV    DX, OFFSET mensagem

    MOV    AH, 09h
    INT    21h
```

```

MOV     AH, 4Ch
INT     21h

lea dx, mensagem
mov ah, 9
int 21h      ; output string at ds:dx

mov ax, 4c00h ; exit to operating system.
int 21h
ends

end start ; set entry point and stop the assembler.

```

Observação

A definição de *strings* pode ser realizada tanto com apóstrofes (aspas simples) como com aspas (aspas inglesas).

Aparentemente não existirá diferença do ponto de vista externo do programa, mas internamente sim. Os programas com extensão **.EXE** são executados um pouco mais devagar e são maiores que os programas com extensão **.COM**.

Apesar de a ferramenta **emu8086** ser muito prática e agradável de trabalhar, em alguns casos, como na definição de programas do tipo **.EXE**, ela torna o código um pouco complexo. Não por culpa dela, mas da própria estrutura interna desse tipo de programa que é mais complexa do que a de programas **.COM**.

Execute o comando de menu **file/new**, escolha qualquer uma das opções de *template* apresentadas, acione as teclas **<Ctrl> + <A>** para selecionar todo o texto existente e em seguida acione a tecla **** para limpar toda a área de texto. Na sequência escreva o código do programa a seguir:

```

TITLE    Teste de Segmento 3

#MAKE_EXE#

DADOS SEGMENT 'DATA'
      mensagem DB "Ola, Mundo$"
DADOS ENDS

PILHA SEGMENT STACK 'STACK'
      DW 0100h DUP(?)
PILHA ENDS

CODIGO SEGMENT 'CODE'
INICIO PROC FAR
      MOV    AX, DADOS
      MOV    DS, AX
      MOV    ES, AX

      MOV    DX, OFFSET mensagem

      MOV    AH, 09h
      INT    21h

      MOV    AH, 4Ch
      INT    21h
      RET

INICIO ENDP
CODIGO ENDS
END INICIO

```

A partir deste ponto com os comandos de menu **file/save** grave o programa anterior com o nome **SEGM3**, de forma que fique semelhante à Figura 10.11.

```

01 TITLE   Teste de Segmento 3
02
03 #MAKE_EXE#
04
05 DADOS   SEGMENT 'DATA'
06         mensagem DB "Ola, Mundo$"
07 DADOS   ENDS
08
09 PILHA   SEGMENT STACK 'STACK'
10         DW 0100h DUP(?)
11 PILHA   ENDS
12
13 CODIGO  SEGMENT 'CODE'
14         INICIO PROC FAR
15             MOV     AX, DADOS
16             MOV     DS, AX
17             MOV     ES, AX
18
19             MOV     DX, OFFSET mensagem
20
21             MOV     AH, 09h
22             INT     21h
23
24             MOV     AH, 4Ch
25             INT     21h
26             RET
27         INICIO ENDP
28 CODIGO  ENDS
29 END INICIO

```

Figura 10.11 - Programa SEG3 na ferramenta emu8086.

O programa anterior está sendo escrito de forma mais simples que a versão obtida por meio do *template*. Nesta versão está se utilizando o estilo de escrita de códigos de programas em *Assembly* semelhante à estrutura utilizada pelos *assemblers* **TASM** (Turbo Assembler - Borland) e **MASM** (MS-Assembler - Microsoft).

A linha **01** do programa utiliza a diretiva **TITLE** para indicar a identificação de um nome interno para o programa. Essa diretiva não é exclusiva para programas com extensão **.EXE**, e pode também ser utilizada em programas com extensão **.COM**. Aliás, é uma forma elegante de identificar os programas na primeira linha de código.

Na linha **03** encontra-se a definição do identificador de tipo de arquivo a ser gerado. Neste caso, está sendo definido o identificador **#MAKE_EXE#**. Em programas com extensão do tipo **.COM** pode-se fazer uso do identificador **#MAKE_COM#** em conjunto com a diretiva **org 100h** para indicar o local onde os dados devem ser manipulados na memória. O identificador **#MAKE_EXE#**, para ser usado, exige que seja determinada a área de dados por meio das diretivas **SEGMENT** e **ENDS** com a definição do parâmetro **'DATA'**.

O programa **SEG3** usa a definição das diretivas **SEGMENT** e **ENDS** de uma forma um pouco diferente da forma utilizada no programa **SEG2**. Para tanto, observe a seguinte sintaxe:

```

[nome] SEGMENT <STACK> <parâmetro>
        [corpo do seguimento]
[nome] ENDS

```

O rótulo *nome* é obrigatório e o identificador **STACK** após **SEGMENT** é utilizado após a diretiva **SEGMENT** quando da definição da pilha. Após a definição do nome de identificação do segmento e da diretiva **SEGMENT**, torna-se necessário identificar o segmento em uso com um parâmetro de reconhecimento de seu tipo de operação entre os símbolos de apóstrofos, que pode ser:

- ◆ **DATA** - identifica a área de definição de dados do programa.
- ◆ **STACK** - define o tamanho da área de pilha.
- ◆ **CODE** - define o trecho de código do programa.

Os rótulos de definição e identificação dos segmentos de dados, pilha e código nas linhas **06-08**, **10-12** e **14-30** estão grafados em português. Quanto aos parâmetros entre os símbolos de apóstrofos, são de certa forma, opcionais, devendo ser mantidos na sua forma original para maior legibilidade do código.

Para verificar detalhes existentes, execute o programa com o comando de menu **assembler/compile and load in the emulator**. Em seguida na janela **emulator: SEGM3.exe** acione o comando de menu **view/stack**, de forma que a área de trabalho fique semelhante à Figura 10.12. Se necessitar, faça a distribuição das janelas e ajuste as suas barras de rolagem.

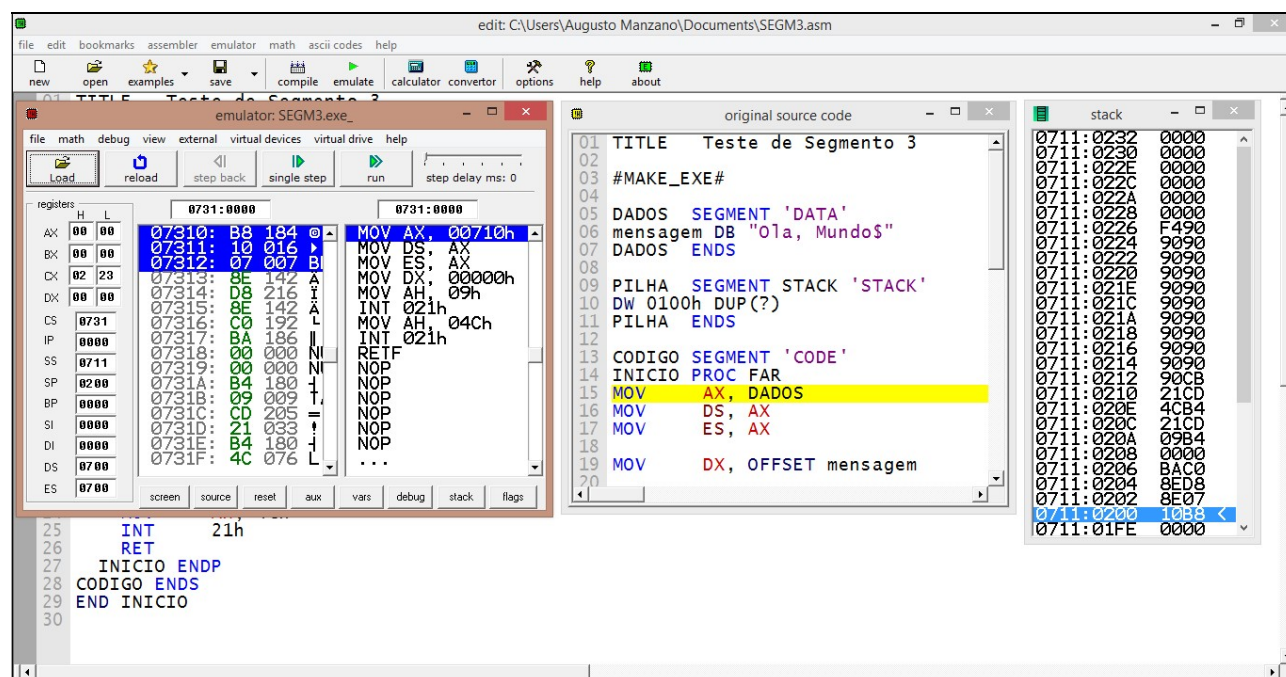


Figura 10.12 - Definição da área de trabalho para o programa SEGM3.

Antes de proceder à execução passo a passo do programa, é necessário levar em consideração algumas linhas de código que aparentemente, ao ser solicitada a execução do programa, não são executadas:

- ◆ O endereço do segmento de dados é definido na memória pelo trecho de código situado entre as linhas **06** e **08**. Os registradores de segmento **DS** e **ES** apontam o endereço dessa área, que neste caso marca o segmento **0700h**.
- ◆ O endereço do segmento de pilha é definido na memória pelo trecho de código situado entre as linhas **10** e **12**. O registrador de segmento **SS** aponta o endereço dessa área, que neste caso marca o segmento **0711h**, a partir do deslocamento **0200h**, como pode ser constatado no registrador de segmento **SP** (**SS:SP = 0711:0200**). A indicação **SS:SP** mostra o endereço de posição atual da pilha.
- ◆ O endereço de segmento de código é definido na memória pelo trecho de código situado entre as linhas **14** e **30**. O registrador de segmento **CS** aponta o endereço dessa área, que neste caso marca o endereço de segmento **0731**, a partir do endereço de deslocamento **0000h**, como pode ser constatado no registrador de segmento **IP** (**CS:IP = 0731:0000**).

É importante ressaltar mais uma vez que possivelmente no computador do leitor os valores de endereço de memória aqui apresentados sejam diferentes. É importante estar atento a esse detalhe.

À medida que o programa for executado, esses valores podem sofrer algumas alterações. Nesse momento inicial as áreas **memory** (área de apresentação do código na parte central da tela ao lado direito da área **registers**) e **disassemble** (área de apresentação do código na parte direita da tela) apresentam, respectivamente, o conteúdo existente na memória e a linha de código a ser executada.

Na tela **original source code** a linha de código do programa **MOV AX, DADOS** (referente à linha de código **15**) é definida para a área **disassemble** como **MOV AX, 00710h**, que está armazenado a partir do endereço de memória **0731:0000h**. O endereço **00710h** é o local onde se encontra a definição da área de dados do programa que está entre as linhas **06** e **08**. Mais adiante este valor será associado aos registradores de segmento **DS** e **ES**. Observe esses dados na Figura 10.13.

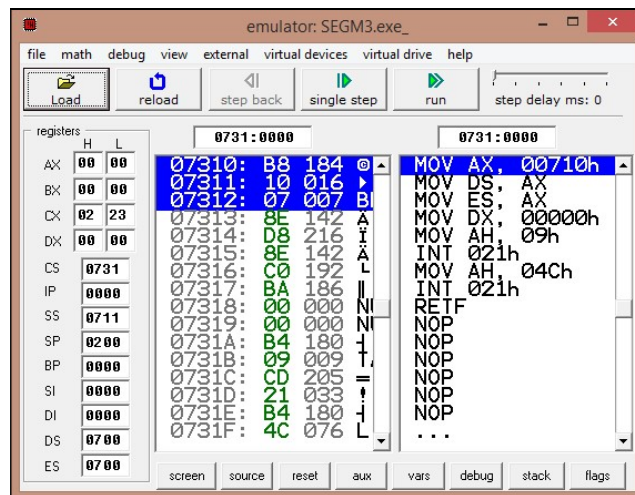


Figura 10.13 - Valores das posições de memória.

Acione a tecla de função <F8> (primeira vez), note que o registrador **AX** é armazenado com o valor **0710h** e o registrador de deslocamento **IP** passa a ter o valor **0003h**, como pode ser constatado na Figura 10.14. Observe que o valor de **AX** nesse momento é o que estava associado à instrução **MOV AX, 00710h** (**CS:IP = 0731:0003**) na área **disassemble**.

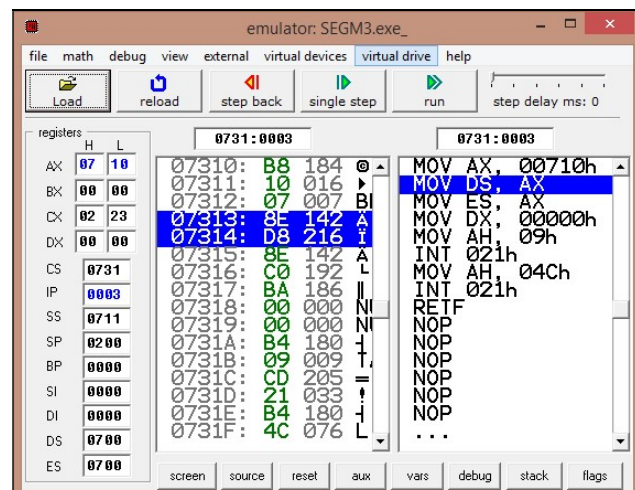


Figura 10.14 - Valores das posições de memória após <F8> - primeira vez.

É importante ressaltar que um registrador de segmento, como é o **DS**, não pode receber um endereço de forma direta, como ocorre com os registradores gerais. Por esta razão a próxima instrução a ser executada (**MOV DS, AX**, referente à linha 16) movimenta o valor do registrador **AX** para o registrador **DS** de forma indireta.

Se observar a área **memory** da Figura 10.14, verá que as instruções marcadas são **07313 8E 142 Ä** e **07314 D8 216 Ĩ**. Os valores **07313h** e **07314h** correspondem ao endereço físico de memória. Os códigos **8E** e **D8** correspondem ao *opcode* de execução da instrução **MOV DS, AX**. Os valores **142** e **216** que aparecem do lado direito são a representação decimal dos valores **8E** e **D8** e os caracteres **Ä** e **Ĩ** apresentados na quarta coluna da área **memory** correspondem aos caracteres ASCII referentes ao valor indicado na terceira e quarta colunas.

A área **memory** (parte central da janela) mostra a listagem do programa em linguagem de máquina e a área **disassemble** (lado direito) mostra o programa em linguagem Assembly. Nesse momento, se for acionado o botão **debug** (sexto botão na parte inferior da tela **emulator: SEGMENT3.exe**), será apresentada a janela **Debug** como indicado na Figura 10.15.

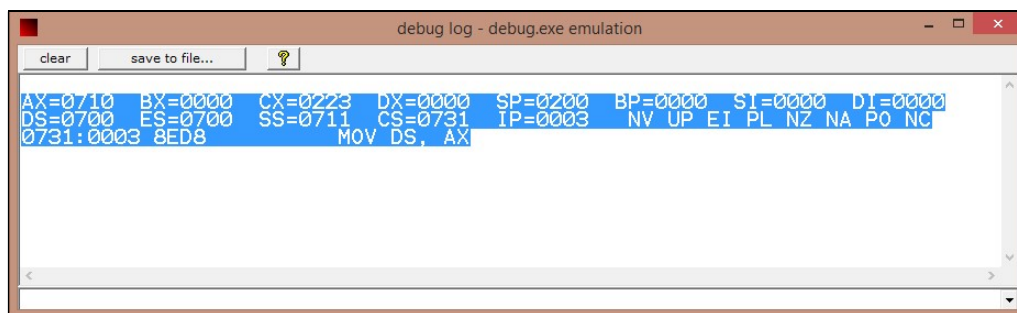


Figura 10.15 - Janela Debug.

A Figura 10.15 indica na terceira linha o local de memória onde se encontra a instrução, sendo o endereço de memória **0731:0003**, o código *opcode* **8ED8** e no extremo direito dessa linha é indicado a instrução em Assembly **MOV DS, AX**. Vale lembrar que o código de máquina (*opcode*) **8ED8** é a instrução equivalente em linguagem Assembly **MOV DS, AX**.

Olhe novamente a Figura 10.14 e observe que o próximo passo fará com que o valor do registrador **IP** que aponta o endereço **0003h** indique o endereço **0005h**. Haja vista a indicação do valor do endereço de memória **07315** estar logo abaixo da parte selecionada do código na área **memory**.

Para verificar a ocorrência de **MOV DS, AX**, acione pela segunda vez a tecla de função **<F8>** e observe as alterações no registrador de segmento **IP** de **0003h** que aponta para a próxima linha do programa, endereço de segmento **0005h**, e do registrador de deslocamento **DS** com o valor **0710** em que se encontra o início da área de segmento de dados, como pode ser verificado na Figura 10.16.

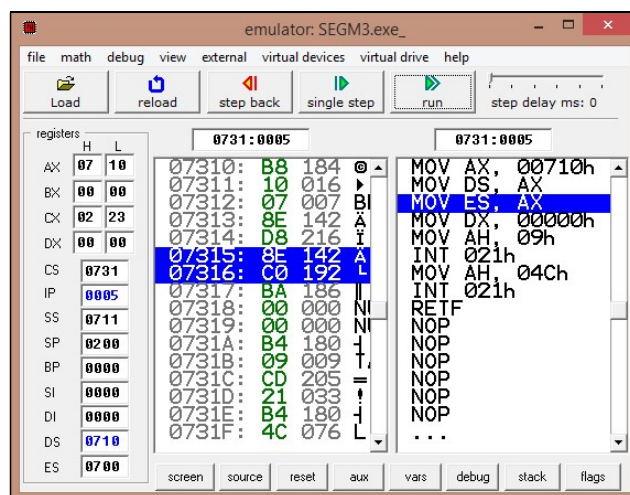


Figura 10.16 - Valores das posições de memória após **<F8>** - segunda vez.

É sabido que os registradores de segmento **DS** e **ES** em programas com extensão **.EXE** necessitam indicar o mesmo ponto de endereço. Os registradores em questão são diferentes, e por esta razão é necessário também movimentar o valor armazenado no registrador geral **AX** para o registrador de segmento **ES** pela linha de código **MOV ES, AX**.

Para certificar essa nova ação, acione a tecla de função **<F8>** pela terceira vez e observe a alteração dos valores do registrador de deslocamento **IP** para o valor **0007h** e do registrador de segmento **ES** com o valor **0710h**, como indica a Figura 10.17.

Os registradores de segmento **DS** e **ES**, após a terceira etapa de execução do programa, possuem o mesmo valor, ou seja, **0710h**, e assim devem estar. A título de curiosidade acione o botão **aux** e selecione na lista a opção **memory**. No campo ao lado esquerdo do botão **update** entre o valor **0710:0000** e acione com o ponteiro do *mouse* o botão **update**. Note que a mensagem **Ola, Mundo** está armazenada a partir do deslocamento **0000h** do segmento **0710h**. A Figura 10.18 mostra a ocorrência.

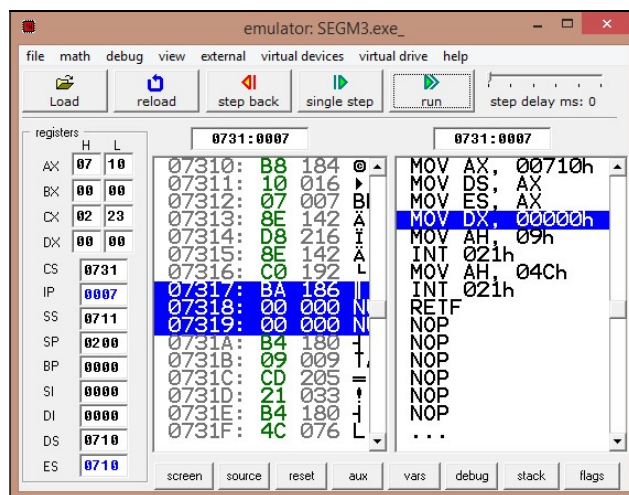


Figura 10.17 - Valores das posições de memória após <F8> - terceira vez.

A janela **random access memory** possibilita dois modos de visualização do estado da memória. Os dados podem ser vistos como tabela, semelhante à Figura 10.18 por meio da opção **table**, ou podem ser vistos como uma listagem por meio da opção **list** como mostra a Figura 10.19. Para fechar a janela, basta acionar o botão **X** no lado direito de sua barra de título.

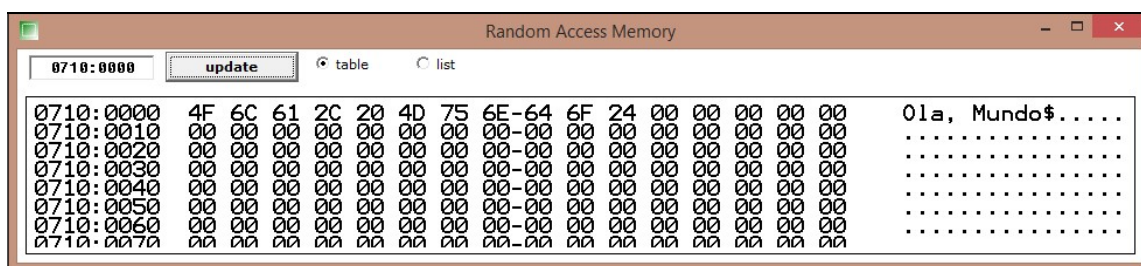


Figura 10.18 - Janela Random Access Memory com a indicação do endereço de segmento de dados (tabela).

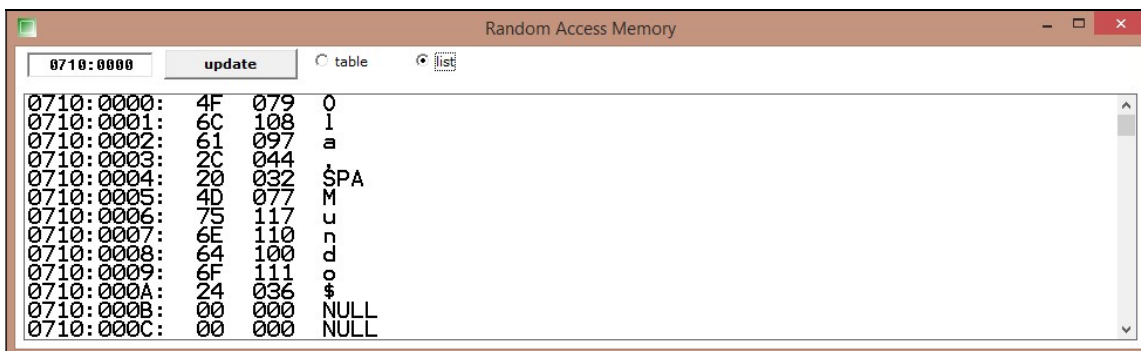


Figura 10.19 - Janela Random Access Memory com a indicação do endereço de segmento de dados (lista).

A linha de código do programa **MOV DX, OFFSET mensagem** (linha 19) ou **MOV DX, 00000h** na área **disassemble** define o valor de deslocamento (*offset*) do início da área de dados (que é **00000h**) para o registrador geral **DX**. Isso faz com que o programa visualize o conteúdo na área de dados da memória. Na sequência acione a tecla de função <F8> pela quarta vez e observe que a única alteração ocorrida é com relação ao valor do registrador de deslocamento **IP**, que passa a ter o valor de deslocamento **000Ah**, como mostra a Figura 10.20.

Apesar de não surtir o efeito na ferramenta **emu8086** (pois a diretiva **ASSUME** não é processada), é possível manter a definição da diretiva **ASSUME** no sentido de manter compatibilidade com os programas assemble-dores **TASM** e **MASM**.

Execute o comando de menu **file/new**, escolha qualquer uma das opções de *template* apresentadas, acione as teclas **<Ctrl> + <A>** para selecionar todo o texto existente e em seguida acione a tecla **** para limpar toda a área de texto. Na sequência escreva o código do programa a seguir:

```
TITLE    Teste de Segmento 4

#MAKE_EXE#

DADOS    SEGMENT 'DATA'
          mensagem DB "Ola, Mundo$"
DADOS    ENDS

PILHA     SEGMENT STACK 'STACK'
          DW 0100h DUP(?)
PILHA     ENDS

CODIGO    SEGMENT 'CODE'

          ASSUME CS:CODIGO, DS:DADOS, SS:PILHA

          INICIO PROC FAR
              MOV     AX, DADOS
              MOV     DS, AX
              MOV     ES, AX
              MOV     DX, OFFSET mensagem
              MOV     AH, 09h
              INT     021h
              MOV     AH, 4Ch
              INT     21h
              RET
          INICIO ENDP
CODIGO    ENDS
END INICIO
```

A partir deste ponto, com os comandos de menu **file/save**, grave o programa anterior com o nome **SEGM4** de forma que fique semelhante à Figura 10.22.

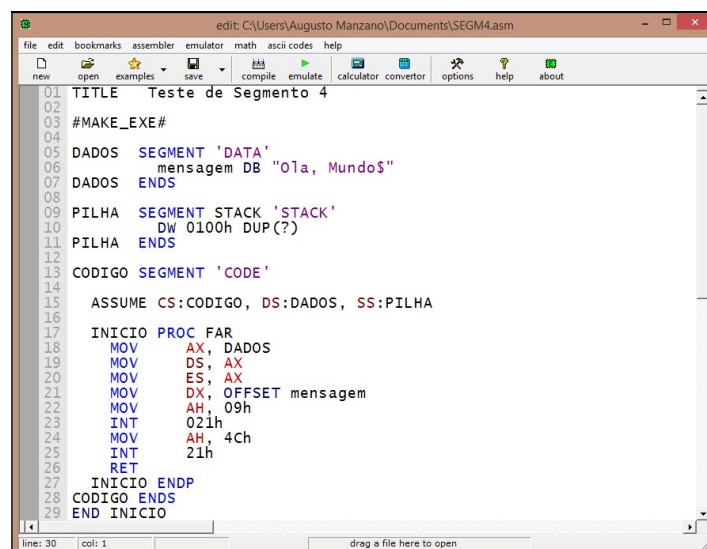


Figura 10.22 - Programa SEGM4 na ferramenta emu8086.

Observação

Para maiores detalhes sobre compatibilidade da ferramenta **emu8086** com as ferramentas **TASM** e **MASM**, consulte a documentação *on-line* da ferramenta. Acesse **Help/MASM/TASM compatibility**.

Atente para a linha grafada em negrito (que será a linha **15**) com o uso da diretiva **ASSUME**. Quando utilizada a diretiva **ASSUME**, ela permite associar de forma explícita cada rótulo definido ao seu respectivo segmento de memória.

10.3 - Bibliotecas em Assembly

Muitos recursos usados ou desenvolvidos para alguns programas tornam-se úteis para outros programas. O fato de copiar um determinado trecho de programa para ser usado em outro, apesar de válido, pode se tornar algo cansativo e propenso a erros.

Um recurso de programação bastante adequado é o desenvolvimento de uma biblioteca de recursos genéricos que possam ser utilizados em vários outros programas. Com essa atitude se economiza tempo de desenvolvimento e evita-se a ocorrência de muitos erros de escrita ou de lógica, pois se faz uso da filosofia de reaproveitamento de código.

A biblioteca em *Assembly* é um arquivo em formato texto, o qual possui uma sequência de rotinas de programa (podem ser procedimentos ou macros) que serão anexadas ao programa em execução. Para demonstrar esse recurso considere o seguinte código:

```
ESCREVA  MACRO
  PUSH  AX
  MOV   AH, 9h
  INT   21h
  POP   AX
ENDM
```

Execute no programa **emu8086** o comando de menu **file/new**, escolha qualquer uma das opções, acione simultaneamente as teclas **<Ctrl> + <A>** para selecionar o código existente e acione em seguida a tecla ****. Informe o código anterior, gravando-o com o nome **BIBLIO.inc**, de forma que fique semelhante à Figura 10.23.

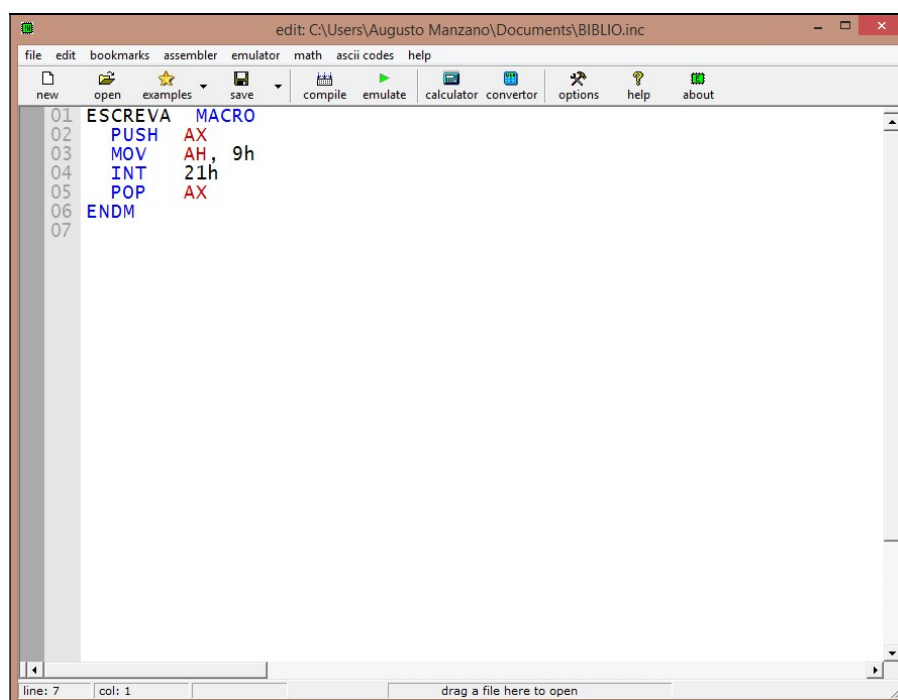


Figura 10.23 - Biblioteca BIBLIO na ferramenta emu8086.

Observação

A extensão **.inc** é apenas uma sugestão, pois o arquivo de biblioteca pode ter qualquer nome e extensão, desde que respeitados os limites da máquina.

Após a montagem da biblioteca, acesse **file/new**, escolha qualquer uma das opções, acione simultaneamente as teclas **<Ctrl> + <A>** para selecionar o código existente e acione em seguida a tecla ****. Na sequência informe este código:

```
TITLE    Teste de Segmento 5

#MAKE_EXE#

INCLUDE 'biblio.inc'

DADOS    SEGMENT 'DATA'
          mensagem DB "Ola, Mundo$"
DADOS    ENDS

PILHA     SEGMENT STACK 'STACK'
          DW 0100h DUP(?)
PILHA     ENDS

CODIGO    SEGMENT 'CODE'
          ASSUME CS:CODIGO, DS:DADOS, SS:PILHA
          INICIO PROC FAR
              MOV     AX, DADOS
              MOV     DS, AX
              MOV     ES, AX
              MOV     DX, OFFSET mensagem

          ESCREVA

              MOV     AH, 4Ch
              INT     21h
              RET
          INICIO ENDP
CODIGO    ENDS
END INICIO
```

Execute os comandos de menu **file/save**, depois grave o programa anterior com o nome **SEGM5**, de forma que fique semelhante à Figura 10.24.

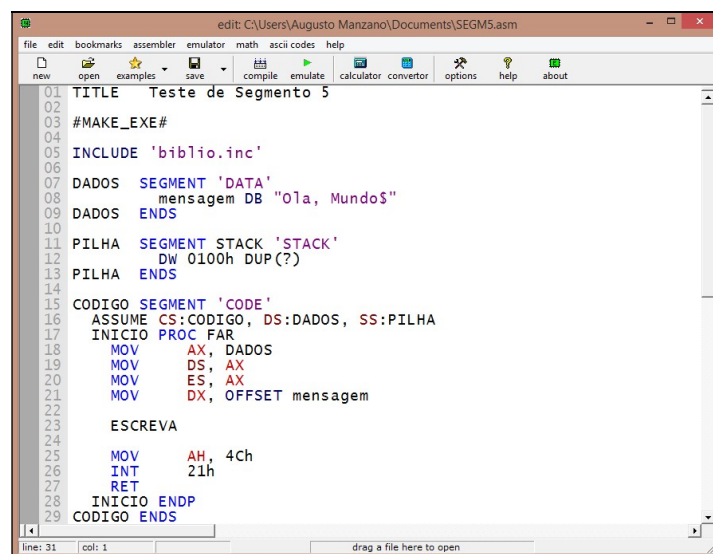


Figura 10.24 - Programa SEGM5 na ferramenta emu8086.

Atente para a linha **05** do programa na qual se encontra a diretiva **INCLUDE** seguida do nome do arquivo de biblioteca entre os símbolos de apóstrofo. A diretiva **INCLUDE** é responsável por vincular ao programa um arquivo de biblioteca externo, possibilitando ao programa ter acesso aos recursos definidos na biblioteca, como é demonstrado na linha **23** quando da chamada da macro **ESCREVA**, como se essa instrução fosse da própria linguagem de programação *Assembly*.

O recurso de biblioteca aumenta a eficácia e eficiência de um programador de computadores. Para ampliar um pouco mais a biblioteca **biblio.inc**, serão acrescentados alguns recursos como limpar tela, posicionar cursor em um determinado ponto da tela, colocar o teclado em espera e manipular o cursor.

Para abrir o arquivo de biblioteca na ferramenta **emu8086**, proceda à execução do comando **file/open**. Certifique-se de que está em aberto a pasta que foi usada para a gravação da biblioteca. Em seguida, forneça para o campo **Nome do arquivo** o nome **biblio.inc** e acione o botão **Abrir**.

Escreva a rotina de macro a seguir para o arquivo de biblioteca **BIBLIO.inc**. O código em questão deve ser informado a partir da linha de código **08**, logo abaixo da rotina **ESCREVA**, deixando uma linha em branco:

```
POSICAO MACRO linha, coluna
    PUSH AX
    PUSH DX
    MOV AH, linha
    MOV AL, coluna
    DEC AH
    DEC AL
    MOV DX, AX
    MOV AH, 02h
    INT 10h
    POP DX
    POP AX
ENDM
```

A macro **POSICAO** coloca o cursor em uma coordenada de tela fornecida como parâmetro de linha e de coluna. A tela do monitor de vídeo em modo texto é mapeada com 80 colunas (numeradas de **0d** até **79d**) e 24 linhas (numeradas de **0d** até **23d**). O parâmetro **linha** externamente aceita valores entre **1d** e **24d** e o parâmetro **coluna** externamente aceita valores entre **1d** e **80d**. Internamente os valores fornecidos pelos parâmetros **linha** e **coluna** são diminuídos em 1 pela ação da instrução **DEC**.

Na macro **POSICAO**, antes de executar qualquer ação, está sendo utilizada a instrução **PUSH** para guardar na pilha os valores atuais dos registradores gerais **AX** e **DX**. Isso se faz necessário porque a macro **POSICAO** é um elemento externo ao programa em operação. Essa macro utiliza os registradores gerais **AX** e **DX** para sua ação. Assim sendo, é melhor garantir que se há algum valor importante nos registradores gerais, ele fica armazenado na pilha e pode ser recuperado pela instrução **POP**.

A instrução **PUSH** tem por finalidade armazenar (escrever) na pilha um dado do tipo *word*. Essa ocorrência é utilizada quando se deseja armazenar na pilha temporariamente um dado a ser utilizado em seguida, como se esse efeito fosse o de passagem de parâmetro encontrado em linguagens de alto nível. As operações de escrita na pilha não podem ser realizadas com a definição de valores imediatos. A instrução **PUSH**, quando em operação, efetua o decremento de **02h** sobre o registrador **SP**, além de colocar o operando **MSB** (mais significativo) na posição **SS:[SP+1]** e o operando **LSB** (menos significativo) na posição **SS:[SP]**.

A instrução **POP** retira um dado do tipo *word* da pilha. As características aplicadas à instrução **POP** são as mesmas aplicadas à instrução **PUSH**. A instrução **POP**, quando em operação, efetua o incremento de **02h** sobre o registrador **SP**, além de receber para o operando **MSB** (mais significativo) o conteúdo da posição **SS:[SP+1]** e para o operando **LSB** (menos significativo) receber o conteúdo da posição **SS:[SP]**.

Após armazenar os valores dos registradores gerais **AX** e **DX**, a rotina movimenta os valores dos parâmetros **linha** e **coluna**, respectivamente, para os registradores **AH** (mais significativo) e **AL** (menos significativo). Depois diminuem em 1 os valores dos registradores **AH** e **AL**, ajustando o valor real da posição do cursor, e movimenta o conjunto de valores do registrador geral **AX** para o registrador geral **DX**.

É necessário que o registrador geral **DX** tenha na sua parte mais significativa (**DH**) o valor do parâmetro **linha** e na sua parte menos significativa (**DL**) o valor do parâmetro **coluna**. Depois o valor da função **02h** (responsável por posicionar o

cursor) é armazenado na parte mais significativa (**AH**) do registrador geral **AX** e executa-se a ação da instrução **INT 10h** para que o cursor seja propriamente posicionado.

Informe a rotina de macro a seguir para o arquivo de biblioteca **BIBLIO.inc**. O código em questão deve ser informado a partir da linha de código **22**, logo abaixo da rotina **POSICAO**, deixando uma linha em branco:

```
TECLE    MACRO
    PUSH    AX
    MOV     AH, 00h
    INT     16h
    POP     AX
ENDM
```

A macro **TECLE** coloca o teclado em modo de espera e aguarda até que alguma tecla seja acionada. No momento em que alguma tecla é acionada, a rotina é encerrada. Na macro **TECLE**, antes de executar qualquer ação, está sendo utilizada a instrução **PUSH** para guardar na pilha os valores atuais do registrador geral **AX**. Valor que será recuperado pela instrução **POP**.

Após armazenar o valor do registrador geral **AX** na pilha, o programa armazena o valor da função **00h** na parte mais significativa (**AH**) do registrador geral **AX** e executa a ação da instrução **INT 16h** para que o teclado entre em modo de espera. No momento em que uma tecla é acionada, o seu valor ASCII é armazenado no registrador **AH** e a rotina é encerrada.

Informe a rotina de macro a seguir para o arquivo de biblioteca **BIBLIO.inc**. O código em questão deve ser informado a partir da linha de código **29**, logo abaixo da rotina **TECLE**, deixando uma linha em branco:

```
CURSORG  MACRO
    PUSH    AX
    PUSH    CX
    MOV     AX, 0100h
    MOV     CX, 000Ah
    INT     10h
    POP     CX
    POP     AX
ENDM
```

Na macro **CURSORG**, antes de executar qualquer ação, está sendo utilizada a instrução **PUSH** para guardar na pilha os valores atuais dos registradores gerais **AX** e **CX**, os quais podem, posteriormente, ser recuperados pela instrução **POP**.

Após armazenar os valores dos registradores gerais **AX** e **CX**, a rotina movimenta os valores **0100h** e **000Ah** para os registradores gerais **AX** e **CX** e executa-se a ação da instrução **INT 10h** para que o cursor seja apresentado em um formato maior que o tradicional. O valor **0100h** permite manipular o cursor e o valor **000Ah** permite mudar seu tamanho para grande após a chamada da interrupção **10h**.

Informe a rotina de macro a seguir para o arquivo de biblioteca **BIBLIO.inc**. O código em questão deve ser informado a partir da linha de código **39**, logo abaixo da rotina **CURSORG**, deixando uma linha em branco:

```
CURSORP  MACRO
    PUSH    AX
    PUSH    CX
    MOV     AX, 0100h
    MOV     CX, 0506h
    INT     10h
    POP     AX
    POP     CX
ENDM
```

Na macro **CURSORP**, antes de executar qualquer ação, está sendo utilizada a instrução **PUSH** para guardar na pilha os valores atuais dos registradores gerais **AX** e **CX**, os quais podem, posteriormente, ser recuperados pela instrução **POP**.

Após armazenar os valores dos registradores gerais **AX** e **CX**, a rotina movimenta os valores **0100h** e **0506h** para os registradores gerais **AX** e **CX** e executa-se a ação da instrução **INT 010h** para que o cursor seja apresentado em seu formato tradicional. O valor **0100h** permite manipular o cursor e o valor **0506h** permite mudar seu tamanho para pequeno após a chamada da interrupção **10h**.

Defina todas as macros anteriores e grave o arquivo **BIBLIO.inc**. As Figuras 10.25 e 10.26 apresentam, respectivamente, o trecho de rotinas de macro a partir da linha **08** e a partir da linha **39**

```

01  ESCREVA  MACRO
02  PUSH    AX
03  MOV     AH, 9h
04  INT     21h
05  POP     AX
06  ENDM
07
08  POSICAO  MACRO linha, coluna
09  PUSH    AX
10  PUSH    DX
11  MOV     AH, linha
12  MOV     AL, coluna
13  DEC     AH
14  DEC     AL
15  MOV     DX, AX
16  MOV     AH, 02h
17  INT     10h
18  POP     DX
19  POP     AX
20  ENDM
21
22  TECLE    MACRO
23  PUSH    AX
24  MOV     AH, 00h
25  INT     16h
26  POP     AX
27  ENDM

```

Figura 10.25 - Biblioteca BIBLIO na ferramenta emu8086 - a partir linha 08.

```

28
29  CURSORG  MACRO
30  PUSH    AX
31  PUSH    CX
32  MOV     AX, 0100h
33  MOV     CX, 000Ah
34  INT     10h
35  POP     CX
36  POP     AX
37  ENDM
38
39  CURSORP  MACRO
40  PUSH    AX
41  PUSH    CX
42  MOV     AX, 0100h
43  MOV     CX, 0506h
44  INT     10h
45  POP     AX
46  POP     CX
47  ENDM
48

```

Figura 10.26 - Biblioteca BIBLIO na ferramenta emu8086 - a partir linha 39.

Para fazer um teste de execução do conjunto de macros armazenadas em uma biblioteca, execute o comando de menu **file/new** e escolha qualquer uma das opções. Acione simultaneamente as teclas **<Ctrl> + <A>** para selecionar o texto atual, em seguida acione a tecla **** para remover o texto existente e na área de edição em branco e codifique o programa seguinte:

```

TITLE    Teste de Segmento 6

#MAKE_EXE#

INCLUDE 'biblio.inc'

DADOS  SEGMENT 'DATA'
        msg0 DB "Tecle algo para prosseguir", 0Dh, 0Ah, 24h
        msg1 DB "Ola, Mundo 1", 0Dh, 0Ah, 24h
        msg2 DB "Ola, Mundo 2", 0Dh, 0Ah, 24h
        msg3 DB "Ola, Mundo 3", 0Dh, 0Ah, 24h
DADOS  ENDS

PILHA  SEGMENT STACK 'STACK'
        DW 0100h DUP(?)
PILHA  ENDS

CODIGO SEGMENT 'CODE'
        ASSUME CS:CODIGO, DS:DADOS, SS:PILHA
        INICIO PROC FAR
            MOV    AX, DADOS
            MOV    DS, AX
            MOV    ES, AX

            POSICAO 03, 05
            MOV    DX, OFFSET msg1
            ESCREVA

            POSICAO 20, 01
            MOV    DX, OFFSET msg0
            ESCREVA
            TECLE

            POSICAO 07, 10
            MOV    DX, OFFSET msg2
            ESCREVA

            POSICAO 22, 01
            MOV    DX, OFFSET msg0
            ESCREVA
            TECLE

            POSICAO 11, 40
            MOV    DX, OFFSET msg3
            ESCREVA

            POSICAO 24, 01
            MOV    DX, OFFSET msg0
            ESCREVA
            CURSORG
            TECLE
            CURSORM

            MOV    AH, 4Ch
            INT    21h
            RET

```

```

INICIO ENDP
CODIGO ENDS
END INICIO

```

Execute os comandos de menu **file/save** e grave o programa com o nome **SEGM6.asm**. A Figura 10.27 apresenta a imagem do resultado da execução do programa.



Figura 10.27 - Resultado da execução do programa.

10.4 - Apresentação de Valores Negativos

Todos os programas utilizados anteriormente manipularam apenas valores positivos. Quanto aos valores negativos, são estes calculados e armazenados na memória com o complemento por dois.

Neste tópico, utilizando o comando **NEG** (*Negate*), é possível transformar valores negativos em positivos e vice-versa. O comando **NEG** afeta o comportamento de alguns registradores de estado, destacando-se entre eles o **SF**.

Quando uma operação de subtração ocorre entre dois valores (comando **SUB**), o registrador de estado **SF** é afetado. Se for executada uma operação de $9 - 7$, o resultado obtido na memória será **2** e o registrador de estado **SF** será sinalizado com valor **0** (sem efeito). Se for realizada a operação $7 - 9$, o resultado obtido na memória será **FEh** (tipo *byte*) ou **FFFEh** (tipo *word*) e o registrador de estado **SF** será sinalizado com o valor **1**, indicando que o valor armazenado na memória é negativo com base no complemento por dois.

O valor **FEh** pode tanto ser **-2d** como o valor **254d** (algo semelhante a esse efeito havia sido apresentado em capítulos anteriores). A forma de interpretação do valor vai depender da análise do registrador de estado **SF**. Se quiser interpretar o valor **FEh** como **-2d**, é necessário considerar o valor **1** do registrador de estado **SF**. Caso contrário, se não levar em conta o registrador de estado **SF** estar com **1** ou com **0**, o valor **FEh** pode ser interpretado como **254d**.

Tome como base um programa que faça a subtração de dois valores numéricos de apenas um dígito lidos no teclado. Observe o código do programa seguinte:

```
TITLE   Teste de Segmento 7
```

```
#MAKE_EXE#
```

```
DADOS  SEGMENT 'DATA'
```

```
msg1 DB 'Entre valor 1 (de 0 a 9) .....: ', 24h
```

```
msg2 DB 0Dh, 0Ah, 'Entre valor 2 (de 0 a 9) .....: ', 24h
```

```
msg3 DB 0Dh, 0Ah, 'Resultado .....: ', 24h
```

```
msg4 DB 0Dh, 0Ah, 'Valor invalido', 24h
```

```
DADOS  ENDS
```

```

PILHA SEGMENT STACK 'STACK'
    DW 0100h DUP(?)
PILHA ENDS

CODIGO SEGMENT 'CODE'
    ASSUME CS:CODIGO, DS:DADOS, SS:PILHA
    INICIO PROC FAR
        MOV     AX, DADOS
        MOV     DS, AX
        MOV     ES, AX

        MOV     DX, OFFSET msg1
        MSG
        CALL    entrada
        MOV     BH, AL

        MOV     DX, OFFSET msg2
        MSG
        CALL    entrada
        MOV     BL, AL

        MOV     DX, OFFSET msg3
        MSG
        SUB     BH, BL
        JS      negativo
        JGE     positivo
    negativo:
        NEG     BH
        MOV     AL, 2Dh
        MOV     AH, 0Eh
        INT     10h
        JMP     mostra

    positivo:
        JMP     mostra

    mostra:
        MOV     AL, BH
        MOV     DL, AL
        ADD     AL, 30h
        MOV     AH, 0Eh
        INT     10h
        FIM
        RET
    INICIO ENDP
CODIGO ENDS

fim MACRO
    MOV     AH, 4Ch
    INT     21h
ENDM

msg MACRO
    MOV     AH, 09h
    INT     21h
ENDM

entrada PROC NEAR
    MOV     AH, 01h
    INT     21h

```

```

CMP     AL, 30h
JL      erro
CMP     AL, 40h
JGE     erro
JMP     fim_validacao
erro:
    LEA     DX, msg4
    MSG
    FIM
fim_validacao:
SUB     AL, 30h
RET
entrada ENDP

```

END INICIO

Acione simultaneamente as teclas <Ctrl> + <A> para selecionar o texto atual, em seguida acione a tecla para remover o texto existente na área de edição em branco. Codifique o programa anterior e com os comandos de menu **file/save** grave-o com o nome **SEGM7**. As Figuras 10.28 (resposta positiva) e 10.29 (resposta negativa) apresentam, respectivamente, o resultado para os valores 9 e 2; 2 e 9.

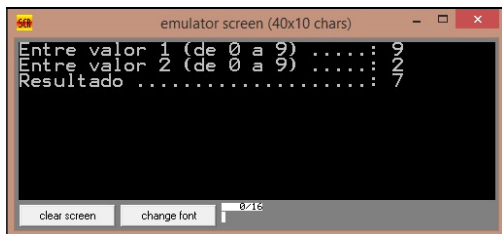


Figura 10.28 - Resultado positivo.

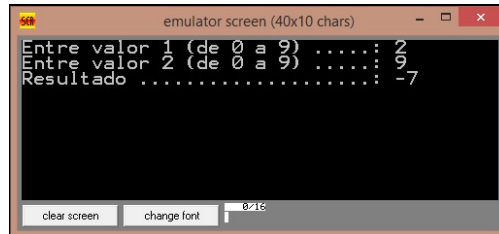


Figura 10.29 - Resultado negativo.

Como vários detalhes do programa são conhecidos e foram anteriormente explicados, eles não necessitam ser revistos, mas é pertinente considerar os detalhes ainda desconhecidos. As Figuras 10.30, 10.31 e 10.32 apresentam a imagem do programa completo.

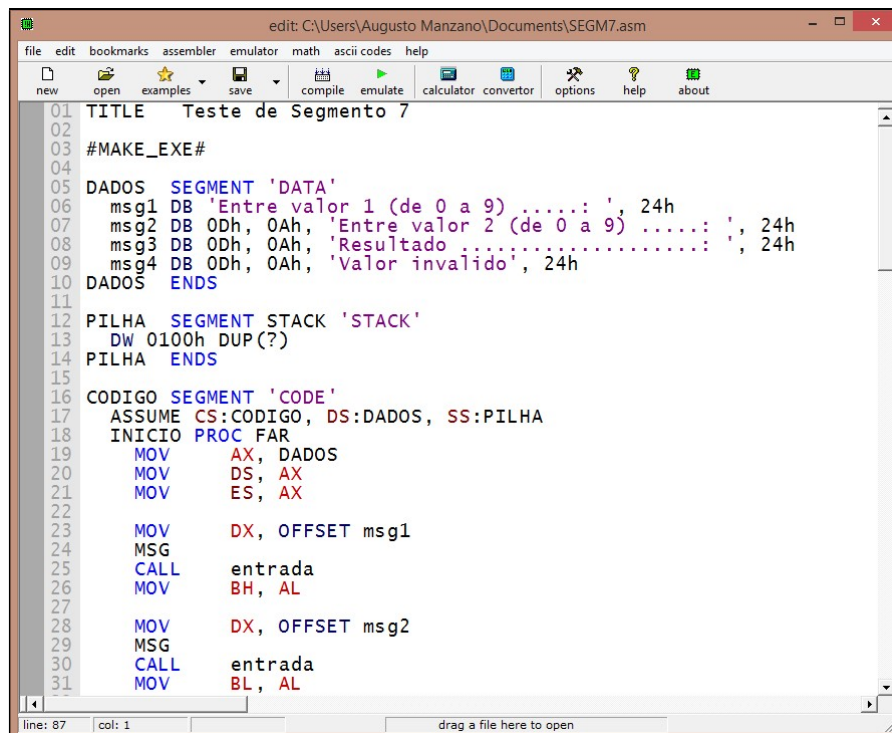


Figura 10.30 - Programa SEGM7 na ferramenta emu8086 - parte 1 (linhas 01 até 31).

```

32
33     MOV     DX, OFFSET msg3
34     MSG
35     SUB     BH, BL
36     JS      negativo
37     JGE     positivo
38     negativo:
39     NEG     BH
40     MOV     AL, 2Dh
41     MOV     AH, 0Eh
42     INT     10h
43     JMP     mostra
44
45     positivo:
46     JMP     mostra
47
48     mostra:
49     MOV     AL, BH
50     MOV     DL, AL
51     ADD     AL, 30h
52     MOV     AH, 0Eh
53     INT     10h
54     FIM
55     RET
56     INICIO ENDP
57     CODIGO ENDS
58
59     fim MACRO
60     MOV     AH, 4Ch
61     INT     21h
62     ENDM

```

Figura 10.31 - Programa SEG7 na ferramenta emu8086 - parte 2 (linhas 32 até 62).

```

63
64     msg MACRO
65     MOV     AH, 09h
66     INT     21h
67     ENDM
68
69     entrada PROC NEAR
70     MOV     AH, 01h
71     INT     21h
72     CMP     AL, 30h
73     JL      erro
74     CMP     AL, 40h
75     JGE     erro
76     JMP     fim_validacao
77     erro:
78     LEA     DX, msg4
79     MSG
80     FIM
81     fim_validacao:
82     SUB     AL, 30h
83     RET
84     entrada ENDP
85
86     END INICIO
87

```

Figura 10.32 - Programa SEG7 na ferramenta emu8086 - parte 3 (linhas 63 até 87).

Na Figura 10.31 considere a explanação da ação de algumas linhas do programa. Na linha **35** ocorre a instrução **SUB BH, BL**. Nesse caso, se o valor do registrador mais significativo **BH** for menor que o valor do registrador menos significativo **BL**, ocorre a alteração do registrador de estado **SF** para **1**, indicando que o valor armazenado no registrador mais significativo é negativo.

Na linha **36** na instrução **JS negativo**, verifica-se se o valor do registrador de estado **SF** é **1**. Nesse caso, se **SF** for **1**, o programa desvia para o trecho **negativo** definido a partir da linha **39**.

Na linha 37 na instrução **JGE positivo**, verifica-se se o valor do registrador mais significativo **BH** é igual ou maior que o registrador menos significativo **BL**. Caso a instrução **JGE** resulte um valor lógico verdadeiro, o programa desvia para o trecho **positivo** definido a partir da linha 46.

O trecho de código a partir da linha 46 apresenta o valor armazenado em memória. Para estar nesse trecho, a instrução **JS negativo** da linha 36 foi executada, pois o registrador de estado **SF** está com o valor 1. Primeiramente na linha 40 é utilizada a instrução **NEG BH**, que transforma o valor negativo (complemento de dois) em seu equivalente positivo. A instrução **NEG** muda o valor de positivo para negativo e de negativo para positivo quando for necessário, alterando o valor armazenado na memória, utilizando o complemento por dois. Nesse caso, o valor do registrador mais significativo **BH** é **F9h**. A instrução **NEG BH** da linha 40 transforma o valor **F9h** no valor 7.

A linha 41 movimenta o valor **2Dh** para o registrador menos significativo **AL** que é o código ASCII do símbolo de sinal negativo, em seguida as linhas 41 e 43 apresentam o sinal na tela do monitor de vídeo. Na sequência (linha 44), o programa chama o trecho identificado pelo rótulo **mostra** e apresenta o valor numérico armazenado na memória, que nesse caso será 7.

10.5 - Biblioteca de Funções Externas

O programa **emu8086** oferece uma biblioteca de funções externas genéricas comuns e muito utilizadas em vários programas a serem desenvolvidos em linguagem de programação *Assembly 8086*.

A biblioteca padrão do programa denominada **emu8086.inc** é um arquivo de programa em formato texto com a definição interna de *macros* e *procedimentos*. Para o efetivo uso da biblioteca dentro de um programa em desenvolvimento, é necessário usar a diretiva **INCLUDE** (como demonstrado anteriormente).

O arquivo de biblioteca pode ter qualquer tipo de extensão. No caso do programa **emu8086**, é aconselhável que o arquivo de biblioteca possua a extensão **.inc** e de preferência esteja gravado na pasta **Inc** dentro do diretório do programa **C:\emulbinaries\inc**. Se não estiver no diretório **inc**, o arquivo de biblioteca deve estar na mesma pasta em que o programa em desenvolvimento está gravado.

Neste tópico são estudadas as funções existentes na biblioteca **emu8086.inc**. Segundo o próprio Alexander Popov, desenvolvedor do produto, é possível que o usuário da ferramenta (aluno) não entenda todo o conteúdo e recursos disponibilizados pela biblioteca. No entanto, é importante saber o que a biblioteca faz.

O autor da ferramenta acrescenta que para utilizar qualquer uma das funções disponibilizadas é necessário colocar no início do programa-fonte em desenvolvimento a linha **INCLUDE 'emu8086.inc'**.

O arquivo de biblioteca **emu8086.inc** do programa **emu8086** tem funções agrupadas em duas categorias operacionais, sendo funções de macro e funções de procedimento.

As funções de macro são as seguintes:

- ◆ **PUTC caractere** - macro com a definição de um parâmetro, que apresenta como saída o caractere informado como parâmetro na posição atual do cursor.
- ◆ **PRINT mensagem** - macro com a definição de um parâmetro, que apresenta uma mensagem na tela, mantendo o cursor na mesma linha de apresentação.
- ◆ **PRINTN mensagem** - macro com a definição de um parâmetro, que mostra uma mensagem na tela. Após a apresentação o cursor é posicionado na próxima linha.
- ◆ **CURSOROFF** - macro sem a definição de parâmetros, que desabilita a apresentação do cursor na tela.
- ◆ **CURSORON** - macro sem a definição de parâmetros, que habilita a apresentação do cursor na tela.
- ◆ **GOTOXY coluna, linha** - macro com a definição de dois parâmetros, que coloca o cursor em uma determinada coordenada de tela.

As funções de procedimento são as seguintes:

- ◆ **SCAN_NUM** - obtém um valor numérico com mais de um dígito, podendo ser positivo ou negativo. Antes da diretiva **END** é necessário utilizar a declaração **DEFINE_SCAN_NUM**.
- ◆ **PRINT_STRING** - procedimento usado para imprimir mensagem terminada com caractere nulo na posição atual do cursor. Antes da diretiva **END** é necessário utilizar a declaração **DEFINE_PRINT_STRING**.

- ◆ **PTHIS** - utilizado para imprimir mensagem terminada com caractere nulo na posição atual do cursor, semelhante ao procedimento **PRINT_STRING**, mas com a diferença de receber o endereço da sequência de caracteres que formam a mensagem a partir da pilha. Para sequências de caracteres terminadas em zero, é necessário que a definição da mensagem ocorra logo após a chamada do procedimento. Antes da diretiva **END** é necessário utilizar a declaração **DEFINE_PTHIS**.
- ◆ **GET_STRING** - lê do teclado uma sequência de caracteres terminada em nulo. Esse procedimento é interrompido quando utilizada a tecla <Enter>. Antes da diretiva **END** é necessário utilizar a declaração **DEFINE_GET_STRING**.
- ◆ **CLEAR_SCREEN** - limpa a tela e posiciona o cursor no seu topo esquerdo. Antes da diretiva **END** é necessário utilizar a declaração **DEFINE_CLEAR_SCREEN**.
- ◆ **PRINT_NUM** - apresenta um valor numérico negativo. Antes da diretiva **END** é necessário utilizar as declarações **DEFINE_PRINT_NUM** e **DEFINE_PRINT_NUM_UN**.
- ◆ **PRINT_NUM_UN** - apresenta um valor numérico positivo. Antes da diretiva **END** é necessário utilizar as declarações **DEFINE_PRINT_NUM** e **DEFINE_PRINT_NUM_UN**.

Ao fazer a compilação de um programa-fonte com a chamada de uma determinada biblioteca, a ferramenta **emu8086** localiza a macro ou procedimento na biblioteca e a executa no programa-fonte. As estruturas de rotinas macros e procedimentos possuem vantagens e desvantagens.

No caso de macros ocorre a substituição no programa-fonte do código da macro pelo código real. Assim sendo, o programa-fonte torna-se maior. Caso o uso de uma macro seja muito frequente, o programa pode também ficar enorme. Nesse caso é aconselhável utilizar procedimentos. No entanto, as macros possibilitam o uso de parâmetros externos e os procedimentos não.

No caso de usar procedimentos, o compilador processa as declarações sem copiá-las para o código-fonte. Nesse caso, o compilador, quando encontra a instrução **CALL**, substitui o nome de procedimento com o seu endereço de memória, onde o procedimento foi declarado, efetua a sua ação e retorna após sua chamada quando encontra a instrução **RET**.

Para exemplificar o uso da biblioteca **emu8086.inc**, considere o programa seguinte:

```
;*****
;*      Programa: BIBLIOT1.ASM      *
;*****

INCLUDE 'emu8086.inc'

#MAKE_EXE#

.DATA
    msg1 DB 'Entrada e Apresentacao', 0d
    msg2 DB 'Tecle algo para continuar...', 0d
    msg3 DB 'Entre um valor numerico ...: ', 0d

.CODE

    ; Desabilita cursor
    CURSOROFF

    ; Poe cursor na Coluna = 30 / Linha = 12
    ; armazena msg1 em SI
    ; apresenta o conteudo de msg1 na tela

    GOTOXY    29d, 11d
    PRINT     'Programa para Teste'

    ; Poe cursor na Coluna = 01 / Linha = 24
    ; armazena msg2 em SI
    ; apresenta o conteudo de msg2 na tela

    GOTOXY    00d, 23d
    LEA       SI, msg2
```

```

CALL      print_string

; Aguarda que algo seja teclado

MOV       AH, 00h
INT       16h

; Habilita cursor

CURSORON

; Limpa a tela

CALL      clear_screen

; Poe cursor na Coluna = 29 / Linha = 01
; armazena msg1 em SI
; apresenta o conteudo de msg1 na tela

GOTOXY    28d, 00d
LEA       SI, msg1
CALL      print_string

; Poe cursor na Coluna = 01 / Linha = 05
; armazena msg3 em SI
; apresenta o conteudo de msg3 na tela

GOTOXY    00d, 04d
LEA       SI, msg3
CALL      print_string

; Efetua a entrada de um valor numerico em CX

CALL      scan_num

; Transfere o valor de CX para AX

MOV       AX, CX

; Apresenta mensagem de saida

CALL      pthis
          DB 0Dh, 0Ah, 'Foi fornecido o valor .....: ', 0d

; Apresenta o valor armazenado em AX
CALL      print_num

; Finaliza programa

INT       20h

; Rotulos de definicao das operacoes da biblioteca
DEFINE_SCAN_NUM
DEFINE_PRINT_STRING
DEFINE_PRINT_NUM
DEFINE_PRINT_NUM_UN
DEFINE_PTHIS
DEFINE_CLEAR_SCREEN
END

```

Execute no programa **emu8086** o comando de menu **file/new/com template** e a partir da linha **06** do editor de texto escreva o programa anterior, gravando-o com o nome **BIBLIOT1.asm**.

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **File/Save** com o nome **BIBLIOT1**.

Execute o programa com o comando de menu **assembler/compile and load in the emulator**. Acione o botão **run** ou a tecla de função **<F9>** para executar o programa e observar todas as ocorrências de funcionalidade inseridas pela biblioteca **emu8086.inc**.

A seguir apresenta-se uma rápida descrição da funcionalidade de cada trecho do programa. Explicações mais detalhadas sobre a funcionalidade de cada recurso da biblioteca encontram-se no próximo tópico.

```
INCLUDE 'emu8086.inc'
```

Neste trecho o código faz referência ao uso da biblioteca **emu8086.inc**. Se essa linha for omitida, ocorrem erros de compilação em todos os trechos em que houver a menção de qualquer um dos recursos externos em uso.

```
CURSOROFF
```

O trecho de programa com a chamada da macro anterior desabilita o cursor. Desta forma, o cursor não será apresentado.

```
; Poe cursor na Coluna = 30 / Linha = 12
; armazena msg1 em SI
; apresenta o conteudo de msg1 na tela
```

```
GOTOXY    29d, 11d
PRINT     'Programa para Teste'
```

As duas linhas indicam o uso das macros que posicionam o cursor na tela do monitor de vídeo e apresentam a mensagem **Programa para Teste**. Observe que é usado na linha **GOTOXY** um valor menor do que o desejado para posicionamento.

```
; Poe cursor na Coluna = 01 / Linha = 24
; armazena msg2 em SI
; apresenta o conteudo de msg2 na tela
```

```
GOTOXY    00d, 23d
LEA      SI, msg2
CALL     print_string
```

O trecho anterior na linha **LEA SI, msg2** transfere o valor de endereçamento da área de dados em que se encontra definida a variável **msg2** para o registrador de apontamento **SI** (*source index*). Na sequência a instrução **CALL** chama o procedimento **print_string** existente na biblioteca **emu8086.inc**.

```
; Aguarda que algo seja teclado
```

```
MOV      AH, 00h
INT      016h
```

O trecho anterior já havia sido usado anteriormente e tem por finalidade fazer uma pausa no teclado e colocá-lo em modo de espera até que alguma tecla seja acionada. Com isso aparece uma tela de saudação do programa.

```
; Habilita cursor
```

```
CURSORON
```

Após a apresentação da tela de saudação, o cursor é novamente habilitado.

```
; Limpa a tela

CALL      clear_screen
```

Na sequência da execução de eventos o programa limpa a tela de saudação e deixa-a em branco para nova apresentação.

```
; Poe cursor na Coluna = 29 / Linha = 01
; armazena msg1 em SI
; apresenta o conteudo de msg1 na tela
```

```
GOTOXY    28d, 00d
LEA        SI, msg1
CALL       print_string
```

```
; Poe cursor na Coluna = 01 / Linha = 05
; armazena msg3 em SI
; apresenta o conteudo de msg3 na tela
```

```
GOTOXY    00d, 04d
LEA        SI, msg3
CALL       print_string
```

O trecho anterior faz a apresentação das mensagens armazenadas nas variáveis **msg1** **msg3** de acordo com o endereço de apontamento do registrador **SI**. A mensagem da variável **ms3** indica a entrada de um valor numérico.

```
; Efetua a entrada de um valor numerico em CX
```

```
CALL       scan_num
```

A instrução **CALL scan_num** executa a chamada do procedimento que faz o tratamento da entrada de um valor numérico que pode ser positivo ou negativo.

```
; Transfere o valor de CX para AX
```

```
MOV        AX, CX
```

Após fazer a entrada do valor numérico e proceder à sua ação de tratamento, por meio da linha anterior o programa transfere o conteúdo do registrador geral **CX** para o registrador geral **AX**. O registrador geral **CX** foi usado como uma área de armazenamento temporário do valor informado anteriormente.

```
; Apresenta mensagem de saída
```

```
CALL       pthis
DB 0Dh, 0Ah, 'Foi fornecido o valor .....: ', 0d
```

```
; Apresenta o valor armazenado em AX
```

```
CALL       print_num
```

Por fim o programa chama o procedimento **pthis** que apresenta a mensagem **Foi fornecido o valor**, e depois apresenta o valor propriamente dito por meio da chamada do procedimento **print_num**.

```
; Finaliza programa
```

```
INT        020h
```

O trecho anterior faz o término de um programa **.COM**, como descrito anteriormente em outros capítulos.

```

; Rotulos de definicao das operacoes da biblioteca
DEFINE_SCAN_NUM
DEFINE_PRINT_STRING
DEFINE_PRINT_NUM
DEFINE_PRINT_NUM_UN
DEFINE_PTHIS
DEFINE_CLEAR_SCREEN
END

```

Inicialmente, antes da diretiva **END** define-se o fim das diretivas do código da biblioteca **emu8086.inc**.

10.6 - Manipulação de Cadeias

Na programação *Assembly* encontra-se alguns comandos exclusivos para a manipulação de cadeias (*strings*) de caracteres, tais como: **MOVSB**, **MOVSW**, **CMPSB**, **CMPSW**, **SCASB** e **SCASW**. Esses comandos para serem operacionalizados fazem uso dos registradores de índice **SI** e **DI** que dependendo do estado da definição do valor do registrador de direção **DF** podem incrementar ou decrementar o deslocamento dos comandos de manipulação de cadeias na memória. O incremento ou decremento ocorre a partir do formato da instrução, tamanho do operando ou do estado do *flag* de direção representado pelo registrador **DF**. O valor do incremento e decremento será de **1** para a manipulação de um *byte* e será de **2** para a manipulação de um *word* (OLIVEIRA, 2013 & LITERÁK, 2013). Se o valor do registrador **DF** for **0** ocorrerá incremento, se o valor for **1** ocorrerá decremento. O registrador **DF** pode ser configurado com os comandos **STD** para definir valor **1** (sentido da direita para à esquerda) e **CLD** para definir valor **0** (sentido da esquerda para à direita).

Os comandos **MOVSB** e **MOVSW** movimentam um *byte* (**MOVSB**) ou um *word* (**MOVSW**) da fonte endereçada pelos registradores de origem **SI:DI** para o destino endereçado pelos registradores **ES:DI** sem afetar nenhum registrador de estado (*flags*) e atualiza os registradores de índice **SI** e **DI** a partir do tamanho do operando ou instrução em uso. Os registradores de índice **SI** e **DI** são incrementados quando o sinalizador de direção é limpo e decrementado quando o sinalizador de direção está definido (LITERÁK, 2013). A operação de movimentação copia os dados da origem para o destino. Os comandos **MOVSB** e **MOVSW** são referenciados como comandos **MOVS** (*MOVE Strings*) e os dados manipulados são definidos em variáveis.

Para demonstrar a ação de cópia de conteúdo de uma posição de memória para outra posição considere um programa que peça a leitura de uma primeira sequência de caracteres, armazene-a em uma variável e mostre-a. Depois peça a leitura da segunda sequência de caracteres, armazene-a em outra variável e mostre-a. O programa deve copiar o conteúdo da variável da primeira entrada na variável da segunda entrada e mostrar o resultado da ocorrência. Assim sendo, execute o comando de menu **file/new** e escolha qualquer uma das opções. Acione simultaneamente as teclas **<Ctrl> + <A>** para selecionar o texto atual, em seguida acione a tecla **** para remover o texto existente e na área de edição em branco e codifique o programa seguinte, gravando-o com o nome **MANIPCAD01** e atente para o trecho em negrito:

```

;*****
;*   Programa: MANIPCAD01.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA

msg1e DB 'Entre a 1a. cadeia de caracteres .....: ', 0
msg2e DB 'Entre a 2a. cadeia de caracteres .....: ', 0
msg1s DB '1a. cadeia de caracteres .....: ', 0
msg2s DB '2a. cadeia de caracteres .....: ', 0
msg3s DB 'Agora 2a. cadeia de caracteres possui ...: ', 0

```

```
cadeia1 DB 30d DUP ('x')
cadeia2 DB 30d DUP ('x')
```

```
.CODE
```

```
; Ajuste do acesso a memoria
```

```
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
```

```
; Entrada da primeira sequencia de caracteres
```

```
LEA     SI, msg1e
CALL    PRINT_STRING
LEA     DI, cadeia1
MOV     DX, 30d
CALL    GET_STRING
PUTC    13d
PUTC    10d
```

```
; Saida da primeira sequencia de caracteres em cadeia1
```

```
LEA     SI, msg1s
CALL    PRINT_STRING
MOV     SI, DI
CALL    PRINT_STRING
PUTC    13d
PUTC    10d
```

```
; Entrada da segunda sequencia de caracteres
```

```
LEA     SI, msg2e
CALL    PRINT_STRING
LEA     DI, cadeia2
MOV     DX, 30d
CALL    GET_STRING
PUTC    13d
PUTC    10d
```

```
; Saida da segunda sequencia de caracteres em cadeia2
```

```
LEA     SI, msg2s
CALL    PRINT_STRING
MOV     SI, DI
CALL    PRINT_STRING
PUTC    13d
PUTC    10d
```

```
; Operacao de copia da fonte para o destino
```

```
LEA SI, cadeia1 ; fonte
LEA DI, cadeia2 ; destino
CLD             ; DF = 0
MOV CX, 30d     ; tamanho da cadeia
REP MOVSB       ; repete cada caractere ate 30
```



```

; Saida da primeira sequencia de caracteres em cadeia2

LEA    SI, msg3s
CALL   PRINT_STRING
LEA    DI, cadeia2
MOV    DX, 30d
MOV    SI, DI
CALL   PRINT_STRING
PUTC   13d
PUTC   10d

INT     20h

DEFINE_PRINT_STRING
DEFINE_GET_STRING
END

```

Os detalhes gerais do programa anterior já são conhecidos. Assim, atente para o trecho em negrito e observe a transferência dos conteúdos das variáveis **cadeia1** e **cadeia2** respectivamente para os registradores de índice **SI** e **DI** por meio da instrução **LEA**. Após configurar o acesso as variáveis é executada a instrução **CLD** que coloca o registrador **DF** em **0** e estabelece que a leitura dos caracteres nas variáveis serão procedidos da esquerda para a direita. A linha de instrução **MOV CX, 30d** transfere para o registrador geral **CX** o valor decimal **30** que informa a quantidade máxima de caracteres das variáveis em uso e a instrução **REP MOVSB** faz a repetição (**REP**) do comando **MOVSB** por trinta vezes até percorrer toda a extensão do tamanho das variáveis em uso.

Os comandos **CMPSB** e **CMPSW** são usados para comparar um *byte* (**CMPSB**) ou um *word* (**CMPSW**) efetuando uma subtração entre o *byte* ou *word* endereçado no registrador **ES:DI** de destino em relação a origem endereçada no registrador **DS:SI** dentro do segmento de dados, sem devolver o resultado da subtração efetivada, mas afetando os registradores de estado **AF**, **CF**, **OF**, **PF**, **SF** e **ZF** (OLIVEIRA, 2013). O incremento e decremento para **CMPSB** ocorrerá em 1 e o incremento e decremento para **CMPSW** ocorrerá em 2 (LITERÁK, 2013). Os comandos **CMPSB** e **CMPSW** são referenciados como instruções **CMPS** (*CoMPare Strings*) e os dados manipulados são definidos em variáveis.

Para demonstrar a ação de comparação de conteúdo de uma posição de memória para outra posição considere um programa que peça a leitura de uma primeira sequência de caracteres e armazene-a em uma variável. Depois peça a leitura da segunda sequência de caracteres e armazene-a. O programa deve comparar os conteúdos informados e apresentar mensagem informando se são iguais ou diferentes. Assim sendo, execute o comando de menu **file/new** e escolha qualquer uma das opções. Acione simultaneamente as teclas **<Ctrl> + <A>** para selecionar o texto atual, em seguida acione a tecla **** para remover o texto existente e na área de edição em branco e codifique o programa seguinte, gravando-o com o nome **MANIPCAD02** e atente para o trecho em negrito:

```

;*****
;*   Programa: MANIPCAD02.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA

msg1e DB 'Entre a 1a. cadeia de caracteres .....: ', 0
msg2e DB 'Entre a 2a. cadeia de caracteres .....: ', 0
msg1s DB '1a. cadeia de caracteres .....: ', 0
msg2s DB '2a. cadeia de caracteres .....: ', 0
msg3s DB 'As cadeias sao iguais.', 0
msg4s DB 'As cadeias sao diferentes.', 0

cadeia1 DB 30d DUP ('x')
cadeia2 DB 30d DUP ('x')

```

.CODE

; Ajuste do acesso a memoria

```
MOV AX, @DATA
MOV DS, AX
MOV ES, AX
```

; Entrada da primeira sequencia de caracteres

```
LEA SI, msg1e
CALL PRINT_STRING
LEA DI, cadeia1
MOV DX, 30d
CALL GET_STRING
PUTC 13d
PUTC 10d
```

; Saida da primeira sequencia de caracteres em cadeia1

```
LEA SI, msg1s
CALL PRINT_STRING
MOV SI, DI
CALL PRINT_STRING
PUTC 13d
PUTC 10d
```

; Entrada da segunda sequencia de caracteres

```
LEA SI, msg2e
CALL PRINT_STRING
LEA DI, cadeia2
MOV DX, 30d
CALL GET_STRING
PUTC 13d
PUTC 10d
```

; Saida da segunda sequencia de caracteres em cadeia2

```
LEA SI, msg2s
CALL PRINT_STRING
MOV SI, DI
CALL PRINT_STRING
PUTC 13d
PUTC 10d
```

; Operacao de comparacao da fonte com o destino

```
LEA SI, cadeia1 ; fonte
LEA DI, cadeia2 ; destino
CLD ; DF = 0
MOV CX, 30d ; tamanho da cadeia
REPE CMPSB ; repete cada caractere ate 30
```

JL difere

```
LEA SI, msg3s
CALL PRINT_STRING ; escreve mensagem para condicao verdadeira
JMP fim
```

difere:

```
LEA SI, msg4s
CALL PRINT_STRING ; escreve mensagem para condicao falsa
```

```

fim:
PUTC    13d
PUTC    10d

INT      20h

DEFINE_PRINT_STRING
DEFINE_GET_STRING
END

```

Note que este programa no trecho em negrito muito se assemelha ao trecho em negrito do programa anterior. Observe que são trechos muito semelhantes, exceto pelo uso do comando **CMP**SB. Perceba que está se fazendo uso das instruções de salto no tratamento da condição para apresentar a mensagem devida a condição ocorrida.

Os comandos **SCASB** e **SCASW** são usados para comparar um *byte* (SCASB) ou um *word* (SCASW) de um caractere que esteja indicado no registrador de origem **AL** (para SCASB) ou **AX** (para SCASW) com o dado apontado no endereço de destino dos registradores **ES:DI**, afetando os registradores de estado **AF**, **CF**, **OF**, **PF**, **SF** e **ZF**. Essas instruções não efetuam a subtração, mas afetam os *flags* como se tivesse realizada a subtração. (LITERÁK, 2013).

O programa seguinte solicita a entrada de uma cadeia e a entrada de um caractere e apresenta mensagens informando se o caractere informado existe ou não na cadeia informada. Assim sendo, execute o comando de menu **file/new** e escolha qualquer uma das opções. Acione simultaneamente as teclas **<Ctrl> + <A>** para selecionar o texto atual, em seguida acione a tecla **** para remover o texto existente e na área de edição em branco e codifique o programa seguinte, gravando-o com o nome **MANIPCAD03** e atente para o trecho em negrito:

```

;*****
;*   Programa: MANIPCAD03.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA

msg1e DB 'Entre uma cadeia de caracteres ....: ', 0
msg2e DB 'Entre apenas um caractere .....: ', 0
msg3s DB 'A cadeia possui o caractere informado.', 0
msg4s DB 'O caractere nao existe na cadeia.', 0

cadeia1 DB 30d DUP ('x')
cadeia2 DB 01d DUP ('x')

.CODE

; Ajuste do acesso a memoria

MOV AX, @DATA
MOV ES, AX

; Entrada da primeira sequencia de caracteres

LEA     SI, msg1e
CALL    PRINT_STRING
LEA     DI, cadeia1
MOV     DX, 30d
CALL    GET_STRING
PUTC    13d
PUTC    10d

```

; Entrada apenas do caractere

```
LEA    SI, msg2e
CALL   PRINT_STRING
LEA    DI, cadeia2
MOV    DX, 30d
CALL   GET_STRING
PUTC   13d
PUTC   10d
```

; Operacao de comparacao da fonte com o destino

```
LEA DI, cadeia1 ; fonte
MOV AL, cadeia2 ; destino
CLD             ; DF = 0
MOV CX, 30d     ; tamanho da cadeia
REPE SCASB      ; repete cada caractere ate 30

JGE difere
LEA SI, msg3s
CALL PRINT_STRING ; escreve mensagem para condicao verdadeira
JMP fim
difere:
LEA SI, msg4s
CALL PRINT_STRING ; escreve mensagem para condicao falsa
fim:
PUTC 13d
PUTC 10d

INT 20h

DEFINE_PRINT_STRING
DEFINE_GET_STRING
END
```

Além das instruções de manipulação de cadeias apresentadas existem outras instruções que não são tratadas aqui. No entanto, o leitor poderá a partir desta base ampliar seu conhecimento pesquisando a respeito de outras instruções com esta finalidade.

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69

Capítulo 11

RECURSOS ESPECÍFICOS

Este capítulo aborda as funções da biblioteca externa que acompanha o programa emu8086. São apresentados endereçamento e acesso à memória nos formatos direto, indireto com registrador, indexado, base indexada, base indexada com deslocamento e matrizes.

11.1 - Funcionalidades da biblioteca emu8086.inc

Anteriormente viu-se uma rápida descrição das funções existentes na biblioteca **emu8086.inc** encontrada na ferramenta **emu8086**. Cabe neste ponto um estudo mais detalhado de cada função em particular.

11.1.1 - Funções de macro

A biblioteca **emu8086.inc** apresenta seis macros que oferecem os seguintes recursos operacionais:

PUTC *caractere*

Função que apresenta como saída o caractere informado como parâmetro na posição atual do cursor. O caractere pode ser um valor em notação binária, decimal, octal ou hexadecimal correspondente a um valor da tabela ASCII. Pode também ser informado o caractere desejado entre aspas ou apóstrofes.

```
;*****
;*      Programa: BIBLIOT2.ASM      *
;*****
```

```
INCLUDE 'emu8086.inc'
```

```
.MODEL small
.STACK 512d
```

```
.CODE
```

```
    PUTC      65d
    PUTC      154o
    PUTC      06Fh
    PUTC      00100000b
    PUTC      "M"
    PUTC      165o
    PUTC      110d
    PUTC      064h
    PUTC      'o'
    INT       020h
END
```

O programa anterior exibe a mensagem **Alo Mundo**, constituída das diversas formas de definição de caracteres ASCII para a sua composição. Salve o programa com o nome **BIBLIOT2.asm**.

GOTOXY *coluna, linha*

Função que coloca o cursor em uma determinada coordenada de tela. É necessário levar em consideração que a tela em modo texto padrão tem 80 colunas (numeradas de 0 até 79) e 24 linhas (numeradas de 0 até 23). No momento de definir a posição, é necessário levar em consideração o valor da posição em si. Para posicionar o cursor na coluna um, linha um, devem ser utilizadas as coordenadas zero e zero.

```
;*****  
;*      Programa: BIBLIOT3.ASM      *  
;*****
```

```
INCLUDE 'emu8086.inc'
```

```
.MODEL small  
.STACK 512d
```

```
.CODE
```

```
    GOTOXY    0d, 0d  
    PUTC      'A'  
    GOTOXY    1d, 1d  
    PUTC      '\'  
    GOTOXY    2d, 2d  
    PUTC      'o'  
    GOTOXY    3d, 3d  
    PUTC      ' '  
    GOTOXY    4d, 4d  
    PUTC      'M'  
    GOTOXY    5d, 5d  
    PUTC      'u'  
    GOTOXY    6d, 6d  
    PUTC      'n'  
    GOTOXY    7d, 7d  
    PUTC      'd'  
    GOTOXY    8d, 8d  
    PUTC      'o'  
    INT       20h  
END
```

O programa anterior apresenta a mensagem **Alo Mundo** num formato diagonal. Salve-o com o nome **BIBLIOT3.asm**.

PRINT *mensagem*/**PRINTN** *mensagem*

Funções que exibem uma mensagem na tela do monitor de vídeo. A rotina **PRINT** apresenta uma mensagem e mantém o cursor na mesma linha. A rotina **PRINTN** apresenta uma mensagem e movimenta o cursor para a próxima linha.

```
;*****  
;*      Programa: BIBLIOT4.ASM      *  
;*****
```

```
INCLUDE 'emu8086.inc'
```

```
.MODEL small  
.STACK 512d
```

```
.CODE

PRINTN    'Alo Mundo 1'
PRINTN    'Alo Mundo 2'
PRINT     'Alo Mundo 3'
PRINT     'Alo Mundo 4'
INT       20h

END
```

O programa anterior mostra quatro mensagens com **Alo Mundo**. A primeira e a segunda linhas aparecem uma abaixo da outra num formato diagonal. A mensagem da terceira linha de programa é apresentada abaixo da segunda mensagem, mas a quarta mensagem surge imediatamente ao lado da terceira mensagem. Salve o programa com o nome **BIBLIOT4.asm**.

CUSOROFF/CUSORON

Funções que ocultam ou apresentam o cursor. A rotina **CUSOROFF** desabilita a apresentação do cursor. A rotina **CUSORON** habilita a apresentação do cursor.

```
;*****
;*      Programa: BIBLIOT5.ASM      *
;*****

INCLUDE 'emu8086.inc'

.MODEL small
.STACK 512d

.CODE

CURSOROFF
PRINTN    'Sem cursor - tecle algo'

MOV       AH, 00h
INT       16h

CURSORON
PRINTN    'Com Cursor - tecle algo'

MOV       AH, 00h
INT       16h

INT       20h

END
```

O programa anterior apresenta duas mensagens. A primeira não mostra o cursor na tela. A segunda mensagem apresenta o cursor. Salve o programa com o nome **BIBLIOT5.asm**.

11.1.2 - Funções de procedimento

A biblioteca **emu8086.inc** apresenta sete procedimentos que oferecem os seguintes recursos operacionais:

GET_STRING/PRINT_STRING

O procedimento **GET_STRING** lê via teclado uma sequência de caracteres terminada em nulo. A sequência de caracteres é escrita no *buffer* utilizando o par de registradores **DS:DI** e armazena o tamanho do *buffer* no registrador geral **DX**. Esse procedimento é interrompido quando usada a tecla <Enter>. Antes da diretiva **END** no programa principal é necessário utilizar a declaração **DEFINE_GET_STRING** para que o funcionamento da rotina **GET_STRING** ocorra sem problemas.

O procedimento **PRINT_STRING** apresenta uma sequência de caracteres terminada com caractere nulo na posição atual do cursor. A mensagem a ser impressa é obtida a partir do *buffer* identificado pelo par de registradores **DS:SI**. Antes da diretiva **END** no programa principal é necessário utilizar a declaração **DEFINE_PRINT_STRING** para que a rotina **PRINT_STRING** funcione sem problemas.

```
;*****  
;*      Programa: BIBLIOT6.ASM      *  
;*****  
  
INCLUDE 'emu8086.inc'  
  
org 100h  
  
.DATA  
    tamanho EQU 30d + 1d  
    buffer  DB  tamanho DUP ('x')  
    msg1     DB  'Entre seu nome: ', 0  
    msg2     DB  'Ola, ', 0  
  
.CODE  
    LEA      SI, msg1  
    CALL     PRINT_STRING  
  
    LEA      DI, buffer  
    MOV      DX, tamanho  
    CALL     GET_STRING  
    PUTC     13d  
    PUTC     10d  
  
    LEA      SI, msg2  
    CALL     PRINT_STRING  
  
    MOV      SI, DI  
    CALL     PRINT_STRING  
  
    INT      20h  
  
    DEFINE_PRINT_STRING  
    DEFINE_GET_STRING  
END
```

O programa anterior apresenta na área de definição de dados (**.DATA**) duas novidades. A primeira é o uso da diretiva **EQU** que permite a definição de constantes, que neste caso está sendo representada pelo rótulo denominado **tamanho**. A segunda novidade é com relação ao valor da constante que está definido como **30d + 1d**. Observe o uso do operador de adição entre os dois valores para determinar o valor **31d**. Certamente pode ser definido diretamente o valor **31d**, mas há possibilidade de estabelecer valores com operadores aritméticos, como foi feito. Salve o programa com o nome **BIBLIOT6.asm**.

Ainda na área de definição de dados (.DATA) há a definição da variável **buffer** do tipo **DB** com o valor estabelecido para a constante **tamanho**. Está sendo estabelecido na memória um espaço para a variável denominada **buffer** com tamanho **31d** contendo uma série de caracteres **x**.

Na área de código (.CODE) o programa apresenta em tela a mensagem **Entre seu nome:**, manipulada pelo procedimento **PRINT_STRING**, pelas linhas de código **LEA SI, msg1** e **CALL PRINT_STRING**.

Em seguida o programa solicita o nome pela linha de código **CALL GET_STRING**. Nessa etapa acompanhe a definição do deslocamento da área de *buffer* através da linha de código **LEA DI, buffer** e do armazenamento do tamanho da área de *buffer* através da linha de código **MOV DX, tamanho**. Está sendo transferido o endereço do conteúdo informado pelo teclado para o par de registradores **DS:DI**. Depois o programa retorna o carro e muda de linha com a instrução **PUTC 13d** e **PUTC 10d**.

Após a execução das etapas anteriores o programa exibe a mensagem armazenada na variável **msg2** e resgata dos registradores **DS:DI** o endereço em que se encontra a mensagem entrada pelo procedimento **GET_STRING**.

Antes da diretiva **END** que finaliza o código do programa estão definidos os rótulos **DEFINE_PRINT_STRING** e **DEFINE_GET_STRING**, que devem ser utilizados antes da diretiva **END**, pois representam as rotinas de macros internas que estabelecem para o procedimento em uso os rótulos locais usados pelos procedimentos **GET_STRING** e **PRINT_STRING**.

PTTHIS

O procedimento **PTTHIS** exibe a mensagem terminada com caractere nulo na posição atual do cursor, semelhante ao procedimento **PRINT_STRING**, mas com a diferença de receber o endereço da sequência de caracteres que formam a mensagem a partir da pilha. Para sequências de caracteres terminadas em zero, é necessário que a mensagem ocorra logo após a chamada do procedimento. Antes da diretiva **END** no programa principal é necessário utilizar a declaração **DEFINE_PTHIS** para que o funcionamento da rotina **PTTHIS** ocorra sem problemas.

```
;*****  
;*      Programa: BIBLIOT7.ASM      *  
;*****  
  
INCLUDE 'emu8086.inc'  
  
org 100h  
  
.DATA  
    tamanho EQU 30d + 1d  
    buffer  DB  tamanho DUP ('x')  
  
.CODE  
  
    CALL    pthis  
           DB 'Entre seu nome: ', 0  
  
    LEA     DI, buffer  
    MOV     DX, tamanho  
    CALL    GET_STRING  
    PUTC    13d  
    PUTC    10d  
  
    CALL    pthis  
           DB 'Ola, ', 0  
  
    MOV     SI, DI  
    CALL    PRINT_STRING  
  
    INT     20h
```

```

DEFINE_PRINT_STRING
DEFINE_GET_STRING
DEFINE_PTHIS
END

```

As chamadas são efetuadas no código anterior a partir da instrução **CALL pthis**. É definida a mensagem a ser apresentada imediatamente após a chamada do procedimento **pthis**. Salve o programa com o nome **BIBLIOT7.asm**.

CLEAR_SCREEN

Procedimento que limpa a tela e posiciona o cursor no seu topo esquerdo. Antes da diretiva **END**, no programa principal é necessário utilizar a declaração **DEFINE_CLEAR_SCREEN**.

Para esse tipo de rotina de procedimento não será apresentado um exemplo de programa, pois esse efeito será mostrado no próximo exemplo.

SCAN_NUM/PRINT_NUM/PRINT_NUM_UN

O procedimento **SCAN_NUM** obtém um valor numérico com mais de um dígito fornecido pelo teclado, podendo ser um valor positivo ou negativo. Esse procedimento armazena o resultado da entrada no registrador geral **CX**. Antes da diretiva **END** no programa principal é necessário utilizar a declaração **DEFINE_SCAN_NUM** para que a rotina **SCAN_NUM** funcione sem problemas.

Os procedimentos **PRINT_NUM** e **PRINT_NUM_UN** apresentam, respectivamente, os valores numéricos negativos e positivos existentes no registrador geral **AX**. Antes da diretiva **END** no programa principal é necessário utilizar as declarações **DEFINE_PRINT_NUM** e **DEFINE_PRINT_NUM_UN** para que as rotinas de procedimento **PRINT_NUM** e **PRINT_NUM_UN** funcionem sem problemas.

```

;*****
;*      Programa: BIBLIOT8.ASM      *
;*****

```

```
INCLUDE 'emu8086.inc'
```

```
org 100h
```

```

.DATA
    tamanho EQU 30d + 1d
    idade    DW 0
    buffer    DB tamanho DUP ('x')
    msg1      DB 'Entre seu nome ....: ', 0
    msg2      DB 'Entre sua idade ....: ', 0
    msg3      DB 'Ola, ', 0
    msg4      DB ' voce tem ', 0
    msg5      DB ' anos.', 0

```

```

.CODE
    LEA      SI, msg1
    CALL     PRINT_STRING
    LEA      DI, buffer
    MOV      DX, tamanho
    CALL     GET_STRING
    PUTC     13d
    PUTC     10d

    LEA      SI, msg2
    CALL     PRINT_STRING
    CALL     SCAN_NUM
    MOV      idade, CX
    PUTC     13d
    PUTC     10d

```

```

CALL    CLEAR_SCREEN
LEA     SI, msg3
CALL    PRINT_STRING
MOV     SI, DI
CALL    PRINT_STRING
LEA     SI, msg4
CALL    PRINT_STRING
MOV     AX, idade
CALL    PRINT_NUM
LEA     SI, msg5
CALL    PRINT_STRING

INT     20h

DEFINE_PRINT_STRING
DEFINE_GET_STRING
DEFINE_SCAN_NUM
DEFINE_CLEAR_SCREEN
DEFINE_PRINT_NUM
DEFINE_PRINT_NUM_UN
END

```

Observe no programa anterior as linhas de código **CALL SCAN_NUM** e na sequência **MOV idade, CX**. Assim que o valor é capturado e armazenado na memória (registrador geral **CX**), ele é transferido para a variável **idade**. Para fazer a apresentação do valor da variável **idade**, o programa executa as linhas de código **MOV AX, idade** e **CALL PRINT_NUM**. Salve o programa com o nome **BIBLIOT8.asm**.

11.2 - Endereçamento e acesso à memória

Anteriormente foi feito um rápido comentário a respeito das formas de acesso e uso de endereçamento de memória. Neste capítulo, devido à necessidade de estudar tabelas em memória (matrizes), é preciso aprofundar este tema.

Segundo o exposto por Hyde (2003), existem dezessete formas de acesso à memória do microprocessador 8086/8088, sendo uma de acesso de deslocamento direto, quatro de acesso com o modo de endereçamento indireto com registradores, quatro de acesso com o modo de endereçamento indexado, quatro de acesso de endereçamento de base indexada e quatro de acesso de endereçamento de base indexada mais deslocamento. Observe a Tabela 11.1.

Tabela 11.1 - Definição do Endereçamento de Memória

Direto	Indireto com Registrador	Indexado	Base Indexada	Base Indexada mais Deslocamento
ds:[posição]	[BX]	desloc[BX]	[BX][SI]	desloc[BX][SI]
	[BP]	desloc[BP]	[BX][DI]	desloc[BX + DI]
	[SI]	desloc[SI]	[BP][SI]	[BP + SI + desloc]
	[DI]	desloc[DI]	[BP][DI]	[BP][DI][x]

A indicação do rótulo **desloc** na tabela refere-se à definição de um valor de deslocamento a ser informado. A tabela anterior apresenta as formas válidas de definição de deslocamentos de memória para a manipulação de dados que podem ser utilizadas em um programa escrito em linguagem de programação *Assembly* para um microprocessador 8086/8088.

Para os casos de acesso a endereçamento de memória não direto, Hyde (2003) sugere um diagrama para facilitar a memorização das estruturas de endereçamento: indireto com registrador, indexado, base indexada e base indexada mais deslocamento, como é indicado na Figura 11.1.

Deslocamento	[BX]	[SI]
	[BP]	[DI]

Figura 11.1 - Endereçamento de memória (Adaptado de Hyde, 2003).

Para saber o endereçamento de memória correto, basta escolher pelo menos um item de cada coluna para ter o modo de endereçamento de memória válido. São válidos os seguintes endereçamentos de memória:

- ◆ Escolhe-se o deslocamento da coluna da esquerda, não se escolhe o registrador [BX] da coluna do meio, escolhe-se o registrador [DI] da coluna da direita e obtém-se o endereçamento **deslocamento[DI]**.
- ◆ Escolhe-se o deslocamento da coluna da esquerda, o registrador [BX] da coluna do meio, escolhe-se o registrador [DI] da coluna da direita e obtém-se o endereçamento **deslocamento[BX][DI]**.
- ◆ Não se escolhe o deslocamento da coluna da esquerda, não se escolhe o registrador [BX] da coluna do meio, escolhe-se o registrador [SI] e obtém-se o endereçamento [SI].
- ◆ Não se escolhe o deslocamento da coluna da esquerda, escolhe-se o registrador [BX] da coluna do meio, o registrador [DI] da coluna da direita e obtém-se o endereçamento [BX][DI].

De acordo com o exposto, qualquer combinação de endereçamento diferente da forma permitida pelo diagrama indicada na Figura 11.1 é considerado inválido. Hyde (2003) apresenta como endereçamento inválido de memória a definição **deslocamento[DX][SI]**, pois o registrador [DX] não está definido na coluna do meio do diagrama.

É pertinente lembrar que as operações de acesso à memória são efetuadas normalmente com a instrução **MOV** e dependendo do programa *assemblador* em uso, é necessário utilizar para algumas operações a instrução **LEA**.

A partir da definição dos princípios de formas de acesso aos endereçamentos de memória, cabe ressaltar os pontos de conceituação apontados por Hyde (2003) em seu trabalho, identificados a seguir.

11.2.1 - Acesso direto

O modo de acesso direto ao endereçamento de memória é baseado no uso de uma constante de 8 *bits*, ou seja, uma constante de um *byte*, que representa o valor de acesso a um determinado endereço de memória do segmento de dados em uso.

Por exemplo, a instrução **MOV AL, DS:[1234h]** transfere para o registrador menos significativo **AL** uma cópia do valor do *byte* residente no endereço de deslocamento **1234h** do segmento **DS**. O registrador menos significativo **AL** passa a ter o valor armazenado no endereço de memória **1234h**. É possível ter acesso ao conteúdo de memória armazenado no endereço de deslocamento **1234h**, como mostra a Figura 11.2.

Em contrapartida, se fosse usada a instrução **MOV DS:[1234h], AL**, seria armazenado o valor do registrador menos significativo **AL** no endereço de memória **1234h**, como indica a Figura 11.3.

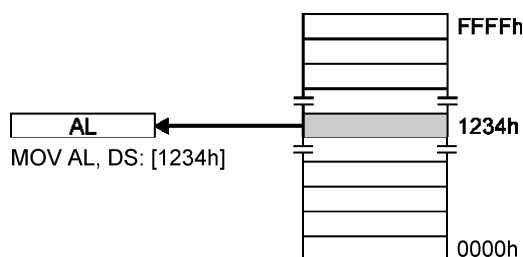


Figura 11.2 - Endereçamento por deslocamento (Adaptado de Hyde, 2003).

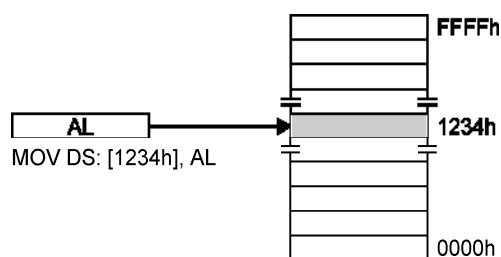


Figura 11.3 - Endereçamento por deslocamento (Adaptado de Hyde, 2003).

É importante levar em consideração que essa forma de acesso é muito utilizada com valores simples. Para valores em posições indexadas (matrizes) é necessário utilizar outras formas de acesso à memória.

Para exemplificar uma ação de acesso direto à memória de forma simples, considere o seguinte programa:

```

;*****
;*      Programa: MEMO1.ASM      *
;*****

.MODEL small
.STACK 512d

.CODE

    MOV     AL, 5d
    MOV     DS:[010Ch], AL
    SUB     AX, AX
    MOV     AL, DS:[010Ch]

    HLT

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **File/Save** com o nome **MEMO1**, de forma que fique semelhante à imagem da Figura 11.4.

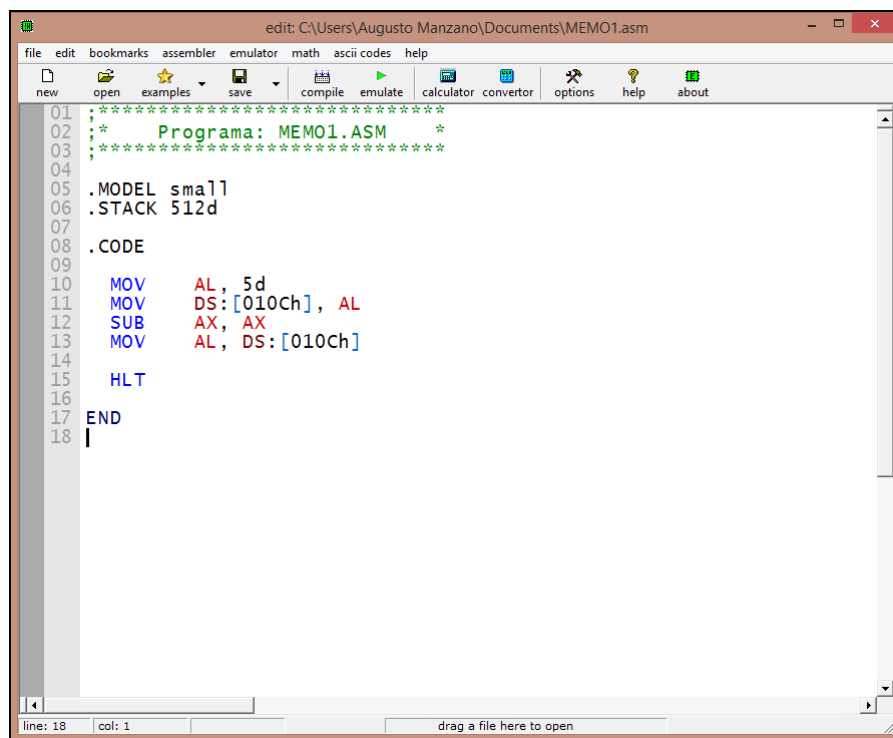


Figura 11.4 - Programa MEMO1 na ferramenta emu8086.

Antes de qualquer comentário a respeito da ação principal do programa, observe a instrução **HLT** (*halt*) de encerramento do programa. Ela para a execução do programa, interrompendo o processador.

Em seguida peça a execução do programa por meio da tecla de função **<F5>** e observe atentamente os detalhes na janela **emulator: MEMO1.exe**, conforme a Figura 11.5.

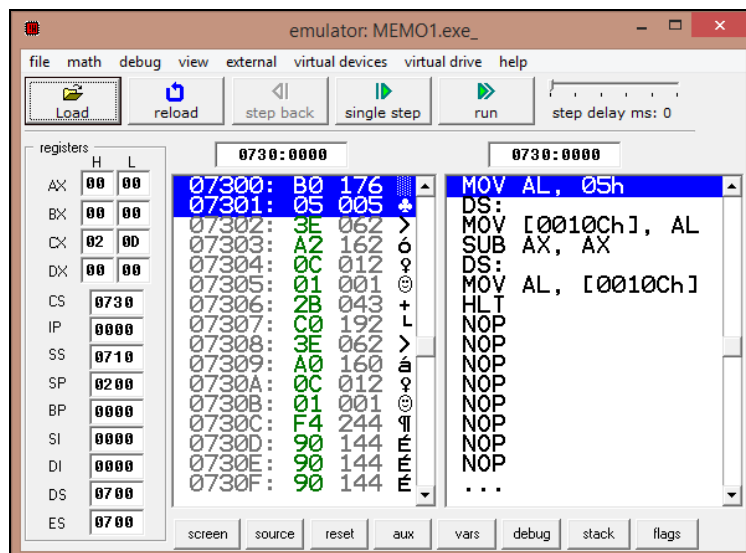


Figura 11.5 - Janela Emulator: MEMO1.exe_ na ferramenta emu8086.

Perceba a apresentação em separado da área **disassembler** (lado direito), Figura 11.6, com a definição do código do programa escrito em *Assembly* e da área **memory** (área central), Figura 11.7, com a definição do código em linguagem de máquina (*opcodes*).

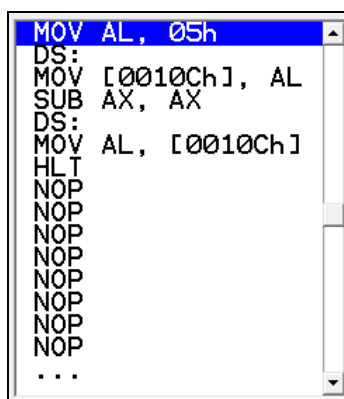


Figura 11.6 - Código em Assembly.

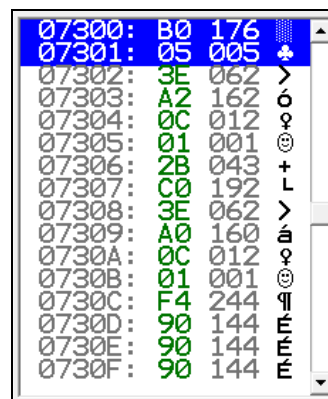


Figura 11.7 - Código em Opcode.

Na Figura 11.6 o código apresentado é ligeiramente diferente do código-fonte definido no editor do programa **emu8086**. As instruções **MOV DS:[010Ch], AL** e **MOV AL, DS:[010Ch]** são indicadas na área de código, Figura 11.6, respectivamente, como **MOV [0010Ch], AL** e **MOV AL, [0010Ch]**. O registrador **DS** fará referência ao endereço de segmento que estiver em uso pelo microprocessador. Basta acionar pela primeira vez a tecla de função **<F8>** para proceder à execução passo a passo do programa. A Figura 11.8 mostra as alterações na janela **emulator: MEMO1.exe_**.

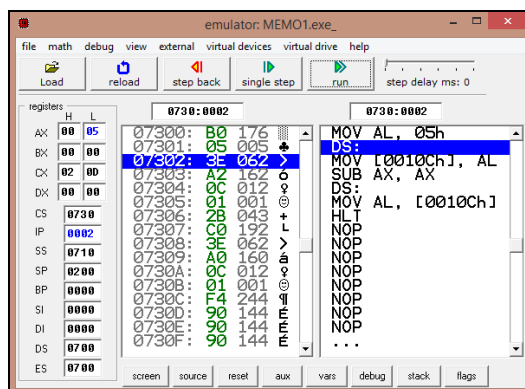


Figura 11.8 - Janela Emulator: MEMO1.exe_ (primeira execução de <F8>).

Ocorre a definição do valor **05h** para o registrador menos significativo **AL**, como indica a Figura 11.8. Em seguida acione a tecla de função **<F8>** pela segunda vez para continuar a ação do programa. A Figura 11.9 exhibe a mudança do valor do registrador **IP** de **0002h** para **0003h**.

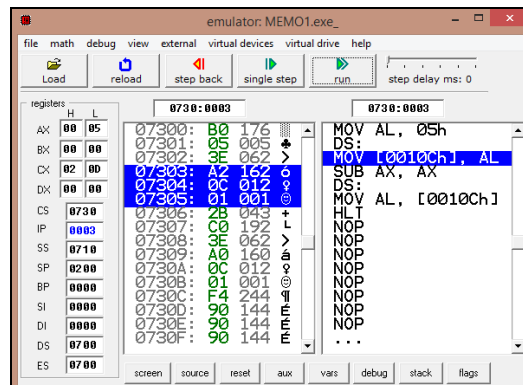


Figura 11.9 - Janela MEMO1.exe_ (segunda execução de **<F8>**).

No momento, acione na tela **emulador: MEMO1.exe_** a opção de menu **view/memory** para que seja apresentada a tela **Random Access Memory**. No campo existente do lado esquerdo superior, ao lado do botão **update**, entre o endereço de memória **0700:010C** no formato **segmento:deslocamento** (é importante lembrar que o endereço de segmento apresentado no computador do leitor pode ser diferente, esteja atento). Como se deduziu o segmento **0700h**? Simples, o programa iniciou sua execução no endereço de segmento **0730h** e como se está utilizando o conceito de acesso direto, este é feito num endereço de memória do segmento de dados em uso. Isto posto, percebe-se que o programa está operando no endereço de segmento **0730h**, desta forma o microprocessador direciona o armazenamento do dado para o início do segmento em operação que para este caso é o endereço de segmento **0700h**. A Figura 11.10 mostra a janela **Random Access Memory** posicionada no endereço **0700:010C**.

Observe na Figura 11.10 que a área de memória de **0700:010C** até **0700:017C** possui armazenado o conteúdo de código **54h** que já estava na memória.

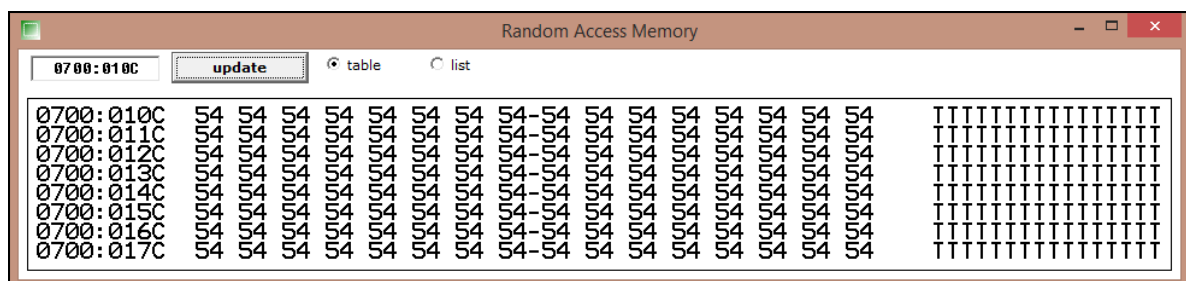


Figura 11.10 - Janela Random Access Memory no endereço **0730:010C**.

Em seguida acione a tecla de função **<F8>** pela terceira vez e observe a mudança do registrador **IP** para **0006h** e o posicionamento sobre a instrução **SUB AX, AX** para zerar o registrador **AX**. Note também o armazenamento do valor **05h** na primeira posição da janela **Random Access Memory** como na Figura 11.11.

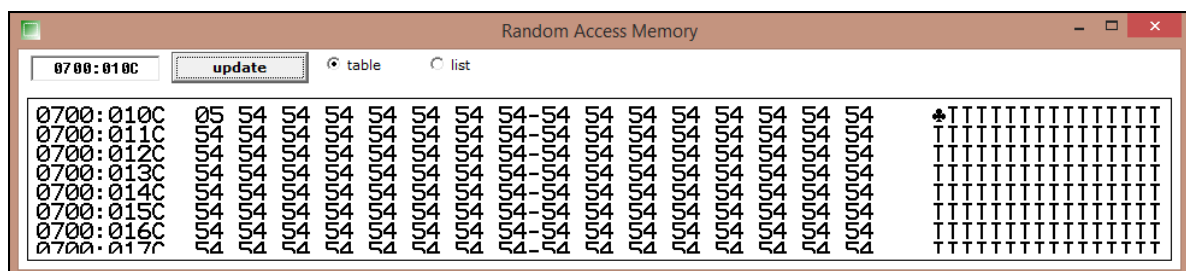


Figura 11.11 - Janela Random Access Memory no endereço **0730:010C** com valor **05h**.

A partir desse instante o registrador geral **AX** será zerado pela instrução **SUB AX, AX**, como indicado na Figura 11.12, e a próxima instrução a ser executada é a preparação para a transferência de volta do valor **05h** que se encontra armazenado no endereço de memória **0700:010C** para o registrador menos significativo **AL**. Lembre-se de que esse armazenamento foi efetivado pela linha de instrução **MOV DS:[010Ch], AL**. Em seguida, acione a tecla de função <F8> pela quarta vez e veja o resultado da operação na Figura 11.13.

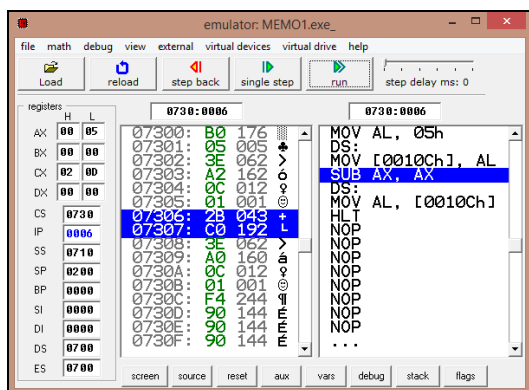


Figura 11.12 - Janela Emulador: MEMO1.exe_ (terceira execução de <F8>).

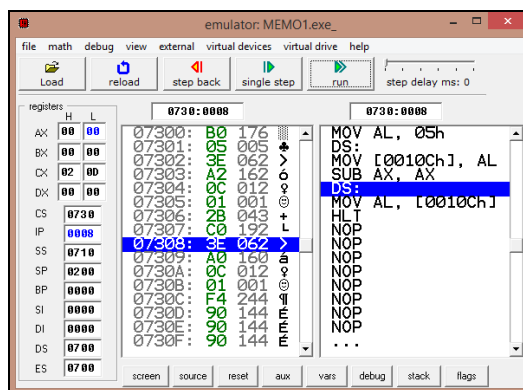


Figura 11.13 - Janela Emulador: MEMO1.exe_ (quarta execução de <F8>).

Acione a tecla de função <F8> pela quinta vez. A execução do programa estará sobre a instrução **MOV AL, [0010Ch]** que efetua a transferência do valor **05h** armazenado no endereço **0700:010C** de volta para o registrador **AL**, como indicado na Figura 11.14. Em seguida acione a tecla de função <F8> pela sexta vez e observe a mudança do valor do registrador **AL** de **00h** para **05h** e o posicionamento sobre a instrução **HLT**, como mostra a Figura 11.15.

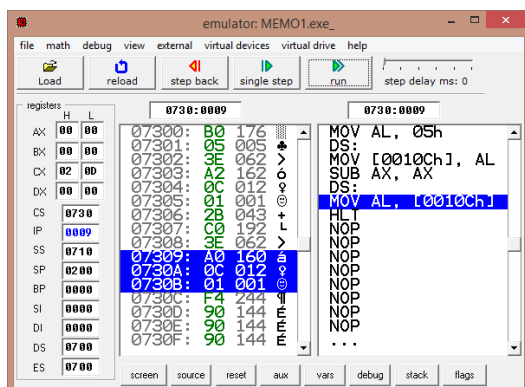


Figura 11.14 - Janela Emulador: MEMO1.exe_ (quinta execução de <F8>).

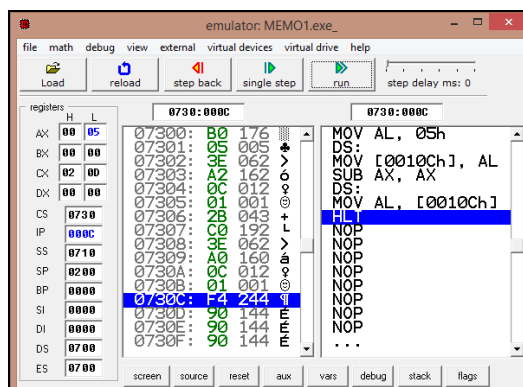


Figura 11.15 - Janela Emulador: MEMO1.exe_ (sexta execução de <F8>).

Pela sétima e última vez acione a tecla de função <F8>, com o ponteiro do *mouse* selecione o botão **OK** da caixa de mensagem **message** e na janela **emulador: MEMO1.exe_** selecione o comando de menu **file/close the emulator**.

11.2.2 - Acesso indireto com registradores

O modo de acesso indireto com registrador permite obter o endereço de uma determinada posição de memória com a utilização de um dos registradores **[BX]**, **[BP]**, **[DI]** e **[SI]**. Como padrão, os registradores **[BX]**, **[DI]** e **[SI]** usam como registrador de segmento o **DS** e o **[BP]** utiliza como registrador de segmento o **SS**.

Por exemplo, a instrução **MOV AL, [BX]** transfere para o registrador menos significativo **AL** uma cópia do valor do *byte* residente no endereço de deslocamento identificado pelo registrador **[BX]** do segmento **DS**. Desta forma, o registrador menos significativo **AL** passa a ter o valor armazenado no endereço de memória identificado por **[BX]**, como mostra a Figura 11.16.

Se usada a instrução **MOV AL, valor[BX]** o registrador menos significativo **AL** será armazenado com os valores existentes da área de dados identificada pela variável **valor**, apontada pelo registrador de segmento **DS** até a posição limite

(a ser definida no programa) a ser obtida com o uso do registrador **[BX]**. Para utilizar esse recurso, é preciso definir um laço que desloque o endereço a partir de uma posição inicial. Vá até um valor que determine a posição final de memória. A informação **[BX]** da Figura 11.16 também pode ser entendida como **[DI]** e **[SI]**.

Para utilizar o registrador de pilha **[BP]**, deve ser considerado como registrador de segmento o **SS**. A Figura 11.17 apresenta a estrutura de funcionamento desse tipo de operação.

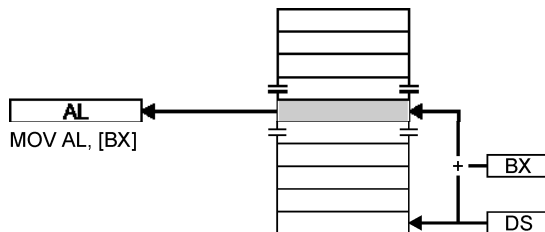


Figura 11.16 - Endereçamento indireto com registro (Adaptado de Hyde, 2003).

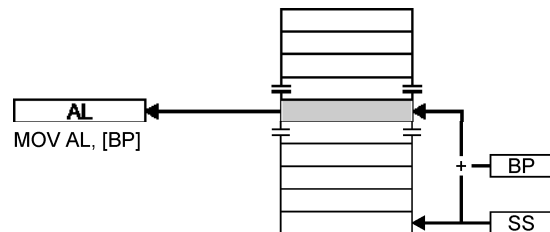


Figura 11.17 - Endereçamento indireto com registro (Adaptado de Hyde, 2003).

Para exemplificar a ação de acesso indireto com registradores de memória de forma simples, considere o seguinte programa:

```
;*****
;*   Programa: MEMO2.ASM   *
;*****

org 100h

.DATA
    valor DB 05d, 04d, 03d, 02d, 01d

.CODE
    MOV     CX, 05h
    LEA     BX, valor
laco:
    MOV     AL, [BX]
    ADD     AL, 30h
    MOV     AH, 0Eh
    INT     10h
    INC     BX
    LOOP    laco
    HLT

END
```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **MEMO2**, de forma que fique semelhante à imagem da Figura 11.18.

Observe o trecho de código definido entre o rótulo **laco:** e a instrução **LOOP laco** e a utilização das instruções **MOV AL, [BX]** e **INC BX**. A instrução **MOV AL, [BX]** movimenta um dos conteúdos identificados na área de dados com a variável **valor**. À medida que se obtêm o conteúdo de memória e sua respectiva apresentação, o programa acrescenta 1 ao valor existente no registrador **[BX]**. Desta forma é possível obter todos os valores definidos na variável **valor**.

Em seguida peça a execução do programa pela tecla de função **<F5>** e acione também uma vez a tecla de função **<F8>**. Observe os detalhes apresentados na janela **MEMO2**, como indica a Figura 11.19.

Observe atentamente as informações apresentadas nas áreas **disassemble** e **memory**. Atente para o posicionamento sobre a instrução do programa **MOV CX, 00005h** e o posicionamento no endereço de memória **0700:0107**.

A tabela seguinte apresenta um paralelo entre o código escrito no editor e o código assumido pelo simulador. Ocorrem algumas pequenas mudanças, entre elas a substituição da instrução **LEA BX, valor** pela instrução **MOV BX, 00102h**, indicando o endereço de deslocamento que marca o início da área de dados denominada **valor**.

```

01  ;*****
02  ; Programa: MEMO2.ASM
03  ;*****
04
05  org 100h
06
07  .DATA
08  valor DB 05d, 04d, 03d, 02d, 01d
09
10  .CODE
11  MOV CX, 05h
12  LEA BX, valor
13  laco:
14  MOV AL, [BX]
15  ADD AL, 30h
16  MOV AH, 0Eh
17  INT 10h
18  INC BX
19  LOOP laco
20  HLT
21
22  END
23

```

Figura 11.18 - Programa MEMO2 na ferramenta emu8086.

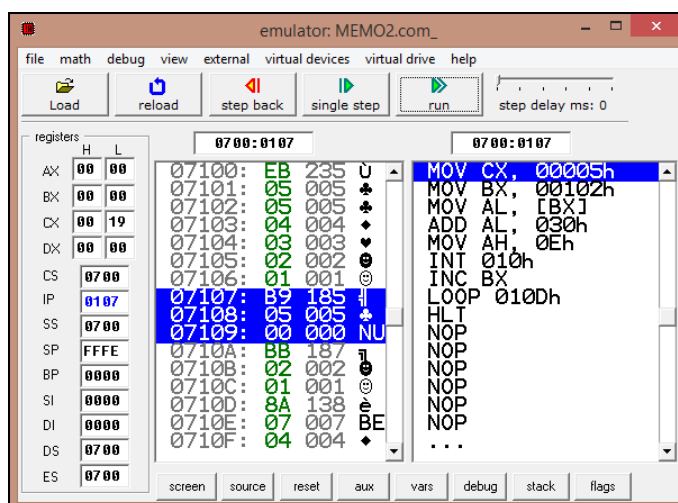


Figura 11.19 - Janela MEMO2 na ferramenta emu8086 (<F8> - primeira vez).

A linha de código **MOV AL, [BX]** transfere o valor apontado pelo registrador **[BX]** para o registrador menos significativo **AL**. Em seguida esse valor é apresentado na tela do monitor de vídeo. O registrador **[BX]** possibilita obter o primeiro valor da área de dados (**valor**), que está armazenado na posição **0** (zero). Por este motivo, antes da instrução **LOOP** se encontra a instrução **INC BX**, que acrescenta 1 (um) ao valor atual do registrador **[BX]**. A Tabela 20.2 demonstra a ocorrência interna dos valores do registrador **[BX]**.

Em vez de acionar a tecla de função **<F8>**, utilize em conjunto as teclas de atalho **<Shift> + <F8>** para executar o programa a partir da instrução **MOV CX, 00005h** que enviará o valor **0005h** para o registrador **CX**, como mostra a Figura 11.20. O valor armazenado no registrador **CX** será utilizado para controlar a ação da instrução **LOOP**.

Tabela 11.2 – Ocorrência de valores do registrador [BX]

Código no Editor	Código no Simulador
MOV CX, 05h	MOV CX, 00005h
LEA BX, valor	MOV BX, 00102h
laco:	
MOV AL, [BX]	MOV AL, [BX]
ADD AL, 30h	ADD AL, 030h
MOV AH, 0Eh	MOV AH, 0Eh
INT 10h	INT 010h
INC BX	INC BX
LOOP laco	LOOP 010Dh
HLT	HLT

Em seguida, execute pela terceira vez as teclas de atalho <Shift> + <F8> para que a instrução **MOV BX, 00102h** seja processada. A Figura 11.21 mostra a etapa atual de execução em que o registrador **BX** assume o valor **0102h**.

Agora acione as teclas de atalho <Shift> + <F8> pela quarta vez para se posicionar no próximo passo. A Figura 11.22 mostra o resultado da ação. Note o posicionamento na instrução **ADD AL, 030h** e a alteração do valor do registrador **AL** com o valor de **[BX]** que neste momento é **05h**.

Pela quinta vez acione as teclas de atalho <Shift> + <F8> e observe o envio do valor **30h** para o registrador **AL**, como mostra a Figura 11.23. Lembre-se de que o procedimento de somar **30h** ao conteúdo do registrador **AL** que está com valor **05h** é tornar esse valor **35h** que é o código ASCII do valor **5d** para ser, na sequência, apresentado na tela do monitor de vídeo.

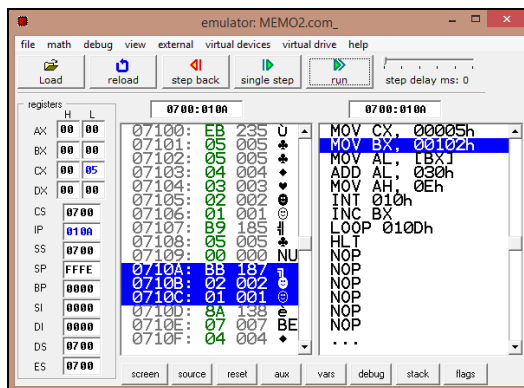


Figura 11.20 - Janela MEMO2 na ferramenta emu8086 (<Shift> + <F8> - segunda vez).

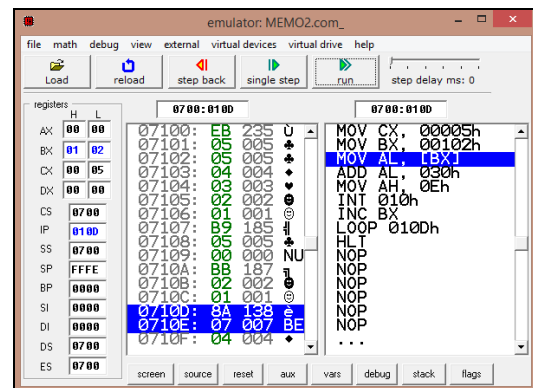


Figura 11.21 - Janela MEMO2 na ferramenta emu8086 (<Shift> + <F8> - terceira vez).

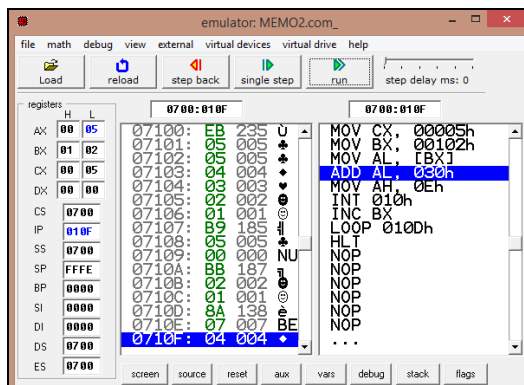


Figura 11.22 - Janela MEMO2 na ferramenta emu8086 (<Shift> + <F8> - quarta vez).

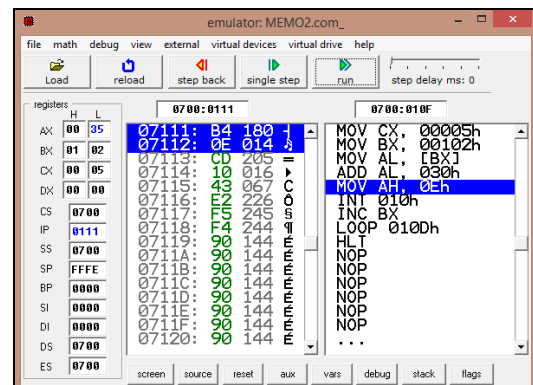


Figura 11.23 - Janela MEMO2 na ferramenta emu8086 (<Shift> + <F8> - quinta vez).

Assim que o valor **5d** estiver armazenado na memória, ele precisa ser apresentado. Assim sendo, execute as teclas de atalho <Shift> + <F8> para proceder à sexta execução passo a passo. Nessa etapa o valor **0Eh** será armazenado no

registrador **AH**. Esse valor corresponde ao código de escrita de um caractere quando do uso da instrução **INT 10h**. A Figura 11.24 mostra essa etapa da execução.

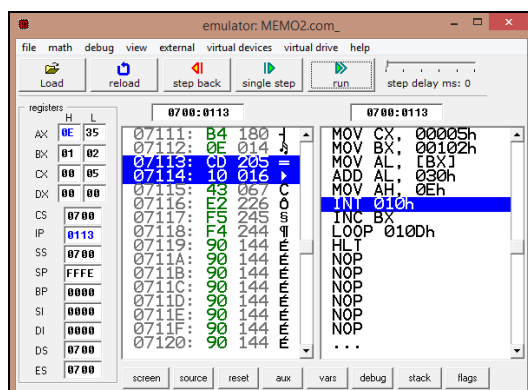


Figura 11.24 - Janela MEMO2 na ferramenta emu8086 (<Shift> + <F8> - sexta vez).

Perceba a inserção do valor **0Eh** no registrador menos significativo **AH** por intermédio da linha de instrução **MOV AH, 0Eh**. Na sequência execute pela sétima vez as teclas de atalho **<Shift> + <F8>**.

Será executada a instrução **INT 10h** que por meio do código **0Eh** armazenado no registrador **AH** fará a apresentação do conteúdo do registrador **AL** na tela do monitor de vídeo na sua forma decimal. A Figura 11.25 apresenta o resultado da ação.

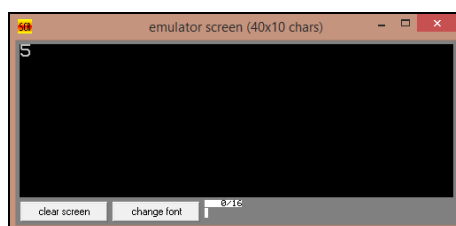


Figura 11.25 - Apresentação do resultado do programa.

Após a apresentação do valor 5 feche a janela **emulator screen** e observe que será executada a instrução **INC BX**. Acione o conjunto de teclas de atalho **<Shift> + <F8>** pela sétima vez e observe a alteração do valor do registrador **BX** para **0103h**, como na Figura 11.26. O valor **0002h** de **BX** passa a ser **0003h**.

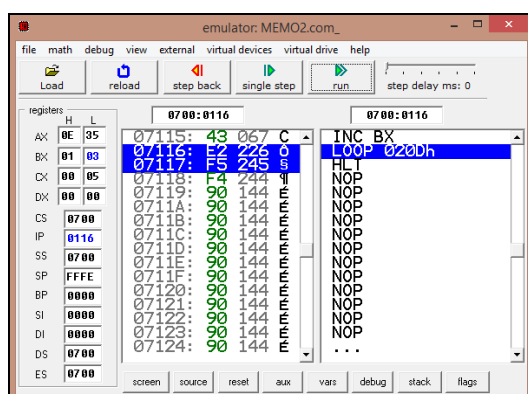


Figura 11.26 - Janela MEMO2 na ferramenta emu8086 (<Shift> + <F8> - sétima vez).

Ao ser executada a instrução **INC BX**, definiu-se o novo valor de endereço de memória a ser usado pela instrução **MOV AL, [BX]** assim que o laço ocorrer. Isso é repetido até que o registrador geral **CX** tenha o valor **0**. Vá repetindo as ações do programa para ver esse efeito ocorrer.

11.2.3 - Acesso indexado

O modo de acesso indexado (também denominado endereçamento de base ou endereçamento indexado) permite obter-se o endereço de uma posição de memória pelo deslocamento sequencial de um índice definido a partir de um dos registradores [BX], [BP], [DI] e [SI].

É interessante lembrar que os registradores [BX], [DI] e [SI] usam como registrador de segmento o DS e o [BP] utiliza como registrador de segmento o SS.

Por exemplo, a instrução **MOV AL, valor[BX]** transfere para o registrador menos significativo AL uma cópia do valor do byte residente no endereço de deslocamento identificado pelo registrador [BX] do segmento DS denominado **valor**. Desta forma, o registrador menos significativo AL passa a ter o valor armazenado no endereço de memória identificado por [BX].

É possível ter acesso ao conteúdo de memória armazenado a partir do endereço de deslocamento indireto [BX], como mostra a Figura 11.27.

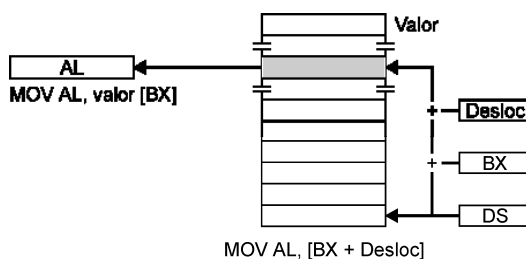


Figura 11.27 - Endereçamento indireto com registro (Adaptado de HYDE, 2003).

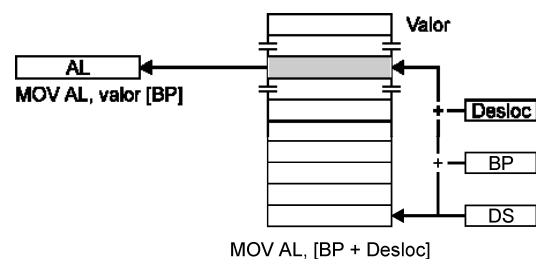


Figura 11.28 - Endereçamento indireto com registro (Adaptado de HYDE, 2003).

A Figura 11.27 indica que por meio da instrução **MOV AL, valor[BX]** (ou instrução **MOV AL, [BX + DESLOC]**, em que **DESLOC** pode ser um valor de 8 ou 16 *bits*) o registrador menos significativo AL é armazenado com os valores existentes da área de dados identificada pela variável **valor**, apontada pelo registrador de segmento DS até a posição limite (a ser definido no programa) a ser obtida com o uso do registrador [BX]. Para fazer isso, é necessário definir um laço de repetição. A informação [BX] da Figura 11.27 pode ser entendida como [DI] ou [SI].

```
;*****  
;* Programa: MEM03.ASM *  
;*****
```

```
org 100h
```

```
.DATA
```

```
valor DB 05d, 04d, 03d, 02d, 01d
```

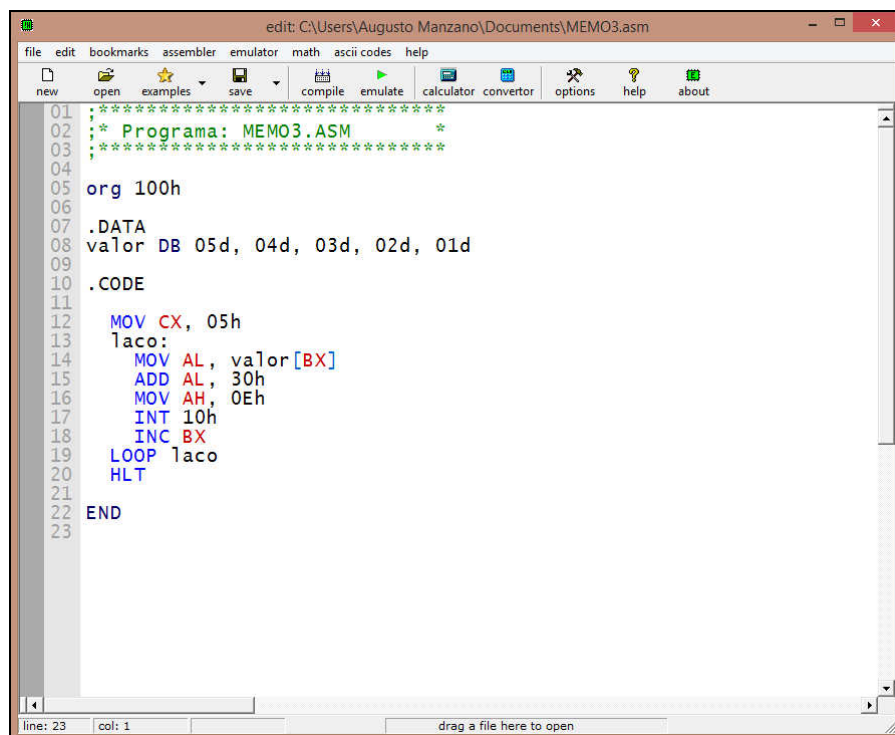
```
.CODE
```

```
MOV CX, 05h  
laco:  
    MOV AL, valor[BX]  
    ADD AL, 30h  
    MOV AH, 0Eh  
    INT 10h  
    INC BX  
    LOOP laco  
    HLT
```

```
END
```

Para usar o registrador de pilha [BP], deve ser considerado como registrador de segmento o SS. A Figura 11.28 mostra a estrutura de funcionamento desse tipo de operação. Para exemplificar o acesso indexado de memória, considere o seguinte programa:

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **MEMO3**, de forma que fique semelhante à imagem da Figura 11.29.



```

01  ;*****
02  ;* Programa: MEMO3.ASM
03  ;*****
04
05  org 100h
06
07  .DATA
08  valor DB 05d, 04d, 03d, 02d, 01d
09
10  .CODE
11
12  MOV CX, 05h
13  laço:
14  MOV AL, valor[BX]
15  ADD AL, 30h
16  MOV AH, 0Eh
17  INT 10h
18  INC BX
19  LOOP laço
20  HLT
21
22  END
23

```

Figura 11.29 - Programa MEMO3 na ferramenta emu8086.

Observe então o trecho de código definido entre o rótulo **laço:** e a instrução **LOOP laço**. Note atentamente a utilização das instruções **MOV AL, valor[BX]** e **INC BX**. A instrução **MOV AL, valor[BX]** movimenta um dos conteúdos identificados na área de dados pela variável **valor**. À medida que se obtém memória e sua respectiva apresentação, o programa acrescenta 1 ao valor existente no registrador **[BX]**. Desta forma, é possível obter todos os valores definidos na variável **valor**. O acesso indireto à memória por registrador é útil quando se necessita trabalhar com matrizes.

Em seguida peça a execução do programa pela tecla de função **<F5>** e acione também uma vez a tecla de função **<F8>**. Observe os detalhes apresentados na janela **MEMO3.com**, como indica a Figura 11.30.

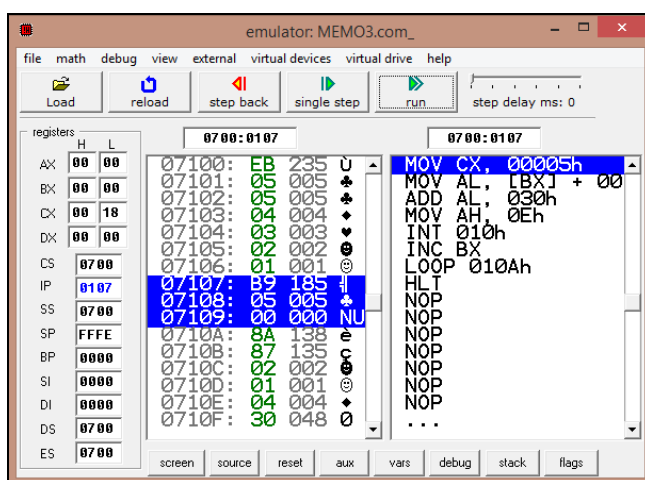


Figura 11.30 - Janela MEMO3.com _ na ferramenta emu8086.

Observe atentamente as informações apresentadas nas áreas **disassemble** e **memory**. A primeira instrução do programa **MOV CX, 00005h** está posicionada no endereço de memória **0700:0107**. Vale a pena fazer um paralelo entre o código escrito no editor e o código assumido pelo simulador. Atente para a Tabela 11.3.

Tabela 11.3 - Código no editor e no simulador

Código no Editor	Código no Simulador
MOV CX, 05h	MOV CX, 00005h
laco:	
MOV AL, valor[BX]	MOV AL, [BX] + 00102h
ADD AL, 030h	ADD AL, 030h
MOV AH, 0Eh	ADD AH, 0Eh
INT 010h	INT 010h
INC BX	INC BX
LOOP laco	LOOP 010Ah
HLT	HLT

Considere a linha de código **MOV AL, valor[BX]** (código no editor) com a linha de código **MOV AL, [BX] + 00102h** (código no simulador).

O primeiro valor (valor **05d**) definido na área de dados representada pela variável **valor** está posicionado no endereço de memória **0102h** do segmento **0700h**, como pode ser observado na área **memory** da janela **emulator: MEMO2.com_** na Figura 11.30.

O uso do registrador **[BX]** na linha de código **MOV AL, valor[BX]** (que internamente é interpretada como **MOV AL, [BX] + 00102h**) possibilita obter o primeiro valor da área de dados (**valor**) que está armazenado na posição **0** (zero). Por este motivo é que antes da instrução **LOOP** se encontra a instrução **INC BX**, que acrescenta **1** (um) ao valor atual do registrador **[BX]**. A Tabela 11.4 demonstra a ocorrência interna dos valores do registrador **[BX]**.

Tabela 11.4 – Ocorrência interna dos valores do registrador [BX]

Registrador [BX]	Instrução MOV AL, [BX] + 00102h	Conteúdo de Memória
0	0 + 00102h = 00102h	5
1	1 + 00102h = 00103h	4
2	2 + 00102h = 00104h	3
3	3 + 00102h = 00105h	2
4	4 + 00102h = 00106h	1

Utilize em conjunto as teclas **<Shift> + <F8>** para executar o programa. Observe a cada movimentação as alterações dos valores nos registradores e a apresentação dos valores na tela do monitor de vídeo.

O programa **MEMO3** pode também ser escrito de uma maneira um pouco diferente da forma apresentada. A Tabela 11.5 exibe as versões **MEMO4** e **MEMO5** com outras formas de referência de uso do acesso indireto com registrador.

Tabela 11.5 – Comparativo de acesso indireto com registrador

MEMO4.asm	MEMO5.asm
<pre>.CODE MOV CX, 05h laco: MOV AL, [BX + valor] ADD AL, 30h MOV AH, 0Eh INT 10h INC BX LOOP laco HLT END</pre>	<pre>.CODE MOV CX, 05h laco: MOV AL, [BX] + valor ADD AL, 30h MOV AH, 0Eh INT 10h INC BX LOOP laco HLT END</pre>

Internamente os programas deste tópico serão interpretados e executados da mesma forma.

11.2.4 - Acesso de base indexada

O acesso de endereçamento de base indexada é uma variação do endereçamento indireto com registradores. Ele utiliza os registradores de base **BX** e **BP** vinculados aos registradores de índice **SI** e **DI**. As Figuras 11.31 e 11.32 apresentam as formas permitidas de uso.

O acesso a uma determinada posição de memória por meio de endereçamento de base indexada é conseguido com a soma de um registrador de base **BX** ou **BP** com um registrador de índice **SI** ou **DI**. Por exemplo, se o registrador **BX** estiver com o valor **0102h** e o registrador **SI** estiver com o valor **03h**, a instrução **MOV AL, [BX + SI]** carrega o registrador menos significativo **AL** com o valor **0105h**.

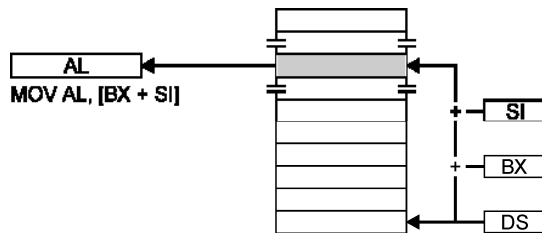


Figura 11.31 - Endereçamento de base indexada (Adaptado de Hyde, 2003).

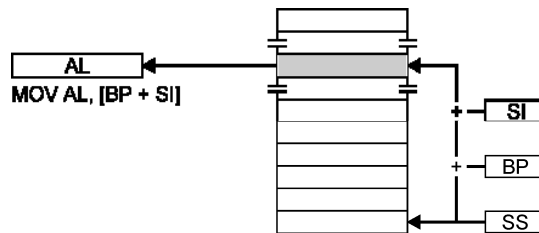


Figura 11.32 - Endereçamento de base indexada (Adaptado de Hyde, 2003).

Para exemplificar o acesso de base indexada de memória considere o seguinte programa:

```
;*****
;*   Programa: MEMO6.ASM   *
;*****

org 100h

.DATA
    valor DB 05d, 04d, 03d, 02d, 01d

.CODE

MOV    CX, 05h
LEA    BX, valor
laco:
    MOV    AL, [BX + SI]
    ADD    AL, 30h
    MOV    AH, 0Eh
    INT    10h
    INC    SI
    LOOP   laco
    HLT

END
```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **MEMO6**, de forma que fique semelhante à imagem da Figura 11.33.

Peça a execução do programa pela tecla de função **<F5>** e acione também uma vez a tecla de função **<F8>**. A partir desse ponto vá acionando o conjunto de teclas **<Shift> + <F8>** e observe a cada ciclo do laço as mudanças ocorridas nos registradores **BX** e **SI**.

```

01  ;*****
02  ; Programa: MEMO6.ASM
03  ;*****
04
05  org 100h
06
07  .DATA
08  valor DB 05d, 04d, 03d, 02d, 01d
09
10  .CODE
11
12  MOV CX, 05h
13  LEA BX, valor
14  laco:
15  MOV AL, [BX + SI]
16  ADD AL, 30h
17  MOV AH, 0Eh
18  INT 10h
19  INC SI
20  LOOP laco
21  HLT
22
23  END
24

```

Figura 11.33 - Programa MEMO6 na ferramenta emu8086.

O uso dos registradores **[BX + SI]** na linha de código **MOV AL, [BX + SI]** possibilita a obtenção dos valores armazenados da área de dados (**valor**). O primeiro valor está armazenado na posição (representado pelo registrador **SI**) de memória **0** (zero), por este motivo é que antes da instrução **LOOP** se encontra a instrução **INC SI**, que acrescenta 1 (um) ao valor atual do registrador **[SI]**. A Tabela 11.6 demonstra a ocorrência interna dos valores dos registradores **[BX + SI]**.

Tabela 11.6 - Ocorrência interna dos valores dos registradores **[BX + SI]**

Registrador [SI]	Instrução MOV AL, [BX + SI]	Conteúdo de Memória
0	00102h + 0 = 00102h	5
1	00102h + 1 = 00103h	4
2	00102h + 2 = 00104h	3
3	00102h + 3 = 00105h	2
4	00102h + 4 = 00106h	1

11.2.5 - Acesso de base indexada mais deslocamento

O acesso de endereçamento de base indexada mais deslocamento é uma variação do endereçamento indexado com endereçamento de base indexada. O acesso de base indexada mais deslocamento utiliza os registradores de base **BX** e **BP** vinculados aos registradores de índice **SI** e **DI** e ao valor de deslocamento (constante de 8 ou 16 *bits*). As Figuras 11.34 e 11.35 apresentam as formas permitidas de uso.

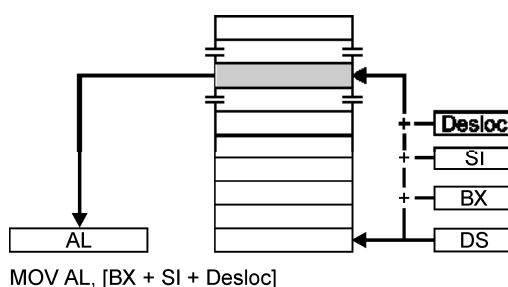


Figura 11.34 - Base indexada com deslocamento (Adaptado de Hyde, 2003).

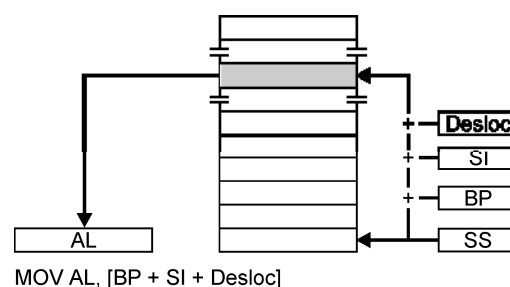


Figura 11.35 - Base indexada com deslocamento (Adaptado de Hyde, 2003).

O acesso a uma determinada posição de memória pelo endereçamento de base indexada com deslocamento é conseguido com a soma de um registrador de base **BX** ou **BP** com um registrador de índice **SI** ou **DI** mais a constante de **DESLOC**. Por exemplo, se o registrador **BX** estiver com o valor **0102h** e o registrador **SI** estiver com o valor **03h**, e a constante for um valor como **02h**, a instrução **MOV AL, [BX + SI + 02h]** carrega o registrador menos significativo **AL** com o valor **0107h**.

Para exemplificar a ação de acesso de base indexada com deslocamento de memória, considere o seguinte programa:

```
;*****
;*   Programa: MEMO7.ASM   *
;*****

org 100h

.DATA
    valor DB 05d, 04d, 03d, 02d, 01d

.CODE

MOV     CX, 05h
LEA     BX, valor - 02h
laco:
    MOV     AL, [BX + SI + 02h]
    ADD     AL, 30h
    MOV     AH, 0Eh
    INT     10h
    INC     SI
    LOOP    laco
    HLT

END
```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **MEMO7**, de forma que fique semelhante à imagem da Figura 11.36.

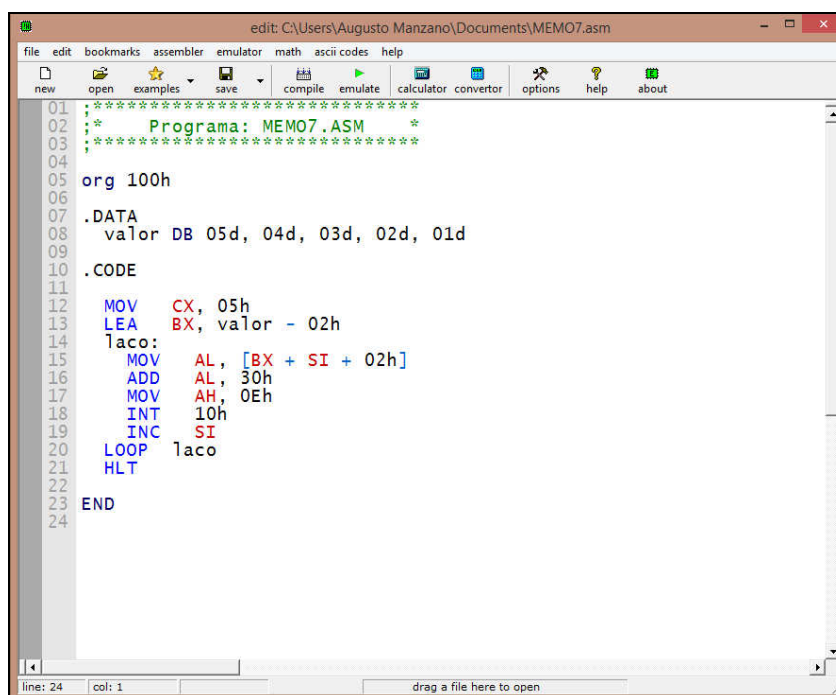


Figura 11.36 - Programa MEMO7 na ferramenta emu8086.

Execute o programa com a tecla de função <F5> e acione também uma vez a tecla de função <F8>. A partir desse ponto vá acionando o conjunto de teclas <Shift> + <F8> e observe a cada ciclo do laço as mudanças ocorridas nos registradores **BX** e **SI**. Atente para o fato da definição do valor constante de deslocamento **02h**.

Para exemplificar o uso da constante, foi alterado na linha de código **LEA BX, valor – 02h** o endereço de início da área de dados **valor** em duas posições a menos na memória. Torna-se possível exemplificar de forma didática o funcionamento do acesso ao endereçamento de base indexada mais deslocamento.

O uso de **[BX + SI + 02h]** na linha de código **MOV AL, [BX + SI + 02h]** possibilita obter os valores armazenados da área de dados (**valor**). A Tabela 11.7 demonstra a ocorrência interna dos valores dos registradores **[BX + SI + 02h]**.

Tabela 11.7 - Ocorrência interna dos valores dos registradores **[BX + SI + 02h]**

Registrador [SI]	Instrução MOV AL, [BX + SI + 02h]	Conteúdo de Memória
0	00100h + 0 + 02h = 00102h	5
1	00100h + 1 + 02h = 00103h	4
2	00100h + 2 + 02h = 00104h	3
3	00100h + 3 + 02h = 00105h	2
4	00100h + 4 + 02h = 00106h	1

11.3 - Prefixos de anulação

O padrão de funcionamento de deslocamentos dentro de um determinado segmento é sempre feito dentro do próprio segmento, ou seja, o deslocamento ocorre no segmento em uso. No entanto, é possível utilizar deslocamentos em segmentos diferentes daquele em uso. É necessário utilizar um conceito denominado *override segment* (segmento anulado ou prefixo de anulação).

Por exemplo, imagine que se queira acessar o endereço de deslocamento **1234h** do segmento de pilha **ES**. Neste caso, deveria ser utilizada a instrução **MOV AX, ES:[1234h]**. De forma semelhante, para acessar esse deslocamento no segmento de código deveria ser utilizada a instrução **MOV AX, CS:[1234h]**. A indicação **ES**: é um prefixo de anulação. Segundo Hyde (2003), o prefixo **:DS** não é de anulação (*override segment*).

O prefixo de anulação pode ser usado com as formas de acesso de memória indireto com registrador e indexado, sendo válidas as seguintes definições:

- ◆ Indireto com registrador:


```
MOV AL, CS:[BX]
MOV AL, DS:[BP]
MOV AL, SS:[SI]
MOV AL, ES:[DI]
```
- ◆ Indexado:


```
MOV AL, SS:DESLOC[BX]
MOV AL, ES:DESLOC[BP]
MOV AL, CS:DESLOC[SI]
MOV AL, SS:DESLOC[DI]
```

O tema deste tópico é um pouco mais complexo, por isso foge aos objetivos iniciais desta obra. Ele é citado apenas a título de ilustração, portanto não será exemplificado.

No entanto, para os mais curiosos, no arquivo de biblioteca **emu8086.inc** que acompanha a ferramenta **emu8086** existem algumas rotinas que utilizam prefixos, além do programa exemplo **ToBin.asm**. Esse material deve ser utilizado como fonte de consulta adicional a esta obra.

11.4 - Operações com Matrizes

As matrizes estão relacionadas ao conjunto de dados armazenado em memória num formato de lista (matriz de uma dimensão) ou tabela (matriz com mais de uma dimensão, normalmente denominada matriz de duas dimensões).

Em vários exemplos aplicados neste capítulo foi utilizada a matriz de uma dimensão de forma implícita. O uso de matriz de forma explícita é conseguido com a sintaxe:

```
vetor    DW 5 DUP (0)
```

Neste caso é definida uma variável denominada **vetor** que tem a capacidade de manipular até cinco valores do tipo *word* inicializados com o valor **0** (zero). Se for necessário trabalhar com uma matriz com mais de uma dimensão (duas dimensões), deve ser utilizada a sintaxe:

```
matriz   DW 5 DUP (3 DUP (0))
```

Neste caso é definida uma variável **matriz**, a qual é inicializada com 15 *words* com valor zero. Essa forma seria algo similar a uma tabela de cinco linhas e três colunas.

Para exemplificar e ilustrar a entrada e apresentação de cinco valores inteiros positivos em uma matriz, considere o seguinte código de programa:

```
#MAKE_COM#

; Programas do tipo .COM são iniciados no endereço CS:0100h
org 100h

;*****
;*      Programa: MEM08.ASM      *
;*****

INCLUDE 'emu8086.inc'

.DATA
    tamanho EQU 05h
    vetor    DB tamanho DUP ('X')
    msg1a    DB 'Entre o ', 0h
    msg1b    DB 'o. valor numerico: ', 0h
    msg2a    DB '0 ', 0h
    msg2b    DB 'o. valor informado equivale a: ', 0h
    valor    DB ?

.CODE
    PRINTN 'Programa Matriz'
    PRINTN 'Entre valores na faixa de 0 ate 255'
    PUTC 13d
    PUTC 10d

    MOV CX, tamanho
    MOV valor, 01h
entrada:
    PUSH CX
    LEA SI, msg1a
    CALL PRINT_STRING
    MOV AL, valor
    CALL PRINT_NUM
    INC valor
    LEA SI, msg1b
    CALL PRINT_STRING
    CALL SCAN_NUM
    MOV vetor[BX], CL
    INC BX
    POP CX
    PUTC 13d
    PUTC 10d
    LOOP entrada
```



```

    PUTC 13d
    PUTC 10d

    MOV CX, tamanho
    MOV valor, 01h
saida:
    LEA SI, msg2a
    CALL PRINT_STRING
    MOV AL, valor
    CALL PRINT_NUM
    INC valor
    LEA SI, msg2b
    CALL PRINT_STRING
    MOV AL, vetor[BX - tamanho]
    CALL PRINT_NUM
    INC BX
    PUTC 13d
    PUTC 10d
    LOOP saida

    HLT
    DEFINE_PRINT_NUM
    DEFINE_PRINT_NUM_UN
    DEFINE_PRINT_STRING
    DEFINE_SCAN_NUM
END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **MEMO8**, de forma que fique semelhante à imagem das Figuras 11.37, 11.38 e 11.39.

```

edit: C:\Users\Augusto Manzano\Documents\MEMO8.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
01 ;*****
02 ;* Programa: MEMO8.ASM *
03 ;*****
04
05 INCLUDE 'emu8086.inc'
06
07 .DATA
08 tamanho EQU 05h
09 vetor DB tamanho DUP ('X')
10 msg1a DB 'Entre o ', 0h
11 msg1b DB 'o valor numerico: ', 0h
12 msg2a DB '0 ', 0h
13 msg2b DB 'o valor informado equivale a: ', 0h
14 valor DB ?
15
16 .CODE
17 PRINTN 'Programa Matriz'
18 PRINTN 'Entre valores na faixa de 0 ate 255'
19 PUTC 13d
20 PUTC 10d
21
22 MOV CX, tamanho
23 MOV valor, 01h
24 entrada:
25 PUSH CX
26 LEA SI, msg1a
27 CALL PRINT_STRING
28 MOV AL, valor
29 CALL PRINT_NUM
30 INC valor
31 LEA SI, msg1b

```

Figura 11.37 - Programa MEMO8 com a área de dados.

```

16 .CODE
17 PRINTN 'Programa Matriz'
18 PRINTN 'Entre valores na faixa de 0 ate 255'
19 PUTC 13d
20 PUTC 10d
21
22 MOV CX, tamanho
23 MOV valor, 01h
24 entrada:
25 PUSH CX
26 LEA SI, msg1a
27 CALL PRINT_STRING
28 MOV AL, valor
29 CALL PRINT_NUM
30 INC valor
31 LEA SI, msg1b
32 CALL PRINT_STRING
33 CALL SCAN_NUM
34 MOV vetor[BX], CL
35 INC BX
36 POP CX
37 PUTC 13d
38 PUTC 10d
39 LOOP entrada
40
41 PUTC 13d
42 PUTC 10d
43
44 MOV CX, tamanho
45 MOV valor, 01h
46 saida:

```

Figura 11.38 - Programa MEMO8 com rotina de entrada de dados.

```

16 .CODE
17 PRINTN 'Programa Matriz'
18 PRINTN 'Entre valores na faixa de 0 ate 255'
19 PUTC 13d
20 PUTC 10d
21
22 MOV CX, tamanho
23 MOV valor, 01h
24 entrada:
25 PUSH CX
26 LEA SI, msg1a
27 CALL PRINT_STRING
28 MOV AL, valor
29 CALL PRINT_NUM
30 INC valor
31 LEA SI, msg1b
32 CALL PRINT_STRING
33 CALL SCAN_NUM
34 MOV vetor[BX], CL
35 INC BX
36 POP CX
37 PUTC 13d
38 PUTC 10d
39 LOOP entrada
40
41 PUTC 13d
42 PUTC 10d
43
44 MOV CX, tamanho
45 MOV valor, 01h
46 saida:

```

Figura 11.39 - Programa MEMO8 com rotina de saída de dados.

A Figura 11.37 apresenta a área de dados do programa. Observe na linha 8 a definição da constante **tamanho** com o valor **05h** (valor máximo de elementos a serem inseridos na variável **vetor**). Desejando trabalhar com um número maior ou menor de valores, basta ajustar o valor **05h** para o desejado.

A linha 9 mostra uma matriz de uma dimensão denominada **vetor** e inicializada na memória com cinco posições contando com o caractere X. Da linha 10 até a linha 13 são definidas as variáveis com as mensagens de orientação do programa.

A linha **14** apresenta a variável **valor** inicializada com um valor desconhecido, representado pelo caractere **?**.

Da linha **22** até a linha **39** encontra-se o trecho de programa responsável pela entrada dos valores e seu armazenamento na memória.

A linha **22** mostra o registrador geral **CX** com o valor armazenado na constante **tamanho**. Isso é necessário, uma vez que a instrução **LOOP** usa o registrador geral **CX** para controlar o número de vezes do laço de repetição em si.

Na linha **23** é exibida a variável **valor** com o valor **01h**, a qual será usada para armazenar o valor do indicador de número de ações do programa.

Da linha **24** até a linha **39** é executada uma série de ações. A primeira delas é armazenar na pilha (linha **25**) o valor do registrador geral **CX**. Isso é necessário devido ao fato de **SCAN_NUM** (biblioteca **emu8086.inc**) utilizar o registrador geral **CX** para a ação da entrada de um valor numérico. Atente para as linhas **33** e **34**, em que está definida a manipulação de transferência do valor armazenado no registrador menos significativo **CL** para o **vetor[BX]**.

O trecho correspondente à saída de dados, Figura 11.33, situa-se entre as linhas **44** e **59**. Observe a linha da instrução da linha **62** correspondente a **MOV AL, vetor[BX - tamanho]**. Está sendo feito um balanceamento do deslocamento a partir de **vetor[BX - tamanho]**. Isso é necessário porque a rotina de entrada movimenta o deslocamento de **[BX]**.

Em relação à utilização de uma matriz de duas dimensões, pode-se utilizar como exemplo o programa **matrix.asm** encontrado na pasta **Samples** que acompanha o programa **emu8086**, o qual inverte os valores existentes entre as linhas e colunas de uma determinada matriz que possui como código:

```
; matrix transpose sample (reverse rows with columns).

name "matrix"

org 100h

jmp start ; go to code...

msg db "to the view matrix click vars button,", 0dh,0ah
    db " and set elements property to 3 for these items:", 0dh,0ah, 0ah
    db "    matrix    ", 0dh,0ah
    db "    row1      ", 0dh,0ah
    db "    row2      ", 0dh,0ah, 0dh,0ah
    db "or add print-out support to this program...", 0dh,0ah, '$'

matrix_size equ 3

; ----- matrix -----
matrix      db 1,2,3
row1        db 4,5,6
row2        db 7,8,9
;-----

i dw ?
j dw ?

start:
mov i, 0
next_i:

    ; j = i + 1
    mov cx, i
    inc cx
    mov j, cx
    next_j:

        mov si, i
        mov bx, j
```

```

    mov al, matrix_size
    mov cx, si
    mul cl
    mov si, ax
    mov dl, matrix[si][bx]

    mov si, i
    mov al, matrix_size
    mul bl
    mov bx, ax
    xchg matrix[bx][si], dl

    mov bx, j
    mov al, matrix_size
    mov cx, si
    mul cl
    mov si, ax
    mov matrix[si][bx], dl

    inc j
    cmp j, matrix_size
    jb next_j
inc i
cmp i, matrix_size/2
jbe next_i

; print message....
lea dx, msg
mov ah, 9
int 21h
; wait for any key press...
mov ah, 0
int 16h
ret

```

A Figura 11.40 exemplifica a ação do programa **matrix.asm** que mostra a primeira linha da matriz torna-se a sua primeira coluna, a segunda linha torna-se a segunda coluna e a terceira linha torna-se a terceira coluna. A seguir é apresentado na íntegra o código do programa **matrix.asm**.

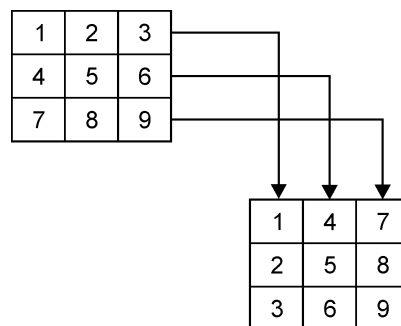


Figura 11.40 - Exemplo de funcionalidade do programa MATRIX.

Para executar e visualizar esse programa, é necessário utilizar alguns outros recursos da ferramenta **emu8086** ainda não apresentados. Primeiramente acione a tecla de função **<F5>** para iniciar o processo de execução do programa.

Depois, conforme orientação no próprio código do programa, acione o comando de menu **view/variables** na janela de depuração para que a caixa de diálogo **variables** seja apresentada como na Figura 11.41.

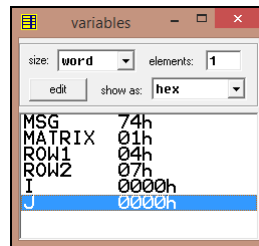


Figura 11.41 - Caixa de diálogo Variables.

A caixa de diálogo **variables** apresenta todas as variáveis definidas no programa. No entanto, não estão sendo apresentados todos os valores das variáveis **MATRIX**, **ROW1** e **ROW2**, apenas mostra o primeiro valor de cada uma delas. Para poder visualizar os três valores de cada variável, selecione com o ponteiro do *mouse* a variável **MATRIX** na lista apresentada. No campo **elements** informe sobre o valor 1 ou valor 3. Repita a mesma ação para as variáveis **ROW1** e **ROW2**, de forma que se tenha um visual semelhante à Figura 11.42.

A Figura 11.42 mostra a mesma sequência de valores definida no código de programa. Coloque o programa em execução e observe ao final as informações da janela **variables**, como indica a Figura 11.43 contendo a inversão dos valores da matriz.

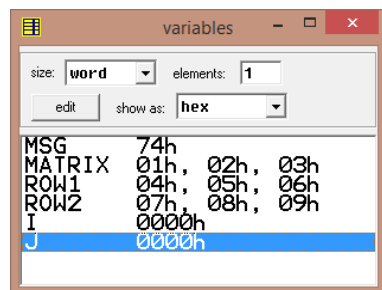


Figura 11.42 - Caixa de diálogo variables com a apresentação dos valores.

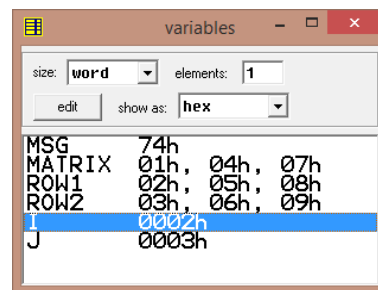


Figura 11.43 - Caixa de diálogo variables com a inversão da matriz.

Para finalizar a apresentação preliminar de introdução à linguagem de programação *Assembly* e no sentido de mostrar mais alguns recursos do programa **emu8086**, carregue para o ambiente de programação o programa **DIVIDE1**, acione a tecla de função **<F5>** e quando for apresentada a tela **emulador: DIVIDE1.exe_**, selecione na parte inferior da tela o botão **aux**. Nas opções apresentadas selecione **listing** e será exibido dentro do programa **Bloco de notas** o código do programa escrito, sendo do lado direito indicado o programa em linguagem *Assembly* e do lado esquerdo é indicado o programa escrito em código de máquina como indica a Figura 11.41.

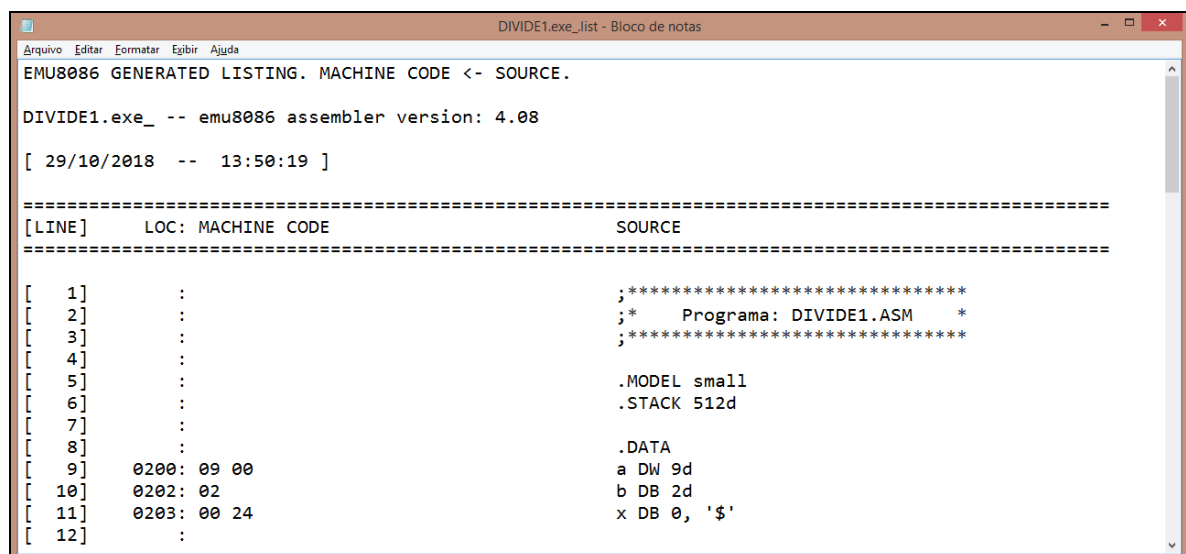


Figura 11.41 - Caixa de diálogo Variables.

No relatório apresentado junto ao programa **Bloco de notas** é indicado além do código em linguagem de máquina e linguagem de montagem a apresentação do cabeçalho de um programa no formato **.EXE**.

11.5 - Programas Exemplo

Além da biblioteca **emu8086.inc**, na pasta **samples** há uma série de programas exemplo que podem ser utilizados como mecanismos de aprendizado e aprofundamento de alguns detalhes operacionais da linguagem de programação de computadores *Assembly 8086/8088*.

Para ter acesso aos exemplos que acompanham a ferramenta, pelo menu principal do programa **emu8086** execute o comando de menu **file/examples** e será apresentada uma lista de programas exemplos como indica a Figura 11.42 ou por meio da pequena seta ao lado direito do botão **examples** como mostra a Figura 11.43.

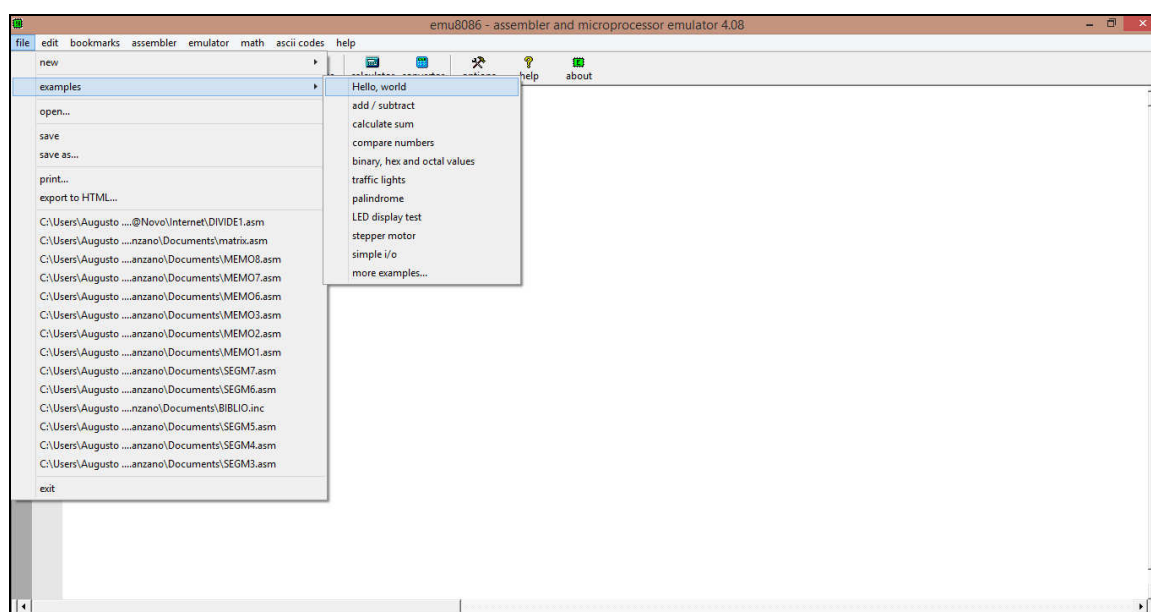


Figura 11.42 - Opções de Exemplos (File/Examples).

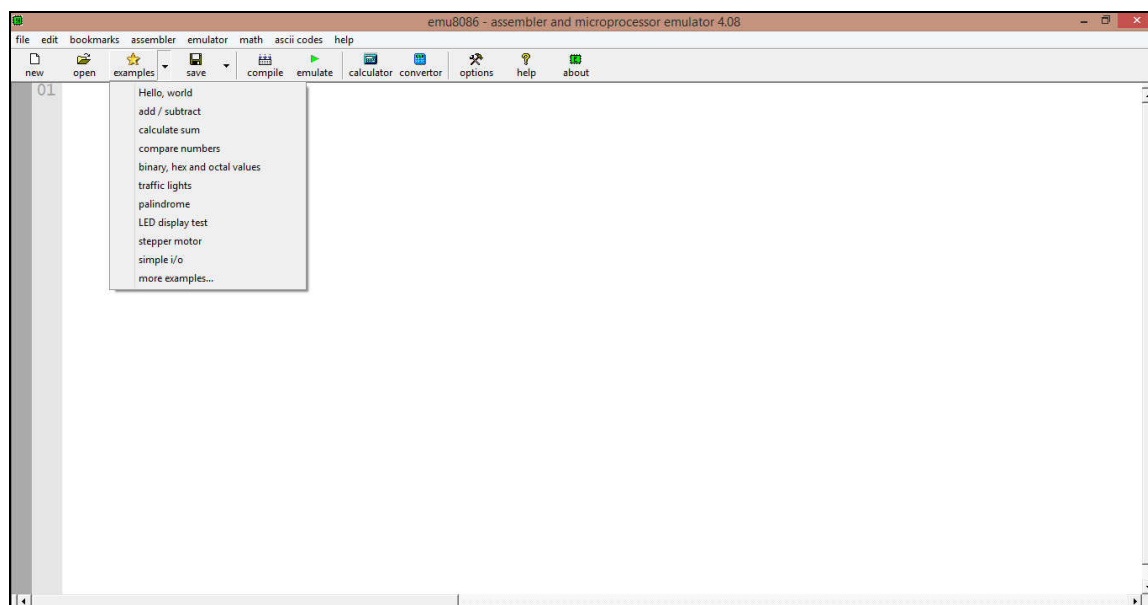


Figura 11.43 - Opções de Exemplos (Botão na barra de ferramentas).

As Figuras 11.42 e 11.43 apresentam um menu com alguns exemplos de programa. Atente para a opção **more samples...** (que indica a existência de mais programas a serem carregados e estudados). Se este botão for acionado, é apresentada a caixa de diálogo **Abrir**, Figura 11.44, que também pode ser aberta com o comando de menu **file/open**, com seleção da pasta **samples**.

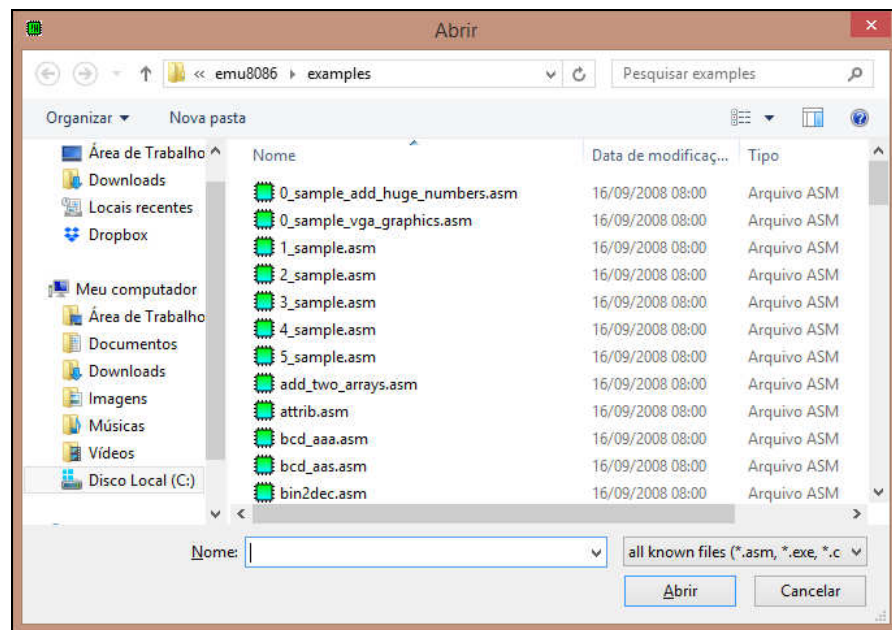


Figura 11.44 - Caixa de diálogo "Abrir" com mais exemplos.

Abra os programas, execute-os e estude os seus códigos. A ferramenta de simulação **emu8086** é voltada para o estudo e a aprendizagem. Por esta razão, além de seu foco didático em mostrar detalhes internos de um microprocessador padrão 8086, ela traz uma extensa coleção de programas para auxiliar por meio de exemplos a aprendizagem. Daqui para frente é só se divertir e aprofundar o conhecimento iniciado neste livro!

Anotações

[illegible]

```

0D82:0100 B402      MOV     AH,02
0D82:0102 B241      MOV     DL,41
0D82:0104 CD21      INT     21
0D82:0106 CD20      INT     20
0D82:0108 69        DB      69

```

Capítulo 12

COMO EM ALTO NÍVEL

Ao longo da apresentação dos capítulos anteriores foram mostradas diversas técnicas simples de trabalho para que se pudesse produzir alguns programas em linguagem de programação Assembly. Neste sentido, foram focadas ações básicas de entrada, processamento e saída. Este capítulo apresenta alguns exemplos de programas complementares, utilizando-se a visão da programação de alto nível para produzir códigos em baixo nível equivalentes para programadores acostumados com o modo de programação em alto nível. São aqui apresentados pequenos programas com enfoque contextualizado a algumas ações básicas para efetuar algumas ações de processamento a partir dos recursos internos existente no ambiente de programação emu8086. Os exemplos de programas apresentados caracterizam-se por serem uma solução possível, mas não necessariamente são as melhores soluções para os problemas apresentados, são as soluções mais didáticas possíveis dentro do escopo deste trabalho.

12.1 - Tomada de decisão

A linguagem de programação Assembly não possui de forma explícita, como ocorre nas linguagens de alto nível, instruções de ação direta para a efetivação de tomadas de decisão. Para que este procedimento ocorra é necessário fazer uso de duas ou mais instruções para tal finalidade. Isto ocorre com o uso dos conceitos de operadores relacionais, operadores lógicos, decisões simples, decisões compostas e decisões seletivas.

A tomada de decisão é efetivada com o uso da instrução **CMP** seguida de uma instrução de desvio condicional que fará o direcionamento do fluxo do programa para o ponto desejado. Além da instrução **CMP** pode-se também fazer uso das instruções de operações lógicas **AND** (conjunção), **OR** (disjunção inclusiva), **NOT** (negação) e **XOR** (disjunção exclusiva) e **NOT** (negação).

Os desvios efetivados após o uso de uma das instruções **CMP**, **AND**, **OR**, **NOT** e **XOR** são feitos com o uso de instruções de saltos que efetuam ações similares as ações do uso dos operadores relacionais. A Tabela 12.1 apresenta um paralelo entre os operadores relacionais de uma linguagem de alto nível com as instruções de saltos da linguagem de programação Assembly dos microprocessadores padrão 8086/8088.

Tabela 12.1 - Tabela Comparativa de Uso das Relações Lógicas

Operador Relacional	Instrução Assembly
=	JE
>	JG
>=	JGE
<	JL
<=	JLE
<>	JNE

Além da existência de instruções para efetivação de ações similares ao uso de operadores relacionais, existem as instruções de ação para efetivação de operadores lógicos. A título de revisão apresenta-se a seguir as tabelas verdades

para os operadores lógicos a serem utilizados com a definição de duas condições. Veja as tabelas 12.2, 12.3, 12.4 e 12.5.

Tabela 12.2 - Operador lógico AND

Tabela verdade do operador lógico de conjunção – AND		
Condição 1	Condição 2	Resultado lógico
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Falso
Falso	Verdadeiro	Falso
Falso	Falso	Falso

Tabela 12.3 - Operador lógico OR

Tabela verdade do operador lógico de disjunção inclusiva - OR		
Condição 1	Condição 2	Resultado lógico
Verdadeiro	Verdadeiro	Verdadeiro
Verdadeiro	Falso	Verdadeiro
Falso	Verdadeiro	Verdadeiro
Falso	Falso	Falso

Tabela 12.4 - Operador lógico XOR

Tabela verdade do operador lógico de disjunção exclusiva – XOR		
Condição 1	Condição 2	Resultado lógico
Verdadeiro	Verdadeiro	Falso
Verdadeiro	Falso	Verdadeiro
Falso	Verdadeiro	Verdadeiro
Falso	Falso	Falso

Tabela 12.5 - Operador lógico NOT

Tabela verdade do operador lógico de negação - NOT	
Condição	Resultado lógico
Verdadeiro	Falso
Falso	Verdadeiro

As ações para os operadores **AND**, **OR** e **XOR** possuem instruções próprias em linguagem *Assembly*. No entanto, para uso do operador lógico **NOT** deve-se usar as instruções de salto que possuem na composição de seus nomes o segundo caractere definido como **N** para as instruções **JNx** ou **JNyy**, onde **x** pode ser **A, B, E, G, L, O, P, S** e **Z** e **yy** pode ser **AE, BE, GE** e **LE**. No caso das ações **AND** e **OR** estas serão executadas sem o uso desses operadores por meio de tomadas de decisão e para o uso do conceito **XOR** será usado a respectiva instrução.

Por uma questão de legibilidade os exemplos a seguir fazem um paralelo entre a forma escrita em alto nível e sua equivalência em baixo nível, e no sentido de manter uma visão didática apresentam em baixo nível alguns detalhes escritos que poderão com o tempo ser removidos

12.1.1 - Decisão simples

A seguir é indicada a forma de representação de uma tomada de decisão simples em pseudocódigo português estruturado, similar a forma existente nas linguagens de programação de alto nível e sua equivalência em linguagem de programação *Assembly*.

Tomada de Decisão Simples em Português Estruturado

```
se ( <elemento 1> <operador relacional> <elemento 2> ) então
    [instruções executadas após condição ser verdadeira]
fim_se
[instruções executadas após condição ser falsa ou ser verdadeira]
```

```

se:
    CMP <elemento 1>, <elemento 2>
    <instrução condicional de salto> fim_se
entao:
    [instruções executadas após condição ser verdadeira]
fim_se:
    [instruções executadas após condição ser falsa ou ser verdadeira]

```

A indicação **elemento 1** e **elemento 2** caracterizam-se por ser a definição do uso de variáveis e/ou constantes na formação de uma condição. Esta relação pode ser composta com a partir de: variáveis versus variáveis ou variáveis versus constantes.

Na definição da condição em português estruturado encontra-se a indicação de uso de um **operador relacional** que está paralelamente assinalado no corpo do código em *assembly* como sendo uma **instrução condicional de salto**.

Após o rótulo **entao:** são definidas as instruções de ação para a condição verdadeira. Caso a condição não seja satisfeita a **instrução condicional de salto** desviará o fluxo de execução do programa para o trecho de instruções definidos após o rótulo **senao:**.

Para exemplificar o uso de tomada de decisão simples considere um programa que efetue a entrada de um valor numérico e mostre a mensagem **"Valor acima de 10"** se o valor informado for maior que dez. Caso contrário o programa não deverá apresentar nenhuma mensagem.

```

;*****
;*   Programa: DECISA01.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA
    valor DW 0
    msg1 DB 'Entre um valor numerico: ', 0
    msg2 DB 'Valor acima de 10.', 0

.CODE
    LEA SI, msg1
    CALL PRINT_STRING
    CALL SCAN_NUM
    MOV valor, CX
    PUTC 13d
    PUTC 10d

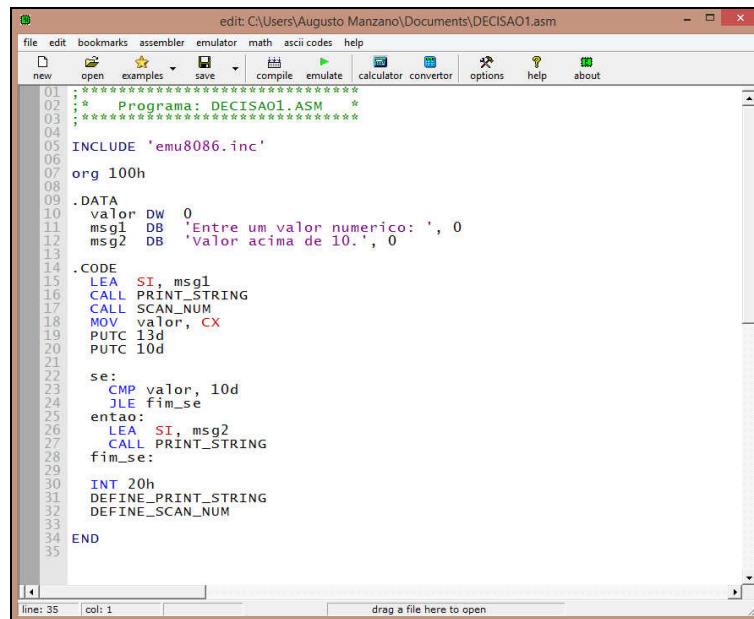
    se:
        CMP valor, 10d
        JLE fim_se
    entao:
        LEA SI, msg2
        CALL PRINT_STRING
    fim_se:

    INT 20h
    DEFINE_PRINT_STRING
    DEFINE_SCAN_NUM

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISA01**.



```
01  ;*****  
02  ; Programa: DECISA01.ASM  
03  ;*****  
04  
05  INCLUDE 'emu8086.inc'  
06  
07  org 100h  
08  
09  .DATA  
10      valor DW 0  
11      msg1 DB 'Entre um valor numerico: ', 0  
12      msg2 DB 'Valor acima de 10.', 0  
13  
14  .CODE  
15      LEA SI, msg1  
16      CALL PRINT_STRING  
17      CALL SCAN_NUM  
18      MOV valor, CX  
19      PUTC 13d  
20      PUTC 10d  
21  
22  se:  
23      CMP valor, 10d  
24      JLE fim_se  
25  entao:  
26      LEA SI, msg2  
27      CALL PRINT_STRING  
28  fim_se:  
29  
30      INT 20h  
31      DEFINE_PRINT_STRING  
32      DEFINE_SCAN_NUM  
33  
34  END  
35
```

Figura 12.1 - Programa DECISA01 na ferramenta emu8086.

Observe na linha **05** a instrução **INCLUDE 'emu8086.inc'** que faz a chamada da biblioteca **emu8086.inc** que possui as funções **PRINT_STRING** / **DEFINE_PRINT_STRING** e **SCAN_NUM** / **DEFINE_SCAN_NUM**. Desta forma, aproveita-se os recursos comuns que podem ser usados na escrita de qualquer programa e direciona-se o foco de trabalho para as ações mais incomuns.

Na linha **10** (**valor DW 0**) está sendo definida uma variável de nome **valor** com valor inicial **0** (zero). Caso queira definir uma variável em memória sem um valor inicial, basta no lugar do valor usar o caractere **?** (interrogação) com a sintaxe **<variável> DW ?**. Esta situação será exposta mais adiante.

A linha **15** (**LEA SI, msg1**) coloca, aponta, no registrador de ponteiro **SI** o endereço de memória onde se encontra a variável **msg1**. Esta forma está sendo utilizada devido ao fato do procedimento **PRINT_STRING** da biblioteca **emu8086.inc** indicado na linha **16** fazer uso do registrador de ponteiro **SI** para acessar o conteúdo *string* da variável indicada. Neste caso, variável **msg1**.

O procedimento **SCAN_NUM** na linha **17** efetua a leitura de um número inteiro fornecido via teclado e o armazena no registrador geral **CX**, devido a esta ocorrência é que a linha **18** (**MOV valor, CX**) transfere o valor do registrador geral **CX** para a variável **valor**.

O ponto do programa que mais interessa é o trecho de código escrito entre as linhas **22** e **28** que estabelecem as instruções de uma tomada de decisão simples. A linha **23** (**CMP valor, 10d**) efetua a comparação do conteúdo da variável **valor** com o valor decimal **10** e em conjunto com a linha **24** (**JLE fim_se**) que faz a vez a tomada de decisão e verifica se a condição estabelecida é falsa ou verdadeira. Na linha **24** está ocorrendo o uso da instrução **JLE** que faz o papel de uso de um operador relacional. Assim sendo, a condição considerada para a tomada da decisão é **valor > 10** a qual executará as instruções das linhas de **25** até **27** caso a condição seja verdadeira e apresentará a mensagem “**Valor acima de 10.**”. No entanto, se a linha **24** detectar que o conteúdo da variável **valor** é menor ou igual a **10** efetua o seu desvio para a linha **28** identificada com o rótulo **fim_se**: e desta forma não apresentará a mensagem.

Aos olhos de alguns programadores a condição estabelecida nas linhas **23** e **24** pode parecer estar em sentido contrário, pois a instrução **JLE** (*Jump on Less or Equal*) efetua um desvio quando a condição verificada com a instrução **CMP** for menor ou igual ao valor estabelecido. Mas tudo é uma questão de ponto de vista, ou seja, de ótica condicional, pois se o pensamento estabelecido para a condição é **valor > 10** e sendo esta condição verdadeira ocorrerá automaticamente a execução das linhas de instrução de **25** até **27** após a avaliação da instrução da linha **24** e quando o resultado da condição **valor > 10** for falso, ou seja, quando o conteúdo da variável **valor** for menor ou igual a **10** será então des-

viado o fluxo de execução do programa para a linha **28**. Neste ponto é bom ter o máximo de cuidado para não criar na mente conceitos equivocados.

12.1.2 - Decisão Composta

A seguir é indicada a forma de representação de uma tomada de decisão composta em pseudocódigo português estruturado, similar a forma existente nas linguagens de programação de alto nível e sua equivalência em linguagem de programação *Assembly*.

Tomada de Decisão Composta em Português Estruturado

```
se ( <elemento 1> <operador relacional> <elemento 2> ) então
    [instruções executadas após condição ser verdadeira]
senão
    [instruções executadas após condição ser falsa]
fim_se
[instruções executadas após condição ser falsa ou ser verdadeira]
```

Tomada de Decisão Composta em Assembly

```
se:
    CMP <elemento 1>, <elemento 2>
    <instrução condicional de salto> senao
entao:
    [instruções executadas após condição ser verdadeira]
    JMP fim_se
senão:
    [instruções executadas após condição ser falsa]
fim_se:
[instruções executadas após condição ser falsa ou ser verdadeira]]
```

A indicação **elemento 1** e **elemento 2** caracterizam-se por ser a definição do uso de variáveis e/ou constantes na formação de uma condição. Esta relação pode ser composta com a partir de: variáveis versus variáveis ou variáveis versus constantes.

Na definição da condição em português estruturado encontra-se a indicação de uso de um **operador relacional** que está paralelamente assinalado no corpo do código em *assembly* como sendo uma **instrução condicional de salto**.

Após o rótulo **entao**: são definidas as instruções de ação para a condição verdadeira e após o término da execução dessas instruções o programa por meio da instrução **JMP fim_se** efetua um salto para o rótulo **fim_se**: e encerra a execução do bloco de tomada de decisão. Caso a condição não seja satisfeita a **instrução condicional de salto** desviará o fluxo de execução do programa para o trecho de instruções definidos após o rótulo **senao**:

Para exemplificar o uso de tomada de decisão simples considere um programa que efetue a entrada de um valor numérico e mostre a mensagem "**Valor igual ou acima de 10**" se o valor informado for maior ou igual a dez. Caso contrário o programa deverá apresentar a mensagem "**Valor abaixo de 10**".

```
;*****
;*   Programa: DECISA02.ASM   *
;*****
```

```
INCLUDE 'emu8086.inc'
```

```
org 100h
```

```

.DATA
    valor DW 0
    msg1 DB 'Entre um valor numerico: ', 0
    msg2 DB 'Valor igual ou acima de 10.', 0
    msg3 DB 'Valor abaixo de 10.', 0

.CODE
    LEA SI, msg1
    CALL PRINT_STRING
    CALL SCAN_NUM
    MOV valor, CX
    PUTC 13d
    PUTC 10d

se:
    CMP valor, 10d
    JL senao
entao:
    LEA SI, msg2
    CALL PRINT_STRING
    JMP fim_se
senao:
    LEA SI, msg3
    CALL PRINT_STRING
fim_se:

INT 20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISAO2**, de forma que fique semelhante à imagem da Figura 12.2.

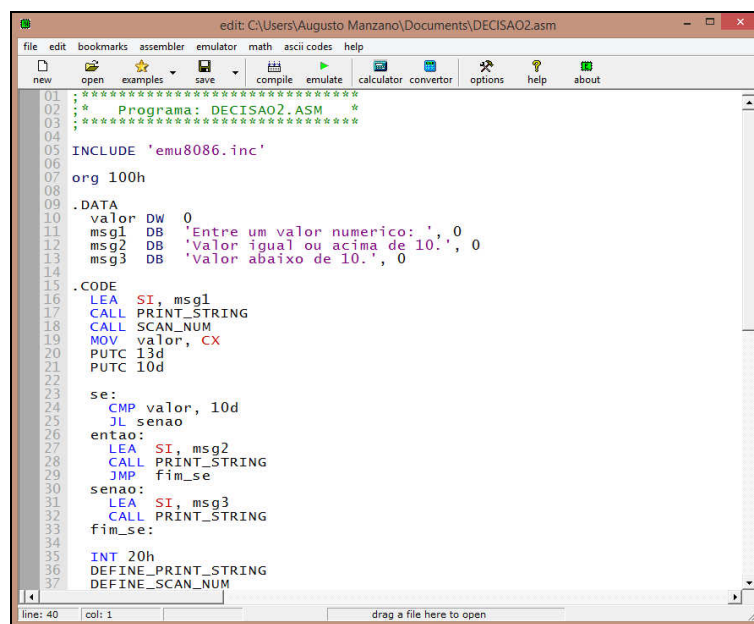


Figura 12.2 - Programa DECISAO2 na ferramenta emu8086.

O ponto do programa que mais interessa é o trecho de código escrito entre as linhas 23 e 33 que estabelecem as instruções de uma tomada de decisão composta. A linha 24 (**CMP valor, 10d**) efetua a comparação do conteúdo da variá-

vel **valor** com o valor decimal **10** e em conjunto com a linha **25** (**JL fim_se**) que faz a vez a tomada de decisão e verifica se a condição estabelecida é falsa ou verdadeira. Na linha **25** está ocorrendo o uso da instrução **JL** que faz o papel de uso de um operador relacional. Assim sendo, a condição considerada para a tomada da decisão é **valor >= 10** a qual executará as instruções das linhas de **27** até **29** caso a condição seja verdadeira e apresentará a mensagem “**Valor igual ao acima de 10.**”. No entanto, se a linha **25** detectar que o conteúdo da variável **valor** é menor que **10** efetua o seu desvio para a linha **30** identificada com o rótulo **senão**: e executa a apresentação da mensagem “**Valor abaixo de 10.**”.

12.1.3 - Decisões com Duas Condições

Neste tópico serão apresentados exemplos que farão uso dos conceitos de aplicação dos operadores lógicos **AND**, **OR** e **XOR**. Todos os exemplos aqui apresentados estarão baseados na estrutura de tomada de decisão composta com base no seguinte estilo.

Para a efetivação de uma ação de tomada de decisão com base no uso do conceito do operador lógico de conjunção **AND** considere a estrutura de código seguinte:

Tomada de Decisão com Operador AND em Português Estruturado

```
se (condição1) .e. (condição2) então
    [instruções executadas após condição1 e condição2 serem verdadeiras]
senão
    [instruções executadas caso uma ou ambas as condições sejam falsas]
fim_se
```

Tomada de Decisão com Operador AND em Assembly

```
se:
    CMP <condição1>
    <instrução condicional de salto> senao
e:
    CMP <condição2>
    <instrução condicional de salto> senao
entao:
    [instruções executadas após condição1 e condição2 serem verdadeiras]
    JMP fim_se
senão:
    [instruções executadas caso uma ou ambas as condições sejam falsas]
fim_se:
```

O programa seguinte efetua a entrada de um valor numérico inteiro em uma variável chamada **valor** e apresenta a mensagem “**Valor esta na faixa de 10 a 90**” caso o valor fornecido seja maior ou igual a 10 e menor ou igual a 90. Caso esta condição não seja satisfeita o programa apresentará a mensagem “**Valor esta fora da faixa de 10 a 90**”. Note que este programa faz uso do operador lógico de conjunção com a condição **valor >= 10** e **valor <= 90**.

```
;*****
;*   Programa: DECISA03.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h
```



```

.DATA
    valor    DW 0
    msg1     DB 'Entre um valor numerico: ', 0
    msg2     DB 'Valor esta na faixa de 10 a 90.', 0
    msg3     DB 'Valor esta fora da faixa de 10 a 90.', 0

.CODE
    LEA      SI, msg1
    CALL     PRINT_STRING
    CALL     SCAN_NUM
    MOV      valor, CX
    PUTC     13d
    PUTC     10d

se:
    CMP     valor, 10d
    JL      senao
e:
    CMP     valor, 90d
    JG      senao
entao:
    LEA      SI, msg2
    CALL     PRINT_STRING
    JMP      fim_se
senao:
    LEA      SI, msg3
    CALL     PRINT_STRING
fim_se:

INT        20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISAO3**, de forma que fique semelhante à imagem da Figura 12.3.

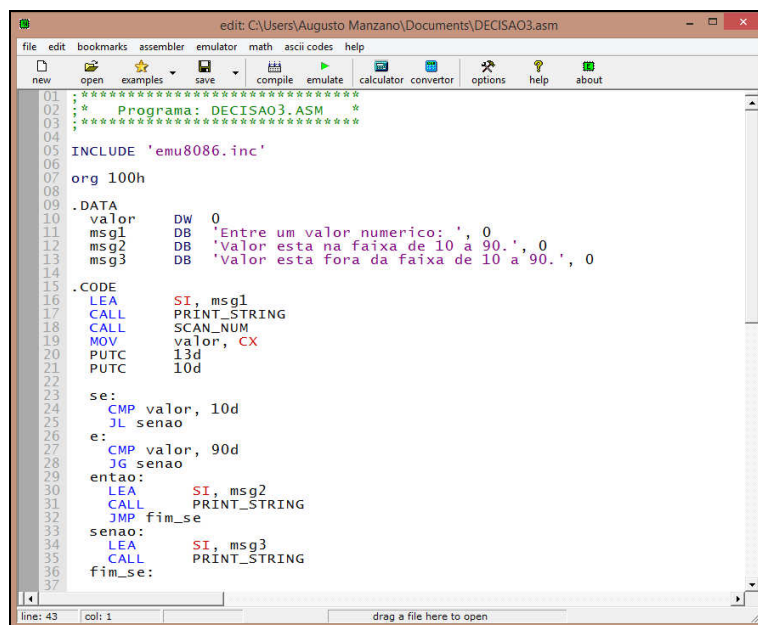


Figura 12.3 - Programa DECISAO3 na ferramenta emu8086.

O ponto do programa que mais interessa é o trecho de código entre as linhas **23** e **36** que estabelecem as instruções de uma tomada de decisão composta com o uso da ação de um operador lógico de conjunção **AND** (e em português). A linha **24** (**CMP valor, 10d**) efetua a comparação do conteúdo da variável **valor** com o valor decimal **10** (primeira condição) e a linha **25** (**JL senao**) faz o desvio para a linha **33** (identificada pelo rótulo **senao**;) caso o conteúdo da variável **valor** não seja maior ou igual a **10** apresentando a mensagem “**Valor esta fora da faixa de 10 a 90.**” e encerra a tomada da decisão. Se a condição estabelecida nas linhas **24** e **25** for maior ou igual a **10** o fluxo de execução do programa avança automaticamente para as linhas de **26** até **28** e efetua nas linhas **27** (**CMP valor, 90d**) e **28** (**JG senao**) a verificação da segunda condição que caso não seja menor ou igual a **90** desvia o fluxo de execução do programa para a linha **33** e apresenta a mensagem “**Valor esta fora da faixa de 10 a 90.**” e encerra a tomada da decisão. Caso o resultado da condição das linhas **27** e **28** não seja verdadeiro o fluxo do programa segue automaticamente a sequência estabelecida entre as linhas **29** e **32** apresentando a mensagem “**Valor esta na faixa de 10 a 90.**” E desviando o fluxo do programa para a linha **36** onde ocorrer o encerramento da decisão.

Para a efetivação de uma ação de tomada de decisão com base no uso do conceito do operador lógico de disjunção inclusiva **OR** considere a estrutura de código seguinte:

Tomada de Decisão Composta com Operador OR em Português Estruturado

```
se (condição1) .ou. (condição2) então
    [instruções executadas quando pelo menos uma das condições for verdadeira]
senão
    [instruções executadas caso ambas as condições sejam falsas]
fim_se
```

Tomada de Decisão Composta com Operador OR em Assembly

```
se:
    CMP <condição1>
    <instrução condicional de salto> senao
ou:
    CMP <condição2>
    <instrução condicional de salto> senao
entao:
    [instruções executadas quando pelo menos uma das condições for verdadeira]
    JMP fim_se
senão:
    [instruções executadas caso ambas as condições sejam falsas]
fim_se:
```

O programa a seguir solicita a sexo de uma pessoa e apresenta uma mensagem informando se o sexo fornecido é ou não válido. Será permitida a entrada dos valores **1** para sexo masculino e **2** para sexo feminino, qualquer outra entrada resultará na apresentação de uma mensagem de erro. As mensagens a serem apresentadas são: “**Sexo valido**” para informações de sexo como **1** ou **2** e “**Sexo invalido**” para qualquer outra entrada diferente de **1** ou **2**. Note que este programa faz uso do operador lógico de disjunção inclusiva com a condição **sexo = 1 ou sexo = 2**.

```
;*****
;*   Programa: DECISA04.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h
```

```

.DATA
sexo      DW  0
msg1      DB  'Informe sexo - [1] Masculino ou [2] Feminino: ', 0
msg2      DB  'Sexo invalido.', 0
msg3      DB  'Sexo valido.', 0

.CODE
LEA       SI, msg1
CALL      PRINT_STRING
CALL      SCAN_NUM
MOV       sexo, CX
PUTC      13d
PUTC      10d

se:
    CMP    sexo, 1d
    JE     senao
ou:
    CMP    sexo, 2d
    JE     senao
entao:
    LEA     SI, msg2
    CALL    PRINT_STRING
    JMP     fim_se
senao:
    LEA     SI, msg3
    CALL    PRINT_STRING
fim_se:

INT        20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISAO4**, de forma que fique semelhante à imagem da Figura 12.4.

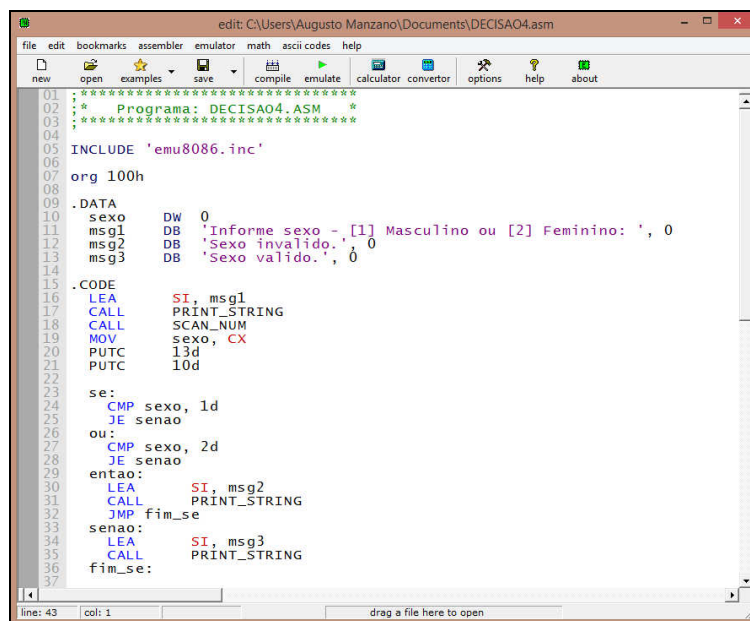


Figura 12.4 - Programa DECISAO4 na ferramenta emu8086.

O ponto do programa que interessa é o trecho de código entre as linhas **23** e **36** que estabelecem as instruções de uma tomada de decisão composta com o uso da ação de um operador lógico de disjunção inclusiva **OR** (**ou** em português). A linha **24** (**CMP sexo, 1d**) efetua a comparação do conteúdo da variável **sexo** com o valor decimal **1** (primeira condição) e a linha **25** (**JE senao**) faz o desvio para a linha **33** (identificada pelo rótulo **senao**;) caso o conteúdo da variável **sexo** seja igual a **1** e neste caso será apresentada a mensagem “**Sexo válido**” e faz-se o encerramento da tomada de decisão. Caso o sexo informado não seja **1** o programa continua o seu fluxo de execução a partir da linha **26** onde encontra a segunda condição: linha **27** (**CMP sexo, 2d**) que se verificada com resposta verdadeira pela linha **28** (**JE senao**) faz o desvio para a linha **33** e apresenta a mensagem “**Sexo valido.**” direcionando para o encerramento da tomada de decisão. Caso as condições das linhas **24** e **25** e das linhas **27** e **28** não sejam verdadeiras ocorrerá a execução das instruções das linhas de **29** até **32**. Neste caso, será apresentada a mensagem “**Sexo invalido.**” e a instrução da linha **32** (**JMP fim_se**) direcionará o fluxo para o encerramento da tomada de decisão.

Para a efetivação de uma ação de tomada de decisão com uso do operador lógico de disjunção exclusiva **XOR** considere a estrutura de código seguinte:

Tomada de Decisão Composta com Operador XOR em Português Estruturado

```
se (condição1) .xou. (condição2) então
    [instruções executadas quando condição1 verdadeira e condição2 falsa ou quando]
    [condição1 falsa e condição2 verdadeira]
senão
    [instruções executadas caso as condições sejam falsas ou sejam verdadeiras]
fim_se
```

Tomada de Decisão Composta com Operador XOR em Assembly

```
se:
    MOV <registrador>, <variável condição1>
    XOR <registrador>, <variável condição2>
    <instrução condicional de salto> senao
entao:
    [instruções executadas quando condição1 verdadeira e condição2 falsa ou quando]
    [condição1 falsa e condição2 verdadeira]
    JMP fim_se
senão:
    [instruções executadas caso as condições sejam falsas ou sejam verdadeiras]
fim_se:
```

A indicação **variável condição1** e **variável condição2** caracterizam-se por ser a definição dos valores condicionais a serem avaliados.

O próximo programa será usado para definir a composição de um par de dança entre dois participantes, sendo permitido apenas a formação de pares de sexos diferentes. Pares de mesmo sexo não serão permitidos. Assim sendo, o programa apresentará a mensagem “**O 1o participante dança com o 2o participante.**” se o sexo informado para o primeiro participante for masculino e para o segundo participante for feminino ou vice-versa. Caso sejam informados sexos feminino ou masculino para ambos os participantes o programa deverá apresentar a mensagem “**O 1o participante nao dança com o 2o participante.**”.

```

;*****
;*   Programa: DECISA05.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA
sexo1    DW  0
sexo2    DW  0
msg1     DB  'Informe sexo participante 1 - [1] Masc. ou [2] Fem. : ', 0
msg2     DB  'Informe sexo participante 2 - [1] Masc. ou [2] Fem. : ', 0
msg3     DB  '0 1o participante dança com o 2o participante.', 0
msg4     DB  '0 1o participante nao dança com o 2o participante.', 0

.CODE
LEA       SI, msg1
CALL      PRINT_STRING
CALL      SCAN_NUM
MOV       sexo1, CX
PUTC      13d
PUTC      10d

LEA       SI, msg2
CALL      PRINT_STRING
CALL      SCAN_NUM
MOV       sexo2, CX
PUTC      13d
PUTC      10d

se:
MOV       AX, sexo1
XOR       AX, sexo2
JE        senao
entao:
LEA       SI, msg3
CALL      PRINT_STRING
JMP       fim_se
senao:
LEA       SI, msg4
CALL      PRINT_STRING
fim_se:

INT       20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISA05**, de forma que fique semelhante à imagem da Figura 12.5.

```

01 ;*****
02 ;* Programa: DECISAO5.ASM *
03 ;*****
04
05 INCLUDE 'emu8086.inc'
06
07 org 100h
08
09 .DATA
10  sexo1 DW 0
11  sexo2 DW 0
12  msg1 DB 'Informe sexo participante 1 - [1] Masc. ou [2] Fem. : ',
13  msg2 DB 'Informe sexo participante 2 - [1] Masc. ou [2] Fem. : ',
14  msg3 DB 'O 1o participante dança com o 2o participante.', 0,
15  msg4 DB 'O 1o participante nao dança com o 2o participante.', 0
16
17 .CODE
18  LEA SI, msg1
19  CALL PRINT_STRING
20  CALL SCAN_NUM
21  MOV sexo1, CX
22  PUTC 13d
23  PUTC 10d
24
25  LEA SI, msg2
26  CALL PRINT_STRING
27  CALL SCAN_NUM
28  MOV sexo2, CX
29  PUTC 13d
30  PUTC 10d
31
32  se:
33  MOV AX, sexo1
34  XOR AX, sexo2
35  JE senao
36  entao:
37  LEA SI, msg3

```

Figura 12.5 - Programa DECISAO5 na ferramenta emu8086.

O ponto do programa que interessa é o trecho de código entre as linhas 32 e 43 que estabelecem as instruções de uma tomada de decisão composta com o uso do operador lógico de disjunção exclusiva **XOR** (xou em português estruturado). A linha 33 (**MOV AX, sexo1**) movimenta para o registrador geral **AX** o valor armazenado na variável **sexo1**, em seguida a linha 34 (**XOR AX, sexo2**) efetua a comparação do conteúdo da variável **sexo1** armazenado no registrador geral **AX** com o valor armazenado na variável **sexo2** (a instrução **XOR** não opera sua ação direta sobre duas variáveis). A linha 35 (**JE senao**) faz o desvio para a linha 40 (identificada pelo rótulo **senao**;) caso o conteúdo avaliado pela instrução **XOR** seja 0 (zero). O valor 0 (zero) obtido pela instrução **XOR** acontece quando os valores avaliados são iguais, ou seja, é considerado falso. Caso os valores avaliados pela instrução **XOR** sejam diferentes o retorno será um valor 1 (um) que equivale a verdadeiro. Sendo a condição verdadeira será apresentada a mensagem “O 1o participante dança com o 2o participante.”, caso contrário ocorre a apresentação da mensagem “O 1o participante nao dança com o 2o participante.”.

12.1.4 - Decisões Sequências

A aplicação de decisões sequenciais se caracterizam quando se faz uso de várias tomadas de decisão simples ou compostas uma após a outra. Para a efetivação de uma ação de tomada de decisão sequencial considere a estrutura de código seguinte:

Tomada de Decisão Sequencial em Português Estruturado

```

se (condição1) então
    [instruções executadas após condição1 ser verdadeira]
fim_se

```

```

se (condição2) então
    [instruções executadas após condição2 ser verdadeira]
fim_se

```

```

se1:
    CMP <condição1>
    <instrução condicional de salto> fim_se1
entao1:
    [instruções executadas após condição1 ser verdadeira]
fim_se1:

se2:
    CMP <condição2>
    <instrução condicional de salto> fim_se2
entao2:
    [instruções executadas após condição2 ser verdadeira]
fim_se2:

```

No sentido de exemplificar este tipo de ocorrência considere um programa que solicite a entrada de um valor numérico (variável **N**) e apresente uma das seguintes mensagens: "**Você entrou o valor 1.**" se for dada a entrada do valor numérico 1; "**Você entrou o valor 2.**" se for dada a entrada do valor numérico 2; "**Você entrou um valor muito baixo**" se for dada a entrada de um valor numérico menor que 1 ou "**Você entrou um valor muito alto**" se for dada a entrada de um valor numérico maior que 2.

```

;*****
;*   Programa: DECISA06.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA
    N    DW 0
    msg1 DB 'Voce entrou o valor 1.', 0
    msg2 DB 'Voce entrou o valor 2.', 0
    msg3 DB 'Voce entrou um valor muito baixo.', 0
    msg4 DB 'Voce entrou um valor muito alto.', 0
    msg5 DB 'Entre um valor numerico: ', 0

.CODE
    LEA    SI, msg5
    CALL   PRINT_STRING
    CALL   SCAN_NUM
    MOV    N, CX
    PUTC   13d
    PUTC   10d

    se1:
        CMP N, 1d
        JNE fim_se1
    entao1:
        LEA SI, msg1
        CALL PRINT_STRING
    fim_se1:

    se2:
        CMP N, 2d
        JNE fim_se2
    entao2:
        LEA SI, msg2

```

```

CALL PRINT_STRING
fim_se2:

se3:
    CMP N, 1d
    JGE fim_se3
entao3:
    LEA SI, msg3
    CALL PRINT_STRING
fim_se3:

se4:
    CMP N, 1d
    JLE fim_se4
entao4:
    LEA SI, msg4
    CALL PRINT_STRING
fim_se4:

INT      20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

```

END

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho <Ctrl> + <A> do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISAO6**, de forma que fique semelhante à imagem da Figura 12.6.

```

edit: C:\Users\Augusto Manzano\Documents\DECISAO6.asm
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
01 *****
02 * Programa: DECISAO6.ASM *
03 *****
04
05 INCLUDE 'emu8086.inc'
06
07 org 100h
08
09 .DATA
10     N DW 0
11     msg1 DB 'Voce entrou o valor 1.', 0
12     msg2 DB 'Voce entrou o valor 2.', 0
13     msg3 DB 'Voce entrou um valor muito baixo.', 0
14     msg4 DB 'Voce entrou um valor muito alto.', 0
15     msg5 DB 'Entre um valor numerico:', 0
16
17 .CODE
18     LEA SI, msg5
19     CALL PRINT_STRING
20     CALL SCAN_NUM
21     MOV N, CX
22     PUTC 13d
23     PUTC 10d
24
25     se1:
26     CMP N, 1d
27     JNE fim_se1
28     entao1:
29     LEA SI, msg1
30     CALL PRINT_STRING
31     fim_se1:
32
33     se2:
34     CMP N, 2d
35     JNE fim_se2
36     entao2:
37     LEA SI, msg2

```

Figura 12.6 - Programa DECISAO6 na ferramenta emu8086.

Observe no programa o uso da sequência de tomadas de decisão uma após a outra sinalizadas com os rótulos **se1...entao1...fim_se1** até **se4...entao4...fim_se4**. Quando um nome de rótulo é usado para uma ação este não pode ser usado para outra ação. Desta forma, torna-se necessário fazer uso de nomes diferentes.

12.1.5 - Decisão Seletiva

A tomada de decisão seletiva é uma alternativa ao uso de tomadas de decisão sequenciais. Essa estrutura lógica de condição é útil e pode ser usada em situações em que se possui um grande número de verificações lógicas a serem

realizadas. A desvantagem desta estrutura de decisão está no fato desta ser usada apenas para condições que usem o operador relacional de igualdade. Para a efetivação de uma ação de tomada de decisão seletiva considere a estrutura de código seguinte:

Tomada de Decisão Seletiva em Português Estruturado

```
caso <variável>
  seja <opção 1> faça
    [ação para condição1 verdadeira]
  seja <opção 2> faça
    [ação para condição2 verdadeira]
  seja <opção 3> faça
    [ação para condição3 verdadeira]
senão
  [ação para nenhuma condição satisfeita]
fim_caso
```

Tomada de Decisão Seletiva em Assembly

```
caso:
  seja01:
    CMP <condição1>
    JNE seja02
    JE faca01
  faca01:
    [ação para condição1 verdadeira]
    JMP fim_caso
  seja02:
    CMP <condição2>
    JNE seja03
    JE faca02
  faca02:
    [ação para condição2 verdadeira]
    JMP fim_caso
  seja03:
    CMP <condição3>
    JNE senao
    JE faca03
  faca03:
    [ação para condição3 verdadeira]
    JMP fim_caso
  senao:
    [ação para nenhuma condição satisfeita]
fim_caso:
```

Desenvolver um programa de computador que leia um valor numérico entre os valores 1 e 12 e apresente por extenso o nome do mês correspondente ao valor entrado. Caso sejam fornecidos valores menores que 1 e maiores que 12, o programa deve apresentar a mensagem "Valor invalido".

```

;*****
;*   Programa: DECISA07.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA
N      DW 0
msg00  DB 'Entre um valor numerico: ', 0
msg01  DB 'Janeiro', 0
msg02  DB 'Fevereiro', 0
msg03  DB 'Marco', 0
msg04  DB 'Abril', 0
msg05  DB 'Maio', 0
msg06  DB 'Junho', 0
msg07  DB 'Julho', 0
msg08  DB 'Agosto', 0
msg09  DB 'Setembro', 0
msg10  DB 'Outubro', 0
msg11  DB 'Novembro', 0
msg12  DB 'Dezembro', 0
msg13  DB 'Valor invalido', 0

.CODE
LEA     SI, msg00
CALL    PRINT_STRING
CALL    SCAN_NUM
MOV     N, CX
PUTC    13d
PUTC    10d

caso:
seja01:
CMP N, 01d
JNE seja02
JE faca01
faca01:
LEA SI, msg01
CALL PRINT_STRING
JMP fim_caso
seja02:
CMP N, 02d
JNE seja03
JE faca02
faca02:
LEA SI, msg02
CALL PRINT_STRING
JMP fim_caso
seja03:
CMP N, 03d
JNE seja04
JE faca03
faca03:
LEA SI, msg03
CALL PRINT_STRING
JMP fim_caso
seja04:
CMP N, 04d
JNE seja05
JE faca04
faca04:

```

```

    LEA SI, msg04
    CALL PRINT_STRING
    JMP fim_caso
seja05:
    CMP N, 05d
    JNE seja06
    JE faca05
faca05:
    LEA SI, msg05
    CALL PRINT_STRING
    JMP fim_caso
seja06:
    CMP N, 06d
    JNE seja07
    JE faca06
faca06:
    LEA SI, msg06
    CALL PRINT_STRING
    JMP fim_caso
seja07:
    CMP N, 07d
    JNE seja08
    JE faca07
faca07:
    LEA SI, msg07
    CALL PRINT_STRING
    JMP fim_caso
seja08:
    CMP N, 08d
    JNE seja09
    JE faca08
faca08:
    LEA SI, msg08
    CALL PRINT_STRING
    JMP fim_caso
seja09:
    CMP N, 09d
    JNE seja10
    JE faca09
faca09:
    LEA SI, msg09
    CALL PRINT_STRING
    JMP fim_caso
seja10:
    CMP N, 10d
    JNE seja11
    JE faca10
faca10:
    LEA SI, msg10
    CALL PRINT_STRING
    JMP fim_caso
seja11:
    CMP N, 11d
    JNE seja12
    JE faca11
faca11:
    LEA SI, msg11
    CALL PRINT_STRING
    JMP fim_caso
seja12:
    CMP N, 12d
    JNE senao
    JE faca12

```

```

faca12:
    LEA SI, msg12
    CALL PRINT_STRING
    JMP fim_caso
senao:
    LEA SI, msg13
    CALL PRINT_STRING
fim_caso:

INT      20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

```

END

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho <Ctrl> + <A> do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISAO7**, de forma que fique semelhante à imagem da Figura 12.7.

```

001  ; *****
002  ; Programa: DECISAO7.ASM
003  ; *****
004
005  INCLUDE 'emu8086.inc'
006
007  org 100h
008
009  .DATA
010  N DW 0
011  msg00 DB 'Entre um valor numerico: ', 0
012  msg01 DB 'Janeiro', 0
013  msg02 DB 'Fevereiro', 0
014  msg03 DB 'Marco', 0
015  msg04 DB 'Abril', 0
016  msg05 DB 'Maio', 0
017  msg06 DB 'Junho', 0
018  msg07 DB 'Julho', 0
019  msg08 DB 'Agosto', 0
020  msg09 DB 'Setembro', 0
021  msg10 DB 'Outubro', 0
022  msg11 DB 'Novembro', 0
023  msg12 DB 'Dezembro', 0
024  msg13 DB 'Valor invalido', 0
025
026  .CODE
027  LEA SI, msg00
028  CALL PRINT_STRING
029  CALL SCAN_NUM
030  MOV N, CX
031  PUTC 13d
032  PUTC 10d
033
034  caso:
035  seja01:
036  CMP N, 01d
037  JNE seja02

```

Figura 12.7 - Programa DECISAO7 na ferramenta emu8086.

Observe no programa que para executar a ação de uma estrutura **caso...seja...faça...fim_caso** usa-se uma sequência de operações de comparação **CMP** com auxílio das instruções de salto **JNE**, **JE** e **JMP**.

12.1.6 - Decisão Encadeada

A tomada de decisão encadeada ocorre quando se utilizam tomadas de decisão simples ou compostas uma dentro de outra, onde para a tomada de decisão mais interna ser efetivada depende da verificação da tomada de decisão mais externa. Para uso de uma ação de tomada de decisão encadeada considere a estrutura de código seguinte:

Tomada de Decisão Encadeada em Português Estruturado

```
se (<condição1>) então
  se (<condição2>) então
    [ação para condição 1 e condição 2 verdadeiras]
  senão
    [ação para condição 1 verdadeira e condição 2 falsa]
  fim_se
senão
  [ação para condição 1 falsa]
fim_se
```

Tomada de Decisão Encadeada em Assembly

```
se1:
  CMP <condição1>
  <instrução condicional de salto> senao1
entao1:
  se2:
    CMP <condição2>
    <instrução condicional de salto> senao2
  entao2:
    [instruções executadas após condição ser verdadeira]
    JMP fim_se2
  senao2:
    [ação para condição 1 verdadeira e condição 2 falsa]
  fim_se2:
    JMP fim_se1
senao1:
  [ação para condição 1 falsa]
fim_se1:
```

Para fazer uso deste recurso considere um programa que informe ao usuário se certo valor fornecido está abaixo de 10, entre 10 e 50 ou acima de 50.

```
;*****
;*   Programa: DECISA08.ASM   *
;*****
```

```
INCLUDE 'emu8086.inc'
```

```
org 100h
```

```
.DATA
```

```
  N      DW 0
  msg1   DB 'Entre um valor: ', 0
  msg2   DB 'Valor entre 10 e 50.', 0
  msg3   DB 'Valor acima de 50.', 0
  msg4   DB 'Valor abaixo de 10.', 0
```

```
.CODE
```

```
  LEA     SI, msg1
  CALL    PRINT_STRING
  CALL    SCAN_NUM
  MOV     N, CX
  PUTC    13d
  PUTC    10d
```

```

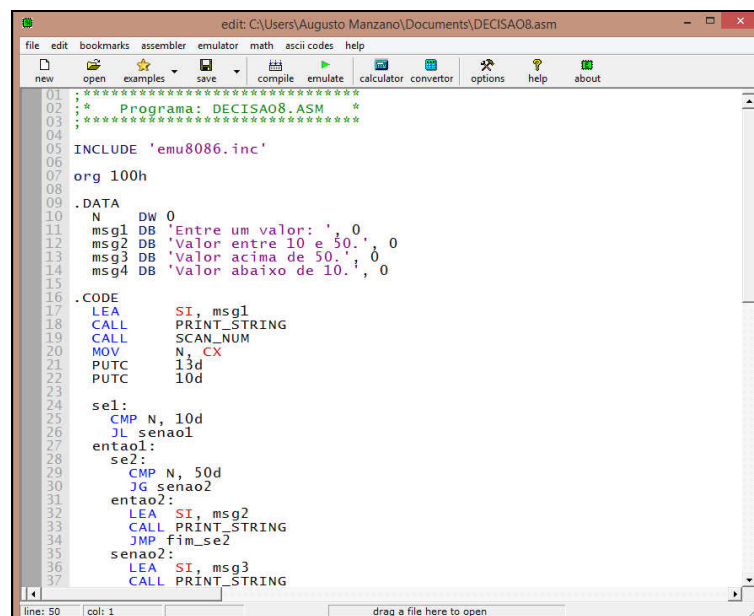
se1:
    CMP N, 10d
    JL senao1
entao1:
    se2:
        CMP N, 50d
        JG senao2
    entao2:
        LEA SI, msg2
        CALL PRINT_STRING
        JMP fim_se2
    senao2:
        LEA SI, msg3
        CALL PRINT_STRING
    fim_se2:
        JMP fim_se1
senao1:
    LEA SI, msg4
    CALL PRINT_STRING
fim_se1:

INT      20h
DEFINE_PRINT_STRING
DEFINE_SCAN_NUM

```

END

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho <Ctrl> + <A> do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **DECISAO8**, de forma que fique semelhante à imagem da Figura 12.8.



```

01 ;
02 ; *****
03 ; Programa: DECISAO8.ASM
04 ; *****
05 INCLUDE 'emu8086.inc'
06
07 org 100h
08
09 .DATA
10 N DW 0
11 msg1 DB 'Entre um valor: ', 0
12 msg2 DB 'valor entre 10 e 50.', 0
13 msg3 DB 'valor acima de 50.', 0
14 msg4 DB 'valor abaixo de 10.', 0
15
16 .CODE
17 LEA SI, msg1
18 CALL PRINT_STRING
19 CALL SCAN_NUM
20 MOV N, CX
21 PUTC 13d
22 PUTC 10d
23
24 se1:
25 CMP N, 10d
26 JL senao1
27 entao1:
28 se2:
29 CMP N, 50d
30 JG senao2
31 entao2:
32 LEA SI, msg2
33 CALL PRINT_STRING
34 JMP fim_se2
35 senao2:
36 LEA SI, msg3
37 CALL PRINT_STRING

```

Figura 12.8 - Programa DECISAO8 na ferramenta emu8086.

12.2 - Laços de Repetição

A linguagem de programação *Assembly* não possui de forma explícita, como ocorre nas linguagens de alto nível, instruções de ação direta para a efetivação de laços. Os laços de repetição são uma estrutura de programação que repete determinado trecho de código certo número de vezes, podendo-se classificá-los como *laços de repetição interativa* ou *laços de repetição iterativa*. São *interativos* quando ocorre a intervenção de um usuário do programa para repetir a próxima ação, são *iterativos* quando executam as repetições previstas de forma automática um número de vezes conhecido. Os exemplos de laços apresentados a seguir são laços do tipo iterativo, quanto aos laços interativos ficam a cargo do leitor implementá-los.

As estruturas de laços de repetição escritas em linguagem de programação *Assembly* são mais simples de serem implementadas do que são as estruturas de tomadas de decisão. Assim sendo, os exemplos de uso das formas de laços existentes a seguir partem de um programa exemplo que apresenta no monitor de vídeo a mensagem “**Linguagem Assembly**” por 5 vezes, iniciando a contagem da variável *I* de 1 até 5 com incremento 1. Serão apresentados três laços, sendo: *controle condicional pré-teste*, *controle condicional pós-teste* e *controle condicional seletivo*.

12.2.1 - Laço Condicional Pré-Teste

O laço de repetição condicional pré-teste executa as instruções subordinadas de um bloco adjacente no período em que o resultado lógico da condição permanece verdadeiro. No momento em que o resultado lógico da condição se tornar falso, a execução do laço é automaticamente encerrada. Para uso de uma ação de laço condicional pré-teste verdadeiro considere a estrutura de código seguinte:

Laço Condicional Pré-teste em Português Estruturado

```
I = <valor inicial>
enquanto (I <= <valor final>) faça
    [ação executada enquanto condição é verdadeira]
    I = I + 1
fim_enquanto
```

Laço Condicional Pré-teste em Assembly

```
MOV I, <valor inicial>
enquanto:
    CMP I, <valor final>
    JG fim_enquanto
faca:
    [ação executada enquanto condição é verdadeira]
    INC I
    JMP enquanto:
fim_enquanto
```

Para fazer uso deste tipo de laço de repetição considere o programa seguinte:

```
;*****
;*   Programa: LAC03.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h
```

```

.DATA
    I      DW 0
    msg1 DB 'Linguagem Assembly', 0

.CODE

    MOV I, 1d
    enquanto:
        CMP I, 5d
        JG fim_enquanto
    faca:

        LEA SI, msg1
        CALL PRINT_STRING
        PUTC 13d
        PUTC 10d

        INC I
        JMP enquanto
    fim_enquanto:

    INT 20h
    DEFINE_PRINT_STRING

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO3**, de forma que fique semelhante à imagem da Figura 12.9.

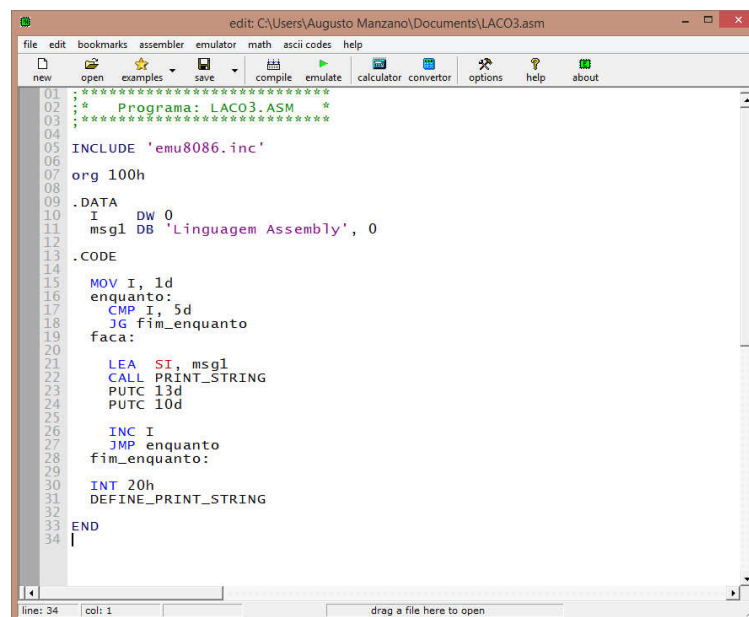


Figura 12.9 - Programa LACO3 na ferramenta emu8086.

A partir da estrutura de laço condicional pré-teste considere o desenvolvimento de um programa que apresente valores de 0 (zero) a 9 (nove) sendo um valor por linha. Assim sendo, observe o código seguinte.

```

;*****
;*   Programa: APLICASM1.ASM   *
;*****

org 100h

```



```

.DATA
    I DW 0

.CODE

    MOV DL, '0' ; 30h
    MOV I, 1d
    enquanto:
        CMP I, 10d
        JG fim_enquanto
    faca:
        MOV AH, 02h
        INT 21h
        PUSH AX
        MOV AL, 10d
        MOV AH, 0Eh
        INT 10h
        POP AX
        PUSH AX
        MOV AL, 13d
        MOV AH, 0Eh
        INT 10h
        POP AX
        INC DL
        INC I
        JMP enquanto
    fim_enquanto:

    INT 20h

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **File/Save** com o nome **APLICASM1**, de forma que fique semelhante à imagem da Figura 12.10.

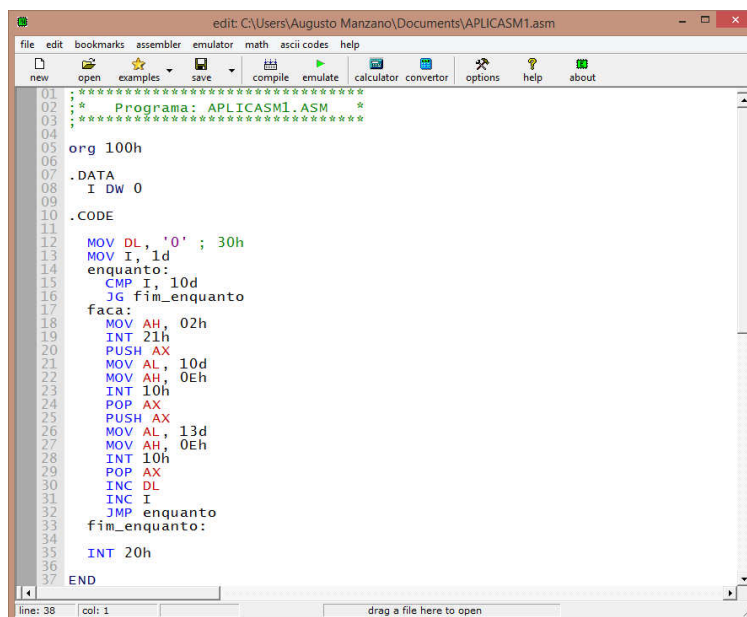


Figura 12.10 - Programa APLICASM1 na ferramenta emu8086.

O programa anterior não está fazendo usos dos recursos existentes na biblioteca **emu8086.ini** por esta razão os efeitos usados anteriormente pela macro **PUTC** para movimentar o cursor para a próxima linha do monitor de vídeo com os

códigos **10d** e **13d** estão sendo descritos entre as linhas **20** e **24** para execução do código **10d** e entre as linhas **25** e **29** para execução do código **13d**. Os códigos **10d** e **13d** podem ser usados na ordem **13d** e **10d** e efetuam o retorno de carro, ou seja, o cursor volta para a posição inicial da linha em uso quando se usa o código **13d** e a movimentação do cursor para a próxima linha. Quando se faz uso da tecla **<Enter>** esta tecla gera internamente a execução dos códigos **10d** e **13d**.

A linha de código **12 (MOV DL, '0' ; 30h)** carrega para a parte baixa do registrador **DX** o caractere **0** (zero) entre aspas simples, o qual pode ser substituído pelo valor **30h** (representação do valor zero na tabela ASCII) e que será impresso pela execução das linhas **18 (MOV AH, 02h)** e **19 (INT 21h)**. Lembre-se de que o valor **02h** carregado na parte alta do registrador **AX** é usado para efetuar a apresentação do caractere armazenado na parte baixa do registrador **DX** por meio da instrução da linha **19**.

As linhas de código **13**, **14-17** e **31-33** são responsáveis pela execução de dez passos do laço condicional pré-teste como orientado. Atenção maior é em relação a linha de código **30 (INC DL)** dentro do laço que efetua a soma de mais **1** sobre o valor armazenado na parte baixa do registrador **DX** e assim pega o próximo caractere numérico.

12.2.2 - Laço Condicional Pós-Teste

O laço de repetição condicional pós-teste executa no mínimo uma vez as instruções subordinadas de um bloco adjacente e mantém a execução no período em que o resultado lógico da condição permanece falso. No momento em que o resultado lógico da condição se torna verdadeiro, a execução do laço é automaticamente encerrada. Para uso de uma ação de laço condicional pré-teste verdadeiro considere a estrutura de código seguinte:

Laço Condicional Pós-teste em Português Estruturado

```
I = <valor inicial>
repita:
    [ação executada até que a condição torne-se verdadeira]
    I = I + 1
até_que (I > <valor final>)
```

Laço Condicional Pós-teste em Assembly

```
MOV I, <valor inicial>
repita:
    [ação executada até que a condição torne-se verdadeira]
    INC I
ate_que:
    CMP I, <valor final>
    JLE repita
```

Para fazer uso deste tipo de laço de repetição considere o programa seguinte:

```
;*****
;*   Programa: LAC04.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h
```

```

.DATA
I      DW 0
msg1 DB 'Linguagem Assembly', 0

.CODE

MOV I, 1d
repita:

    LEA SI, msg1
    CALL PRINT_STRING
    PUTC 13d
    PUTC 10d

    INC I
ate_que:
CMP I, 5d
JLE repita

INT 20h
DEFINE_PRINT_STRING

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO3**, de forma que fique semelhante à imagem da Figura 12.11.

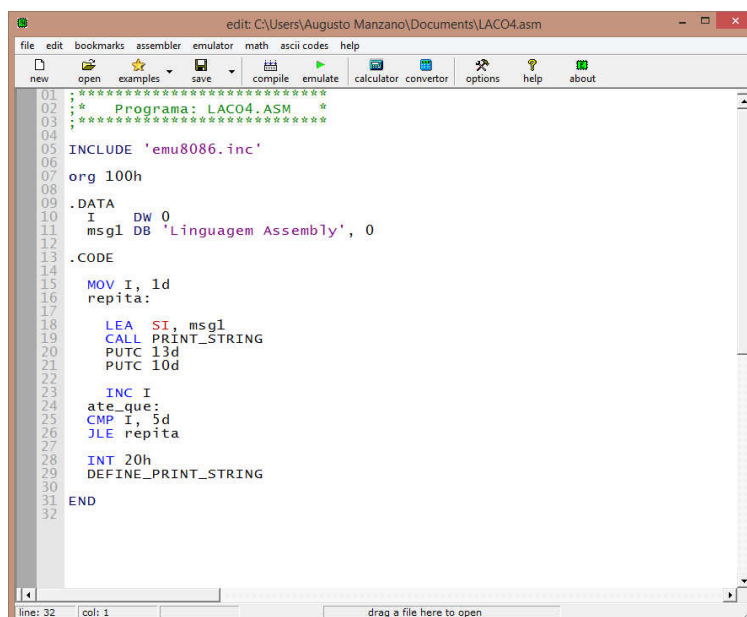


Figura 12.11 - Programa LACO4 na ferramenta emu8086.

A partir da estrutura de laço condicional pós-teste considere o desenvolvimento de um programa que apresente numa linha as letras do alfabeto em formato maiúsculo e numa segunda linha as letras do alfabeto em formato minúsculo. Assim sendo, observe o código seguinte.

```

;*****
;*   Programa: APLICASM2.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA
    I DW 0

.CODE

    MOV DL, 'A' ; 41h
    MOV I, 1d
repita1:
    MOV AH, 02h
    INT 21h
    INC DL
    INC I
ate_que1:
    CMP I, 26d
    JLE repita1

    PUTC 13d
    PUTC 10d

    MOV DL, 'a' ; 61h
    MOV I, 1d
repita2:
    MOV AH, 02h
    INT 21h
    INC DL
    INC I
ate_que2:
    CMP I, 26d
    JLE repita2

    INT 20h

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **APLICASM2**, de forma que fique semelhante à imagem da Figura 12.12.

Este programa é de certa forma semelhante ao programa anterior, exceto a estrutura de laço de repetição em uso. O trecho situado entre as linhas **14** e **23** apresenta o alfabeto em caracteres maiúsculos e o trecho de linhas entre **28** e **37** apresenta o alfabeto em caracteres minúsculos.

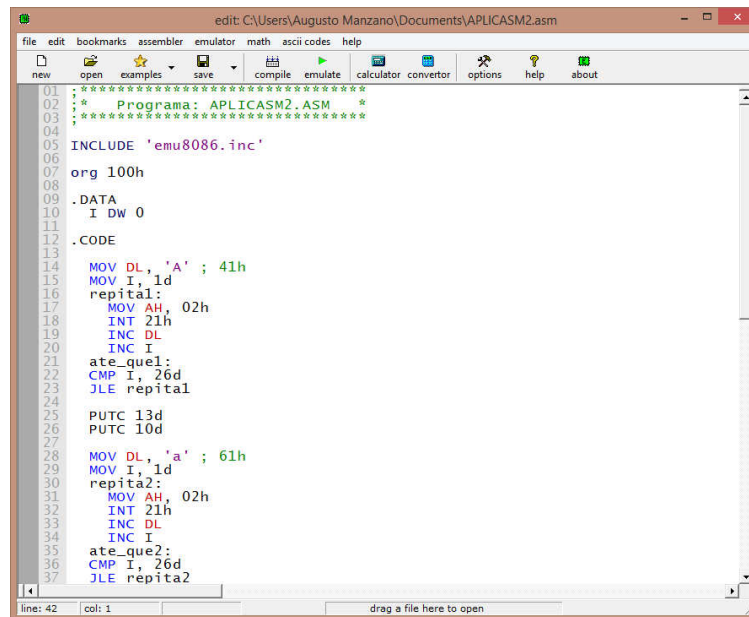


Figura 12.12 - Programa APLICASM2 na ferramenta emu8086.

12.2.3 - Laço Incondicional

O laço de repetição incondicional executa um bloco adjacente certo número de vezes definido. Para uso de uma ação de laço condicional pré-teste verdadeiro considere a estrutura de código seguinte:

Laço Incondicional em Português Estruturado

para I de [<inicio>] **até** [<fim>] **passo** 1 **faça**
 [<instruções executadas durante o ciclo de contagem da variável de controle>]
fim_para

Laço Incondicional em Assembly

```

MOV I, <valor inicial>
para:
    CMP I, <valor final>
    JG fim_para
faca:
    [ação executada até que a condição torne-se verdadeira]
    INC I
    JMP para
fim_para:

```

Para fazer uso deste tipo de laço de repetição considere o programa seguinte:

```

;*****
;*   Programa: LAC05.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

```

```

.DATA
    I      DW 0
    msg1 DB 'Linguagem Assembly', 0

.CODE

    MOV I, 1d
para:
    CMP I, 5d
    JG fim_para
faca:

    LEA SI, msg1
    CALL PRINT_STRING
    PUTC 13d
    PUTC 10d

    INC I
    JMP para
fim_para:

    INT 20h
    DEFINE_PRINT_STRING

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **LACO5**, de forma que fique semelhante à imagem da Figura 12.13.

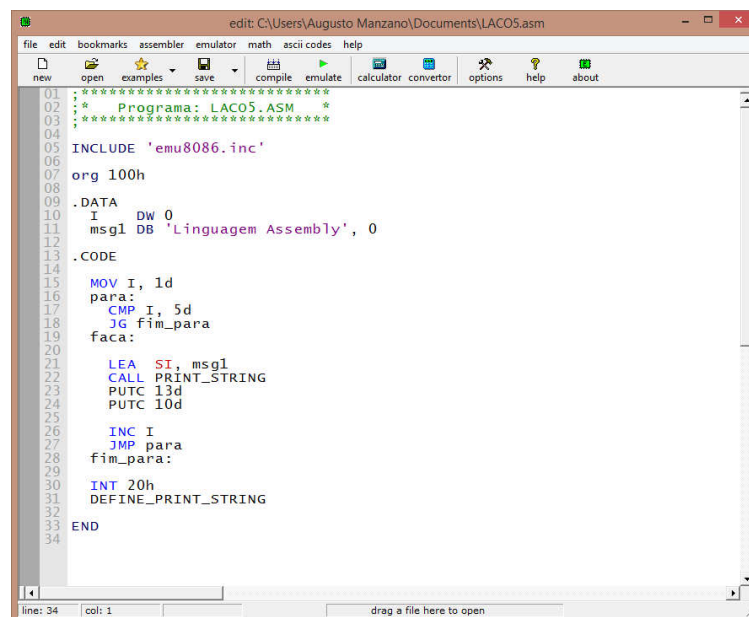


Figura 12.13 - Programa LACO5 na ferramenta emu8086.

A partir da estrutura de laço incondicional considere o desenvolvimento de um programa que apresente linearmente todos os caracteres da tabela ASCII situados na faixa de valores de **0Eh** até **7Fh**. Assim sendo, observe o código seguinte.

```

;*****
;*  Programa: APLICASM3.ASM  *
;*****

```

```

org 100h

.DATA
    I DW 0

.CODE

    MOV DL, 0Eh
    MOV I, 1d
    para:
        CMP I, 114d
        JG fim_para
    faca:
        MOV AH, 02h
        INT 21h
        INC DL
        INC I
        JMP para
    fim_para:

    INT 20h

END

```

Execute no programa **emu8086** o comando de menu **file/new/com template**, acione as teclas de atalho **<Ctrl> + <A>** do editor de texto e escreva o programa anterior, gravando-o por meio dos comandos de menu **file/save** com o nome **APLICASM3**, de forma que fique semelhante à imagem da Figura 12.14.

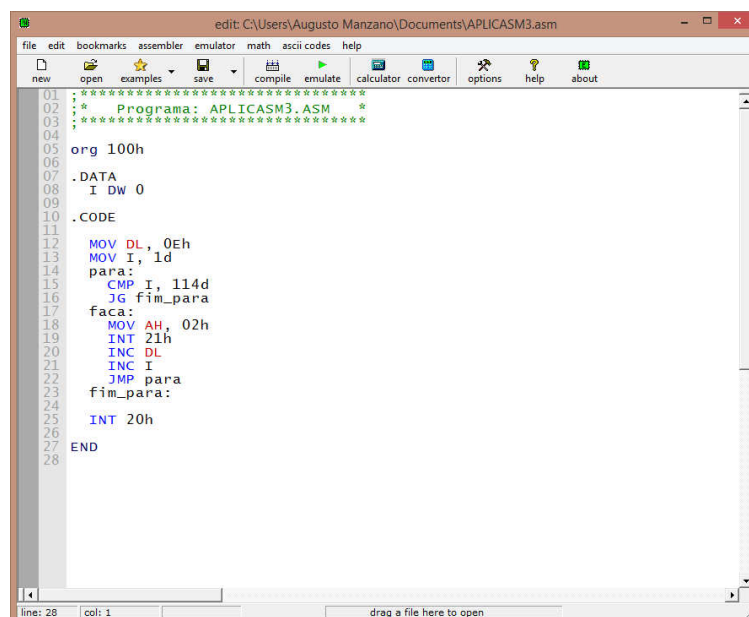


Figura 12.14 - Programa APLICASM3 na ferramenta emu8086.

Observe que o programa inicia a apresentação dos caracteres ASCII a partir do código **0Eh** e estende por 114 caracteres a apresentação dos caracteres da tabela.

12.3 - Demonstrações Assembly

Este tópico tem por objetivo a partir do exposto neste e nos demais capítulos desta obra apresentar como demonstração alguns exemplos de programas escritos em linguagem de baixo nível *Assembly 8086/8088*. Os exemplos apresentados baseiam-se numa situação problema simples. Deguste mentalmente cada uma das situações expostas e observe

os detalhes apresentados. As explicações principais sobre a ação de cada programa estão definidas dentro do próprio código como linhas de comentários precedidas do caractere ponto e vírgula.

Como sugestão complementar de estudo abra o arquivo de biblioteca **emu8086.inc** existente no diretório (pasta) de trabalho **C:\lemu\binaries\inc** e observe os procedimentos definidos para o tratamento das operações de entrada e saída. Estude também os programas apresentados no diretório **C:\lemu\binaries\examples**, além do modo de ajuda do programa. São ricas fontes de informação existente dentro da própria ferramenta de trabalho.

ORDENAÇÃO DE VALORES

Desenvolver um programa de computador que efetue a leitura de três valores numéricos e apresente estes valores dispostos em ordem crescente.

```
;*****
;*   Programa: ORDENA.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA

    A DW ? ; Definição de variáveis para entrada de dados
    B DW ?
    C DW ?

    msg1 DB          'Entre o 1o. valor: ', '$'
    msg2 DB 0Dh, 0Ah, 'Entre o 2o. valor: ', '$'
    msg3 DB 0Dh, 0Ah, 'Entre o 3o. valor: ', '$'

    msg4 DB 0Dh, 0Ah, 0Dh, 0Ah, 'Valor 1: ', '$'
    msg5 DB 0Dh, 0Ah,          'Valor 2: ', '$'
    msg6 DB 0Dh, 0Ah,          'Valor 3: ', '$'

.CODE

; //////////////////////////////////
; // Entrada de dados //
; //////////////////////////////////

; Efetua a entrada do 1o. valor (Bloco de ação 1)

LEA  DX, msg1 ; Pega a mensagem da variável msg1
MOV  AH, 09h  ; Põe 09h na parte baixa de AX
INT  21h      ; Imprime msg1 de DX por meio do código 09h
CALL SCAN_NUM ; Efetua a leitura do valor e põe em CX
MOV  A, CX    ; Transfere o valor de CX para a variável A

; Efetua a entrada do 2o. valor (Bloco de ação 2)

LEA  DX, msg2 ; Pega a mensagem da variável msg2
MOV  AH, 09h  ; Ações similares ao bloco 1
INT  21h      ;
CALL SCAN_NUM ;
MOV  B, CX    ; Transfere o valor de CX para a variável B

; Efetua a entrada do 3o. valor (Bloco de ação 3)

LEA  DX, msg3 ; Pega a mensagem da variável msg3
MOV  AH, 09h  ; Ações similares ao bloco 1
INT  21h      ;
```



```

CALL SCAN_NUM ;
MOV C, CX ; Transfere o valor de CX para a variável C

; //////////////////////////////////////
; // Ordenação crescente dos valores //
; //////////////////////////////////////

; se (A > B) então
; troca os valores de A com B e de B com A

MOV BX, A ; Armazena valor da variável A em BX
MOV CX, B ; Armazena valor da variável B em CX
se1:
    CMP BX, CX ; compara BX e CX
    JLE fim_se1 ; se BX > CX segue após então1
entao1:
    ; e efetua a troca de valores
    XCHG BX, CX ; entre os registradores BX e CX
fim_se1:
    ;
MOV A, BX ; Armazena valor do registrador BX em A
MOV B, CX ; Armazena valor do registrador CX em B

; se (A > C) então
; troca os valores de A com C e de C com A

MOV BX, A ; Armazena valor da variável A em BX
MOV CX, C ; Armazena valor da variável C em CX
se2:
    CMP BX, CX ; compara BX e CX
    JLE fim_se2 ; se BX > CX segue após então2
entao2:
    ; e efetua a troca de valores
    XCHG BX, CX ; entre os registradores BX e CX
fim_se2:
    ;
MOV A, BX ; Armazena valor do registrador BX em A
MOV C, CX ; Armazena valor do registrador CX em C

; se (B > C) então
; troca os valores de B com C e de C com B

MOV BX, B ; Armazena valor da variável B em BX
MOV CX, C ; Armazena valor da variável C em CX
se3:
    CMP BX, CX ; compara BX e CX
    JLE fim_se3 ; se BX > CX segue após então3
entao3:
    ; e efetua a troca de valores
    XCHG BX, CX ; entre os registradores BX e CX
fim_se3:
    ;
MOV B, BX ; Armazena valor do registrador BX em B
MOV C, CX ; Armazena valor do registrador CX em C

; //////////////////////////////////////
; // Saída de dados //
; //////////////////////////////////////

LEA DX, msg4 ; Pega a mensagem da variável msg3
PUSH AX
MOV AH, 09h
INT 21h
POP AX
MOV AX, A ; Transfere para AX da variável A
CALL print_num ; Escreve o conteúdo de AX.

LEA DX, msg5 ; Pega a mensagem da variável msg5
PUSH AX

```

```

MOV AH, 09h
INT 21h
POP AX
MOV AX, B ; Transfere para AX da variável B
CALL print_num ; Escreve o conteúdo de AX.

LEA DX, msg6 ; Pega a mensagem da variável msg6
PUSH AX
MOV AH, 09h
INT 21h
POP AX
MOV AX, C ; Transfere para AX da variável C
CALL print_num ; Escreve o conteúdo de AX.

INT 20h

DEFINE_SCAN_NUM
DEFINE_PRINT_NUM_UN
DEFINE_PRINT_NUM

END

```

CHECA TRIÂNGULO

Elaborar um programa que leia três valores numéricos para os lados de um triângulo, considerando lados como A, B e C. Verificar se os lados fornecidos formam um triângulo, e se for esta condição verdadeira, deve ser indicado o tipo de triângulo formado: isósceles, escaleno ou equilátero. Leve em consideração que triângulo é uma forma geométrica (polígono) composta de três lados, e o valor de cada lado deve ser menor que a soma dos outros dois lados. Assim sendo, é um triângulo quando $A < B + C$, quando $B < A + C$ e quando $C < A + B$, considerando como lados as variáveis A, B e C. Tendo certeza de que os valores informados para os três lados formam um triângulo, deve-se então analisar os valores fornecidos para estabelecer o tipo de triângulo que será formado: isósceles, escaleno ou equilátero. Um triângulo é isósceles quando possui dois lados iguais e um diferente, sendo $A=B$ ou $A=C$ ou $B=C$; é escaleno quando possui todos os lados diferentes, sendo $A < B$ e $B < C$ e é equilátero quando possui todos os lados iguais, sendo $A=B$ e $B=C$.

```

;*****
;*   Programa: TRIANGULO.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA

A   DW ? ; Definição de variáveis para entrada de dados
B   DW ?
C   DW ?

msg1 DB 'Entre o valor do lado [A]: ', '$' ; 0Dh e 0Ah
msg2 DB 0Dh, 0Ah, 'Entre o valor do lado [B]: ', '$' ; são usados
msg3 DB 0Dh, 0Ah, 'Entre o valor do lado [C]: ', '$' ; para pular
                                           ; linhas
msg4 DB 0Dh, 0Ah, 0Dh, 0Ah, 'Medidas nao formam um triangulo.', 0
msg5 DB 0Dh, 0Ah, 0Dh, 0Ah, 'Triangulo isosceles.', 0
msg6 DB 0Dh, 0Ah, 0Dh, 0Ah, 'Triangulo equilatero.', 0
msg7 DB 0Dh, 0Ah, 0Dh, 0Ah, 'Triangulo escaleno.', 0

.CODE

```

```

; //////////////////////////////////
; // Entrada de dados //
; //////////////////////////////////

; Efetua a entrada do 1o. valor

LEA DX, msg1 ; Pega a mensagem da variável msg1
MOV AH, 09h ; Põe 09h na parte baixa de AX
INT 21h ; Imprime msg1 de DX por meio do código 09h
CALL SCAN_NUM ; Efetua a leitura do valor e põe em CX
MOV A, CX ; Transfere o valor de CX para a variável A

; Efetua a entrada do 2o. valor

LEA DX, msg2 ; Pega a mensagem da variável msg2
MOV AH, 09h
INT 21h
CALL SCAN_NUM
MOV B, CX ; Transfere o valor de CX para a variável B

; Efetua a entrada do 3o. valor

LEA DX, msg3 ; Pega a mensagem da variável msg3
MOV AH, 09h
INT 21h
CALL SCAN_NUM
MOV C, CX ; Transfere o valor de CX para a variável C

; //////////////////////////////////
; // Verificação se medidas formam um triângulo //
; // e efetivação da apresentação das mensagens //
; // de saída //
; //////////////////////////////////

se1:
MOV AX, A ;
MOV AX, B ;
MOV BX, C ;
ADD BX, AX ; A < B + C
CMP AX, BX ;
JNL senao1 ;
e1_1:
MOV AX, B ;
MOV AX, A ;
MOV BX, C ;
ADD BX, AX ; B < A + C
CMP AX, BX ;
JNL senao1 ;
e1_2:
MOV AX, C ;
MOV AX, A ;
MOV BX, B ;
ADD BX, AX ; C < A + B
CMP AX, BX ;
JNL senao1 ;
entao1:
se2:
MOV AX, A ;
MOV BX, B ;
CMP AX, BX ; A <> B
JNE senao2 ;
e2_1:
MOV AX, B ;

```

```

    MOV  BX, C ;
    CMP  AX, BX ; B <> C
    JNE  senao2 ;
entao2:
    LEA  SI, msg6 ; Triangulo equilatero.
    CALL PRINT_STRING
    JMP  fim_se2
senao2:
    se3:
        MOV  AX, A ;
        MOV  BX, B ;
        CMP  AX, BX ; A = B
        JE   senao3 ;
    ou3_1:
        MOV  AX, A ;
        MOV  BX, C ;
        CMP  AX, BX ; A = C
        JE   senao3 ;
    ou3_2:
        MOV  AX, B ;
        MOV  BX, C ;
        CMP  AX, BX ; C = B
        JE   senao3 ;
    entao3:
        LEA  SI, msg7 ; Triangulo escaleno.
        CALL PRINT_STRING
        JMP  fim_se3
    senao3:
        LEA  SI, msg5 ; Triangulo isosceles.
        CALL PRINT_STRING
    fim_se3:
    fim_se2:
    JMP  fim_se1
senao1:
    LEA  SI, msg4 ; Medidas nao formam um triangulo.
    CALL PRINT_STRING
fim_se1:

INT 20h

DEFINE_SCAN_NUM
DEFINE_PRINT_STRING

END

```

TABUADA DE UM NÚMERO QUALQUER

Elaborar um programa que leia um valor inteiro qualquer entre 1 e 10 e apresente como resultado sua tabuada desse número. Caso o usuário informe um valor fora da faixa de uso do programa o programa deve apresentar como resultado a mensagem de advertência “Entre valores entre 1 e 10.”.

```

;*****
;*   Programa: TABUADA.ASM   *
;*****

INCLUDE 'emu8086.inc'

org 100h

.DATA

N DW ? ; Definição de variáveis para entrada de dados
I DW 0
R DW 0

msg1 DB 'Entre o valor de tabuada (1-10): ', 24h
msg2 DB ' X ', 0
msg3 DB ' = ', 0
msg4 DB 'Entre valores entre 1 e 10.', 0
espaco DB ' ', 0

.CODE

; //////////////////////////////////
; // Entrada de dados //
; //////////////////////////////////

LEA DX, msg1
MOV AH, 09h
INT 21h
CALL SCAN_NUM
MOV N, CX
PUTC 13d
PUTC 10d

; //////////////////////////////////
; // Processamento e apresentação da tabuada //
; //////////////////////////////////

PUTC 13d
PUTC 10d
se1:
    CMP N, 1d
    JL senao1
e1:
    CMP N, 10d
    JG senao1
entao1:
    MOV I, 1d
    para:
        CMP I, 10d
        JG fim_para
    faca:

        MOV AX, N ;
        MOV BX, I ; Efetua o cálculo da
        MUL BX ; tabuada ==> R = N * I
        MOV R, AX ;

    se2:
        CMP N, 10d ; Coloca um espaço em branco
        JGE fim_se2 ; antes do valor da variável N caso a
    entao2: ; unidade para tabular o valor
        LEA SI, espaco ; com uma dezena seja < 10.

```

```

    CALL PRINT_STRING ; Em C = "%2d" e PASCAL = N:2.
fim_se2:
;

MOV  AX, N
CALL PRINT_NUM      ; Escreve o valor N

LEA  SI, msg2
CALL PRINT_STRING   ; Escreve o simbolo X

se3:
;
    CMP I, 10d      ; Coloca um espaço em branco
    JGE fim_se3     ; antes do valor da variável I caso a
entao3:             ; unidade para tabular o valor
    LEA SI, espaco  ; com uma dezena seja < 10.
    CALL PRINT_STRING ; Em C = "%2d" e PASCAL = N:2.
fim_se3:
;

MOV  AX, I
CALL PRINT_NUM      ; Escreve o valor I

LEA  SI, msg3
CALL PRINT_STRING   ; Escreve o simbolo =

se4:
;
    CMP R, 100d     ; Coloca um espaço em branco
    JGE fim_se4     ; antes do valor da variável R caso o
se5:               ; valor a ser apresentado seja < que
    CMP R, 10d      ; uma centena.
    JGE fim_se5     ; Coloca mais um espaço em branco antes
    LEA SI, espaco  ; do valor da variável R caso o valor.
    CALL PRINT_STRING ; a ser apresentado seja < que uma dezena.
fim_se5:
;
entao4:           ;
    LEA SI, espaco  ;
    CALL PRINT_STRING ; Em C = "%3d" e PASCAL = N:3.
fim_se4:
;

MOV  AX, R
CALL PRINT_NUM      ; Escreve o valor R

PUTC 13d
PUTC 10d
INC I
JMP para

fim_para:
JMP fim_se1
senao1:
MOV AH, 02h
MOV DL, 07h ; toca bip
int 21h
LEA SI, msg4
CALL PRINT_STRING
fim_se1:

INT 20h

DEFINE_SCAN_NUM
DEFINE_PRINT_STRING
DEFINE_PRINT_NUM
DEFINE_PRINT_NUM_UN$
END

```

12.4 - Linguagem Assembly versus Código de Máquina

É muito comum escutar as pessoas dizerem que programar em linguagem *Assembly* é difícil, que quem programa em linguagem *Assembly* é louco, que programação em linguagem *Assembly* é isso ou aquilo. Enfim muito é dito e pouco é falado.

Após o discorrer sobre os exemplos apresentados nesta obra, o próprio leitor poderá responder as essas perguntas com muita facilidade, pois programar em linguagem *Assembly* não é difícil e não tão pouco para loucos. Programar em linguagem *Assembly* é um pouco diferente de programar em linguagens de alto nível. Mas o fato de ser diferente, de aplicar os preceitos de lógica de programação com outra ótica não faz em absoluto desta linguagem ser mais difícil ou mais fácil que programar em linguagens como: C, PASCAL, BASIC, COBOL, entre tantas outras.

Qual o motivo que muitas pessoas confundem e afirmam ser a linguagem *Assembly* mais difícil de aprender? A ideia errônea vem do fato de confundir a linguagem *Assembly* com a linguagem de código de máquina. Perceba que são duas as linguagens de programação em baixo nível: a linguagem em código de máquina e a linguagem *Assembly*. A linguagem de máquina (código de máquina) que foi levemente apresentada nos primeiros capítulos deste livro é baseada em instruções e dados escritos em valores hexadecimais que são armazenados nos endereços de memória que também são definidos em valores hexadecimais. Assim sendo, pode-se até dizer que esta linguagem de comunicação com a máquina seja de difícil aprendizagem. Mas tudo é uma questão de ponto de vista, pois talvez seria mais difícil escrever um programa em código de máquina utilizando-se valores binários. Note que cada valor hexadecimal equivale a quatro valores binários. Escrever um programa diretamente em código de máquina baseado em valores hexadecimais é mais rápido e mais seguro que escrever o mesmo programa com códigos binários, pois a simples troca de um valor 1 (um) com um valor 0 (zero) muda tudo. Assim sendo, se for perguntado para uma pessoa que programa equipamentos computacionais em linguagem de máquina baseada em valores binários, se essa tarefa é difícil com certeza ele dirá que não, pois tudo é uma questão de costume e de um ponto de vista mais leigo. Na medida em que se avança em certo conhecimento, aquilo que parecia ser difícil torna-se fácil. Isto chamasse evolução técnica e intelectual.

Melhor que falar é demonstrar. Tome por base o programa mais simples que se pode escrever em qualquer linguagem de programação, ou seja, o programa: "alô mundo". Assim sendo, observe o código de programa definido a partir do endereço de memória 0100 para as versões escritas nas linguagens: *assembly*, *máquina em hexadecimal* e *máquina em binário*.

org 100h		
MOV AH, 09h	0100: B4 09	0000000100000000: 10110100 00001001
LEA DX, msg	0102: BA 09 01	0000000100000010: 10111010 00001001 00000001
INT 21h	0105: CD 21	0000000100000101: 11001101 00100001
INT 20h	0107: CD 20	0000000100000111: 11001101 00100000
msg DB 'Alo mundo.', 24h	0109: 41 6C 6F 20	0000000100001001: 01000001 01101100 01101111 00100000
	6D 75 6E 64	01101101 01110101 01101110 01100100
	6F 2E 24	01101111 00101110 00100100
RET	0114: C3	0000000100010100: 11000011

Agora tenha em mente o seguinte, tudo que é fácil resolva hoje, o que é difícil deixe para ser resolvido amanhã, pois amanhã será fácil devido a experiência acumulada do hoje. Deixe para depois de amanhã tudo aquilo que hoje seja considerado impossível. Dentro do exposto neste trabalho, meramente introdutório, espero ter auxiliado a você leitor ou educando a introduzir este tema em sua vida profissional e a partir daqui iniciar um estudo mais aprofundado. Ao profissional da educação, espero sinceramente ter lhe proporcionado um material que venha auxiliar uma introdução didática e que também facilite o início de seu trabalho.

Como última lição desta obra carregue o programa **TABUADA.asm** no ambiente de programação **emu8086** e acione o botão **emulate** quando surgir a caixa de diálogo **emulator: Tabuada.asm_** acione o botão **aux** e escolha a opção **listing** que apresentará a tela do programa **Bloco de notas** com o código do programa escrito em linguagem *Assembly* e em código de máquina em hexadecimal.

0D82:0100 B402	MOV	AH,02
0D82:0102 B241	MOV	DL,41
0D82:0104 CD21	INT	21
0D82:0106 CD20	INT	20
0D82:0108 69	DB	69
0D82:0109 64	DB	64
0D82:010A 61	DB	61
0D82:010B 64	DB	64
0D82:010C 65	DB	65
0D82:010D 206573	AND	[DI+73],AH
0D82:0110 7065	JO	0177
0D82:0112 63	DB	63
0D82:0113 69	DB	69
0D82:0114 66	DB	66
0D82:0115 69	DB	69
0D82:0116 63	DB	63
0D82:0117 61	DB	61

Parte

IV

Apêndice


```

0D82:0100 B402      MOV     AH,02
0D82:0102 B241      MOV     DL,41
0D82:0104 CD21      INT     21
0D82:0106 CD20      INT     20
0D82:0108 69        DB      69

```

Apêndice



REFERÊNCIA OPERACIONAL

Este apêndice apresenta a listagem das instruções 8086/8088 em modo *Assembly* e em modo *Opcode* (código em linguagem de máquina), além de outras informações importantes.

O microprocessador 8086/8088 possui um conjunto de instruções que não segue um tamanho homogêneo. Dependendo da instrução em uso ou de seu formato operacional, a quantidade de *bits* usados será diferente, o que leva à necessidade de o programador conhecer a estrutura interna de formação das instruções e de seus operandos. Uma instrução 8086/8088 pode ocupar de 1 até 6 *bytes*, como pode ser constatado na coluna **Byte** da tabela **Instruções Assembly e Opcodes**. A característica de formatação heterogênea da composição de uma instrução 8086/8088 e de seus operandos pode parecer complexa ou de difícil administração, mas em contrapartida, fornece ao programador uma grande possibilidade de escrever códigos de programas mais versáteis. Assim sendo, uma instrução pode ser escrita em uma linha de acordo com a seguinte estrutura:

1°. Byte							
7	6	5	4	3	2	1	0
OPCODE						D	W

2°. Byte							
7	6	5	4	3	2	1	0
MOD		REG				R/M	

3°. Byte							
7	6	5	4	3	2	1	0
DESLOCAMENTO MENOS SIGNIFICATIVO							

4°. Byte							
7	6	5	4	3	2	1	0
DESLOCAMENTO MAIS SIGNIFICATIVO							

5°. Byte							
7	6	5	4	3	2	1	0
DADO MENOS SIGNIFICATIVO							

6°. Byte							
7	6	5	4	3	2	1	0
DADO MAIS SIGNIFICATIVO							

A formatação de uma instrução pode ocorrer de algumas formas. Uma instrução pode ser composta pelos *bytes* 1, 2, 3, 4, 5 e 6, como pode ser composta apenas pelo *byte* 1 ou pelos *bytes* 1 e 2, bem como pode ser composta com os *bytes* 1, 5 e 6, ou qualquer outra forma, desde que sempre esteja definido inicialmente o *byte* 1.

Para facilitar o entendimento do exposto anteriormente, seguem as nomenclaturas utilizadas na representação da estrutura das instruções da linguagem Assembly 8086/8088. As instruções usadas para a programação da linguagem Assembly podem usar como indicado anteriormente de um até seis *bytes* de memória, que podem estar dispostos segundo a seguinte estrutura:

- ◆ Instrução – ocupa de um a dois *bytes*;
- ◆ dado – ocupa de um a dois *bytes*;
- ◆ endereço – ocupa um, dois ou quatro *bytes*.

O *byte* 1 caracteriza-se por ser a forma mais simples de representação de uma instrução. Possui os bits 7, 6, 5, 4, 3 e 2 para representar o OPCODE (código de operação) em si, sucedidos dos bits 1 e 0, que possuem como significado o seguinte:

- ◆ O *bit* 1 do *byte* 1 é representado pela indicação D, a qual define o sentido de direção que uma operação de movimentação de dados terá sobre um registrador. Se D=0, indica que o conteúdo do campo REG (*byte* 2) será interpretado como fonte; se D=1, indica que o conteúdo do campo REG (*byte* 2) será interpretado como destino.
- ◆ O *bit* 0 do *byte* 1 é representado pela indicação W, a qual define o modo como a operação será realizada de acordo com o dado definido, podendo esse dado ser um *word* ou um *byte*. Se W=0, operação realizada como *byte*; se W=1, operação realizada como *word*.

O *byte* 2, quando utilizado, efetua o modo de endereçamento e acesso à memória por meio dos registradores. Possui sua estrutura segmentada em três partes, sendo MOD (bits 7 e 6), REG (bits 5, 4 e 3) e R/M (bits 2, 1 e 0), que possuem o seguinte significado:

- ◆ A parte MOD é usada para indicar o modo de endereçamento e definição dos operandos em relação ao uso de registradores ou memória, ou seja, informa de onde é obtido o operando. Olhe a tabela **Modo (MOD)**.

Modo (MOD)	
Código	Descrição
00	A operação será em modo memória sem deslocamento, exceto quando o campo R/M do <i>byte</i> 2 for igual a 110, pois neste caso o deslocamento será de 16 bits.
01	A operação será em modo memória com deslocamento de 8 bits.
10	A operação será em modo memória com deslocamento de 16 bits.
11	A operação será em modo registrador. Neste caso, consultar a tabela Registrador/Memória (R/M).

- ◆ A parte REG indica o registrador que será usado na operação do *byte* 2, a partir do valor setado no *bit* 0 do *byte* 1. Olhe a tabela **Registrador (REG)**.

Registrador (REG)			
Código	W = 0	W = 1	Segmento
000	AL	AX	ES
001	CL	CX	CS
010	DL	DX	SS
011	BL	BX	DS
100	AH	SP	-
101	CH	BP	-
110	DH	SI	-
111	BH	DI	-

- ◆ A parte R/M é usada para indicar as operações de cálculo de endereços efetivos sobre a memória e sobre o registrador. Olhe a tabela **Registrador/Memória (R/M)**.

Registrador/Memória (R/M)					
MOD = 11			Cálculo do Endereço Efetivo		
R/M	W = 0	W = 1	MOD = 00	MOD = 01	MOD = 10
000	AL	AX	[BX + SI]	[BX + SI] + DESLB	[BX + SI] + DESLW
001	CL	CX	[BX + DI]	[BX + DI] + DESLB	[BX + DI] + DESLW
010	DL	DX	[BP + SI]	[BP + SI] + DESLB	[BP + SI] + DESLW
011	BL	BX	[BP + DI]	[BP + DI] + DESLB	[BP + DI] + DESLW
100	AH	SP	[SI]	[SI] + DESLB	[SI] + DESLW
101	CH	BP	[DI]	[DI] + DESLB	[DI] + DESLW
110	DH	SI	DESLW	[BP] + DESLB	[BP] + DESLW
111	BH	DI	[BX]	[BX] + DESLBb	[BX] + DESLW

- ◆ As indicações DESLB e DESLW se relacionam, respectivamente, à ação de deslocamento de *byte* ou *word* em memória.
- ◆ A estrutura do *byte* 2 é utilizada para a definição do conteúdo **ModR/M**. A tabela **Estrutura Padrão do Byte ModR/M** mostra a estrutura de formação de um byte ModR/M (*byte* 2). Observe que o byte ModR/M é dividido em três grupos operacionais.

Estrutura Padrão do Byte ModR/M								
ModR/M Byte 2	MOD		REG			R/M		
	7	6	5	4	3	2	1	0
	3°.		2°.			1°.		

- ◆ O campo **MOD** (*bits* 7 e 6) definido como terceiro grupo deve ser combinado com o campo **R/M** (*bits* 2, 1 e 0) definido como primeiro grupo no sentido de possibilitar a formação de 32 valores, sendo oito valores para os registradores de 8 *bits* (AL, CL, DL, BL, AH, CH, DH e BH) ou 16 *bits* (AX, CX, DX, BX, SP, BP, SI e DI), mais 24 valores para os modos de endereçamento representados como *byte* **ModR/M**.
- ◆ Os *bits* de 0 até 2 indicam a definição de registro (R) ou memória (M) para o campo **R/M**, os *bits* de 3 até 5 indicam a definição de registro ou de segmento de registro para o campo **Reg** e os *bits* de 6 até 7 indicam para o campo **Mod** como o campo **R/M** deve ser interpretado.
- ◆ A partir das tabelas **Modo (MOD)**, **Registrador (REG)**, **Registrador/Memória (R/M)** e **Estrutura Padrão do Byte ModR/M** fica definida como endereço efetivo a junção do primeiro e do terceiro grupos (MOD + R/M), o que possibilita obter os seguintes endereços:

DS: [BX+SI] 00xxx000
DS: [BX+DI] 00xxx001
SS: [BP+SI] 00xxx010
SS: [BP+DI] 00xxx011
DS: [SI] 00xxx100
DS: [DI] 00xxx101
DS: DESLW 00xxx110
DS: [BX] 00xxx111
DS: [BX+SI+DESLB] 01xxx000
DS: [BX+DI+DESLB] 01xxx001
SS: [BP+SI+DESLB] 01xxx010

SS: [BP+DI+DESLB]	01xxx011
DS: [SI+DESLB]	01xxx100
DS: [DI+DESLB]	01xxx101
SS: [BP+DESLB]	01xxx110
DS: [BX+DESLB]	01xxx111
DS: [BX+SI+DESLW]	10xxx000
DS: [BX+DI+DESLW]	10xxx001
SS: [BP+SI+DESLW]	10xxx010
SS: [BP+DI+DESLW]	10xxx011
DS: [SI+DESLW]	10xxx100
DS: [DI+DESLW]	10xxx101
SS: [BP+DESLW]	10xxx110
DS: [BX+DESLW]	10xxx111
AL AX	11xxx000
CL CX	11xxx001
DL DX	11xxx010
BL BX	11xxx011
AH SP	11xxx100
CH BP	11xxx101
DH SI	11xxx110
BH DI	11xxx111

- ◆ A partir das tabelas **Modo (MOD)**, **Registrador (REG)**, **Registrador/Memória (R/M)** e **Estrutura Padrão do Byte ModR/M** ficam definidos para o segundo grupo (REG) os seguintes registradores:

AL AX	xx000xxx
CL CX	xx001xxx
DL DX	xx010xxx
BL BX	xx011xxx
AH SP	xx100xxx
CH BP	xx101xxx
DH SI	xx110xxx
BH DI	xx111xxx

- ◆ A partir das tabelas **Modo (MOD)**, **Registrador (REG)**, **Registrador/Memória (R/M)** e **Estrutura Padrão do Byte ModR/M** ficam definidos para o segundo grupo (REG) os seguintes registradores de segmento:

ES	xx000xxx
CS	xx001xxx
SS	xx010xxx
DS	xx011xxx

Os *bytes* 3 e 4, quando utilizados, permitem definir os valores de deslocamento em memória. Esses *bytes* são utilizados apenas em uma parte do conjunto de instruções.

Os *bytes* 5 e 6, quando utilizados, permitem definir os valores dos dados a serem operacionalizados com a instrução em uso. Esses *bytes* são também utilizados apenas em uma parte do conjunto de instruções.

A tabela seguinte apresenta a estrutura geral do modo de operação em relação aos registradores e à memória.

Modo de Operação											
#	Mod	Reg/Mem	Endereço Efetivo	Reg/Opcodes em Valores Hexadecimais							
				0	1	2	3	4	5	6	7
				000	001	010	011	100	101	110	111
1	00	000	[BX+SI]	00	08	10	10	20	28	30	38
2	00	001	[BX+DI]	01	09	11	19	21	29	31	39
3	00	010	[BP+SI]	02	0A	12	1A	22	2A	32	3A
4	00	011	[BP+DI]	03	0B	13	1B	23	2B	33	3B
5	00	100	[SI]	04	0C	14	1C	24	2C	34	3C
6	00	101	[DI]	05	0D	15	1D	25	2D	35	3D
7	00	110	DESLW	06	0E	16	1E	26	2E	36	3E
8	00	111	[BX]	07	0F	17	1F	27	2F	37	3F
9	01	000	[BX+SI]+DESLB	40	48	50	58	60	68	70	78
10	01	001	[BX+DI]+DESLB	41	49	51	59	61	69	71	79
11	01	010	[BP+SI]+DESLB	42	4A	52	5A	62	6A	72	7A
12	01	011	[BP+DI]+DESLB	43	4B	53	5B	63	6B	73	7B
13	01	100	[SI]+DESLB	44	4C	54	5C	64	6C	74	7C
14	01	101	[DI]+DESLB	45	4D	55	5D	65	6D	75	7D
15	01	110	[BP]+DESLB	46	4E	56	5E	66	6E	76	7E
16	01	111	[BX]+DESLB	47	4F	57	5F	67	6F	77	7F
17	10	000	[BX+SI]+DESLW	80	88	90	98	A0	A8	B0	B8
18	10	001	[BX+DI]+DESLW	81	89	91	99	A1	A9	B1	B9
19	10	010	[BP+SI]+DESLW	82	8A	92	9A	A2	AA	B2	BA
20	10	011	[BP+DI]+DESLW	83	8B	93	9B	A3	AB	B3	BB
21	10	100	[SI]+DESLW	84	8C	94	9C	A4	AC	B4	BC
22	10	101	[DI]+DESLW	85	8D	95	9D	A5	AD	B5	BD
23	10	110	[BP]+DESLW	86	8E	96	9E	A6	AE	B6	BE
24	10	111	[BX]+DESLW	87	8F	97	9F	A7	AF	B7	BF
25	11	000	AL / AX	C0	C8	D0	D8	E0	E8	F0	F8
26	11	001	CL / CX	C1	C9	D1	D9	E1	E9	F1	F9
27	11	010	DL / DX	C2	CA	D2	DA	E2	EA	F2	FA
28	11	011	BL / BX	C3	CB	D3	DB	E3	EB	F3	FB
29	11	100	AH / SP	C4	CC	D4	DC	E4	EC	F4	FC
30	11	101	CH / BP	C5	CD	D5	DD	E5	ED	F5	FD
31	11	110	DH / SI	C6	CE	D6	DE	E6	EE	F6	FE
32	11	111	BH / DI	C7	CF	D7	DF	E7	EF	F7	FF

No sentido de entender a distribuição da tabela **Modo de Operação**, considere a instrução **DIV BX**, levando-se em consideração que estejam os registradores **AX** com o valor do dividendo e **BX** com o valor do divisor. Como orientado no capítulo três, o *Opcod* para essa operação é **F7F3**, sendo **F7h** o código em linguagem de máquina para a instrução **DIV BX**, como pode ser verificado na tabela **Instruções Assembly e Opcodes (DIV BX = F7 / 6)** e **F3h** é a indicação de que a operação será efetuada com o registrador **BX**, como pode ser comprovado na tabela **Modo de Operação**.

Tome por base o valor **F3h**, convertendo-o em seu equivalente binário (**11110011b**). Em seguida separe os termos **11-110-011** para identificar assim **Mod-Reg/Opcod-Reg/Mod**. Junte os termos **Mod** e **Reg/Mod** e ter-se-á o valor **11-011** (linha 28 da tabela), localize na tabela a coluna referente ao código **Reg/Opcod** com valor **110** (coluna 6) e veja que o valor apresentado em hexadecimal é **F3** referente às operações sobre os registradores **BX** e **BL**.

Neste sentido, a tabela de **Notação Utilizada para Assembly/Opcodes** descreve os símbolos e seus significados que são utilizados nas tabelas **Modo de Operação** e **Instruções Assembly e Opcodes**.

Notação Utilizada para Assembly/Opcodes	
Símbolo	Significado
DRB	Indica deslocamento relativo para a próxima instrução: JMP, CALL, entre outras sinalizadas como segmento de <i>bytes</i> .
DRD	Indica deslocamento relativo para a próxima instrução: JMP, CALL, entre outras sinalizadas como segmento de <i>double words</i> .
DRW	Indica deslocamento relativo para a próxima instrução: JMP, CALL, entre outras sinalizadas como segmento de <i>words</i> .
MB	Operação realizada em endereço de memória com tamanho de um byte de DS:SI ou ES:DI (usado somente em instruções com <i>strings</i>).
MW	Operação realizada em endereço de memória com tamanho de um word de DS:SI ou ES:DI (usado somente em instruções com <i>strings</i>).
PW	Operação realizada com ponteiro de memória distante, indicado pela definição WORD ptr DS:[AX].
PTR16:16	Definição de operação com uso de valor imediato de ponteiro distante de segmento e deslocamento de memória. O formato 16:16 corresponde à definição de segmento:deslocamento descrita na forma NNNN:NNNN, em que NNNN são valores entre 0000h e FFFFh.
RW	Operação realizada em registro com tamanho de um word, podendo ser efetuado nos registradores AX, CX, DX, BX, SP, BP, SI ou DI.
RMB	Definição de operação realizada em endereços de registro ou endereços de memória com tamanho de um byte.
RMW	Definição de operação realizada em endereços de registro ou endereços de memória com tamanho de um word.

Notação Utilizada para Assembly/Opcodes	
Símbolo	Significado
VIB	Definição de valor imediato de tamanho de um byte entre 00h e FFh a ser definido para uma dada operação.
VIW	Definição de valor imediato de tamanho de um word entre 0000h e FFFFh a ser definido para uma dada operação.

A tabela de **Instruções Assembly e Opcode** apresenta a relação alfabética das Instruções Assembly em valores hexadecimais e os dados são apresentados com as simbologias descritas de acordo com as tabelas anteriores. Essa tabela apresenta as instruções suportadas pelo programa **emu8086**. Por esta razão, nela não são descritas algumas instruções 8086/8088, como INT3, LOCK e WAIT.

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
AAA	37	AAD AAM AAS	1
AAD	D5	AAA AAM AAS	2
AAM	D4	AAA AAD AAS	2
AAS	3F	AAA AAD AAM	1
ADC RMB, AL	10	AAD SBB SUB	2-4
ADC RMW, AX	11	AAD SBB SUB	2-4
ADC AL, RMB	12	AAD SBB SUB	2-4
ADC AX, RMW	13	AAD SBB SUB	2-4
ADC AL, VIB	14 VIB	AAD SBB SUB	3-4
ADC AX, VIW	15 VIW	AAD SBB SUB	3-4
ADC RMB, VIB	80 GRP1/2 VIB	AAD SBB SUB	3-6
ADC RMW, VIW	81 GRP1/2 VIW	AAD SBB SUB	3-6
ADC RMW, VIB	83 GRP1/2 VIB	AAD SBB SUB	3-6
ADD RMB, AL	00	ADC SBB SUB	2-4
ADD RMW, AX	01	ADC SBB SUB	2-4
ADD AL, RMB	02	ADC SBB SUB	2-4
ADD AX, RMW	03	ADC SBB SUB	2-4
ADD AL, VIB	04 VIB	ADC SBB SUB	3-4
ADD AX, VIW	05 VIW	ADC SBB SUB	3-4
ADD RMB, VIB	80 GRP1/0 VIB	ADC SBB SUB	3-6
ADD RMW, VIW	81 GRP1/0 VIW	ADC SBB SUB	3-6
ADD RMW, VIB	83 GRP1/0 VIB	ADC SBB SUB	3-6

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
AND RMB, AL	20	TEST OR NOT NEG XOR	2-4
AND RMW, AX	21	TEST OR NOT NEG XOR	2-4
AND AL, RMB	22	TEST OR NOT NEG XOR	2-4
AND AX, RMW	23	TEST OR NOT NEG XOR	2-4
AND AL, VIB	24 VIB	TEST OR NOT NEG XOR	3-4
AND AX, VIW	25 VIW	TEST OR NOT NEG XOR	3-4
AND RMB, VIB	80 GRP1/4 VIB	TEST OR NOT NEG XOR	3-6
AND RMW, VIW	81 GRP1/4 VIW	TEST OR NOT NEG XOR	3-6
AND RMW, VIB	83 GRP1/4 VIB	TEST OR NOT NEG XOR	3-6
CALL PTR16:16 (FAR)	9A	RET	5
CALL NOME (NEAR)	E8	RET	3
CALL RMW (NEAR)	FF GRP5/2	RET	2
CALL PW (FAR)	FF GRP5/3	RET	2-4
CBW	98	CWD	1
CLC	F8	STC CMD	1
CLD	FC	CMPS LODS MOVS SCAS STD STOS	1
CLI	FA	STI	1
CMC	F5	CLC STC	1
CMP RMB, AL	38	SUB CMPS SCAS	2-4
CMP RMW, AX	39	SUB CMPS SCAS	2-4
CMP AL, RMB	3A	SUB CMPS SCAS	2-4
CMP AX, RMW	3B	SUB CMPS SCAS	2-4
CMP AL, VIB	3C VIB	SUB CMPS SCAS	3-4
CMP AX, VIW	3D VIW	SUB CMPS SCAS	3-4
CMP RMB, VIB	80 GRP1/7 VIB	SUB CMPS SCAS	3-6
CMP RMW, VIW	81 GRP1/7 VIW	SUB CMPS SCAS	3-6
CMP RMW, VIB	83 GRP1/7 VIB	SUB CMPS SCAS	3-6
CMPSB	A6	CMP SCAS	1
CMPSW	A7	CMP SCAS	1
CWD	99	CBW	1
DAA	27	DAS	1
DAS	2F	DAA	1

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
DEC AX	48	INC SUB	1-2
DEC CX	49	INC SUB	1-2
DEC DX	4A	INC SUB	1-2
DEC BX	4B	INC SUB	1-2
DEC SP	4C	INC SUB	1-2
DEC BP	4D	INC SUB	1-2
DEC SI	4E	INC SUB	1-2
DEC DI	4F	INC SUB	1-2
DEC RMB	FE GRP4/1	INC SUB	2-4
DEC RMW	FF GRP5/1	INC SUB	2-4
DIV BL	F6 GRP3a/6	MUL	2
DIV BX	F7 GRP3b/6	MUL	2
HLT	F4	STI CLI	1
IDIV BL	F6 GRP3a/7	IMUL	2-4
IDIV BX	F7 GRP3a/7	IMUL	2-4
IMUL RMB	F6 GRP3a/5	IDIV	2
IMUL RMW	F7 GRP3b/5	IDIV	2
IN AL, VIB	E4 VIB	OUT	2
IN AX, VIB	E5 VIB	OUT	2
IN AL, DX	EC	OUT	2
IN AX, DX	ED	OUT	1
INC AX	40	ADD DEC	1
INC CX	41	ADD DEC	1
INC DX	42	ADD DEC	1
INC BX	43	ADD DEC	1
INC SP	44	ADD DEC	1
INC BP	45	ADD DEC	1
INC SI	46	ADD DEC	1
INC DI	47	ADD DEC	1
INC RMB	FE GRP4/0	ADD DEC	2-4
INC RMW	FF GRP5/0	ADD DEC	2-4
INT VIB	CD VIB	INTO	2

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
INTO	CE	INT	1
IRET	CF	INT INTO	1
JA/JNBE NOME	77 DRB	JMP	2
JAE/JNB NOME	73 DRB	JMP	2
JB/JC/JNAE NOME	72 DRB	JMP	2
JBE/JNA NOME	76 DRB	JMP	2
JCXZ NOME	E3 DRB	JMP	2
JE/JZ NOME	74 DRB	JMP	2
JG/JNLE NOME	7F DRB	JMP	2
JGE/JNL NOME	7D DRB	JMP	2
JL/JNGE NOME	7C DRB	JMP	2
JLE/JNG NOME	7E DRB	JMP	2
JMP NOME	E9 DRW	Todas as demais de saltos	2
JMP PTR16:16	EA DRD	Todas as demais de saltos	2-4
JMP NOME	EB DRB	Todas as demais de saltos	2
JMP RMW	FF GRP5/4	Todas as demais de saltos	2-4
JMP PW	FF GRP5/5	Todas as demais de saltos	2-4
JNC NOME	73 DRB	JMP	2
JNE/JNZ NOME	75 DRB	JMP	2
JNO NOME	71 DRB	JMP	2
JNP/JPO NOME	7B DRB	JMP	2
JNS NOME	79 DRB	JMP	2
JO NOME	70 DRB	JMP	2
JP/JPE NOME	7A DRB	JMP	2
JS NOME	78 DRB	JMP	2
LAHF	9F	SAHF	1
LDS AX, PW	C5	Não possui	2-4
LEA AX, RMW	8D	MOV	2-4
LES AX, PW	C4	Não possui	2-4
LODSB	AC	MOVS STOS	1
LODSW	AD	MOVS STOS	1
LOOP NOME	E2 DRB	Não possui	2

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
LOOPE/LOOPZ NOME	E1 DRB	Não possui	2
LOOPNE/LOOPNZ NOME	E0 DRB	Não possui	2
MOV RMB, AL	88	MOVS	2-4
MOV RMW, AX	89	MOVS	2-4
MOV AL, RMB	8A	MOVS	2-4
MOV AX, RMW	8B	MOVS	2-4
MOV RW, ES	8C	MOVS	2
MOV ES, RW	8E	MOVS	2
MOV AL, MB	A0 VIW	MOVS	2-4
MOV AX, MW	A1 VIW	MOVS	2-4
MOV MB, AL	A2 VIW	MOVS	2-4
MOV MW, AX	A3 VIW	MOVS	2-4
MOV AL, VIB	B0 VIB	MOVS	2-3
MOV CL, VIB	B1 VIB	MOVS	2-3
MOV DL, VIB	B2 VIB	MOVS	2-3
MOV BL, VIB	53 VIB	MOVS	2-3
MOV AH, VIB	B4 VIB	MOVS	2-3
MOV CH, VIB	B5 VIB	MOVS	2-3
MOV DH, VIB	B6 VIB	MOVS	2-3
MOV BH, VIB	B7 VIB	MOVS	2-3
MOV AX, VIW	B8 VIW	MOVS	2-3
MOV CX, VIW	B9 VIW	MOVS	2-3
MOV DX, VIW	BA VIW	MOVS	2-3
MOV BX, VIW	BB VIW	MOVS	2-3
MOV SP, VIW	BC VIW	MOVS	2-3
MOV BP, VIW	BD VIW	MOVS	2-3
MOV SI, VIW	BE VIW	MOVS	2-3
MOV DI, VIW	BF VIW	MOVS	2-3
MOV RMB, VIB	C6 /0 VIB	MOVS	3-6
MOV RMW, VIW	C7 /0 VIW	MOVS	3-6
MOVSB	A4	MOV LODS STOS	1
MOVSW	A5	MOV LODS STOS	1

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
MUL AL	F6 GRP3a/4	DIV	2
MUL AX	F7 GRP3B/4	DIV	2
NEG RMB	F6 GRP3a/3	AND NOT OR XOR	2-4
NEG RMW	F7 GRP3b/3	AND NOT OR XOR	2-4
NOP	90	Não possui	1
NOT RMB	F6 GRP3a/2	AND NEG OR XOR	2
NOT RMW	F7 GRP3b/2	AND NEG OR XOR	2
OR RMB, AL	08	AND NEG NOT XOR	2-4
OR RMW, AX	09	AND NEG NOT XOR	2-4
OR AL, RMB	0A	AND NEG NOT XOR	1
OR AX, RMW	0B	AND NEG NOT XOR	1
OR AL, VIB	0C VIB	AND NEG NOT XOR	-
OR AX, VIW	0D VIW	AND NEG NOT XOR	-
OR RMB, VIB	80 GRP1/1 VIB	AND NEG NOT XOR	3-6
OR RMW, VIW	81 GRP1/1 VIW	AND NEG NOT XOR	3-6
OR RMW, VIB	83 GRP1/1 VIB	AND NEG NOT XOR	3-6
OUT VIB, AL	E6 VIB	IN	2
OUT VIB, AX	E7 VIW	IN	2
OUT DX, AL	EE	IN	1
OUT DX, AX	EF	IN	1
POP ES	07	PUSH	1
POP SS	17	PUSH	1
POP DS	1F	PUSH	1
POP AX	58	PUSH	1
POP CX	59	PUSH	1
POP DX	5A	PUSH	1
POP BX	5B	PUSH	1
POP SP	5C	PUSH	1
POP BP	5D	PUSH	1
POP SI	5E	PUSH	1
POP DI	5F	PUSH	1
POP RMW	8F /0 VIW	PUSH	2

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
POPA	61	PUSHA	1
POPF	9D	PUSHF	1
PUSH ES	06	POP	1
PUSH CS	0E	POP	1
PUSH SS	16	POP	1
PUSH DS	1E	POP	1
PUSH AX	50	POP	1
PUSH CX	51	POP	1
PUSH DX	52	POP	1
PUSH BX	53	POP	1
PUSH SP	54	POP	1
PUSH BP	55	POP	1
PUSH SI	56	POP	1
PUSH DI	57	POP	1
PUSH VIW	68 VIW	POP	1
PUSH VIB	6A VIB	POP	1
PUSH RMW	FF GRP5/6	POP	2-4
PUSHA	60	POPA	1
PUSHF	9C	POPF	1
RCL RMB, VIB	C0 GRP2/2 VIB	RCR ROL ROR	3-5
RCL RMW, VIB	C1 GRP2/2 VIB	RCR ROL ROR	3-5
RCL RMB, 1	D0 GRP2/2	RCR ROL ROR	2-4
RCL RMW, 1	D1 GRP2/2	RCR ROL ROR	2-4
RCL RMB, CL	D2 GRP2/2	RCR ROL ROR	2-4
RCL RMW, CL	D3 GRP2/2	RCR ROL ROR	2-4
RCR RMB, VIB	C0 GRP2/3 VIB	RCL ROR ROL	3-3
RCR RMW, VIB	C1 GRP2/3 VIB	RCL ROR ROL	3-5
RCR RMB, 1	D0 GRP2/3	RCL ROR ROL	2-4
RCR RMW, 1	D1 GRP2/3	RCL ROR ROL	2-4
RCR RMB, CL	D2 GRP2/3	RCL ROR ROL	2-4
RCR RMW, CL	D3 GRP2/3	RCL ROR ROL	2-4
REP/REPNE/REPNZ	F2	Não possui	1

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
REPE/REPZ	F3	Não possui	1
RET VIW	C2 VIW	CALL	3
RET	C3	CALL	1
RETF VIW	CA VIW	CALL	3
RETF	DRB	CALL	1
ROL RMB, VIB	C0 GRP2/0 VIB	RCL RCR ROR	3
ROL RMW, VIB	C1 GRP2/0 VIB	RCL RCR ROR	3-5
ROL RMB, 1	D0 GRP2/0	RCL RCR ROR	2-4
ROL RMW, 1	D1 GRP2/0	RCL RCR ROR	2-4
ROL RMB, CL	D2 GRP2/0	RCL RCR ROR	2
ROL RMW, CL	D3 GRP2/0	RCL RCR ROR	2
ROR RMB, VIB	C0 GRP2/1 VIB	RCL RCR ROR	3
ROR RMW, VIB	C1 GRP2/1 VIB	RCL RCR ROR	3-5
ROR RMB, 1	D0 GRP2/1	RCL RCR ROR	2-4
ROR RMW, 1	D1 GRP2/1	RCL RCR ROR	2-4
ROR RMB, CL	D2 GRP2/1	RCL RCR ROR	2
ROR RMW, CL	D3 GRP2/1	RCL RCR ROR	2
SAHF	9E	LAHF	1
SAR RMB, VIB	C0 GRP2/7 VIB	SAL SHL SHR	3-5
SAR RMW, VIB	C1 GRP2/7 VIB	SAL SHL SHR	3-5
SAR RMB, 1	D0 GRP2/7	SAL SHL SHR	2-4
SAR RMW, 1	D1 GRP2/7	SAL SHL SHR	2-4
SAR RMB, CL	D2 GRP2/7	SAL SHL SHR	2-4
SAR RMW, CL	D3 GRP2/7	SAL SHL SHR	2-4
SBB RMB, AL	18	SUB ADD ADC	2-4
SBB RMW, AX	19	SUB ADD ADC	2-4
SBB AL, RMB	1 ^a	SUB ADD ADC	2-4
SBB AX, RMW	1B	SUB ADD ADC	2-4
SBB AL, VIB	1C VIB	SUB ADD ADC	3-4
SBB AX, VIW	1D VIW	SUB ADD ADC	3-4
SBB RMB, VIB	80 GRP1/3 VIB	SUB ADD ADC	3-6
SBB RMW, VIW	81 GRP1/3 VIW	SUB ADD ADC	3-6

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
SBB RMW, VIB	83 GRP1/3 VIB	SUB ADD ADC	3-6
SEG. CS (SEGMENTO)	2E	-	-
SEG. DS (SEGMENTO)	3E	-	-
SEG. ES (SEGMENTO)	26	-	-
SEG. SS (SEGMENTO)	36	-	-
SCASB	AE	CMP	1
SCASW	AF	CMP	1
SHL/SAL RMB, 1	D0 GRP2/4	SAR SHR	2-4
SHL/SAL RMB, CL	D2 GRP2/4	SAR SHR	2-4
SHL/SAL RMB, VIB	C0 GRP2/4 VIB	SAR SHR	2-4
SHL/SAL RMW, VIB	C1 GRP2/4 VIB	SAR SHR	2-4
SHL/SAL RMW, 1	D1 GRP2/4	SAR SHR	2-4
SHL/SAL RMW, CL	D3 GRP2/4	SAR SHR	2-4
SHR RMB, VIB	C0 GRP2/5 VIB	SHL SAL SAR	3
SHR RMW, VIB	C1 GRP2/5 VIB	SHL SAL SAR	3
SHR RMB, 1	D0 GRP2/5	SHL SAL SAR	2-4
SHR RMW, 1	D1 GRP2/5	SHL SAL SAR	2-4
SHR RMB, CL	D2 GRP2/5	SHL SAL SAR	2-4
SHR RMW, CL	D3 GRP2/5	SHL SAL SAR	2-4
STC	F9	CLC CMC	1
STD	FD	CLD LODS MOVS SCAS STOS	1
STI	FB	CLI	1
STOSB	AA	LODS MOVS	1
STOSW	AB	LODS MOVS	1
SUB RMB, AL	28	ADC ADD SBB	2-4
SUB RMW, AX	29	ADC ADD SBB	2-4
SUB AL, RMB	2A	ADC ADD SBB	2-4
SUB AX, RMW	2B	ADC ADD SBB	2-4
SUB AL, VIB	2C VIB	ADC ADD SBB	3-4
SUB AX, VIW	2D VIW	ADC ADD SBB	3-4
SUB RMB, VIB	80 GRP1/5 VIB	ADC ADD SBB	3-6
SUB RMW, VIW	81 GRP1/5 VIW	ADC ADD SBB	3-6

Instruções Assembly e Opcodes			
Instrução	Opcodes e Dados	Relaciona-se com as Instruções	Bytes
SUB RMW, VIB	83 GRP1/5 VIB	ADC ADD SBB	3-6
TEST RMB, AL	84	AND CMP	2-4
TEST RMW, AX	85	AND CMP	2-4
TEST AL, VIB	A8 VIB	AND CMP	3-4
TEST AX, VIW	A9 VIW	AND CMP	3-4
TEST RMB, VIB	F6 GRP3a/0 VIB	AND CMP	3-6
TEST RMW,VIW	F7 GRP3b/0 VIW	AND CMP	3-6
XCHG RMB, AL	86	Não possui	1
XCHG AL, RMB	86	Não possui	2-4
XCHG RMW, AX	87	Não possui	1
XCHG AX, RMW	87	Não possui	2-4
XCHG CX, AX	91	Não possui	2
XCHG DX, AX	92	Não possui	2
XCHG BX, AX	93	Não possui	2
XCHG SP, AX	94	Não possui	2
XCHG BP, AX	95	Não possui	2
XCHG SI, AX	96	Não possui	2
XCHG DI, AX	97	Não possui	2
XLATB	D7	Não possui	1
XOR RMB, AL	30	OR AND NOT NEG	2-4
XOR RMW, AX	31	OR AND NOT NEG	2-4
XOR AL, RMB	32	OR AND NOT NEG	2-4
XOR AX, RMW	33	OR AND NOT NEG	2-4
XOR AL, VIB	34 VIB	OR AND NOT NEG	3-4
XOR AX, VIW	35 VIW	OR AND NOT NEG	3-4
XOR RMW, VIW	81 GRP1/6 VIW	OR AND NOT NEG	3-6
XOR RMB, VIB	80 GRP1/6 VIB	OR AND NOT NEG	3-6
XOR RMW, VIB	83 GRP1/6 VIB	OR AND NOT NEG	3-6

Na coluna que apresenta o relacionamento entre as instruções, as indicações CMPS LODS MOVS SCAS STOS devem ser lidas como CMPSB, CMPSW, LODSB, LODSW, MOVSB, MOVSW, SCASB e SACASW.

Algumas instruções relacionadas possuem o mesmo valor de *opcode*. Isso ocorre com algumas instruções que pertencem ao mesmo grupo operacional. Destacam-se seis grupos operacionais de instruções, a saber: GRP1, GRP2, GRP3a, GRP3b, GRP4 e GRP5.

O grupo GRP1 relaciona-se às instruções ADD, OR, ADC, SBB, AND, SUB, XOR ou CMP representadas pelos *opcodes* 80, 81, 82 e 83. A definição de qual instrução estará associada ao *opcode* indicado depende do valor numérico após a barra, podendo ser 0, 1, 2, 3, 4, 5, 6 ou 7. Desta forma, /0 representa ADD, /1 representa OR, /2 representa ADC, /3 representa SBB, /4 representa AND, /5 representa SUB, /6 representa XOR e /7 representa CMP.

O grupo GRP2 relaciona-se às instruções ROL, ROR, RCL, RCR, SHL/SAL, SHR ou SAR representadas pelos *opcodes* D0, D1, D2 e D3. A definição de qual instrução estará associada ao *opcode* indicado depende do valor numérico após a barra, podendo ser 0, 1, 2, 3, 4, 5 ou 7. Desta forma, /0 representa ROL, /1 representa ROR, /3 representa RCL, /4 representa SHL, /5 representa SHR e /7 representa SAR. A operação /6 é nula para esse grupo.

Os grupos GRP3a e GRP3b relacionam-se às instruções TEST, NOT, NEG, MUL, IMUL, DIV ou IDIV representadas, respectivamente, pelos *opcodes* F6 e F7. A definição de qual instrução estará associada ao *opcode* indicado depende do valor numérico após a barra, podendo ser 0, 2, 3, 4, 5, 6 ou 7. Desta forma, /0 representa TEST, /2 representa NOT, /3 representa NEG, /4 representa MUL, /5 representa IMUL, /6 representa DIV e /7 representa IDIV. A operação /1 é nula para esses grupos.

O grupo GRP4 relaciona-se ao uso das instruções INC e DEC representadas pelo *opcode* FE. A definição de qual instrução estará associada ao *opcode* indicado depende do valor numérico após a barra, podendo ser 0 ou 1. Desta forma, /0 representa INC e /1 representa DEC. As demais operações são nulas para esse grupo.

O grupo GRP5 relaciona-se ao uso das instruções INC, DEC, CALL (1), CALL (2), JMP (1), JMP (2) ou PUSH representadas pelo *opcode* FF. A definição de qual instrução estará associada ao *opcode* indicado depende do valor numérico após a barra, podendo ser 0, 1, 2, 3, 4, 5, ou 6. Desta forma, /0 representa INC, /1 representa DEC, /2 representa CALL como modo 1, /3 representa CALL como modo 2, /4 representa JMP como modo 1, /5 representa JMP como modo 2 e /6 representa PUSH. A operação /7 é nula para esse grupo.

Observe em seguida o mapa de *opcodes* dividido em parte 1, parte 2 e parte de extensão. Para interpretação do mapa, veja primeiramente a linha e depois a coluna.

Mapa de opcodes - Parte 1								
	0	1	2	3	4	5	6	7
0	ADD RMB,AL	ADD RMW,AX	ADD AL,RMB	ADD AX,RMW	ADD AL,VIB	ADD AX,VIW	PUSH ES	POP ES
1	ADC RMB,AL	ADC RMW,AX	ADC AL,RMB	ADC AX,RMW	ADC AL,VIB	ADC AX,VIW	PUSH SS	POP SS
2	AND RMB,AL	AND RMW,AX	AND AL,RMB	AND AX,RMW	AND AL,VIB	AND AX,VIW	SEG. ES	DAA

Mapa de opcodes - Parte 1								
	0	1	2	3	4	5	6	7
3	XOR RMB,AL	XOR RMW,AX	XOR AL,RMB	XOR AX,RMW	XOR AL,VIB	XOR AX,VIW	SEG. SS	AAA
4	INC AX	INC CX	INC DX	INC BX	INC SP	INC BP	INC SI	INC DI
5	PUSH AX	PUSH CX	PUSH DX	PUSH BX	PUSH SP	PUSH BP	PUSH SI	PUSH DI
6	-	-	-	-	-	-	-	-
7	JO NOME	JNP NOME JNO NOME	JB NOME JC NOME JNAE NOME	JAE ROT08 JNB NOME	JE NOME JZ NOME	JNE NOME JNZ NOME	JBE NOME JNA NOME	JA NOME JNBE NOME
8	GRP1	GRP1	GRP1	GRP1	TEST RMB,AL	TEST RMW,AX	XCHG RMB,AL XCHG AL,RMB	XCHG RMW,AX XCHG AX,RMW
9	NOP	XCHG CX AX	XCHG DX AX	XCHG BX AX	XCHG SP AX	XCHG BP AX	XCHG SI AX	XCHG DI AX

Mapa de opcodes - Parte 1								
	0	1	2	3	4	5	6	7
A	MOV AL,MB	MOV AX,MW	MOV MB,AL	MOV MW,AX	MOVSB	MOVSW	CMPSB	CMPSW
B	MOV AL,VIB	MOV CL,VIB	MOV DL,VIB	MOV BL,VIB	MOV AH,VIB	MOV CH,VIB	MOV DH,VIB	MOV BH,VIB
C	-	-	RET VIW	RET	LES AX,PW	LDS AX, PW	MOV RMB,VIB	MOV RMW,VIW
D	GRP2	GRP2	GRP2	GRP2	AAM	AAD	-	XLATB
E	LOOPNZ NOME LOPPNE NOME	LOOPZ NOME LOOPE NOME	LOOP NOME	JCZ NOME	IN AL,VIB	IN AX,VIB	OUT VIB,AL	OUT VIB,AX
F	LOCK	-	REP REPNE REPNZ	REPE REPZ	HLT	CMC	GRP3a	GRP3b

Mapa de opcodes - Parte 2								
	8	9	A	B	C	D	E	F
0	OR RMB,AL	OR RMW,AX	OR AL,RMB	OR AX,RMW	OR AL,VIB	OR AX,VIW	PUSH CS	-

Mapa de opcodes - Parte 2								
1	SBB RMB,AL	SBB RMW,AX	SBB AL,RMB	SBB AX,RMW	SBB AL,VIB	SBB AX,VIW	PUSH DS	POP DS
2	SUB RMB,AL	SUB RMW,AX	SUB AL,RMB	SUB AX,RMW	SUB AL,VIB	SUB AX,VIW	SEG. CS	DAS
3	CMP RMB,AL	CMP RMW,AX	CMP AL,RMB	CMP AX,RMW	CMP AL,VIB	CMP AX,VIW	SEG. DS	AAS
4	DEC AX	DEC CX	DEC DX	DEC BX	DEC SP	DEC BP	DEC SI	DEC DI
5	POP AX	POP CX	POP DX	POP BX	POP SP	POP BP	POP SI	POP DI
6	-	-	-	-	-	-	-	-
7	JS NOME	JNS NOME	JP NOME JPE NOME	JPO NOME	JL NOME JNGE NOME	JGE NOME JNL NOME	JLE NOME JNG NOME	JG NOME JNLE NOME

Mapa de opcodes - Parte 2								
8	MOV RMB,AL	MOV RMW,AX	MOV AL,RMB	MOV AX,RMW	MOV RW,ES	LEA AX,RMW	MOV ES,RW	POP RMW
9	CBW	CWD	CALL PTR16:16	WAIT	PUSHF	POPF	SAHF	LAHF
A	TEST AL,VIB	TEST AX,VIW	STOSB	STOSW	LODSB	LODSW	SCASB	SCASW
B	MOV AX,VIW	MOV CX,VIW	MOV DX,VIW	MOV BX,VIW	MOV SP,VIW	MOV BP,VIW	MOV SI,VIW	MOV DI,VIW
C	-	-	REFF VIW	RETF	INT3	INT VIB	INTO	IRET
D	-	-	-	-	-	-	-	-
E	CALL NOME	JMP NOME	JMP PTR16:16	JMP NOME	IN AL,DX	IN AX,DX	OUT DX,AL	OUT DX,AX

Mapa de opcodes - Parte 2								
F	CLC	STC	CLI	STI	CLD	STD	GRP4	GRP5

Mapa de opcodes - Parte de Extensão								
	0	1	2	3	4	5	6	7
GRP1	ADD	OR	ADC	SBB	AND	SUB	XOR	CMP
GRP2	ROL	ROR	RCL	RCR	SHL/SAL	SHR	-	SAR
GRP3a	TEST	-	NOT	NEG	MUL	IMUL	DIV	IDIV
GRP3b	TEST	-	NOT	NEG	MUL	IMUL	DIV	IDIV
GRP4	INC	DEC	-	-	-	-	-	-
GRP5	INC	DEC	CALL	CALL	JMP	JMP	PUSH	-

Para a criação deste apêndice, foram utilizadas informações dos manuais dos microprocessadores da Intel e da AMD que constam na bibliografia deste trabalho, além da obra *The 8086/8088 Primer* de Sthephen P. Morse.

```

0D82:0100 B402      MOV     AH,02
0D82:0102 B241      MOV     DL,41
0D82:0104 CD21      INT     21
0D82:0106 CD20      INT     20
0D82:0108 69        DB      69

```

Apêndice

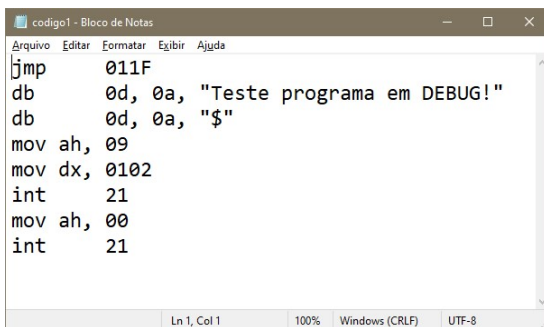
B

ENHANCED DEBUG

Este apêndice abrange detalhes operacionais gerais do utilitário de depuração, montagem e desmontagem *Enhanced DEBUG* usado em algumas partes desta obra.

O programa Microsoft DEBUG e Enhanced DEBUG caracteriza-se por ser um utilitário de depuração (permite verificar o código de programa em baixo nível), montagem (permite usar o utilitário para escrever um programa em baixo nível) e desmontagem (permite olhar internamente o código de um programa em baixo nível). É possível com esta ferramenta fazer alterações e ajustes em um código de programa compilado em baixo nível, tanto **".COM"**, como **".EXE"**.

Para a demonstração dos comandos existentes no utilitário *Enhanced DEBUG* seguem alguns arquivos a serem criados como material de apoio a algumas operações. Observe, respectivamente, os arquivos em formato texto indicados nas figuras A.1, A.2 e A.3 contendo os códigos básicos de dois programas e um texto descritivo definidos como suporte a algumas operações exemplificadas junto ao utilitário DEBUG. Lembre-se de deixar esses arquivos no mesmo local onde o programa DEBUG se encontra.

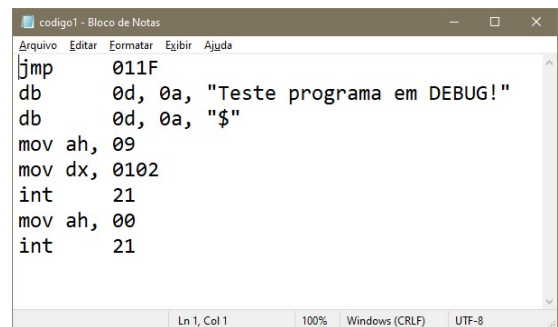


```

codigo1 - Bloco de Notas
Arquivo  Editar  Formatar  Exibir  Ajuda
jmp      011F
db       0d, 0a, "Teste programa em DEBUG!"
db       0d, 0a, "$"
mov ah, 09
mov dx, 0102
int      21
mov ah, 00
int      21
Ln 1, Col 1      100%  Windows (CRLF)  UTF-8

```

Figura A.1 - Código Assembly com uso de dados em formato byte.

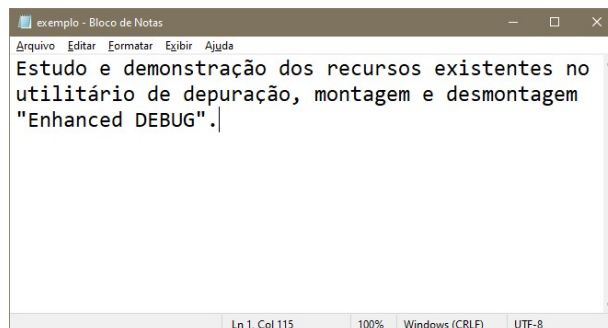


```

codigo1 - Bloco de Notas
Arquivo  Editar  Formatar  Exibir  Ajuda
jmp      011F
db       0d, 0a, "Teste programa em DEBUG!"
db       0d, 0a, "$"
mov ah, 09
mov dx, 0102
int      21
mov ah, 00
int      21
Ln 1, Col 1      100%  Windows (CRLF)  UTF-8

```

Figura A.2 - Código Assembly com uso de dados em formato word.



```

exemplo - Bloco de Notas
Arquivo  Editar  Formatar  Exibir  Ajuda
Estudo e demonstração dos recursos existentes no
utilitário de depuração, montagem e desmontagem
"Enhanced DEBUG".
Ln 1, Col 115    100%  Windows (CRLF)  UTF-8

```

Figura A.3 - Texto com mensagem simples "exemplo.txt".

O programa *Enhanced DEBUG* pode ser executado a partir do *prompt* do sistema operacional com algumas definições iniciais:


```
debug [[unidade:] [caminho] nome do arquivo [parâmetros do arquivo]]
```

onde os parâmetros indicados como,

```
[unidade:] [caminho] nome do arquivo
```

são a localização do nome do arquivo executável que se deseja verificar a partir da unidade e caminho opcionalmente indicados, tendo possivelmente a indicação

```
[parâmetros do arquivo]
```

como sendo qualquer eventual informação de linha de comando exigida pelo arquivo executável a ser verificado.

Como comentado, dependendo de como se faz a chamada do programa tem-se uma diferente ocupação de memória. Veja a seguir duas situações comportamentais do programa *Enhanced DEBUG* carregado com e sem o uso de parâmetro a partir de seu modo padrão de operação.

Veja os detalhes a partir da figura A.4 para o uso da execução dos programas de depuração sem o uso de parâmetro, a partir da orientação indicada por Sedory (2017):

- ◆ Aloca os 64 KB do primeiro segmento de memória que esteja livre para uso, neste caso, representado pelo endereço de segmento 06AF;
- ◆ Os registradores de segmentos DS, ES, SS e CS são definidos com o valor do endereço do segmento de memória alocado com os 64 KB. Desta forma. DS=ES=SS=CS="posição do segmento", ou seja, 06AF;
- ◆ O ponteiro de instrução (IP) fica definido inicialmente, sempre por padrão, na posição de deslocamento 0100;
- ◆ O ponteiro de pilha SP fica definido como FFFE;
- ◆ Os registradores AX, BX, CX, DX, BP, SI e DI são sinalizados como zero;
- ◆ Os bits dos registradores de flags OF, DF, SF, ZF, AF, PE e CF são zerados respectivamente com os valores NV, UP, PL, NZ, NA, PO e NC, exceção ao flag de interrupção IF configurado com valor 1 a partir do valor EI.

```
D:\>debug
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0100 NV UP EI PL NZ NA PO NC
06AF:0100 C3                      RET
```

Figura A.4 - Programa "Enhanced DEBUG" carregado sem o uso de parâmetro.

Veja os detalhes a partir da figura A.5 para o uso da execução dos programas de depuração com o uso de parâmetro a partir de um programa ".COM", a partir da orientação indicada por Sedory (2017):

- ◆ Aloca pelo menos 64 KB do primeiro segmento de memória que esteja livre para depurar programas ou examinar arquivos especificados na linha de comando, neste caso, representado pelo endereço de segmento 06B4;
- ◆ Os registradores de segmentos DS, ES, SS e CS são definidos com o valor do endereço do segmento de memória alocado com os 64 KB. Desta forma. DS=ES=SS=CS="posição do segmento", ou seja, 06B4. Para arquivos maiores que 64KB, será necessário definir diferentes valores de segmento para acessar todos os bytes carregados na memória além dos primeiros 64 KB;
- ◆ O ponteiro de instrução (IP) fica definido inicialmente, sempre por padrão, na posição de deslocamento 0100;
- ◆ O ponteiro de pilha SP fica definido como FFFE;
- ◆ Os registradores BX, DX, BP, SI e DI são sinalizados como zero;
- ◆ Os registradores AX, CX ficam com valores diferentes de zero. O valor de CX (e por vezes em BX também) representa o tamanho em hexadecimal do arquivo carregado na memória. Para arquivos com menos de 64 KB (256 bytes), apenas CX é usado. Por exemplo: Se for carregado um arquivo de 360.247 bytes, então BX será 0005 e CX será 7F37 (57F37h = 360.247d). Se for carregado um arquivo com exatamente 65.536 bytes esses registradores serão BX com 0001 e CX com 0000. No entanto, devido ao deslocamento automático de carga no

endereço 0100h os últimos 256 bytes do arquivo serão carregados no início do próximo segmento de 64 KB. O valor de AX caracteriza por ser o valor inicial do programa passado como parâmetro carregado e apontado no endereço 0100h, que não vem ao caso.

- ◆ Os bits dos registradores de flags OF, DF, SF, ZF, AF, PE e CF são zerados respectivamente com os valores NV, UP, PL, NZ, NA, PO e NC, exceção ao flag de interrupção IF configurado com valor 1 a partir do valor EI.

```
D:\>debug teste.com
-r
AX=FFFF BX=0000 CX=0061 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06B4 ES=06B4 SS=06B4 CS=06B4 IP=0100 NV UP EI PL NZ NA PO NC
06B4:0100 B80300          MOV     AX,0003
-

```

Figura A.5 - Programa "Enhanced DEBUG" carregado com o uso de parâmetro.

A obtenção da relação de comandos do programa é obtida a qualquer momento a partir do uso do comando "?" (sinal de interrogação) no *prompt* de comando do programa. As figuras A.7 e A.8 apresentam a relação dos comandos existentes no utilitário *Enhanced DEBUG* (MASLOCH, 2021 & SEDORY, 2017).

```
assemble      A [address]
compare       C range address
dump          D[B|W|D] [range]
dump MCB chain DM
enter         E address [list]
fill          F range list
go            G [=address] [breakpoints]
hex add/sub   H value1 value2
input         I[W|D] port
load file     L [address]
load sectors  L address drive sector count
move          M range address
name          N [[drive:][path]filename [arglist]]
output        O[W|D] port value
proceed       P [=address] [count]
proceed return PR
quit          Q
register       R [register [value]]
FPU register  RN
toggle 386 regs RX
search        S range list
trace         T [=address] [count]
trace mode    TM [0|1]
unassemble   U [range]
[more]

```

Figura A.7 - Relação de comandos do utilitário Enhanced DEBUG (1/2).

```
dump MCB chain DM
enter         E address [list]
fill          F range list
go            G [=address] [breakpoints]
hex add/sub   H value1 value2
input         I[W|D] port
load file     L [address]
load sectors  L address drive sector count
move          M range address
name          N [[drive:][path]filename [arglist]]
output        O[W|D] port value
proceed       P [=address] [count]
proceed return PR
quit          Q
register       R [register [value]]
FPU register  RN
toggle 386 regs RX
search        S range list
trace         T [=address] [count]
trace mode    TM [0|1]
unassemble   U [range]
view flip     V
write file    W [address]
write sectors W address drive sector count
-

```

Figura A.8 - Relação de comandos do utilitário Enhanced DEBUG (2/2).

Observe que alguns parâmetros são definidos entre colchetes "[" e "]", o que indica que são parâmetros opcionais. Veja a seguir o significado de cada parâmetro dentro do contexto de operação do programa.

PARÂMETRO	SIGNIFICADO
address (endereço)	Endereço de memória expresso com valor numérico hexadecimal.
arglist (lista de argumentos)	Parâmetros a serem informados a um programa carregado com o DEBUG.
breakpoints (pontos de interrupção)	Definição de um ponto de parada para a efetivação de testes e acompanhamentos de programas.
count (número)	Definição do número de setores consecutivos cujo conteúdo será carregado.
drive (unidade)	Indicação da unidade de disco a ser utilizada.
list (lista)	Sequência de bytes hexadecimais separados por espaço ou dados no formato ASCII incluídos em cadeias definidas com aspas simples ou inglesas.
path (caminho)	Indicação da pasta ou diretório de acesso em disco.
port (porta)	Definição de porta lógica a ser utilizada em alguma operação.
range (faixa)	Definição de um segmento de memória a partir de dois valores separados por espaço ou, em alguns casos, a partir de um valor inicial.
register (registrador)	Indicação de um registrador a ser utilizado.
sector (setor)	Indicação em hexadecimal de um setor de disco a ser utilizado para a operação.
value (número)	Definição para uso de um valor numérico no formato hexadecimal.

Além da ajuda interno, obtido por meio do comando "?", é possível obter informações adicionais de auxílio do programa no *prompt* do sistema antes de fazer seu carregamento em memória. Para tanto, use após o de *prompt* "C:>" a instrução "debug /?" e observe o resultado apresentado, como indicado na figura A.9.

```

C:\>debug /?
DEBUG version 1.32b

DEBUG [/F] [[drive:][path]filename [arglist]]

    /F      enable page flipping
    filename file to debug or examine
    arglist  parameters given to program

For a list of debugging commands, run DEBUG and type ? at the prompt.

C:\>_

```

Figura A.9 - Apresentação de informações adicionais do utilitário Enhanced DEBUG.

O uso do parâmetro "/F" coloca o programa em execução na página 1 do vídeo, sem este parâmetro o programa entra na página 0. O uso do argumento "filename" caracteriza a chamada do DEBUG com a indicação de um arquivo a ser verificado ou modificado e o parâmetro "arglist" caracteriza-se por ser a indicação de eventuais parâmetros que o programa carregado em "filename" venha a fazer uso. Desta forma, são possíveis de uso algumas combinações, como:

```

C:> debug /?
C:> debug /f
C:> debug programa
C:> debug programa /arg
C:> debug /f programa
C:> debug /f programa /arg
C:> debug c:\teste\programa
C:> debug c:\teste\programa /arg
C:> debug /f c:\teste\programa
C:> debug /f c:\teste\programa /arg

```

A seguir são comentados e apresentados em ordem alfabética os comandos operacionalizados. Atenta para as indicações de textos em **negrito** e normal. Em texto **negrito** estará a indicação das entradas a serem realizadas por você e em texto normal estará o que o ambiente mostra para você.

A [endereço] - assemble (montar)

Este comando é usado para definir instruções de códigos de um programa na memória, devendo-se iniciar sua ação, sempre a partir do endereço **0100** do segmento **CS** que estiver ativo, neste caso, **06AF**. Neste modo de operação devem ser indicadas instruções em linguagem Assembly compatível com microprocessadores Intel/AMD dentro dos limites de 16/32 bits. O comando "**A**" mantém ativo na memória o último endereço usado para a definição de alguma operação. Isto significa que este comando ao ser usado sucessivamente faz a colocação do ponteiro **IP** no próximo endereço de acesso após o endereço anteriormente utilizado. Neste sentido, este comando é semelhante ao comando "**D**" (Dump) que também "lembra" a localização de seu último uso. A montagem de um programa na memória é encerrada quando a tecla "<Enter>" é acionada em uma linha vazia.

Observação

Para a demonstração dos exemplos a seguir escreva diretamente cada código na posição indicada ou abra respectivamente os arquivos "**codigo1.txt**" e "**codigo2.txt**", selecione o texto com as teclas de atalho "<Ctrl>+<a>", depois execute o atalho "<Ctrl>+<c>" e dentro do programa DEBUG a partir da posição **0100** execute as teclas de atalho "<Ctrl>+<v>".

Exemplo

Observe o resultado da operação na figura A.10 utilizando-se estrutura de 16 bits de dados.

```

-a 0100
06AF:0100 jmp     011F
06AF:0102 db      0d, 0a, "Teste programa em DEBUG!"
06AF:011C db      0d, 0a, "$"
06AF:011F mov ah, 09
06AF:0121 mov dx, 0102
06AF:0124 int     21
06AF:0126 mov ah, 00
06AF:0128 int     21
06AF:012A
-g =0100

Teste programa em DEBUG!

Program terminated (0000)
~

```

Figura A.10 - Exemplo de codificação de programa em Assembly em modo 16 bits.

Veja o resultado da operação de uso do comando "**A**" na figura A.11 utilizando-se estrutura de 32 bits de dados. Para maiores referências consulte o comando "**G**".

```

-rx
386 regs on
-a 0100
06AF:0100 jmp     0123
06AF:0102 dw     0d, 0a, "Teste programa em DEBUG!"
06AF:011E dw     0d, 0a, "$"
06AF:0123 mov     ah, 09
06AF:0125 mov     dx, 0102
06AF:0128 int     21
06AF:012A mov     ah, 00
06AF:012C int     21
06AF:012E
-g =0100

Teste programa em DEBUG!

Program terminated (0000)
-

```

Figura A.11 - Exemplo de codificação de programa em Assembly em modo 32 bits.

C faixa de endereço - compare (comparar)

Este comando é usado para estabelecer a comparação dos bytes de dois blocos de memória. Sendo os conteúdos dos blocos iguais o comando mostra, na sequência, apenas seu *prompt*, mas se houver alguma diferença o programa apresenta os bytes que se diferem lado a lado com a indicação do local de memória onde encontra-se a diferença a partir do formato **endereço1 byte1 byte2 endereço2**. O parâmetro **faixa de endereço** (obrigatório) do comando "C" pode ser definido a partir de duas formas de uso: com a especificação dos valores de endereços inicial e final com **valor1**, **valor2** e **valor3**, em que **valor1** e **valor2** marcam o início e fim do endereço de origem e **valor3** marca o início do endereço de destino a serem comparados utilizando-se implicitamente a mesma distância definida entre **valor1** e **valor2**; ou com a especificação de um endereço inicial e seu comprimento de ação utilizando o argumento "L" que poder ser escrito tanto com caractere maiúsculo quanto caractere minúsculo.

Exemplos

Observe o resultado da operação na figura A.12.

```

-c 200 280 300
-
-c 100 120 200
06AF:0100 C3 00 06AF:0200
-
-c 100 L10 200
06AF:0100 C3 00 06AF:0200
-

```

Figura A.12 - Exemplo de comparação de áreas de memória.

D [B|W|D] [faixa] [comprimento] - dump (despejar)

Este comando mostra (despeja) o conteúdo existente em determinada área de memória indicada no parâmetro **faixa**. O comando "D" é utilizado com todos os parâmetros opcionais, onde **faixa** permite definir a área de memória a ser apresentada podendo fazer uso dos indicativos **valor1** para determinar o endereço inicial e **valor2** para determinar opcionalmente o endereço final, **comprimento** estabelece o tamanho a ser apresentado a partir do endereço inicial indicado como **valor1**.

Observação

Para a demonstração dos exemplos a seguir execute na linha de comando do sistema operacional a chamada da instrução "debug /f exemplo.txt".

Exemplo

C:> debug /f exemplo.txt

Veja o resultado da operação na figura A.13. Perceba que os caracteres acentuados em português são omitidos por não serem caracteres ASCII puros.

```
-d
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130 6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140 65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura...o, m
06AF:0150 6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160 74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170 44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-
```

Figura A.13 - Exemplo de despejo de memória.

Observe outras formas de realização do despejo de dados da memória indicada na figura A.14.

```
-d 0100 0120
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 - r
-d 0100 L 30
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
-
```

Figura A.14 - Outros exemplos de despejo de memória.

O comando "D" pode ser usado com as variantes "B" como comando "DB", "W" como comando "DW" e "D" como comando "DD" no sentido de apresentar o conteúdo de memória disposto na forma de *byte* "B", *word* "W" e *double word* "D" e alterar o modo de operação do DEBUG utilizando-se ou não os detalhes já comentados. Veja na figura A.15 exemplos dos resultados apresentados com "DB", "DW" e "DD".

```
-db 0100 0120
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 - r
-dw 0100 0120
06AF:0100 7345 7574 6F64 6520 6420 6D65 6E6F 7473
06AF:0110 6172 A7C3 A3C3 206F 6F64 2073 6572 7563
06AF:0120 7372
-dd 0100 0120
06AF:0100 75747345 65206F64 6D656420 74736E6F
06AF:0110 A7C36172 206FA3C3 20736F64 75636572
06AF:0120 736F7372
-db ss:sp
06AF:FFFE 00 00 - ..
-dw ss:sp
06AF:FFFE 0000
-dd ss:sp
06AF:FFFE
-
```

Figura A.15 - Exemplos de despejo de memória com "DB", "DW" e "DD".

Quando um dos comandos "DB", "DW" e "DD" é usado ocorre a mudança do modo de ação do ambiente DEBUG, ou seja, a partir do uso de um desses comandos o ambiente fica configurado ao modo solicitado até que outro comando dessa categoria seja executado. Para manter o desempenho em modo padrão garanta a execução do comando "DB".

DM – dump MCB chain (despejar cadeia MCB)

Este comando descarta a apresentação de uma cadeia MCB (*Memory Conflict Buffer* - Buffer de Conflito de Memória). O comando "DM" apresenta a lista PSP (*Process Segment Prefix* - Prefixo do Segmento de Processo) a partir do segmento de memória que esteja ativo até encontrar um MCB que não tenha uma letra de assinatura "M", "Z" ou o endereço MCB que extrapole o limite de 1 MB.

Exemplo

Observe o resultado da operação na figura A.16.

```
- dm
PSP: 06AF
016F 4D 0008 0001
0171 4D 0008 0019
018B 4D 0191 0004 DEBUG
0190 4D 0191 051D DEBUG
06AE 5A 06AF 9950
9FFF 4D 0008 2800 SC
C800 5A 0000 17FF
-
```

Figura A.16 - Exemplo de despejo MCB.

As colunas são as seguintes, a partir do indicativo da terceira linha após **PSP:06AF** considere o descrito a seguir para todas as linhas apresentadas.

- ◆ 1a coluna - **018B** - endereço do segmento do MCB.
- ◆ 2a coluna - **4D** - assinatura do MCB: "4D" para vincular "M" ao MCB e "5A" para vincular "Z" de outra forma.
- ◆ 3a coluna - **0191** - proprietário do MCB: valores abaixo de "50" são valores especiais do sistema, sendo "0" para um MCB não usado e "8" é o SC/SD/S proprietário do sistema MCB em uso; valores mais altos representam geralmente segmentos de processo (bloco de memória precedido por um MCB, que pertence ao bloco em si).
- ◆ 4a coluna - **0004** - quantidade de parágrafos do MCB: "0" indica bloco de memória vazio.
- ◆ 5a coluna - **DEBUG** - nome do proprietário do MCB, podendo ser grafado sem nome quando estão livres ou nomeados quando ocupados: MCB do sistema têm um nome de identificação com até duas letras, quando com mais letras o nome do MCB é obtido a partir dele mesmo tendo então até 8 letras.

E endereço [lista] - enter (entrada)

Este comando é usado para fazer a entrada de dados ou instruções em Assembly diretamente em endereços de memória indicados ou examinar o conteúdo de certa posição. O comando **"E"** opera com dois parâmetros, sendo o primeiro chamado **endereço** obrigatório e opcionalmente com um segundo chamado **lista**. Se o primeiro parâmetro for usado sem o segundo o comando entra em modo examinar/alterar e se ambos forem usados o comando opera diretamente no modo entrada de dados.

Exemplo

C:> **debug /f exemplo.txt**

Veja o resultado da operação na figura A.17 a partir da execução do comando **"d 0100 0105"** no *prompt* DEBUG. Atente para os valores apresentados **45** para a letra **"E"**, **73** para a letra **"s"**, **74** para a letra **"t"**, **75** para a letra **"u"**, **64** para a letra **"d"** e **6F** para a letra **"o"**.

```
-d 0100 0105
06AF:0100 45 73 74 75 64 6F - Estudo
-
```

Figura A.17 - Área de despejo com palavra "Estudo" indicada.

A partir da palavra **"Exemplo"** armazenada entre a faixa de endereços de **0100** até **0105** é possível com o comando **"E"** fazer a alteração deste conteúdo, por exemplo, para **"Ensaio"**. Para tanto, entre no *prompt* DEBUG a instrução **"e 0101 6e,73,61,69"** respectivamente para as letras **"n"**, **"s"**, **"a"**, **"i"** e **"o"** (ou informe: **e 0101 "nsai"**) e em seguida execute a instrução **"d 0100 0105"** para ver o resultado indicado junto a figura A.18.

```
-d 0100 0105
06AF:0100 45 73 74 75 64 6F - Estudo
-e 0101 6e,73,61,69
-d 0100 0105
06AF:0100 45 6E 73 61 69 6F - Ensaio
-
```

Figura A.18 - Resultado após o uso do comando "E".

Para examinar o conteúdo de memória com a possibilidade de realizar alteração use a instrução "**e 0101**". Neste caso é apresentado o valor da posição seguindo de um ponto. Neste momento, pode-se informar o novo valor para a posição e acionar a tecla "<Enter>" uma vez para registrar a alteração ou usar a tecla de "<Espaço>" para colocar o modo do comando no próximo endereço da memória para alterar ou não a "nova" posição. Caso efetue alguma alteração de forma incorreta use a tecla "<menos>" para voltar a uma posição e realizar sua alteração. Após realizar as ações desejadas basta acionar um último "<Enter>" para finalizar a operação.

Para alterar a palavra "**Ensaio**", tornando-a "**Estudo**" execute no *prompt* DEBUG a instrução "**e 0101**" e entre os detalhes indicados em negrito. As referências "<Espaço>" e "<Enter>" indicam que essas teclas devem ser acionadas durante a edição:

```
-e 0101
06FA:0101  6E.73 "<Espaço>" 73.74 "<Espaço>" 61.75 "<Espaço>" 69.64 "<Enter> 2x".
```

Na sequência execute, novamente, a instrução "**d 0100 0105**" e observe o resultado indicado junto a figura A.19.

```
-e 0101
06AF:0101  6E.73  73.74  61.75  69.64  6F.
-d 0100 0105
06AF:0100  45 73 74 75 64 6F      -      Estudo
-
```

Figura A.19 - Resultado após o uso do comando "E" a partir de posição definida.

F faixa lista - fill (preencher)

Este comando é usado para realizar o preenchimento de determinada área de memória com certo conjunto de bytes. Desta forma, uma das suas finalidades é realizar a limpeza de certa área de memória. O comando "F" usa dois parâmetros obrigatórios sendo o primeiro para a indicação do endereço de memória a ser tratado e o comprimento do preenchimento a ser executado (**faixa**) e o segundo (**lista**) a ser usado no preenchimento da área de memória.

Exemplo

```
C:> debug /f exemplo.txt
```

Inicialmente execute o comando "**d 0100**" para ver o conteúdo de parte do texto carregado e na sequência e efetue o uso das instruções "**f 0100 010F 'DEBUG '**" e "**d 0100**". Veja na figura A.20 a ocorrência no uso do comando "F".

```
-d 0100
06AF:0100  45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110  72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120  72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130  6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140  65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura....o, m
06AF:0150  6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160  74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170  44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-f 0100 010F 'DEBUG '
-d0100
06AF:0100  44 45 42 55 47 20 44 45-42 55 47 20 44 45 42 55 DEBUG DEBUG DEBU
06AF:0110  72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120  72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130  6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140  65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura....o, m
06AF:0150  6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160  74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170  44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-
```

Figura A.20 - Resultado do preenchimento de dados com o comando "F".

Para realizar a limpeza dos dados de uma área de memória basta executar o uso do comando "F" o preenchimento desta área com o valor numérico "00" a partir da instrução "**f 0100 FFFF 00**". No entanto, o uso do comando "F" requer alguns cuidados pois se definido sobre uma área de memória dados que possua informações do programa DEBUG ou informações de uso do próprio sistema poderá inviabilizar sua ação ou gerar efeitos inesperados. Mantenha atenção e foco sobre isso.

G [[=endereço] [pontos de interrupção]] - go (ir)

Este comando é usado para executar um programa que esteja definido em memória ou usado para definir pontos de interrupção no código de um programa. O comando "G" usa dois parâmetros opcionais, sendo o primeiro a indicação do endereço onde o programa se inicia na memória e o segundo a definição do ponto de interrupção (**HALT**) até onde o programa se estende na memória. O segundo parâmetro pode ser omitido quando o programa possui internamente em seu código alguma instrução de encerramento ou interrupção. É possível definir para o segundo parâmetro até dez pontos de interrupção a partir do uso do comando "G" separando os endereços de interrupção com espaços em branco. Quando um ponto de interrupção é definido ocorre a interrupção da execução do programa no endereço imediatamente antes ao ponto definido para sua verificação.

Exemplo

Para realizar um teste de execução no uso do comando "G" é necessário ter em memória o código de um programa definido para execução. Veja na figura A.21 um código de programa definido a partir do comando "A" e sua execução a partir do uso da instrução "g =0100".

```
-a 0100
06AF:0100 jmp      011F
06AF:0102 db       0d, 0a, "Teste programa em DEBUG!"
06AF:011C db       0d, 0a, "$"
06AF:011F mov  ah, 09
06AF:0121 mov  dx, 0102
06AF:0124 int   21
06AF:0126 mov  ah, 00
06AF:0128 int   21
06AF:012A
-g =0100

Teste programa em DEBUG!

Program terminated (0000)
-
```

Figura A.21 - Exemplo de execução de programa com comando "G".

O comando "G" pode ser usado sem parâmetros e neste caso apenas executa o código que estiver indicado no registrador "CS" a partir da posição indicada no registrador "IP", se usado com a indicação do endereço, por exemplo, como em "0100" inicia o programa nesta posição a partir do segmento "CS" e se usado, por exemplo, com o formato "g 0100 012A" inicia o programa na posição 0100 e encerra a execução do programa antes do ponto de interrupção estabelecido no endereço 012A, ou seja, encerra o programa no endereço 0128.

Quando forem definidos mais de um ponto de interrupção a partir do primeiro ponto de interrupção definido (como em 012A) é possível fazer usos sucessivos do comando "G" para avançar até o próximo ponto de interrupção. No entanto, esses pontos de interrupção somente podem ser definidos nos endereços de memória que tenham códigos válidos para os *opcodes* Assembly sob risco das ações de parada definidas deixarem de surtir efeito na operação.

H valor1 valor2 - hex add/sub (adição/subtração hexadecimal)

Este comando é usado para realizar simultaneamente as operações de adição e subtração de dois valores numéricos hexadecimais. O comando "H" usa dois parâmetros obrigatórios representados por **valor1** e **valor2**. Esses parâmetros podem ser operacionalizados com valores de até quatro dígitos. O resultado da diferença ocorre a partir do **valor2** sobre o **valor1** e utiliza o efeito de *complemento de dois* para representar valores que estejam acima do limite máximo permitido.

Exemplo

Observe na figura A.22 alguns exemplos de operações utilizando-se valores numéricos.

```

-h 2 1
0003 0001
-h 9 1
000A 0008
-h 1 9
000A FFF8
-h 9876 1234
AAAA 8642
-

```

Figura A.22 - Resultados de adições e subtração com o comando "H".

I [W|D] porta - input (porta de entrada)

Este comando é usado para realizar entrada, ou seja a leitura, de uma porta x86 e mostrar seu conteúdo. O comando "I" usa um parâmetro obrigatório que deve ser indicado a partir de uma porta de 16 bits especificada entre os valores **0000** até **FFFF**. Os comandos **"IW"** e **"ID"** fazem a inserção de valores no formato **word** ou **dword** (*double-word*). Este comando permite ler as portas de entrada e aos códigos escritos nos endereços das portas de saída.

L [endereço] [[unidade] [setor] [número]] - load file (carrega arquivo)

Este comando é usado para realizar o carregamento de um arquivo especificado como argumento do programa DEBUG ou usado após o comando **"N"**. É possível com o comando **"L"** carregar os setores especificados de um disco na memória sem levar em consideração o sistema de arquivos em uso. O argumento **endereço** é o local da memória onde os dados são copiados (use apenas quatro dígitos para mantê-los dentro da área de memória alocada), o argumento **unidade** refere-se a um valor de mapeado da unidade de disco do computador, como: **0** para **A:**, **1** para **B:**, **2** para **C:** e assim por diante, o argumento **setor** conta de 0 até o maior setor do volume existente e **número** refere-se ao número de setores copiados para a memória.

Quando o comando **"L"** é usado sem parâmetros, o arquivo especificado na linha de comando é carregado na memória, começando automaticamente na posição de deslocamento **0100** do segmento **"CS"** em uso desde que antecipadamente tenha sido usado o comando **"N"**, se usado com o parâmetro **endereço** é possível carregar para a memória o arquivo ou o conteúdo dos setores especificados memória, se usado todos os parâmetros é possível carregar o conteúdo de setores específicos do disco em vez de carregar um arquivo.

Exemplo

Veja na figura A.23 o exemplo do carregamento do arquivo **"exemplo.txt"** a partir da execução do programa DEBUG sem a passagem deste arquivo como argumento do programa a partir da instrução **"C:> debug /f"**.

```

-d 0100
06AF:0100 C3 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0110 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0120 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0130 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0140 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0150 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0160 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0170 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
-n exemplo.txt
-l
-d 0100
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra...o dos recu
06AF:0120 72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130 6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140 65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura....o, m
06AF:0150 6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160 74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170 44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-

```

Figura A.23 - Carregamento do arquivo "exemplo.txt".

Para o carregamento de setores de disco pode-se fazer uso de uma instrução de chamada semelhante a seguinte:

-l 0100 2 0 1

Em que o conteúdo solicitado é carregado no endereço **0100** do setor **0** da unidade **2** (disco **C:**) com a quantidade de setores definida com **1**. O comando **"L"** usa os valores dos registradores **"DX"** e **"CX"** para que nestes estejam definidos o tamanho dos dados carregados no formato de 32 bits. Caso a leitura não seja possível será apresentada a mensagem de erro **"Unknown command reading drive"** com a indicação da unidade de disco lida.

M faixa endereço - move (movimentar)

Este comando é usado para copiar o conteúdo de certo intervalo de endereço de memória para outro intervalo de endereço de memória dentro do limite de 64 KB operacionalizado pelo programa DEBUG. Antes de escrever certo conteúdo na área de destino o programa DEBUG armazena inicialmente os bytes da origem com o comando **"M"** para efetivar sua ação. É importante tomar cuidado de não se fazer a especificação de áreas de memória fora da abrangência de deslocamento dentro do segmento de memória em uso. Os parâmetros **faixa** (endereço de origem: local inicial e final) e **endereço** (local inicial de destino) são obrigatórios.

Exemplo

C:> debug /f exemplo.txt

Veja na figura A.24 o exemplo da efetivação de cópia do conteúdo da área de memória de **0100** até **0106** para o endereço **0200**.

```
-m 0100 0106 0200
-d 0100
06AF:0100  45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110  72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120  72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130  6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140  65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura....o, m
06AF:0150  6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160  74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170  44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-d 0200
06AF:0200  45 73 74 75 64 6F 20 00-00 00 00 00 00 00 00 00 Estudo .....
06AF:0210  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0220  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0230  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0240  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0250  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0260  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
06AF:0270  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00 .....
```

Figura A.23 - Movimentação (cópia) de bytes em memória.

N [[unidade:] [caminho]] arquivo [lista de argumentos] - name (nome)

Este comando é usado para estabelecer em memória o nome do arquivo a ser lido com o comando **"L"** ou escrito com o comando **"W"**. O comando **"N"** usa um parâmetro obrigatório chamado **arquivo** que se refere ao nome de referência de um arquivo no sistema operacional, o qual poderá ou não fazer uso do parâmetro **lista de argumentos**. Além desses parâmetros há opcionalmente a indicação de **unidade** referente a unidade de disco a ser usada e **caminho** para indicar o local onde o arquivo será ou é referenciado.

Exemplo

C:> debug /f

Observe na figura A.25 o uso do comando **"N"** para o carregamento do arquivo **"exemplo.txt"** para a memória. Atente para os detalhes indicados na figura.

```

-d 0100
06AF:0100 C3 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0110 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0120 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0130 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0140 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0150 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0160 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
06AF:0170 00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 .....
-n exemplo.txt
-l
-d 0100
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130 6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140 65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura...o, m
06AF:0150 6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160 74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170 44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-

```

Figura A.25 - Carregamento do arquivo "exemplo.txt".

O [W|D] porta valor - output (porta de saída)

Este comando é usado para realizar o envio, ou seja a saída, de um valor a uma porta x86. O comando "O" usa dois parâmetros obrigatórios que devem ser indicados a partir de um número entre **0000** e **FFFF** para produzir um byte para uma porta de 16 bits, também especificada, entre os valores **0000** até **FFFF**. Os comandos "OW" e "OD" produzem valores no formato **word** ou **dword** (*double-word*).

P [=endereço] [numero] - proceed (continuar)

Este comando é usado para realizar a continuidade de execução de um trecho de programa que esteja sendo operado no modo passo a passo semelhantemente ao uso do comando "T" após o uso do comando "G". O comando "P" executa de uma só vez as instruções de um bloco de programa (sub-rotina CALL), diferentemente do comando "T" que faz esta ação linha a linha não importando o formato do programa. O comando "P" faz uso de dois parâmetros opcionais alternativamente: o parâmetro **=endereço** permite estabelecer de qual endereço de deslocamento o programa será executado; o parâmetro **número** especifica a quantidade de vezes que o comando "P" poderá ser executado.

PR - proceed return (prossiga retorno)

Este comando é usado para realizar um salto para fora da sub-rotina atual, passando pelo próximo RET, RETF ou IRET que não esteja aninhado sem parar. O comando "PR" não faz uso de nenhum parâmetro.

Q - quit (sair)

Este comando é usado para encerrar a execução do programa DEBUG e retornar o controle ao sistema operacional. O comando "Q" não faz uso de nenhum parâmetro.

R [registro [valor]] - register (registrador)

Este comando é usado para dar acesso aos registradores de memória tanto na apresentação de seus conteúdos como para alteração dos mesmos. O comando "R" pode trabalhar com um parâmetro opcional chamado **registro** que pode por sua vez fazer uso ou não de um parâmetro chamado **valor**. O comando "R" sem a definição de parâmetro apresenta o estado dos registros gerais atuais, exibidos como valores de 16 ou 32 bits (dependendo do uso do comando "RX"), e o conteúdo dos demais registradores. Se usado com o comando o nome de um registrador específico este apresentará o conteúdo do registrador e se usado o parâmetro com o nome do registrador e um valor fará a inserção do valor indicado no registrador apontado.

Exemplo

C:> debug /f

Observe na figura A.26 o uso do comando "R" a partir de algumas ocorrências.

```
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0100 NV UP EI PL NZ NA PO NC
06AF:0100 C3 RET
-r ax
AX 0000 :
-r ax 1234
-r
AX=1234 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0100 NV UP EI PL NZ NA PO NC
06AF:0100 C3 RET
-r ax
AX 1234 :abcd
-r
AX=ABCD BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0100 NV UP EI PL NZ NA PO NC
06AF:0100 C3 RET
-r
```

Figura A.26 - Resultados operacionais do uso do comando "R".

RF [lista de sinalizadores] - register flag (sinalizadores: registrador de estado)

Este comando é usado para apresentar ou modificar os valores dos sinalizadores do registrador de estado. O comando "RF" não é apresentado no modo ajuda com o uso do comando "?" mas é um dos comandos mais importantes do programa DEBUG. Este comando é operado apenas com valores de 16 bits e seu uso sem parâmetro apresenta a relação dos *flags* (sinalizadores) com a sinalização de seus bits em uma única linha e permite modificação dos valores de estado de cada bit ou de todos os bits do registrador. Os bits do registrador de estado que podem ser apresentados e/ou alterados são: **OF, DF, IF, SF, ZF, AF, PF, CF** que respectivamente poderão estar configurados com o valor "1" representados pelos rótulos **NV, UP, DI, PL, NZ, NA, PO** e **NC** ou configurados com o valor "0" com os rótulos **OV, DN, EI, NG, ZR, AC, PE** e **CY**. Veja resumidamente as definições desses valores no quadro e figura A.27.

Flag Register (Registrador de Estado)								
Flag	OF	DF	IF	SF	ZF	AF	PF	CF
0	NV	UP	DI	PL	NZ	NA	PO	NC
1	OV	DN	EI	NG	ZR	AC	PE	CY

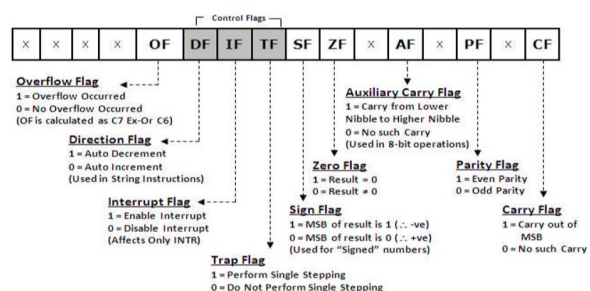


Figura A.27 - Flag Register: Robotic Eletronics

<https://roboticelectronics.in/wp-content/cache/all/flag-register-in-8086/index.html>

Exemplo

C:> **debug /f**

Observe na figura A.28 o uso do comando **"RF"** a partir de algumas ocorrências.

```
-rf
NV UP EI PL NZ NA PO NC :
-rf
NV UP EI PL NZ NA PO NC :ov di
-rf
OV UP DI PL NZ NA PO NC :
-

```

Figura A.28 - Resultados operacionais do uso do comando "RF".

A figura A.26 apresenta a alteração do estado do registrador **OF** e **IF** com seus valores alterados de **NV (0)** para **OV (1)** em **OF** e de **EI (1)** para **DI (0)** em **IF**.

As alterações de valores nos registradores de sinalização podem ser efetivadas com ou sem o uso de espaços em branco após o *prompt* DEBUG, como por exemplo **"ovei"** como mostra a figura A.29.

```
-rf
NV UP EI PL NZ NA PO NC :
-rf
NV UP EI PL NZ NA PO NC :ov di
-rf
OV UP DI PL NZ NA PO NC :
-rf
OV UP DI PL NZ NA PO NC :ovei
-rf
OV UP EI PL NZ NA PO NC :
-

```

Figura A.29 - Alteração de registradores sem uso de espaço em branco entre os flags.

RN - FPU register (apresenta o conteúdo do registrador FPU)

Este comando é usado para apresentar o estado do registrador de FPU (Floating-Point Unit – Unidade de Ponto Flutuante) que permite ao computador fazer uso de valores de ponto flutuante (valores do conjunto de números reais). O registrador FPU possui um vetor de memória com oito posições que podem ser acessadas como uma pilha: a posição **"ST0"** refere-se ao registrador que está no topo da pilha e **"ST7"** refere-se ao registrador no final da pilha. Os valores numéricos de ponto flutuante operacionalizados no registrador FPU possui, para reduzir erros de arredondamento, o comprimento de 80 bits. O comando **"RN"** não faz uso de nenhum parâmetro.

Exemplo

C:> **debug /f**

Observe na figura A.30 o uso do comando **"RN"** a partir de algumas ocorrências.

```
-rn
CW=037F SW=FFFF TW=3E80 IP=00007400 DP=00000000
ST0=0000.0000000000000000 ST1=0000.0000000000000000
ST2=0000.0000000000000000 ST3=0000.0000000000000000
ST4=NaN ST5=NaN
ST6=empty ST7=empty
-

```

Figura A.30 - Apresentação conteúdo do registrador "FPU" com o comando "RN".

RX - toggle 386 regs (alternar registrador 386)

Este comando é usado para alternar os modos de operação do programa DEBUG entre os formatos 16/32 bits. O comando **"RX"** não faz uso de nenhum parâmetro.

Exemplo

```
C:> debug /f
```

Observe na figura A.31 o uso do comando "RX".

```
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0100 NV UP EI PL NZ NA PO NC
06AF:0100 C3 RET
-rx
386 regs on
-r
EAX=00000000 EBX=00000000 ECX=00000000 EDX=00000000 ESP=0000FFFE EBP=00000000
ESI=00000000 EDI=00000000 EIP=00000100 EFL=00007202 NV UP EI PL NZ NA PO NC
DS=06AF ES=06AF SS=06AF CS=06AF FS=0000 GS=0000
06AF:0100 C3 RET
-rx
386 regs off
-r
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0100 NV UP EI PL NZ NA PO NC
06AF:0100 C3 RET
-
```

Figura A.31 - Alteração dos modos 16/32 bits.

S faixa lista - search (procurar)

Este comando é usado para efetuar na memória a pesquisa de uma cadeia de bytes. O comando "S" faz uso obrigatório de dois parâmetros, sendo **faixa** para especificar a área de memória a ser pesquisada e **lista** para determinar a cadeia de bytes a ser pesquisada. O comando pode resultar duas formas de saída: a primeira quando nada é encontrado e mostra apenas o próximo *prompt* DEBUG ou quando encontra o conteúdo pesquisado e mostra o endereço inicial de onde a sequência é iniciada.

Exemplo

```
C:> debug /f exemplo.txt
```

Observe na figura A.32 o uso do comando "S" com algumas variações.

```
-d 0100
06AF:0100 45 73 74 75 64 6F 20 65-20 64 65 6D 6F 6E 73 74 Estudo e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130 6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140 65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura....o, m
06AF:0150 6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160 74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170 44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-s 0100 012F "tudo"
06AF:0102
-s 0100 012F "xpto"
-s 0100 012F 74 75 64 6F
06AF:0102
-s 0100 012F 65 66 67
-
```

Figura A.32 - Alteração dos modos 16/32 bits.

T [=endereço] [número] - trace (rastrear)

Este comando é usado para rastrear as instruções de um programa disposto na memória, uma de cada vez, passo a passo. O comando "T" pode fazer uso de dois parâmetros opcionais, sendo **endereço** a indicação do ponto de origem que se deseja rastrear e **número** é a indicação de quantas vezes se deseja executar passo a passo o rastreamento. Se o comando "T" isoladamente rastreará apenas a instrução do código de programa que estiver apontada a partir dos valores definidos nos registradores "CS" e "IP" e fará a interrupção de sua ação apresentando os valores existentes nos registradores, além de indicar a próxima instrução a ser rastreada. Se usado com seus parâmetros permite definir o

ponto de início do rastreamento (apenas o primeiro parâmetro) ou com o segundo parâmetro realizar a partir do endereço apontado um certo número de repetições do comando.

Exemplo

C:> **debug /f**

Observe na figura A.33 o uso do comando "T". Atente para a definição de um código de programa que permite fazer uso do comando algumas vezes. Veja que a cada execução do comando "T" o registrador **IP** tem seu valor alterado apontando para a próxima linha a ser executada. A primeira execução do comando "T" salta o programa para a linha **011F** conforme indicado na instrução **jmp 011F** na linha **0100**. As duas próximas linhas representam os dados armazenados na memória e que são tratados no programa.

```
-a 0100
06AF:0100 jmp      011F
06AF:0102 db       0d, 0a, "Teste programa em DEBUG!"
06AF:011C db       0d, 0a, "$"
06AF:011F mov ah, 09
06AF:0121 mov dx, 0102
06AF:0124 int      21
06AF:0126 mov ah, 00
06AF:0128 int      21
06AF:012A
-t
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=011F NV UP EI PL NZ NA PO NC
06AF:011F B409          MOV     AH,09
-t
AX=0900 BX=0000 CX=0000 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0121 NV UP EI PL NZ NA PO NC
06AF:0121 BA0201       MOV     DX,0102
-t
AX=0900 BX=0000 CX=0000 DX=0102 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0124 NV UP EI PL NZ NA PO NC
06AF:0124 CD21        INT      21
-t
Teste programa em DEBUG!
AX=0924 BX=0000 CX=0000 DX=0102 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0126 NV UP EI PL NZ NA PO NC
06AF:0126 B400        MOV     AH,00
-t
AX=0024 BX=0000 CX=0000 DX=0102 SP=FFFE BP=0000 SI=0000 DI=0000
DS=06AF ES=06AF SS=06AF CS=06AF IP=0128 NV UP EI PL NZ NA PO NC
06AF:0128 CD21        INT      21
-t
Program terminated (0000)
-~
```

Figura A.33 - Execução passo a passo com comando "T".

Outra maneira de executar o programa e ver cada um dos conteúdos gerados na memória de uma única vez é por meio da instrução **"t -0100 5"**.

TM [0|1] - trace mode (modo de rastreamento)

Este comando é usado para mostrar ou definir o modo de rastreamento do programa. São possíveis o uso de dois modos de operação com **"TM0"** e **"TM1"**. Exceto a indicação dos modos de operação o comando **"TM"** não usa nenhum parâmetro. Os modos **"TM0"** e **"TM1"** afetam a forma como o comando **"T"** é operado. Se selecionado o modo **"TM0"** (modo padrão) executa rastreamento no estilo do programa *Enhanced DEBUG* e o modo **"TM1"** executa rastreamento no estilo *Microsoft DEBUG*.

U [faixa] - unassemble (desmontar)

Este comando é usado para desmontar as instruções de um programa na forma de código de máquina. O código desmontado tem a aparência semelhante a estrutura de um código montado. O comando **"U"** utiliza opcionalmente um

parâmetro chamado **faixa**. Caso o comando seja usado sem o parâmetro atuará automaticamente sobre o endereço de deslocamento **0100** desmontando 32 bytes, mas com a definição do parâmetro é possível definir o endereço inicial e final de atuação ou a definição do endereço inicial e seu comprimento com o código "L".

Exemplo

C:> debug /f exemplo.txt

Observe na figura A.34 o uso do comando "U" sem a definição de parâmetro, na figura A.35 com definição de parâmetro de faixa inicial/cumprimento e na figura A.36 com definição de parâmetro de faixa inicial e final.

```
-u
06AF:0100 45          INC     BP
06AF:0101 7374        JAE     0177
06AF:0103 7564        JNZ     0169
06AF:0105 6F          OUTSW
06AF:0106 206520      AND     [DI+20],AH
06AF:0109 64          FS:     (unused)
06AF:010A 65          GS:     (unused)
06AF:010B 6D          INSW
06AF:010C 6F          OUTSW
06AF:010D 6E          OUTSB
06AF:010E 7374        JAE     0184
06AF:0110 7261        JB      0173
06AF:0112 C3          RET
06AF:0113 A7          CMPSW
06AF:0114 C3          RET
06AF:0115 A36F20      MOV     [206F],AX
06AF:0118 646F        FS:OUTSW
06AF:011A 7320        JAE     013C
06AF:011C 7265        JB      0183
06AF:011E 637572      ARPL    [DI+72],SI
-
```

Figura A.34 - Execução do comando "U" sem parâmetro.

```
-U 0100 L 10
06AF:0100 45          INC     BP
06AF:0101 7374        JAE     0177
06AF:0103 7564        JNZ     0169
06AF:0105 6F          OUTSW
06AF:0106 206520      AND     [DI+20],AH
06AF:0109 64          FS:     (unused)
06AF:010A 65          GS:     (unused)
06AF:010B 6D          INSW
06AF:010C 6F          OUTSW
06AF:010D 6E          OUTSB
06AF:010E 7374        JAE     0184
-
```

Figura A.35 - Execução do comando "U" com parâmetro de faixa inicial e comprimento.

```
-U 0100 010D
06AF:0100 45          INC     BP
06AF:0101 7374        JAE     0177
06AF:0103 7564        JNZ     0169
06AF:0105 6F          OUTSW
06AF:0106 206520      AND     [DI+20],AH
06AF:0109 64          FS:     (unused)
06AF:010A 65          GS:     (unused)
06AF:010B 6D          INSW
06AF:010C 6F          OUTSW
06AF:010D 6E          OUTSB
-
```

Figura A.36 - Execução do comando "U" com parâmetro de faixa inicial e final.

V - view flip (visualizar inversão)

Este comando quando em uso permite que a saída do programa ocorra na página de vídeo 0 (zero) quando o efeito de inversão de página no sistema é suportado. O comando "V" não possui nenhum parâmetro e seu efeito pode, dependendo de diversos fatores alheios ser imperceptível. Quando o argumento "/f" é usada no *prompt* do sistema na chamada do programa DEBUG o modo de operação em uso é a página 1 de vídeo, enquanto a saída do programa, geralmente, ocorrerá na página 0 de vídeo. Dentre os comandos existentes este é um que pode não parecer muito útil.

W [[endereço] [unidade setor número]] - write file/sectors (escrever arquivo/setores)

Este comando escreve em disco um arquivo ou setores específicos que estejam em memória. O comando "W" pode ser usado com ou sem parâmetros. Para usar o comando sem parâmetro é necessário que o arquivo a ser gravado tenha sido carregado inicialmente com o programa DEBUG ou que se utilize antes o comando "N". Caso haja a necessidade de gravar um conteúdo de memória é necessário considerar o número de bytes a serem gravados em um arquivo de disco definidos junto aos registradores gerais "BX" e "CX". O parâmetro **endereço** especifica o ponto inicial de memória a ser considerado na gravação, o parâmetro **unidade** especifica a unidade de disco usada para a gravação, o parâmetro **setor** especifica o endereço do primeiro setor a ser gravado e o parâmetro **número** especifica a quantidade de setores a ser usada na gravação. Para fazer uso do comando "W" é necessário tomar certos cuidados: caso tenha ocorrido antes de executar o comando "W" o uso dos comandos "G", "T", "P" e "R" é necessário redefinir, primeiro, os registradores gerais "BX" e "CX"; não use o comando para gravar arquivos nos formatos ".EXE" e ".HEX".

Exemplo

C:> debug /f exemplo.txt

Observe na figura A.37 o uso do comando "W" sem a definição de parâmetro.

```
-d 0100 0105
06AF:0100 45 73 74 75 64 6F      -          Estudo
-e 0101 6e,73,61,69
-d 0100 0105
06AF:0100 45 6E 73 61 69 6F      -          Ensaio
-w
Writing 0077 bytes
-q

D:\>debug
-n exemplo.txt
-l
-d 0100 0105
06AF:0100 45 6E 73 61 69 6F      -          Ensaio
-
```

Figura A.37 - Execução do comando "W" sem parâmetro após alteração de arquivo.

Observe na figura A.38 o uso do comando "W" com a definição de parâmetro gravando o conteúdo de memória do arquivo "exemplo.txt" em um arquivo chamado "teste.txt" a partir do endereço "CS:0100". A figura A.39 mostra o resultado da gravação no arquivo "TESTE".

```
D:\>debug exemplo.txt
-d 0100
06AF:0100 45 6E 73 61 69 6F 20 65-20 64 65 6D 6F 6E 73 74 Ensaio e demonst
06AF:0110 72 61 C3 A7 C3 A3 6F 20-64 6F 73 20 72 65 63 75 ra....o dos recu
06AF:0120 72 73 6F 73 20 65 78 69-73 74 65 6E 74 65 73 20 rsos existentes
06AF:0130 6E 6F 20 75 74 69 6C 69-74 C3 A1 72 69 6F 20 64 no utilit..rio d
06AF:0140 65 20 64 65 70 75 72 61-C3 A7 C3 A3 6F 2C 20 6D e depura....o, m
06AF:0150 6F 6E 74 61 67 65 6D 20-65 20 64 65 73 6D 6F 6E ontagem e desmon
06AF:0160 74 61 67 65 6D 20 22 45-6E 68 61 6E 63 65 64 20 tagem "Enhanced
06AF:0170 44 45 42 55 47 22 2E 00-00 00 00 00 00 00 00 00 DEBUG".....
-n teste.txt
-w cs:0100
Writing 0077 bytes
-
```

Figura A.38 - Execução do comando "W" com parâmetro no endereço "CS:0100".

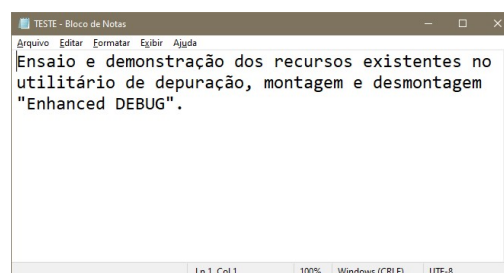


Figura A.39 - Arquivo gerado após o uso da instrução "w CS:0100".

O comando **"W"** pode ser usado de forma mais completa como **"w cs:0100 2 3A 3F"** e neste caso far-se-á a gravação dos dados em memória iniciando-se no endereço **"cs:0100"** na unidade de disco **"C:"** começando no setor lógico **"3A"** por **"3F"** setores.

Bibliografia

ABEL, P. **IBM PC Assembly Language and Programming**. New Jersey, USA: Prentice-Hall, 2001.

AMD. **AMD64 Architecture Programmer's Manual: Application Programming**. Santa Clara, USA: Advanced Micro Devices, v. 1, 2003.

_____. **AMD64 Architecture Programmer's Manual: General-Purpose and System Instruction**. Santa Clara, USA: Advanced Micro Devices, v. 3, 2003.

_____. **AMD64 Architecture Programmer's Manual: System Programming**. Santa Clara, USA: Advanced Micro Devices, v. 2, 2003.

BREY, B. B. **The Intel Microprocessors**. New Jersey, USA: Prentice-Hall, 1998.

_____. **8086/8088, 80286, 80386 and 80486 Assembly Language Programming**. New Jersey, USA: Prentice-Hall, 1994.

CURSO IBM DE PROGRAMAÇÃO EM 32 BITS. **Turbo Assembler**. São Paulo: Planeta, V. 3, p. 561-620, 1998.

DANDAMUDI, S P. **Introduction to Assembly Language Programming: From 8086 to Pentium Processors**. 3. ed. New York, USA: Springer-Verlag, 2000.

DETMER, R. C. **Essentials of 80x86 Assembly Language**. Canada: Jones and Bartlett Publishers, 2007.

HYDE, R. **The Art of Assembly Language**. San Francisco, CA (USA): No Starch Press, 2003.

IBM. **PC DOS 7 Technical Update: Document Number GG24-4459-00**. Florida, NW (USA), 1995.

INTEL. **IA-32 Intel Architecture Software Developer's Manual: Basic Architecture**. V. 1. Santa Clara, USA: Intel Corporation, 2003.

_____. **IA-32 Intel Architecture Software Developer's Manual: Instruction Set Reference**. V. 2. Santa Clara, USA: Intel Corporation, 2003.

_____. **IA-32 Intel Architecture Software Developer's Manual: System Programming Guide**. V. 3. Santa Clara, USA: Intel Corporation, 2003.

_____. **iAPX 86/88, 186/188 User's Manual: Hardware Reference**. Santa Clara, USA: Intel Corporation, 1985.

_____. **iAPX 86/88, 186/188 User's Manual: Programmer's Reference**. Santa Clara, USA: Intel Corporation, 1987.

_____. **MCS-86 Assembly Language Reference Guide**. Santa Clara, USA: Intel Corporation, 1978.

_____. **Intel 64 and IA-32 Architectures Software Developer's Manual: Instruction Set Reference, A-M. - Volume 2A**, Junho de 2009, Publicação número 253666-031US.

_____. **Intel 64 and IA-32 Architectures Software Developer's Manual: Instruction Set Reference, N-Z. Volume 2B**: Junho de 2009, Publicação número 253667-031US

- IRVINE, K. **Assembly Language for Intel-Based Computer**. New Jersey, USA: Prentice-Hall, 1998.
- LANCHARRO, E. A. et al. **Informática Básica**. São Paulo: Makron Books, 1991.
- LITERÁK, L. **80x86 Instruction Set. Czech Republic**. Disponível em: <http://www.penguin.cz/~literak/intel/intel.html>. Acesso em 25 fev. 2013.
- LOURENÇO, A. C. **Sistemas Numéricos e Álgebra Booleana**. São Paulo: Érica, 1994.
- LVSEVEN. **Emu8086 Microprocessor Emulator**. Disponível em: <https://lvseven.weebly.com/blog/emu8086-microprocessor-emulator>. Acesso em 25 nov. 2018.
- MANZANO, A. L. N. G.; MANZANO, M. I. N. G. **Estudo Dirigido de Informática Básica**. 7. ed. São Paulo: Érica, 2007.
- MASLOCH, C. **IDebug manual**. Ulukai.org, 2021. Disponível em: <https://ulukai.org/ecm/doc/idebug.pdf>. Acesso em: 5, mai. 2021.
- MEYER, M.; BABER, R.; PFAFFENBERGER, B. **Nosso Futuro e o Computador**. Porto Alegre: Bookman, 2000.
- MORSE, S. P. **The 8086/8087 Primer: An Introduction to Their Architecture, System Design and Programming**. 2. ed. USA: Hayden Books, 1987.
- NORTON, P. **Desvendando o PC e PS/2**. Rio de Janeiro: Campus, 1993.
- NORTON, P.; SOCHA, J. **Linguagem ASSEMBLY para IBM PC**. Rio de Janeiro: Campus, 1987.
- OLIVEIRA, F. O. **Tutorial de Linguagem Assembly**. Brasil. Disponível em: <http://www.ebah.com.br/content/ABAAAA4WQAD/tutorial-linguagem-assembly>. Acesso em 25 fev. 2013.
- QUADROS, Daniel G. A. **PC Assembler com programa exemplo totalmente comentado**. Rio de Janeiro: Campus, 1986.
- SANTOS, J. P. **Turbo Assembler e Macro Assembler**. São Paulo: McGraw-Hill, 1990.
- SEDORY, D. B. **A guide to DEBUG: The Microsoft DEBUG.EXE program**. Site pessoal, 2020. Disponível em: <https://thestarman.pcministry.com/asm/debug/debug.htm>. Acesso em: 3, mai. 2021.
- TOKHEIN R. L. **Introdução aos Microprocessadores**. São Paulo: McGraw-Hill, 1985.
- TRIEBEL, W. A.; SINGH, A. **The 8088 and 8086 Microprocessors: Lab Manual**. New Jersey, USA: Prentice-Hall, 2000.
- VISCONTI, A. C. J. F. **Microprocessadores 8080 e 8085: Hardware**. 9. ed., V. 1. São Paulo: Érica, 1991.
- WEBER, R. F. **Fundamentos de Arquitetura de Computadores**. 3. ed. Porto Alegre: Instituto de Informática de UFRGS, 2004.
- YADAV, A. **Microprocessor 8085, 8086**; USA: University Science Press, 2008.

Programas Fonte

O material de estudo desta obra (programas fonte) pode ser obtido a partir do endereço GitHub:

https://github.com/J-AugustoManzano/livro_Assembly-Intro-8086

A todos um grande abraço e um bom aprendizado!

LINGUAGEM ASSEMBLY

INTRODUÇÃO AO PADRÃO INTEL 8086

O objetivo deste trabalho é ser um veículo de apresentação dos princípios preliminares e básicos que norteiam o início do aprendizado da linguagem de programação Assembly 8086/8088 de microcomputadores IBM-PC em baixo nível. Não há pretensão de explorar o tema com profundidade. Este livro é direcionado tão somente a leitores iniciantes, é uma introdução a atividade de programação em baixo nível.

Os exemplos de desenvolvimento apresentados são desenvolvidos a partir do uso das ferramentas "Enhanced DEBUG" e "emu8086" com a escrita de códigos de programação em linguagem de máquina por meio de "opcodes" e em linguagem de montagem "assembly".

